

Hardware Verification

A Holistic Guide

PARTS I–VII — FIRST EDITION, SEPTEMBER 2026

Marco Paci · Fondazione Chips-IT

2026-09-18

Citations verified against the full-text corpus

Hardware Verification: A Holistic Guide

Marco Paci · Fondazione Chips-IT

First edition, September 2026.

© 2026 Fondazione Chips-IT. Some rights reserved — see **Licence**, below.

Purpose of this edition. This edition serves two uses: a reference for engineers doing verification work, and teaching material for engineers moving into verification from design. It is not a vendor document and recommends no commercial tool.

Sources. Every numbered reference in this book resolves to a document held in full text, and every claim carrying a reference was checked against the page cited rather than against recollection of it. Where the literature does not settle a question, the book says so. Where a figure comes from a single source, that source and its date are named, because a statistic without a date is not a statistic.

Example systems. The reference SoC and the flagship SoC used throughout are constructed teaching examples with fixed, published parameters. They are not any company's product, and no claim about them should be read as a claim about silicon. Claims about real organisations are a separate matter and are made only where a published source states them; each is cited to that source.

Trademarks. AMBA is a registered trademark of Arm Limited (or its subsidiaries or affiliates) in the US and/or elsewhere; Arm publishes trademark usage guidelines at arm.com. AXI, AHB, APB and ACE are protocol designations within the AMBA family and are used here descriptively. IEEE is a registered trademark in the U.S. Patent and Trademark Office, owned by The Institute of Electrical and Electronics Engineers, Incorporated. Verilog is a registered trademark of Cadence Design Systems, Inc. Other product, organisation and standard names appearing in this book may be the trademarks of their respective owners. They are used editorially, in the owner's favour, with no intent to infringe.

No endorsement. None of the organisations whose specifications, publications or results are cited in this book has reviewed, approved, sponsored or endorsed it. That includes Arm, IEEE, Accellera, RISC-V International, and every company named in a cited case study. Where this book characterises a published result, the characterisation is the author's and any error in it is the author's.

Quotation from standards. Short passages of normative text are quoted, with attribution to the document, its issue and its clause, for the purpose of identifying and explaining the requirement under discussion. Those quotations rest on the statutory right of quotation for purposes of criticism, review, illustration and teaching — Article 5(3)(d) of Directive 2001/29/EC as implemented in national law — and **not** on any specification licence: the AMBA specification licence grants copying for the purpose of developing products that comply with the specification, which is not the purpose of this book, and the specifications' proprietary notices reserve reproduction to the publisher's written permission. Accordingly no standard is reproduced in whole or in substantial part, and **no figure or table from any standard is reproduced**; where this book needs a diagram of something a standard specifies, the diagram is drawn from scratch. Reproducing a figure or a table would require written permission, which attribution does not substitute for.

Assistance disclosure. This book was written with substantial assistance from large language models. The systems used were Anthropic's Claude — models in the Claude Opus family for drafting, adversarial review and page-level citation checking, and a Claude Fable model for orchestration across chapters. The tools are not authors: responsibility for every claim, and for the decisions about what the book argues, rests with the named author. The editorial procedure — a closed source corpus, an independent review pass for each chapter, page-level verification of citations, and mechanical gates on numbers,

code, hedges and cross-references — is described in the preface, together with its known ceiling, so that a reader who wants to judge the method can.

Licence. This book is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International licence (CC BY-NC-SA 4.0), whose legal code is at <https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode> and travels with the book as the file `LICENSE`. You may copy, redistribute, quote at length and adapt it for non-commercial purposes, provided you credit *Marco Paci / Fondazione Chips-IT, Hardware Verification: A Holistic Guide, first edition, September 2026*, indicate any changes, link to the licence, and place any adaptation under the same terms. The licence covers this book and nothing else: the documents it cites are not part of it and are not relicensed by it — the primary sources are held outside it and remain subject to their publishers' terms, and the quotations from standards described above rest on the statutory right of quotation rather than on this licence.

Errors. Corrections are welcome and will be recorded. An error found in a book of this size is a contribution to it, not an embarrassment to it.

Contents

Figures	10
Tables	12
Preface	14
Notation and conventions	18
Part I — Foundations	23
1. Why Verification Exists	24
1.1 The problem: correctness is not the default	25
1.2 The numbers: how often the industry fails	26
1.3 The effort: who pays, and how much	28
1.4 The economics: what a bug costs, and when	29
1.5 Public history: when escapes became famous	32
1.6 The reference SoC: the running example of this book	33
2. The Verifier's Mindset	39
2.1 Designer and verifier: two minds, one specification	40
2.2 Thinking in terms of breakage	41
2.3 Independence of judgment	44
2.4 The value of doubt	45
3. The Verification Problem	50
3.1 The state-space explosion	50
3.2 Controllability and observability	54
3.3 The oracle problem	56
3.4 From exhaustive to risk-driven verification	59
4. The Verification Lifecycle	64
4.1 The shape of the lifecycle	64
4.2 Planning defines deliverables and closure criteria	67
4.3 From bring-up to regression maturity	69
4.4 Maturity stages as a shared language	72
4.5 Closure and sign-off	73
4.6 Roles and reviews assign responsibility across the lifecycle	75
4.7 Schedule reality	77
Part II — Planning and Measurement	85
5. Verification Planning	86
5.1 From specification to feature list	87
5.2 The anatomy of a vplan row	90
5.3 Making the plan executable and traceable	95

5.4 The plan review as a rite	96
5.5 Risk: deciding how deep to go, feature by feature	97
5.6 Planning against an incomplete specification	99
6. Coverage: Theory and Practice	106
6.1 The three coverage families and their taxonomy	106
6.2 Code coverage and its structural limits	108
6.3 Functional coverage from specification intent	109
6.4 The scope and limits of 100% coverage	111
6.5 Designing the coverage model: an engineering act	111
6.6 Worked example: a coverage model for the neural accelerator	114
6.7 Shaping the crossbar coverage model	117
6.8 Closure: the workflow from holes to done	119
6.9 Exclusions as reviewable artifacts	121
7. Metrics-Driven Sign-off	127
7.1 Metrics for shared project status	127
7.2 Dashboard instruments	128
7.3 Reading bug-rate curves	131
7.4 The sign-off checklist: aggregating the evidence	133
7.5 Worked example: the dashboard, two weeks before tape-out	135
7.6 Escape analysis and the next verification plan	137
Part III — Dynamic Verification	145
8. Testbench Architecture	146
8.1 Five separate responsibilities	146
8.2 Layering: signal, transaction, sequence	148
8.3 Scoreboard patterns	150
8.4 Reference models: three species	154
8.5 Monitors and passive checking	155
8.6 Worked example: the crossbar environment, step by step	157
8.7 Worked example: the DMA environment and end-to-end checking	162
8.8 The environment as a design in its own right	163
9. Stimulus: Directed to Random to Portable	169
9.1 Directed tests: precision, and its ceiling	170
9.2 The constrained-random turn	171
9.3 Writing good constraints	173
9.4 Seeds and reproducibility	180
9.5 Sequences and stimulus layering	181
9.6 Graph-based and portable stimulus	183
10. UVM as a Methodology	190
10.1 The problem: reuse across projects, teams and companies	190

10.2 The component taxonomy and why it exists	192
10.3 The factory and the configuration database	194
10.4 Phasing: why a testbench needs a state machine	196
10.5 Sequences and the sequencer-driver handshake	197
10.6 The register layer	198
10.7 Reporting as infrastructure	202
10.8 Reuse in practice: the same agent, one level up	202
10.9 What UVM does not solve	203
10.10 When not to use UVM	204
11. Assertion-Based Verification	209
11.1 Assertions as executable specifications	210
11.2 The anatomy of SVA	211
11.3 Assert, assume, cover: three claims about one property	213
11.4 Where assertions live	214
11.5 Protocol checking: the interface as a contract	215
11.6 White-box assertions and internal invariants	221
11.7 Assertion quality	225
11.8 The same text, two engines	226
12. Regression Engineering	233
12.1 Regression suites and admission policies	234
12.2 The (test, seed) matrix	236
12.3 Continuous integration for hardware	237
12.4 Failure triage at scale	238
12.5 Flakiness and regression reliability	240
12.6 Compute economics	243
Part IV — Static and Formal	251
13. Static Verification	252
13.1 The entry gate	253
13.2 Lint: from noise to policy	255
13.3 Clock-domain crossing	258
13.4 Structural checks and protocol checks	260
13.5 Reset-domain crossing	263
13.6 X-propagation	265
14. Formal Property Verification	272
14.1 Proof and its scope	272
14.2 Properties and assumptions	275
14.3 Bounded and unbounded proof	278
14.4 Abstraction and excluded behavior	281
14.5 Convergence: the undetermined property	284

14.6 Formal sign-off	288
15. Formal Apps and Equivalence	297
15.1 The app: formal as a product	297
15.2 Connectivity checking: proving the wiring	299
15.3 Register checks: the map against its description	301
15.4 Unreachability: a proof instead of a waiver	302
15.5 Auto-generated properties and specification review	304
15.6 Logic and sequential equivalence checking	305
15.7 Datapath equivalence at word level	308
16. Hybrid Flows	315
16.1 Dividing the work	316
16.2 Assumption and constraint duality, as a flow	318
16.3 Coverage unification	321
16.4 Choosing an engine, row by row	324
16.5 The handoff	326
Part V — Beyond RTL Simulation	333
17. Acceleration and Emulation	334
17.1 Simulation's cycle limit	334
17.2 Three platforms and their costs	337
17.3 Use models: in-circuit and virtual	339
17.4 The transactor boundary	341
17.5 Economics: one machine, many claimants	346
18. FPGA Prototyping and HW/SW Co-verification	354
18.1 The prototype as a different instrument	355
18.2 Virtual platforms	358
18.3 Hybrid co-simulation	361
18.4 Firmware-driven verification	363
18.5 Bug ownership and triage	367
19. Gate-Level, Timing and Power-Aware Verification	374
19.1 The role of gate-level simulation	374
19.2 Reading a gate-level failure	376
19.3 SDF back-annotation and what a timing check asserts	379
19.4 Power-aware verification	382
19.5 Run selection and scheduling	389
20. Post-Silicon Validation	396
20.1 Changes from model to physical part	397
20.2 Design for debug before tape-out	399
20.3 Bring-up: the first hours and days	402
20.4 Triage and localization costs	406

20.5 Escapes and the feedback loop	410
20.6 Workarounds and fix decisions	413
Part VI — Specialized Domains	421
21. Analog and Mixed-Signal Verification	422
21.1 Where the digital abstraction stops	423
21.2 Real number modelling	424
21.3 AMS co-simulation	426
21.4 Behavioral models and model correlation	428
21.5 Digital verification responsibilities	431
22. Safety Verification	438
22.1 Obligations imposed by safety standards	438
22.2 Failure analysis as a verification input	440
22.3 Fault injection and fault simulation	443
22.4 Safety mechanisms and their verification	447
22.5 Tool qualification and the evidence trail	450
23. Security Verification	458
23.1 The adversary defines the scope	459
23.2 The threat model as the plan's root	460
23.3 Security properties and how they differ	464
23.4 Information flow and its verification	466
23.5 Side channels	469
23.6 Where security verification lives in the flow	471
Part VII — The Human and the Machine	479
24. Teams, Process and Ramp-up	480
24.1 Roles and outcome ownership	481
24.2 Review purposes and mechanics	483
24.3 Maturity stages as a shared language	486
24.4 Onboarding: designer → verifier	487
24.5 Process quality and limits	491
25. AI and ML in Verification	499
25.1 Classical machine learning in verification	500
25.2 Evidence for large language models	503
25.3 Plausible by construction	508
25.4 The human gates	511
25.5 Governance	513
26. The Road Ahead	522
26.1 The bottleneck moves	523

26.2 Shift-left and shift-right	525
26.3 What does not change	527
26.4 Verification as a profession	529
Appendix A — Glossary	537
Appendix B: Reading paths	570
Appendix C: The literature and its scope	577
Appendix D: Solutions to the exercises	586
Index	641

Figures

Figure 1.1	The four stages at which a bug can be found	30
Figure 2.1	The reconvergence model drawn for the DMA of this chapter's opening	41
Figure 3.1	The three-link chain a bug must survive to be found	54
Figure 4.1	The shape of a verification lifecycle	65
Figure 5.1	The five fields of a well-formed vplan row	90
Figure 6.1	The coverage-closure loop	119
Figure 7.1	How the sign-off checklist aggregates its evidence and what the review is then allowed to decide	134
Figure 8.1	The crossbar environment	158
Figure 9.1	The two families of stimulus constraint drawn as nested regions	173
Figure 10.1	The crossbar testbench as a UVM component hierarchy	193
Figure 11.1	Backward \$stable: at cycle 1 a fresh address differs from the prior idle value and fires; at cycle 10 withdrawal leaves the...	211
Figure 11.10	Delimiter 2 captures at 4 inside the window; delimiter 9 has no capture by 11 and fails	221
Figure 11.11	One bit is set through cycle 12; two at 13 and 14 fail the invariant; one returns at 15	222
Figure 11.12	Two representations of one fact	223
Figure 11.13	The two level guards	223
Figure 11.14	Accepted address 2 ends at 4096 and holds; crossing address 4-5 is unaccepted and unjudged; acceptance 9 fails	224
Figure 11.15	Key load 1 holds idle; load 7 fails in flight	225
Figure 11.16	Two equivalent formulations on one trace	226
Figure 11.17	Both forms on one pair of requests	227
Figure 11.18	The invariant across two caches	227
Figure 11.2	Three sequence forms	212
Figure 11.3	Reset falls between cycles 3 and 4	212
Figure 11.4	Same-tick consequent	213
Figure 11.5	Forward obligation: offers at cycles 1	216
Figure 11.6	Per-ID bookkeeping: addresses at cycles 1 and 3 leave two writes under one ID; responses at 5 and 7 retire them	216
Figure 11.7	Beats at cycles 3 and 4 match queued length; a third beat at cycle 11 sets lenerr at wlast	218
Figure 11.8	Immediate acceptance leaves attempts vacuous and cawstall empty; only the cycle-9-to-10 stall runs the check	219
Figure 11.9	On a subordinate port	220
Figure 12.1	The path a commit takes to become evidence	234
Figure 13.1	Where each static check sits relative to the per-commit gate	254
Figure 14.1	One assumption drawn on two traces	277

Figure 14.2	Induction starts an interval at an arbitrary state	280
Figure 14.3	The counterexample a proof engine returns	281
Figure 14.4	The order in which to work an undetermined property	286
Figure 15.1	Equivalence checking as a second path from the same origin	306
Figure 16.1	Engine assignment belongs to the plan row rather than to the block	316
Figure 17.1	Where the transaction boundary cuts an accelerated environment	342
Figure 18.1	A hybrid co-simulation partition	361
Figure 19.1	The reference SoC's two power domains and every signal that crosses between them	383
Figure 20.1	Post-silicon work is a loop rather than a line	414
Figure 21.1	The two engines meet at one boundary	427
Figure 22.1	The reduction sequence that takes a fault list from every site crossed with every fault model down to the faults a campaign...	445
Figure 23.1	The flagship's root of trust	462
Figure 24.1	Four reviews with four different questions	484
Figure 25.1	One missing delay, wrong in two directions	508
Figure 25.2	The strengthened cover on two requests	509
Figure 25.3	The same generated property on a pipeline that loads the selector one cycle before the request rises	510
Figure 25.4	The gates a generated artifact passes before it is admitted to a suite	511

Tables

Table 1.1	The reference SoC element by element	33
Table 3.1	The per-channel programming interface of the reference SoC's DMA engine	51
Table 3.2	One DMA channel after expert abstraction	52
Table 3.3	What the 4 KB-crossing assertion of the worked example establishes about the DMA backend and what it leaves untouched	58
Table 4.1	An excerpt from the crossbar's feature list in week one	68
Table 4.2	The one-page evidence summary the crossbar's sign-off review starts from	74
Table 4.3	The review calendar in lifecycle order	75
Table 5.1	Four rows of the DMA vplan	92
Table 5.2	Five DMA features ranked on the two axes that set verification depth	98
Table 6.1	The four attributes of the accelerator's job coverage model	115
Table 7.1	Six questions to put to a bug curve that has gone flat	132
Table 7.2	The week-40 block dashboard that the sign-off review reads	135
Table 8.1	The five responsibilities a testbench separates	147
Table 8.2	Scoreboard structure as a function of the ordering the design actually guarantees	153
Table 11.1	What the four property directives mean to each of the two engines that read them	213
Table 12.1	The four tiers of a regression suite	235
Table 12.2	The flagship SoC's nightly candidate list before any re-tiering	245
Table 14.1	The crossbar's formal results in the shape a sign-off package presents them	291
Table 14.2	The assumption register those results lean on	291
Table 15.1	The catalogue of formal apps	298
Table 15.2	The three equivalence flows side by side	307
Table 15.3	The two verification-plan rows a connectivity app earns at SoC top level	310
Table 16.1	What a cross-engine coverage report may and may not claim	322
Table 17.1	One verification problem per row	336
Table 17.2	A simulator, an emulator and an FPGA prototyping platform compared row by row	337
Table 17.3	The fraction of an accelerated platform's peak rate a testbench actually obtains	343
Table 19.1	The power-down and power-up sequences for the reference SoC's switched domain	384
Table 19.2	The useful combinations of the three independent choices that make up a gate-level run	389
Table 20.1	What changes when the design becomes a part	397
Table 20.2	On-die design-for-debug instruments	399
Table 20.3	The industrial path from a failed test run to a resolved bug	406

Table 20.4	Classifying a silicon escape backwards	411
Table 20.5	The three forward-looking options for an open bug once the masks exist	415
Table 21.1	Three published real number modelling results	425
Table 21.2	A model register entry laid out one field per row	431
Table 22.1	Three plan rows, in this book's five-column format	452
Table 23.1	The flagship's threat model as rows	462
Table 23.2	The four parts of an information-flow rule	467
Table 23.3	Six security questions	471
Table 25.1	What language models have been reported to do in this field	506

Preface

Purpose

Hardware functional verification is taught in pieces. There are good books on simulation-based methodology and formal property verification, good coverage papers, and standards documents for each language involved. Very little literature explains how these subjects bear on one another. Little of it explains how a decision in a verification plan determines what a proof can discharge, that is, which of the plan's claims a proof can settle on its own. Nor does it explain what a coverage number means once formal results are merged into it, or which questions survive every instrument a project can afford. These are decisions verification leads make, but the existing literature leaves gaps between its subjects.

In the author's assessment, several figures repeated in this field are older than their use suggests, and some trace to no primary measurement. Among them are the figures for where verification effort goes, how often first silicon succeeds, and what a bug costs at each stage. The book's evidence policy requires it to identify which numbers withstand scrutiny and name those that do not; this book does so.

Scope

It is an account of hardware functional verification as a single discipline. It covers planning and measurement; simulation-based verification and the methodologies built on it; and assertions. It takes up formal property verification and equivalence checking; static analysis, clock- and reset-domain crossing; and gate-level and power-aware verification. Its subjects include acceleration, emulation and FPGA prototyping; hardware-software co-verification; and analog-mixed-signal, safety and security verification. It covers post-silicon validation, and the economics and metrics by which all of it is judged. It treats these as one subject because a project cannot choose just one, and the decisions concern where the boundary between two techniques should fall.

The book treats verification as an argument about evidence rather than a sequence of activities. An argument here is a case built from evidence, not a disagreement. A verification plan is a set of claims; coverage is a sample of a model somebody wrote; a proof holds under assumptions somebody discharged or did not. Every technique in this book is presented as an instrument producing a particular kind of evidence, with a stated reach and stated blind spots. An instrument here is a means of measurement, and its reach is the range of behavior it can make a statement about. The recurring question is which instrument answers which question, at what cost, and what remains unanswered.

Limits

It is not a manual for a methodology library, and not a tool tutorial. It names tool categories (simulator, formal tool, emulator, linter) and does not recommend products.

Where a technique is inseparable from a standard, the standard is named precisely, because that is the document a reader has to open to check the claim. The name gives the standard's issue and its clause. The issue is the edition of the document, which matters because a specification's requirements change from one issue to the next.

Unlike a survey of published work, this book takes positions and labels them as such. Where the literature is divided, the division is described rather than averaged away. Where published evidence is thin, as it is in several areas of this field, the book states the gap without substituting confident prose.

Intended readers

The book serves two readers with different needs.

The first is a verification engineer seeking unfamiliar parts of the discipline: a simulation-based verification engineer encountering formal property verification, or a formal engineer asked about coverage closure. Coverage closure is the point at which every item of a verification plan has been either covered or consciously excluded. Chapters are sufficiently self-contained for direct entry, and Appendix B gives reading paths stating what each equips readers to do and leaves out.

The second is an engineer moving into verification from design. That reader is the reason the book develops its arguments from first principles rather than assuming a methodology background, and the reason a single worked example system recurs from beginning to end.

Constructed example systems

Almost every example in this book runs on one of two constructed systems: a modest reference SoC, and a later, larger flagship SoC from the same fictional lineage. Their parameters are fixed and published, they do not change between chapters, and a bug found in Chapter 1 is still the same bug in Chapter 26.

The choice of constructed examples is deliberate: it gives up lookup and checking against a datasheet, and what it buys is completeness. An example can be complete: every parameter the argument needs is available, the same crossbar can be examined through six different techniques, and a coverage model can be given cell by cell. A coverage model is the set of situations a project has decided to count, one cell for each combination of values. Examples drawn from real silicon are complete only where the owner chose to publish, which is rarely where a teaching argument needs them.

The book distinguishes these examples from claims about real organizations. Reports of what a named company did come from and cite a published source, usually that company's own conference paper. Where it reports what the industry does in aggregate, it names the study and its year.

The evidence discipline

The book was written against a closed corpus of documents held in full text: papers, conference proceedings, standards and books. The rule was that a claim either carries a reference into that corpus, or is presented as the author’s position, or does not appear. Nothing is cited from memory of a paper.

Three practices in the prose follow from this rule.

Statistics are dated. A figure about first-silicon success or about where verification effort goes is a measurement of a particular year, and the year is given. First-silicon success is the outcome in which the first fabricated parts are already production-worthy. The author considers several widely repeated numbers in this field older than their use suggests.

Quantities are derived where they can be. When a number can be computed from an example’s own parameters, the book shows the derivation so readers can identify any step they dispute.

Absent evidence is reported as absent. There is no industry-wide data on several questions this book has to address. The author considers reporting this absence more useful than a plausible estimate, because it identifies questions for which readers’ own measurements would be worth more than any citation.

Production and review

The book was written with substantial help from large language models, specifically Anthropic’s Claude: models in the Claude Opus family for drafting, adversarial review and citation checking, and Claude Fable and Opus models for coordination across chapters. The method is disclosed here because readers are entitled to assess it.

Chapters were drafted against the corpus and the style contract. The style contract is the set of rules this book’s prose was written and checked against. An independent adversarial pass then reviewed each chapter, tasked with finding unsupported claims, misattributed citations, incorrect arithmetic and internal contradictions. After the drafting and that review, a third pass applied the findings, with authority to reject the findings it judged wrong. Citation checks were done at page level against the source document. Mechanical gates address the defects that reading misses. A mechanical gate is a check a program settles alone, before anyone reads the text. The gates catch unresolved citation markers. They catch numbers, code blocks or cross-references moved during editing without a stated reason. They catch lost hedges, since dropping *typically* or *up to* strengthens claims beyond their sources. They also catch a caption missing from a figure or a table, and a reference that never resolves to the figure or the table it names. A single term in this book is redefined against common industry usage, and that term is *validation*. In this book the word means checking a design on real hardware, and everything done on a model is verification. A further gate watches that redefined term for drift in either direction. Eight terms neighboring the redefined one are counted rather than enforced, because a gate can compare occurrences but cannot read a sense. A count that

changes deserves a person’s attention rather than a build failure. Each gate was tested against a deliberate defect, and trust in it depended on observing its failure on that defect.

The author’s position on language models editing long documents has three parts. The first is that severe and silent damage is sparsely distributed through a document. The second is that rewriting inflates certainty while appearing to preserve meaning. The third is that on already-clean text, precision in deciding what needs changing collapses. That position rests on the author’s reading of the field and on the editing of this book itself. Each rewritten chapter was checked, independently, against an inventory of its propositions. An inventory of propositions is the list of the claims the chapter made before the rewriting, and each claim is quoted from the sentence that carried it. The check against the inventory kept finding a few serious losses among many propositions preserved, typically a dropped condition or a reservation turned into a certainty. The editorial passes and their mitigations follow from that experience rather than from intuition. Where neither that experience nor the literature offers a measurement, the method reports the absence without borrowing confidence from an adjacent result.

In the author’s assessment, the procedure is limited in two ways. The first is that automated reviewers that share an author’s blind spots miss what the author missed. The second is that source grounding improves an automated reviewer’s judgment of the claims presented to that reviewer far more than it improves the same reviewer’s choice of what to check. Source grounding is putting the cited source in front of the reviewer, so that a claim is judged against an opened page and not from memory. The defects that this method catches are misattribution, contradiction and arithmetic. The method is least effective at catching fluent but wrong passages that nobody thought to question. Readers who find such a passage have found something the process could not find, and the author would like to know.

Acknowledgements

Use of generative artificial intelligence. Anthropic’s Claude was used in preparing this book, as described in “Production and review” above: models in the Claude Opus family for drafting text, for adversarial review of drafts, and for verifying citations against the cited pages, and Claude Fable and Opus models for coordination across chapters. No text was published without being read. The systems are tools, have no authorship and receive no author credit; the author accepts full responsibility for all content, including errors missed by review.

Personal acknowledgements to be written by the author.

Notation and conventions

A reader who wants to check this book rather than trust it needs to know how it is written. This page states the conventions once.

Typography

Monospace marks anything that exists in a file or on a wire: signal names, identifiers, coverpoint and property names, code, file paths, and command fragments. *Italic* marks a term at the point where it is defined, and marks emphasis of a distinction. **Bold** marks the subject of a paragraph in the structured boxes described below, and nothing else.

References and how to check a claim

Citations appear as a superscript number keyed to a numbered list at the end of the chapter. Numbering is per chapter and follows first appearance, so the same source can be reference 3 in one chapter and reference 11 in another; the reference list gives the full citation each time, so no cross-chapter lookup is needed.

Each entry also carries the identifier of the source in the project's corpus. That identifier is not decoration: it means the full text of that document was held and opened, and the claim was checked against the page, not against a summary or an abstract.

Works the corpus does not hold in full text are never cited for a claim. They appear, where they are worth knowing about, under **Further reading**, marked as not held. The distinction is deliberate: a reference is a promise that somebody opened the document.

Standards

A normative document is identified the first time a chapter relies on it, by issuing body, designation, issue or version, and — where a specific requirement is asserted — clause. For example, a requirement of the AXI4 protocol is attributed to *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c (ID012621), with the clause that states it. The designation is given in the form the document itself prints, which is not always the form catalogues use.

Two consequences follow, and both matter for checking. First, the issue is part of the citation, because specifications change: the current issue of Arm IHI 0022 no longer contains AXI4 at all, so an AXI4 requirement can only be attributed to the last issue that specifies it. Second, where a requirement is quoted rather than paraphrased, the quotation is short and marked as a quotation, so that a reader can see where the specification's words end and this book's begin.

Pointers into this book, and pointers out of it

A section sign followed by a bare number always points inside this book: §7.3 is the third section of Chapter 7. It never means a section of anything else, so a reader who follows one never leaves the volume.

A clause of an external document is always preceded by that document’s designation, and its clause identifier carries the form that document uses — Arm IHI 0022H.c §A5.2.2, IEEE 1800-2023 §16.12. The designation is what disambiguates: without one, the pointer is this book’s. Where a document numbers its sections in the same bare style this book does, the word is spelled out instead, as in *the integration specification, section 3.2*, so that no bare §N.M in these pages can be mistaken for an outside reference.

Figures and tables

Every figure and every table is numbered by chapter — Figure 8.1 is the first figure of Chapter 8 — and carries a caption that says what the reader is looking at without requiring the surrounding paragraph. Figure captions sit below the figure, table captions above the table. Cross-references use the number, and the lists of figures and tables in the front matter give the page.

The numbers are assigned by the build from symbolic labels in the sources, so a reference in the text and the float it points at cannot drift apart when a figure is inserted or removed. A reference to a label that does not exist, and a label defined twice, both fail the build rather than producing a wrong number. An uncaptioned figure or table is reported by the build with its line number, so the guarantee in the paragraph above is **checked on every build rather than asserted**.

No figure or table in this book is reproduced from a standard or from another publication. Where a diagram of something a specification defines is needed, it is drawn for this book from the specification’s text.

The chapter apparatus

Chapters open with a statement of the problem on one of the example systems, in engineering terms, followed by a paragraph mapping the sections. Four structured elements recur:

Scenario boxes state a situation, the approach taken and the reason it is the right approach, in that order. They are constructed examples on the example systems unless a source is cited.

In practice boxes report what teams do, including the parts that are inconvenient. Each is either cited to a published source or marked as a constructed illustration; the book does not present unsourced practice as reporting.

Pitfalls list common mistakes as a symptom and a cure, on the assumption that a reader recognises the symptom before the cause.

Summary closes the chapter with the claims it established, not with a recap of its topics.

The example systems

Bare names refer to the **reference SoC**: “the crossbar”, “the DMA engine”, “the core”, “the neural accelerator” are that system’s blocks, with the parameters fixed in the front matter of Chapter 1 and unchanged thereafter. Parts of the larger, later **flagship SoC** are always qualified as such — “the flagship’s compute cluster”, “the flagship’s mesh NoC” — because several blocks exist in both generations at different scales, and an unqualified name would read as the same block twice.

A supporting cast of other devices — a safety-critical microcontroller, a secure element, an analog front end, and a set of IP blocks including a GPU, a video codec, an Ethernet controller and a flash controller — appears where a point needs a device the reference SoC does not have. These are introduced where they are used.

The DMA engine’s parts carry the same names in every chapter. A *transfer descriptor* is the record software writes to program one transfer. A *2D row* is one line of a two-dimensional transfer, a contiguous run of bytes, and the *stride* is the signed address step from the start of one row to the start of the next. The *burst splitter* is the DMA stage that splits each transfer into bursts that the AXI address rules allow; it sits between the register frontend and the AXI backend, and the *4 KB boundary* it must respect is the address line that AXI forbids a burst to cross. A *DMA channel* is an independent transfer engine inside the DMA, with a state of its own and a share of the AXI backend. The design’s own code, and the bug report of Chapter 1, keep the burst splitter’s older name, the midend transfer legalizer, in identifiers such as `cg_legalizer_stress` and `midend_txn_done`; the prose never uses it.

Vocabulary

The glossary in Appendix A is binding, not summarising: where it defines a term, this book means that and nothing else. One case is worth flagging here because the industry uses the word both ways. **Validation** in this book means checking a design on real hardware — on fabricated silicon, or in the lab on a prototype. Everything done on a model, before there is hardware, is **verification**. The distinction being drawn is hardware against model, not fabricated against programmable.

Each chapter is written to be read on its own. A term the glossary defines is recalled, in a clause of its own, the first time a chapter uses it, even when an earlier chapter introduced it; the recall paraphrases the glossary row and never replaces it. Acronyms the field treats as words (AXI, RTL, UVM, DMA, DUT, FFT, ECC) are not expanded. Where one word carries two senses in this book, the chapter that uses the second sense says which one it means: *redundancy* is a second reading of a specification in Chapter 2 and replicated hardware in Chapter 22, *cone of influence* is the logic a proof has to consider in Chapter 15 and the netlist region a fault can reach in Chapter 22.

Quantities

Memory sizes and address boundaries are powers of two: 4 KB is 4,096 bytes, and the 4 KB boundary of the AXI protocol is that boundary. Data rates and clock frequencies are powers of ten: 200 MHz is 200×10^6 Hz. Large ratios are written in scientific notation, as 10^{12} , when the exponent is the point being made.

Two protocol encodings recur and are stated wherever they are used, because both are a common source of off-by-one errors: a burst length field encodes the number of beats minus one, and a coverage bin count is a count of cells in a model, not of tests.

Where a quantity in an example can be derived from that example's published parameters, the derivation is given. Where it cannot, its source is cited. A number with neither is a defect, and finding one is worth reporting.

Exercises and solutions

Each chapter ends with exercises in calculation, engineering judgement and reading an artifact. Their identifiers combine the chapter and exercise numbers. The statements use the example systems defined above; any trace or log supplied in an exercise is constructed input for that exercise. Appendix D gives the reasoning and the relevant section, including the assumptions needed for a calculation. An answer that depends on an unstated premise should identify that premise.

Timing diagrams

Timing diagrams place sampled signal values and property evaluation on the same cycle axis. Lettered payloads distinguish values without assigning an address or a key. The caption identifies the obligation and the trace that illustrates it; a diagram is a worked example, not a simulation result or a proof of the property. Reset behavior and registered checker outputs are read according to the sampling rules stated alongside the property.

Glossary and analytical index

Appendix A retains the English definitions and adds Italian equivalents for terminology. The translation column does not change the meaning of a definition. The analytical index at the end of the book points to printed page numbers and uses cross-references for aliases. Its entries help locate a discussion; the chapter and glossary establish the term's meaning.

PART I

Foundations

Why verification exists, how a verifier thinks, and why the problem is formally impossible — so it is managed as risk.

1. Why Verification Exists

Verification exists because a defect that survives it grows more expensive the later it is found. The reference SoC of this book carries the case: its DMA engine copies rectangular blocks of memory one line at a time, and each line is a contiguous run of bytes, a *2D row*. Consecutive rows start one address step apart; that step is the *stride*, and a negative stride walks memory downwards. The *burst splitter* is the stage that cuts every transfer into bursts that the address rules allow. It sits between the DMA's registers and the AXI backend (AXI: the industry-standard on-chip bus protocol). The rule the splitter must obey above all is the 4 KB address boundary, which AXI forbids a burst to cross: every burst stays inside one address region of that size, so it never spans two subordinates. The DMA has two independent transfer engines, and each of them is a *DMA channel*. The burst splitter splits a two-dimensional transfer incorrectly when the transfer has a negative stride and its row crosses that boundary, and the wrong split corrupts data only while the second DMA channel is competing for the AXI backend that the two channels share. Two hundred and fourteen directed DMA tests had passed for months without ever combining a negative stride and a row that crosses the boundary with a second channel competing for the backend. Software programs each transfer by writing a small record that names the source and destination addresses together with the lengths and strides, the *transfer descriptor*. A constrained-random test generating legal-but-unlikely descriptors by the thousands found the mis-split in one night, as a single failing seed of the nightly regression, eleven days before tape-out, the day the design database is frozen and sent for fabrication; the scoreboard, the testbench component that compares what the design did against what it should have done, reported the mismatch.

Found in simulation, the bug costs a day to fix. Missed there, it survives bring-up, the phase in which a fabricated part and its environment first exchange checked transactions, because the corruption needs a rare alignment and a busy second DMA channel, and reaches a customer as intermittent buffer corruption under load. The same defect then costs weeks of a senior team's time to root-cause with only what the part exposes to look at, a respin with its refabrication and requalification, and a missed launch window. That difference, multiplied across every block of every chip, is why verification consumes most of the engineering effort of a chip project.

CHAPTER MAP

§1.1 defines a bug escape and a respin; §1.2 states how the industry measures verification effectiveness, by required silicon spins for ASICs and by escaped bugs for FPGAs, and where those measures now stand; §1.3 establishes why verification dominates project effort and headcount; §1.4 develops the economics of when a bug is found; §1.5 reads the famous escapes for what they teach; §1.6 introduces the reference SoC that carries this book's examples.

1.1 The problem: correctness is not the default

A digital design is a machine built from millions of interacting state elements, written by humans working from a specification that is itself written by humans. **Functional verification** establishes, before fabrication, that the design implements its specification: that it does what it is supposed to do, and does not do what it must not. It is distinct from **validation**, which checks a design on real hardware rather than on a model (Chapter 20 states the distinction in full). Much of the literature uses the same term more loosely, as an umbrella over pre- and post-silicon work, and the section on effort below flags one place where that looser sense distorts a cost figure. It is distinct too from manufacturing test, which asks not whether the design is right but whether each fabricated die is free of physical defects.

A **bug escape** is a design flaw that survives verification and reaches the next stage: full-chip integration, silicon, or the field. A **respin** is a new fabrication cycle with corrected masks, and it is the consequence when an escape reaches silicon and cannot be worked around. It is also the industry's bluntest verification metric, because it is unambiguous: a project either shipped first silicon or paid for another pass. **First-silicon success**, the fraction of projects whose first fabricated silicon is production-worthy, is therefore the effectiveness headline of the discipline.

A mature, forty-year-old industry still fails at correctness most of the time, and Chapter 3 develops the theory of why: state spaces are too large to enumerate, and deciding whether an observed response is correct is itself a problem, the oracle problem. The practical shape of the difficulty is already visible in the DMA bug of §1.4, which required the *conjunction* of three individually harmless conditions, because designs fail at the intersections: between features, between blocks, between clock domains, between hardware and software. A 10G Ethernet MAC with time-sensitive-networking queues fails the same way, since the timestamping is right and the frame preemption is right while the bug lives where a high-priority frame preempts an in-flight one in the same window the timestamp is captured. Time-sensitive-networking queues are priority queues whose transmission is scheduled, and frame preemption is the interruption of a frame already in flight for a higher-priority one. And the intersections multiply faster than the features do: by 2024, 84 percent of IC/ASIC projects incorporate one or more embedded processors, 95 percent have two or more asynchronous clock domains, 83 percent include security features, and 44 percent follow at least one safety-critical development standard ^[1], and every one of those numbers is a layer of requirements that burdened far fewer projects twenty years ago.

Scenario. The reference SoC integrates a neural accelerator that shares the crossbar with two DMA channels. The accelerator was verified standalone, every convolution mode and weight precision passing, and so was the DMA. **Approach.** Integration verification is still budgeted for the *combination*, accelerator streaming weights while DMA channel 1 fills the next buffer, because block-level correctness does not compose into system-level correctness. **Why.** Arbitration, ordering, and

back-pressure behaviors exist only when both actors are active, so no standalone environment exercises them.

1.2 The numbers: how often the industry fails

The numbers of this section come from two industry-wide surveys that recur throughout the book: a worldwide, double-blind study of 1,886 participants covering 2014, the largest independent one of its kind at the time ^[2], and its 2024 successor (597 eligible participants, ± 4 percent margin at 95 percent confidence), published as separate IC/ASIC and FPGA trend reports ^[1, 3]. Survey data has known biases, candidly documented: regional participation shifts can distort year-to-year language-adoption trends ^[1]. Three editions are now held here, and two tracks run through them: first-silicon success stood at about thirty-two percent in the earliest edition and twenty-four percent in the middle one, both above the rate this chapter reports below; the share of IC/ASIC projects finishing behind their original schedule read sixty-eight percent in the earliest edition and sixty-six percent in the middle one, both below the figure the current edition records ^[1, 4, 5]. The effectiveness trend is consistent across those three editions, and this book claims that consistency for that series alone, because the schedule figures move in two directions and take no trend word. The series runs on a two-year cadence, and at the time of writing the edition quoted here is the most recent published: a further edition was announced for this year's DVCon India, but its figures were not published in time for this book.

Both are Wilson Research Group studies, commissioned by their authors at Mentor Graphics and executed by an outside firm to keep respondents anonymous, and preserving the wording of the earlier Collett International Research studies of 2002 and 2004 so that trends stay comparable; no Collett study followed 2004, and the Wilson series was created to fill that void ^[2]. The 2004 Collett sample was 201 eligible participants against 1,886 in 2014 ^[2].

First-silicon success is falling

In 2014, only about 30 percent of IC/ASIC projects achieved first-silicon success ^[2], and a decade later the figure is 14 percent, the lowest recorded in twenty years, capping a decline that runs unbroken from 2012 to 2024 ^[1]. Six out of seven chip projects therefore pay for at least one extra fabrication cycle. The respin is so common that experienced organizations budget for it.

Plain functional bugs of the kind verification exists to catch remain the leading contributor to respins, ahead of analog, timing, power, and physical categories; the studies label those bugs “Logic or functional flaws”, and multiple flaws can contribute to one respin, so the categories sum past 100 percent [1, 2]. Among the root causes of those functional flaws, design errors lead, and problems traced to changing, incorrect, or incomplete specifications are a persistent second theme [1, 2]. A large share of “hardware bugs” are not coding mistakes but ambiguities and contradictions in what the design was *supposed* to do, discovered late, which is why Chapter 5 treats the verification plan as an instrument for interrogating the specification rather than a list of tests.

Schedule tells the same story: in 2014, 61 percent of IC/ASIC projects were behind their original schedule [2], and by 2024 the historical average of roughly 66 percent had worsened to 75 percent [1]. Verification does not bear sole blame for slips, but it sits on the critical path at the end of the project, so it inherits every upstream delay *and* absorbs the blame for its own.

FPGAs: the illusion of a free respin

FPGA projects have no mask sets, so “respin” seems costless: fix the RTL (register-transfer level, the synthesizable description of the design), rebuild the bitstream, reprogram the part. The data does not bear that out. Asked in 2024 how many non-trivial bugs escaped into production, only 13 percent of FPGA projects answered *zero*, so 87 percent shipped production systems with non-trivial bugs; the study calls that figure notably worse than IC/ASIC first-silicon success rates, while noting that FPGA projects have no metric directly comparable to a respin count [3]. The IC/ASIC first-silicon success rate and the share of FPGA projects that shipped non-trivial bugs therefore measure different things, and once the first is read as a failure share, the gap between those two headline percentages is smaller than the margin of error the studies declare. The diagnosis is cultural: many FPGA teams deliberately minimize pre-lab verification, racing to the lab to validate the design there, a strategy that worked for the simple FPGAs of the past and fails for today’s SoC-class devices [3]. And a field escape is not free even when reprogramming is: in aerospace systems, replacing or re-programming an FPGA requires removing system covers and triggers a full system revalidation, compounding cost and delay [3].

Scenario. A product team reuses the reference SoC’s I/O subsystem in an FPGA-based industrial controller. The schedule review proposes cutting simulation-based verification to three weeks, on the ground that bugs can be shaken out on the bench. **Approach.** The verification lead answers with the escape data [3] and a targeted compromise: full constrained-random verification for the newly modified camera-interface datapath, lab-only validation for the unmodified, previously verified UART and SPI paths. **Why.** Lab debug has a fraction of simulation’s observability: pins and probe points rather than internal state, so bugs found there cost multiples to root-cause. Spending pre-lab effort *where the design is new* buys the most risk reduction per engineer-week, and reuse of verified IP is what lets some projects legitimately spend less on verification [1].

What the trend data is really saying

The outcome got worse across twenty years of methodology, and the 14 percent invites reading as an indictment of that methodology: constrained-random stimulus, functional coverage, assertions, UVM (the standard testbench methodology, IEEE Std 1800.2-2020, *IEEE Standard for Universal Verification Methodology* [6], Chapter 10), and formal verification. But design complexity has grown along every measured axis: gate counts, embedded processors (52 percent of designs in 2004, 71 percent by 2014 [2], 84 percent in 2024, with 58 percent of projects integrating a RISC-V core and 59 percent some form of AI accelerator [1]), clock domains, security, safety. Holding respin rates anywhere near flat against that headwind is itself a productivity gain, delivered by the techniques this book teaches. The 2014 data still supported an optimistic reading: designs had grown dramatically, yet required spins were not getting any worse [2]. The 2024 decline suggests the headwind is currently winning [1]. The remedy's direction is unchanged across the decade: in 2024, projects using formal verification and mature methodologies experienced fewer spins [1]. Staffing, available verification time and adoption of these techniques have not kept pace.

1.3 The effort: who pays, and how much

Nearly every verification talk and textbook repeats that verification consumes “70 percent of the effort.” Exactly this claim has been called “subjective or unsubstantiated”, folklore repeated from publication to publication without primary data [2]. Yet the folklore is well rooted, and each of its versions measures a different whole: as a share of design effort, verification consumes about 70 percent in the SoC era [7]; as a share of design time, multiple chip projects cite approximately 70 percent spent in verification [8]; and as a share of overall time and resources, validation in the umbrella sense (pre- and post-silicon together rather than the real-hardware checking this book means by the word) is estimated at 70 percent in SoC design [9]. That third figure is a reminder to read the scope before the number: the same source puts *post-silicon* validation at half, the figure §1.4 cites.

Measured, the average share of total project time spent in verification was **57 percent** in 2014, essentially unchanged from 2012, with wide dispersion in both directions and a growing tail of projects spending more than 80 percent of their time verifying [2]. Projects assembling pre-verified IP legitimately spend far less, and projects with high proportions of newly developed IP spend far more [1], so the defensible summary is the majority of project time, with “how much more than half” set mostly by how much of the design is new.

Headcount is the other axis, and its trend is steeper. Between 2007 and 2014, demand for design engineers grew at a 3.7 percent compound annual rate, while demand for verification engineers grew at **12.5 percent**, and by 2014 the mean peak number of verification engineers on a project exceeded the mean peak number of design engineers [2]. The growth has since cooled (2016–2024: 2.4 percent for design, 3 percent for verification on IC/ASIC projects [1]), but from that already-inverted baseline a five-to-one ratio of verification to design engineers is not unusual

in some market segments such as processors [1]. The FPGA world is now living the IC/ASIC world's 2007–2014 transition: FPGA verification headcount grew at 8.5 percent CAGR against 4.7 percent for design between 2012 and 2024 [3]. A 2006 guideline reads, two decades later, as conservative for some segments: verification engineers may number up to twice the RTL designers [7].

Designers verify too: in 2024, IC/ASIC design engineers spent on average 49 percent of their time on verification [1], so nearly half of the “design” headcount is functionally verification effort. With a dedicated verification team that equals or outnumbers the design team, well over half of all engineering hours go to verification on a typical project developing new IP.

Within verification, the largest single consumer of an engineer's time is **debug**, ahead of test planning, testbench development and running tests, in 2014 and again in 2024, for both IC/ASIC and FPGA engineers [1, 2, 3]. The management consequence is the same in both years: debug time is unpredictable and varies wildly between projects, so verification schedules built by extrapolating the previous project's effort keep failing [1]. Much of this book's advice is debug-time reduction: self-checking testbenches (Chapter 8), assertions that catch bugs at their point of origin (Chapter 11), disciplined triage (Chapter 12).

In this scenario: staffing the reference SoC

The reference SoC is a small chip by industry standards. A plausible staffing has four RTL designers and five dedicated verification engineers (core and instruction-set simulator (ISS) co-simulation; crossbar and DMA; accelerator; I/O and peripherals; integration and regression infrastructure), with each designer additionally spending near half their time on block-level verification of their own RTL [1]. An out-of-order server core illustrates the scaling behind the five-to-one processor-segment ratio [1]: the design team grows by perhaps three times, while the state space (speculation, replay, coherence, ordering) grows combinatorially, and verification headcount must chase the state space rather than the gate count. That asymmetry, effort scaling with *behavior* rather than *structure*, is why verification's share keeps rising, and Chapter 3 states it formally.

1.4 The economics: what a bug costs, and when

Verification is an economic activity, funded because bugs found late cost more than bugs found early.

Two quantities move in opposite directions as the project advances: the **cost to find and fix** a bug rises while the **time available** to deal with it shrinks [9]. A third quantity sits behind the cost and the time, **observability**: how much of the design's internal state is visible when something goes wrong. In RTL simulation any internal signal is observable; on an FPGA prototype only a few thousand pre-selected signals are observable; on silicon the observable signals are only those the design-for-debug hardware routes to a pin or a trace buffer [9], so less visibility means slower root-causing at exactly the project stage where days are most expensive. The collapse is worst

where the failure leaves nothing to look at. On silicon, an HBM controller (HBM: a stacked high-bandwidth memory) can stall mid-step in its PHY training sequence, the handshake in which the physical layer of a link calibrates itself before data can flow. The stall yields a link that never comes up and a bus with no traffic on it, while in simulation the same stall is a signal a testbench can force, step through and print. [Figure 1.1](#) walks the four stages.



Figure 1.1 — The four stages at which a bug can be found, from block RTL simulation to the field, each labelled with the observability available at that stage and the form a fix takes there. The two degrade together: as the fix moves from an edit and a recompile to a recall or a respin, observability falls from any internal signal at any time to only what the design-for-debug hardware routes to a pin or a trace buffer.

- **Simulation is slow but transparent.** RTL simulation runs roughly a billion times slower than the target silicon at full application-class scale: a workload that takes seconds on the chip, booting an operating system, would take years in an RTL simulator [9]. Simulation-found bugs are cheap because of perfect visibility and an instant fix, but simulation alone cannot reach system-level behavior, which is why emulation and prototyping exist (Chapter 17).
- **Post-silicon debug is measured in weeks.** On IBM’s POWER8, one of the most instrumented and best-staffed programs in the industry, medium-severity post-silicon bugs took on average about 10 days per bug to root-cause, with the 90th percentile at 20 days [9].
- **A respin is refabrication plus requalification.** An escape that reaches silicon and cannot be worked around forces redesign and re-validation as well as expensive refabrication [9]. The mask and fab costs are the visible part, and the dominant term is usually time.
- **The market window dwarfs everything.** Missing a product’s launch window can mean millions to billions of dollars of lost revenue, or missing the market entirely, whether that window is the holiday season or a customer’s design-in deadline, and a bug that reaches the field can add recall costs and reputation damage on the same scale [9].

A widely repeated rule of thumb says the cost of a bug grows by an order of magnitude at each stage: block, chip, silicon, field. The corpus offers no measured cost-per-stage series, so the “10×” is a mnemonic rather than data, but every mechanism above pushes in its direction and none pushes back.

The industry also pre-pays in silicon itself: in some modern SoCs, over 20 percent of die area is devoted to design-for-debug and validation instrumentation [9], spent on observability for bugs not yet found. And post-silicon validation as a whole is estimated in multiple studies at up to 50 percent of a design’s overall cost, measured as time [9]. Pre-silicon verification effort is what keeps that downstream machinery from becoming the primary bug-finding mechanism.

Scenario. During integration of the reference SoC, a formal tool proves a protocol assertion false on the crossbar: under sustained back-pressure, two write transactions from different managers, in flight to the same subordinate, can interleave their data beats, the individual data transfers that make up a burst. The interleaving is an ordering violation, since AXI4 obliges an interconnect to forward merged write data in address order. The counterexample, the input sequence the proof engine returns to exhibit the failure, is 11 cycles long. **Approach.** The team fixes the arbiter and re-proves the property, total cost: two engineer-days. **Why.** This bug class is difficult to diagnose downstream: traffic must combine sustained back-pressure with concurrent writes, which directed tests or random runs can miss, and the lab symptom would be memory corruption that is difficult to reproduce. Caught at RTL by an assertion, it costs days, and the 2024 data associates exactly this maturity with fewer spins ^[1]; caught in the field, it is the “millions to billions” scenario ^[9].

Worked example: anatomy of a near-escape

Here is the opening’s DMA bug, written up the way a competent team files it. The report keeps the burst splitter’s older name, the midend transfer legalizer, as the design’s code identifiers do. Every field of it gets its own chapter later (severity and triage in Chapter 12, escape analysis in Chapter 7, coverage crosses in Chapter 6, where a coverage cross is the combination of two or more coverage axes into cells), and the shape matters now: a bug report is an economic document, recording not only what broke but what the escape *would have cost* and what process change prevents its siblings.

Bug ID:	SOC-1284
Title:	DMA ch0 2D negative-stride transfer corrupts data at 4 KB boundary when ch1 is concurrently active
Found by:	Nightly constrained-random regression (scoreboard mismatch), 11 days before tape-out
Stage found:	RTL simulation, full-chip environment
Severity:	Critical – silent data corruption, no error response raised
Root cause:	Midend transfer legalizer mis-splits a negative-stride 2D row that crosses a 4 KB AXI boundary; the malformed burst is masked unless ch1 owns the backend in the same window (arbitration timing hides it otherwise).
Escape analysis:	214 directed DMA tests passed; none combined negative stride × 4 KB crossing × dual-channel activity. Functional coverage model had no cross of these three dimensions – the hole was invisible to our own metrics.
Cost if escaped:	intermittent field corruption → weeks of bring-up debug at ~10 days/bug post-silicon norms, likely respin + requal
Actions:	(1) fix legalizer split logic; (2) add coverage cross stride_sign × boundary_cross × channel_concurrency; (3) extend scoreboard with end-to-end data check for 2D descriptors; (4) sweep sibling risk: audit all legalizer corner cases against the AXI 4 KB rule.

The directed tests, each written to check one imagined behavior, all passed. The constrained-random environment found the unimagined combination, and the score-

board, not any human, noticed the corruption. A coverage model is the region of behavior that its dimensions and their values mark out, and this model could not even *represent* the hole. Automated checking makes that division of labor portable: a video codec team runs the same discipline against a bit-exact reference decoder, because “the picture looks right” is not a verdict a regression can produce unattended. The one bug already argues for two of this book’s parts: stimulus and checking (Part III), and measurement (Part II).

The verification gap

The pattern of this chapter is a widening **verification gap** between what projects must verify and what they can, and it decomposes into three: an *inherent* gap (unavoidable complexity growth: more IP, more cores, more protocols, more software content); a *transient* gap (skills and tooling that lag but catch up); and a *self-induced* gap (schedule pressure forcing shortcuts, poor environment quality, metrics collected for their own sake, best-known methods sitting unused at an estimated 20 percent productivity cost) ^[10]. The decomposition is a management tool, assigning *innovate* to the inherent gap, *invest* to the transient one and *introspect* to the self-induced one ^[10]. Much of this book is aimed at the third, the gap a team creates and can close without waiting for anyone’s roadmap.

1.5 Public history: when escapes became famous

Most escapes end in an erratum in a datasheet, a software workaround or a stepping nobody outside the team remembers; a stepping is what a bring-up team calls a respin. A few became public, and the narrative that follows is public history rather than cited fact; where the corpus supports a claim, the citation says so.

In 1994 a mathematics professor doing number-theory research found that his new Pentium machines occasionally divided incorrectly, because for rare combinations of operands the floating-point divide unit returned results wrong in the low-order digits. The root cause, as it emerged publicly, was a handful of missing entries in a lookup table used by the division algorithm, a deterministic flaw hard to hit by chance. Intel initially argued that a typical user might never encounter the error in the lifetime of the machine; the argument was statistically defensible and commercially disastrous. The story reached the mainstream press, and the company was eventually forced to offer replacement of the affected chips already in the market ^[8], a recall whose direct cost was widely reported at the time and whose reputational cost was larger. The case remains the canonical example of why failure costs drive verification ^[8].

The FDIV case has implications beyond divider verification. First, *rare* is not *safe*: a fault that a single operation almost never encounters is still met by millions of chips executing billions of operations, and the field finds what the testbench did not, a theme that returns as coverage philosophy in Chapter 6. Second, escape cost is set by the market rather than by the bug: the flaw affected almost no real computation and still forced a replacement program, because trust questioned in public is re-

priced for the whole product line. Third, the aftermath rather than the bug changed the industry: formal verification of floating-point datapaths moved into mainstream practice at processor vendors, and the case study still opens the era’s verification texts [8].

Processor vendors today routinely publish errata sheets, catalogs of post-silicon bugs with workarounds, a practice so normalized that an erratum is only news when it cannot be worked around. And in 2018, the public disclosures of speculative-execution side channels showed that a design can be functionally flawless, with every architectural result correct, and still leak secrets through its microarchitecture: a side channel is a route by which a protected value can be inferred from an observable physical quantity rather than read. The escape there lay in the *specification*, which never forbade the leak, rather than in the implementation. Each famous escape widened what verification is expected to cover, from correctness to safety to security (Chapters 22 and 23), and the data of this chapter shows the industry absorbing exactly those layers [1].

1.6 The reference SoC: the running example of this book

Throughout this book, one design carries the examples: a microcontroller-class, single-core RISC-V SoC for low-power edge applications (*The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508, ratified, RISC-V International [11]). It is a composite, technically faithful to the class of edge-AI microcontrollers shipping today, deliberately anonymous, and made of these elements (Table 1.1):

Table 1.1 — The reference SoC element by element: the design that carries this book’s examples, described in one phrase per element with the canonical parameters — widths, counts, capacities, frequencies — that every later chapter reuses unchanged. Read it as a cast list rather than as a specification; a bare mention of the core, the crossbar or the DMA engine anywhere in the book means the row named here, with these numbers.

Element	Description	Canonical parameters
the core	4-stage, in-order, single-issue RV32IMC core with optional custom DSP/SIMD extensions	~50k gates; OBI-like instruction/data interfaces; M/U privilege modes; CLINT-style interrupts with a fast-IRQ extension
the crossbar	full AXI4 crossbar interconnect (<i>AMBA® AXI and ACE Protocol Specification</i> , Arm IHI 0022H.c, 2021) [12]	5 managers (core-I, core-D, DMA×2, debug) × 4 subordinates (SRAM, boot ROM, APB bridge, neural accelerator); 32-bit address, 64-bit data; ID width 4
the DMA engine	modular multi-channel DMA: frontend (registers), burst splitter (splits each transfer into bursts the AXI address rules allow; code identifiers keep the name <code>midend</code>), backend (AXI4 manager)	2 channels; 1D/2D transfers; max burst 256 beats; 16-entry × 64-bit datapath FIFO in the backend; per-channel error reporting
the I/O subsystem	autonomous I/O engine moving data between peripherals and memory without core intervention	lightweight per-peripheral DMA; UART, SPI, I2S, camera interface

Element	Description	Canonical parameters
the neural accelerator	fixed-point convolution engine for quantized neural networks	2/4/8-bit weights; 16-lane MAC datapath; streams weights from memory; register-programmed convolution jobs
the memory	multi-banked tightly-coupled SRAM, word-interleaved, single-cycle	512 KB in 4 banks; logarithmic interconnect toward core and DMA
the peripherals	APB subsystem (<i>AMBA® APB Protocol Specification</i> , Arm IHI 0024, Issue E, 2023) ^[13]	UART, SPI, I ² C, GPIO, timer, event/interrupt unit
clocking	3 asynchronous domains	system @ 200 MHz, peripheral @ 50 MHz, always-on RTC @ 32 kHz
power	2 UPF (Unified Power Format, the standard power-intent description — IEEE Std 1801-2024, <i>IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems</i> ^[14]) power domains	always-on (RTC, wake-up logic) + switchable system domain; retention on the event unit
debug	RISC-V debug module over JTAG	run-control + abstract memory access

The choice is representative by the numbers. The SoC has an embedded processor (like 84 percent of 2024 IC/ASIC projects), a RISC-V one (like 58 percent), an AI accelerator (like 59 percent), multiple asynchronous clock domains (like 95 percent), and security- and safety-relevant variants in later chapters (like 83 and 44 percent respectively) ^[1]. When this small chip generates a hard verification problem, it is the same problem the median project faces.

Each element is also a deliberate teaching seam. The crossbar carries protocol verification (handshakes, ordering, ID routing) and the formal methods of Part IV. The DMA carries data-mover verification: burst splitting, the 4 KB boundary, error responses, and transfer-descriptor corner cases. The core carries ISA-level verification against a golden ISS, privileged state and interrupt corner cases, and the accelerator carries datapath and quantization corner cases plus memory contention with the DMA. The accelerator’s unit of work is one convolution, programmed through its registers, and the book calls it a *convolution job*. The three clock domains carry clock-domain crossing (CDC) verification (Chapter 13), the two power domains carry low-power verification (Chapter 19), and the register files everywhere carry register-model verification, which checks the testbench’s model of the design’s registers against the design’s own. And a handful of *recurring bugs*, of which SOC-1284 above is the first, resurface across chapters, each time caught or missed by a different technique, which puts the techniques on constant ground.

When scale matters, the examples move up a generation. The **flagship SoC** is its successor: it inherits the first generation’s DMA architecture, register discipline and verification assets. It adds these elements: a host cluster of four application-class RV64GC cores running an OS, whose MESI directory across four private L1s makes coherence a verification problem; a 4×4 mesh NoC and an LPDDR4 subsystem, large enough to move interconnect and memory verification to emulation and system scale; a root of trust that puts security inside the SoC; and processor trace and bus

monitors, the instrumentation a post-silicon bring-up team uses to debug. Its automotive derivative, the flagship-A, carries safety at full-SoC size. The subsystems keep standalone lives too: the **compute cluster** (8 cores sharing multi-banked memory) and the mesh NoC are block-level examples before they are the flagship's. Specialized chapters call in whatever their domain needs: an automotive brake-controller MCU, a secure element, an analog front-end. All obey the same rule: same parameters everywhere, no variants, and a bare name always means the reference SoC's.

IN PRACTICE

Escape analysis should turn records of respins and escaped bugs into changes to the next verification plan. Extrapolating the last project's effort is unreliable because debug, the largest effort component, varies unpredictably between projects ^[1]. Schedule pressure contributes to the self-induced verification gap ^[10]; the verification schedule loses time allowance when design slips are absorbed into it, and the total delay can then appear to originate in verification. The next project's plan should incorporate the escape post-mortem and budget verification staffing at least one-to-one against design, before counting the designers' own verification time ^[1, 2]; passing tests should contribute to the sign-off argument described in Chapter 7 without determining its conclusion.

PITFALLS

1. **Treating verification as a phase that starts when RTL is done.** Symptom: verification team "waiting for a stable drop." Cure: verification starts with the specification, so plan, testbench architecture and assertions proceed in parallel with design (Chapter 4).
2. **Equating "all tests pass" with "verified."** Symptom: green dashboard, no coverage model, tape-out confidence by acclamation. Cure: passing tests measure only what was imagined, so measure coverage and hunt what was not (Chapters 5–7).
3. **Staffing verification as the designers' side job.** Symptom: no dedicated verification owners; designers verify their own interpretation of the spec. Cure: independent verification ownership, staffed to the industry's ratios, with designers' ~50 percent verification time budgeted explicitly ^[1].
4. **Counting on the lab.** Symptom: "it's an FPGA / we'll catch it in bring-up." Cure: 87 percent of FPGA projects ship non-trivial bugs ^[3] and lab observability is a fraction of simulation's ^[9], so move effort pre-lab for everything newly designed.
5. **Costing a respin as masks-plus-fab.** Symptom: risk register prices an escape at the refabrication invoice. Cure: price the schedule slip and the market window, whose corpus-documented range is millions to billions ^[9].
6. **Skipping the escape post-mortem.** Symptom: bug fixed, everyone relieved, nothing changes. Cure: every escape (and near-escape) is a defect in the *process*;

file the SOC-1284-style analysis and change the plan, the coverage model, or the checklist it exposed.

SUMMARY

- Verification exists because escapes are ruinous: bug cost rises and fix time shrinks at every project stage, from editable RTL to recalled product [9].
- The industry, measured: 14 percent IC/ASIC first-silicon success in 2024, a twenty-year low [1], and 87 percent of FPGA projects shipping non-trivial bugs [3].
- Functional flaws remain the leading cause of respins; their root causes are design errors, with changing or ambiguous specifications a persistent second theme [1, 2].
- Verification is the majority of project effort: 57 percent of project time on average a decade ago [2], verification headcount at or above design headcount [1, 2], and debug the biggest single time sink within it [1].
- The classic “70 percent” figure is folklore with roots in the literature [2, 7, 8, 9]; the defensible claim is “more than half, growing with the fraction of new IP.”
- Verification is economics: risk reduction bought with engineer-time, and the techniques that buy it cheapest early (assertions, formal, constrained-random with strong checkers) are the subject of this book. Perfection is not on offer, and Chapter 3 explains why.
- The reference SoC introduced here is the book’s running example; its parameters do not change, and its bugs will return.

EXERCISES

Exercise 1.1 (calculation). Take the 2024 IC/ASIC first-silicon success rate of §1.2 and compute how many projects out of a hundred pay for at least one further fabrication cycle. Reconcile that figure with the “six out of seven” the same paragraph states, showing the arithmetic that connects them. Name the population the rate describes and the population it does not.

Exercise 1.2 (judgement). A program review places the 2024 IC/ASIC first-silicon success rate and the share of FPGA projects that shipped non-trivial bugs side by side on one slide, and concludes that FPGA verification is the weaker discipline. Decide what the two figures together let the slide conclude, and name the two properties of those figures that limit that conclusion. Point to the sentence of §1.2 that carries each property.

Exercise 1.3 (calculation). Take the staffing of the reference SoC in §1.3 together with the average share of an IC/ASIC design engineer’s time that went to verification in 2024, counting each engineer as one full-time equivalent. Compute the fraction of that project’s engineering hours that goes to verification. Compare the result with the claim §1.3 closes on, and state which of the two staffing numbers dominates the sum.

Exercise 1.4 (judgement). A program manager budgets verification at 70 percent of project time, citing the figure the literature repeats, on a project that assembles mostly pre-verified IP. Decide what §1.3 licenses the plan to budget instead, and give the measured figure that section reports together with the two directions in which it disperses. Name the scope error in the variant of the repeated figure that comes from the post-silicon literature.

Exercise 1.5 (artifact). Read the write-up below, filed on the crossbar ordering violation of §1.4, against the template of the worked example in the same section. Name the two absent fields that carry the report’s economic argument, and state what the report loses without each. Name the pitfall of the chapter’s list that each omission matches.

Title:	Crossbar write-data beats of two managers interleave at one subordinate port under sustained back-pressure
Found by:	Formal proof of a protocol assertion, 11-cycle counterexample
Stage found:	RTL, block-level formal environment
Severity:	Critical, ordering violation with silent memory corruption
Root cause:	The write arbiter releases the grant mid-burst while an earlier burst is still unfinished on the write-data channel.
Actions:	(1) fix the write arbiter; (2) re-prove the property.

FURTHER READING

From the corpus: the two 2024 Wilson Research Group trend reports, IC/ASIC ^[1] and FPGA ^[3], are short, still the most recent published edition at the time of writing, and worth reading in full; Foster’s DAC 2015 paper ^[2] adds the methodological background of the study series and the 2007–2014 trend arcs. Mishra et al.’s post-silicon tutorial ^[9] is the best single source on the economics of late-stage bugs. Bergeron’s opening chapter ^[7] and Bertacco’s introduction ^[8] give the early-2000s view of the same crisis, instructive because the ratios have barely moved. The Oracle Labs “verification gap” presentation ^[10] is a compact management vocabulary for everything in this chapter. The 2020 reports, IC/ASIC ^[4] and FPGA ^[15], and the 2022 white papers, IC/ASIC ^[5] and FPGA ^[16], are held here too and supply the two intermediate points of the effectiveness series.

Not in the corpus (cited here as general references only): Wile, Goss and Roesner, *Comprehensive Functional Verification* (Morgan Kaufmann, 2005), the classic full-lifecycle treatment; the Collett International Research functional verification studies of 2002 and 2004, the ancestors of the Wilson series, known today primarily through the trend baselines they established; and the contemporary press coverage of the 1994 Pentium FDIV recall, which put the direct charge at roughly half a billion dollars, a figure quoted everywhere and supported nowhere in this book’s corpus, and therefore kept out of the chapter’s argument.

REFERENCES

1. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
2. **P17** H. D. Foster, "Trends in Functional Verification: A 2014 Industry Study," Proc. 52nd Design Automation Conference (DAC '15), San Francisco, CA, 2015.
3. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
4. **R3** H. D. Foster, "2020 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2020.
5. **R5** H. D. Foster, "2022 Wilson Research Group IC/ASIC Functional Verification Trends," Siemens EDA white paper, 2022.
6. **S9** IEEE, "IEEE Std 1800.2-2020, IEEE Standard for Universal Verification Methodology Language Reference Manual (Revision of IEEE Std 1800.2-2017)," IEEE, 2020.
7. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
8. **B3** V. Bertacco, "Scalable Hardware Verification with Symbolic Simulation," Springer, 2006.
9. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
10. **D16** R. Narayan and T. Symons, "I created the Verification Gap," Proc. DVCon U.S., 2015.
11. **S14** RISC-V International, "The RISC-V Instruction Set Manual Volume I: Unprivileged Architecture, Version 20250508 (Ratified)," RISC-V International, May 2025.
12. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
13. **S19** Arm Limited, "AMBA APB Protocol Specification, ARM IHI 0024, Issue E (ID022823)," Arm Limited, February 2023.
14. **S10** IEEE, "IEEE Std 1801-2024, IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems (Revision of IEEE Std 1801-2018)," IEEE, 2024.
15. **R4** H. D. Foster, "2020 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2020.
16. **R6** H. D. Foster, "2022 Wilson Research Group FPGA Functional Verification Trends," Siemens EDA white paper, 2022.

2. The Verifier's Mindset

Verification requires an independent interpretation of the specification. The DMA engine shows what the difference between designing and verifying costs when one mind holds both roles. Its designer wrote the register frontend, the AXI4 backend tuned for full-bandwidth bursts, and, between them, the burst splitter, the stage that splits each transfer into bursts that the AXI address rules allow. She verifies the block herself with a directed suite written in one week: register defaults and read/write paths, 1D transfers of assorted lengths and alignments, a 2D transfer, a deliberately erroneous address to check the per-channel error report, and completion interrupts. Everything passes, and the block is declared verified and integrated.

Three months later a system-level constrained-random regression fails on one seed in four hundred. Constrained-random is the automatic generation of legal but unlikely stimulus under constraints. A seed is the value that fixes a run's random draws, so that the run can be repeated. The scoreboard is the checker that compares the memory image produced by the DMA against a reference prediction. The scoreboard flags corrupted data only where three conditions meet: the transfer is 2D, with a *negative* row stride; the row straddles the 4 KB boundary, the address line that AXI forbids a burst to cross (Arm IHI 0022H.c §A3.4.1) ^[1]; and DMA channel 1 is concurrently streaming audio buffers. A row stride is the signed address step from the start of one row to the start of the next. Each DMA channel is an independent transfer engine, and the DMA has two of them sharing one AXI backend. The burst splitter, which must split any burst that would cross the 4 KB boundary, computes the split point on the assumption that addresses ascend. A negative stride defeats that assumption, but the splitter emits a wrongly split burst only when a switch between the two DMA channels also disturbs its internal state, and the write then lands partly in the wrong 4 KB region. None of the designer's tests could have found this, not because she tested carelessly but because of *how she thought about testing*: her stimulus exercised the block the way it was meant to be used, and her checks confirmed her own understanding of the spec. Her judgment of what counted as "valid" excluded negative strides across the 4 KB boundary, and that exclusion pruned away exactly the region of the input space where the bug lived.

CHAPTER MAP

§2.1 sets the two mindsets against one specification and shows why a designer who verifies her own block verifies her interpretation rather than the specification; §2.2 develops thinking in terms of breakage and the corner cases the specification never mentions; §2.3 states what independence of judgment requires; §2.4 makes systematic doubt the verifier's core professional asset.

2.1 Designer and verifier: two minds, one specification

A designer constructs a component that correctly implements the intended function, with the required performance and, ideally, without defects. A verifier assumes bugs are present and seeks them by stressing protocols, exploring corner cases and applying zero tolerance toward inconsistencies [2]. The designer's success is a mechanism that works and the verifier's a demonstration of where it does not, so the two are different orientations toward one specification, and each produces systematically different decisions in stimulus, in checking, and in what gets questioned rather than trusted.

The framing is Verilab's, set out in a DVCon paper as a series of concrete choices a verification engineer has to make rather than as an attitude to be adopted [2].

In 2024 design engineers spent on average 49% of their time on verification activities [3]. In some segments, such as processors, teams field up to five verification engineers per designer [3]. The distinction is therefore between *mindsets* rather than job titles, and a designer doing verification with a design mindset is the situation of the opening. Hiring screens candidates on languages, on methodologies and on tool experience ("knows UVM", "has experience with assertions"; UVM is the standard testbench methodology, the class library and interface set that teams build their environments on), and leaves unmeasured what matters most to verification effectiveness: the engineer's mindset. Tools can be learned in months, whereas the mindset usually arrives only through years of experience, mentoring, and lessons from failures [2].

Self-verification is structurally blind

Bergeron's *reconvergence model* accounts for why the opening had to end the way it did: verification reconciles two paths from a common origin. The first path is a transformation such as writing RTL from a specification, and the second is an independent path back to that origin. What is actually verified is determined by where the two paths reconverge [4].

When the DMA designer verifies her own block she reads the specification once and forms an interpretation, and from that *same* interpretation she writes both the RTL and the tests with their expected results, so the two paths reconverge at her interpretation rather than at the specification. If that interpretation is wrong anywhere, say by reading the 4 KB rule as a constraint on *ascending* burst addresses, the verification faithfully confirms that the design matches what she believed the spec to mean [4]. The rule constrains address boundaries, not address direction, and the Note in Arm IHI 0022H.c §A3.4.1 gives the reason: the prohibition "prevents a burst from crossing a boundary between two Subordinates" [1]. Figure 2.1 sets the two paths side by side.



Figure 2.1 — The reconvergence model drawn for the DMA of this chapter's opening: the same specification, the same RTL, and two ways of arriving at the tests. In the self-verification half one interpretation feeds both the RTL and the tests, so the comparison can only confirm that interpretation — including where it is wrong. In the independent half a second engineer's reading of the specification feeds the testbench instead, and the two paths reconverge at the specification itself. One node differs between the two halves; §2.3 adds that the independent interpretation bought by this second reading pays off only if the verifier's judgment is independent too.

Three mechanisms reduce human-introduced error in any process: automation, mistake-proofing (poka-yoke) and redundancy. Design remains too creative an act for the first two, so hardware verification rests on the third, the simplest and the most costly: a *different person* independently interprets the specification and checks the implementation against it. Verification rests on redundancy and consumes about 70% of design effort (see Chapter 1 for the economics). The industry pays willingly, because a respin costs more than the redundancy [4].

Scenario. The crossbar team is short-staffed, and the engineer who wrote the DMA's AXI backend offers to “help verify” the crossbar, a block she did not design. **Approach.** The team accepts the offer because she can interpret the crossbar specification independently. She reads the crossbar spec fresh, builds her own expectation of ID-ordering rules, and her checks encode *her* reading, not the crossbar designer's. An ID-ordering rule says which responses may come back out of order and which must keep the order they were issued in. **Why.** Independence attaches to the *artifact*, not the person. A designer can apply her design skills to verifying someone else's block while forming an independent interpretation. What redundancy forbids, as the reconvergence model implies [4], is only the closed loop of the opening, one mind on both paths.

2.2 Thinking in terms of breakage

If the first shift is *who* interprets the spec, the second is *what the testbench is trying to make happen*: the designer's instinct is to make the block work, the verifier's discipline to make it fail. The line runs between *interoperating* with a design and *stress-ing* it, because every testbench must interoperate at some level while a testbench built with a design mindset slides into cooperating with the very behavior it should be interrogating [2].

A verification component is the reusable testbench-side model of an interface, with a driver to stimulate it and a monitor to observe it. The case for such a component on a protocol that requires clock-data recovery rests on a paradox of robustness. The design mindset says that the protocol specifies CDR, so the monitor should implement CDR as robustly as possible. The verification mindset notices that the more ro-

bust the recovery algorithm, the *less checking it performs*, since a tolerant monitor adapts to data rates that are out of spec and masks the bug it exists to catch. The right answer is the least tolerant observer that the reference clock permits, even though no real receiver would be built that way, because verification does not set out to replicate reality; it sets out to check the design as thoroughly as possible, “even if it means doing so in spite of reality” [2]. The same trap waits on a 10G Ethernet MAC, where a receive monitor accepts every frame, including the ones a compliant link partner would have rejected, because its elastic-buffer model absorbs clock drift beyond what the specification permits. An elastic buffer is the queue a receiver uses to take up the difference between the transmitter’s clock rate and its own.

The same logic recurs wherever the testbench could be *helpful*:

- **Handshakes.** A protocol says a receiver NACKs a corrupted transfer. A design-minded verification component dutifully auto-NACKs, and the auto-NACK both lets a simulation that should have failed keep running and masks an unprovoked error from the DUT (the design under test). The verification-minded component logs an error and fails the test; NACK generation exists only under explicit test-case control for error injection [2].
- **Status outputs.** The I/O subsystem’s FIFOs export `full` and `empty` flags, and every test wants them to pace stimulus. If they are used unchecked, they leave the testbench regulating itself by signals that may themselves be buggy. Check them first, with a white-box assertion against the FIFO pointers, or with an independent count of items in and out, and only then rely on them. A white-box assertion is one that checks internal design state rather than only the ports. The same rule bans “snooping” a DUT’s internal clock as the testbench’s own timebase [2].
- **Error recovery.** A testbench does not need to survive the errors it detects. Once a check fires, downstream checking is likely compromised anyway, so the correct default is to stop on first error [2].

Corner cases: “invalid” is not a stimulus filter

The designer’s second reflex after helpfulness is validity pruning: *that input can’t happen, so don’t test it*. Whether a scenario is valid or invalid is irrelevant, since the only question is whether it *can happen*, and if it can, the design’s response to it must be verified. In one reported production case a designer would dismiss a zero-radius circle as meaningless input, while software upstream in that very system generated such circles routinely, without that designer knowing it. In the other, a low-power mode bit was tested by the obvious sequence (write 1, check entry; write 0, check exit) and passed until tape-out; writing 0 to the bit *when it was already 0* turned out to put the device into low-power mode [2]. Nothing in the specification suggests either test.

Scenario. The inline AES-GCM crypto engine sits on the memory path, and its feature list describes encryption under a loaded key, so the team assumes nobody starts an encryption without loading one, which describes intent rather than reachability. The register interface permits it. **Approach.** Enumerate what the interface

allows rather than what the block is for, and write a test for each: start an operation with an empty or stale key slot; overwrite the key while an operation is in flight; program a length field that disagrees with the number of beats actually streamed; fail the authentication tag on a message whose plaintext the engine has already pushed downstream. **Why.** Every one of these is reachable from the register interface and none appears in any feature list. On a data-mover a permitted-but-unintended input corrupts data and a scoreboard catches it; here the same class of input leaks key material or plaintext, and in the last case the engine may compute exactly the right bytes and still have released them to a consumer that should never have seen them. That release is a security failure rather than a functional one, and it is a fault in *when* data moved, invisible to a value-comparing oracle, the mechanism that decides whether an observed response is correct.

The same lens on the opening starts from the DMA spec: 1D and 2D transfers; strides; bursts of up to 256 beats, the individual data transfers that make up a burst; and, inherited from the AXI protocol, the rule that no burst may cross a 4 KB address boundary (Arm IHI 0022H.c §A3.4.1) ^[1]. The designer's test list came from the feature list. The verifier instead interrogates each feature for its breaking points and crosses them: stride sign $\times$ boundary proximity $\times$ channel concurrency $\times$ burst length. "Negative stride" is on the list not because anyone plans to use it that way but because the register field is signed, so it can happen. The cross with "row lands within one burst of a 4 KB boundary" and "other channel active" is exactly the cell where the burst splitter's bug lives, and a constrained-random generator steered by that coverage model visits that cell as a matter of course, which is how the bug was caught.

Worked example: attacking the burst splitter. The design's own code still calls it the midend legalizer, and the identifiers in this section keep that name. First, the property: the designer thinks "the splitter splits at 4 KB", the verifier states the *prohibition* at the output and lets the tool hunt for any way to violate it:

```
// No INCR write burst emitted by the DMA backend may cross a 4 KB page,
// regardless of how the midend legalized the transfer. (FIXED bursts hold
// their address; WRAP bursts stay inside their aligned container.)
// 16-bit arithmetic on purpose: an illegal awsize (>3 on this 64-bit bus)
// must overflow the comparison and fail – not wrap silently and pass.
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && awready && (awburst == 2'b01)) |->
      (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

Second, the functional-coverage model that encodes the breakage hypothesis is a covergroup, SystemVerilog's construct for declaring which situations must be observed before coverage is called closed. Its coverpoints each sample one axis of the space into the bins worth counting, and its cross demands their combinations. The covergroup below would have exposed the directed suite's blind spot as a block of empty bins long before any failure:

```

covergroup cg_legalizer_stress @(posedge desc_accepted);
  cp_stride : coverpoint desc_stride_sign { bins pos = {0}; bins neg = {1}; }
  cp_bnd    : coverpoint bytes_to_4kb_boundary {
    bins at_bnd = {0};
    bins in_burst = {[1:2048]}; // crossing reachable in one burst, or
    exactly filled by one
    bins far = {[2049:4095]}; }
  cp_len    : coverpoint desc_burst_beats {
    bins single = {1}; bins mid = {[2:255]}; bins max_beats = {256}; }
  cp_ch1    : coverpoint ch1_active { bins idle = {0}; bins busy = {1}; }
  x_attack  : cross cp_stride, cp_bnd, cp_len, cp_ch1;
endgroup

```

The property is the zero-tolerance check; the cross is the map of where to apply pressure. Even the checker gets the breakage treatment: its 16-bit arithmetic makes an illegal `awsiz` fail loudly rather than wrap past the check. Neither artifact took more skill than the directed tests, the ones written out by hand to do the same thing on every run. The property and the covergroup only asked a different question: not “does it transfer?” but “how could the split go wrong, and under what interference?”

2.3 Independence of judgment

Redundancy buys an independent *interpretation*, and it pays off only if the verifier also maintains independent *judgment*, the willingness to reach and defend conclusions the designer disagrees with; three disciplines protect it.

Predictions derive from the specification. The verifier’s reference for correct behavior, the scoreboard’s expected data image and the monitor’s legal responses, must be derived from the spec and never reverse-engineered from what the RTL does. A checker tuned until the design passes has collapsed the reconvergent path back onto the implementation, so the design is once again verified against itself. The same holds at planning level: the verification plan must be based on the *intent* of the design rather than on its implementation, and the corner cases that the implementation creates, like the burst splitter’s split arithmetic, are verified in addition, after the intent-based plan stands ^[5]. When a test fails, “the testbench must be wrong” and “the design must be wrong” deserve exactly equal initial credence, and the spec rather than seniority arbitrates.

DUT outputs require independent checks. Every time the testbench consumes a DUT output without independent checking, whether a status flag, a ready signal or an interrupt line, it delegates a piece of its judgment back to the object under suspicion ^[2]. The DMA’s per-channel error report is a case in point: a lazy environment uses it as ground truth for “this transfer failed”, so a bug that *under-reports* errors silences the very tests meant to catch it. The independent environment predicts from stimulus which transfers must fail and treats the error report as one more output to be checked.

Verifiers connect architectural intent with implementation. Designers implement at module level; verifiers, building an environment that must exercise and

check the whole block, necessarily operate one level of abstraction up. That position makes the verification engineer the natural liaison between architect and designer. The verifier is often the first to detect that specification and implementation have diverged, and is the one whose effort suffers most from inconsistency and “nice-to-have” late additions. Exercising the role takes a professional willingness to say “no, this is not the way to do it” [2], and to say it to a designer with more years on the block, to a schedule that wants the block closed, and sometimes to the verifier’s own earlier plan.

Scenario. During integration, the DMA designer reviews the verifier’s failing seed and replies: “negative stride across a page boundary is a software error; document it as a restriction and move on.” The tape-out review is in two weeks. **Approach.** The verifier does not argue about intent but asks the operative question: can the programming interface express it? It can: the stride register is signed, and nothing in the frontend rejects the transfer descriptor, the record that programs one DMA transfer. So either the RTL is fixed, or the frontend must *detect and reject* the case and a test must prove the rejection. “Software will never do it” is accepted only as a documented, checked restriction, never as an untested assumption. **Why.** The verifier’s judgment, that reachable means testable, is what the team pays independence for [2].

2.4 The value of doubt

What ties the mindset together is a disciplined, impersonal doubt: not cynicism about designers, but a refusal to accept comfort as evidence. The doubt is impersonal because it points equally at the testbench and at the verifier’s own reasoning.

Passing regressions require checked outcomes. A regression at 100% pass is not good news until the checks are known to be real. The rule of *no coverage without checking* is the operational form of that doubt, because a coverage bin that is hit while nothing verified the outcome converts ignorance into false confidence and is worse than an empty bin. A passing-test count used as a progress metric produces the same false confidence, since the verification mindset treats testcases as vehicles toward checked coverage closure and nothing more [2]; a growing pass rate over an unchecked design measures only the testbench’s tolerance. Coverage closure is the point at which every item of the verification plan has been covered or consciously excluded. When everything passed in the opening, the reading was that *no check had fired*.

Failure-free blocks require scrutiny. Bugs cluster where scrutiny hasn’t been, so a block that has never failed a test is either simple or under-attacked, and the verifier’s zoom-out instinct is to ask which and to reallocate effort accordingly. A flash controller that has never failed a data-integrity test invites both readings: the low-density parity-check (LDPC) code in its ECC pipeline may be genuinely solid, or the error-injection campaign may only ever have injected patterns the code was always going to correct. The regular self-audit that decides between the two readings is

zoom-out thinking: each week it re-asks what most needs breaking, weighed against the design's hot spots, the schedule and what can move post-tapeout, and it asks of every environment feature, every metric and every meeting what that item is trying to accomplish [2].

The testbench requires falsifiable checks and debug visibility. Verification engineers spend more of their time on debug than on any other activity [3], and a good share of what they debug is the testbench itself. Doubt therefore extends to engineering for falsifiability: the messaging must be rich enough to debug from logfiles, since much of a testbench's behavior, such as zero-time execution and dynamic objects, never appears in waveforms, and the interfaces must be structured so that a waveform shows *who drove what* [2].

The plan defines measurable success. The verification gap is the shortfall between what a project must verify and what it can. A substantial part of that gap is self-induced: schedule pressure forces shortcuts, best-known methods are skipped as low priority, and engineer productivity varies up to tenfold [6]. If success is not defined, it cannot be falsified, so the verification plan is the definition of what "verified" will mean for this design (see Chapter 5) [5].

Scenario. Four weeks from the coverage-closure milestone, the weekly zoom-out review notices what day-to-day work cannot: the neural accelerator has not failed a test in a month, while the DMA bug absorbed everyone's attention. **Approach.** A verifier is reallocated to attack the accelerator. The accelerator's unit of work is one convolution, programmed through its registers, and the book calls it a *convolution job*. Her first find is an environment defect rather than a design one: because the messages never say which convolution job or which multiply-accumulate lane a mismatch belongs to, earlier failures had been triaged slowly and twice waived as "environment noise". She fixes the messaging first; the reopened failures then yield a real bug in a low-precision weight mode. When the schedule pushes back, the verification plan, agreed before the crunch, arbitrates: accelerator-versus-DMA memory contention stays pre-tapeout, performance corners move to post-silicon. The crunch is the end-of-project concentration of the hardest verification work against a date that cannot move. **Why.** The zoom-out cadence, logfile-first debuggability and pre/post-tapeout prioritization are the disciplines that keep doubt operative when pressure peaks [2].

IN PRACTICE

- **Small teams cross-verify.** Two designers exchange blocks, which preserves interpretation independence [4] and leaves the social side to manage, where reciprocal politeness ("I won't press on your block if you don't press on mine") is the failure mode.
- **The designer is a source, not an oracle.** The verifier should interview the designer for hot spots, asking which parts she was least sure about, while keeping pass/fail authority and the checkers on the verification side.

- **Self-verification requires independent artifact review.** When the DMA designer must verify her own block, the following artifact reviews should compensate for the shared interpretation: the verification plan (vplan, Chapter 5) and the coverage model are reviewed by someone else against the spec, assertions are written from the spec text rather than the RTL, and an independent engineer owns the scoreboard's reference model, breaking the interpretation loop at the checks even if one person writes the stimulus.
- **Mindset is taught, not assumed.** Since design engineers already spend about half their time verifying ^[3], mindset training should be explicit, with review checklists that ask “what would break this?”, bug post-mortems that trace escapes to mindset decisions, and mentoring pairs across the design/verification line (see Chapter 24).

PITFALLS

1. **Mirroring the design in the testbench.** Symptom: the monitor implements the same clever algorithm as the DUT, and everything passes. Cure: build the least tolerant observer the spec allows; predict from the spec, not the RTL ^[2].
2. **Validity pruning.** Symptom: “that can't happen” removes stimulus before anyone checks whether it is reachable. Cure: if the interface can express it, drive it; validity claims become checked rejections, not silence ^[2].
3. **Trusting unchecked DUT outputs.** Symptom: tests pace themselves on `full/empty` flags or DUT clocks that no check covers. Cure: check a signal before the environment relies on it ^[2].
4. **Counting tests instead of checked coverage.** Symptom: progress reported as “412 of 415 passing”. Cure: report checked functional coverage; treat testcases as vehicles ^[2].
5. **The helpful testbench.** Symptom: auto-NACKs, silent retries, error recovery paths that keep failing simulations alive. Cure: fail fast and loud; stop on first error by default ^[2].
6. **Independence in title only.** Symptom: the checker is “fixed” until the design passes, with the designer arbitrating. Cure: the spec arbitrates, and discrepancies close with a spec clarification, a design fix, or a testbench fix, each recorded as such.

SUMMARY

- Designer and verifier hold different orientations toward one specification, constructing correct function versus proving where it breaks; the difference is a mindset rather than a job title, and mindset, not tool knowledge, is the strongest determinant of verification effectiveness ^[2].
- Verifying one's own design reconverges on the designer's interpretation rather than the specification, so a misread spec escapes by construction; independent

interpretation, which is redundancy, is the mechanism hardware verification is built on [4].

- A testbench built for interoperability can tolerate behavior that verification must reject. Verification may legitimately ignore reality to maximize rigor [2].
- “Invalid” is not a stimulus filter. If a scenario is reachable, its outcome must be verified: the bugs that kill projects live outside the intended-use envelope [2].
- Judgment must stay independent at every scale: reference models are derived from the spec, no unchecked DUT signal is trusted, and the professional duty to say no is supported by a vplan agreed before schedule pressure [5].
- Doubt is symmetric, covering the design, the testbench, the metrics and the schedule. No coverage without checking; no pass rate without asking what the checks would have caught [2].

EXERCISES

Exercise 2.1 (judgement). The schedule leaves the DMA designer verifying her own block. The team proposes that she write the assertions and the scoreboard’s reference model, reading both off the RTL she wrote, while a second engineer writes the stimulus. Decide where the two paths of the reconvergence model of §2.1 meet under that division of labor, and give the division that the chapter’s practice list prescribes instead. Name the artifact whose ownership decides the outcome.

Exercise 2.2 (judgement). A verification component is needed for a protocol that requires clock-data recovery, and the team offers the receiver IP’s own recovery algorithm for the monitor, on the ground of fidelity to a real receiver. Decide which observer §2.2 requires, and give the mechanism by which the more tolerant one loses checking power. Name the form the same mistake takes on the receive monitor of a 10G Ethernet MAC in §2.2.

Exercise 2.3 (calculation). Count the coverpoints of `cg_legalizer_stress` in §2.2 and the bins each one declares, then compute the number of cells the cross `x_attack` demands. Name the axis whose second bin exists only because a register field is signed, and quote the reason §2.2 gives for placing that bin in the model.

Exercise 2.4 (calculation). Take the property `p_no_4kb_crossing` of §2.2 together with the bus width its comment names. Compute the largest byte offset inside a 4 KB region at which an INCR write burst of 256 beats, at the largest legal `awsize`, can start and still satisfy the property. Compare that offset with the upper edge of the `in_burst` bin of `cg_legalizer_stress`, stating what the two quantities share and how they differ.

Exercise 2.5 (artifact). Read the regression summary below, produced by the DMA’s new random environment. State what the summary establishes about the burst splitter, and name the rule of §2.4 that the summary’s last line puts in question once its first two lines are offered as closure evidence. Name the pitfall of the chapter’s list that the tests line matches on its own.


```

regression summary
  tests run / tests passed      415 / 415
  cg_legalizer_stress  x_attack 100%
  scoreboard comparisons executed 0

```

FURTHER READING

From the corpus. Montesano & Litterick, *Verification Mind Games* ^[2], seven pages of concrete mindset calls, is this chapter's backbone. Bergeron's *Writing Testbenches using SystemVerilog*, Chapter 1 ^[4], develops the reconvergence model and the human factor. The *Verification Methodology Manual*, Chapter 2 opening ^[5], connects mindset to planning discipline. The 2024 Wilson Research Group IC/ASIC report ^[3] supplies this chapter's workforce data, and Narayan & Symons' *I Created the Verification Gap* ^[6] is a candid practitioner taxonomy of how teams inflict the gap on themselves.

Outside the corpus (not citable here, worth owning): Wile, Goss & Roesner, *Comprehensive Functional Verification* (Morgan Kaufmann, 2005), the broadest treatment of the verification engineer's role; Myers, *The Art of Software Testing* (Wiley, 1979), the original articulation, from software, of testing as a destructive rather than constructive act; Weinberg, *The Psychology of Computer Programming* (1971), on egoless engineering and why independent review works on humans, not just on code.

REFERENCES

1. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
2. **D21** J. Montesano and M. Litterick, "Verification Mind Games: how to think like a verifier," DVCon, 2014.
3. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
4. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
5. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
6. **D16** R. Narayan and T. Symons, "I created the Verification Gap," Proc. DVCon U.S., 2015.

3. The Verification Problem

The first question a verification plan for the reference SoC’s DMA engine must answer is what *testing all the cases* costs on a block of two channels and a few registers. Each channel exposes 192 bits of architecturally visible state and the two together expose 384, so the register states number 2 to the 384th power, roughly 10 to the 115th, and stripping the sixteen read-only status bits still leaves 2 to the 368th programmable. And that counts only the *register* values. §3.1 adds up the contributors to the state space one by one: the register map, the backend FIFO, the product of the two channels, and the input history that reaches them. Exhaustive testing of this block is therefore out of reach, and every competent verification plan is built on top of that answer.

CHAPTER MAP

§3.1 computes the state space and shows why exhaustive verification is impossible and what formal verification changes about that; §3.2 develops controllability and observability; §3.3 states the oracle problem; §3.4 makes the pivot from completeness to managed risk.

3.1 The state-space explosion

A synchronous digital design is a finite-state machine: a set of flip-flops and memory bits (the state), combinational logic that computes the next state from the current state and the inputs, and outputs. Finiteness invites the thought that the states can all be visited, but the arithmetic rules it out, because each state bit doubles the number of possible states and a design with n state bits has 2^n of them ^[1]. Even a 256-bit RAM, thirty-two bytes of storage, already yields about 2^{256} possible contents, and exhaustive simulation of all input combinations is “practically impossible” for designs of any realistic size ^[2]; the reference SoC’s 512 KB SRAM holds over four million state bits, and the core alone, at ~50k gates, carries a few thousand.

Exhaustive testing of nontrivial designs is therefore generally infeasible, so testing provides at best *probabilistic* assurance that the design is correct ^[3]. Simulation visits only a small fraction of a system’s possible configurations, which is why the discovery of bugs leans heavily on someone’s expertise in choosing *which* few configurations to visit ^[1], and why Bergeron states the consequence in one sentence: “simulation can show only the presence of bugs, never prove their absence” ^[4].

Worked example: the state space of one DMA channel

The block a schedule might reasonably call “simple” is a single DMA channel, an independent transfer engine of the reference SoC’s DMA with its own registers and its own state, sharing the AXI backend with the other channel. Its frontend exposes this per-channel programming interface (Table 3.1):

Table 3.1 — The per-channel programming interface of the reference SoC’s DMA engine: each register, its width in bits, and the role it plays in programming or reporting a transfer. The widths rather than the register count are what the worked example adds up — they sum to the 192 bits of architecturally visible state a single channel exposes.

Register	Width	Role
SRC_ADDR	32	source base address
DST_ADDR	32	destination base address
NUM_BYTES	32	bytes per row (1D length)
SRC_STRIDE	32	signed source row stride (2D)
DST_STRIDE	32	signed destination row stride (2D)
NUM_REPS	16	row count for 2D transfers
CFG	8	enable, 1D/2D, fixed-address flags, IRQ enables
STATUS	8	busy, done, error code from the AXI backend

The widths sum to 192 bits of architecturally visible state, or $2^{192} \approx 6.3 \times 10^{57}$ distinct register configurations for *one channel*, and no simulation farm reaches a space of that size at any rate. One million new configurations per second per machine is generous by an order of magnitude for RTL simulation of this block, and a thousand such machines together cover 10^9 configurations per second. Even at that rate, the cross-product of just the two address registers, $2^{64} \approx 1.8 \times 10^{19}$ pairs, would keep that farm busy for almost six centuries.

And the register file is the *shallow* part of the problem:

- **Internal state.** More state sits in the AXI backend and in the burst splitter, the DMA stage that splits each transfer into bursts that the AXI address rules allow. A modest 16-entry $\times$ 64-bit datapath FIFO alone contributes 1024 bits, or $2^{1024} \approx 10^{308}$ configurations, dwarfing the register space by 250 orders of magnitude; pointers, outstanding-transaction counters and the splitter’s working registers come on top.
- **Two channels.** The canonical DMA bug of this book, the 2D negative-stride mis-split from Chapter 1, manifests *only when DMA channel 1 is also active*, so bugs of this kind live in the product of the two channels’ states: $2^{384} \approx 3.9 \times 10^{115}$.
- **Sequence, not just state.** A state is only meaningful if a legal input history can reach it, and the input history space is larger still, because each cycle the environment chooses valid/ready handshakes, response codes, and back-pressure on several AXI channels: call it ten binary choices per cycle, so a thousand-cycle test is one path out of 2^{10000} . Which states are actually *reachable* is not even known a

priori, since computing the reachable set is the job formal tools take on, and it is where they hit their own wall.

As transistor budgets grow, with some fixed fraction of each new chip spent on state elements, the verification problem grows not linearly with design size but exponentially ^[1], and this is the engine behind the “verification gap” of Chapter 1: design capacity compounds, verification capacity does not, and the gap widens ^[1, 5]. A physical number sharpens this: simulation models run roughly a million times slower than the finished chip, so even months of simulation on many machines examine only a few *minutes* of the device’s operating life ^[5]. The factor of a million is an order-of-magnitude claim, because where a simulator lands against silicon depends on design size, on how much of the system is modelled and on how much instrumentation is compiled in. The band this book declares runs from tens of thousands at one end to a billion at the other, and the factor of a million sits in its middle. On any figure in that band, the campaign samples a vanishing fraction of the part’s operating life.

Abstraction shrinks the space, and it is still too big

The standard first response to these numbers is correct: *most of those states are equivalent*. For the burst splitter’s control logic, a source address of `0x1000_0004` and `0x2000_0004` behave identically, because what matters is the offset within the 4 KB region, the length relative to burst and region boundaries, and the sign of the stride; so an expert partitions each input dimension into equivalence classes and takes the cross-product of classes instead of bits. For one channel of the DMA the classes come out like this (Table 3.2):

Table 3.2 — One DMA channel after expert abstraction: each input dimension partitioned into the equivalence classes a skilled verifier would treat as behaving alike, with the number of classes in each. The counts multiply, so the figure to read off the table is their product — about nine million cases, no longer cosmological and still far beyond any hand-written suite.

Dimension	Expert equivalence classes	Count
source offset in 4 KB region	aligned / near-boundary / mid-region...	~16
destination offset in region	same	~16
length	sub-beat, one beat, sub-burst, exact burst, multi-burst, crosses 0/1/2+ 4 KB boundaries	~10
stride	positive/negative × smaller/equal/larger than row, zero	~7
repetitions	1, 2, many, max	~4
other channel	idle / active same bank / active other bank / erroring	~4
backend back-pressure	ready delay 0...7 cycles	~8
error response	none / read error / write error / both	~4

The product is about nine million cases, after an aggressive abstraction that already embodies judgment about what “matters” and is therefore itself a place where bugs

hide, since a class boundary drawn wrongly is a corner nothing will ever test. Nine million is far beyond directed testing, because a team writing and maintaining even a thousand hand-crafted tests is exceptional, and Chapter 1's escape data shows what the untested residue does. This double reduction, from 10^{57} raw states to 10^6 – 10^7 *meaningful* cases to some *sampled and measured* subset, is the intellectual structure of simulation-based verification: first abstraction, then stimulus automation (constrained random, Chapter 9), then measurement (coverage, Chapter 6). Constrained random is the automatic generation of legal but unlikely stimulus under constraints. Since exhaustive simulation is impossible, a sense of *relative* completeness is established through a coverage metric, and defining that metric is “a selective, and thus subjective, process” [2]. The metric therefore reports how much of what was chosen for measurement has been exercised, not how much of the design's behavior has been verified, and escapes live in the gap between what was measured and what the design actually does.

Formal verification changes the shape of the problem

Formal verification solves the problem partly, and the shape of that “partly” matters for the rest of the book. The technique at issue is formal property verification, the decision by mathematical reasoning of whether a design can violate a stated property. Formal property verification does not sample the state space but reasons about all of it, using symbolic representations and mathematical proof, so a proven property holds for *every* reachable state and input sequence, not the visited ones [3]. Model checking is an exhaustive state-space search made tractable by clever data structures, but its worst-case cost is exponential in the number of storage elements, so the *state explosion problem* reappears in formal form [2], and in practice this restricts full proofs to small or well-abstracted targets (arbitration, interface protocols, control kernels) while simulation remains the mainstay of verification for large designs [2]. A directory-based coherent hub shows what *abstracted* is doing there: the MESI state of every line in every cache is a product space no tool will enumerate, so the proof runs against a model cut down to two caches and one address, small enough to converge and still large enough to contain the livelock and starvation cases that make coherence worth proving. A bounded proof is a search carried only up to a fixed number of cycles, and it states that limit outright, since the property is guaranteed only within that proof bound [6]. One experienced practitioner considers it self-evident that formal verification cannot fully investigate all the functionality of a real circuit [5].

A rigorous discipline pushes further: Bormann’s *complete verification*, in which a completeness checker proves that a set of operation properties leaves no verification gaps. An operation property covers one whole operation of the design rather than one rule at a time, which is what lets a checker ask whether the set has left anything out. Industrial projects using complete verification reported that almost all such modules functioned correctly after fabrication, and that the few overlooked faults were traced to misapplication of the methodology [5]. The qualifier about misapplication is part of the result rather than a footnote to it. The provenance is specific: regular productive use at Siemens and at Infineon from 1999, on a project list whose only named part is the Infineon TriCore2 automotive processor [5].

Bormann’s result is the strongest counterexample to “completeness is impossible,” and Chapter 14 treats it with respect, but reaching that result took a different way of writing specifications, a dedicated methodology, and effort concentrated on selected modules, so completeness at that standard is a *purchasable* property for critical blocks rather than a property a chip project has by default. For the SoC as a whole (hardware, firmware, three clock domains, analog at the edges) the verification problem stands.

3.2 Controllability and observability

A bug is not found when it exists; it is found when three things happen in sequence. The stimulus must drive the design into the buggy condition, which *activates* the bug; the wrong value must travel from where it was born to somewhere the environment can see it, which is how it *propagates*; and something there must know the value is wrong, which is how it is *checked*. If any link breaks, the bug survives the test that technically “executed” it, which is why simulation logs full of passing tests routinely contain bugs that were activated and never propagated, and errors that propagated and were never checked. Figure 3.1 locates the escape at each link.

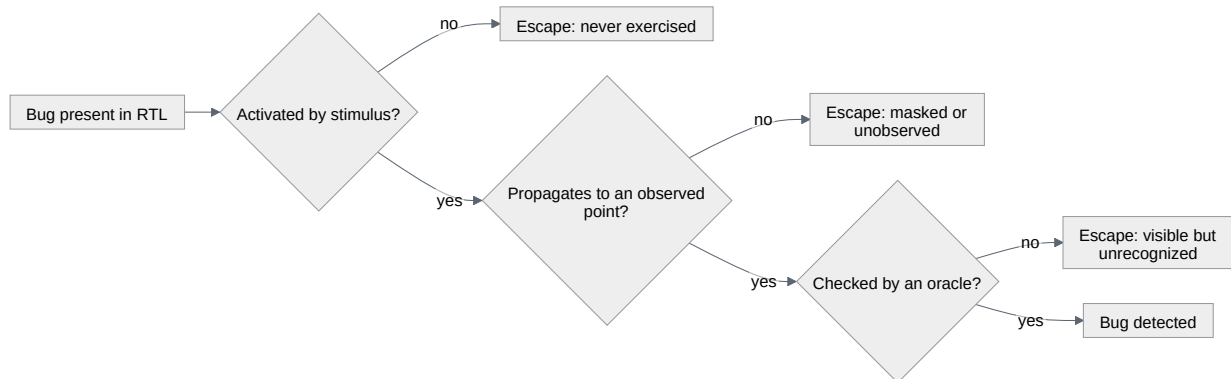


Figure 3.1 — The three-link chain a bug must survive to be found — activated by stimulus, propagated to an observed point, checked by an oracle — with the escape that follows from breaking each link. The three escapes are distinct failure modes rather than degrees of one; §3.3 takes up the third, where the wrong value is visible and nothing recognizes it as wrong.

Controllability is the ability to steer the design into a given internal condition using only drivable interfaces, and **observability** is the ability to see the consequences of

an internal event at the interfaces that can be watched. Both degrade with distance: pure black-box verification, which drives and observes a design only through its external interfaces with no knowledge of the implementation, “suffers from an obvious lack of visibility and controllability”, since it is often difficult to set up an interesting state combination or to isolate a piece of functionality, and equally difficult to observe the internal response ^[4]. That is its cost, not an argument against black-box testing, which checks the design against its specification rather than its own structure.

The trade-off that follows is that **smaller verification targets offer greater controllability and observability** ^[4]. At the block level the inputs of the DMA’s burst splitter are the testbench’s outputs, so the testbench can stall any handshake, inject any error response, and present on any cycle any transfer descriptor, the record that programs one DMA transfer. Wrapped inside the SoC, the same splitter is reachable only through the core, which executes driver code, and then through the crossbar under arbitration, so most of its input space is no longer directly reachable. Which features are verified standalone and which only integrated is a planning decision: Chapter 5 assigns every plan row to a level, Chapter 8 builds the environments that serve them, and the criterion is that a feature must be verified at a level where the testbench can both provoke it and see it fail ^[4].

Scenario. The reference SoC’s core has a custom SIMD extension in which a bug corrupts the saturation flag, but only when an interrupt is taken in the same cycle the SIMD instruction writes it (a canonical bug, the kind of defect this book follows from chapter to chapter). **Approach.** Verification runs at core level with co-simulation against an instruction-set simulator and *random* interrupt injection: the testbench fires IRQs at random cycles for thousands of runs and the checker compares architectural state instruction by instruction. **Why.** The buggy condition is a one-cycle coincidence between two independently-timed events, so it is a controllability problem: at SoC level, aligning a timer interrupt with one specific instruction inside a firmware loop is nearly impossible to arrange deliberately and worse to reproduce. At core level with direct IRQ lines, randomization can exercise the coincidence, and the co-sim oracle checks a corrupted flag in the same cycle it commits. (Compare Chapter 9 on why random stimulus finds coincidences humans do not imagine.)

Observability failures are quieter and therefore more dangerous. Two more of the book’s canonical bugs break the last two links. The UART’s RX overrun flag is never cleared if software reads the status register in the same cycle a stop-bit error arrives, and for months that condition was *activated* in regressions while no checker looked at the flag afterwards: visible state, never checked. The event unit’s retention register loses its configuration on power-domain re-entry when a wake event races isolation release, and plain RTL simulation did not model the power domains, so the corruption had *nothing to propagate through* and was caught only by power-aware gate-level simulation (Chapter 19).

A structurally weak link is answered by changing the design, and the grey-box prescriptions are standard practice: if observability is the problem, add sample points readable through memory-mapped registers; if controllability is the problem, add provisions to force error and exception conditions that are otherwise prohibitively slow to provoke ^[4]. Grey-box verification is black-box verification augmented with design hooks for controllability and observability. On the reference SoC, the DMA's per-channel error code in `STATUS` and the event unit's readback path exist because someone asked at design time how verification would see them. A NAND flash controller carries an error-injection register into its low-density parity-check (LDPC) ECC pipeline for the same reason: the multi-bit patterns that exercise the uncorrectable path cannot be produced from the flash interface at any rate a regression can wait for. Assertions serve the same end from inside: bound into the RTL, they monitor internal signals and catch violations locally, which is why assertion-based verification is credited with improving observability and debugging ^[2], flagging the error at its birthplace rather than a thousand cycles later.

The controllability and observability budget is the best a project will ever have and only shrinks: an RTL simulator makes virtually any internal signal observable at any time; FPGA prototyping falls to a few thousand pre-selected signals, chosen before the bitstream is built; and in silicon, on the order of a hundred signals reach trace buffers and pins, with controllability limited to the configuration hooks the architecture defined in advance ^[7]. The narrowing of both budgets from simulation to silicon is one of the key factors distinguishing everything pre-silicon from post-silicon validation, the work of checking a design on the fabricated part rather than on a model ^[7]. A bug not caught in simulation must be caught somewhere it is orders of magnitude harder to see (see Chapter 20). This asymmetry, as much as mask costs, is the economic argument for pre-silicon rigor.

3.3 The oracle problem

Where controllability and observability are solved, so that the stimulus reaches the corner and the response emerges at a monitored interface, the harder question is **how anyone knows the response is right**. A simulator produces what the design *did*, while correctness is a relation between that and what it *should have done*, and “should” lives in a specification written in prose, in the architect's head, or nowhere. The mechanism that pronounces pass or fail is the **oracle**, and building one is routinely harder than building the stimulus. The asymmetry is stark: deciding how to apply stimulus is relatively easy, since its timing and content are fully controlled, but verifying the response is difficult, because it requires planning how to determine the expected response and then how to check the design against it ^[4].

A testbench without a programmed oracle is a waveform generator that finds bugs only if a human stares at the output, which does not scale past the first afternoon. Everything in this book assumes **self-checking** environments, where the verdict is computed rather than eyeballed. The practical question is *which* oracle: every practical oracle is partial, checking a projection of correctness rather than correctness itself, and the craft is in combining them so their blind spots do not overlap.

Reference models are the most complete oracle: an independent executable of the specification runs on the same stimulus, and outputs are compared continuously [4]. The catch is that reference models rarely exist, except where the specification is an executable model, which is the typical situation for processors, where the model is golden by definition [4]. The reference SoC's core is verified this way: an instruction-set simulator (ISS), a fast software model of the architecture, runs in lockstep with the RTL (a step of the model for every step of the design), and every committed instruction's architectural state is compared. The ISS defines *architectural* correctness, so a bug that corrupts no architectural state (a performance bug, a speculative access that violates a protocol, a security leak through timing) is invisible to it by construction. A video codec is the other classic definitional oracle, since the standard ships a bit-exact reference decoder, and the projection is just as narrow: bit-exact output says nothing about rate control, about latency, or about what the encoder does when its line buffer back-pressures.

Scoreboards are the workhorse where no reference model exists: for data-moving designs the testbench applies the specified *transform* to each input transaction (for the DMA: split each transfer into legal bursts, then re-address), stores the expected output transactions in queues and compares actual outputs against them as they emerge, checking at end of test that nothing expected was lost [4]. The projection is narrower: a scoreboard proves data integrity and completeness but is typically blind to *when* things happened (latency, throughput, starvation, so an arbiter can be grossly unfair past a data-integrity scoreboard [4]) and to *how* (a protocol violation that still delivers correct data sails through).

Assertions are oracles for local rules: properties bound inside the design that check invariants and protocol obligations every cycle, on every test that runs [2, 4]. They are the sharpest partial oracle, detecting at the point of origin, over the narrowest scope of one rule each.

Offline and human comparison survives at the edges: some responses are easy for a human and awkward for a program (a human recognizes “a filled red circle” instantly), and some flows dump outputs for post-simulation comparison against reference files [4]. It should be used sparingly, because a verdict arriving after simulation ends arrives far from the state that caused it, and errors are best detected while the model is still near the failing state, when diagnosis is cheap [4].

Worked example: partial oracles on the DMA backend

One rule from the AXI protocol sits at the heart of this book's canonical DMA bug: no burst may cross a 4 KB address boundary (Arm IHI 0022H.c §A3.4.1 [8], whose words are “A burst must not cross a 4KB address boundary.”). As an assertion bound into

the DMA's AXI write channel, the same property Chapter 2 arrived at, qualifiers and all:

```
// No INCR write burst emitted by the DMA backend may cross a 4 KB page,
// regardless of how the midend legalized the transfer. (FIXED bursts hold
// their address; WRAP bursts stay inside their aligned container.)
// 16-bit arithmetic on purpose: an illegal awsize (>3 on this 64-bit bus)
// must overflow the comparison and fail – not wrap silently and pass.
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
  (awvalid && awready && (awburst == 2'b01)) |->
    (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

What this oracle does and does not establish (Table 3.3):

Table 3.3 — What the 4 KB-crossing assertion of the worked example establishes about the DMA backend and what it leaves untouched, one question per row answered for that single property. Only the first row is a yes, and the vacuity row is the one to dwell on: a backend that hangs forever satisfies the property without ever exercising it.

Question	Does <code>a_no_4kb_crossing</code> answer it?
Did any issued burst cross a 4 KB boundary?	Yes — every INCR burst, every cycle, every test; FIXED and WRAP bursts cannot cross by construction.
Did the split bursts carry the right data?	No — it never looks at data.
Do the split bursts add up to the programmed transfer?	No — it sees bursts one at a time, not their sum.
Does a legal transfer ever complete at all?	No — a backend that hangs forever satisfies it <i>vacuously</i> : the implication never triggers, so the property never fails.
Was the <i>read</i> channel also legal?	No — separate assertion needed.

The canonical bug illustrates the division of labor: at the boundary the burst splitter mis-splits a negative-stride 2D row, the contiguous run of bytes that makes up one line of a two-dimensional transfer. If the splitter emits a burst that *crosses* the boundary, the assertion fires on the spot. But in the actual bug the emitted bursts were individually legal and the *data* landed wrong, so only one oracle could catch it: the scoreboard's byte-accurate memory model, which compares final memory contents against the transform of the programmed descriptor. Each covered the other's blind spot, so the question is which projection of correctness each oracle owns, and whether any is unowned.

One failure mode defeats *all* oracles at once, the **common-mode error**: every oracle encodes someone's reading of the specification. If the scoreboard author misreads the same ambiguous sentence the designer misread (whether the stride applies before or after the first 2D row), model and design agree, the test passes, and the bug is structurally invisible to the environment. Specification issues (changing, incorrect, incomplete specs) are among the leading root causes of the functional flaws that force a respin, a new fabrication cycle with corrected masks [9]. The defenses are or-

ganizational rather than technical: independent interpretation (the verifier reads the spec rather than the RTL, Chapter 2), review of the oracle itself, and treating every model/design disagreement as a specification question whose resolution belongs in the spec. A passing regression establishes that the design and the testbench agree, not that either of them is right.

3.4 From exhaustive to risk-driven verification

The three results compound: not all of the state space can be visited, not all of the visited fragment can be reached or seen, and not all of what is seen can be checked. Verification is therefore never truly complete: it can show the presence of errors, not their absence, and given enough time an error *will* be found. The effort spent identifying an error must be weighed against its likely severity ^[4].

The governing analogy is statistical hypothesis testing ^[4]: the hypothesis “my design is correct” cannot be proved, only subjected to trials capable of falsifying it, with confidence calibrated to the power of those trials. The power of a trial, in the statistical sense, is the probability that it detects a fault when the design has one. The formal side says the same thing: the weakness of all observation-based quality assurance is that errors are found only with certain *probabilities* ^[5].

First: effort must go where risk is, not where habit is. Risk is the product of two estimates: how likely a bug is, and what its escape would cost. Likelihood concentrates in novelty and intersections: a bug is likelier in new logic than in silicon-proven reuse, and likelier in corner-dense arithmetic than in regular datapath. A bug is likelier at cross-domain seams (clock, power, block boundaries) than in interior logic, and likelier in volatile spec areas than in frozen ones. Cost concentrates where escapes are expensive to find and fix downstream: unworkaroudable functions, safety paths, anything whose post-silicon observability is near zero ^[7]. Verification effort is expensive: roughly 70 % of ASIC R&D cost by the classic “verification gap” complaint, as stated in 2009 ^[5], with verification headcount growing at a compound annual growth rate (CAGR) of 12.5 % against 3.7 % for design across 2007–2014 ^[10]. Spending *more* is not, by itself, working: in 2024, 14 % of projects reached first-silicon success, the state in which the first fabricated part is production-worthy ^[9]. The answer is to spend *aimed*: rank the features by risk and let the ranking drive the depth of each verification, the technique and the order of work. Made explicit and traceable, that ranking is the verification plan (Chapter 5).

Second: techniques must be matched to the shape of each risk. The techniques of Parts III–V are partial answers with different profiles along this chapter’s three axes, not competitors. Random simulation covers the easy-to-reach majority of scenarios quickly but leaves hard-to-activate corners, and those corners root-cause the industry’s bug escapes, the design flaws that survive verification into a later stage ^[11]. A directed test is a stimulus sequence whose content and order are written out by hand, so it does the same thing on every run. Directed tests reach a known corner precisely but cost engineer-days each, which is why their automated generation is a research field of its own ^[11]. Formal proof is exhaustive with respect to the properties written, on the blocks where it converges ^[2, 3].

Scenario. The reference SoC’s finite verification effort is allocated across the chip. **Approach.** The crossbar’s ordering rules go to formal property verification: they are control-dominated and protocol-defined, a write-interleaving bug would leave the chip unusable with no workaround, and per-property exhaustiveness is worth the setup cost on a block that size ^[2]. The DMA gets a constrained-random environment with a byte-accurate scoreboard and protocol assertions, because its risk lives in descriptor corner cases no one can enumerate by hand. The core gets ISS co-simulation with random interrupt injection, because its oracle problem (every instruction, every architectural register) is only tractable against an executable golden model, a model trusted because it is itself the specification ^[4]. The UART, reused and silicon-proven, gets a thin directed suite, including one test written from a firmware bug report about the overrun flag. The event unit’s retention behavior is deferred to power-aware gate-level simulation, and the deferral is *written in the plan* as an accepted RTL-simulation gap. **Why.** Every allocation follows risk: depth increases with novelty $\times$ escape-cost, silicon history justifies reuse credit, and the plan records each known blind spot with its owner and scheduled check.

The pivot redefines what “done” means, the subject of Part II. If completeness is impossible, *done* cannot mean “verified everything”; it means “reached the agreed evidence threshold on every planned item, and the risks we chose not to retire are enumerated and accepted by people entitled to accept them.” Coverage supplies the evidence, and since a coverage model is a selective, subjective definition of relative completeness ^[2], the model must be reviewed and not just filled. Sign-off (Chapter 7) is then a risk-acceptance decision, not a certificate of correctness. A tape-out deadline does not establish that residual risk has been reviewed. In 2024, 86 % of projects required a respin ^[9]; that rate alone does not establish the cause.

IN PRACTICE

Without explicit risk ranking, the work completed by tape-out determines verification scope, leaving no record of whether the unverified features were lower priority. The explicit alternative is a risk register kept next to the vplan and re-ranked when the spec changes, so that schedule pressure cuts *named items in review*, converting “we ran out of time” into “we accepted risk X, signed by Y.” Reviewing RTL while merging scoreboard code unreviewed leaves the oracle unchecked, so a checker defect

can let incorrect results pass throughout a regression. Crediting a scenario as covered when no oracle checks its result confuses activation with checking.

PITFALLS

1. **Promising completeness.** The plan says “fully verified” and management hears “no bugs possible.” Cure: report coverage of a *stated model* with its exclusions, and never write “complete” without its object [2].
2. **Counting passing tests as progress.** Ten thousand green tests with weak oracles measure agreement, not correctness. Cure: track coverage and bug-discovery curves, and audit what each test checks.
3. **Activation without a checker.** A scenario is marked covered because stimulus reached it, but no oracle owned it. Cure: every vplan item names its oracle, the check and not the stimulus (Chapter 5).
4. **Verifying at the wrong level.** A block corner is deferred to SoC tests where it is uncontrollable and unobservable. Cure: choose the level per feature from controllability/observability analysis, not from schedule convenience [4].
5. **Oracle derived from the design.** The model was written by watching RTL waveforms, so it reproduces the design’s misunderstandings. Cure: derive oracles from the specification independently, treating every mismatch as a spec question first [9].
6. **Uniform effort.** Every block gets the same depth regardless of novelty or escape cost, over-verifying reused IP while the new accelerator starves. Cure: explicit risk ranking drives the budget, and the ranking is reviewed like code.

SUMMARY

- The state space of even a trivial block is beyond enumeration: one DMA channel’s 192 bits of architecturally visible state give $\sim 10^{57}$ configurations [1, 2].
- Simulation can show the presence of bugs, never their absence; testing yields probabilistic assurance, not proof [3, 4].
- Finding a bug requires activation, propagation and checking; controllability and observability bound the first two and both collapse progressively from simulation to FPGA to silicon [4, 7].
- Every practical oracle (reference model, scoreboard, assertion) checks a projection of correctness, so they are combined until blind spots do not overlap, with a guard against common-mode spec misreadings [2, 4, 9].
- Formal verification is exhaustive per property, not per design: it relocates the wall (state explosion, bounded proofs, chosen properties) rather than removing it [2, 5, 6].
- Residual risk must be identified and bounded before it is accepted; verification effort follows likelihood $\times$ escape cost [4].

- “Done” is a risk-acceptance decision made in the open; coverage is its evidence and the plan is its contract (Chapters 5–7).

EXERCISES

Exercise 3.1 (calculation). Add the widths in the per-channel register table of §3.1 and confirm the figure the section reads off that table for one channel. Identify the register the chapter sets aside when its opening counts programmable bits, and compute the programmable bit count of one channel and of the two-channel pair. State which of the two counts the predicate “architecturally visible” attaches to, and why the two are not interchangeable in an expression of the form two to the Nth.

Exercise 3.2 (calculation). Multiply the class counts of the equivalence-class table of §3.1 and compare the product with the figure the section reads off that table. Take the size of the hand-written suite §3.1 calls exceptional, assume one case per test, and compute the fraction of the product such a suite covers. State what the remainder is, in the terms §3.1 uses for it.

Exercise 3.3 (judgement). A schedule proposal moves verification of the SIMD saturation-flag corruption of §3.2 out of the core-level environment and into SoC-level firmware tests. Decide which of the chapter’s two axes the proposal degrades, and give the mechanism of the degradation as §3.2 states it. Give the criterion §3.2 sets for assigning a feature to a level, and name the pitfall the proposal matches.

Exercise 3.4 (judgement). A block-level run of the DMA backend reports no failure of `a_no_4kb_crossing` and no failure of anything else. State the two readings that report admits, using the projection table of §3.3. Name the oracle §3.3 assigns to the question the assertion leaves unanswered, and give the check that oracle performs at end of test.

Exercise 3.5 (judgement). The DMA specification leaves it ambiguous whether the row stride applies before or after the first row, and the scoreboard’s author reads that sentence the way the designer read it. The regression passes on every seed. State what the passing regression establishes, name the failure mode §3.3 gives it, and give the defenses that section prescribes. Explain why adding a further oracle of the same kind is not among them.

Exercise 3.6 (artifact). Read the plan row below and state what its status line is entitled to claim. Name the field left empty, the pitfall of the chapter’s list the row commits, and the oracle §3.3 shows to be the only one that catches the bug named in the feature line. Give the reason §3.3 supplies for that oracle being the only one. The plan row’s coverage line names a cross cell, the combination of one value from each of two or more coverpoints in a coverage cross.

plan row	DMA-2D-A
feature	2D row straddling a 4 KB page, negative stride, other channel concurrently active
level	block
stimulus	constrained random descriptors, seeds swept nightly
oracle	

coverage status	cross cell hit covered
--------------------	---------------------------

FURTHER READING

From the corpus. Bertacco's opening chapters are the clearest short account of why scalability is *the* verification problem ^[1]. Kern & Greenstreet's survey maps formal verification's foundations and limits ^[3]. Yuan's introduction walks the whole technique landscape (simulation, coverage, model checking, symbolic methods) with unusual candor about each one's wall ^[2]. Bergeron's early chapters develop black/white/grey-box verification, levels of verification and self-checking strategy ^[4]. Bormann's dissertation is the serious counterpoint: how far "complete verification" can be pushed, and at what methodological price ^[5]; Schwarz shows the same formal machinery applied across the hardware/firmware boundary ^[6]. Jayasena & Mishra survey automated directed-test generation, the modern response to the corner-case residue ^[11], and Mishra's post-silicon tutorial quantifies how observability decays after tape-out ^[7].

Not in the corpus (cited on reputation, not verified for this book): Wile, Goss & Roesner, *Comprehensive Functional Verification* (Morgan Kaufmann, 2005), chapter 1, for an industrial framing of the verification problem and the "verification cycle"; Meyer, *Principles of Functional Verification* (Newnes, 2003), for self-checking and the limits of simulation.

REFERENCES

1. **B3** V. Bertacco, "Scalable Hardware Verification with Symbolic Simulation," Springer, 2006.
2. **B6** J. Yuan, C. Pixley, and A. Aziz, "Constraint-Based Verification," Springer, 2006.
3. **P20** C. Kern and M. R. Greenstreet, "Formal Verification in Hardware Design: A Survey," ACM Transactions on Design Automation of Electronic Systems, vol. 4, no. 2, pp. 123-193, 1999.
4. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
5. **T2** J. Bormann, "Complete Functional Verification," Ph.D. dissertation, University of Kaiserslautern, Germany, 2009 (English translation of the German dissertation 'Vollständige funktionale Verifikation', April 2009).
6. **T1** M. Schwarz, "Formal Hardware/Firmware Co-Verification of Optimized Embedded Systems," Ph.D. dissertation (Dr.-Ing.), Technische Universität Kaiserslautern, Germany, 2021.
7. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
8. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
9. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
10. **P17** H. D. Foster, "Trends in Functional Verification: A 2014 Industry Study," Proc. 52nd Design Automation Conference (DAC '15), San Francisco, CA, 2015.
11. **P10** A. Jayasena and P. Mishra, "Directed Test Generation for Hardware Validation: A Survey," ACM Computing Surveys, vol. 56, no. 5, Article 132, 2024.

4. The Verification Lifecycle

A verification program must state formally and repeatedly how much of its work is done and when the rest will be finished. On the reference SoC, three of the four teams answer that question in terms that cannot be checked: the core team reports being about 80% done, unchanged from the previous month, the DMA team a count of tests running, the I/O team a delivery date of *soon*. None names a quantity measured against a plan, so none is checkable and none constrains the schedule built around it. The crossbar's verification team answers in terms that can be checked: the crossbar is at stage 2 of 3, the environment runs all five managers against all four subordinates, nightly regression is at 340 seeds, functional coverage is at 71% against a reviewed plan, and the bug-open curve peaked two weeks ago. Stage 3 entry (coverage closure and the sign-off review) is gated on the two checklist lines still open: two ordering properties proven only up to a proof bound, and the starvation metric not yet wired into the dashboard. The proof bound is the number of cycles the proof covers: inside it no violation exists, beyond it nothing is claimed. At the current closure rate, the sign-off review is six weeks out.

The difference is not talent or tooling but that the crossbar team runs verification as a *lifecycle*: a sequence of phases with defined entry and exit criteria, measurable maturity stages, scheduled reviews with named owners, and a plan that connects the first feature list to the final sign-off signature. This chapter walks that cycle with the crossbar as the running example, from its feature list in week one to its sign-off review, and then sets it against the industry data on what schedules actually do.

CHAPTER MAP

§4.1 sets out the phases and milestones of the lifecycle and its feedback loops; §4.2 sets out planning deliverables and their reported benefits; §4.3 develops bring-up and the long middle, where regression and coverage become the instruments of management; §4.4 develops a three-stage maturity model (bring-up → mature → verified); §4.5 treats closure and the sign-off review; §4.6 gives the review calendar; and §4.7 treats schedule reality, where effort goes, why most projects run late, and how to plan against the crunch instead of into it.

4.1 The shape of the lifecycle

Verification is a process, not a pile of testbenches ^[1]. Like any process it has a shape: phases that follow one another, milestones that gate the transitions, and feedback loops that send work backward when reality disagrees with the plan. The shape is stable across companies and design domains even when the vocabulary differs, and [Figure 4.1](#) draws it in the terms this book uses.

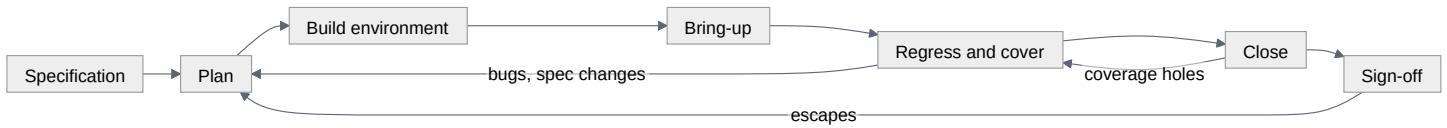


Figure 4.1 — The shape of a verification lifecycle: six phases from the specification to sign-off, and the three feedback loops that send work backwards — bugs and specification changes into planning, coverage holes from closure back into the regression phase, and escapes found after sign-off into the plan of the next project. The loops matter as much as the phases, and a cycle of this shape runs once per verification level rather than once per chip: the crossbar’s own cycle ends at the sign-off review of §4.5, and its result is what the SoC-level cycle consumes.

Six phases, three loops:

1. **Plan.** Extract what must be verified from the specification, decide how, estimate the effort, and get the result reviewed. The verification plan can start as soon as the architectural specification is complete, before RTL exists, and is augmented later with implementation-specific items once the micro-architecture exists [1]. Ideally the implementation side of the plan is finished before RTL coding begins and before any testbench code is written [2].
2. **Build.** Construct the verification environment: interfaces, drivers, monitors, scoreboard, coverage infrastructure. The build phase overlaps heavily with RTL development, because the environment should be ready to receive the first RTL drop rather than start when it arrives.
3. **Bring-up.** Make the environment and the design talk: first transaction in, first response checked, first smoke tests passing in a repeatable, self-checking way.
4. **Regress and cover.** The long middle: constrained-random stimulus ramps up, the coverage model fills in, regressions run at a steady cadence, and the bug curve rises, peaks and, with luck, falls. The bug curve is the count of open bugs plotted over time, and it is the phase’s main instrument of convergence.
5. **Close.** Convert “we ran a lot” into “we covered what we planned”: chase coverage holes, disposition every open bug, justify every exclusion.
6. **Sign-off.** A formal review in which the team presents the evidence against the plan and the organization accepts the residual risk. Sign-off is a decision rather than a discovery (see Chapter 3), because the design is never proven correct and the review only decides the evidence is sufficient.

The loops carry the lessons: a bug found in the middle phase may reveal a feature missing from the plan and sends the work back to planning, a specification change re-opens the feature list, and an escape found after sign-off, in the lab or in a customer system, must flow back into the plan of the next project. An escape is a bug that crosses a sign-off boundary undetected and turns up on the far side of the boundary that was meant to catch it.

Milestones, not percentages. Each phase transition should be gated by a milestone that is a *demonstration*: “Agent coding 80% done” is not checkable; “a single packet goes through the DUT and the scoreboard checks it” or “register reset-value test passes” is either true or false [3]. An agent is the reusable part of a testbench that drives and monitors exactly one interface, and how far its coding has pro-

gressed is a state rather than a demonstration. The vocabulary of the typical milestones varies while the pattern does not: **plan review** (the plan exists and stakeholders signed it), **first transaction** (environment and DUT communicate, self-checking), **feature complete** (all planned stimulus and checkers implemented), **RTL freeze** (the design stops moving except for bug fixes), **coverage closure** (every plan item covered or consciously excluded), **verification sign-off** (the review that gates tape-out).

One lifecycle per level. The cycle runs once per verification *level*, not once per chip. A design decomposes into physical and logical partitions, from blocks and cores through subsystems to the full ASIC and the board, and each level of granularity suits a different verification objective: smaller partitions offer far better controllability and observability, larger ones verify integration and system behavior [1]. Controllability is the ability to steer the design into a chosen internal condition from the interfaces a testbench can drive, and observability is the amount of internal state visible while debugging. Block-level plans and their outcomes roll up into the system-level plan, and system verification tracks progress against milestone views built on those per-IP results [4]. Some tests are deliberately planned to port across levels, block to system or simulation to lab, and that must be decided during planning, because portable tests are constrained to the features common to both environments [2].

Scenario. The crossbar, with 5 managers (core instruction, core data, two DMA ports, debug) $\times$ 4 subordinates (SRAM, boot ROM, APB bridge, neural accelerator), a 32-bit address, 64-bit data and ID width 4, gets its own block-level lifecycle: its own plan, environment, regression, and sign-off review. The SoC-level plan does *not* re-verify AXI protocol compliance on every port; it assumes the block-level sign-off and focuses on integration: address-map consistency, arbitration under real traffic from the actual core and DMA, and contention between the DMA and the neural accelerator on the SRAM banks. **Approach.** The SoC lifecycle gates its own bring-up milestone on the crossbar reaching stage 2 (defined in Section 4.4): no point wiring a full-chip environment around an interconnect that cannot yet route ID-tagged responses correctly. **Why.** Levels exist to put each class of bug where it is cheapest to find: a protocol bug found at block level costs hours, while the same bug found in full-chip simulation costs days, because controllability and observability have collapsed [1].

The verification lifecycle and the design lifecycle share one anchor, the specification, which is the golden reference: when the testbench and the RTL disagree the specification arbitrates, which is why the specification must exist before the implementation and must be written independently of that implementation [1]. When the specification itself turns out to be ambiguous, the lifecycle has surfaced a real project bug: specifications that change, or that are incorrect or incomplete, are consistently reported as a leading concern among verification engineers and project managers; those same specification defects are also one of the recurring root-cause categories for the functional flaws that escape to silicon [5].

4.2 Planning defines deliverables and closure criteria

Chapter 5 dissects the verification plan as an artifact: feature extraction, plan structure, traceability.

The verification effort for a modern design runs 100% to 200% of the RTL design effort [1], and no team would spend the design half of that budget without a specification document: the verification half has a specification document of its own, and that document is the verification plan [1]. The planning process creates the instrument that lets a verification manager estimate the effort and time remaining to the degree of certainty required [1].

Planning is staged like design itself: a design is specified as requirements, then architecture, then implementation, and the verification planning process should follow the same refinement, either as separate cross-referenced documents or as successive refinement of one document [2]. In lifecycle terms:

- **Verification requirements**, identified as early as possible, ideally while the architectural design is still being carried out, and reviewed as part of the project's technical assessment reviews [2]. Three questions structure them: what the design *does* (the intended function, traceable to the specification), what the design *must not do* (the errors the environment must be able to detect, since a checker only finds the failures it was told to look for), and what is explicitly *out of scope* (conditions the design is not expected to survive) [2]. The requirements review deliberately includes stakeholders from inside and outside the project team, because completeness is the property no single perspective can check [2].
- **Environment requirements** are the decisions that are cheap now and expensive later, and the plan settles five of them: the response-checking strategy (with random stimulus the expected response must be *computed*, not hard-coded [2]), the coverage sampling interfaces, the end-of-test conditions (an apparently simple question, "when is a test done?", that hides real bugs if answered lazily [2]), the reproducibility mechanics (any simulation must be trivially re-runnable with the same configuration and seed [2]), and the decision on which tests must port across environments.
- **Implementation plan**: who builds what, in what order, with what estimate.

On IBM's POWER8 processor, preparation for bring-up began at high-level design: the post-silicon tool team engaged early enough that the design team built the requirements into the chip (hardware irritators, a class of built-in logic whose only job is to perturb the design as it runs; non-functional running modes that stress a unit in ways no application ever would; and checkers embedded in the processor itself), with bare-metal exercisers as the primary test vehicle. IBM counted any bug first found by an operating-system test or by real software as an escape [6].

Two planning practices shape the schedule directly.

First: the reuse audit and the written assumption list. A reuse assessment identifies, for every piece of design and verification collateral, whether it exists and to what extent it is reused, modified, or new, because the recurring failure mode is as-

suming “reuse” means 100% reuse and ending up with an under-resourced, under-scoped project [7]. A dual-role USB controller is where this bites hardest: the compliance suite and the device-side stack arrive as genuine reuse, while the host-side link training and the U0-U3 power-state transitions are new work that the word “reuse” on a schedule slide hides. Alongside the assessment, every verification assumption (“feature X is verified at SoC level, not here”; “the standard protocol driver is adequate because we won’t implement the custom extension”) must be documented and reviewed face to face with the designers, usually over several review rounds, because unspoken assumptions are how design features fall through the cracks [7].

Second: deliverables with a definition of done. The work breaks into deliverables that group everything related to one feature (testbench infrastructure, stimulus, checks, coverage), each with an effort estimate and a *definition of done* that is an action rather than a state: “completely coded, committed, running and passing in regression”, “a single packet goes through the DUT”, never “80% coded” [3]. The resulting schedule is reviewed with stakeholders, on whether it meets the larger team’s needs and whether design can meet verification’s input dates, and it aims for linear progress: evenly sized deliverables, evenly spaced dates, later deliverables building on earlier ones [3]. Linear progress of that kind is what makes verification status legible from outside the team, since at any moment the lead can answer *what have we verified, what is left, are we on schedule, when will we finish* [3].

Modern practice pushes one step further: the plan should be *executable*, a machine-readable structure whose items link directly to the coverage points, checks, and test results that close them, so that progress reporting is automatic and the plan stays alive instead of rotting in a document repository [7]. An executable plan also makes reviews auditable, because review records attach to specific plan objects [7]. Chapter 5 returns to the mechanics; the lifecycle consequence is that every later phase (regression, closure, sign-off) reads its progress *from the plan*, and the plan reads its status from the regression database, automatically.

Planning has a price and a measured return: a meaningful verification planning process adds 10–20% to the verification front end and returns a 10–40% saving on the overall verification schedule, plus at least a 20% reduction in implementation and closure effort [7]. These savings describe a single organization’s reported experience; they do not establish a ranking of lifecycle phases by return on investment.

Worked example: the crossbar’s plan, week one. The verification lead reads the interconnect chapter of the SoC specification and the AXI4 protocol requirements (Arm IHI 0022H.c) [8], and produces a first feature list, an excerpt of which is already in vplan form (Table 4.1):

Table 4.1 — An excerpt from the crossbar’s feature list in week one, already in verification-plan form: each row states what must be shown for one sub-feature and, in the last column, the evidence that will be accepted as closing it. That

last column is the load-bearing one: naming it in week one is the closure planning that makes §4.5 possible. The final row carries what the team has decided *not* to verify, with the assumptions section reviewed with design as its evidence.

ID	Feature	Sub-feature	What must be shown	Closure evidence
XB-01	Address decode	Legal routing	Every manager reaches every subordinate it is allowed to reach, per the address map	Functional coverage: manager × subordinate cross
XB-02	Address decode	Decode error	Access to an unmapped address returns DECERR without hanging the port	Directed test + error-response coverage
XB-03	ID routing	Response routing	Read data returns to the issuing manager with the issuing ID, under arbitrary interleaving	Scoreboard check + ID coverage
XB-04	Ordering	Same-ID order, no write interleave	Transactions with the same ID complete in issue order; write data beats of one burst never interleave with another burst's on the same port (write-data interleaving is deprecated in AXI4 and later, and an interconnect that combines writes from different managers must forward the write data in address order — Arm IHI 0022H.c §A5.2.2 ^[8])	Protocol assertions (formal + sim)
XB-05	Arbitration	Fairness under load	No manager is starved when all five managers target one subordinate	Latency metric + starvation assertion
XB-06	Back-pressure	Stall robustness	Correct behavior with arbitrary ready de-assertion on every channel	Constrained-random with stall injection; assertion density on handshakes
—	<i>Out of scope</i>	—	Cache-coherent extensions (not implemented); APB-side peripheral behavior (verified in the peripheral subsystem plan)	Assumptions section, reviewed with design

Naming closure evidence in week one is what makes Section 4.5 possible: completion criteria and the coverage that correlates to each plan item are defined at planning time, not discovered at the end ^[7]. Alongside the feature list, the lead produces the deliverable schedule (environment skeleton: 2 weeks; single-path bring-up: 1 week; all-paths random with scoreboard: 3 weeks; protocol assertion bind-in: 1 week; coverage model: 2 weeks) and the assumptions list. The whole package goes to the plan review, and Section 4.6 says who sits in the room and what that room adds to this list.

4.3 From bring-up to regression maturity

Bring-up establishes checked transactions

Bring-up opens with the environment and the first RTL drop both in place and nothing working, and the discipline that shortens it is the deliverable thinking of plan-

ning, driving toward *demonstrations*. The first milestone is not “environment complete” but “one read transaction from the core-data manager to SRAM, driven, monitored, and checked by the scoreboard”, then one write, then one transaction per manager-subordinate pair. Each step is a definition-of-done that either passes in a committed, repeatable simulation or does not [3].

Two practices distinguish teams that exit bring-up quickly:

- **Self-checking from the first transaction.** The temptation is to eyeball waveforms for a few weeks and “add checking later”, but the response-checking strategy was designed during planning [2] and comes up *with* the stimulus, because an unchecked passing test is a false sense of progress and the checker itself needs debugging time. Most bugs found in bring-up are environment bugs, and finding them is the phase doing its job.
- **Regression from the first passing test.** As individual self-checking tests come alive, they go straight onto the master regression list, which runs at a regular interval, classically nightly, so that anything the design or environment breaks is caught within a day [1]. A regression suite exists to guarantee backward compatibility: the change that fixes today’s bug routinely breaks yesterday’s verified functionality, and only re-running everything catches it [1].

The classic bring-up failure mode is architectural big-bang: months of up-front testbench infrastructure work with no running demonstration, which both hides schedule slippage and makes progress invisible to outsiders [3]. The cure is the simplest environment that can pass the first deliverable, and that environment is refactored afterwards under the protection of the regression suite.

Scenario. The crossbar environment comes up in week 3 with one manager agent and one subordinate memory model. The first three failures all look like environment bugs: the driver violates a handshake it was supposed to test, the scoreboard mispredicts response ordering for the boot-ROM port, and the address map in the environment disagrees with the one in the RTL. Only the third of them is something else: that disagreement turns out to be a *real* discrepancy against the specification, caught because two teams implemented the same table independently.

Approach. All three go through the same bug-tracking flow, environment bugs included, since errors found at any stage, in the design itself or in the verification environment, are trackable issues [1]. **Why.** The environment also requires verification: an untracked environment bug can stop a testbench from checking its intended behavior. The address-map discrepancy is bring-up’s first return on investment: an integration bug found at the cheapest possible moment.

The long middle: regression and coverage maturity

After bring-up the lifecycle enters its longest phase, and its character changes from building things to *running and reading* them: constrained-random stimulus takes over the bulk of stimulus generation, the coverage model planned in week one gets implemented and starts reporting, and the regression becomes the project’s heart-beat.

Regression engineering gets its own chapter (see Chapter 12), but the lifecycle-level mechanics are these. The suite grows until it no longer fits in a night and then splits: a basic-functionality subset every night, the full suite over the weekend, random tests re-run with fresh seeds ranked by their incremental coverage contribution so the most valuable ones run first [1]. That threshold arrives at very different dates for different IPs, since a tile-based GPU’s shader-cluster regression stops fitting in a night long before a peripheral’s does, and its split falls by workload class: short kernels nightly, full frame renders at the weekend. Mature environments run several regressions per day, and a documented cadence is four runs a day, five days a week: each run produces a pass or fail result plus coverage that is snapshotted into a tracking database, after which the bulky raw results can be discarded [4].

What turns this machinery into *management* is metrics over time, chosen during planning: a successful metrics-driven process requires organizing their execution, classification, and reporting in the planning phase, because metrics are expensive to build and maintain and every measurement should have a stated purpose [9]. They pay off as trend lines: bug-open and bug-closure rates plotted over time, percentage of tests written, coverage progression, regression runtime per block, the charts that let a lead spot anomalies and answer “are we on schedule? are we converging?” at a glance [4, 9]. Even indirect signals earn their place, since how often code is checked in correlates with the stability and maturity of design and testbench [9]. For an interconnect like the crossbar the plan’s metric set includes bus-specific ones, functional coverage of operation mixes plus utilization, queuing, back-pressure, and latency [9], which is how feature XB-05 (starvation) gets measured rather than argued about.

The middle phase is dominated by debug: debugging takes more of a verification engineer’s time than any other single activity [5, 10], and debug effort varies enormously between projects, which is why schedules extrapolated from the previous project keep failing [5]. The lifecycle answer is not to schedule debug precisely but to keep the debug loop short: fast regressions, reproducible seeds, and good failure bucketing, the grouping of failures by signature so that one root cause is debugged once rather than many times.

The middle phase is also where the plan’s multi-engine choices show their return: the crossbar plan assigned feature XB-04, same-ID ordering and write-interleaving rules, to protocol assertions run in both simulation and formal, and it is formal that pays off:

```

Bug #217 – crossbar: write-data interleaving at a subordinate port
Found by:    formal protocol assertion (write-interleave rule), 11-cycle trace
Phase:       regression/coverage (week 14); RTL freeze candidate build
Symptom:     with WREADY back-pressure on the SRAM port and two managers
              holding writes in flight to it, the crossbar interleaves the
              write data beats of the two bursts – AXI4 obliges it to forward
              merged write data in address order, one burst at a time.
Root cause:  write-arbiter grant not held for full burst when the
              downstream stalls exactly on the first data beat.
Escape risk: silent data corruption; wrong data written, no error response.
Disposition: RTL fix + directed sim test derived from the trace + the
              assertion stays enabled in every future regression.

```

An 11-cycle counterexample for a bug that constrained-random had not hit in six weeks of nightly regression shows the sim/formal division of labor working as planned (see Chapters 14 and 16), and it only happened because the *plan* put the ordering rules in formal’s column back in week one. A counterexample is the input sequence a proof engine returns to exhibit a failure, and this one carries no more history than the failure needs. The last line of the report closes the loop from the lifecycle diagram: every bug’s fix enters the regression as a permanent test or assertion.

4.4 Maturity stages as a shared language

“How done are you?” is the question the lifecycle must answer continuously for people who do not live inside the testbench: program managers, the SoC integration team, other block teams that consume the results. Percentages fail at it: “80% done” is uncheckable, coverage percentages without a reviewed model are noise, and from outside a healthy coverage asymptote is indistinguishable from “we’ll suddenly get more productive next week” [3]. A coverage curve draws the same line on the chart whether it flattens because the plan is nearly covered or because the team has stopped making progress.

What works is a small number of named **maturity stages**, each with a written checklist of *demonstrable* exit criteria, a pattern that open-source silicon projects publish for every block (OpenTitan is the best-known example [11]). A block is *at* stage N while it works through that stage’s checklist and *exits* into stage N+1 when every line is ticked. Any three-stage variant looks essentially like this:

- **Stage 1: bring-up complete.** The verification plan exists and has passed review, the environment instantiates the current RTL and drives and checks real transactions automatically, and a smoke suite passes in regression, so the plan and bring-up phases have exited.
- **Stage 2: verification mature.** All planned stimulus and checkers are implemented; the functional coverage model is implemented and reporting; regression runs at its target cadence with a managed pass rate; most planned features have been exercised; the bug-open curve has peaked and closure is outpacing discovery. The block is then trustworthy enough for others to build on, and this is the gate the SoC lifecycle waits for.
- **Stage 3: verified, sign-off ready.** Coverage is closed against the plan, with every exclusion written down and reviewed; the full regression has been passing over a sustained window; every open bug has a disposition; the sign-off review has accepted the evidence.

Three properties make the model a *language* rather than another status report:

The criteria are demonstrations, one checkbox at a time. Like milestone definitions of done [3], each checklist line is either objectively true or objectively false: “coverage model implemented and reporting in nightly regression” rather than “coverage well underway”. A stage claim can therefore be audited in an hour by someone outside the team.

The stages compose upward. Each block reports one stage; the SoC dashboard is a table of blocks $\times$ stages; program-level milestone views aggregate per-IP plan results into a single picture [4]. To the question this chapter opens on, “stage 2, six weeks to sign-off” is a complete and comparable answer, because the same words mean the same thing for the crossbar, the DMA and the core.

The stages tie metrics to process maturity. Using metrics to make a process quantitatively managed is the core idea hardware verification inherited from the Capability Maturity Model tradition in software engineering [9]. The stage model is that idea made small: each stage is a bundle of metrics thresholds plus reviewed artifacts, and moving between stages is a measured event rather than a feeling.

Worked example: the crossbar’s stage-2 exit checklist (excerpt, week 16, the status this chapter opens on).

```
[x] Verification plan reviewed; all review actions closed
[x] All 5x4 manager/subordinate paths driven and checked in regression
[x] Protocol assertion set bound on all 9 ports; enabled in sim and formal
[x] Functional coverage model implemented for XB-01..XB-07; reporting nightly
[x] Nightly regression >= 300 seeds, pass rate >= 95%, no test disabled
[x] Bug-open curve past peak (trailing 3-week closure rate > discovery rate)
[ ] Formal ordering proofs (XB-04): 2 properties still bounded-only -> action
[ ] Starvation metric (XB-05) wired into regression dashboard -> action
```

Two open boxes mean the crossbar has *not yet exited* stage 2 and cannot claim stage 3 entry, and everyone can see exactly why and what remains.

Stages describe *verification maturity*, not RTL quality: a block can be stage 2 with fifty open bugs, because the claim is that the machinery to find and track them is mature, not that the design is clean. An HBM controller can report stage 2 while carrying a stack of open bugs against its refresh and thermal corner cases, since what stage 2 claims is the training-sequence stimulus, the coverage on those corners, and the nightly regression that keeps surfacing those bugs. Conflating maturity with cleanliness produces a dashboard of green process indicators over a red design.

4.5 Closure and sign-off

Closure redeems or exposes the lifecycle’s early investments: if the plan named its closure evidence per feature, as the crossbar plan did in Section 4.2, closure is a checklist-driven convergence. If it did not, closure becomes an argument about what “done” should have meant, at the worst possible time and under the worst possible pressure.

The closure phase converges several strands of evidence at once, each treated in depth in Chapters 6 and 7:

- **Functional coverage against the plan.** Every plan item reaches its target, or is *excluded with a written reason* that survives review. Declared exclusions are a feature of closure rather than a failure, since the plan itself put some conditions out of scope from day one [2]; what is forbidden is the silent exclusion.
- **Code coverage analyzed.** Not necessarily 100%, but analyzed, with unreached code either explained (unused configuration, defensive logic) or exposed as a stimulus hole.
- **Regression health over a sustained window.** All tests passing, repeatedly, across seeds, not a single lucky green run.
- **Bug metrics.** The open-bug curve at or near zero for the block, the discovery rate flat despite continued stimulus. The classical ship criterion bundles exactly these strands: the design can ship when the RTL passes all the planned tests *and* the coverage and bug-rate metrics satisfy the team, and not before [1].

Then comes the **sign-off review**, the lifecycle’s last gate and its most human one, whose function is not to generate new evidence but to *judge* the existing evidence against the plan, in front of the people who will live with the consequences. The review walks the plan, not the testbench, and it walks three things in turn: every feature row with its closure evidence, its exclusions and their reasons; the open-bug dispositions; and the assumptions list from planning, which is re-read line by line because an assumption that was reasonable in week one (“the custom protocol extension will not be implemented”) may have rotted since. An executable plan makes this audit mechanical rather than archaeological, because results and review records are already linked to plan objects [7].

Worked example: the crossbar sign-off summary (the one-page table the review starts from). Table 4.2 brings the evidence and remaining obligations into one review.

Table 4.2 — The one-page evidence summary the crossbar’s sign-off review starts from, one row per strand: plan features against the plan written in §4.2, functional and code coverage, formal results, regression history, bug dispositions, the assumptions re-read at sign-off, and escapes handed to later levels. What the table admits is the point: three formal properties proven only to a bound, fourteen coverage exclusions, one waived bug and one assumption escalated upward are the residual risk the organization is asked to accept. A summary showing none of it would describe an unexamined design rather than a clean one.

Evidence	Status	Notes
Plan features XB-01..XB-07	7/7 closed	XB-04 closed by formal proof (unbounded) + sim assertions; XB-07 added at plan review, closed by directed test + coverage bin
Functional coverage	100% of model	14 exclusions, all reviewed (list attached)
Code coverage	98.7% stmt / 96.2% branch	Residue: defensive decode default — reviewed

Evidence	Status	Notes
Formal	41 proven, 3 bounded $\geq$ 40 cycles	Bounded set reviewed and accepted as residual risk
Regression	21 consecutive clean nightlies, 5 clean weekend fulls	~9,400 seeds cumulative
Bugs	63 filed / 63 dispositioned	58 fixed+regressed, 4 not-a-bug (spec clarified), 1 waived (doc erratum)
Assumptions	9/9 re-reviewed at sign-off	1 escalated: APB bridge (Arm IHI 0024E ^[12]) timeout behavior — moved to SoC-level plan
Escapes to later levels	1	Address-map bug #003 also existed in SoC integration spec — fed back

The review’s output is a *decision*, the organization accepting the residual risk the table sets out, and a record of who accepted it.

The residue shows up occasionally however disciplined the closure is: in 2024 only 14% of IC/ASIC projects achieved first-silicon success, the lowest value in twenty years ^[5], and in the same year 87% of FPGA projects reported non-trivial escapes into production ^[13]. Sign-off is risk management’s final act rather than correctness’s, which is why the lifecycle’s last loop exists: every escape must flow back into the planning of the next block, the next level, the next project.

4.6 Roles and reviews assign responsibility across the lifecycle

Plans, stages and closure evidence are machinery. Reviews are where human judgment inspects it, and they are the part most often skipped under pressure, which is backwards: reviews are cheap, and they catch the class of error no tool can, the error of *intent*.

Two principles govern the cast. First, **the team owns the plan**: an RTL designer’s responsibility is not to write RTL but to produce a working design, so the entire design team contributes to and reviews the verification plan ^[1]. Second, **completeness needs outsiders**: the requirements review pulls in design, verification and software engineers from inside and outside the project team, because each perspective sees holes the others cannot ^[2]. Reviews then recur at prescribed stages, from planning through implementation and debug to closure, with representatives from the design, test, systems, and verification teams ^[7].

The review calendar, in lifecycle order (Table 4.3):

Table 4.3 — The review calendar in lifecycle order, from the requirements review held when the architectural specification is complete to the sign-off review that gates tape-out: each row gives when the review happens, who is in the room, and the question that review exists to answer. The third column is the one to read closely — the reviews that judge

completeness deliberately seat people from outside the verification team, and the sign-off row seats everyone in the rows above it plus management.

Review	When	Who	What is being judged
Requirements / plan review	Architectural spec complete; part of the project's technical assessment reviews ^[2]	Design, verification, software, system stakeholders	Is the feature list complete? Are the must-not-do's and out-of-scopes right?
Assumptions review	During planning; re-run whenever the spec moves	Verification team with the designers, face to face, usually several rounds ^[7]	Every unstated "someone else covers this" made explicit and agreed
Schedule / deliverables review	Plan approved, before build	Verification lead with consuming teams ^[3]	Do the deliverables and dates meet the larger team's needs? Can design meet verification's input dates?
Environment and code review	During build and continuously	Verification peers	Is the testbench itself trustworthy? (It is a design too.)
Coverage-model review	Before the long middle	Designers + verification	Does the model measure the <i>intent</i> , not the implementation? (see Chapter 6)
Progress / triage review	Weekly through the middle phase	Lead + team, escalation to program	Trend lines: bug curves, coverage, regression health — on schedule? converging? ^[4]
Coverage-closure review	Entering closure	Verification + design leads	Every hole chased or excluded with a written reason; no silent exclusions
Sign-off review	End of closure, gating tape-out (or block release)	All of the above plus management	The full evidence package against the plan; explicit acceptance of residual risk

The progress review is the oldest form of coverage. A testcase status table with owner and completion columns is itself a functional coverage model, one tracked through managerial procedure instead of tooling: engineers update it, and it is reviewed at regular meetings to decide whether the project is on schedule and where resources should move ^[2]. Modern dashboards automate the collection; the meeting, and the judgment exercised in it, remain.

Designers review the verification team's checkers and coverage model for *intent*, asking whether these checks encode what the design is supposed to do, but do not own them: the verifier's independence of judgment (see Chapter 2) is the value proposition, and the specification, not the designer, arbitrates when testbench and RTL disagree. Verification peers review each other's environment code, because a testbench bug that silently stops testing manufactures false confidence ^[1], worse than a design bug because nothing downstream is looking for it. Leads review trends rather than transactions, and the sign-off review is the one room where all three of these vantage points meet.

Scenario. At the crossbar’s plan review the firmware engineer, invited under protest (“it’s an interconnect, what would software know?”), asks how the debug manager port is verified while the core is halted. The feature list has core-I, core-D, and DMA traffic in every cross, but no scenario where the debug port is the *only* active manager issuing abstract memory accesses into a quiesced system. One row is added to the plan (XB-07), and it produces one coverage bin and one directed test, a test whose content and order are written out by hand. In week 16 that test catches an arbiter assumption that would have made post-silicon debug intermittently unreliable, and post-silicon debug is the one tool that matters most when everything else is broken. **Approach.** The outsiders stay in the review, especially the ones with no stake in the block’s internals. **Why.** Requirements reviews include stakeholders from outside the project team [2], whose independent perspective can expose omissions such as the missing debug-port scenario.

4.7 Schedule reality

Everything above describes the lifecycle as it should run; the industry data describes how it actually runs.

Verification dominates project effort. Figures like “70% of a project’s overall effort is spent in verification” have circulated for years [10]; the measured 2014 industry mean was 57% of total project time, with a rising share of projects spending more than 80% [10]. The spread is structural: projects at the low end lean on pre-verified IP reuse, projects at the high end carry high proportions of newly developed IP [5], which is why the crossbar (configured from a mature in-house pattern) and the neural accelerator (new) sit at opposite ends of the reference SoC’s effort table despite comparable gate counts.

Headcount tells the same story. The mean peak number of verification engineers on a project overtook design engineers years ago [10]. Between 2007 and 2014 demand for verification engineers grew at a 12.5% compound annual rate while demand for designers grew at 3.7% [10]; between 2016 and 2024 both rates cooled, to roughly 3% for verification engineers against 2.4% for designers, but in some segments, processors among them, a 5-to-1 ratio of verification to design engineers is not unusual [5]. The boundary itself is soft: design engineers spend about half their own time on verification, 47% in 2014 and 49% in 2024 [5, 10]. Verification is therefore not *a* task on the critical path; for most of the project it *is* the critical path.

Most projects run late anyway. Around two-thirds of IC/ASIC projects historically finished behind their original schedule (67% in 2007 and again in 2012, 61% in 2014) [10], and in 2024 the figure rose to 75% [5]. Meanwhile first-silicon success fell to 14%, the worst in twenty years, with logic and functional flaws still the leading cause of respins [5]. A respin is a new fabrication cycle with corrected masks, and a bug that reaches silicon can force one. The industry spends more than half its effort on verification, is late doing it and still ships bugs, and that combination is not an argument that verification fails but a measure of how hard the problem from Chapter 3 is in industrial practice.

Where the time actually goes. Within the verification team, debug is the single largest consumer of engineer time, larger than testbench development, test writing, or test planning [5, 10]. Worse for the planner, debug effort is unpredictable and varies significantly from project to project, which invalidates the most natural estimation method, extrapolating from the last project [5]. Effort is also back-loaded across the lifecycle: planning and environment build consume effort at a modest, steady rate; the long middle consumes compute time and debug time in bulk; and closure concentrates the highest-pressure work (the last coverage holes, the intermittent failures, the hardest bugs) into the weeks when the tape-out date is least movable, and that concentration is the **crunch**.

The crunch has a failure mode with a name, the guillotine: when the ship decision arrives on a fixed date and cuts off whatever verification was still in flight, what gets shipped is “whatever happened to work”, and the features that fall in the basket may be the ones the market needed [1]. The defense was built in Section 4.2: a plan that defines first-time success and *prioritizes* features makes the inevitable end-of-project cuts a conscious decision about named risks rather than an accident of which tests happened to be written first [1]. First-time success is the set of features the plan requires to work in first silicon.

The **verification gap** is the distance between the verification a project needs and the verification it gets. One practitioner taxonomy decomposes it into contributors (complexity, skills, schedule, quality, metrics, reuse) and, more usefully, into tractability classes: gaps that are *inherent* (complexity that must simply be absorbed), *transient* (skills that can be developed), and *self-induced* [14]. The self-induced ones are the lifecycle’s business: aggressive schedules and milestones create pressure that forces shortcuts, “pay me now or pay me later” [14]. Sloppy environment engineering shows up as slow edit-compile-run loops and weak regression management; the simple absence of best-known methods is estimated to cost around 20% of productivity on its own [14]. Individual skill spread reaches up to 10× in productivity across engineers [14], which makes team composition a first-order schedule variable that no amount of methodology hides.

Planning against that reality means scheduling in demonstrable deliverables with linear progress, so that slippage is visible in weeks rather than quarters [3]; buying schedule with planning up front, where the leverage is (10-20% front-end cost against a 10-40% return on the verification schedule in one reported experience) [7]; keeping the maturity stages truthful, so that “80% done” cannot conceal stage 1 un-

til RTL freeze; and protecting the regression cadence and the debug loop, because that is where the majority of all verification hours will be spent whether or not the plan allowed for them [5, 10].

Scenario. Week 18 of the reference SoC: RTL freeze slips three weeks because the neural accelerator’s datapath is being reworked, and the program manager proposes to “absorb” the slip by compressing verification closure, while the tape-out date does not move. **Approach.** The crossbar lead brings the plan. The first-time-success list is re-reviewed with the stakeholders: XB-01 through XB-07 stay non-negotiable; the stretch goals (full QoS characterization, exhaustive sparse-ID stress) are explicitly moved out of the sign-off gate and into an erratum-risk register that management signs. The stage checklists do not change. **Why.** If schedule pressure reduces verification scope, a prioritized plan makes the cuts explicit, and a signed risk register records who accepts the resulting risk [1, 14].

IN PRACTICE

- **The plan must follow specification changes.** A reviewed plan becomes stale if it remains a snapshot from month one while the design changes. The plan should either be executable and linked to the regression database [7] or follow this update rule: every specification change and every not-a-bug dispute must land as a plan diff, reviewed in the weekly meeting.
- **Stage claims require evidence.** The symptoms are checklists claimed without evidence, “temporary” test disablement to keep the pass rate green, and coverage closed by deleting bins, the counters that record which values the stimulus has reached. The countermeasure is social, not technical: stage transitions are *reviewed* events with an outsider in the room, and a disabled test is treated as an open bug against the environment.
- **Slower regressions delay debug feedback.** This increases the cost of the debug work that occupies the largest share of verification engineers’ time [5, 10]. Regression turnaround should be tracked as a metric, with a runtime spike investigated like a failing test [9].
- **Tape-out constraints require explicit risk review.** If sign-off dates are negotiated backward from tape-out, the available evidence must still be reviewed. The realistic goal is not to prevent the negotiation but to enter it with a prioritized plan and a residual-risk list, so the compression is explicit and signed [1].
- **The review calendar must be maintained under load.** Skipping the coverage-model review and leaving assumptions unread removes checks on verification scope. An unreviewed assumption (“the other team covers this”) can leave a design feature unverified [7].

PITFALLS

1. **Reporting percent-done.** Symptom: “80% done” three months in a row. Cure: maturity stages with demonstrable checklists; report the stage and the open checklist lines [3].
2. **Big-bang environment development.** Symptom: months of infrastructure work, nothing running end to end. Cure: deliverable-driven bring-up; first checked transaction is milestone one [3].
3. **Deferring checking.** Symptom: waveform eyeballing “until the env stabilizes”. Cure: self-checking from the first transaction; the response strategy was planned before the first test [2].
4. **Assuming reuse means 100%.** Symptom: under-scoped project discovered at bring-up. Cure: a written reuse assessment (exists / reused / modified / new) for every piece of collateral [7].
5. **Inventing closure criteria at closure.** Symptom: month-long arguments about what “done” means, at the worst time. Cure: closure evidence named per feature in the plan, in week one [7].
6. **Treating sign-off as proof.** Symptom: shock when silicon comes back with a bug despite “100% coverage”. Cure: sign-off is documented risk acceptance, with the residual-risk list kept explicit and every escape fed back into the next plan [5, 13].

SUMMARY

- The lifecycle is six phases (plan, build, bring-up, regress-and-cover, close, sign-off) with feedback loops, run once per verification level, block lifecycles rolling up into the SoC’s.
- Milestones and stage transitions must be demonstrations (“this test passes in regression”), never state estimates (“80% coded”) [3].
- The plan is the specification of an effort that costs 100–200% of RTL design [1], and disciplined planning measurably buys back schedule [7].
- A three-stage maturity model (bring-up complete, verification mature, verified) with written exit checklists gives every block the same auditable answer to “how done are you?”; open-source silicon projects publish exactly this pattern.
- Reviews are the layer of human redundancy, the independent second reading of the specification by someone who did not write it: requirements with outsiders, assumptions face-to-face with designers, and closure and sign-off against the plan, at prescribed stages from planning to closure [2, 7].
- Verification consumes over half of project time (57% measured in 2014) [10], most projects run late (75% in 2024) [5], and debug dominates engineer time and resists estimation [5, 10].
- Sign-off is risk acceptance, not proof: 14% first-silicon success and 87% FPGA escape rates say the residue is real [5, 13], so it is made explicit and signed, and fed back into the next cycle.

EXERCISES

Exercise 4.1 (calculation). Take the bug row of the crossbar sign-off summary in §4.5 and verify that its dispositions partition the set of filed bugs, showing the arithmetic. Then account for the plan-features row of the same table, which closes more features than the plan excerpted in §4.2 lists, and name the review that added the difference. Name the class of hole that §4.6 credits that review with catching.

Exercise 4.2 (calculation). Sum the crossbar’s deliverable schedule in §4.2, then take the band that §4.2 reports for the saving that a planning process returns on the overall verification schedule, apply that band to the sum, and give the result in weeks. Give the reason §4.1’s phase list supplies for treating that sum as smaller than the schedule the saving applies to. State the qualification §4.2 attaches to the reported percentages.

Exercise 4.3 (judgement). A program manager reads a dashboard placing the crossbar at stage 2 and the DMA engine at stage 1, and schedules SoC integration bring-up on the inference that the crossbar’s RTL carries fewer open bugs. Decide what the two stage claims entitle a reader to conclude and what they do not, and name the property of the stage model in §4.4 that draws that line. State which of the two claims the SoC lifecycle is entitled to act on and for what, naming the section that grants it.

Exercise 4.4 (judgement). A lead sizes the next project’s closure window by scaling this project’s closure window, on the ground that the team, the flow and the tools are unchanged. Decide what §4.7 allows to carry across a project boundary and what it forbids, and name the activity that consumes the largest share of engineer time together with the property of that activity which defeats the extrapolation. Give the two structural reasons the same section offers for the spread in verification’s share of project effort, and name the third variable it locates inside the team itself.

Exercise 4.5 (artifact). Read the stage claim below against the stage-3 exit criteria of §4.4 and the closure strands of §4.5. Name the ticked line that the blank field contradicts, the ticked line that the disabled-test row contradicts, and the field that §4.5 forbids leaving blank. State what the chapter’s In practice list requires the reviewer to do with the disabled tests, and what that requirement does to the last row of the claim. State what §4.4 says about a checklist line still open at the stage a block claims, and apply it to the unticked line.

```
Block: DMA engine
Stage claimed: 3 (verified, sign-off ready)
[x] Coverage closed against the plan
[x] Full regression passing over a sustained window
[x] Every open bug has a disposition
[ ] Sign-off review has accepted the evidence
Nightly regression pass rate: 95%
Tests disabled to hold that pass rate: 2 (intermittent; revisit after RTL freeze)
Coverage exclusions, with reasons:
Open bugs: 7 filed, 7 dispositioned
```

FURTHER READING

Corpus sources.

- [2] *Verification Methodology Manual for SystemVerilog*, ch. 2: the planning process as staged refinement (requirements, environment, implementation), and the fullest treatment of what a plan must contain.
- [1] Bergeron, *Writing Testbenches Using SystemVerilog*, ch. 2, ch. 3 and ch. 7: issue tracking (what counts as a trackable issue); the plan's role, first-time success, levels of verification; simulation and regression management.
- [7] "Best Practices in Verification Planning": executable plans, reuse assessment, assumption reviews, and the cost/benefit data quoted in this chapter.
- [3] "Proven Strategies for Better Verification Planning": deliverables, definitions of done, linear progress; the antidote to percent-done scheduling.
- [4] "Smarter Verification Management": regression data tracking, trend charts, and multi-engine and milestone rollups at SoC scale.
- [9] "Metrics in SoC Verification": the metrics landscape, its CMM lineage, and planning-phase metric design.
- [10] Foster, "Trends in Functional Verification: A 2014 Industry Study" (DAC 2015): the effort, headcount, schedule, and respin baseline.
- [5, 13] Wilson Research Group 2024 IC/ASIC and FPGA trend reports, the source of the current numbers: 14% first-silicon success, 75% behind schedule, 87% FPGA escape rate.
- [14] "I Created the Verification Gap": a practitioner's self-critical taxonomy of why the gap between needed and delivered verification exists.
- [11] OpenTitan Signoff Checklist: the stage item lists behind Section 4.4's pattern, as lowRISC publishes them, twenty-two items to reach V1, twenty-two more for V2, thirteen for V3. [15] fixes what those stage names mean, and [16] is the same checklist filled in for one block.

Not in the corpus (cited here as pointers only):

- Wile, Goss, Roesner, *Comprehensive Functional Verification*: the classic end-to-end treatment of the verification cycle at industrial (IBM) scale.
- Carter & Hemmady, *Metric-Driven Design Verification*: a book-length treatment of the metrics-driven process sketched in Sections 4.3 and 4.5.

REFERENCES

1. B2 J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. B1 J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
3. D7 P. Marriott, J. Vance, and J. McNeal, "Proven Strategies for Better Verification Planning," DVCon 2022 Workshop, 2022.
4. D26 D. Zhang, "Smarter Verification Management with vManager Platform," Proc. DVCon China, 2021.

5. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
6. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
7. **D9** B. Ehlers, C. Vargas, and P. Carzola, "Best Practices in Verification Planning," Proc. DVCon U.S., 2013.
8. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
9. **D20** A. Meyer and H. Foster, "Metrics in SoC Verification: Not just for coverage anymore," Proc. DVCon U.S., 2012.
10. **P17** H. D. Foster, "Trends in Functional Verification: A 2014 Industry Study," Proc. 52nd Design Automation Conference (DAC '15), San Francisco, CA, 2015.
11. **O10** lowRISC, "Signoff Checklist," lowRISC/opentitan, doc/project_governance/checklist/README.md, commit 4e5c99db524e99425cb78ca6f2714e5287fa800c, accessed 2026-09-02.
12. **S19** Arm Limited, "AMBA APB Protocol Specification, ARM IHI 0024, Issue E (ID022823)," Arm Limited, February 2023.
13. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
14. **D16** R. Narayan and T. Symons, "I created the Verification Gap," Proc. DVCon U.S., 2015.
15. **O11** lowRISC, "OpenTitan Hardware Development Stages," lowRISC/opentitan, doc/project_governance/development_stages.md, commit 4e5c99db524e99425cb78ca6f2714e5287fa800c, accessed 2026-09-02.
16. **O12** lowRISC, "AES Checklist," lowRISC/opentitan, hw/ip/aes/doc/checklist.md, commit 4e5c99db524e99425cb78ca6f2714e5287fa800c, accessed 2026-09-02.

PART II

Planning and Measurement

The plan as a contract and measurement as ground truth: without these two, verification is opinion.

5. Verification Planning

Verification planning begins before any testbench exists, when the specification is the only artifact available. The specification of the reference SoC's burst splitter says: *"The burst splitter brings each transfer into line with the AXI address rules, splitting any burst that would cross a 4 KB boundary."* The burst splitter is the DMA stage that sits between the register frontend and the AXI backend, and the 4 KB boundary is the address line that AXI forbids a burst to cross: every burst stays inside one 4 KB address region, so it never spans two subordinates. A 2D transfer moves several lines of bytes, and a 2D row is a single one of those lines. Consecutive rows start one stride apart, and the stride is the signed address step from the start of one row to the start of the next. Three questions follow. The first is whether the split holds for 1D transfers only or also for 2D. The second concerns 2D transfers, in which the check applies per row: whether the split still holds when that stride is negative and consecutive rows walk *down* through memory toward the boundary instead of up. The third is whether the answer changes when the other channel is active and the two channels share the splitter's state. The specification is silent on all three. The method this chapter sets out records them as a line of the verification plan, a plan row. That row later traps the worst bug in the block: a 2D negative-stride transfer that the burst splitter splits wrongly at the 4 KB boundary, corrupting data only while channel 1 runs. The bug is found by the constrained-random environment, which generates legal but unlikely stimulus under constraints, and it is caught by the scoreboard, the testbench component that compares what the design did against what it should have done. The plan, though, pointed both the environment and the scoreboard at that corner of the input space: no directed test would have been written for a question that did not exist until the plan asked it.

Chapter 4 placed planning in the lifecycle: when it happens, what it must produce, what it costs. This chapter sets out how it is done: the method by which a specification becomes a feature list, a feature a measurable row, and a document an instrument.

CHAPTER MAP

§5.1 extracts features from a specification adversarially, by interface, by function and by architecture, with the designer interview and the *must-not-happen* list as parts of extraction. §5.2 gives the anatomy of a vplan row, feature → attributes → measurable pass/fail criterion → method assignment, with the one-row-one-metric discipline that keeps closure measurable. §5.3 states what makes a plan *executable* and *traceable*, and §5.4 sets out the plan review and who must attend. §5.5 shows how risk sets each feature's verification depth, and §5.6 treats planning against an incomplete or moving specification.

5.1 From specification to feature list

The specification is the golden reference: when the testbench and the RTL disagree, the specification arbitrates, so it must exist before the implementation; and a “specification” that merely describes what the RTL does is unverifiable, because it changes every time the implementation changes ^[1]. Two documents usually feed the plan: the architectural (functional) specification, from which planning can begin, and the implementation specification, which later contributes implementation-specific items ^[1]. For the DMA engine, the first defines transfers, channels and error semantics; the second reveals that a burst splitter exists at all.

Feature extraction enumerates, from these documents, everything that must be shown to work. It is the step with the highest cost of omission in verification, because of a brutal asymmetry: a behavior is verified if it is in the plan, and only if it is in the plan ^[1]. Coverage does not save an omission, since it measures progress against the model this list defines (see Chapter 6), and a feature that extraction misses is invisible to every metric downstream.

Reading the spec adversarially

A designer reading a specification asks “how do I build this?”, while a verifier asks “how does this break?” (see Chapter 2). Without that shift, planning reads the specification with a design mindset: a scenario per specified feature, error injection only for the errors the design detects, so it finds the obvious bugs and leaves the obscure ones, every bit as critical, unhunted ^[2]. Systematic interrogation replaces it: three passes over the design, each with its own question set ^[1]:

Interfaces first. For every interface, enumerate what it implies: which transactions, which ranges of values, which sequences and densities, which protocol violations the design must sustain, and which interactions exist with the other interfaces ^[1]. On the DMA frontend this yields: every register readable and writable per its access policy (read-only, read-write, write-one-to-clear); a configuration written while the channel runs is rejected or deferred, never half-applied; back-to-back writes to the trigger register start exactly one transfer. On the backend: legal AXI on every DMA channel, correct accumulation of the two AXI error responses (SLVERR, a subordinate signalling failure, and DECERR, no subordinate at that address), and behavior under arbitrary `ready` back-pressure. A DMA channel is a transfer engine with its own descriptor and state, and the DMA has two of them, sharing the one AXI backend.

Functions second. Follow the data through the specified transformations: the relevant configurations, the transformations and their sequences, the *sensitive values* that trigger each, where transformed data must end up, how ordering is affected, what error-detection mechanisms exist and how they report, and what happens to erroneous data ^[1]. For the burst splitter, “sensitive values” is the phrase that does the work: transfer lengths of 0, 1, one beat less than max burst (256 beats), exactly max burst, one more; source and destination alignments that do and do not straddle the 4 KB boundary; 2D strides positive, zero, and negative. A beat is a single data transfer within a burst.

Architecture third. From knowledge of the chosen implementation, list the conditions that push the design toward its limits: buffer overflow and underflow, resource bottlenecks, multiple requests colliding on them, one transformation path blocking another [1]. This pass turns the two-channel structure of the DMA into a feature generator: two channels contending for one backend port, one channel's error not perturbing the other's stream, splitter state shared or not shared. The third question of the opening, negative stride near the boundary *while channel 1 is active*, belongs to this pass, and could not come from the functional spec alone, because the sharing of splitter state between channels is an implementation fact.

Alongside the three passes, a practical taxonomy from one published planning workshop sorts scenarios into four bins: *isolated features* are one behavior at a time, the backbone of bring-up and debug; *mixed features* are the key combinations, where interconnect and multi-channel bugs live; *legal exceptions* are abnormal but supported cases such as soft reset mid-transfer, error responses and FIFO-full, and they must appear in the design spec; and *illegal scenarios* are cases the design does not support, where the spec must say whether “unsupported” means ignored or recovered by reset [3]. The last two bins are chronically under-populated in first-draft plans, and they are where the planning rule applies with full force: a verification effort that does not inject errors is incomplete [2].

Interviewing the designer

The specification, however good, does not contain the whole feature list. Team members beyond the plan's author, the system architects and the RTL designers, contribute features invisible in the document: some are not obvious to anyone unfamiliar with the design's purpose, others exist only once a particular implementation is chosen [1]. One methodology makes the point structurally: implementation-specific requirements create corner cases *that only the designer is aware of*, since the specification never mentions the FIFO that the designer inserted; the FIFO's full and empty conditions must be verified all the same, and the methodology recommends that the designer capture them as coverage properties in the RTL [4]. The same advice governs coverage attributes, whose most effective source is a brainstorming session among several verification engineers and designers familiar with the specifications [5].

The interview is extraction, not review. Useful questions, on the reference SoC's DMA: *What was the hardest part to get right?* (the splitter's resumption address arithmetic, an instant plan priority). *What state is shared between the channels?* (the answer decides whether “channel isolation” is one feature or ten). *What did you assume the software will never do?* (each answer is either an out-of-scope entry, reviewed and signed, or a new feature, never an unexamined hope). The interview also runs in reverse, since planning early enough leaves room to request design changes that make hard features verifiable: pre-loadable counters, sample points, error-injection hooks (see Chapter 3's observability discussion) [1].

What must not happen

A specification describes intended behavior, while verification detects errors, and it can only detect the errors it is looking for [4]. A feature list of only affirmative sentences is therefore half a list. One methodology elevates the negative half to a rule: define what the design *must not do*, as requirements enumerating the relevant and probable errors, since there are infinitely many ways to fail and judgment selects which the environment must see [4]. The checker architecture of Chapter 8 is downstream of this list: a scoreboard detects exactly the wrong behaviors that someone decided to look for, and correctness at the end of the project is inferred from nothing stronger than the *failure to find inconsistencies* [4].

Must-not-happen features are also where the DMA's canonical bug was pre-caught. That canonical bug is the defect this book keeps returning to: the burst splitter mis-splits a negative-stride 2D row that crosses the 4 KB boundary, and only while the other DMA channel is active. “Data is delivered uncorrupted, in order, exactly once” is the affirmative feature. Its negative twin is the sentence *no transfer shall ever write a byte to an address outside the destination window that the splitter computed for it, under any interleaving of the DMA channels*, and the scoreboard's address check implements exactly that. That check, not any directed expectation, fired on the negative-stride bug. The move fits any block whose correctness is numerical rather than structural: on the radar front-end, “the FFT pipeline computes the transform” says nothing about the fixed-point contract, so the negative list carries it as *no FFT stage shall saturate without raising its saturation flag*, because an unreported saturation turns a target into noise and leaves no trace. Symmetrically, the third list, what is explicitly *out of scope*, bounds the effort: conditions the design need not survive are written down, so “we do not verify descriptor chains longer than memory” is a reviewed decision, not an accident [4]. A transfer descriptor is the record software writes to program one DMA transfer: the source and destination addresses, the lengths and strides, and the channel that will run it. One caution from the same source: “if it is not specified, do not verify it” applies to genuine don't-cares, while an *incomplete* specification is not a don't-care but a defect to be fixed [4]; Section 5.6 returns to that boundary.

Worked example: the DMA feature list, first cut

After the three passes, the interview and the negative list, the DMA plan's feature list opens like this (excerpt; the full list runs to roughly sixty entries):

```
DMA-F01 Frontend: register access per access policy (R0/RW/W1C), reset values
DMA-F02 Frontend: config write during active transfer rejected, error flagged
DMA-F03 Burst splitter: 1D transfer split into legal AXI bursts, max 256 beats
DMA-F04 Burst splitter: burst split at 4 KB boundary, all alignments, len 0/1/max±1
DMA-F05 Burst splitter: 2D transfer, per-row split into legal bursts, stride > 0, =
0, < 0
DMA-F06 Burst splitter: 4 KB split correct under 2D negative stride [arch pass +
interview]
DMA-F07 Burst splitter: channel 0/1 splitter state fully independent [interview]
DMA-F08 Backend: AXI protocol compliance under arbitrary back-pressure
DMA-F09 Backend: SLVERR/DECERR reported in the issuing channel's status only
DMA-N01 MUST NOT: write outside the legal destination window (any interleaving)
```

```

DMA-N02  MUST NOT: raise done interrupt before last write response accepted
DMA-N03  MUST NOT: propagate channel A error into channel B stream
DMA-X01  OUT OF SCOPE: descriptor fetch from uncached device memory (not supported;
         software constraint documented, reviewed with design in week 3)

```

The shape matters: affirmative features (F), must-not-happen features (N), out-of-scope entries (X) with a review date, and provenance tags where a feature came from the architecture pass or the interview rather than the spec text. DMA-F06 exists because someone asked a question the specification did not answer.

5.2 The anatomy of a vplan row

The verification plan defines *what* must be verified and *how*, and it is the specification document of the verification environment [1, 5]. The partition is explicit: what must be verified lives in the coverage requirements, and how it is verified lives in the design of the three aspects that every dynamic environment is built from (stimulus generation, response checking, coverage measurement) [5]. A well-formed vplan row is that partition in miniature, five fields each answering one question (Figure 5.1):

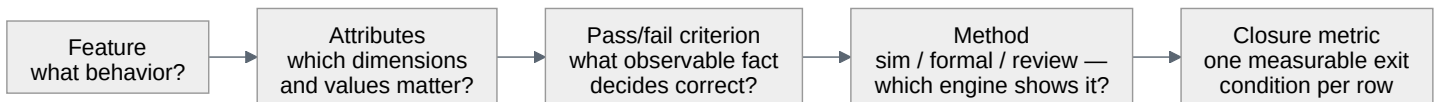


Figure 5.1 — The five fields of a well-formed vplan row, in the order they are written and with the question each one answers: the behavior, the dimensions and values that matter, the observable fact that decides correct, the engine that will show it, and the single measurable condition that closes the row. The arrows are ordered rather than decorative: the attributes belong to a stated feature, and the closure metric is chosen for a stated criterion and a chosen method. The last field is where the one-row-one-metric rule bites: a row that needs two kinds of evidence is two rows.

Feature. The field names one behavior, stated as *conditions plus expected result* rather than as an implementation description, labeled with a unique identifier and cross-referenced to the specification section that defines it [1]. Identifiers are permanent: a deleted requirement retires its identifier, never reused, so an old regression report can never silently point at a new requirement [4]. The label earns its keep at debug time: the checker’s error message carries it, so a failure names the requirement it violated [1].

Attributes. An attribute, in Piziali’s vocabulary, is a parameter or dimension of *the coverage model*: a configuration mode, an opcode, a control-field value, a packet length [5]. The distinction between the model and the device it abstracts is deliberate, and Chapter 6 depends on it. Listing a feature’s attributes and *their values that matter* turns prose into a verifiable space. The value-selection rules differ by kind: for a control field, enumerate every value; for a data field, choose values by how the design processes them [5]. In one published example a maximum-packet-length requirement becomes packet lengths at MAXFL−4, MAXFL−1, MAXFL, MAXFL+1, MAXFL+4 and 65535, crossed with the configuration values that change the limit [4]. MAXFL is the maximum length that the requirement fixes, and the pattern is bound-

ary, both neighbors of the boundary, and the far extreme, each crossed with every configuration that moves it.

Pass/fail criterion. The row states how the response will be judged correct (expected values, timing, protocol) and above all *which errors to look for*: explicit acceptance criteria let an engineer who is not the block’s expert build a reliable check ^[1]. “Transfer completes” is not a criterion. “Every destination byte equals its source byte, in order, exactly once, and the done interrupt follows the last accepted write response” is. A pass/fail criterion is a checker specification, so writing this field well is where the scoreboard of Chapter 8 is designed.

Method. Not every row is a simulation row. One methodology’s response-checking guideline doubles as a method-selection guideline: failures visible at signal level are most efficiently caught by assertions; failures visible per output transaction are most efficiently caught by a scoreboard; statistical failures of performance or fairness are most efficiently caught by offline analysis of a trace ^[4]. The same logic extends across engines: exhaustive, local, control-dominated properties go to formal; data-transport correctness at scale to constrained-random simulation with a scoreboard; and some rows are discharged by *review*, a signed inspection of code or configuration where neither simulation nor proof is the right instrument (see Chapters 14 and 16 for the sim/formal division of labor). Others are discharged by an instrument that the plan does not own. A USB 3.x dual-role controller’s plan sends its link-training and low-power rows, the U0 through U3 transitions, to the specification-mandated compliance suite: the method field reads *compliance suite*, the evidence that closes the row is the suite’s item list, and the plan row records which features the suite discharges and which it leaves open. Those transitions run from the link’s active state through its three low-power states. In one published RISC-V processor verification plan the assignment is dual: formal carries architectural compliance while simulation carries the complex behaviors, custom loop hardware, interrupts, debug mode ^[6]. One published industrial methodology makes this an explicit planning artifact: a scope assessment that decides, before implementation begins, which parts of the effort go to directed tests, which to constrained-random and which to formal, and which kinds of coverage will measure each part ^[7].

Closure metric: one per row. The last field states the evidence that closes the row: a functional coverage group at 100%, a proven property, a passed directed test, a signed review. The discipline is **one row, one metric**: a row needing two kinds of evidence is two rows. Closure is computed row by row (see Chapter 7); a condition such as “coverage *and* the assertion *and* the stress test looks fine” is not automatically evaluable until each term has a measurable acceptance criterion. One published RISC-V plan shows the pattern in its row format: each verification goal is a row carrying its own pass/fail criterion (“check against reference model”), test type (constrained-random) and coverage method (functional coverage), one goal and one measurable discharge ^[6]. A reference model is an independent executable of the specification, run on the same stimulus as the design, so that the two responses can be compared.

The OpenHW Group’s tutorial on writing a verification plan gives its spreadsheet six columns: one for the requirement each row comes from, one for the feature and its sub-feature, one for the verification goal, one for the pass/fail criterion, one for the test type and one for the coverage method; and the tutorial asks the author to record what is to be verified and to leave how to a separate strategy document [8]. A published CORE-V interrupt verification plan is that template filled in, seventy-two feature rows under the same headings [9]. The Verification Academy publishes a pared-back format, one row per requirement in a spreadsheet, with section, title, link and type as the four required columns, and description, weight and goal as the three used most of the time [10].

Worked example: four rows of the DMA vplan

Feature DMA-F06 from Section 5.1 needs two kinds of evidence, so it becomes two rows, F06a and F06b. The must-not-happen entry DMA-N01 is shown alongside them, and its always-on check runs in the same simulations that fill F06a’s cross. It splits into two rows for the same reason, so the two features become four plan rows with four closure metrics (Table 5.1):

Table 5.1 — Four rows of the DMA vplan, carrying the five fields of the row anatomy across two of the features listed in §5.1. DMA-F06 splits into a constrained-random row closed by a covergroup and a formal row closed by a proven property; the must-not-happen entry DMA-N01 splits into the always-on scoreboard check and the injection test whose failure condition is the *absence* of the error message. Two features, four rows, four metrics — the arithmetic the one-row-one-metric rule forces you to notice.

ID	Feature & attributes	Pass/fail criterion	Method	Closure metric
DMA-F06a	2D transfers are split correctly into legal bursts at the 4 KB boundary for all stride signs. Attributes: stride sign $\{+,0,-\} \times$ the 2D row’s relation to the next boundary {more than a burst of headroom, exactly a burst, less than a burst, straddling it} $\times$ burst length {1, 255, 256} $\times$ other-channel {idle, active}	Every AXI burst issued lies within one 4 KB frame; the union of issued bursts equals the transfer’s exact byte set	Constrained-random sim, scoreboard address/data check	Covergroup <code>cg_2d_4kb</code> (below) at 100%
DMA-F06b	Splitter never emits a boundary-crossing burst (any input)	AXI 4 KB rule (Arm IHI 0022H.c §A3.4.1) [11] as a safety property on the backend’s AXI write-address channel	Formal proof, backend standalone	Property <code>p_no_4kb_crossing</code> proven unbounded

ID	Feature & attributes	Pass/fail criterion	Method	Closure metric
DMA-N01a	No write outside the destination window that the splitter computed, any interleaving of the DMA channels	Scoreboard flags any write beat whose address $\notin$ expected window, with feature ID in the message	Constrained-random sim (always-on check)	Check enabled in 100% of regression runs
DMA-N01b	That check demonstrably fires	Injection test drives one write beat past the window's edge; the failure condition is the <i>absence</i> of the error message	Directed sim, error injection	Injection test passes

Splitting N01 keeps its two exit conditions separately traceable. The combined condition “check enabled everywhere *and* proven to fire” joins two kinds of evidence that a tool can evaluate together once each has its acceptance criterion, while two separate rows let that tool track and report each of the two results on its own. The split also gives a home to another planning rule: a checker that has never been seen to fail is not yet evidence. Every planned check earns a companion row for the test that injects the error it is supposed to catch, and the absence of the error message is that test’s failure condition ^[1].

Row DMA-F06b points at an artifact the reader has already met, the property of Chapters 2 and 3, unchanged:

```
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && awready && (awburst == 2'b01)) |->
      (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

What the plan decides here is not the property’s text but its *binding point*. The invariant is a claim about what leaves the block, so the plan row binds it where bursts become observable, on the backend’s AXI write-address channel, and proves it on the backend standalone. Binding the same text inside the burst splitter would prove a claim about an intermediate representation and leave the backend’s own address arithmetic unproven: the plan row would close, and the block would still be wrong.

The closure metric of DMA-F06a is a covergroup, and writing it forces the plan to say what its boundary attribute *means*, which is not a raw address. For each 2D row of the transfer descriptor the monitor computes two byte counts and classifies the row from the pair. The first count is the *headroom*, the bytes from the row’s start address to the next 4 KB boundary in the direction that the stride sign selects. The second is the number of bytes that one maximum-length burst of this row would move, its burst length $\times$ 8 on the 64-bit bus. A 2D row that reaches past the boundary is `straddle`, whatever its bursts do, and one that stays inside its 4 KB region is

classified by headroom against burst bytes. Four cases, mutually exclusive, covering every 2D row exactly once:

```
covergroup cg_2d_4kb @(posedge clk iff midend_txn_done);
  cp_stride_sign : coverpoint txn.stride_sign
  { bins pos = {POS}; bins zero = {ZERO}; bins neg = {NEG}; }
  // txn.bnd_rel: per-row relation, computed by the monitor from the row's
  // headroom (bytes to the next 4 KB frontier, in the stride's direction)
  // and this row's burst bytes. The four values partition every row once.
  cp_boundary_dist : coverpoint txn.bnd_rel
  { bins gt_burst = {GT_BURST}; // stays in frame, more than a burst of headroom
    bins eq_burst = {EQ_BURST}; // stays in frame, exactly a burst of headroom
    bins lt_burst = {LT_BURST}; // stays in frame, less than a burst of headroom
    bins straddle = {STRADDLE}; // reaches past the frontier – must be split
  }
  cp_len : coverpoint txn.burst_len
  { bins one = {1}; bins max_m1 = {255}; bins max = {256}; }
  cp_other_ch : coverpoint other_channel_active;
  x_worst : cross cp_stride_sign, cp_boundary_dist, cp_len, cp_other_ch;
endgroup
```

The worst cases sit in the bin named `straddle` crossed with `neg` and `other_channel_active = 1`: two of the cross's 72 combinations, not the three cells that the burst-length bins would suggest. The missing one is instructive. The generator's constraint solver is the engine that picks each 2D row's values, and it works under a budget that caps a row at 256 beats, so a straddling row is split into pieces and none of those pieces can be a maximum-length burst; the two cells that remain are the 1-beat and the 255-beat ones. That exclusion follows from a budget the team chose rather than from the model itself, since lifting the budget brings the cell back, which is why it belongs on the plan row as a recorded waiver rather than in a footnote. Filling those two cells closes row DMA-F06a only if all other non-excluded cells are also covered and the required address, data and burst-legality checks were active with no unresolved errors in the contributing runs. A scoreboard failure instead identifies a defect, as on SOC-1284, the 2D mis-split escape of Chapter 1. An escape is a bug that crosses a sign-off boundary undetected. A sign-off is the review that judges the evidence against the plan, and the boundary is the point where a block's evidence is accepted and the block passes to the next stage.

A coverpoint is an observed expression that the coverage model partitions into bins, and `cp_other_ch` is the only one here without explicit bins, because a one-bit flag's automatic bins already name both values and label the cross correctly. And *at 100%* means 100% of the cells the model can legitimately reach, fewer than the cross's 72: the six maximum-length straddle cells are possible and are excluded only by the budget the team chose. The exclusion is explicit, with its justification recorded on the plan row (see Chapter 6 for how waivers are written). The model also obeys Chapter 2's rule of pairing: this coverage means something only because the scoreboard check of DMA-N01a runs in the same simulations, and coverage without checking certifies stimulus, not correctness [2].

The covergroup in Chapter 2 crosses the same feature into 36 cells while this one makes 72, and neither is wrong. The stride axis gained a bin because a zero stride, where all 2D rows start at the same address, is its own splitting path rather than a

degenerate case of the other two. The boundary axis changed more deeply: Chapter 2 bins the headroom to the next boundary in *absolute bytes*, against fixed thresholds, while this model bins it *relative to what one burst of this row would move*. A fixed threshold assumes a burst length and cannot identify headroom exactly equal to one burst across varying burst lengths, because that boundary moves with the burst. A coverage model gains a bin when someone learns something about the design, so a closed plan row should record *which version of the model* closed it; otherwise a plan row that read 100% last quarter answers a coarser question, and nothing in the report says so.

One structural warning applies as the rows multiply: the plan stays in the *DUT's* vocabulary (features by design functionality, scenario descriptions, kinds and quantities of coverage) and out of the testbench's, with no component inventories, no test lists, no hard-coded bin values that change with every environment refactor [3]. A plan written in testbench vocabulary is rewritten every time the testbench is; one written in DUT vocabulary survives, and the testbench is derived from it. Each row is assigned to the verification level (block, subsystem, chip) where its behavior is fully contained and observable (see Chapter 3 on controllability, Chapter 8 on environment levels): the bulk lands at block level, and a crowd of rows piling onto one deeply buried unit is the classic sign that the unit needs its own environment [1].

5.3 Making the plan executable and traceable

A plan that lives in a document repository decays, and the industry's recurring complaints are documented: plans that take too long to write, contain nothing the team uses ("nobody reads them"), and rot as the design changes [3]. One published methodology traces all three barriers to a single missing property: the plan is not connected to anything [7]. The cure, introduced in Chapter 4, has a precise definition. An *executable* verification plan is a machine-readable structure that defines the project's scope, lists its features, attaches the testcases, checks and coverage objects to each, and links design requirements down to those objects; that much says what the plan holds, and what makes it "executable" is a further property, that the regression list is generated *from* the plan and that every run's pass/fail and coverage results are annotated *back onto* the plan's objects [7]. What was verified and what was not is then computed rather than remembered, and closure reporting costs nothing because the mapping from results to features already exists [7].

Traceability is the same linkage read in the other direction, and it is built from the identifiers of Section 5.2. Each requirement's unique ID cross-references three things: the specification clause it came from, the testcases and properties that discharge it, and the coverage points that measure it [4]. One audit rule makes the payoff concrete: every feature is annotated with the testcases that verify it, because a feature with no cross-reference is *not being verified*, and the plan is the only place that fact is visible before tape-out rather than after [1]. One published industrial flow adds two practical details that generic planning tools rarely provide out of the box: the first is a requirement-tag field on every testcase and coverage object, linked into

the program's requirements-management system; the second is a per-row *target configuration* field (RTL, gate-level, mixed-signal), so the extracted regression list knows where each test must run [7].

The published template turns the first of those cross-references into a filled field, requiring each planned item to name a source document and the section inside it [8].

On the reference SoC the DMA plan is a version-controlled structured file (the format matters less than the parseability [7]); DMA-F06a names covergroup `cg_2d_4kb` and the scoreboard check; the nightly regression publishes per-row status; and the specification paragraph on burst splitting carries the tag `[DMA-F03..F07]` in its margin. When the specification changes, the tags say which rows are stale, and when a row closes, the dashboard of Chapter 7 says so without a meeting. Chapter 4 described the lifecycle consequence, that every later phase reads progress from the plan; here the point is that *executable* and *traceable* are properties designed into the rows on day one, not export formats bolted on at the end.

5.4 The plan review as a rite

Everything about how the plan review is run follows from one fact, met in Section 5.1: completeness is the property that no single perspective can check, and a designer is responsible for a working design rather than for RTL, so the entire design team, not the plan's author, must contribute to the plan and answer for its completeness [1].

The review includes stakeholders and outside peers. Chapter 4 placed the review in the calendar and named the roster: design, test/product, systems, and verification peers from outside the block, convened at prescribed stages of planning, implementation, debug and closure [7]. Each seat catches a different class of hole: the designer catches features the spec never wrote down (Section 5.1's interview, now on the record), while the architect catches misread intent. The software or systems engineer catches use cases invisible from inside the block, as in Chapter 4's crossbar review, where the firmware engineer's question about the debug port added feature XB-07. The outside peer catches method errors: unmeasurable pass/fail criteria, coverage that certifies stimulus rather than behavior, missing error injection.

Each row undergoes weakness analysis. One published workshop gives the challenge procedure a name, weakness analysis, and three blades, applied row by row [3].

- *Correctness*: whether the plan row is real, which breaks into five checks: that the DUT implements this option; that the specification fully defines the behavior; that the RTL parameters allow it; that the option is a valid use case; and that anyone cares [3]. Rows that fail are deleted, and deleting rows is a review success, because unverifiable and irrelevant rows cost real schedule.
- *Precision*: what exactly is being verified and in what context, the feature's dependencies, its disqualifying conditions, its value ranges and event timing, and

whether different contexts carry different goals [3]. “Verify 2D transfers” fails precision; DMA-F06a’s attribute list passes it.

- **Completeness:** for each influencing variable, how it can change and what the change does to the DUT; for each scenario, what is expected *and what is not expected*; which feature combinations produce unique outcomes that need their own rows [3].

Two more challenges belong in every review. First, the assumptions: every “verified elsewhere,” every “software will never do that,” reviewed face to face with the designers, usually over several rounds, because unspoken assumptions are how features fall through the cracks [7]. Second, the designer’s own “that can’t happen.” Chapter 2 established the mindset, and at plan review it becomes procedure, since whether a case is *valid* is irrelevant and the only question is whether it can occur, in which case the recovery is a feature [2].

Scenario. At the DMA plan review, the block’s designer challenges plan row DMA-F06a: “negative-stride 2D descriptors that straddle a boundary can’t happen, because the driver library always allocates row-aligned buffers.” The verification lead asks two questions. *Is the driver library the only software that will ever program this block?* (No: the register interface is documented, and the neural-accelerator team writes its own descriptors.) *Does the RTL check the condition, or assume it?* (Assume.) **Approach.** The plan row stays and its risk rank rises, because an unchecked assumption reachable by third-party software is where escapes breed, and a new must-not-happen row is added for the frontend: descriptors the design cannot honor are rejected with an error, not silently mangled. **Why.** The challenge and its resolution are recorded on the plan row, linked directly to the plan object, where audits and quality reviews can reach them [7], and the exchange itself does what no coverage metric can: it tests the plan against knowledge that exists only in people.

The review marks a status change: before it the plan is a draft owned by one person, after it the block’s contract, amended only with the same audience (see Chapter 4 for where the reviews sit in the calendar, and Chapter 24 for review culture at team scale).

5.5 Risk: deciding how deep to go, feature by feature

Depth (how much stimulus, how many crosses, which engines, how much of the schedule) is a per-feature decision, and risk is the input that decides it.

The base mechanism is priority, written into the plan rows in three tiers: *must-have* features define first-time success, gate the project’s completion and get thorough verification across all configurations; *should-have* features get their basic operation verified, with depth added only as time allows; *nice-to-have* features are verified “as time allows”, and today’s design schedules almost guarantee that this promise means never [1]. One methodology operationalizes the same idea as requirement

ranking, in which resources go to the highest ranks first and tape-out is decided when the most important requirements are met, plus requirement *ordering*, because dependencies dictate sequence: register access is verified before the configurations that use it ^[4]. The point of the tiers is that when the schedule guillotine falls (see Chapter 4), the cut line was drawn deliberately, months earlier ^[1].

Priority ranks features by consequence. Risk adds the second axis, the likelihood of a bug, and bug risk and complexity risk are among the published axes along which features are isolated for analysis ^[3]. In practice the risk drivers that a plan review scores are five: *novelty* (new logic versus silicon-proven reuse, which the reuse assessment of Chapter 4 feeds directly ^[7]); *structural complexity* (shared state, concurrency, arbitration); *specification quality* (Section 5.6); *controllability and observability* at the chosen level; and the *cost of escape* (silent data corruption versus a flag readable by software). Where consequence and likelihood are both high, the plan buys depth in every currency at once: more attributes and deeper crosses in the coverage model, a second engine on the same behavior (DMA-F06a and F06b, simulation for the transport, formal for the invariant), tighter closure criteria, review of the checker itself.

On DMA-F06 the five drivers read: *novelty* high, a new 2D negative-stride path rather than silicon-proven reuse; *structural complexity* high, splitter resumption arithmetic in state shared between the two DMA channels; *specification quality* high, since the document is silent on this case; *controllability and observability* low, both good at block level because descriptors are programmed directly and every issued burst is visible on the backend interface; *cost of escape* at the worst available value, silent data corruption that no status flag reports. Controllability and observability is the one driver arguing for less depth. Four drivers high, and the favourable one only makes depth affordable rather than unnecessary: high consequence $\times$ high likelihood. The same five questions on DMA-F01 invert every answer, driver by driver: reused register logic; that logic generated with its own tests from one description through a register abstraction layer (RAL, see Chapter 10); a fully specified access policy; complete observability; and a consequence that software can read back. Its plan row therefore buys almost nothing, deliberately. Table 5.2 ranges both features and three others across the same two axes.

Table 5.2 — Five DMA features ranked on the two axes that set verification depth — the consequence of an escape and the likelihood of a bug — with the depth each ranking buys. The rows are deliberately unequal, and that is what a risk assessment produces: DMA-F06, novel and with a silent-corruption consequence, buys a full cross in constrained-random simulation and a formal property besides, while the reused register logic of DMA-F01 buys generated tests and a review of the generator’s configuration and nothing more.

DMA feature	Consequence	Bug likelihood	Planned depth
DMA-F01 register access	medium	low (generated RAL)	Generated register tests; review of the generator config
DMA-F03 1D split into bursts	high	medium	CR sim + scoreboard; boundary-value coverage
DMA-F06 2D neg-stride @ 4 KB	high (silent corruption)	high (novel, shared state, spec silent)	Two rows: full cross in CR sim (F06a) + formal safety property (F06b)

DMA feature	Consequence	Bug likelihood	Planned depth
DMA-N01 window violation	high (silent corruption)	medium	Two rows: always-on scoreboard check (N01a) + its injection test (N01b)
DMA-F09 error routing	medium	medium	CR error injection; per-channel status coverage

The unit of ranking is not always the module. On the GPU the tile binner sorts each triangle into the screen tiles it covers. The same five questions are asked per workload class, because that binner is silicon-proven when it handles one full-screen triangle, while the same binner under thousands of sub-pixel triangles straddling tile edges is novel, poorly observable and expensive to escape. Depth is bought per workload class, and one module appears in rows of three different depths. That is possible only because the rows were written against behaviors rather than blocks.

Depth can also be staged in time. One published RISC-V coverage plan from 2020 makes maturity explicit in the metric: a *compliance basic* level gives stimulus-effectiveness insight while the design is immature and changing, and *compliance extended* becomes the closure goal once the design is stable [6], so the exit criteria tighten as the design earns it. Depth also interacts with method economics: the 2006 estimate is that a directed-test plan stays manageable when testcases number in the low hundreds, while a project with over a thousand identified testcases would need more than a year of hand-writing, and that arithmetic is what pushes high-risk features with many attribute values toward constrained-random automation that closes on coverage rather than on enumeration [1]. The same arithmetic appears on the coverage side: in that 2020 plan, hand-writing the coverage model for a 120-instruction ISA costs roughly 3,600 lines of SystemVerilog at 10–30 lines per instruction, and a 300-instruction extension another ~9,000, so the published conclusion is that generation, not typing, is the only sane source for that class of model [6].

That plan is the verification plan for the OpenHW Group’s CV32E40Pv2, a 32-bit in-order RISC-V core: the core began as ETH Zurich’s RI5CY, Dolphin Design develops it onward, Imperas supplies both the reference model and the generated coverage model, and the plan is public, published in the `core-v-verif` repository as files a reader can inspect row by row [6]. The arithmetic and the conclusion drawn from it are the CV32E40Pv2 programme’s own [6].

5.6 Planning against an incomplete specification

Reality is documented in the 2024 industry data: specifications that change, or arrive incorrect or incomplete, remain a common concern for verification engineers and project managers alike, and design errors have historically been the leading root cause of the functional flaws behind respins, with recent signs of improvement [12]. A respin is a new fabrication cycle with corrected masks, forced by an escape. The two are entangled, because a designer implementing an ambiguous sentence is making a design error whose cause lies in the specification. The planning literature is equally blunt: documentation quality is the primary input to planning; the worst case is a

feature with no documented description, where the verification engineer must do original research before a meaningful plan is possible; and frequent or late design changes can invalidate large portions of planning work outright [7].

The published “detail levels” guidance is the opposite of both denial and paralysis, and it has two branches, one for each state of the specification: where the specification is complete and detailed, the team invests in a complete, detailed plan up front, with coverage and assertions specified in the rows; where it is minimal or still in progress, the team writes a less detailed plan now, fully elaborates only the first few deliverables, adds detail to later rows *as the information is finalized*, and puts explicit planning-and-architecture time inside each later deliverable’s estimate [3]. Change handling follows the same shape: small specification changes are absorbed into future rows; significant ones add new deliverables rather than silently stretching old ones, and their impact is reviewed with the team so the cost is communicated in a form everyone already understands [3]. Two disciplines keep the boundary in place. First, the boundary from Section 5.1: an unspecified genuine don’t-care is excluded from verification, but an *incomplete* specification is a defect, and the plan’s job is to surface it for fixing rather than paper over it [4]. Second, the plan must never become the spec, since a specification that is a consequence of the implementation is unverifiable [1], so every hole the plan finds goes back through the specification’s owner, in writing.

Scenario. The neural accelerator’s unit of work is one convolution, programmed through its registers; this book calls it a *convolution job*. The accelerator enters planning with a specification solid on the datapath (2/4/8-bit weight modes, the 16-lane MAC array, quantization and saturation semantics) and hollow on system behavior: the memory-streaming interface section is headed “TBD pending crossbar arbitration experiments,” and the rules for aborting a job run to a single sentence.

Approach. The verification lead writes the plan anyway, in three zones. Zone 1, fully elaborated rows for the stable datapath (attribute-complete: weight width $\times$ saturation boundary $\times$ accumulator sign crosses), scheduled first. Zone 2, *stub* rows for the streaming interface: feature named, spec reference marked TBD(\$4.2), pass/fail and attributes deliberately blank, each stub carrying a planning-time allowance in its estimate and a named specification owner. Zone 3, a spec-hole log: every question that the extraction passes raised and the spec could not answer (“what does the engine do if the crossbar stalls the weight stream mid-row?”), filed as specification defects with dates, reviewed weekly with the architect. The plan review covers zones 1 and 3 now; zone 2 rows are review-gated on their spec sections landing. **Why.** Waiting for a finished specification would delay planning and the questions that help complete it; planning against guessed behavior risks invalidating the work when the specification changes [3, 7]. The zoned plan assigns specification gaps to an owner and tracks them through the log and weekly review, making the verification team the specification’s first systematic reviewer.

Planning is not a phase that consumes a finished specification; it is the activity that *tests* the specification, years before silicon tests the design, and every question in Section 5.1’s passes that the document cannot answer is a specification bug found at the cheapest possible moment.

IN PRACTICE

The first draft of the feature list should be timeboxed to days, not weeks: the published guidance is low effort for fast results, with more detail added later where it proves necessary [3], because completeness comes from the review rather than from the author polishing alone. The plan should live in version control next to the RTL. It should use a parseable format because its derived artifacts (regression list, dashboard, traceability report) are generated from it [7]; each row should have an owner. The must-not-happen and out-of-scope lists warrant particular scrutiny: an omission from the must-not-happen list leaves a failure unchecked, while a wrong entry on the out-of-scope list excludes behavior that required verification. The budget should allow for planning’s front-loaded cost: the 2013 figures quantified in Chapter 4 are 10–20% added at the front, savings of 10–40% of the overall verification schedule and at least 20% of implementation and closure effort [7], from a single organization’s reported experience.

PITFALLS

1. **Transcription planning.** *Symptom:* the plan is the specification’s headings with “verify that” prepended. *Cure:* run the three extraction passes and the designer interview; zero features tagged [interview] or [arch pass] means extraction never happened.
2. **Unmeasurable rows.** *Symptom:* pass/fail criteria like “works correctly under stress.” *Cure:* every row states the observable fact that decides it and the single metric that closes it; a row that cannot name its metric is split or deleted.
3. **Affirmative-only plans.** *Symptom:* no must-not-happen rows, no error injection, checkers that have never been seen to fail. *Cure:* a negative twin for every data-integrity feature; an injection test per checker whose failure condition is the *absence* of the error message [1].
4. **Testbench-vocabulary plans.** *Symptom:* rows named after agents and sequences; every environment refactor forces a plan rewrite. *Cure:* DUT functionality and scenario descriptions only; the testbench is derived from the plan, never the reverse [3].
5. **The signing-ceremony review.** *Symptom:* a one-hour walkthrough, no rows changed, no assumptions challenged. *Cure:* weakness analysis row by row (correctness, precision, completeness) with designers, systems and outside peers present and the challenges recorded on the rows [3, 7].
6. **Uniform depth.** *Symptom:* every feature gets the same coverage template regardless of novelty or consequence. *Cure:* rank consequence × likelihood per fea-

ture; spend crosses, engines and schedule where both are high, and draw the cut line for the rest in advance [1, 4].

SUMMARY

- A behavior is verified only if it is in the plan, so extraction has the highest cost of omission in verification [1].
 - Extraction is adversarial and runs in three passes (interfaces, functions, architecture), followed by an interview of the designer for the corner cases only the implementation knows and by explicit must-not-happen and out-of-scope lists [1, 4].
 - A vplan row is feature $\rightarrow$ attributes $\rightarrow$ measurable pass/fail criterion $\rightarrow$ method $\rightarrow$ one closure metric: a row needing two kinds of evidence is two rows.
 - Method follows the shape of the failure: assertions for signal-level wrongs, scoreboards for transaction-level wrongs, offline analysis for statistical ones, formal for local invariants, review where no engine fits [4].
 - An executable plan generates the regression list and absorbs the results; traceability ties spec $\leftrightarrow$ feature $\leftrightarrow$ test/coverage by permanent IDs, and a feature with no cross-reference is not being verified [1, 4, 7].
 - The plan review is a rite with a roster (design, systems, test, outside peers) and a procedure: challenge each row's correctness, precision and completeness, and every "that can't happen" [2, 3, 7].
 - Risk (consequence $\times$ likelihood) sets per-feature depth; priority tiers pre-draw the schedule cut line; an incomplete spec is planned in zones, with its holes filed as specification defects [1, 3, 4].
-

EXERCISES

Exercise 5.1 (calculation). Take the two coverage-model line counts §5.5 quotes from a published coverage plan, together with the instruction counts they belong to and the per-instruction rate the same sentence gives as a band. Compute the rate each total implies and locate it in the band. State what follows for quoting those totals as estimates, and name the conclusion §5.5 reports that source as drawing from them.

Exercise 5.2 (artifact). Read the plan row below against the five fields of the row anatomy in §5.2. Name the field left blank and the rule of §5.2 that governs what may be written there. Name the field whose content fails the same section's requirement on pass/fail criteria, and restate that content so that it names an observable fact. Name the cross-reference §5.3 requires of every row and that this row does not carry.

```
ID: DMA-F11
Feature and attributes: Midend: descriptor chaining
Pass/fail criterion: Chained transfers complete correctly under stress
Method: Constrained-random simulation
Closure metric:
```

Exercise 5.3 (calculation). Count the cells of the cross `x_worst` from the attribute list of row DMA-F06a in §5.2, then compute how many of those cells count toward the row’s closure under the convention §5.2 states for a covergroup at full coverage. Name the premise the exclusion rests on, state whether it belongs to the model or to a decision the team took, and name where §5.2 requires it to be recorded. Give the refinements §5.2 names as the reason the earlier, coarser model of Chapter 2 counts fewer cells for the same feature.

Exercise 5.4 (judgement). At the DMA plan review a designer proposes moving row DMA-N02 out of scope, on the ground that the driver polls the status register and never enables the done interrupt. Decide which of the three lists of §5.1 the row belongs on after that argument, and give the property §5.1 requires of every out-of-scope entry. Name the second of the two questions the verification lead asks in §5.4 and state what its answer does to the row’s risk rank.

Exercise 5.5 (judgement). A lead applies one coverage template to every row of the DMA plan. Decide what §5.5 puts in place of a uniform template, then walk the five risk drivers of that section across DMA-F01 and DMA-F06 and give the depth each ranking buys. State why the one favorable driver of DMA-F06 does not reduce its depth.

FURTHER READING

Corpus sources.

- [1] Bergeron, *Writing Testbenches Using SystemVerilog*, ch. 3: on feature extraction, prioritization and testcase derivation, and the source of this chapter’s extraction questions.
- [4] *Verification Methodology Manual for SystemVerilog*, ch. 2: planning as rules and recommendations, with must-not-do requirements, unique identifiers, ranking, response-checking selection, coverage translation.
- [5] Piziali, *Functional Verification Coverage Measurement and Analysis*, ch. 2 and 4: the plan as the verification environment’s specification; attributes and attribute-value selection; the coverage plan (Chapter 6).
- [7] “Best Practices in Verification Planning”: the executable-plan definition, template discipline, assumption reviews, and the planning ROI data.
- [3] “Proven Strategies for Better Verification Planning”: feature isolation, scenario classification, weakness analysis, and planning under incomplete specifications.
- [6] “Planning for RISC-V Success”: a published vplan row format with per-goal pass/fail and coverage method; staged coverage levels; the arithmetic for generated coverage models.

- [2] “Verification Mind Games”: why design-mindset plans miss the obscure bugs (the mindset itself is Chapter 2’s subject).
- [12] Wilson Research Group 2024 IC/ASIC report: specification issues as a persistent root-cause concern behind functional flaws.
- [9] the OpenHW Group’s interrupt verification plan for its CV32E40P core, a real, filled vplan in spreadsheet form; [8] is the same group’s tutorial on writing one, and [13] is where its verification strategy draws the line between the plan and the document that supports it.
- [10] the Verification Academy’s *Coverage Cookbook*: the testplan format behind Section 5.2’s four required columns, and its process for carrying a requirement from the specification into a coverage model.

Not in the corpus (pointers only):

- Haque, Michelson, Khan, *The Art of Verification with VERA*: origin of the interface/function/corner-case extraction ordering that Section 5.1’s three passes adopt.
- Wile, Goss, Roesner, *Comprehensive Functional Verification*: planning at industrial scale, including the specification-review culture this chapter compresses into Section 5.4.

REFERENCES

1. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. **D21** J. Montesano and M. Litterick, "Verification Mind Games: how to think like a verifier," DVCon, 2014.
3. **D7** P. Marriott, J. Vance, and J. McNeal, "Proven Strategies for Better Verification Planning," DVCon 2022 Workshop, 2022.
4. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
5. **B5** A. Piziali, "Functional Verification Coverage Measurement and Analysis," Springer, 2004.
6. **D6** D. Graham, A. Sutton, S. Davidmann, P. Gouedo, X. Aubert, and Y. Pruvost, "Planning for RISC-V Success: Verification Planning and Functional Coverage," Proc. DVCon Europe, 2023.
7. **D9** B. Ehlers, C. Vargas, and P. Carzola, "Best Practices in Verification Planning," Proc. DVCon U.S., 2013.
8. **O1** OpenHW Group, "How to Write a Verification Plan (Testplan)," [openhwgroup/core-v-verif, docs/VerifPlans/VerificationPlanning101.md](https://github.com/openhwgroup/core-v-verif/docs/VerifPlans/VerificationPlanning101.md), commit f3b1f971e0e6b94daae46d279cc50ca390785369, 2021, accessed 2026-09-02.
9. **O4** OpenHW Group, "CV32E40P Interrupt Verification Plan," [openhwgroup/core-v-verif, cv32e40p/docs/VerifPlans/Simulation/interrupts/CV32E40P_interrupts.xlsx](https://github.com/openhwgroup/core-v-verif/cv32e40p/docs/VerifPlans/Simulation/interrupts/CV32E40P_interrupts.xlsx), commit f3b1f971e0e6b94daae46d279cc50ca390785369, accessed 2026-09-02.
10. **B7** Verification Methodology Team, "Coverage Cookbook," Mentor Graphics Verification Academy, 2013 (revised 26 April 2019).
11. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
12. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.

13. [02](#) OpenHW Group, "Verification Planning and Requirements," CORE-V Verification Strategy, [openhwgroup/core-v-verif](#), docs/VerifStrat/source/planning_requirements.rst, commit [f3b1f971e0e6b94deae46d279cc50ca390785369](#), 2020, accessed 2026-09-02.

6. Coverage: Theory and Practice

Coverage records which situations a verification campaign exercised and supports reasoning about those it missed. On the neural accelerator, every test passes in a two-week regression, and code coverage on the accelerator RTL reads 100%: every line and every branch executed. Neither result rules out a failing convolution job: the accelerator’s unit of work is one convolution programmed through its registers, and the job in question comes from an audio keyword-spotting model that uses 2-bit weights and a tensor one pixel wide, launched while the DMA engine streams camera frames through the same SRAM banks. Contention starves the weight prefetcher, and the starved prefetcher wraps its buffer pointer one entry too early, but only when the weight stream ends part-way through a beat, a single data transfer within a burst. The stream ends part-way through a beat only with degenerate geometry and the narrowest precision. The output feature map is subtly wrong. The two weeks of regressions never created this situation, and the verification metrics in place could not reveal its absence. Code coverage records executed lines; the missing configuration combines precision $\times$ geometry $\times$ contention in the specification, beyond the code structure. This chapter develops functional coverage for such combinations and the discipline of trusting it only as far as the model supports.

CHAPTER MAP

§6.1 defines the coverage families and their blind spots; §6.2 examines code-coverage sub-metrics, whose implementation basis makes holes informative but fullness inconclusive. §6.3 covers the effort of deriving models from specification intent, using covergroups, coverpoints, bins and crosses, and the limits of automatic bins. §6.4 relates “100% coverage” of a model to Chapter 3’s impossibility results. §6.5 derives attributes and coverpoints from features, with bins encoding intent. §6.6 compares accelerator models reaching 100% on the same feature to show why reports require model review. §6.7 shapes combinatorial crosses with exclusions justified in the vocabulary of the specification, which makes the claimed reachable space reviewable and lets a checker refute it. §6.8 covers hole analysis, targeted tests, constraint tuning and stopping criteria; §6.9 defines reviewable exclusions for situations that cannot or need not be covered without redefining success.

6.1 The three coverage families and their taxonomy

Simulation provides an existence proof: a particular stimulus produced a particular response, and the checkers accepted it. Coverage complements that evidence by recording which situations the tests created, so the team can reason about the ones they did not. Chapter 4 placed coverage closure in the lifecycle; this chapter develops its measurement.

Practitioners sort coverage into three families. **Code coverage** measures which RTL structures the simulations executed, including lines and branches, expressions and transitions. **Functional coverage** measures which *specified behaviors* occurred, using models the engineer writes deliberately. **Assertion coverage** measures the activity of assertions: which were evaluated, triggered, completed. The distinction concerns who defines each metric and from what source, beyond the choice of tooling. The Verification Academy states that no single metric characterizes the verification process completely, since a regression at full code coverage may still leave specified functionality unverified, and that a complete picture of a project's progress often needs several metrics together [1].

Piziali's founding taxonomy supplies this chapter's framework of two axes. A coverage metric has a *kind* and a *source*. Its kind is **implicit** when the metric is inherent in a representation, such as the statements of a hardware description language, and **explicit** when an engineer invents and defines it; its source is the design's **implementation** or its **specification** [2]. The two axes define four coverage spaces. Code and structural coverage are implicit-implementation; functional coverage built from the specification is explicit-specification; and functional coverage of microarchitectural detail, such as a pipeline's occupancy or an implementation FIFO's high-water mark, is explicit-implementation. The fourth quadrant, implicit-specification, would hold metrics inferred automatically from a natural-language specification, and as of 2004 it had no practical realization [2]. Implicit metrics require no modeling effort or explicit engineering judgment; explicit metrics require effort and depend on the engineering behind them. A **coverage space** is a multi-dimensional region described by attributes and their values, loosely called a *coverage model* in industry [2].

The accelerator's line coverage is implicit-implementation, computed automatically at no modeling cost and fully covered. The absent model of precision $\times$ geometry $\times$ DMA contention would have been explicit-specification. A model of the weight prefetcher's buffer occupancy, where the bug occurs, would have been explicit-implementation; it too was absent. Three quadrants were empty; the quadrant that was filled, implicit-implementation, is the one that asks for no explicit engineering judgment.

By 2024, adoption of code coverage, functional coverage and assertions in the IC/ASIC market had matured rapidly between 2007 and 2014 and since largely stabilized within the margin of error, whereas *constrained-random* simulation kept climbing, a technique that generates legal but unlikely stimulus automatically under declared constraints [3]. Coverage is established practice; assessing what it measures requires examining the model.

6.2 Code coverage and its structural limits

Code coverage instruments the RTL and records which of its structures execute during simulation. Its purpose is to determine whether some design code was left unexercised [4]. Because the metric is implicit in the language, tools extract it automatically without model design, judgment or maintenance. That economy supports enabling it in every flow, with its sub-metric limits understood.

Line and statement coverage report which source lines executed at least once. They expose dead configuration branches and unexercised error paths, including whole features never reached; an untriggered DMA error-reporting path appears as a block of red lines. They reveal nothing about *why* or *in what context* a line ran: a statement that executed once, even under the most favorable conditions, is “covered.”

Branch coverage reports whether each decision point took both directions. It is strictly stronger than statement coverage and still combinatorially weak, because covering decisions individually says nothing about **path coverage**, the sequences of decisions taken together. A suite can hold 100 percent statement coverage while covering only 75 percent of control paths, and paths grow exponentially with the number of control-flow statements [4]. The DMA burst splitter is the stage that splits each transfer into bursts that the AXI address rules allow. One branch of the splitter splits a transfer that would cross the 4 KB boundary, so that every burst stays inside one address region and never reaches a second subordinate. Another branch handles a negative stride, an address step that walks memory downwards. Each branch may be covered without any one transfer taking both branches, and that untaken combination is what contains this book’s canonical corruption bug.

Condition and expression coverage check whether each controlling term in a Boolean expression independently caused the decision. For a branch guarded by `parity == ODD || parity == EVEN`, statement coverage reads 100 percent while expression coverage sits at 50 percent, because no testbench ever set `EVEN`; statement coverage cannot report this oversight. Reaching 100 percent expression coverage is itself extremely difficult [4].

Toggle coverage reports which bits of which signals changed value (0→1 and 1→0). It is the coarsest metric in the family and approximates whether any activity reached a structure at all, useful for integration sanity and power-intent connectivity. It is also the only code metric that sees inside structures the others treat as one object.

FSM coverage extracts state machines and reports visited states and traversed arcs. A five-state machine with 14 possible transitions reaches 100 percent state coverage from a single sequence covering only 36 percent of the transitions. Because transitions come from the implementation, the tool cannot identify unintended or missing intended transitions; the graph needs review against the specification [4]. Some transitions may be unreachable under the protocols feeding the FSM; a property checker determines which [4], as Part IV discusses.

Code coverage is computed **from the implementation, against the implementation**, so it cannot represent anything the designer did not write. The code in an empty module is always 100% covered [4]. If the accelerator’s designer never implemented the 2-bit weight unpacking mode, no code metric will report it missing; every test of the modes that *do* exist passes and the coverage report shows no gap. 100% code coverage means the entire design implementation was executed. It indicates neither correctness nor completeness of the verification suite: low numbers reliably signal a problem, but high numbers do not establish completion [4].

Code coverage therefore serves as a cheap automatic backstop from the first regression: every hole indicates a missing test, dead code or an implementation detail that has not been understood, while full coverage cannot establish completion.

Scenario. The crossbar’s ID field is 4 bits wide, and in this regression the five managers only ever index IDs 0 through 4, because the sequence library pins one ID per manager; the debug manager, ID 4, issues nothing at all. So ID bits 3 and 2 never change value. Line coverage on the ID-decode logic reads 100%: the decoder executes every cycle. Toggle coverage remains at 50%, with four of the eight bit transitions missing. **Approach.** Tracing both stuck bits first suggests an incorrect explanation. Bit 3 appears structurally unused because the five managers never index past 4, but the 4-bit field permits any of 16 AXI4 values; the interconnect widens it so managers need not coordinate their IDs (Arm IHI 0022H.c §A5.1 “AXI transaction identifiers”, §A5.2.3 “Interconnect use of transaction identifiers”) [5]. The regression establishes only a stimulus restriction. Bit 3 is *unreached*; excluding it records a sequence-library decision requiring an owner and expiry (Section 6.9), without establishing structural unreachability. Bit 2 is set only by the debug manager, and the environment never routed that manager’s transactions through the crossbar: the stuck bit records a testbench wiring bug that line coverage could not see. **Why.** Different code metrics expose different blind spots, so disagreements among inexpensive metrics help diagnose gaps. The second thing the scenario shows is that a claim of unreachability needs evidence: misclassification is costly, so the claim must name the clause that establishes it.

6.3 Functional coverage from specification intent

Functional coverage requires up-front modeling effort: the engineer specifies it entirely from specification intent, independently of code structure, whereas code coverage is inferred automatically [6]. It is the only metric family that measures whether specified situations occurred, including those the implementation omits, as in the opening example. The Verification Academy states the same distinction between the two model kinds: a structural coverage model is created automatically, while a functional one, being explicit, must be written by hand before anything can be measured [1].

SystemVerilog is the common language for these models (IEEE 1800-2023, Clause 19). A **covergroup** is the container: a user-defined type that encapsulates the specification of a coverage model, defined once and instantiated as many times as the

environment needs [6]. Inside a covergroup, a **coverpoint** designates one variable or expression to observe and partitions its values into **bins**, where each bin is a counter for a value or a set of values whose occurrence matters to the model. A **cross** enumerates Cartesian combinations of two or more coverpoints, where most interesting bugs arise through attribute interactions, as the breakage thinking of Chapter 2 predicts; that habit asks how the design could break, and under what interference, rather than whether it works. Bins also express negative intent: `ignore_bins` removes values from the space, while `illegal_bins` raises a runtime error when an illegal value occurs, taking precedence over any other bin that would have counted it [6]. Section 6.9 develops this distinction: *ignore* excludes a value from the measurement; *illegal* requires that it never occur, so the second acts as a checker.

Without explicit bins, a coverpoint gets one per value for small domains; otherwise its range is sliced uniformly into at most `auto_bin_max` bins, default 64 [6]. Slicing the accelerator’s 9-bit tensor-width field into 64 uniform bins gives eight values per bin. Width 16, the MAC lane count and a behavioral boundary, then shares the bin `auto[16:23]` with widths 17 through 23. Widths 16 and 17 are exactly the two the datapath treats differently, and the tool has just merged them. Section 6.6 returns to this problem of bins that conceal behavioral differences.

Functional coverage divides in two: **data/stimulus coverage** records which values and configurations were processed, best captured with covergroups; **protocol/activity coverage** records which sequences and control scenarios occurred, best expressed as `cover property` statements over temporal sequences [7]. A 2-bit-weight job under DMA contention for SRAM is data coverage; a write response with two same-ID writes outstanding under back-pressure is activity coverage, expressed directly by a three-line sequence in SystemVerilog Assertions (SVA), the notation in which temporal design rules are written, but awkward to sample in a covergroup.

Assertion coverage, the third family, measures assertion activity. The antecedent of an assertion is the condition that must become true before the assertion checks anything. At minimum, assertion coverage records which assertions were evaluated and which were vacuously satisfied (IEEE 1800-2023 §16.14.8) [6]: an assertion whose antecedent never occurred has verified nothing, a direct application of Chapter 3’s observability argument that what is never observed is not verified. Beyond that, `cover property` directives are declared functional coverage points in their own right, and designers should mark implementation corner cases with them [7] so unverified cases are visible without relying on someone remembering the plan. Assertion coverage applies to checker assertions and to coverage assertions alike, and it is itself measurable and analyzable [2]. This connects to Chapter 11: assertion coverage establishes whether the checkers were active.

The three families provide three complementary observations: implementation execution, occurrence of specified behavior, and checker activity during that behavior. Verification needs all three; Part IV adds a fourth perspective, formal coverage models [8] (see Chapter 16).

6.4 The scope and limits of 100% coverage

Every coverage percentage depends on its denominator. At 100% functional coverage, the accelerator's denominator contains everything the model's author enumerated, rather than everything the accelerator can do. Reaching 100% functional coverage indicates the completeness of the test suite *against the model*; it says nothing about the completeness of that model and, unless bins encode invalid values, nothing about correctness [4].

Chapter 3 established that the reachable state space is finite but too large to enumerate, making exhaustive verification impossible; behavior is therefore partitioned into equivalence classes and each is sampled. A functional coverage model is that partition, made executable: bins are equivalence classes with a counter attached. It therefore inherits the partition's loss: a chosen projection of an intractable space. "100%" measures progress across the projection, never across the space.

Fidelity measures how closely a coverage model captures actual behavioral requirements. A control register with 18 specified values, modeled as 18 bins, has no abstraction gap; a 32-bit address bus whose 2^{32} values are grouped into 16 ranges has a substantial one. Fidelity is also lost by omitting *relationships*: if bit CR3 may only be 1 when CR7 is 1 and the model never encodes that dependency, it cannot distinguish this bug-prone coupling [2]. Fidelity matters because a bug occupies a region of the coverage space, its *footprint*: some bugs occupy a huge one, triggered by any test in the right mode, while others occupy a point requiring one instruction, one fault type and one interrupt to coincide. A high-fidelity model is far more likely to distinguish, and therefore demand, the bug-exposing scenario; a low-fidelity one files it with its harmless neighbors [2]. High fidelity everywhere is unaffordable because it leads to Section 6.7's intractable full cross; fidelity must therefore be allocated where the risk analysis of Chapters 3 and 5 identifies likely bugs.

Coverage measures **sampling**; checkers establish correctness. It records that a situation occurred; scoreboards, reference models and assertions determine whether the design behaved correctly (Chapters 8 and 11). Coverage methodology *assumes* an orthogonal mechanism detects device misbehavior [2]. Without output comparisons, a covergroup merely counts jobs: the unread-log problem of Chapter 2 applies to coverage as well. Verification aims to remove bugs; coverage closure is only a *derivative* of that goal, a stand-in for it that is well defined and measurable [9]. A derivative can be optimized directly: if stimulus is tuned only for bin hits, the campaign can reach 100% with inactive checkers, without evidence that misbehavior would have been detected. Coverage-model reviews must therefore include the checkers observing the same events and identify, for every coverpoint, who detects misbehavior in that scenario.

6.5 Designing the coverage model: an engineering act

Chapter 5 ends with a verification plan, the *vplan* named throughout this chapter: features extracted from the specification, each with a measurable pass/fail intent.

Coverage-model design turns those features into attributes and bins, crosses and sampling events that a simulator can count. The work has two phases: **top-level design**, which writes the model’s semantic description and identifies the attributes and their relationships, and **detailed design**, which answers *what* to sample, *where*, and *when* to sample and correlate attributes [2]. The semantic description comes first, one sentence of the form “record all valid operating conditions of X defined by attributes A, B, C” [2]. Writing it requires the author to state exactly what the model claims to measure, the only thing a reviewer can hold it to.

The judgment concentrates in three places.

Choosing attributes. Attributes come from the specification’s parameters (the accelerator’s weight precision), the environment’s degrees of freedom (DMA contention) and, with restraint, the implementation’s own choices (the prefetch buffer’s occupancy). The first two make specification coverage; the third is implementation coverage in Section 6.1’s taxonomy, legitimate but answering a different question about stress on this microarchitecture [2]. An attribute does not belong in the model when no checker observes it and no risk motivates it.

Choosing bins. Bins implement breakage thinking (Chapter 2). Bins distinguish behavioral changes at boundaries and alignment points, wrap-arounds, hardware resource counts and the value one past a count. Tools default to uniform slicing [6]; deliberate bins are unequal, dense at behavioral changes and coarse where the specification is indifferent. A bin describable only as `auto[128:191]` lacks a requirement-based meaning.

Some behavioral boundaries occur in data rather than configuration. On a video codec, control flow depends on the picture itself, on features such as an all-skip macroblock, a motion vector pinned at the search-window limit, or a coefficient block whose values fall outside the entropy coder’s tables. The monitor must therefore *compute* the bin from the stream rather than read it from a register. These frame classes are relevant when comparison with a bit-exact reference model reveals a mismatch.

Choosing sampling. Sampling location must match the interface or unit whose behavior the attributes describe: job attributes at the accelerator’s register interface and contention where SRAM arbitration happens, rather than wherever an object handle is convenient. Sampling must occur when the values are jointly valid and stable. The accelerator model below samples at job completion with the contention flag *latched during* weight streaming; sampling at job launch would record whether the DMA happened to be busy at an irrelevant instant, giving each filled bin a misleading meaning. Correlation time is a first-class design decision [2].

Scenario. The DMA vplan hands over feature DMA-F06a (Chapter 5): “2D transfers are split correctly into legal bursts at the 4 KB boundary for all stride signs.” The 4 KB boundary that this plan row names is the one AXI4 imposes on every burst (Arm IHI 0022H.c §A3.4.1 “Address structure” [5]); no burst may cross it. The row’s closure metric, the evidence that decides when the feature counts as covered, is the covergroup `cg_2d_4kb`. **Approach.** Bins follow behavioral boundaries: stride

sign, pos, zero, neg (**3** bins); distance from the start of a row to the next 4 KB boundary relative to burst size, gt_burst, eq_burst, lt_burst, straddle (**4** bins); extreme burst lengths, 1, 255, 256 (**3** bins); and activity on the other DMA channel (**2** bins, one per value of a one-bit flag, where automatic bins are meaningful). The cross is $3 \times 4 \times 3 \times 2 = 72$ combinations, each nameable in specification vocabulary. **Why.** The canonical DMA bug occurs in the neg $\times$ straddle $\times$ other-channel-active corner: two of those 72 points rather than three. The team's stimulus generator caps a 2D row, a line of a two-dimensional DMA transfer, at 256 beats. The cap is a budget the team sets on the constraint solver, the engine that picks each row's values; the cap is not a limit the AXI rules impose, so a straddling row splits into pieces and none of them can be the 256-beat maximum. The third of those points, the one at the maximum burst length, is therefore unreachable through a recorded generation choice, independently of stimulus luck, and it requires a waiver instead of further tests (Section 6.9). These bins require the bug scenario; 64 automatic stride bins would have grouped it with other cases.

The covergroup `cg_2d_4kb` below implements the bins of plan row `DMA-F06a` with the sampling rule this section has just prescribed: it latches the other-channel flag for the whole duration of a 2D row and samples once, when that row completes.

```
typedef enum logic [1:0] {STRIDE_POS, STRIDE_ZERO, STRIDE_NEG} stride_sign_e;

typedef enum logic [1:0] {DIST_GT_BURST, DIST_EQ_BURST,
                          DIST_LT_BURST, DIST_STRADDLE} boundary_dist_e;

typedef struct packed {
    stride_sign_e stride;      // sign of the 2D row's stride
    boundary_dist_e boundary;  // row start to the next 4 KB frontier, in bursts
    logic [8:0] beats;        // beats the row emitted, a count and not an
                                // encoded length: the legalized range is 1..256
} dma_row_t;

module dma_row_cov (
    input logic clk,
    input logic rst_n,
    input logic row_start,      // the midend accepted a new 2D row
    input logic row_done,      // the row's last beat was accepted
    input dma_row_t row_attr,   // row attributes, computed by the monitor
    input logic other_ch_active // the other channel has a burst in flight
);

covergroup cg_2d_4kb with function sample(dma_row_t row, logic other_ch);
    option.per_instance = 1;

    cp_stride_sign : coverpoint row.stride {
        bins pos = {STRIDE_POS};
        bins zero = {STRIDE_ZERO};
        bins neg = {STRIDE_NEG};
    }

    cp_boundary_dist : coverpoint row.boundary {
        bins gt_burst = {DIST_GT_BURST};
        bins eq_burst = {DIST_EQ_BURST};
        bins lt_burst = {DIST_LT_BURST};
        bins straddle = {DIST_STRADDLE}; // the row crosses a 4 KB frontier
    }
endmodule
```

```

// Only the extremes carry behavior; the values between them are the
// steady state and no bin claims them.
cp_len : coverpoint row.beats {
    bins one      = {1};
    bins max_m1   = {255};
    bins max      = {256};
    // The legalized range is 1..256 in a 9-bit field, so the midend must
    // never emit anything else: a value here is a bug, not a hole.
    illegal_bins out_of_range = {0, [257:511]};
}

// One bin per value of a one-bit flag: automatic bins are meaningful.
cp_other_ch : coverpoint other_ch;

x_worst : cross cp_stride_sign, cp_boundary_dist, cp_len, cp_other_ch {
    // The generator's solver budget caps a row at 256 beats, so a
    // straddling row splits into pieces and none of them can be the
    // maximum: 3 stride signs x 2 other-channel states = 6 cells, dead
    // by a recorded generation choice and not by geometry. The waiver
    // belongs on plan row DMA-F06a and expires when the budget moves.
    ignore_bins solver_budget = binsof(cp_boundary_dist.straddle) &&
                                binsof(cp_len.max);
}
endgroup

cg_2d_4kb cov = new();

// What the model needs is whether the other channel contended at any
// point of this row, so the flag is latched across the row instead of
// being read at the sampling edge.
logic other_ch_seen;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)          other_ch_seen <= 1'b0;
    else if (row_start)   other_ch_seen <= other_ch_active;
    else if (other_ch_active) other_ch_seen <= 1'b1;
end

// One sample per row, at completion, when all four attributes are
// jointly valid.
always @(posedge clk) begin
    if (rst_n && row_done) cov.sample(row_attr, other_ch_seen);
end
endmodule

```

6.6 Worked example: a coverage model for the neural accelerator

The accelerator (Chapter 1) is a fixed-point convolution engine with configurable 2/4/8-bit weights and a 16-lane MAC datapath; weights stream from memory and registers program jobs. The vplan feature: *quantized convolution jobs execute correctly for all weight precisions and tensor geometries, including under memory contention with the DMA engine.*

Semantic description: *record every completed convolution job as the combination of weight precision, tensor width class, input channel class and whether the DMA contended for the weight-stream SRAM banks during execution.* All four attributes correlate at one instant, job completion (Table 6.1):

Table 6.1 — The four attributes of the accelerator’s job coverage model, with where each value is read from and when it is captured. Three are read off the job registers at launch and the contention flag is latched across the weight-streaming window, so that all four are jointly valid at a single sampling instant, job completion — the choice §6.5 calls the one that determines meaning.

Attribute	Source	Sampled
weight precision (2/4/8-bit)	job registers	at job launch
tensor width (1–256)	job registers	at job launch
input channels (1–512)	job registers	at job launch
DMA contention	SRAM arbiter	latched during weight streaming

Detailed design, in SystemVerilog:

```
typedef enum logic [1:0] {PREC_2B, PREC_4B, PREC_8B} prec_e;

typedef struct packed {
    prec_e      prec;      // weight precision
    logic [8:0] tensor_w;  // tensor width in pixels, 1..256
    logic [9:0] in_ch;     // input channels, 1..512
} job_attr_t;

covergroup cg_accel_job with function sample(job_attr_t job,
                                             logic dma_contended);

    option.per_instance = 1;

    // Enum: automatic bins give one named bin per precision -- acceptable
    // here because every enum value is individually meaningful.
    cp_prec : coverpoint job.prec;

    // Width bins follow the datapath, not the number line: behavior bends
    // at the 16-lane boundary and at the extremes.
    cp_width : coverpoint job.tensor_w {
        bins single      = {1};           // degenerate geometry
        bins sub_lane    = {[2:15]};      // partial lane utilization
        bins lane_exact  = {16};          // exactly one full MAC pass
        bins lane_plus1  = {17};          // full pass + 1-pixel remainder
        bins multi_lane  = {[18:255]};    // steady-state streaming
        bins max_width   = {256};          // architectural maximum
        // Legal range is 1..256 in a 9-bit field: the register checker must
        // reject the rest, so reaching the datapath is a bug, not a hole.
        illegal_bins out_of_range = {0, [257:511]};
    }

    cp_ch : coverpoint job.in_ch {
        bins one_ch      = {1};
        bins few_ch      = {[2:15]};
        bins accum_16    = {16};           // accumulator reuse boundary
        bins many_ch     = {[17:511]};
        bins max_ch      = {512};
        // Legal range is 1..512; same contract as cp_width above.
        illegal_bins out_of_range = {0, [513:1023]};
    }
}
```

```

}

cp_dma : coverpoint dma_contended {
  bins quiet    = {1'b0};
  bins contended = {1'b1};
}

// The cross that would have caught the opening scenario:
// precision x width-class x contention = 3 x 6 x 2 = 36 combinations.
x_prec_width_dma : cross cp_prec, cp_width, cp_dma;

// Channels interact with precision (accumulator packing), not with
// contention: a second, smaller cross instead of one giant one.
x_prec_ch : cross cp_prec, cp_ch;
endgroup

```

The width bins encode a claim: the datapath treats widths 18 and 200 equivalently, both as steady-state multi-lane streaming, but treats 16 and 17 differently; the reviewer can agree or produce a counterexample, a case from the specification that the claim does not cover. The `illegal_bins` encode a different claim: those values must never reach the datapath, so if one does, the simulation stops rather than counting it (Section 6.9). Illegal values are excluded from coverage [6], so `cp_width` still offers 6 bins and `cp_ch` 5. The crosses make a second claim: precision interacts with geometry and contention because both affect weight-unpacker timing, while channel count interacts only with precision. This replaces one cross of $3 \times 6 \times 5 \times 2 = 180$ combinations with $36 (3 \times 6 \times 2) + 15 (3 \times 5) = 51$ justified combinations. The sampling encodes a third: `dma_contended` is latched across the whole weight-streaming window and sampled once, at job completion, when all four attributes are jointly valid. The opening bug occupies `x_prec_width_dma: (PREC_2B, single, contended)`, which this model requires for full coverage.

A counterexample covers the same feature with an inadequate model:

```

covergroup cg_accel_job_vanity with function sample(job_attr_t job,
                                                    logic dma_contended);

  cp_prec  : coverpoint job.prec;
  cp_width : coverpoint job.tensor_w {
    bins lo_half = {[1:128]};
    bins hi_half = {[129:256]};
  }
  cp_dma   : coverpoint dma_contended;
  // no cross
endgroup

```

Because this covergroup keeps the same attributes, the same sampling call and the same sampling instant as the model above, the only things that differ are the two modeling defects. This covergroup compiles and runs, reaching 100% early in the first regression despite misleading measurements. `lo_half` groups degenerate width 1, lane boundary 16 and remainder case 17 into one 128-value bin, hiding bug-exposing scenarios among harmless neighbors [2]. And there is no cross, so “2-bit weights” and “contention” are each ticked without ever having occurred *together*.

A third defect requires no change to the bins: the original model's `sample()` call is replaced with `@(posedge clk iff job_done)`, and the covergroup then reads contention live at that edge. This measures whether the DMA happened to be busy when the job retired, regardless of whether it contended during streaming, and fills the bin either way. The report cannot distinguish these defects from the original model. Functional coverage is exactly as good as the model, which can only be judged by *reading it*; coverage-model review against the vplan and the checkers must therefore be a flow gate.

6.7 Shaping the crossbar coverage model

The crossbar has 5 managers (core-I, core-D, two DMA channels, debug) $\times$ 4 subordinates (SRAM, boot ROM, APB bridge, the accelerator). Route coverage requires 20 combinations for every manager to reach every subordinate. Protocol coverage adds burst type (FIXED, INCR, WRAP: 3), direction (2), a modest 6 burst-length buckets and 4 transfer sizes on the 64-bit data bus. The full cross has $5 \times 4 \times 3 \times 2 \times 6 \times 4 = 2,880$ combinations, or 46,080 with the 16 transaction IDs. Review must justify the combinations introduced by these axes, and many bins are *unreachable by construction*: the instruction port never writes, the boot ROM rejects writes, and the debug manager issues only single-beat INCR accesses.

Unreachable cross bins make 100% unattainable. Late bulk exclusions without per-bin reasons lose the audit trail, while accepting a persistent 61% dashboard can discourage review (Chapter 7 returns to this problem). A **shaped cross** explicitly defines reachable space in the model, with reasons for exclusions.

```
typedef struct packed {
    logic [3:0] mgr_id;    // 5 managers, IDs 0..4
    logic [1:0] sub_id;    // 4 subordinates
    logic [1:0] burst;     // AxBURST
    logic      is_write;   // direction
} xbar_txn_t;

covergroup cg_xbar_route with function sample(xbar_txn_t t);
    cp_mgr : coverpoint t.mgr_id {
        bins core_i = {0}; bins core_d = {1};
        bins dma[] = {[2:3]}; bins dbg = {4};
    }
    cp_sub : coverpoint t.sub_id {
        bins sram = {0}; bins rom = {1}; bins apb = {2}; bins accel = {3};
    }
    cp_burst : coverpoint t.burst {
        bins fixed = {2'b00}; bins incr = {2'b01}; bins wrap = {2'b10};
        // 2'b11 is reserved in AXI4: a manager must never issue it.
        illegal_bins reserved = {2'b11};
    }
    cp_dir : coverpoint t.is_write {
        bins rd = {1'b0}; bins wr = {1'b1};
    }

    x_route : cross cp_mgr, cp_sub, cp_burst, cp_dir {
        // Instruction port is read-only by construction.
        ignore_bins ifetch_wr = binsof(cp_mgr.core_i) && binsof(cp_dir.wr);
    }
endcovergroup
```



```

// Debug manager issues only INCR accesses.
ignore_bins dbg_burst = binsof(cp_mgr.dbg) &&
                      (binsof(cp_burst.fixed) || binsof(cp_burst.wrap));
// Writes to the boot ROM must be rejected: observing the error
// response is a required event, so it lives in its own
// error-response covergroup rather than as a hole in this cross.
ignore_bins rom_wr = binsof(cp_sub.rom) && binsof(cp_dir.wr);
}
endgroup

```

Each `ignore_bins` line ^[6] carries its justification as a comment, in spec vocabulary, reviewable in the same diff as the model. The model’s reserved `AxBURST` encoding is the protocol’s own: Arm IHI 0022H.c §A3.4.1 “Address structure”, Table A3-3 “AxBURST signal encoding”, page A3-50 ^[5]. The shaped cross makes a falsifiable claim about *intended* reachability: protocol checkers must flag a write from a read-only instruction port. The boot-ROM line separates an exclusion from a requirement: the stimulus combination is excluded from this cross, while the error response stays positively required as a coverage point elsewhere, and the requirement moves to the covergroup where it can be measured.

Unshaped, this cross is $5 \text{ managers} \times 4 \text{ subordinates} \times 3 \text{ burst types} \times 2 \text{ directions} = 120$ combinations. The exclusions remove 12 cells (`ifetch_wr` = $1 \times 4 \times 3 \times 1$), 16 (`dbg_burst` = $1 \times 4 \times 2 \times 2$) and 15 (`rom_wr` = $5 \times 1 \times 3 \times 1$), for a total of 43 cells counted separately. Two pairs overlap: `ifetch_wr` with `rom_wr` in 3 cells, `dbg_burst` with `rom_wr` in 2, while `ifetch_wr` and `dbg_burst` are disjoint. The union of the three sets is therefore 38. This leaves **82** justified combinations rather than 120, without expanding to the 2,880 of the full six-axis cross. The tractability criterion is whether a reviewer can understand the complete model. On the compute cluster’s 8-core interconnect, or on the mesh NoC, the on-chip packet network that takes the place of a shared bus once too many blocks must communicate, crosses likewise cover only the two- and three-way interactions identified by risk analysis; assertions address the rest.

On the GPU, a shader cluster’s space comprises *occupancy shapes*, beyond structural axes: thread groups in flight $\times$ divergence class $\times$ memory-access pattern, a cross small enough to enumerate fully. The divergence class of a thread group records how far its threads take different paths through the kernel. The workload, rather than the configuration, decides which cells are reachable: a kernel without branches stays in one divergence bin for its whole run, and no biasing of the generator can move it out of that bin. The model must therefore be crossed against workload class; a hole identifies a missing program shape that could produce the behavior, rather than a missed random choice, changing who is responsible for fixing it.

On a directory-based coherent hub, coverage concerns *protocol states*: combinations of MESI state (a cached line is Modified, Exclusive, Shared or Invalid), directory entry and outstanding snoop across several caches, too large to enumerate manually. A directory entry is the hub’s record of which caches hold a given line. A snoop is a request the hub sends to a cache about a line, and it is outstanding until the cache answers. Formal coverage models identify the combinations over which the invariant was proved and those excluded by assumptions ^[8], the subject of Chapter 16.

6.8 Closure: the workflow from holes to done

Coverage closure is reached when the entire modeled coverage space has been observed [9], with exclusions accounted for. Chapter 4 placed it on the lifecycle timeline; Figure 6.1 draws the loop itself. Traditionally, closure is manual: the team runs the regression across seeds, each seed fixing one run’s random draws, merges the coverage databases, analyzes the remaining holes, modifies constraints or writes directed tests with content and order laid down by hand, and reruns until the target is reached; the merge-and-analyze step alone consumes significant time and compute [10]. Coverage also accumulates across simulators, emulators and formal tools, and a standard API and interchange format, the Unified Coverage Interoperability Standard (UCIS), exists to keep it mergeable across dynamic and static tools and across a project’s lifetime [11] (Chapter 16 covers unified coverage in depth).

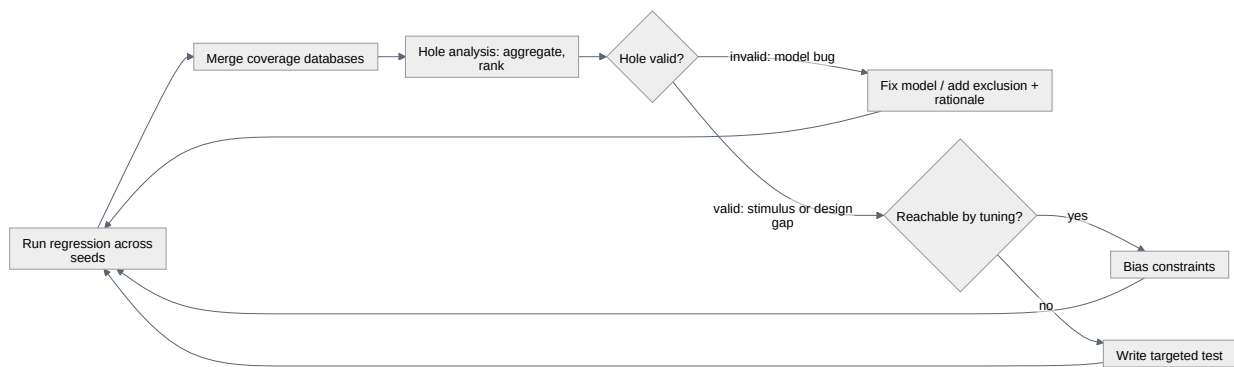


Figure 6.1 — The coverage-closure loop, from a seeded regression through merge and hole analysis to the action each hole earns. The branch that carries the discipline is the validity question: an invalid hole is a defect in the model, answered with an exclusion and its rationale — the path §6.9 exists to keep honest — while a valid hole is answered with biased constraints or a targeted test. Every branch returns to the regression, so nothing is closed without a rerun.

Reading holes. A coverage hole is required stimulus or device behavior not yet observed [2]. Hole-analysis techniques, first reported by IBM’s Haifa laboratory in 2002, reduce raw lists to interpretable regions: **aggregation** coalesces many point-holes into few larger regions; **partitioning** groups holes sharing semantics; **projection** finds attribute values never observed in any combination; and Hamming distance, the number of attribute values in which two holes differ, measures proximity [2]. Fifty scattered holes aggregated as “no 2-bit-weight job has ever run under contention, at any geometry” yield one finding with one root cause requiring one action. A projected hole takes priority over a single missing cross product because it identifies a whole missing stimulus dimension.

Classifying holes. Holes fall into two classes that determine the corrective action. A **valid hole** is an intended observation that has not happened: on the input side, usually a stimulus weakness; on the output or internal side, possibly a design error suppressing the behavior. An **invalid hole** is a bug in the coverage model itself: a combination that cannot or should not occur, included through misunderstanding; its fix is a restriction on the model, an exclusion with a reason, rather than a test [2]. Section 6.9 documents that second path. The Verification Academy publishes the

same triage for code coverage in three classes rather than two, splitting the valid hole by its cause: input stimulus that was never generated, a bug in the design or the testbench that blocks that stimulus from reaching the code, and code left unused in a given configuration and expected to be unreachable in normal operation. Only the third is the class it directs at exclusion [1].

Classification also decides what the stimulus must be *capable* of. On an inline AES-GCM crypto engine, a hole in the failed-authentication bin cannot be addressed by constraint tuning: the environment must be able to corrupt a tag after the engine computes it, truncate a ciphertext mid-stream, or present a key slot that differs from the one used to encrypt the message. A well-behaved driver has no path to perform these manipulations at any seed. Without these operations, the hole identifies missing testbench capability. Injected fault classes require their own coverage model, available for the sign-off review, the formal step that judges the evidence against the plan and accepts the risk that remains.

Filling holes through tuning or targeted tests. The first lever is **generation feedback**: bias constraint distributions so generator probability mass reaches missing observations; input holes respond directly; output and internal holes must first be traced back to the input scenario parameters that provoke them [2]. The second lever, a **targeted test**, encodes the missing scenario directly. Tuning is cheap per hole and preserves the randomness that finds unplanned bugs (Chapter 9); targeting is precise but brittle. Tuning suits families of holes; targeting suits isolated survivors. As only hard-to-reach points remain, progress decelerates sharply [4]; as of 2012, manually closing the last 20% costs up to 80% of a verification team’s time and resources [12].

That cost motivates closure automation research and products. The published approaches range from automated constraint regeneration, where uncovered bins are read back from the merged database and converted mechanically into constraints for the next run [10], to solver-level distribution, where the constraint solver of the constrained-random engine distributes solutions to satisfy the declared coverage model, converging measurably faster than undirected constrained-random generation [9]. Learning-based coverage-directed generation learns and inverts the mapping from generator directives to achieved coverage, using Bayesian networks, Markov models, evolutionary methods or inductive logic programming, which infers rules from examples [12]. One instance: a deep reinforcement-learning agent generating stimulus on the fly for a compression-encoder DUT hit 133 of 136 hard functional bins in 500 training episodes where uniform-random stimulus plateaued at 28 [13]. Chapter 25 examines when these techniques transfer beyond demonstrations; every one optimizes coverage closure, the derivative of bug removal [9]. Automation strengthens the need for Section 6.4’s checking review: machine-optimized 100% arrives faster, with even less human examination of scenarios.

An older learning-based example provides a more specific industrial attribution than the reinforcement-learning case. On the Storage Control Element of an IBM z-series system, a training set of 160 tests had left 223 coverage tasks uncovered; a Bayesian network trained on the correlation between test-generator directives and observed

coverage then synthesized new directive files and closed every one of them within 250 further test-cases, whereas an expert’s hand-written directive file reached only two-thirds of them in over 400 [14]. A different lever acts on the regression rather than on the generator: instead of biasing what is produced, learn which already-generated tests are worth simulating. The Radar Signal Processing Unit of Infineon’s AURIX TC3XX is a large, highly configurable block whose tests are described by 300 configuration fields; there, classifiers trained on previously simulated runs nearly closed the same coverage model while simulating 18% fewer tests [15].

Stopping criteria. Closure requires a full *reviewed* model, a current exclusion file and a stable period without model refinements. Both stimulus and model evolve during closure. **Coverage model feedback** is part of the method: models start exploratory, with coarse, low-fidelity coverage and goals around 80%, and are refined toward production models with goals of 95 to 100 percent as understanding matures [2]; discovering during closure that new attributes are needed is part of that refinement. Sustained 100% on a stable model, with active checkers, provides the closure evidence used in Chapter 7.

6.9 Exclusions as reviewable artifacts

The crossbar’s instruction-fetch port never issues a write, by construction of the port; the accelerator’s register file has reserved fields; parameterized RTL carries branches this configuration can never take. Unreviewable exclusions create risk, and they take recognizable forms: a bulk waiver at 11 pm before a milestone, an exclusion file never diffed, a tool-generated “excluded: unreachable” annotation that contains twelve merely difficult bins.

Four rules make exclusions reviewable:

1. **Every exclusion carries its reason, inline.** For example, “instruction port is read-only by construction; see integration spec section 3.2” explains the exclusion. Section 6.7’s shaped cross shows the pattern: the justification travels in the same artifact as the exclusion, so the review is a code review.
2. **Exclusions are classified by kind**, with different owners and expiry: *structural* exclusions tied to parameters expire on parameterization changes; *specification* exclusions of forbidden combinations expire on specification revisions; *risk-accepted* exclusions of reachable cases judged too costly never expire and therefore require the most senior signature. The third category makes explicit the risk acceptance hidden by bulk waivers.
3. **Exclusion and prohibition have different requirements.** Using `ignore_bins` for a forbidden combination instead of `illegal_bins` [6] both shrinks the measured space and removes error detection. Under Section 6.8’s invalid-hole rule, a model misunderstanding requires an explicit restriction [2]; a condition that the design must actively reject requires a checker as well.
4. **Exclusions are versioned and re-derived.** The exclusion file lives with the RTL and the vplan, is re-audited when either changes, and appears in the sign-off re-

view as a first-class document. For code coverage, formal unreachable analysis can *prove* many structural exclusions (see Chapter 15).

Published exclusion records illustrate these rules. The OpenHW Group’s uncovered-coverage table for a RISC-V core that it develops gives every entry a cause or explanation, an action, a future action against the RTL or against the tests, and a risk of leaving the item uncovered, a risk graded no, low, medium or high; the entries name the open issues they wait on, and one of them asks in writing whether a proof engine could settle the question instead ^[16]. The companion waiver list applies the same discipline across eight sheets of code-coverage and functional-coverage items, each carrying a written reason and, where one exists, the number of its tracking issue ^[17].

For any uncovered bin, the team should be able to produce within a minute either a coverage plan or a written, attributed reason that remains valid. The review must distinguish coverage supported by those records from 100% obtained by redefining the target.

IN PRACTICE

Code coverage should run from the first smoke test, with triage deferred until the design stabilizes because early holes change with it. Functional coverage models should be written alongside the testbench, because writing them after stimulus exists risks modeling only what those tests already do. They require formal review against the vplan and checker inventory: every coverpoint names its feature and the checker that catches misbehavior in it. During closure, the regression dashboard merges code, functional and assertion coverage across the farm using the cross-tool interchange format ^[11]; a weekly triage meeting should review aggregated holes rather than raw bins, assigning each family a verdict: tune, target, fix the model, or exclude with a reason. Skipping classification can leave an unexplained 4% at sign-off. Closure staffing must account for costly individual residual holes ^[12]. A hole that resists three targeted tests warrants investigation for a bug or specification misunderstanding. Exclusion files require contested decisions to be resolved in review; an unchallenged waiver does not demonstrate that it was read.

PITFALLS

1. **Treating 100% code coverage as done.** Symptom: green structural metrics presented as verification quality. Cure: explain code-coverage holes as a minimum requirement, then assess functional coverage against the vplan ^[4].
2. **Automatic bins on meaningful fields.** Symptom: bins named `auto[112:127]` in the report. Cure: deliberate bins named in spec vocabulary; auto bins only where every value is equally (un)interesting ^[6].
3. **Coverage without checkers.** Symptom: coverpoints sampling events no scoreboard or assertion observes. Cure: identify the checker that detects misbehavior for every sampled scenario ^[2, 9].

4. **The full cross.** Symptom: one cross with thousands of mostly-unreachable bins, stuck at 61% forever. Cure: shape crosses with justified inline `ignore_bins`, using several small, defensible crosses in place of one oversized cross.
5. **Sampling at the wrong moment.** Symptom: bins fill but targeted replays of “covered” scenarios find nothing resembling them. Cure: sample when attributes are jointly valid; latch condition flags across the window they describe.
6. **Bulk late waivers.** Symptom: coverage jumps 8% the week before sign-off on a single exclusion-file commit. Cure: classify and justify exclusions, assign owners and review them as code, leaving no unattributed waivers.

SUMMARY

- Code coverage measures the implementation, functional coverage the specification, and assertion coverage the checkers; each answers a different question and none substitutes for another [2].
- Automatic code coverage exposes incompleteness but cannot establish completion; unimplemented features are structurally invisible [4].
- A functional coverage model is an executable set of equivalence classes (Chapter 3): “100%” measures progress across that model and says nothing about its completeness or fidelity [2, 4].
- Coverage measures sampling, while checkers establish correctness: closure is a measurable derivative of bug removal and can be optimized without removing bugs, so checkers must connect it to that goal [2, 9].
- Models derive attributes from specifications, place bins at behavioral boundaries and shape crosses to reachable space, sampling jointly valid attributes [2].
- Closure cycles through merging, analysis, classification and tuning or targeting; the last 20% can consume up to 80% of effort, justifying automation and requiring effective checking [10, 12].
- Legitimate exclusions are reviewable artifacts with classifications, reasons, owners and versions; forbidden behavior still requires a checker [2, 6].

EXERCISES

Exercise 6.1 (calculation). Take the five-state machine of §6.2 with its transition count, together with the two percentages the same sentence reports for the single sequence run against it. Compute how many transitions that sequence traversed and check the count back against the percentage, showing the arithmetic. State the least number of arcs any single sequence visiting every state must traverse, and name the two things §6.2 says a tool reading this metric cannot decide on its own.

Exercise 6.2 (calculation). Take the tensor-width field of the accelerator’s job model in §6.6 with the width declared for it, and the default value of `auto_bin_max` given in §6.3. Compute how many values one automatic bin would hold, then give the range of the automatic bin containing the field’s architectural maximum, using the

bin name §6.3 quotes for the lane boundary to fix the alignment. Name what else falls inside that bin and the line of the covergroup in §6.6 that decides how the model must treat it.

Exercise 6.3 (judgement). A closure tool reads the uncovered bins back from a merged database, converts them mechanically into constraints, and brings the accelerator’s model from a plateau to full coverage overnight. Decide what that result establishes about the campaign and what it does not, and name the property of coverage closure §6.4 states that makes the distinction necessary. List the stopping criteria of §6.8 the result does not by itself satisfy, and give the reason §6.8 offers for tightening the checking review when closure is automated.

Exercise 6.4 (judgement). Two teams model the accelerator’s job feature with the same attributes, the same bins and the same crosses, and both report full coverage of the contention cross; one latches the contention flag across the weight-streaming window and samples at job completion, the other reads the flag live at the clock edge where the job retires. Decide what each report is evidence for, and name the design decision of §6.5 that separates them. State the conclusion §6.6 draws about coverage reports from the fact that the two look identical, and name the cell of the model that the chapter’s opening failure occupies.

Exercise 6.5 (artifact). Read the exclusion record below against the four rules of §6.9 and the shaped cross of §6.7. State which of the three `ignore_bins` lines of that cross covers this combination, if any, and what follows for the record’s stated reason. Name the two fields the record leaves unusable and the rules of §6.9 that require them. Give the classification the record must declare if the team’s real ground is a restriction in the sequence library, using the distinction §6.2 draws for the crossbar’s two stuck identifier bits.

```
Bin: x_route: (dbg, apb, incr, wr)
Status: excluded
Reason: not reachable
Kind:
Owner: unassigned
Expiry: none
Risk if left uncovered: low
```

FURTHER READING

Corpus sources. Piziali’s *Functional Verification Coverage Measurement and Analysis* [2] develops the taxonomy and model-design method, fidelity and hole analysis. Bergeron’s *Writing Testbenches using SystemVerilog* [4] concisely explains the scope and limits of 100% coverage. The VMM book [7] codifies the data-versus-activity split and the coverage/assertion guidelines. IEEE 1800-2023 Clause 19 [6] is definitive for covergroup, bin, cross and option semantics; the UCIS standard [11] for coverage interchange (with Chapter 16). On closure automation: a review of ML-based coverage-directed generation [12], automated constraint regeneration [10], solver-level coverage-driven distribution [9], deep-RL closure [13], and coverage models for formal

verification [8]. The Verification Academy's *Coverage Cookbook* [1] accompanies this chapter with the executable testplan format whose four required columns Chapter 5 uses, and its statement that deriving a functional coverage model requires interpreting the available specifications.

Non-corpus references (not citable above): O. Lachish, E. Marcus, S. Ur, A. Ziv, "Hole Analysis for Functional Coverage Data," DAC 2002, the original hole-analysis paper; H. Foster, A. Krolnik, D. Lacey, *Assertion-Based Design*, the companion discipline to Section 6.3's assertion coverage.

REFERENCES

1. **B7** Verification Methodology Team, "Coverage Cookbook," Mentor Graphics Verification Academy, 2013 (revised 26 April 2019).
2. **B5** A. Piziali, "Functional Verification Coverage Measurement and Analysis," Springer, 2004.
3. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
4. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
5. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
6. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
7. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
8. **D11** X. Feng, X. Chen, and A. Muchandikar, "Coverage Models for Formal Verification," Proc. DVCon U.S., 2017.
9. **D10** M. Teplitsky, A. Metodi, and R. Azaria, "Coverage Driven Distribution of Constrained Random Stimuli," Proc. DVCon U.S., 2015.
10. **D15** M. Kodi, S. S. Patil, and R. Nair, "Fully Automated Functional Coverage Closure," Proc. DVCon U.S., 2019.
11. **S3** Accellera Systems Initiative, "Unified Coverage Interoperability Standard (UCIS), Version 1.0," Accellera, June 2012.
12. **P19** C. Ioannides and K. I. Eder, "Coverage-Directed Test Generation Automated by Machine Learning – A Review," ACM Transactions on Design Automation of Electronic Systems, vol. 17, no. 1, Article 7, 2012.
13. **D1** E. Ohana, "Closing Functional Coverage With Deep Reinforcement Learning: A Compression Encoder Example," Proc. DVCon U.S., 2023.
14. **P16** S. Fine and A. Ziv, "Coverage Directed Test Generation for Functional Verification using Bayesian Networks," Proc. 40th Design Automation Conference (DAC '03), Anaheim, CA, pp. 286–291, June 2003.
15. **P4** N. Masamba, K. Eder, and T. Blackmore, "Supervised Learning for Coverage-Directed Test Selection in Simulation-Based Verification," arXiv preprint arXiv:2205.08524v3, October 2022.
16. **O6** OpenHW Group, "CV32E40Pv2 Uncovered Code Coverage Explanation, RTL v1.8.3," [openhwgroup/programs/Project-Descriptions-and-Plans/CV32E40Pv2/Milestone-data/RTL_v1.8.3/CV32E40Pv2_uncovered_coverage_explanation.xlsx](https://openhwgroup.org/programs/Project-Descriptions-and-Plans/CV32E40Pv2/Milestone-data/RTL_v1.8.3/CV32E40Pv2_uncovered_coverage_explanation.xlsx), commit f7968ac5a0e6438549057fe45311abf6244b9a3b, accessed 2026-09-02.

17. **05** OpenHW Group, "CV32E40Pv2 Waiver List, RTL v1.8.3," [openhwgroup/programs, Project-Descriptions-and-Plans/CV32E40Pv2/Milestone-data/RTL_v1.8.3/CV32E40Pv2_waiver_list.xlsx](https://github.com/openhwgroup/programs/tree/master/Project-Descriptions-and-Plans/CV32E40Pv2/Milestone-data/RTL_v1.8.3/CV32E40Pv2_waiver_list.xlsx), commit f7968ac5a0e6438549057fe45311abf6244b9a3b, accessed 2026-09-02.

7. Metrics-Driven Sign-off

Sign-off requires quantitative evidence for every block and an irreversible decision based on it. This chapter develops the instruments and their interpretation through the reference SoC's review, two weeks before tape-out and eleven months into verification. Intuition ranks the neural accelerator as risky because it is the newest block and its verification team the least experienced; it ranks the DMA as safe after three weeks without new failures and with an all-green regression dashboard. The metrics review says almost exactly the opposite. All the accelerator's plan rows are discharged but one, a contention cross that its block-level bench cannot reach. A plan row is discharged when the evidence that closes it exists and is linked to it, and a cross is a coverage item that counts the runs in which two or more conditions hold together. Coverage is otherwise closed, and bug discovery flattened *while* stimulus was still being added. The DMA's curve also flattened, but no new stimulus went in during the same three weeks; the last severe bug arrived late in an unmodeled corner, with coverage crosses around it at 71%. Intuition measured team seniority and recent silence, whereas the instruments correctly assessed the project's verification state.

CHAPTER MAP

§7.1 explains why SoC-scale verification needs shared metrics and what they cost. §7.2 covers per-feature status, coverage trends, bug curves, regression health and engineering time; §7.3 distinguishes clean designs from exhausted stimulus in flat bug curves. §7.4 aggregates plan and coverage evidence with static and formal results, bug dispositions and reproducibility. §7.5 applies the checklist under deadline pressure: identical flat curves lead to sign-off, conditional sign-off with a dated waiver, or a hold. §7.6 uses blameless escape analysis to update the next project's plan.

7.1 Metrics for shared project status

Chapter 4 introduced sign-off as the lifecycle's last gate: a review that judges evidence against the plan (see Chapter 4). This chapter examines the evidence's origins, misleading interpretations and use in decisions. **Metrics-driven verification (MDV)** manages the project on quantitative measurements collected continuously from the verification process, rather than on status estimates collected from people. MDV is presented as today's universally accepted baseline methodology for verification management ^[1]; the remaining choices concern what to measure and how to maintain reliable measurements.

Intuition works on small projects, where a few people can hold the entire design in their heads; on an SoC integrating many complex IP blocks, no single person has a

view spanning the project, and the intuition of one or even several people may stop portraying its real state ^[2]. The reference SoC has ten to fifteen engineers, each knowing one or two blocks deeply and the rest indirectly. The DMA's safety assessment failed structurally because it was made one step away from the work, not through carelessness. Large verification environments require measurements to establish project status ^[2].

First, **no single metric portrays the project**. Every measurement gives one view with significant limitations; only a wide range of metrics avoids a picture distorted by the drawbacks of any one of them, and that range has to be planned together and *correlated* during analysis ^[2]. Coverage percentage without the bug curve rewards stimulus that hits bins without checking behavior, since bins are the counters that a coverage model keeps over sets of values. A green pass rate without coverage rewards a regression that no longer challenges the design. A falling bug count without regression health may just mean the regression is broken. Sign-off therefore requires reading these instruments together.

Second, **metrics are expensive and must guide action**. Functional metrics must be architected, written and maintained like any other verification code, with both simulation and maintenance costs ^[2]. A coverage metric left unread after each regression is probably irrelevant, and reaching 100% on an irrelevant metric may look like progress without being productive work, because collected data alone does not provide useful information ^[3]. The **metrics gap** is a metrics explosion in which teams instrument for the sake of instrumenting and lose track of what matters and what does not ^[4]. A proposed metric warrants implementation only if a change in its value changes a decision.

Metrics serve to coordinate a distributed team. The team defines and agrees on **shared goals** when writing the plan, establishing what “done” means and tracking it automatically thereafter ^[5]. With an executable plan (see Chapter 5), each row links to the coverage points, checks and results that discharge it, and status is *computed* from that evidence; this makes the sign-off meeting of Section 7.5 a review of evidence rather than a negotiation between personalities.

7.2 Dashboard instruments

A verification management platform, whether a commercial tool or an in-house database, reports what has been verified, what remains and whether the process producing the evidence is healthy, at any moment without a meeting. The closed regression loop collects data on each iteration: plan, run, collect results, triage failures, merge coverage, analyze against the plan, and react ^[5]. A coverage model names the attributes and the values that a design must be exercised over, and calling one golden means a project keeps a single such model and takes it as the specification of what counts as covered. One team rebuilt the entire stack in-house around a single golden coverage model, treating the regression loop itself as the methodology rather than any one tool ^[1]. That team, Graphcore, measured the effect of moving its coverage model out of SystemVerilog into an in-house Python implementation: the move

meant that coverage could be collected in an offline generation step with no simulator at all; collection over the same 200 seeds fell from 40,493 s to 3,718 s, a 10.9× speedup; and its authors qualify both gains, because a model that is not timing-accurate can carry only coverage independent of cycle-accurate information [1]. Of the many quantities the loop *can* record [2], five families belong on the project dashboard.

1. Per-feature status. This is the primary instrument because it reports against the verification plan, which this book calls the vplan. For each vplan row, including the feature and its checks, coverage and closure evidence (see Chapter 5), the dashboard shows discharged / in progress / blocked / waived. Requirements traceability runs through this instrument: from specification requirement to plan row to coverage point to test result, so that a change in the specification automatically becomes visible as a hole in the evidence [5]. The other dashboard metrics are *leading indicators* that approximate per-feature status. The choice of which indicator best approximates per-feature status is a per-block judgment: on a flash controller, whose job is surviving corrupted input, that role falls to error-injection coverage. Error-injection coverage records which of the planned bit-error patterns, burst lengths and worn-block conditions the regression actually injected and checked against the data-integrity oracle, the mechanism that decides whether an observed response is correct. Coverage blind to the injection axis conveys nearly no information about that block.

2. Coverage trends. Coverage curves show percentage over time by block and category, merged across simulation, formal and any other engine that discharges plan items [5]. A single snapshot (“the DMA is at 96%”) is nearly information-free: the same number is alarming if it was 96% three weeks ago with stimulus running since, but encouraging if it was 78% last Monday. Plotting coverage over time reveals its trajectory [2]. Chapter 6 defines coverage requirements; this chapter examines their progress.

3. Bug curves. Bug-open and bug-closure rates over time are among the simplest trend reports [2]; Section 7.3 addresses their interpretation, including the misreadings discussed below.

4. Regression health. This includes pass rate over time by block and configuration, check-in-to-result turnaround, farm utilization, simulation time per block, and anomalies in these measures. A sudden runtime spike caused by a recent change is actionable that week; a month later, diagnosing the slowdown requires reconstructing the change [2]. Regression health also includes *test effectiveness*: ranking tests by coverage contributed and bugs found, so that the regression can be reordered or re-weighted instead of growing monotonically until it no longer fits in a night [2]. Late in a project, pruning *coverage collection* reduces the overhead of collection, and that overhead scales roughly linearly with the number of bins; crosses that are already fully hit go on incurring that overhead at every run; one team therefore cut a seeded regression from 46.6 to 38.2 hours of compute by dropping coverpoints already hit in the previous run, at the cost of slightly lower reported coverage on the pruned points [1]. Dropping those coverpoints is a bounded and deliberate loss of measurement in exchange for compute, and it belongs in the sign-off record as well as in the script that implements it. Chapter 12 treats the regression factory in depth.

5. Debug-time share. Engineering time complements the evidence metrics. In the 2024 industry data, verification engineers spend more of their time on debug than on any other activity, and debug effort is unpredictable and varies significantly between projects [6, 7]. The two earlier editions of the same study are held here as well, and they report the same ordering and attach the same warning about one practice: planning a project's effort and schedule from the previous project's data [8, 9]. That variance makes extrapolation from the last project's debug effort unreliable for estimation (see Chapter 4). Tracking where the hours go, even coarsely through triage annotations in the bug tracker, provides the one early-warning signal the other four instruments miss: a project's *evidence* metrics may look flat because the whole team is occupied with one pathological debug, which the time records distinguish from inactivity.

Scenario. Week 38 on the reference SoC. The DMA's coverage curve has been flat for three weeks and its bug curve flat for the same three weeks. Alongside the flat curves, instrument 5 records the block's two engineers logging those weeks almost entirely against triage and debug of one severe find from week 35: a transfer aborted mid-descriptor and then resumed, which the burst splitter restarts from a stale pointer while the other of the DMA's two channels is holding the backend FIFO that they share. The burst splitter is the DMA stage that splits each transfer into bursts that the AXI address rules allow. Each transfer is set going by a transfer descriptor, the record that software writes to program one DMA transfer. **Approach.** The lead interprets the flat curves as stimulus starvation because no new stimulus went in to exercise new conditions. **Why.** Metrics describe process activity, so their interpretation must account for the work performed. A flat week with a two-person team debugging one bug has zero evidentiary value about the rest of the block; only debug-time tracking distinguishes it from a quiet week on a clean design.

Two disciplines preserve the provenance of these measurements. **Only clean runs contribute:** failed simulations and stale builds cannot supply coverage evidence;

only error-free verification tasks qualify [3]. **Each data point requires versioned provenance:** which RTL revision, which testbench revision, which tool versions, which seed and which configuration produced it [2], because a trend drawn across incompatible baselines cannot support a valid comparison.

7.3 Reading bug-rate curves

The reported cumulative bug-discovery curve is S-shaped: slow during bring-up, the phase in which environment and design first exchange checked transactions, while the environment can barely exercise the design; steep as constrained-random stimulus and maturing checkers explore the design; then flattening as discoveries diminish. Constrained-random is the automatic generation of legal but unlikely stimulus under constraints. The curve's derivative, bugs found per week, rises to a peak and then decays. Verification managers use a flat discovery tail as a stopping signal because the classical ship criterion requires the RTL to pass all planned tests *and* the review to be satisfied with the coverage and bug-rate metrics [10].

The S-shape is received wisdom rather than a measurement: no bug-rate curve for a named project appears anywhere in this book's cited sources, so it describes what teams report seeing, without a fitted curve. Published points sit on the rising limb, never on the tail: a formal deployment on a live Intel graphics project caught around 15 bugs across designs in its first two weeks and 84 within eight weeks, on totals that the paper itself states differently in its abstract and in its conclusion [11].

The flat tail is therefore **ambiguous**. A bug curve records discoveries, and a discovery needs three things to happen at once: stimulus that reaches the buggy condition, a checker that catches the misbehavior, and an engineer who triages the failure into the tracker. The curve flattens when bugs run out or any one of those three steps stops. Two curves produce the same picture: one that is flat because the design is clean, and one that is flat because the instruments are blind. Progress metrics are snapshots and trends of what has happened, like stock-market indices or batting averages, and they cannot be used to predict the future [10]. Extrapolating the bug curve applies this limitation to predictions of remaining bugs.

Stimulus saturation causes this ambiguity. Coverage closure is the convergence of every item of the plan to covered or consciously excluded status. Constrained-random coverage closure is non-linear: it rises steeply at first and then exhibits diminishing returns later in the project [1]. The mechanism proposed here is that a fixed constraint set keeps re-sampling the same region of the state space: bugs reachable inside it are found in the steep phase, and after that more seeds mostly re-confirm what is known. Chapter 6's closure discipline addresses saturation; a saturated regression can produce a flat bug curve that is mistaken for evidence of cleanliness.

The curve's shape also depends on what the block is made of. An HBM controller barely draws an S: discovery is dominated by the PHY training state machine, and late finds cluster in retraining corners after a thermal event, after a refresh collision, or following an interrupted sequence. A flat tail there says nothing until the review establishes how many training entries the regression attempted in the flat window, because between training experiments the curve is necessarily flat.

A flat tail requires corroborating instruments, applying the correlation discipline of Section 7.1 ^[2] to one decision (Table 7.1):

Table 7.1 — Six questions to put to a bug curve that has gone flat, with the answer pattern each returns when the flatness is genuine and the pattern it returns when the flatness is an instrument failure instead. The two columns describe the same picture read two ways, which is why the curve is never read by itself.

Question to the flat curve	Flat because clean	Flat because blind
Was new stimulus added during the flat window — new sequences, new constraints, new <i>irritators</i> (background traffic generators whose only job is to perturb the DUT)?	Yes — and it found nothing	No — same tests, more seeds
Is functional coverage still moving?	Plateaued <i>at target</i> , holes dispositioned (see Chapter 6)	Plateaued <i>below</i> target, or bins nobody reviewed
What is the fresh-seed yield (failures per thousand new seeds)?	Measured, and ~zero	Not measured, or seeds re-explore the same space
Did checker strength change?	Checkers stable or strengthened	Checkers waived/disabled to “stabilize” regression
Is the triage queue empty?	Yes — failures were dispositioned	A backlog hides undiagnosed discoveries
Where were the <i>last</i> bugs found?	Scattered, low severity, in modeled corners	Clustered, severe, in a corner the plan never modeled

The last row explains the DMA hold in Section 7.5. Because bugs cluster, a severe bug found late in a corner that the coverage model did not describe is one *sample* from a region of the state space that the instruments were not watching. The response must treat the find as evidence about the model as well as the design: extend the coverage model and the stimulus around it, then require the curve to flatten again under the expanded model, rather than fixing the bug and declaring the tail restored. This reading is practitioner judgment: the corpus supports the underlying facts, since clustering of discoveries by stimulus category is the kind of correlation that a metrics database exists to expose ^[2]. The inference rule, however, is the author's.

One additional comparison is **bug-open vs bug-closure** [2]. The gap between them is the debug backlog, and its trend informs scheduling: a discovery curve that flattens while the open-bug count stays high means finding has paused while fixing catches up. The two lines must converge to close that backlog; parallel plateaus are insufficient.

7.4 The sign-off checklist: aggregating the evidence

Chapter 4 defines sign-off as the final lifecycle review in which people accept residual risk on the record. The checklist makes **evidence aggregation** explicit: every claim that verification is complete must trace to an artifact that a skeptical reviewer could independently reproduce and use to assess the claimed completion. Its shape varies by team; its content, at minimum, does not:

1. **Every plan row discharged or dispositioned.** Each feature row of the vplan (see Chapter 5) points to the coverage, checks and results that discharge it, or carries a written disposition explaining why it will not. With an executable plan and requirements traceability, a database query establishes this status [5]. An untraceable plan row remains undischarged regardless of recollections.
2. **Coverage closed or waived, with recorded waivers.** Sign-off records functional coverage at target and analyzed code coverage under Chapter 6's closure discipline. Every remaining hole carries a **waiver** with five fields: *the hole*, *the reason*, *the risk argument*, *an owner*, *an expiry*. Chapter 6's three exclusion kinds (structural, specification-driven, risk-accepted) gain one sign-off rule: a risk-accepted waiver used to meet a deadline requires a real expiry date and senior signature because it is not a don't-care. Only a genuine don't-care writes "none" there: a hole that the project will never need to revisit. A waiver with an empty expiry is permanent by accident.
3. **Static verification clean.** Lint, CDC and RDC sign-off requires review of their waiver files (see Chapter 13). Static results remain valid only if their waivers were justified.
4. **Formal results with reviewed assumptions.** Proofs discharge plan rows only with their assumptions and constraints; an over-constrained full proof leaves behavior unexamined (see Chapter 14). The review examines the assume list as well as proof status.
5. **Open-bug disposition.** Every open entry in the tracker is classified. Severities on this project read S1 (blocks tape-out), S2 (fix, or waive with an argument) and S3 (documentation or cosmetic); Chapter 12 covers triage. Three dispositions are available: *fix before tape-out*, *waive as known errata* (with the software work-around named and the documentation owner assigned), or *defer* (with the argument why it cannot manifest in this product's use cases). An open bug with no disposition remains unevaluated, and recording that disposition is what the ship criterion of Section 7.3 calls *satisfaction with the bug-rate metrics*, made explicit in the record.

6. **Regression health over a sustained window.** All planned tests must pass across seeds and configurations throughout a *window*; a single green run cannot establish sustained health. The evidence must be audited: only error-free runs on the tagged release candidate contribute [3].
7. **Reproducibility.** Every number on the dashboard must be traceable to RTL revision, testbench revision, tool versions, configuration and seed [2], which makes any result regenerable on demand. Escape analysis (Section 7.6) needs that record to establish exactly what ran.

These inputs converge on one review; Figure 7.1 traces each to the recorded decision.

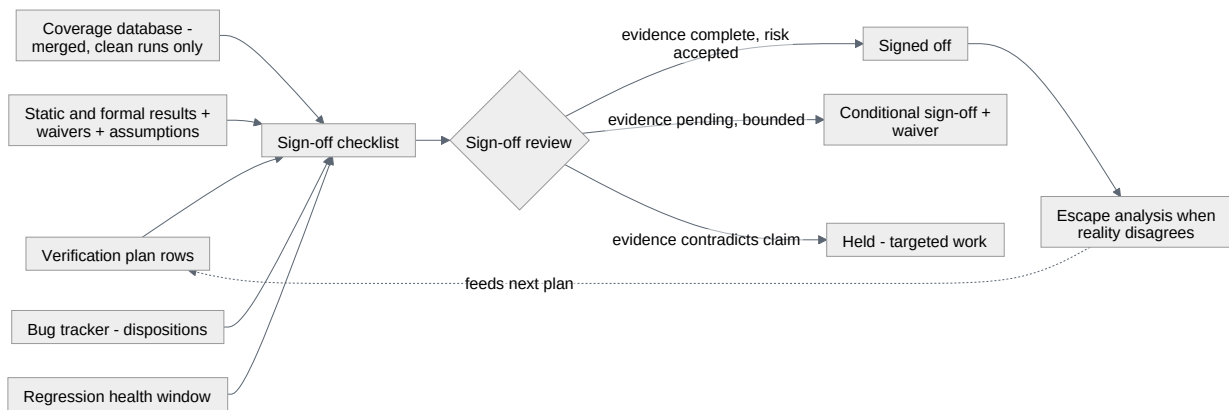


Figure 7.1 — How the sign-off checklist aggregates its evidence and what the review is then allowed to decide. Each input on the left is one checklist item reduced to the artifact a skeptic could re-derive it from; the review has three outcomes rather than two, and the middle one is a decision taken now against named evidence arriving by a named date. The dashed edge is what makes sign-off part of a loop rather than an endpoint: escape analysis returns to the plan (§7.6).

The checklist has two requirements. It is **written before the end**: the closure evidence per row was named at planning time (see Chapter 4), so the checklist is the plan’s last column, established before deadline pressure could affect its requirements. It is **decision-oriented**: each item resolves to signed off, conditionally signed off, or held, where *conditional* means the decision is taken now against named evidence arriving by a named date. A conditional sign-off requires a testable condition.

The evidence required depends on the block. A 10G Ethernet MAC with time-sensitive-networking queues signs off substantially on a compliance suite, an externally defined test set whose verdict is non-negotiable and comparable across vendors. The suite is insufficient alone because conformance says nothing about the block’s own queue-shaping configuration space. Such blocks carry two evidence columns: the suite’s verdict, and the coverage model measuring everything the suite never reaches.

Worked example: checklist excerpt (the crossbar, week 40).

#	Item	Evidence	Status
1	All 41 plan rows discharged	vplan report r2841, auto-linked results; features XB-01..XB-07, 7/7 closed	PASS
2	Coverage closed or waived, on two evidence lines	functional: merge of sim+formal, tag xbar_rc3, 100% of the model, 14 exclusions, all reviewed (true don't-cares, expiry "none"); code: 98.7% statement / 96.2% branch, residue = defensive decode defaults, dispositioned in review	PASS
3	Static clean	lint 0 issues; CDC report waiver file v7 re-reviewed	PASS
4	Formal protocol properties	41 proven, 3 bounded ≥ 40 cycles — bounded set reviewed and accepted as residual risk; assume list reviewed against integration constraints	PASS
5	Open bugs dispositioned	1 open S3 (documentation mismatch), waived as errata, doc owner assigned	PASS
6	Regression window	21 consecutive clean nightlies + 5 clean weekend fulls, ~9,400 seeds cumulative, tag xbar_rc3	PASS
7	Reproducibility	all results regenerable from tag + seed manifest	PASS

The evidence column expands Chapter 4's crossbar sign-off summary item by item: the same review and numbers in one artifact at two levels of detail. The write-inter-leaving bug that formal found in week 14 appears nowhere on the checklist because it was found, fixed and regressed; the ordering property that caught it now sits among the formal item's unbounded proofs as permanent evidence. That item's three *bounded* proofs hold only up to a depth, and the checklist requires the review to explicitly accept the residual risk beyond that depth. Sign-off certifies that bug-finding procedures reached agreed completion and residual risk was explicitly accepted; it guarantees neither a history without bugs nor exhaustive answers.

7.5 Worked example: the dashboard, two weeks before tape-out

At week 40, the reference SoC's sign-off review (see Chapter 4) covers three blocks: the crossbar, neural accelerator and DMA engine. The dashboard has one row per block and uses Section 7.2's instrument families, with debug-time share in the fresh-stimulus column and arrows showing three-week trends (Table 7.2):

Table 7.2 — The week-40 block dashboard that the sign-off review reads: one row per block, one column per instrument family, arrows giving the three-week trend. The bug-discovery column is flat for all three blocks and the review still reaches a different verdict on each, so the columns that do the separating are the plan rows, the direction of the coverage, and whether fresh stimulus went in at all.

Block	Plan rows	Func cov	Code cov (stmt)	Open bugs S1/S2/S3	7-day pass rate	Bug discovery (3 wk)	Fresh stimulus (3 wk)
the crossbar	41/41	100% → (closed-or-waived)	98.7% →	0 / 0 / 1	100% →	flat	yes — new irritator sequences, zero finds

Block	Plan rows	Func cov	Code cov (stmt)	Open bugs S1/S2/S3	7-day pass rate	Bug discovery (3 wk)	Fresh stimulus (3 wk)
the neural accelerator	37/38	98.9% ↑	97.8% ↑	0 / 0 / 2	99.8% ↑	flat	yes — quantization corners, zero finds
the DMA engine	44/47	96.0% →	98.2% →	0 / 1 / 3	99.2% →	flat	no — team in debug (Section 7.2 scenario)

The pass-rate column is a seven-day snapshot; each block’s checklist uses a longer window, including 21 nightlies for the crossbar. The review assesses each block against its plan.

The crossbar: signed off. The checklist excerpt above is its evidence, and the flat discovery curve is corroborated against Section 7.3’s table: fresh stimulus went in and found nothing, coverage at target, triage queue empty, last finds months old and scattered. The plan named closure evidence per row in week one, and formal took the protocol rows as far as they go, with three of its properties holding only up to a proof bound and that limitation recorded. After eleven minutes, the review records sign-off in the record of the meeting, with the S3 errata disposition attached.

The neural accelerator: conditional sign-off, with a waiver. The accelerator’s unit of work is one convolution, programmed through its registers, and this book calls it a *convolution job*. One plan row is not discharged: the cross of 2-bit weight mode $\times$ concurrent DMA traffic on the shared SRAM banks. The block-level bench cannot generate real DMA contention; the SoC-level environment can, and that regression is still accumulating the cross. Other evidence supports cleanliness: coverage rises toward target while discovery stays flat under continuing quantization-corner stimulus. Both dispositioned bugs are S3: a performance-counter off-by-one on job restart, waived as errata with a software workaround, and a register-description mismatch. The 99.8% pass rate requires review because checklist item 6 asks for planned tests passing *repeatedly*: the 0.2% is a known testbench race on job restart, filed against the environment rather than the design. The review confirms that disposition instead of dismissing the number as noise. The review decides now rather than deferring, and grants conditional sign-off on a waiver that fills all five fields. The hole is the undischarged cross. The reason is that no block-level bench can generate real contention. The risk argument is that the 2-bit datapath is identical to the verified 4-bit path except for the unpack stage, and that contention exercises arbitration already proven on the crossbar. The owner is the SoC-level verification lead. The expiry is week 41: the cross closes in SoC regression by then, or the block returns to review.

The DMA engine: held. Per-feature status prevents sign-off. Three of forty-seven plan rows are undischarged, the worst status among these blocks; checklist item 1 alone therefore holds the DMA. All three were opened in week 35 in reaction to the data-corruption find described below, and none carries evidence yet.

The DMA reports 96% coverage, a 99.2% pass rate, one open S2 with a fix under review, and flat discovery. But the corroboration of Section 7.3’s table fails on four of

six axes. No new stimulus entered the entire trend window because the two engineers were debugging one bug from week 35; the regression has repeated the same stimulus since. Fresh-seed yield is therefore unmeasured. Coverage is plateaued *below* target, and in the region of the last find it sits at 71%.

In week 35, fresh seeds in the SoC-level regression exposed data corruption in a transfer aborted mid-descriptor and then resumed: the burst splitter restarts the interrupted 2D row from a pointer that it never invalidated, and re-issues bursts that it had already committed. A 2D row is the contiguous run of bytes that makes up one line of a two-dimensional transfer. The corruption is reproducible only when the other channel is holding the backend FIFO at the instant of the abort. Abort-and-resume had no plan row, coverpoint or cross because every one of the burst splitter's plan rows assumes a transfer running to completion. The DMA's plan rows for splitting a 2D transfer cover a different case. One of those crosses demands a transfer with a negative stride, split wrongly at the 4 KB boundary, that corrupts data only while channel 1 is simultaneously active. A negative stride is an address step that walks memory downwards, and the 4 KB boundary is the address line that AXI forbids a burst to cross: every burst stays inside one 4 KB address region, so it never spans two subordinates. The scoreboard is the testbench structure that holds what the design should have produced and compares it with what the design did. Constrained-random stimulus plus the scoreboard reaches such a wrongly split transfer where directed splitter tests never would, and the plan says in advance where to look. That failure is SOC-1284, the escape of Chapter 1, and it arrives after this review. The week-35 find lacked such a cross: the green dashboard described coverage around an unseen region, and 71% cannot measure the hole itself because it has no bins.

A late, severe find in an unmodeled corner samples an unwatched region (Section 7.3), so the review treats this descriptor failure as evidence of inadequate instruments. The block is held two weeks, with work aimed at the blind spot beyond the individual bug. The three open rows must be discharged and widened into a family for abort and resume as a *class*, and that family needs three additions: a cross of abort point $\times$ transfer geometry $\times$ other-channel state; constrained-random abort injection while both channels contend for the backend; and a checker on the violated invariant, which is that a resumed transfer issues exactly the bytes that the aborted one had not yet committed. The discovery curve must flatten under the *new* instruments, with their expanded stimulus and checking, before the checklist is reviewed again to assess whether the block now meets its requirements.

The review records three decisions, each with named residual risk, based on evidence rather than speaker seniority.

7.6 Escape analysis and the next verification plan

An **escape** is a bug found beyond a boundary that was supposed to catch it: a block bug found at SoC level, an RTL bug found in gate-level simulation, or a silicon bug found in the lab. Escapes are widespread across the discipline. In the 2024 industry data, only 14% of IC/ASIC projects achieved first-silicon success, the lowest value re-

corded in twenty years [6], and 87% of FPGA projects reported non-trivial bugs escaping into production [12]. Since complete prevention is unavailable (Chapter 3), an escape provides a costly, targeted audit of the verification process.

Escape analysis turns that audit into a structured post-mortem. For each escape, the team answers five questions, in order, in writing:

1. **Bug mechanism.** The analysis records the precise mechanism at bug-report level, beyond the symptom shown by a failing test.
2. **Stimulus gap.** The analysis identifies the condition never generated and whether it reflects a constraint gap, missing sequence or unplanned scenario.
3. **Checker or platform gap.** The analysis distinguishes *reached but unchecked* behavior (a checker gap) from behavior *unreachable at this abstraction* (a platform gap), and the two gaps require different corrective work.
4. **Misleading completion metrics.** The analysis identifies the dashboard that showed completion despite the hole: missing coverage bins, an outdated waiver, or a starved bug curve interpreted as clean.
5. **Changes to planning.** New plan rows and coverage crosses must address the *class* of bug, never only the instance. A checker or platform is added and a sign-off checklist item amended. Each change, including the new plan rows and coverage crosses, has an owner and is filed into the next project's planning inputs (see Chapter 5).

The analysis must be **blameless**, examining the process rather than assigning personal blame. Career consequences can encourage participants to distort post-mortem reports, undermining the metrics used to assess the project. Fear-driven verification and lack of introspection contribute to the gap between required and actual verification; teams can address the self-induced part by examining their own contribution [4]. Escape analysis institutionalizes that introspection. In question 4, the analysis should allow an engineer to report that metrics were correct but the plan omitted the case, without having to defend herself.

Scenario: escape post-mortem, week 44. The event unit's retention registers lose configuration on power-domain re-entry when a wake event races isolation release. Retention here means enhanced functionality on selected sequential elements, so that their values survive the power-down of their primary supply. A power domain is a group of instances treated together for power management, typically sharing one supply, and power-domain re-entry is the moment that supply comes back on. Isolation is the defined behavior of a signal while its driving logic is unpowered, and isolation release is the moment that behavior ends. The escape was found in UPF-aware gate-level simulation *after* RTL sign-off (see Chapter 19); UPF is the Unified Power Format, IEEE Std 1801-2024, *IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems* [13]. **Approach.** The five questions are answered in writing: (1) Mechanism: a wake event arriving in the same cycle as isolation release corrupts the restore path of the retention flops. (2) Stimulus: RTL-level power tests stepped through save/sleep/restore sequences with directed timing; nothing ever randomized wake-event arrival against

the power-sequence FSM, so the race was never generated. (3) Checkers: the RTL power tests checked configuration *after* restore completed, leaving the race window itself unobserved. Plain RTL simulation does not model loss of state in the switched domain at all, so part of the behavior was unrepresentable below power-aware simulation. The unobserved race window and the unrepresentable state loss leave both a platform gap and a checker gap. (4) Metrics: the power-management plan rows were discharged by those directed tests, the ones written out by hand to do the same thing on every run; the coverage model had no cross of wake-timing $\times$ power-state transition, so 100% of what was measured showed completion despite the plan's omission of the race. (5) Changes: a new plan-row family "retention behavior under randomized wake timing", the missing cross added to the coverage model, power-aware simulation pulled earlier in the schedule instead of arriving with gate-level simulation, and a new sign-off checklist item requiring *power-intent behavior verified on a power-aware platform* rather than inferred from RTL, with the owner of the low-power methodology named. **Why.** The engineer who wrote the directed tests executed the plan she was given; the plan could not see the race. Blameless analysis turns one late, expensive discovery into four permanent process improvements.

Escape analysis closes the sign-off loop by updating the checklist of Section 7.4: instead of remaining static, that document accumulates findings from every post-mortem conducted by the organization. Checklist items such as "retention verified on power-aware platform" and "fresh-seed yield reported for any block whose curve is flat" record earlier escapes, placing their corrective requirements in the review to prevent recurrence. The results become new rows in the two artifacts every future project uses, the plan and the checklist, so lessons enter the required work rather than remaining in a separate lessons-learned document.

IN PRACTICE

A hybrid infrastructure combines a vendor verification-management platform for plan-to-result traceability and multi-engine coverage merge [5] with in-house scripts and databases. Some build the entire MDV stack in-house for flexibility and engine independence, at a real cost: they now own that infrastructure and must train engineers who have never seen it [1]. A standing weekly metrics review should keep dashboards in active use, preventing a month of neglect in which their data goes unexamined.

Tying metrics to dates or bonuses creates incentives to add easy coverage bins or "temporarily" exclude failing tests; either change can make the dashboard misleading. Fear-driven and ulterior-motive-driven verification are listed alongside design complexity as causes of verification gaps, the shortfalls between what a project must verify and what it can [4]. An exclusion justified in March can be stale in September, so sign-off re-reviews waiver files rather than counting them. Scheduling coverage closure as linear disregards its diminishing-returns tail [1, 4]. Of Section 7.2's five instruments, debug-time share warrants explicit tracking: debug dominates verification time and varies unpredictably between projects [6], so it is most likely to explain

schedule consumption. That last step is our inference; the corpus supplies the dominance and the variance, nothing about who tracks it. MDV should collect fewer metrics and act on all of them to obtain value [2, 3].

PITFALLS

1. **Unverified dashboard status.** *Symptom:* green process indicators conceal design failures because status is inferred from color without examining provenance. *Cure:* every green status must trace to a reproducible artifact that sign-off can review.
2. **Reading a flat bug curve as clean.** *Symptom:* “no new bugs in three weeks” offered as closure evidence by itself. *Cure:* review fresh stimulus, target coverage, fresh-seed yield, checker integrity, triage backlog and the location of the last finds.
3. **Turning a metric into a target.** *Symptom:* coverage percentage tied to a milestone date; bins multiply, hard crosses vanish, the number climbs while knowledge does not. A milestone is a phase-gating event that a project has to demonstrate it has reached rather than estimate. *Cure:* targets live on plan rows and dispositions; percentages are instruments, and instruments are audited (see Chapter 6).
4. **Waivers without owner or expiry.** *Symptom:* a continually growing waiver file contains reasons the reviewers cannot explain. *Cure:* every waiver retains five fields (hole, reason, risk argument, owner, expiry); only a true don’t-care may use “none” for expiry, and sign-off re-reviews every waiver.
5. **Counting dirty runs.** *Symptom:* coverage merged from failing simulations or mixed RTL revisions inflates closure. *Cure:* only error-free runs on the tagged baseline contribute [3]; provenance is part of the number.
6. **Blame in post-mortems.** *Symptom:* defensive escape analyses allow the same escape class to recur next project. *Cure:* blameless analysis studies the process and must produce changed plan and checklist rows.

SUMMARY

- On an SoC, individual intuition has a structural limit: no person spans the project, and unmeasured status remains unknown [2]. Metrics establish shared status if the team limits their number, correlates them and acts on them.
- Five instruments track per-feature plan status (the primary measure), coverage *trends*, bug curves, regression health and debug-time share. No single one of them is trustworthy alone [2].
- A flat bug curve is ambiguous because clean designs and stimulus starvation produce the same shape. Its interpretation requires fresh-stimulus data, coverage position, seed yield, checker integrity and the location of the last finds.
- A late, severe bug in an unmodeled corner samples an unwatched region. The instruments must expand, with flat discovery demonstrated again under them.

- Sign-off aggregates discharged plan rows, closed-or-waived coverage, clean static and formal results with assumptions reviewed, bug dispositions, and results reproducible from tagged baselines. The outcome is signed off, conditional (a waiver with all five fields: hole, reason, risk argument, owner, expiry), or held.
- The 2024 industry study, still the latest published at the time of writing, reports widespread escapes: 14% first-silicon success and 87% of FPGA projects with non-trivial production escapes [6, 12]. The same study measured first-silicon success at about thirty-two percent in 2020 and twenty-four percent in 2022, and the 2022 edition already called that twenty-four percent the lowest in twenty years [8, 9]. The second of those readings is lower than the first, and the first-silicon figure quoted just above is lower still, so the series records successive lows rather than a contradiction. Blameless escape analysis answers five questions in writing and adds rows to the next plan and sign-off checklist.

EXERCISES

Exercise 7.1 (calculation). Take the two compute figures §7.2 reports for the coverage-pruning experiment, in which coverpoints already hit in the previous run were dropped. Compute the share of compute that the pruning saved. State the price the same sentence attaches to the saving, and state where §7.2 requires that price to be recorded.

Exercise 7.2 (calculation). Take the plan-row column of the week-40 dashboard in §7.5 for the neural accelerator and for the DMA engine. Compute, for each block, the share of its plan rows still undischarged. Decide whether those two shares are what separates the accelerator’s conditional sign-off from the DMA’s hold, and name the checklist item of §7.4 and the field of its waiver that separate them instead.

Exercise 7.3 (judgement). A block arrives at review with coverage at target, an empty triage queue, three weeks of flat bug discovery under fresh irritator stimulus that found nothing, and one severe find eight weeks earlier in a corner the coverage model never described. Decide whether §7.3 lets the review read that flat tail as evidence of cleanliness. Give the row of the chapter’s six-question table that decides it, and state the two changes the response must make before the tail counts as evidence again.

Exercise 7.4 (judgement). Under deadline pressure a team proposes a coverage waiver whose expiry field reads “none”, arguing that the reviewed exclusions in the §7.4 checklist excerpt carry that same value. Decide whether checklist item 2 admits the waiver, and give the property of those exclusions that the proposed waiver lacks. State what item 2 requires in the expiry field of a waiver taken to meet a date, and name the pitfall of the chapter that the proposal matches.

Exercise 7.5 (artifact). Read the escape post-mortem below, filed on the week-35 DMA find of §7.5 against the five questions of §7.6. One of the five answers is absent. Name the absent question, supply the answer §7.5 supports for it, and state what the post-mortem loses while it stays blank. State also the disposition §7.6 requires of every change listed under question 5.

Escape: block bug found at SoC level, DMA engine, week 35

Question 1: a transfer aborted mid-descriptor and then resumed; the burst splitter restarts the interrupted 2D row from a pointer it never invalidated and re-issues bursts it had already committed. Reproducible only while the other channel holds the backend FIFO at the instant of the abort.

Question 2: abort and resume was never generated. Every plan row on the burst splitter assumes a transfer running to completion, so no sequence ever set up an abort.

Question 3: the condition was reachable at this abstraction, since fresh seeds in the SoC-level regression reached it. The corrective work therefore adds a checker on the violated invariant, that a resumed transfer issues exactly the bytes the aborted one had not yet committed.

Question 4:

Question 5: three plan rows opened and widened into a family for abort and resume as a class: a cross of abort point $\times$ transfer geometry $\times$ other-channel state, and constrained-random abort injection while both channels contend for the backend.

FURTHER READING

From the corpus. [2] develops metrics beyond coverage for SoC verification, with categorization, run-time control and reporting; [1] provides a modern industrial case of an in-house MDV stack with a single golden coverage model, coverage pruning and regression diffing. [5] shows verification-management automation: traceability, multi-engine merge, and the regression round trip. [3] (Chapter 6, “Coverage-Driven Verification”) covers metrics for effort allocation, irrelevant metrics and clean-run discipline; [10] states the classical ship criterion and limits of trend metrics. [4] is a short taxonomy of verification shortfalls, including the metrics gap and self-induced gap. The industry effectiveness data cited throughout is from three editions of the Wilson Research Group study, the most recent being the 2024 pair [6, 12], and their methodological ancestor, Foster’s DAC 2015 trend study [7]. The two earlier editions of the Wilson series are held here as well and are short: IC/ASIC [8, 9] and FPGA [14, 15]. Piziali’s coverage taxonomy [16] underpins the measurement side of this chapter and is treated in depth in Chapter 6.

Not in the corpus. Carter & Hemmady, *Metric Driven Design Verification: An Engineer’s and Executive’s Guide to First Pass Success* (Springer, 2007), is a book-length treatment of metrics-driven processes and one of the two works that [2] distinguishes itself from; it is cited as background only.

REFERENCES

1. **D22** S. Hristozkov, J. Pallister, and R. Porter, “No Country For Old Men – A Modern Take on Metrics Driven Verification,” Proc. DVCon Europe, 2021.
2. **D20** A. Meyer and H. Foster, “Metrics in SoC Verification: Not just for coverage anymore,” Proc. DVCon U.S., 2012.
3. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, “Verification Methodology Manual for SystemVerilog,” Springer, 2006.

4. **D16** R. Narayan and T. Symons, "I created the Verification Gap," Proc. DVCon U.S., 2015.
5. **D26** D. Zhang, "Smarter Verification Management with vManager Platform," Proc. DVCon China, 2021.
6. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
7. **P17** H. D. Foster, "Trends in Functional Verification: A 2014 Industry Study," Proc. 52nd Design Automation Conference (DAC '15), San Francisco, CA, 2015.
8. **R3** H. D. Foster, "2020 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2020.
9. **R5** H. D. Foster, "2022 Wilson Research Group IC/ASIC Functional Verification Trends," Siemens EDA white paper, 2022.
10. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
11. **D18** M. V. Achutha Kiran Kumar, E. Seligman, A. Gupta, Ss Bindumadhava, and A. Bharadwaj, "Making Formal Property Verification Mainstream: An Intel Graphics Experience," Proc. DVCon U.S., 2017.
12. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
13. **S10** IEEE, "IEEE Std 1801-2024, IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems (Revision of IEEE Std 1801-2018)," IEEE, 2024.
14. **R4** H. D. Foster, "2020 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2020.
15. **R6** H. D. Foster, "2022 Wilson Research Group FPGA Functional Verification Trends," Siemens EDA white paper, 2022.
16. **B5** A. Piziali, "Functional Verification Coverage Measurement and Analysis," Springer, 2004.

PART III

Dynamic Verification

The simulation core: testbench architecture, stimulus, UVM as a methodology rather than an API, assertions, and the engineering of regression.

8. Testbench Architecture

Testbench architecture separates five responsibilities so that an environment can be debugged and reused with confidence in its checks. The reference SoC's crossbar testbench illustrates the problem: at six weeks of bring-up, one file contains 2,400 lines in a single module. It drives all five manager ports, samples the four subordinate ports, computes what the SRAM should return, and collects coverage in the same forever loop, over the same variables. It works and has found four bugs, but two problems arise in the same week.

A read-data mismatch at 41 μ s has no clear explanation. The crossbar may have returned two responses in an order the protocol permits; the checker may have assumed an order the protocol never promised; or the driver's back-pressure model may have drifted out of step with what it told the checker to expect. All three hypotheses live in the same two hundred lines and share state, so no experiment separates them. After two days in a waveform viewer, the ticket closes as "testbench issue" without an identified cause.

The DMA team also needs to reuse the AXI checking against its backend. Reuse fails because the checker reads driver-written variables: it knows only the intended stimulus, never what appeared on the wires, so it cannot provide an independent oracle, the mechanism that decides whether an observed response is correct (Chapter 3).

Both problems have one architectural cause rather than a coding error: the responsibilities share one program.

CHAPTER MAP

§8.1 separates the five responsibilities of stimulus generation, driving, monitoring, checking and coverage, explaining what fails when they merge. §8.2 develops signal, transaction and sequence layering with vendor-neutral transaction-level communication. §8.3 compares three scoreboard patterns (in-order, multiple in-order streams and unordered matching keys) and their checking costs under reordering. §8.4 covers when to build a reference model, its three species and their maintenance costs. §8.5 explains why monitors never drive and how passive instantiation enables system-level reuse; §8.6 and §8.7 build the crossbar and DMA environments step by step; §8.8 covers environment reviews, configurability and bring-up.

8.1 Five separate responsibilities

A testbench need not be a monolithic block ^[1]. The design under verification is not implemented as one unit, and its environment need not be either. The decomposition separates five responsibilities by their functions and the information they must not share (Table 8.1):

Table 8.1 — The five responsibilities a testbench separates, each with the question it answers and the thing it must not know. The third column carries the argument: every entry in it is an ignorance imposed on purpose, and merging two of these boxes does not merely blur the environment’s structure, it gives one box knowledge that the table forbids, for instance a checker that knows which test is running.

Responsibility	Answers	Must not know
Stimulus generation	<i>What</i> scenario should happen next?	How the protocol wiggles pins
Driving	<i>How</i> is this transaction expressed on the interface?	What the correct response is
Monitoring	What <i>did</i> appear on the interface?	What was intended
Checking	Was it <i>right</i> ?	Which test is running
Coverage	What have we <i>exercised</i> ?	Whether the test passed

Functional verification is the establishment, before fabrication, that a design implements its specification. By 2001 the same principle was already the basis of a language designed for functional verification: that language had a data model, a test generator, a reference model, a checker and coverage metrics tied back to a plan; its checker was split further into a data checker and a temporal checker running concurrently with the design; and its organizing principle was separation of concerns rather than a coding convention retrofitted later ^[2]. Chapter 9 traces this history: the structure preceded the tooling.

Each responsibility earns its own box because each has a different failure mode and a different reason to change. Merging responsibilities removes specific capabilities:

Checking inside the driver. A driver knows its intended stimulus. If the checker lives there, it compares the design against the stimulus generator’s intentions rather than against the specification; and every bug in which the driver itself misbehaves becomes invisible, because both sides of the comparison move together. Checking inside the driver therefore embeds Chapter 3’s common-mode error in the architecture, the same specification misreading encoded in both the design and its oracle.

Monitoring inside the driver. Observation as a side effect of driving cannot watch an interface driven elsewhere. That prevents passive system-level reuse (Section 8.5), including debugging with the same environment watching real traffic while stimulus is disabled.

Coverage inside the checker. Coverage sampled at verdict computation covers only paths the checker reached. Missing observation then looks like missing stimulus, preventing an answer to Chapter 6’s question of what the tests never did.

On a finished project, the self-checking structure is the largest and most complex part of the environment. It is the part that cost the most to author and maintain; it is the part responsible for declaring functional correctness; and it embodies a duplication of the specified functionality of the design. Checking needs planning before bring-up. The verification plan must state which failure modes the checking detects and how it detects them, and that statement must be reviewed, so that a potential failure does not go undetected. The classical categories are data transformation, data ordering, protocol correctness, data losses and design state ^[1] (see Chapter 5).

Implementation-specific failures such as buffer overflows are best expressed as assertions directly in RTL; signal-level relationships belong in the interface declaration containing the signals. Higher-level failures involving data transformation, ordering or end-to-end computation are better detected behaviorally in the testbench, because property languages do not suit end-to-end response checking well ^[1] (Chapter 11). The split must be deliberate and documented, with exactly one owner per failure mode.

Scenario. The crossbar driver predicts the read-back value of each write and stores it in a local array. A review examines a mis-encoded `awsiz`. **Approach.** The driver only drives pins, a monitor reconstructs what the crossbar received, and the predictor uses the *monitored* request. **Why.** With the split, the mis-encoded `awsiz` produces a monitored transaction that differs from the intended one and the read-back check fails. Without it, the driver’s mistake propagates identically into both the design and the expectation, and the test passes.

8.2 Layering: signal, transaction, sequence

An environment spans three levels of abstraction.

At the **signal level** the environment works with pins, edges and handshakes. At the **transaction level**, a transaction is initiated by injecting something into the running simulation and terminated some time later by observing a result; functional verification works at the level of whole transactions rather than the design’s clock cycles ^[2]. At the **sequence level** the environment composes scenarios: ordered and constrained collections of transactions that express intent (“fill the write-data channel of manager 2 while the debug port reads the ROM”). Chapter 9 takes the sequence level as its subject.

The component that converts between the first two levels is the **transactor**, or bus-functional model: repetitive physical-level operations are encapsulated into tasks, and all the tasks for one physical interface or protocol are collected into one model ^[1]. Only the transactor accesses pins; components above it use transactions.

Incorrect pin-sampling time causes apparent design bugs during bring-up. A clocking block makes the sampling moment explicit: with the default `1step` input skew, the value sampled is always the signal's last value immediately before the clock edge [3], which is what a monitor wants and what a naive `@(posedge clk)` read does not reliably give.

Two shapes of communication

Two non-interchangeable communication shapes cover almost all component interactions.

Operational communication is point-to-point and blocking: a producer hands a transaction to one consumer, and back-pressure is meaningful. The sequencer is the component that holds the sequence requests waiting to run and chooses which one the driver receives each time the driver asks, so a sequencer that cannot hand its next item to a busy driver *should* wait. **Observational** communication broadcasts a monitor's observations without blocking or depending on listeners. Its standard form is an analysis port with a single `write()` function broadcasting a value to all subscribers [4]. The port may have zero, one, or many subscribers without affecting the producer's operation; however many are connected, the void `write()` call always completes in the same delta cycle, that is, without advancing simulation time [5].

Observers must never slow a monitor, because that would change measurement timing. Observers are pluggable: a coverage collector, protocol checker and scoreboard subscribe to one stream without structural changes for a fourth. Siemens EDA's Verification Academy publishes a taxonomy of what subscribes to an analysis port: comparators, coverage collectors, and metric analyzers, whose subject is non-functional behavior such as timing, performance and power rather than correctness [6].

Broadcast requires independent copies: each subscriber receives a handle to the *same* transaction object, so every subscriber must copy before operating on it [5]; the standard states normatively that a subscriber's `write()` implementation shall not modify the value passed to it (IEEE Std 1800.2-2020, *IEEE Standard for Universal Verification Methodology*, §12.2.10.2) [4]. A coverage collector that normalizes an address in place silently corrupts the scoreboard's expectation, and the result looks like a design bug.

Chapter 10 covers the library standardizing these connections; the pattern predates libraries and remains independent of them, with one publisher and many independent observers without back-pressure on observation.

Protocol layering

Traditionally a transactor is tied to one physical interface, and such a transactor prevents reuse: each environment needs its own model and its own abstraction level. Layered models follow the protocol's structure. The layers are usually obvious in communication protocols: a USB application layers into packets, transactions and transfers (*Universal Serial Bus 3.2 Specification*, Revision 1.1, USB 3.0 Promoter Group, June 2022) [7]; but they exist elsewhere too, and any system-level function

that can be decomposed into blocks gives a corresponding decomposition of its transactions [1].

A block-level transaction is usually different from a system-level transaction: an Ethernet frame for a MAC block becomes an IP packet for the system reassembling and routing fragments [1]. Layered transactors let both environments share the lower layers instead of forking them. What counts as a transaction at each layer stays a local decision. TLM is the passing of whole transactions between components over standard interface handles rather than through signals. TLM-2.0, part of IEEE 1666 and built on SystemC, supplies its interoperability layer [8], and in practice that interoperability has held only for memory-mapped bus protocols: for serial and interrupt-style interfaces such as SPI, I2C, CAN and UART, each modeling team defines a payload of its own [9]. SystemC is a modeling language whose models sit above the level of signals.

Scenario. The USB controller must pass a compliance suite defined outside the specification, in the USB SuperSpeed Compliance Methodology white paper, and the team also needs its own directed and random tests for the dual-role device/host switch. **Approach.** The harness has three layers: a signal-level transactor per port, a packet layer, and a transaction/transfer layer. The compliance suite drives the top layer; in-house sequences drive whichever layer they need. **Why.** The compliance suite is a fixed body of externally specified stimulus and checking; it is not an environment and does not know about the design's power-state corner cases. Layering lets both consume the same transactors, so the signal layer is written and debugged once.

A third benefit is retargeting. An emulator is a purpose-built machine with a compiler and transactors of its own, big enough to hold a whole design and observable through triggers and state saving. Compact transaction messages replace per-signal traffic to overcome the throughput limits of a simulator or an emulator, and they do so under two conditions: timed and untimed behavior stay separate, and the two sides synchronize on transactions and events rather than clocks [10]. Transactors enable this retargeting; an environment accessing signals from twenty places cannot be moved (see Chapter 17).

8.3 Scoreboard patterns

For some, *scoreboard* denotes the whole self-checking structure, including predictor, expected-data storage and comparison. This chapter uses the narrower definition: a **scoreboard is the data structure that holds data expected to be received**, filled by a predictor and consulted by the comparison function when the output monitor reports something. Data remaining at simulation end was lost in the design, which may or may not be an error [1].

A scoreboard is a data-structure problem decided by one operation, lookup: because the exact order in which outputs appear may be hard to predict, the structure must

be designed to minimize the look-up operation ^[11]. The following three patterns differ in their ordering assumptions.

In-order: one queue. If the design preserves the order of its input stream, store expectations in a queue and check each observed response against the front. A queue beats a dynamic array here because it performs the same adding or removing regardless of position, whereas an array shifts its whole content on every check ^[11]. Lookup is $O(1)$ and failure messages are unambiguous; reordering is reported as a data mismatch, which is correct only if the specification requires order.

A transfer descriptor is the record that programs one DMA transfer. The skeleton of that in-order pattern runs on the completions of one DMA channel's transfer descriptors: the predictor fills the queue, the observation consults its front, and Section 8.6 turns the single queue into one per stream key.

```
package dma_desc_sb_pkg;

typedef enum {XFER_OK, XFER_ERR} xfer_status_e;

typedef struct {
    int unsigned desc_id;    // index of the descriptor in the channel's list
    logic [31:0] bytes;      // bytes the descriptor moves, from the model
    xfer_status_e status;    // the completion status the model predicts
    time t_pred;            // when the programming writes were monitored
} dma_exp_t;

// One channel completes its descriptors in the order they were programmed,
// so the expected store is one queue and the lookup is its front. Two
// channels take one queue each, the second pattern of this section; the
// crossbar's three-coordinate key is Section 8.6.
class dma_ch_scoreboard;
    local dma_exp_t pending [$];
    int unsigned n_matched, n_orphan, n_mismatch;    // read by report()

    // Called by the predictor, which the register monitor feeds with the
    // programming writes it observed. Never called from a driver.
    function void predict(dma_exp_t e);
        e.t_pred = $time;
        pending.push_back(e);
    endfunction

    // Called by the completion monitor. Zero delay, so the checker can
    // never back-pressure the monitor.
    function void observe(int unsigned desc_id, logic [31:0] bytes,
                        xfer_status_e status);

        dma_exp_t head;
        if (pending.size() == 0) begin
            n_orphan++;                // a completion nobody predicted
            $error("dma_sb: orphan completion, descriptor %0d", desc_id);
            return;
        end
        head = pending.pop_front();
        // != and not !=: an x in an observed byte count must never score as
        // a match.
        if (head.desc_id != desc_id || head.bytes != bytes ||
            head.status != status) begin
            n_mismatch++;
            $error("dma_sb: mismatch: expected descriptor %0d, %0d bytes, %s, \
predicted at %0t; observed descriptor %0d, %0d bytes, %s",

```

```

        head.desc_id, head.bytes, head.status.name(), head.t_pred,
        desc_id, bytes, status.name());
    return;
end
n_matched++;
endfunction

// End of test: a descriptor that never completed leaves its expectation
// in the queue and produces no mismatch of its own.
function bit report();
    int unsigned n_left = unsigned'(pending.size());
    $display("dma_sb: matched=%0d orphan=%0d mismatch=%0d leftover=%0d",
            n_matched, n_orphan, n_mismatch, n_left);
    if (n_left != 0)
        $error("dma_sb: %0d descriptors never completed", n_left);
    return (n_orphan == 0) && (n_mismatch == 0) && (n_left == 0);
endfunction
endclass

endpackage

```

Multiple independent in-order streams: one queue per stream. Responses on several concurrent interfaces may be observed in any order while the sequence on each single interface stays in order; one queue per output interface holds that interface’s predictions in the expected order and lets observations be checked in any order across interfaces ^[11]. The pattern generalizes past physical interfaces to any *logical* stream the specification promises to keep ordered. On the crossbar, the guarantee AXI makes is per manager port, per direction and per transaction ID: responses carrying the same ID return to their manager in order, responses with different IDs may be reordered by the interconnect, and reads and writes carry independent ID spaces. The *AMBA® AXI and ACE Protocol Specification*, Arm IHI 0022H.c §A5.1 “AXI transaction identifiers” ^[12], states the ordering rule in one sentence: “All transactions with a given AXI ID value must remain ordered, but there is no restriction on the ordering of transactions with different ID values.” The stream key therefore needs all three coordinates, and Section 8.6 shows what goes wrong with any fewer.

Unordered: matching keys. If no per-stream ordering survives, the checker must find the matching expectation rather than pop it. An index avoids a slow full search: hashing the observed response’s own values produces a likely-unique key, which an associative array maps to the expected data’s location ^[1, 11].

Scenario. The Ethernet MAC’s time-sensitive networking (TSN) queues reorder by design: eight priority queues with a credit-based shaper mean a low-priority frame injected first can legitimately leave last. **Approach.** Key expectations by (destination port, traffic class, stream identifier), all three of which the egress monitor reads off the frame itself, and keep one in-order queue per key (within a traffic class the shaper does not reorder), then check the *relations* the specification actually guarantees (per-class order, no duplication, no loss) rather than the exact global departure sequence. **Why.** A global in-order scoreboard would reject the first legal reordering, while a fully unordered scoreboard would accept a real per-class

ordering bug. The key specifies which ordering is verified and therefore requires review.

The design’s ordering guarantees determine which of the three patterns applies. Table 8.2 sets them side by side, with the lookup each one forces.

Table 8.2 — Scoreboard structure as a function of the ordering the design actually guarantees: what the design preserves, the structure that matches it, the lookup that structure needs, and what the checker gives up in exchange. The last column is where the price is paid — each step down the table tolerates more of the reordering the specification permits, and checks less of it.

Design behavior	Structure	Lookup	Cost to the checker
Preserves order	One queue	Front only, $O(1)$	Cheapest; any reorder = failure
Per-stream order only	Queue per stream key	Front of one queue	Must <i>name</i> the stream key correctly
Reorders freely	Associative index on a content key	Hash, $O(1)$ amortized	Ordering is no longer checked at all

Tagging, losses and inexact answers

Three refinements address tagging, losses and inexact answers. **Data tagging** exploits designs that carry part of their input untouched to an output: once separate checking has established that the payload is not modified, the checker can encode the expected destination, position and transformation *in the payload*, so each output monitor decides correctness on its own. This is the simplest self-checking structure because independent monitors contain all checking intelligence, subject to two limits. Serial numbers should not be embedded in stimulus data, because a system-level environment will need those user fields for higher-level information [11]; and when the payload is too short to hold the tag, tagging must work in concert with a scoreboard, the tag serving as a fast key into it [1].

Losses. Some designs legitimately drop data, and reproducing the discard decision means reproducing the design’s resource state and timing. Usually, an aggregate check is preferable. On an *ordered* stream, the first two patterns only, a transaction ahead of the matched expectation is assumed properly dropped, and the assumption is confirmed against the design’s own drop-count statistics [11]; “ahead of” has no meaning in a content-keyed index. This replaces an intractable prediction with a cheap check of aggregate losses.

Inexact answers. A fixed-point datapath will not match a floating-point model bit for bit; the observed response is then compared against the predicted one within an acceptable margin of error or bit-error rate [11]. The radar front-end’s FFT chain is the canonical case; the plan must justify the margin.

8.4 Reference models: three species

Chapter 3 established that a true reference model is the most complete oracle and that it rarely exists. The alternatives are three species with different costs and one distinction between transfer functions and reference models.

A **transfer function** reproduces the data transformation the design performs, so that the environment can determine which output to expect; it uses the design's configuration descriptor to perform the same transformation, and it is normally used together with a scoreboard. A transfer function is not a reference model: it does not exist a priori, it is not golden by definition, and as simulations run it will contain as many errors as the design itself; because it predicts less accurately, the response monitor has to perform more intelligent checks to deal with the uncertainty, whereas with a true reference model checking is a plain observed-against-expected comparison ^[1].

Species 1: the algorithmic transfer function. This is written from the specification in the testbench language. Examples are the crossbar's address decoder and a model of the DMA's burst splitter, the stage that splits each transfer into bursts that the AXI address rules allow; that model is distinct from the byte-level descriptor model in Section 8.7. An algorithmic transfer function is cheap to build and change but never trusted absolutely: disagreement first requires identifying which of the two implementations misread the specification (Chapter 3's common-mode warning applies in full).

Species 2: the golden executable. The standard supplies the model, so checking compares pixels after decoding the conformance bitstream. The video codec's decoder is this case (see Chapter 3), while its encoder on the same die has no golden executable. The standard constrains the bitstream, not the choices that produced it, so two conformant encoders emit different bitstreams from the same picture and both are correct; there is no executable to compare against. Checking combines weaker claims: the output decodes, it decodes to a picture within the distortion target *the team itself* set (a quality goal of its own making, not a standard-mandated one), and it never pushes the level's buffer-occupancy constraints out of bounds. Each of those claims is checkable, and none of them is golden. One block thus has a bit-exact model in one direction and no model at all in the other, so golden status does not establish completeness.

Species 3: the co-simulated instruction-set simulator. For the reference SoC's core, the model runs alongside the RTL and the comparison happens at instruction retirement: an instruction manager processes instructions as the design retires them, and an ISA scoreboard performs a step-and-compare of PC, registers, CSRs and trap status (*The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, Version 20250508, ratified, RISC-V International) ^[13] against the golden state ^[14]. Two structural requirements support this comparison.

First, reference models need not be cycle-accurate specifications of the hardware, only functionally accurate [2]. Requiring functional rather than cycle accuracy makes a software model fast enough to co-simulate, and it dictates where comparisons may occur: at an architectural boundary such as retirement, never cycle by cycle.

Second, asynchronous events must reach both sides to avoid failures resembling design bugs. A RISC-V environment doing this correctly has its interrupt agent drive random external, timer and software interrupts to the core and *simultaneously notify the reference model of the same interrupts*, so design and model operate on a synchronized view of external events [14]. Without notification, a random interrupt can create an environment-induced divergence; in this example, diagnosis takes a day. The same rule governs any responder whose behavior the model must know about, and error injection, the deliberate driving of erroneous stimulus to check that the design detects the error and recovers from it.

The OpenHW Group's CV32E40Pv2 is a 32-bit in-order RISC-V core, and its environment runs the same program on the RTL and on the reference model, streams the core's internal state and its asynchronous events over an interface named RVVITRACE, and compares state as each instruction retires, flagging a mismatch the moment it appears rather than at the end of the run; the core originated as RI5CY at ETH Zurich, Dolphin Design continued its development, and Imperas supplied the reference model and the coverage model [15].

Where concurrent integration is too hard, both sides can dump outputs for post-processing comparison [1], incurring Chapter 3's detection distance: the verdict follows simulation end, far from the causal state.

The maintenance burden

Every species is a second implementation of the specification and requires maintenance as long as the design lives: every specification change becomes two edits, two reviews and one decision about which side was right. The plan must budget this maintenance (Chapter 5) before it emerges in month four. Without a model, expectations are hard-coded per test; hard-coding needs a known configuration and a known input stream, so it works only for directed testcases; and because the response checked is crafted for that testcase, an unexpected corner that some other test creates by accident goes undetected [1].

Scenario. The UART needs no reference model under Chapter 3's risk allocation, which assigned it a thin directed suite. **Approach.** Oracle architecture leaves that allocation unchanged. **Why.** Models suit stateful, configurable or arithmetic transformations whose expectations would otherwise require manual derivation in every test. A small, silicon-proven block with a trivial transformation is none of those.

8.5 Monitors and passive checking

A monitor never drives stimulus.

Signal direction alone is insufficient: a stimulus task that checks feedback signals is still stimulus, and a monitor that must return flow-control is still a monitor. The distinction is *who initiates*: if a task initiates the transaction under full control of the testbench, it is a stimulus generator; if it waits for a transaction initiated by the design, it is a response monitor ^[1]. A subordinate-side responder answering requests is therefore an agent's driver.

A late response task leaves the testbench unready: internal back-pressure prevents full-throughput verification, output data “spills” and leaves stream gaps, or missing feedback signals cause an output protocol violation attributable entirely to the testbench. The structural answer is the **autonomous monitor**: an independent thread started at construction that continuously watches the interface and pushes extracted data into a FIFO that the rest of the environment drains independently; the arrangement decouples the timing of the transaction from the timing of the checking ^[1].

```
// Passive by construction, but only through the modport: an inputs-only
// clocking block constrains vif.cb.*, and nothing else.
interface axi_mon_if (input logic clk, input logic rst_n);
    logic        awvalid, awready;
    logic [31:0] awaddr;
    logic [3:0]   awid;
    clocking cb @(posedge clk);
    input awvalid, awready, awaddr, awid;
endclocking
// Only the clocking block and reset are reachable through this modport.
modport MON (clocking cb, input rst_n);
endinterface
```

A virtual interface handle is the reference through which a class component reaches an interface's signals. A component holding a plain `virtual axi_mon_if` handle reaches every member of the interface, and `awvalid`, `awready`, `awaddr` and `awid` are ordinary `logic` variables: `vif.awvalid = 1'b1`; compiles and drives. A clocking block on its own constrains `vif.cb.*` and nothing else. A *clocking modport* restricts access; the standard's example, `modport STB (clocking sb);`, is a synchronous testbench modport ^[3]. A monitor handed a `virtual axi_mon_if.MON` cannot assign to the bus at all: the compiler rejects the access, so no code review has to catch it.

Active and passive instantiation

Chapter 10 defines the agent and its active/passive switch, which enables reuse of a block environment at system level. A multi-level RISC-V environment illustrates this: the vector unit's environment actively emulates the scalar core at block level, and the *same* environment is then instantiated in passive mode inside the core-level environment once the real core drives the interface ^[14]. Nothing is rewritten: the stimulus half is switched off and the observation half keeps working. Without that separation, the environment is rebuilt at every integration level; and if that rebuilding is omitted, system verification runs with no checking at all.

Protocol monitors and functional monitors

Two passive monitor roles have separate responsibilities. A **protocol monitor** checks interface traffic legality: handshake stability, response-code legality, no burst crossing a 4 KB boundary, no reserved encodings (Arm IHI 0022H.c §A3.2.1 “Handshake process”, §A3.4.1 “Address structure” and its Table A3-3 “AxBURST signal encoding”) [12]. Its natural implementation is assertions attached to the interface, since signal-level relationships belong in the interface declaration that contains the signals [1] (Chapter 11 covers how they are written and bound). It knows nothing about intent and fires on the offending cycle. A **functional monitor** reconstructs and publishes transactions from pins without judging them.

A monitor that infers behavior it cannot observe stops being a monitor and becomes a second implementation of the design.

This separation also applies across different interface types, which suggests an architectural principle beyond digital design. A wire UVC is a verification component, a reusable testbench-side model of one physical interface with a driver to stimulate and a monitor to observe. Verilog-AMS is the analog and mixed-signal extension of the hardware description language. A mixed-signal block taken from a current LSI Corporation design is an analog driver with filtering and fault detection, and its environment was built as wire UVCs: their driver and monitor functions are written not in the testbench language but in Verilog-AMS; and the components are wrapped in the same self-checking sequences, functional coverage and regression control as a digital agent would be [16].

Scenario. In the flash controller’s low-density parity-check (LDPC) ECC pipeline, a first environment makes its NAND-side monitor recompute the ECC codeword to report “correctable” or “uncorrectable” per page. **Approach.** A predictor across the analysis port performs the recomputation; the monitor reports the raw codeword, the page’s status bits and the design-published syndrome, the check value a decoder derives from a codeword. **Why.** This addresses two problems: debugging cannot distinguish faulty monitor arithmetic from a design bug when the error originates in an unreviewed component; and reusing the design team’s LDPC routine under schedule pressure creates a common-mode check that proves nothing (Chapter 3).

An Ethernet MAC monitor timestamping frames on the wrong clocking edge reports a constant one-cycle offset on every TSN measurement. Every test fails despite a correct design; the cause is the sampling boundary in Section 8.2.

8.6 Worked example: the crossbar environment, step by step

The crossbar is a full AXI4 interconnect: five managers (instruction fetch, data, two DMA channels, debug) by four subordinates (SRAM, boot ROM, APB bridge, neural accelerator), 32-bit address, 64-bit data, ID width 4. The following steps establish the construction order.

Step 0: the checking contract precedes code. Four verdicts, each with a named owner: data integrity per ordered stream and completeness (nothing lost, nothing invented) belong to Step 4’s scoreboard; routing correctness to Step 3’s forward-path check; protocol legality on the nine interfaces to Step 2’s checkers. And three it will *not* produce: relative ordering between different transaction IDs, arbitration fairness, latency bounds. Each verdict needs a named owner and mechanism; documenting the excluded half of the contract prevents reviewers from assuming checks that do not exist (Chapter 3).

Step 1: one agent per interface. Nine agents, each owning exactly one interface [5]. Five manager-side agents are active: sequences feed a driver, a monitor watches. Four subordinate-side agents are responders; by Section 8.5’s criterion they are drivers, since their reply is under testbench control: an SRAM model with programmable latency and back-pressure, a ROM responder that returns an error response to every write, an APB-bridge responder with long variable latency, and an accelerator responder that can stall indefinitely. All nine carry a monitor, including the interfaces the environment itself drives.

Step 2: monitors publish to subscribers. Each monitor broadcasts the transactions it reconstructs to three kinds of subscriber. The first is the per-interface protocol checker. The second is the coverage collector, which samples the shaped route cross `cg_xbar_route` of Chapter 6; the model, its `ignore_bins` and their justifications belong to that chapter, and this environment only supplies the stream. An `ignore_bins` declaration excludes a bin from a coverage model’s score because covering it is not that model’s job. The third is the checking path, which takes two connections on the manager side (requests to the predictor, responses to the scoreboard) and one on the subordinate side (the forward-path comparison). None of those three subscribers can block the monitor, and none may modify the transaction it receives [4]. Figure 8.1 shows the resulting structure.

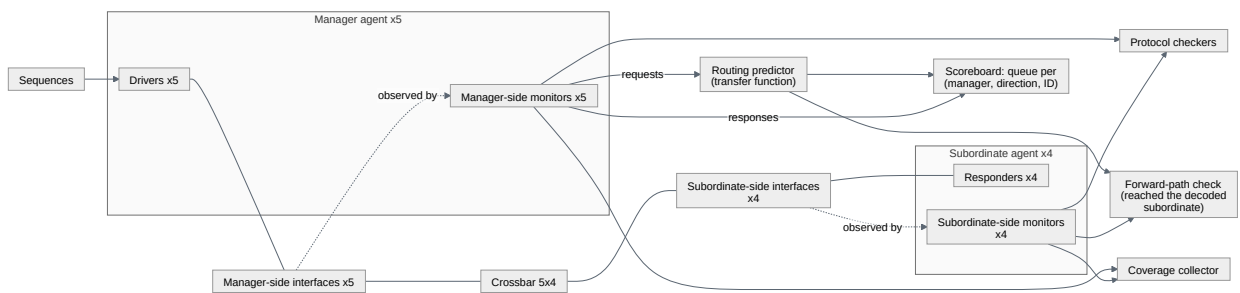


Figure 8.1 — The crossbar environment. Stimulus reaches the design through drivers and interfaces; checking is fed only by monitors, so no checker can observe a value the design never drove. The routing predictor supplies both the scoreboard, which matches responses per `(manager, direction, ID)`, and the forward-path check, which confirms the request reached the decoded subordinate.

Step 3: the routing predictor feeds two paths. The predictor is a transfer function, not a reference model (Section 8.4): from a *monitored* manager-side request it decodes the destination using the runtime configuration descriptor [1]; the address map is data the test can change, never a constant in the checker. It emits one expectation record holding the originating manager, the decoded subordinate, the dir-

action, the manager-side ID, the address and the expected beats. The predictor is not golden: if its map and the RTL parameterization disagree, every region-boundary access produces a mismatch, and the first response is a specification review rather than checker debugging (Chapter 3).

The record feeds two comparisons, one for each crossbar path. The **forward-path check** compares it with the subordinate-side request monitor to establish routing correctness: the address and write data reached only the decoded subordinate. The **return-path check** is Step 4's scoreboard, and it observes on the *manager* side, because read data flows subordinate → crossbar → manager. Comparing against a subordinate-side response monitor checks the environment's own responders and cannot establish whether subordinate 0's read data reached manager 2 rather than manager 3 in ID order.

Step 4: the out-of-order scoreboard. A key needs one dimension per independent source of concurrency, as Section 8.7 also shows for the flagship's eight-channel DMA. Here there are three: five managers issuing at once, two independent ID spaces, and AXI's per-ID ordering within one port and one direction (Arm IHI 0022H.c §A5.1 "AXI transaction identifiers", §A5.2.1 "Read data ordering") [12]. The stream key is the triple (manager port, direction, manager-side ID), one in-order queue per stream [11].

The two-coordinate key (subordinate, ID) fails all three requirements. Manager 1 and manager 2 may both have ID 3 outstanding to subordinate 0, and Chapter 10 deliberately sets that situation up with a virtual sequence, a sequence that coordinates several interfaces at once. One queue then matches one manager's expectation against the other's response. Only some arbitrations produce that pairing, so the fault surfaces as a mismatch that disappears on a seed change rather than as a reproducible bug. The two-coordinate key also merges `ARID` and `AWID`, which name unrelated streams, so a read expectation can be popped by a write response. An interconnect merging several managers must also *widen* subordinate-side IDs to avoid falsely serializing different managers' same-ID transactions. Five managers need $\lceil \log_2 5 \rceil = 3$ manager-identifying bits. Those three bits are added to the 4-bit ID a manager issues, so the same transaction arrives at a subordinate port with a 7-bit ID. Keying the scoreboard there is sound only with that widened ID, and it obliges the predictor to reproduce the design's ID-remap policy exactly. Manager-side keying avoids modeling that policy and keeps the key computable from what the observer sees.

```
typedef enum bit {RD, WR} xbar_dir_e;

typedef struct {
    int unsigned mgr;           // originating manager port, 0..4
    int unsigned sub;          // destination decoded by the predictor, 0..3
    xbar_dir_e dir;            // read and write IDs are independent spaces
    logic [3:0] id;             // manager-side ID, as the manager issued it
    logic [31:0] addr;
    logic [63:0] beats [$];    // expected response data, one entry per beat
    time t_pred;               // when the request was monitored
    int unsigned capture;      // monotonic index, for the failure message
}
```



```

} xbar_exp_t;

class xbar_scoreboard;
    // One expectation queue per (manager port, direction, manager-side ID).
    local xbar_exp_t pending [int][$];
    local int unsigned n_capture;
    int unsigned n_matched, n_orphan, n_mismatch;    // read by report()

    local function int stream_key(int unsigned mgr, xbar_dir_e dir,
                                  logic [3:0] id);
        return (int'(mgr) << 5) | (int'(dir) << 4) | int'(id);
    endfunction

    // Called by the routing predictor when a manager-side monitor
    // reports a legal request. Never called from a driver.
    function void predict(xbar_exp_t e);
        if ($isunknown(e.id)) begin                // never legal, and int'(id) would
            $error("xbar_sb: X/Z in a request ID, manager %0d", e.mgr);
            return;                                // silently file it under key 0
        end
        e.t_pred = $time;
        e.capture = n_capture++;
        pending[stream_key(e.mgr, e.dir, e.id)].push_back(e);
    endfunction

    // Called by a MANAGER-side monitor, so that what is checked is the
    // crossbar's return path and not the environment's own responder.
    // Zero-delay: it never blocks the monitor's thread.
    function void observe(input int unsigned mgr, input xbar_dir_e dir,
                          input logic [3:0] id, input logic [63:0] beats [$]);

        int k;
        xbar_exp_t head;
        if ($isunknown(id)) begin
            $error("xbar_sb: X/Z in a response ID, manager %0d", mgr);
            n_orphan++;
            return;
        end
        k = stream_key(mgr, dir, id);
        if (!pending.exists(k) || pending[k].size() == 0) begin
            n_orphan++;                            // a response nobody asked for
            $error("xbar_sb: orphan %s response, manager %0d ID %0d",
                  dir.name(), mgr, id);
            return;
        end
        head = pending[k].pop_front();             // one manager, direction and ID:
                                                    // these must return in order

        if (head.beats != beats) begin             // != : an x beat never reads as equal
            n_mismatch++;
            $error("xbar_sb: %s mismatch, manager %0d ID %0d: exp %p got %p; \
predicted for subordinate %0d at %0t, capture %0d",
                  dir.name(), mgr, id, head.beats, beats,
                  head.sub, head.t_pred, head.capture);
        end
        else n_matched++;
    endfunction

    // End of test: anything still queued was never delivered.
    function int unsigned leftover();
        int unsigned n = 0;
        foreach (pending[k]) n += unsigned'(pending[k].size());
        return n;
    endfunction

```



```

// A silent scoreboard is a broken scoreboard: all four counts printed,
// three of them failures.
function bit report();
  int unsigned n_left = leftover();
  $display("xbar_sb: matched=%0d orphan=%0d mismatch=%0d leftover=%0d",
          n_matched, n_orphan, n_mismatch, n_left);
  if (n_left != 0)
    $error("xbar_sb: %0d expectations never answered", n_left);
  return (n_orphan == 0) && (n_mismatch == 0) && (n_left == 0);
endfunction
endclass

```

Four details support the checking contract. `predict` is called by the *predictor*, which a monitor feeds, never by a driver (Section 8.1). Both entry points are zero-delay functions, so a checker can never back-pressure a monitor ^[11]. The comparison uses `!==`, not `!=`, because logical inequality on operands containing `x` or `z` yields `1'bx`, and an `if` statement reads that value as false: with `!=`, a single undriven beat scores as *matched*; the comparison could not detect the undriven beat. The `$isunknown` guards prevent `int'(id)`, a two-state cast, from collapsing an unknown ID to `0` and filing a corrupt transaction under a real key. A response that never arrives produces no mismatch, only a queue that never empties, so `leftover()` checks for this usually silent failure ^[1]. `report()` exposes these counters so their results can be inspected.

Step 5: limits of the environment. The canonical crossbar bug is write-data interleaving under back-pressure: beats belonging to two writes in flight to one subordinate appear interleaved on the wire, on a channel that AXI4 obliges the crossbar to keep in address order (Arm IHI 0022H.c §A5.2.2 “Write data ordering”) ^[12]; interleaving write data with different IDs is deprecated in AXI4 and later rather than forbidden, and address order is one burst at a time because there is no `WID`, the signal that would tag each write beat with the ID of its burst. The scoreboard cannot detect this because a tolerant subordinate still stores correct bytes and returns correct responses; the violation relates two transactions rather than either transaction’s data. It was found by protocol assertions in formal, with an 11-cycle counterexample (Chapter 14). A counterexample is the input sequence a proof engine returns to exhibit a failure. The scoreboard checks return-path data and completeness, the forward-path check owns routing, and the protocol monitor owns legality. Formal provides exhaustiveness for the legality properties that matter most; fairness and starvation remain outside all these checks, as Step 0 declares.

Step 6: diagnostic records. Every expectation records its originating manager, decoded subordinate, monotonic capture index and request-monitoring time, enabling messages such as “*RD mismatch, manager 2 ID 3 ... predicted for subordinate 0 at 41,320 ns, capture 8814*” instead of an unexplained difference between two byte queues. The four fields expose the provenance missing from the opening example, in which diagnosis took two days.

8.7 Worked example: the DMA environment and end-to-end checking

The DMA engine (two channels, 1D and 2D transfers, bursts up to 256 beats, a 16-entry $\times$ 64-bit datapath FIFO in the backend, per-channel error reporting) poses a different oracle problem. No single interface establishes transfer correctness; end-to-end checking must determine whether the descriptor produces the correct destination bytes.

A register agent programs the frontend (Chapter 10 covers register abstraction); a byte-accurate memory model behind the backend's AXI manager port serves reads and absorbs writes with randomized latency and back-pressure; a backend monitor publishes every burst; an error/interrupt monitor publishes completion and error events. The oracle is a **descriptor-level golden model**, a C model of the transfer semantics (source, destination, 1D/2D sizes, strides, element width) that is co-simulated through the language's foreign-interface layer and that turns the programmed descriptor into the expected destination memory image.

This model is golden within a narrow scope: data-mover transfer semantics *are* the specification, and the model implements them at byte-movement abstraction rather than burst abstraction. Leaving out the burst splitter's split decisions is what makes the model maintainable. Section 8.4's Species 1 model of the burst splitter predicts those split decisions and is a transfer function as buggy as the design it mirrors; the descriptor model predicts only byte destinations, and it stays golden because it excludes splitting. The two models for one IP differ in the abstraction level each commits to.

Both checking levels are necessary:

Level 1, end-to-end and late. At descriptor completion, checking compares the memory-model image with the golden image byte for byte, and it compares the reported per-channel status with the predicted status. That comparison catches the canonical DMA bug: the burst splitter mis-splits a negative-stride 2D transfer at the 4 KB boundary, and the mis-split corrupts data only while DMA channel 1 is also active. The malformed burst is masked unless channel 1 owns the shared AXI backend in the same window, because arbitration timing hides it otherwise. The stride is the signed address step from the start of one row to the start of the next. Every burst the backend issued was individually legal; only the bytes were wrong.

Level 2, per-burst and immediate. Protocol assertions are bound to the backend's AXI channels, among them the 4 KB-crossing property established in Chapter 2 and re-examined as a partial oracle in Chapter 3. Beside them sits the coverage model `cg_2d_4kb`, which records how the burst splitter treated each 2D row, the contiguous run of bytes that makes up one line of a two-dimensional transfer. Chapter 5's plan row DMA-F06a names that model, and Chapter 6 treats it. The per-burst assertions fire on the offending beat.

Both levels are necessary because detection distance has a cost: errors should be detected as early as possible, while the model is still near the failing state ^[1]. An end-to-end mismatch reports 4 KB of incorrect destination memory at completion, and on

this block it arrives an order of magnitude late, tens of microseconds and hundreds of bursts after the mistake; the assertion identifies the offending beat. The checks are complementary: assertions cannot see wrong data, and image comparison cannot identify *when* the error occurred.

Two further decisions prevent false failures from over-specification. **Exact split sequences over-specify the model:** checking them turns every legitimate change to the burst splitter's arbitration into a regression failure. Checking instead requires bursts to stay inside one 4 KB region each, to cover the descriptor exactly once in aggregate and to match byte counts; coverage records which split shapes appeared. **Asynchronous events reach both sides:** an error response injected by the responder on beat 7 must also reach the golden model, or it predicts a complete image while the design correctly reports a partial transfer. This applies Section 8.4's processor co-simulation rule to the data mover ^[14]. Letting the golden model inspect the burst stream during bring-up diagnostics turns it into a second implementation of the burst splitter, with its own bugs and no independence (Chapter 3).

The burden belongs in the plan: a change to the way transfers are split touches the RTL, the model and the coverage model, and the review must cover all three. The flagship's DMA, with eight channels, 40-bit physical addressing and descriptor chaining, reuses this environment: the golden model adds an address-width parameter and chaining loop, the scoreboard key adds a DMA-channel dimension, and the 4 KB discipline stays unchanged. The canonical DMA bug's flagship descendant needs exactly one new component, a coherence-aware memory model, because the second half of its split writes a line that the host cluster holds in the Modified state, the state in which one cache holds the only up-to-date copy of a line.

8.8 The environment as a design in its own right

An environment that will be reused, reconfigured and debugged for three years is a software design, whose long-term requirements extend beyond immediate bring-up success.

Reuse contract. The self-checking structure belongs in a class with three properties: the class holds no reference to the verification environment around it; it exposes a procedural interface through which stimulus, exceptions and observed responses are reported; and it works with zero delay rather than as a process of its own, forking a blocking thread out of that interface on the rare occasion it needs simulation time; that zero-delay rule is also why inter-process channels and mailboxes are the wrong interface to a checker ^[11]. A checker importing the block-level environment in week three prevents its own system-level instantiation and the reuse planned at kick-off. With that independence, one environment per IP composes into cluster environments and then the top-level environment ^[5]. A verification consultancy rebuilt its scoreboard as a configurable library, so that the design-specific part became configuration and not new code, and it reports two results: its projects typically obtain an approximate 15% reduction in scoreboard code against starting from the empty base class, and they typically complete setup and configuration in-

side a day even for complex designs; the library is published under a permissive open-source license ^[17].

Configuration is data. The design configuration descriptor has to reach the predictor, because the predictor performs the same transformation the design does and that transformation depends on configuration ^[1]. Once configuration is an object rather than a set of compile-time switches, it can be randomized, printed in the failure log and replayed, and the environment stops needing a rebuild per test.

Bring-up debuggability. Four properties support bring-up debuggability: every transaction carries an identifier and a timestamp; every failure message names expected, observed *and* the provenance of the expectation; the environment has a mode with checking disabled, to separate “stimulus cannot reach this” from “the check is wrong”; and the checkers are themselves tested by injecting a known-bad response and confirming the environment fails. Checker credibility requires evidence that it can fail (Chapters 2 and 3). The claim that these four properties repay their own cost within a week of bring-up is an illustrative assumption. All four need budgeting: this is the environment’s largest part, with the greatest complexity and the most maintenance ^[1]. It requires the same review, coding standard, lint and regression discipline as RTL, with its schedule in the plan (Chapters 4 and 5).

IN PRACTICE

Implementation dependencies reduce the separation shown in the diagrams. If predicting the legal set exactly is too expensive, a scoreboard may depend on one implementation detail, such as a reordering window or prefetch depth. The assumption belongs in a comment and the plan, because an undocumented dependency can resurface at the next integration when that detail changes.

Dropping checker-testing under schedule pressure allows green regressions whose checks cannot fail. Two or three permanently broken RTL variants, kept in the repository and run nightly, expose this: the environment is broken if it does not fail on them. The term “scoreboard” can denote the data structure to one engineer and the whole self-checking subsystem to another ^[1]. Stating the intended meaning can prevent disputes about which component is golden.

PITFALLS

1. **Checking from the driver’s intent.** *Symptom:* expectations computed from the transaction object the driver was handed, not from what the monitor saw. *Cure:* predict only from monitored transactions; the driver’s copy never reaches the checker.
2. **A monitor that drives.** *Symptom:* the monitor completes a handshake or asserts a ready signal “to keep things moving”; a real stall bug becomes invisible. *Cure:* hand monitors a modport that exposes only an inputs-only clocking block, so passivity is a compile-time property, and put every reply in a responder.

3. **One global scoreboard queue for a reordering design.** *Symptom:* seed-dependent failures and progressively relaxed checking that eventually checks nothing. *Cure:* name the ordering guarantee the specification actually makes, key the queues by it, and record the choice in the plan.
4. **No end-of-test completeness check.** *Symptom:* every produced transaction matches, so the test passes while missing transactions go unnoticed. *Cure:* fail on orphan responses and on leftovers ^[1], excluding only those the design's own drop-count statistics account for ^[11], and report both counts in every log.
5. **The over-specified reference model.** *Symptom:* the model predicts arbitration order, latency or burst decomposition, and legal design changes break the regression. *Cure:* predict what the specification promises; measure the rest with coverage.
6. **The reconstructing monitor.** *Symptom:* a monitor recomputes ECC, CRC or unobservable protocol state using the design team's routine, and regression stays green on an injected fault. *Cure:* monitors report, predictors compute from an independently written specification (Chapter 3), and known-bad variants prove the checkers can fail.

SUMMARY

- Stimulus, driving, monitoring, checking and coverage are five responsibilities with five failure modes. Merging any two removes a capability: an independent oracle, passive reuse or accurate measurement.
- Signal → transaction → sequence layering uses non-blocking broadcast observation so measurement does not alter behavior ^[4, 5], while also enabling emulator retargeting ^[10].
- A scoreboard is a data structure whose shape is decided by the lookup ^[1, 11]. Its key needs one dimension per independent concurrency source and must be computable from observations; moving from one queue to a content-keyed index trades ordering checks for reordering tolerance.
- A predictor written for the testbench is a transfer function, as buggy as the design and not golden; disagreements require specification review ^[1]. Only a model that *is* the specification is golden, and even then only about one projection of correctness. Co-simulation adds two conditions: compare at an architectural boundary, because models are functionally and not cycle accurate ^[2], and deliver every asynchronous event to both sides ^[14].
- Monitors never initiate or drive; autonomous operation prevents a late testbench from back-pressuring the design ^[1]. They observe the design's output, since checking the testbench's responder only checks its own work. An agent must work identically in active and passive mode ^[4, 5], enabling block-to-system reuse ^[14].
- The environment is a design: a checker's reusability requires independence from its surroundings and a procedural interface ^[11]. The self-checking structure is the

largest and most maintained part of the environment ^[1], so its plan must provide for review and testing.

EXERCISES

Exercise 8.1 (calculation). Take the crossbar parameters listed at the head of §8.6 together with the stream key of Step 4. Compute how many in-order expectation queues that key can address, and how many the two-coordinate key that the same step rejects would address. Step 4 holds that the shorter key fails all three of the concurrency sources it lists: give the reason it fails each one, and name the way Step 4 says that failure shows itself when two managers hold the same identifier at once.

Exercise 8.2 (calculation). Take the manager count and the manager-side ID width of §8.6 together with the widened subordinate-side ID width Step 4 derives. Compute how many distinct stream identities the manager ports can present per direction, and how many identities the widened field can encode. State what keying the scoreboard at the subordinate side would then oblige the routing predictor to reproduce, and give the property of a manager-side key that Step 4 prefers instead.

Exercise 8.3 (judgement). A reviewer proposes feeding the routing predictor’s expectation record to a comparison against the subordinate-side response monitor, on the ground that read data originates at the subordinate. Decide what that comparison establishes and what it cannot, and give the step of §8.6 that carries the reason. State which of Step 0’s verdicts lose their mechanism under the proposal.

Exercise 8.4 (artifact). Read the end-of-test line below, printed by the `report()` function of the crossbar scoreboard in §8.6. State the value `report()` returns for these counts and the reason. Name the design behavior that a nonzero final field records and the thing it leaves open, and name the pitfall of the chapter that an environment without that field matches.

```
xbar_sb: matched=256 orphan=0 mismatch=0 leftover=3
```

Exercise 8.5 (judgement). A reviewer proposes replacing `!=="` with `!=` in the beat comparison of §8.6’s scoreboard, on the ground that the two operators read alike. Decide what the substitution costs, giving the language rule that decides it and the failure it makes invisible. Name the other guard of the same step that defends against the same class of value, and state what that guard prevents.

Exercise 8.6 (judgement). A reviewer proposes replacing the descriptor-level golden model of §8.7 with the burst-splitter model of §8.4’s Species 1, so that one oracle predicts burst splitting as well as destination bytes. Decide what the substitution costs, naming the scope in which the descriptor model is golden and the reason the burst-splitter model cannot inherit that status. State the over-specification §8.7 warns against and its effect on the regression.

FURTHER READING

From the corpus. ^[1] Chapters 5 and 6 are this chapter's direct ancestors: bus-functional models, transaction-level interfaces, the verification harness, and the taxonomy of self-checking techniques (hard-coded response, data tagging, reference model, transfer function, scoreboarding), including the transfer function / reference model distinction Section 8.4 is built on. ^[11] supplies the implementation rules: encapsulation of the self-checking structure, its independence from the environment, its procedural zero-delay interface, and the scoreboard recommendations for in-order queues, per-stream queues, hashing indexes, assumed drops and margin-of-error comparison. ^[5] gives the component vocabulary and the analysis-port communication model, with ^[4] as its normative statement. ^[2] is the historical source for the decomposition and the transaction-level framing; ^[14] is a recent industrial account of a multi-level environment with ISS co-simulation and passive reuse. ^[10] shows what transaction-level layering buys on an emulator, and ^[3] Clauses 14 and 25 define the sampling and modport semantics every monitor depends on. Siemens EDA's Verification Academy *UVM Cookbook* catalogs these patterns with worked scoreboard and agent code. It describes the predictor as a stand-in for the unfinished design, usually ready earlier through higher abstraction, so the testbench can be built and debugged before the design is available ^[6]. Its API details instantiate the vendor-neutral structure described here.

Not in the corpus. Mike Mintz and Robert Ekendahl, *Hardware Verification with SystemVerilog: An Object-Oriented Framework* (Springer, 2007), treats environment architecture as software design at book length. It is background only.

REFERENCES

1. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. **P18** Y. Hollander, M. Morley, and A. Noy, "The e Language: A Fresh Separation of Concerns," Proc. Technology of Object-Oriented Languages and Systems (TOOLS 38), Zurich, Switzerland, pp. 41-50, 2001.
3. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
4. **S9** IEEE, "IEEE Std 1800.2-2020, IEEE Standard for Universal Verification Methodology Language Reference Manual (Revision of IEEE Std 1800.2-2017)," IEEE, 2020.
5. **S11** Accellera Systems Initiative, "Universal Verification Methodology (UVM) 1.2 User's Guide," Accellera, October 2015.
6. **B8** Functional Verification Methodology Team, "UVM Cookbook," Siemens EDA Verification Academy, 2021 (revised 31 January 2022).
7. **S21** USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.
8. **S25** IEEE, "IEEE Std 1666-2023, IEEE Standard for Standard SystemC Language Reference Manual (Revision of IEEE Std 1666-2011)," IEEE, 2023.
9. **D43** G. Delbergue, M. Burton, B. Le Gal, and C. Jegou, "Analysis of TLM-2.0 and its Applicability to Non Memory Mapped Interfaces," Proc. DVCon U.S., 2016.
10. **D33** A. Grove, "UVM hardware assisted acceleration with FPGA co-emulation," Proc. DVCon Europe, 2015.

11. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
12. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
13. **S15** RISC-V International, "The RISC-V Instruction Set Manual Volume II: Privileged Architecture, Version 20250508 (Ratified)," RISC-V International, May 2025.
14. **P25** S. Ahmed, A. Kropotov, R. I. Genovese, B. Homs, E. Merino, F. Urbani, et al., "Verification and Validation (V&V)-in-the-Loop for RISC-V Design: The Holistic Vision of BZL," arXiv preprint arXiv:2604.27013v1, April 2026.
15. **D6** D. Graham, A. Sutton, S. Davidmann, P. Gouedo, X. Aubert, and Y. Pruvost, "Planning for RISC-V Success: Verification Planning and Functional Coverage," Proc. DVCon Europe, 2023.
16. **D19** N. Khan, Y. Kashai, and H. Fang, "Metric Driven Verification of Mixed-Signal Designs," Proc. DVCon U.S., 2011.
17. **D41** J. Andersen, P. Jensen, and K. Steffensen, "Versatile UVM Scoreboarding," Proc. DVCon U.S., 2015.

9. Stimulus: Directed to Random to Portable

Stimulus generation ranges from directed tests to constrained-random and portable scenarios; the DMA example starts with a testbench that is three weeks old. Its driver and its scoreboard, the structure that holds what each transfer should produce and checks every output against it, support eleven directed tests, all passing. Test 1 programs a 64-byte 1D copy and compares the two buffers. Test 7 programs a 2D transfer with a positive row stride, the signed address step from the start of one row to the start of the next. Test 11 programs a destination eight bytes below a 4 KB boundary, the address line that AXI forbids a burst to cross: every burst stays inside one 4 KB address region, so it never spans two subordinates. The burst splitter is the DMA stage that splits each transfer into bursts that the AXI address rules allow; it sits between the register frontend and the AXI backend, and this test checks the splitter's split against the 4 KB rule. Selecting test 12 exposes the limits of directed enumeration.

Candidates include negative strides, both channels at once, and `NUM_REPS` at its maximum. Another is a 2D row, a single line of a two-dimensional transfer, exactly one maximum burst long. It starts exactly one maximum burst below a 4 KB boundary, on DMA channel 0, the first of the DMA's two independent transfer engines, while channel 1 drains. Each is plausible and takes half a day to write and debug; the list has several thousand candidates rather than eleven more. Hand-written tests are slow and find only anticipated bugs.

Constrained-random generation replaces test enumeration with *rules*: a model of legal transfer descriptors, the records that program one DMA transfer each, lets an engine search for the values that satisfy every active constraint and generate descriptors by the million, while coverage measures the results. That engine is the *constraint solver*. This shift from tests to rules introduces a new class of weakness: stimulus that is abundant and legal, and that still reaches none of the situations that matter.

CHAPTER MAP

§9.1 covers directed tests for bring-up, spec compliance and regression anchors, including their scaling limit; §9.2 traces the contributions of dedicated verification languages. §9.3 separates *legality* from *distribution* and examines coverage lost to over-constraining. §9.4 defines the limits of seed-based reproduction. §9.5 layers atomic transactions into virtual sequences coordinating several interfaces. §9.6 covers portable stimulus's capabilities beyond constrained random and its limits.

9.1 Directed tests: precision, and its ceiling

A **directed test** is stimulus whose content and order are written out by hand. Individual testbenches verify individual features: the stimulus is manually crafted to exercise one feature, and the response is checked against the symptoms that would appear if that feature were broken ^[1]. Nothing is chosen at run time; the test does the same thing every time it runs.

Determinism supports three tasks: bring-up, specification compliance and regression anchoring.

Bring-up. The first stimulus a new block sees must be small enough to debug when everything is broken at once. When the DMA engine's first descriptor fails, debugging must establish whether the fault is in the register decode, the burst splitter, the backend, the scoreboard or the clock. A twelve-line directed test answers that in one waveform; a randomized 2D transfer with 47 rows and a negative stride does not isolate the fault.

```
// Directed: the first descriptor the DMA engine ever executes.
task automatic dma_bringup_1d();
    bus_write(SRC_ADDR, SRAM_BASE);
    bus_write(DST_ADDR, SRAM_BASE + 32'h8000);
    bus_write(NUM_BYTES, 32'd64);           // 8 beats of 64 bits
    bus_write(CFG,      CFG_EN_1D_CH0);    // enable, 1D, channel 0
    do @(posedge clk); while (!(bus_read(STATUS) & ST_DONE));
    check_memory_equal(SRAM_BASE, SRAM_BASE + 32'h8000, 64);
endtask
```

Spec compliance. Some stimulus is enumerated by an external authority, and randomizing it violates that enumeration. An HBM controller's initialization and PHY-training sequence, the opening handshake of a high-bandwidth memory interface, is a strictly ordered protocol with mandated wait states between steps: there is exactly one legal opening move, and the test that checks that opening sequence is directed. Any published compliance suite likewise requires its specified tests rather than a random sample from a separately defined space.

Regression anchors. A short deterministic test that touches one feature and finishes in seconds serves as a smoke test: it establishes within a minute of a commit whether the design still boots.

Where it stops. The directed approach can be managed to completion when the total number of test cases is in the low hundreds; a project with over a thousand identified test cases would need more than a year this way, and beyond that some form of testbench automation becomes necessary to complete the task within an acceptable time frame ^[1]. That necessity is a claim about schedule rather than about feasibility. The reference SoC's DMA alone generated several thousand candidates in the paragraph above.

Directed testing also has two failure modes. The first is **bias**: directed test cases can only find anticipated bugs [1]. Hand-written stimulus reflects the author's design model, so errors in that model leave gaps: Chapter 2's opening suite omitted negative strides because its author pictured a 2D copy moving forward.

The second is **decay**: a directed test can pass forever after it stops testing its intended feature. Enlarging a FIFO or memory leaves tests of its full operation passing while they exercise only part of it. The defense is a redundant path: a coverage model, the space of situations a run is measured against, states what the test is supposed to accomplish and requires it to fill 100% of those points, since deterministic stimulus should fill exactly what it targets [1]. A directed test that drops to 60% of its own goal has become stale.

In 2024, random tests are still described as covering the vast majority of easy-to-detect scenarios but unable to reach complex corner cases in reasonable time, with directed tests covering the remainder [2]. One methodology recommends a directed test when a scenario is too complex to describe or too unlikely to happen randomly; a run need not be 100% directed, since a directed sequence can be injected randomly within a stream of random ones, and injecting a directed sequence in that way may expose problems a purely directed run would not [3].

9.2 The constrained-random turn

Constrained-random generation addressed bugs that appear only in convoluted scenarios, where states across several state machines align only through one particular transaction sequence; once a team had written as many directed tests as it could, random simulation was the only reasonable response left [4].

From enumeration to description

The declarative model describes *the set of legal runs*. Constraints are formal, unambiguous specifications of design behavior; in stimulus generation they define what input combinations can be applied and when. Because they are declarative they read like the specification rather than like a program, and because they are executable (stimuli are generated directly from them) they eliminate manual construction of a procedural testbench, raising testbench writing to a higher level of abstraction much as RTL raised design above gates [4].

Through the **generator/monitor duality**, the same constraint that *generates* legal traffic on one block's input can *check* legality on a neighboring block's output, making interface constraints reusable one level up the hierarchy [4]. Chapter 11 makes the same object an assertion; Chapter 14 makes it a formal assumption.

The dedicated languages, and what survived them

Dedicated hardware verification languages developed this model before general standardization, and the **e** language records it. As a commercial product, **e** came from Verisity Ltd., and what put the language on projects was Specman, Verisity's

flagship product; the language’s designers noted that most of Specman was itself written in `e` [5]. In it, a data item is a struct whose fields carry declarative constraints, resolved during the run to generate a random but directed stream of data for the DUT [5]. Three ideas from `e` remain in current practice:

- **Tests as constraint layers.** A test specification, in its simplest form, adds a few constraints on top of an existing environment, extending an already-defined packet type with `keep i == j` and `keep kind != empty` rather than editing the generator.
- **Preference distinct from requirement.** Constraints could be marked soft: a hard constraint applied to the same field by another piece of code overrules them, so a general-purpose default could be overridden by a specific test without touching it.
- **Weights, possibly reactive.** A select-style constraint weighted the generator toward chosen alternatives, and those weights could be integer-valued functions of data sampled from the DUT, so generation was influenced by the current state of the device [5].

SystemVerilog absorbed the model into a general-purpose language, and the semantics are now normative in IEEE Std 1800-2023, *IEEE Standard for SystemVerilog* [6]. Class variables are declared `rand` or `randc`; `randomize()` generates values for all the *active* random variables of an object, subject to the *active* constraints, and returns 1 on success or 0 on failure; `rand_mode()` and `constraint_mode()` move a variable or a constraint block out of that set. Inheritance builds layered constraint systems, so a general-purpose model can be constrained into an application-specific one, and `randomize()` with `add` adds constraints inline at the call site [6], standardizing the `e` test-layering pattern. In the 2024 industry data, constrained-random simulation is the one simulation-based technique whose adoption is still climbing rather than having stabilized [7].

What the trade actually costs

Compared with the directed approach, a coverage-driven random approach trades longer initial testbench development for more productive feature coverage in the long run; the source declines to quantify the gain, because it depends heavily on the team’s commitment and on its skill at writing generators that can be constrained easily from the outside [1]. Two commitments follow, and both apply from day one of testbench development: functional coverage, and a generator built to be constrained from the outside.

First, **functional coverage is required**, because only the coverage model measures what actually ran; the generator describes what could happen. Without functional coverage, a random environment is merely running directed test cases whose stimulus is filled in at random [1].

Second, **generator architecture** needs deliberate design to exercise the required features; it is better to design a generator that can be constrained easily from the outside than to alter its code for each new test ^[1]; this decision belongs with the environment architecture of Chapter 8.

9.3 Writing good constraints

Constraints have two distinct jobs: defining **legality** and shaping the **distribution** of legal stimulus. Conflating them causes bad stimulus.

Legality defines acceptable design inputs

Stimulus constraints come in two families: *environment* constraints, which express the interface protocol and must be obeyed strictly, and *directive* constraints layered on top, which steer the run toward the scenarios currently targeted. The figure shows the two families as three nested regions ^[4] (Figure 9.1):

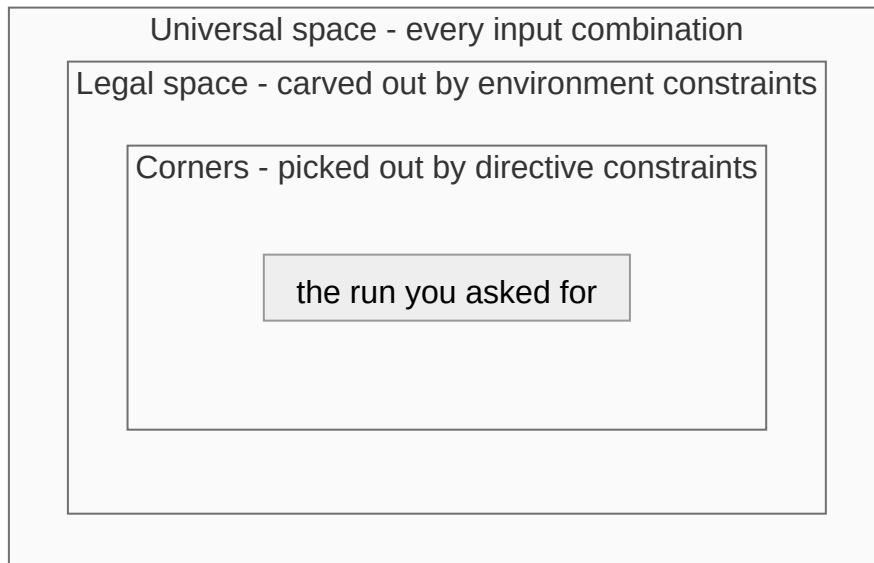


Figure 9.1 — The two families of stimulus constraint drawn as nested regions: environment constraints carve the legal space out of the universal space of every input combination, and directive constraints pick out the corners inside it that give the run you asked for. Which boundary a constraint moves is what decides where it belongs — the outer one is the design's and lives in the generator, the inner one is a test's and lives in a layer above it.

The DMA's 2D descriptor generator labels each constraint by family. The DMA's 64-bit data path and its maximum burst of 256 beats, a beat being one data transfer inside a burst, give 2048 bytes per maximum burst, exactly half of one 4 KB address region.

```
class dma_2d_descriptor;
  rand bit [31:0] src_addr;
  rand bit [31:0] dst_addr;
  rand int unsigned num_bytes; // bytes per row
  rand int src_stride; // signed
  rand int dst_stride; // signed
```

```

rand bit [15:0] num_reps;      // rows
rand bit      channel;
rand bit [31:0] dst_row[];    // the rows this descriptor will touch

// --- legality: the frontend rejects anything else ---
constraint c_bus_alignment {    // 64-bit data path
    src_addr[2:0] == 3'b000;
    dst_addr[2:0] == 3'b000;
    num_bytes[2:0] == 3'b000;
    src_stride[2:0] == 3'b000;
    dst_stride[2:0] == 3'b000;
}
// --- policy, NOT legality: an unmapped address is legal to PROGRAM.
// Disable this one in the error-response test.
constraint c_addressable {
    src_addr inside {[SRAM_BASE : SRAM_BASE + SRAM_SIZE - 1]};
    dst_addr inside {[SRAM_BASE : SRAM_BASE + SRAM_SIZE - 1]};
    foreach (dst_row[i]) // a negative stride must not walk the rows
        dst_row[i] inside {[SRAM_BASE : SRAM_BASE + SRAM_SIZE - 1]};
}
// --- policy: keep the randomized object small enough to solve fast ---
constraint c_solver_budget {
    num_bytes inside {[8 : 2048]}; // <= one 256-beat burst
    num_reps inside {[1 : 64]};
    src_stride inside {[ -8192 : 8192]};
    dst_stride inside {[ -8192 : 8192]};
}
constraint c_rows { // definition, not policy
    dst_row.size() == int'(num_reps);
    foreach (dst_row[i]) dst_row[i] == dst_addr + unsigned'(i * dst_stride);
}
endclass

```

Four details distinguish legality from policy. The `c_addressable` comment distinguishes programming from execution. An address outside the memory map is legal to *program*, because the register field accepts it. What the DMA then does with that address is a decode error reported per DMA channel, and that response is a feature the stimulus must exercise. Restricting stimulus to the uses that the design is intended to see is the error Chapter 2 warns against, and such a restriction can be made permanent in a `constraint` block. Second, `c_solver_budget`'s bounds are engineering choices about solve time, not statements about the DUT; labeling them as policy lets a later test relax them without anyone wondering whether it has just generated something illegal. Third, a bit-select expresses alignment much more cheaply to solve than the equally correct `num_bytes % 8 == 0`. Fourth, `c_addressable` bounds `dst_row[]` and not just the two base addresses, because a negative stride otherwise takes rows outside the map and wraps their addresses through zero. Leaving a derived field unbounded allows unintended addresses.

Uniformity over legal combinations

Random values shall be selected to give a uniform distribution over legal value *combinations*, with every legal combination equally probable, so that randomization explores the whole design space ^[6]. The guarantee concerns the *solution space defined by the constraints*, distinct from the space the coverage model measures. Uniform distribution over the solution space does not reflect the intention of covering as

many functional scenarios as possible; the distribution most tools provide is derived from the constraint model and is not aligned with the coverage model. In one published example, packets constrained only for internal consistency were measured against a coverage model asking for the extreme edges of the address range: over a run of a thousand generated instances most of its objectives were left uncovered, and adding distribution directives closed the coverage completely [8]. The constraint model contains no directive that could steer toward the edges, because it does not encode the importance of the edges.

Scenario. The radar front-end's FFT pipeline takes 16-bit samples in Q1.15, a signed fixed-point format whose values carry a sign bit ahead of the binary point and a fractional part behind it. **Approach.** Every 16-bit pattern is legal, so bare `rand bit signed [15:0]` is a correct generator. Endpoint bugs include saturation at the most-negative value, whose negation is unrepresentable. Uniform sampling over 65,536 values visits each end with probability $1/65,536$. `systemverilog constraint c_dynamic_range { iq dist { 16'sh8000 := 3, // most negative 16'sh7fff := 3, // most positive 16'sh0000 := 2, [16'sh8001 : -1] :/ 46, [1 : 16'sh7ffe] :/ 46 }; }` **Why.** The legal set is unchanged; only the probability mass moved. Weighting distinguished values against broad ranges makes the corner case frequent without adding illegal values.

Shaping: `dist`, ordering, and `soft`

Three constructs shape the distribution without causing solver failure.

`dist` assigns weights to values and ranges: absent any other constraints, the probability that the expression matches an item is proportional to that item's weight. Of the two operators, `:=` applies the weight to each element of a range, while `:/` divides one weight across the whole range. The weights are ratios and need not be normalized to 100. Nonzero weights do not change the solution space, and therefore they cannot cause the constraint solver to fail. Other constraints on the same expression renormalize the remaining choices. Values *absent* from the list or with *total* weight zero are forbidden [6]. A `dist` whose ranges do not cover everything legal is silently an over-constraint.

`solve...` before changes probabilities by ordering choices. With `s -> d == 0` over a 1-bit `s` and a 32-bit `d`, the legal combinations number $1 + 2^{32}$: `s` true pins `d` to zero and admits a single combination, while `s` false leaves `d` free. `s` is true in exactly one of those combinations, and two of them leave `d` at zero: that combination, and the one in which `s` is false and `d` is drawn as zero. Those two give `d == 0` a probability of $2/(1 + 2^{32})$, effectively never. Adding `solve s` before `d` leaves the legal set identical but makes `d == 0` occur with probability slightly over 50%. Variable ordering forces selected corner cases to occur more often and, like `dist`, cannot cause the constraint solver to fail because it does not change the solution space [6]. Variable ordering addresses the pattern in configuration descriptors in which a control bit implies a narrow data case.

`soft` separates preference from requirement. Hard constraints must be satisfied or the constraint solver fails; a soft constraint is discarded when no solution satisfies it alongside the active hard ones. Its purpose is layering: the author of a generic component supplies defaults that a later, more specialized constraint overrides automatically, where a hard default would force the test to disable that default procedurally or to extend the class just to put another default in its place [6]. In the DMA generator, “most descriptors are small” belongs in a soft constraint; “addresses are 8-byte aligned” does not.

Over-constraining and coverage loss

Over-constraining has two forms; only one produces an explicit failure.

Unsatisfiability is the explicit failure. When the constraints admit no solution, not a single input is allowed at the current state, so that state is a dead-end and simulation has to be aborted; over-constraints stall the progress of simulation and, in formal verification, they lead to vacuous proofs, which succeed only because nothing ever satisfied the property’s antecedent, so nothing was checked; tools provide constraint diagnosis for that reason, to identify a minimal set of constraints responsible for the conflict [4]. The antecedent of a property is the condition that must become true before the property checks anything. A failed `randomize()` exposes an unsatisfiable set of constraints within seconds. Checking the return value of `randomize()` can be necessary for exactly that reason, a set of constraints with no solution [6]; treating the check as mandatory is the practitioner’s rule on top, because an unchecked failure leaves the object holding its previous values and the run continues, re-sending the last legal descriptor forever.

The more damaging form remains satisfiable while excluding the feature under test. A lingering constraint can prevent generation of exactly the input sequences that would fill persistently empty coverage points [1], without failures or warnings; the resulting plateau can be mistaken for remaining difficult cases.

Worked example: exclusion of the canonical bug. In week 4 of the DMA campaign, a constraint is added after a failure review, and it excludes the canonical bug. The canonical bug is the recurring DMA defect that this book follows from plan to fix. An engineer sees descriptors with rows straddling a 4 KB boundary and concludes they are illegal because no AXI burst may cross a 4 KB boundary (Arm IHI 0022H.c §A3.4.1 “Address structure”) [9], adding:

```
class dma_desc_overconstrained extends dma_2d_descriptor;
  // Week 4: "the AXI spec says no burst may cross a 4 KB page."
  constraint c_no_page_cross {
    foreach (dst_row[i])
      (32'(dst_row[i][11:0]) + num_bytes) <= 32'd4096;
  }
endclass
```

The constraint is *true of the design’s output* and *false of its input*. The 4 KB rule governs the bursts that the backend may emit (Arm IHI 0022H.c §A3.4.1 states the rule of a burst, and because that rule speaks of bursts rather than of the descriptors the

stimulus programs, the constraint is wrong of the *input*) [9]; splitting a request that would cross a 4 KB boundary is the burst splitter’s job, the feature that the campaign exists to verify. Imposing the DUT’s obligation on stimulus leaves tests passing because they no longer exercise its faulty behavior.

The canonical coverage model `cg_2d_4kb` (Chapter 5) crosses four axes: stride sign, destination-row relation to the next 4 KB boundary, burst length and other-channel activity, giving $3 \times 4 \times 3 \times 2 = 72$ cells. The new constraint makes the `straddle` bin of `cp_boundary_dist` unreachable, and with it every cell containing it: $3 \times 1 \times 3 \times 2 = 18$. Not all eighteen were reachable; deriving the difference supplies the evidence needed by a waiver, the written record that accepts one hole in the evidence and names its reason, its risk and its owner.

A coverpoint is an expression that a coverage model partitions into bins, the counters that record which values a run actually produced. The two axes sample at different scopes: `cp_boundary_dist` classifies a destination row, while `cp_len` samples the length of a burst as the burst splitter actually emits it, after any split. `c_solver_budget` caps a 2D row at 2048 bytes (256 beats on the 64-bit path, one maximum burst, half a 4 KB region), so a row reaches past at most one 4 KB boundary. A straddling row splits into exactly two bursts, and the beat counts of those two bursts sum to the beat count of the row, at most 256, with each burst carrying at least one beat. Neither piece can be 256. `straddle` $\times$ `max` is therefore empty for every stride sign and both channel states: **6 of the 18 were unreachable before anyone wrote the week-4 line**. Subtracting those six from the eighteen leaves twelve, and what `c_no_page_cross` costs is the other **12**. The canonical DMA bug (a negative-stride row mis-split at the 4 KB boundary, corrupting data only when channel 1 is also active) sits in `neg` $\times$ `straddle` $\times$ other-channel-active, one cell per burst-length bin; the `max` one sits in the excluded column, so the constraint deletes **both** of the bug’s live cells.

A plan row is the line of the verification plan that names one feature, the method that tests it and the evidence that closes it; plan row DMA-F06a needs two caveats. The six exclusions depend on *policy*: without the solver budget, a 4104-byte row at offset 4088 splits into 1 beat and 512, with the 512 splitting again into 256 + 256, making `straddle` $\times$ `max` reachable. And the whole argument is destination-side: `c_no_page_cross`, `c_addressable` and `dst_row[]` all speak about writes. Chapter 5’s *100% means 100% of the cells the model can legitimately reach* requires this bookkeeping, separating what is impossible from what was merely decided; an unrecorded exclusion can lead a sign-off to quote 72, the model’s full cell count. A sign-off is the review that judges the evidence against the plan.

The coverage report shows one permanently empty block: a whole axis value missing from every combination it appears in, with no test failures. Mistaken for a difficult corner, the block receives an owner and due date but remains unresolved.

Diagnosing the coverage gap. Three checks identify the cause:

1. **Checking the generator.** A functional-coverage gap could also indicate a fault in a random generator or the verification environment [1]. A whole axis *value* ab-

sent from every combination, rather than scattered missing cells, indicates a disabled stimulus dimension (Chapter 6).

2. **Constraint and coverage comparison.** For each axis of `cg_2d_4kb`, identify the generator variable that changes it. For `cp_boundary_dist`, only a pinning constraint exists.
3. **Targeting the missing cell.** Request the relation defining the missing *cell* so a randomization failure can diagnose the constraint conflict.

```
systemverilog // aim at neg x straddle: a destination row reaching past the frontier
if (!desc.randomize() with { dst_stride < 0; (32'(dst_addr[11:0]) + num_bytes) > 32'd4096; }) $fatal(1, "over-constrained: no descriptor reaches past a 4 KB frontier");
```

The second clause carries the diagnosis. It is satisfiable in `dma_2d_descriptor`: offset 4088 plus a 2048-byte row reaches 6136. It is unsatisfiable in `dma_desc_overconstrained`: `c_rows` places row 0 at `dst_addr`, and `c_no_page_cross` then requires that row to end inside one 4 KB region, which is the negation of the relation the call requests. Targeting only `dst_stride` instead permits an 8-byte row, which fits inside one 4 KB region at every legal offset and therefore cannot expose this conflict.

Distribution shaping. Removing `c_no_page_cross` restores reachability, and straddling is then not even rare. Uniformity still poorly serves two axes: maximum burst length is one of 256 legal values, while legal negative strides receive no more probability mass than positive ones. Shaping both axes corrects their distributions without changing which descriptors are legal:

```
class dma_desc_shaped extends dma_2d_descriptor;
  constraint c_len_extremes {
    num_bytes dist { 2048 := 20, 2040 := 20, [8 : 2032] := 60 };
  }
  constraint c_frontier {
    (32'(dst_addr[11:0]) + num_bytes) dist {
      [0 : 4088] := 35, // where row 0 ends, mod 4 KB
      4096 := 15, // ends before the frontier
      [4104 : 6136] := 50; // ends exactly on it
    } // reaches past it: must be split
  }
  constraint c_stride_sign {
    dst_stride dist { [-8192 : -8] := 55, 0 := 5, [8 : 8192] := 40 };
  }
  constraint c_order { solve num_bytes before dst_addr; }
endclass
```

`c_frontier` constrains the *expression classified by the coverage model* rather than a raw address. The third range maps onto `straddle`. The first two land somewhere among the other three bins without choosing which, since those bins are classified against a burst's worth of headroom rather than against the boundary. The expression names row 0, while `cp_boundary_dist` samples every row. The `c_frontier` constraint therefore targets one bin of four, on the first of up to sixty-four rows; it does not target all four. In this example, software rarely uses the signed stride field, so `c_stride_sign` gives legal negative strides more mass than positive ones to increase their occurrence.

`c_len_extremes` and `c_frontier` both constrain expressions containing `num_bytes`, and `c_order` orders `num_bytes` itself against `dst_addr`. The constructs are legal: no `randc` variable is involved, and each distributed expression contains a random variable. A `randc` variable is a random variable that cycles through every value of its range before any value repeats. Weights hold *absent any other constraints*; where other constraints make some of them unsatisfiable, an implementation is required only to satisfy the constraints [6]. The standard leaves interaction between `dist` and solve order undefined, though permitted (IEEE 1800-2023 §18.5.3 and §18.5.9). Thus `solve num_bytes before dst_addr` expresses an intent of “length first, address after” without guaranteeing that mechanism. Coverage reports establish whether two distributions over overlapping expressions achieved their intended effect. With the three in place and both channels enabled, the negative-stride straddle cells fill within a night’s regression: the scoreboard reports a destination mismatch, and the seed that produced it is the bug report.

Repeating check 2 exposes one remaining axis. `dma_2d_descriptor` cannot change `cp_other_ch`: other-channel activity relates two streams and cannot be shaped within one descriptor generator. Both reachable cells of the canonical bug lie on the other-channel-active side, so this generator, however well shaped, reaches neither. Getting there needs a second descriptor on channel 1 inside the same randomization, which is what Section 9.5 is for.

Two costs: solving and manual shaping

Although constraints are concise, the complexity of solving them is multiplied by the hundreds or thousands of constraints and variables present when verifying a unit- or block-level design, and fast solving is critical to simulation throughput and therefore to coverage. Generic constraint solving is NP-hard; SAT is NP-complete, and its solutions are *believed* to have an exponential worst case; SAT performance varies widely across similar-looking instances, and SAT is one approach among several: linear and integer programming, automatic test pattern generation, binary decision diagrams, Boolean unification and constraint logic programming [4]. The tool chooses its solving engine; the problem does not prescribe one. In practice, small randomized objects and bit-selects instead of arithmetic reduce cost; a generator suddenly taking 40 seconds per transaction warrants a bug report.

Manual shaping adds overhead, motivating Chapter 6’s automation research. Using distribution and ordering directives well demands understanding both their semantics and the solving process, for three reasons. Distribution constraints carry order semantics, and misunderstanding those semantics yields a distribution different from the one intended. In a large model the relations between fields are tangled, so a directive on one field can change the distribution of others. And the engineer must know the whole coverage model, since not all objectives can be met at once. Re-ordering distribution *constraint statements*, rather than items within a `dist` list, suffices to change a working example’s distribution [8]. Hence a review rule: distribution constraints are read as carefully as legality ones, and whoever writes one names the bin it aims at.

9.4 Seeds and reproducibility

Generated stimulus makes each execution a sample of a test’s possibilities. The unit of work becomes a **run**: an execution with a specific seed and a specific set of source files, producing a specific set of messages and coverage metrics [3]. The shorthand “a run is a (test, seed) pair” omits the source identity needed for reproducibility. The same seed against a different RTL revision, a different testbench commit, or a different simulator build is a different experiment.

Reproduction infrastructure must make it *easy* to identify which versions of which sources, tools and libraries were used and to reissue the exact command, especially the seed. Command-line options cannot be source-controlled, so a script should set the detailed options from broad, high-level ones. A testbench reused across simulator and constraint-solver versions may generate different stimulus for the same seed [3].

Random stability keeps seeds useful while the testbench changes. The RNG is localized to threads and objects, so the values one object or thread produces are independent of the others. Each class instance gets its own RNG, seeded from the thread that creates that instance. Each new dynamic thread also gets its own RNG, seeded from its parent thread. Under that hierarchical seeding a whole subsystem can be pinned by manually seeding one root thread. Stability is preserved as long as object creation, thread creation and random-number generation happen in the same order as before, so new objects, threads and draws should be added *after* existing ones, at the end of a code block, to keep previously created work reproducible [6]. Inserting one `$urandom` call at a sequence’s start can therefore change an entire regression; appending preserves the earlier sequence.

Seed count and coverage. Low input coverage after only a few runs calls for further sampling (in the source’s example, 8 of 10 opcodes after three runs); the same result after many runs (twenty-five, in that same example) calls for reviewing the generator’s distribution. A constraint that drives an unfilled point toward certainty is more work, and likely the more productive avenue, especially once hundreds of previous runs have failed to produce the inputs that point needs [1]. A test that misses points it was *designed* to hit needs repair. Points it was never designed to hit belong in a new one, so the existing test keeps working and keeps the coverage it earns [3].

Run length. One long run and many short ones produce equal-quality random data if the source is fully random and the seeds do not repeat a previous sequence, but their project effect differs: a long run is cumbersome to reproduce when the error appears near the end; it exercises a single device configuration; and it can leave the DUT in one region of the state space, where further stimulus adds nothing; many short runs, by contrast, reproduce faster, sweep more configurations and traverse the space more quickly [1]. The state space is the set of configurations the design’s state elements can take. Many short runs are therefore preferable; Chapter 12 develops their regression economy.

Debug cost. Randomness moves effort from writing stimulus to reading failures, and debug already takes the largest share of effort: in 2024, verification engineers

spend more of their time on debug than on any other activity ^[7]. A directed test that fails names the intent it was written to check, whereas a random run that fails supplies a log and a seed. Two inexpensive practices help: have every generated item print a human-readable description of itself, and record the seed in the failure report so triage begins with a command that reproduces the failure. Triage is the sorting of failures into facts before any one of them is debugged.

9.5 Sequences and stimulus layering

Individually constrained transactions cover many cases but cannot express the *relationships* needed to expose further failures. A **scenario** is a sequence of random or directed stimulus of particular interest for the device under test, one that individually constrained-random stimulus is unlikely to produce on its own ^[3]. Three stimulus layers add relationships that the layer below cannot express.

Atomic. Individually constrained transactions suit cases needing no constraints across a *sequence*, such as a configuration descriptor ^[3]; for the DMA, the atomic item is one descriptor.

Scenario. This layer builds sequences of random transactions with defined relationships, each sequence aimed at a particular functional corner. The scenario is described and randomized as an *array* of items solved all at once, letting the constraint solver relate past, current and future items in one constraint set ^[3]. Randomizing three descriptors in turn cannot express “the third descriptor reuses the second’s destination”.

Scenario. A USB controller’s link power states (*Universal Serial Bus 3.2 Specification*, Revision 1.1, USB 3.0 Promoter Group, June 2022, section 7.5, “Link Training and Status State Machine (LTSSM)”) ^[10]. **Approach.** The atomic item is a packet; the scenario traverses U0 to U1, an inactivity timeout to U2, and a remote wake back to U0, with transfers scheduled *relative to* those transitions. U0 is the link’s active state; U1 and U2 are low-power states the link can rest in, and U2 is the deeper of the two. **Why.** Scheduling a transfer in the cycle of low-power entry requires a relationship between two items and a timer, beyond the constraints on either item.

Multi-stream (virtual). The third level addresses one technical limitation: constraints cannot be expressed *across* scenarios in different streams, because those items are generated in different generators and are not inside the object that any one generator randomizes. Expressing cross-stream relations requires that the descriptors for all streams be randomized as a single object. Three arrangements are alternatives: synchronize otherwise independent generators; let a single-stream scenario descriptor forward some of its items to other streams; or use a multi-stream generator that randomizes one descriptor holding a sub-descriptor per stream. The first is enough when the scenario needs only the initial synchronization of normally independent streams (it will force an Ethernet MAC collision by starting all transmit-

ters together) and it is not enough when the scenario needs the *contents* of the individual streams cross-correlated, or individual transactions within them synchronized [3].

Worked example: one object, three streams. Two canonical bugs require conjunctions that a single-stream generator cannot request. The neural accelerator’s weight prefetcher wraps its buffer pointer one entry early when the weight stream ends mid-beat *under DMA contention*, reachable only at 2-bit weights with a width-1 tensor. A tensor’s width is the number of elements in one row of the operand that the accelerator convolves. The DMA’s mis-split at the 4 KB boundary corrupts data only when channel 1 is also active, the axis Section 9.3 could not reach within one descriptor. Both are relations between streams, so both go into one randomization:

```
class contention_scenario;
  rand dma_2d_descriptor dma_ch0; // the descriptor under test
  rand dma_2d_descriptor dma_ch1; // the other channel: what cp_other_ch samples
  rand nn_job_descriptor nn;
  rand int unsigned skew_beats;

  function new();
    dma_ch0 = new();
    dma_ch1 = new();
    nn      = new();
  endfunction

  constraint c_channels {
    dma_ch0.channel == 1'b0;
    dma_ch1.channel == 1'b1;
  }
  constraint c_contend { // one SRAM bank, all three
    nn.wgt_addr[4:3] == dma_ch0.dst_addr[4:3];
    dma_ch1.dst_addr[4:3] == dma_ch0.dst_addr[4:3];
    // where in channel 0's stream the accelerator's weight fetch lands
    skew_beats inside {[0 : (32'(dma_ch0.num_reps) * dma_ch0.num_bytes) >> 3]};
  }
  constraint c_worst_cell {
    nn.weight_bits == 2;
    nn.tensor_width == 1;
    dma_ch0.dst_stride < 0;
  }
endclass
```

The 512 KB SRAM is word-interleaved across four banks; `addr[4:3]` selects a bank on the 64-bit word grid, so equating those bits makes all three active streams contend. `skew_beats` specifies *where* in the transfer the accelerator’s stream ends, since overlap alone does not trigger the bug. It is a beat index into channel 0’s own transfer rather than a fixed cycle count, because a transfer of up to 64 rows runs for thousands of cycles and a constant range would only stagger the starts.

`c_worst_cell` belongs in the *test* targeting this cell rather than in the environment. `c_channels` completes Section 9.3's open axis: a channel 1 descriptor makes `other_channel_active` true while `cg_2d_4kb` samples channel 0's transfer. Constraints specify relations but do not schedule execution. Fixed channel numbers and bank bits ensure the three descriptors *would* contend if run together; the launching sequence must make that happen, or the scenario never fills the target cell.

For an Ethernet MAC with TSN (time-sensitive networking) queues, the atomic item is a frame and the scenario a traffic profile (a burst class at a stated line occupancy). A multi-stream descriptor correlates a high-priority queue profile with a best-effort flood on another queue *in time*, which is required to provoke a shaper-arbitration corner. The shaper is what paces the frames of each queue, and arbitration is what picks the next queue to send when several are ready at once. Layers must be interruptible: the scenario layer may be partially or completely bypassed by the test above it, so generators must be stoppable, at the start or mid-run, to let directed stimulus be injected, and restartable afterward ^[3], supporting Section 9.1's injection mechanism. Expressing all this in a particular methodology is Chapter 10's subject.

9.6 Graph-based and portable stimulus

Everything so far assumes a testbench that owns the interfaces. AMD describes the limits of this assumption at SoC level: at IP level, the constrained-random testbench is excellent, but the initialization sequences it contains are not easily portable to firmware or post-silicon tests, and IP-level tests lack system context; at SoC level, the stimulus is simple directed feature tests, complex scenarios are difficult to create by hand, and run times are long; post-silicon, tests are OS-based, debug takes longer, and failures are difficult to bring back to simulation or to emulation ^[11]. In emulation a design is compiled onto a hardware fabric and runs under stimulus a host supplies, before silicon exists. That source is a conference slide deck: it describes AMD's experience and the flow built in response, rather than survey findings. The three platforms use three incompatible stimulus formats without reuse.

Transaction-level sequences randomize transaction *contents*; the transaction *flow* is not itself randomized, and randomizing *between* sequences is difficult. A constraint class can express one resource-contention configuration with display-controller streams competing for pipes, overlays and interfaces, but that captures one specific scenario, not the ability to start and stop streams while others are in progress or to schedule them arbitrarily; it does not capture the underlying problem, which is tasks contending for limited resources ^[12]. A pipe is one of the display controller's paths for fetching a layer of pixels, and an overlay is one of the layers the controller composites into the output.

PSS. The Portable Test and Stimulus Standard (Accellera Systems Initiative), at version 3.0 in August 2024, defines a single representation of stimulus and test scenarios, usable across levels of integration and configurations, from which implementations can be generated for simulation, emulation, FPGA prototyping and post-silicon. It is a declarative domain-specific language for modeling *scenario spaces*, and its constraint and coverage syntax takes SystemVerilog as its reference ^[13].

A model is a set of class definitions plus rules constraining their legal instantiation. Behavior is an **action**. An action is atomic when it maps directly to a system operation, and compound when it encapsulates a flow of actions in an **activity**. Actions consume and produce passive **objects** whose type fixes the scheduling: a *buffer* is persistent data whose producer must complete before its consumer starts, a *stream* is transient data requiring producer and consumer to run in parallel one-to-one, and a *state* object may be read concurrently but written only exclusively. Actions may also claim **resources** from sized **pools**: a device with two DMA channels is modeled as a pool of two channel resources, so no more than two channel-locking actions may execute at any given time, matching the reference SoC and the standard’s own example. In PSS a **scenario** is one particular solution of all those constraints ^[13], and not the sequence of stimulus the word names in Section 9.5.

What distinguishes PSS from a constraint class is **inference**. Where the specification of intent is partial, the tool shall infer the additional actions and model elements needed to make that specification complete and valid, and shall infer nothing beyond what is required, so one partial specification expands into a variety of scenarios that all satisfy the stated intent while including other behaviors that broaden coverage (PSS 3.0 §5 and §5.3.2) ^[13]. For the intent “do a 2D transfer, then run an inference job”, the tool works out that the transfer needs a free channel, that the inference job’s input buffer needs some action to have produced it, and that the accelerator must have been configured first.

```
// Sketch: both DMA channels and the accelerator, inside one scenario.
buffer mem_block_s { }
resource channel_s { }

component dma_c {
  action transfer_2d {
    lock channel_s ch; // holds one channel for its duration
    rand int in [-8192..8192] dst_stride;
    output mem_block_s block; // a buffer output needs no consumer
  }
}

component nn_c {
  action infer_job {
    rand int in [2,4,8] weight_bits;
  }
}

extend component pss_top {
  dma_c dma;
  nn_c nn;
  pool [2] channel_s channels; // the reference SoC has two
  pool mem_block_s blocks;
```

```

bind channels *;                                // every object needs a same-type pool
bind blocks  *;

action contention {
  activity {
    parallel {                                  // synchronized start, no ordering across
      do dma_c::transfer_2d with { dst_stride < 0; };
      do dma_c::transfer_2d;                    // the other channel
      do nn_c::infer_job    with { weight_bits == 2; };
    }
  }
}

```

Three rules govern this model. Pool depth constrains generation: two channels means at most two channel-locking actions may hold one at a time, each with a distinct instance, and a scenario that would over-subscribe the pool is not generated at all. Reducing `channels` to one makes this activity illegal: capacity constrains generation rather than imposing a queueing delay. Second, every object reference is bound to a pool of its own exact type, as required for model legality; a buffer output may have no consumer, where a stream output may not (PSS 3.0 §5.1.1, §5.1.2). Third, `parallel` guarantees a synchronized start and the absence of any scheduling dependency across branches; two executions with no dependency between them may or may not overlap in time (PSS 3.0 §6.3.1, §6.2.3) ^[13]. `schedule` allows any legal order, including entirely sequential branches, so the action named *contention* may run without its branches ever overlapping.

Retargeting uses *test realization* to map the abstract model’s atomic actions onto target code. The order in which actions execute must conform to the flow the activities dictate *and* be correct with respect to the model rules ^[13]; this is how inferred actions, called by no activity at all, get their place in the generated test. The same `contention` action becomes transaction sequences that drive a bus verification IP in a block-level simulation, and a bare-metal C program running on the core in emulation or on silicon. A bus verification IP is a component supplied ready-made for a standard protocol.

PSS-described intent was compiled into UEFI firmware tests and bare-metal tests, and those tests ran unchanged on an AMD accelerated processing unit, first under emulation and then on post-silicon parts; they reached the hardware through the Cadence Perspec portable-stimulus tool and through an in-house AMD runtime layer called uDREX, whose job was to abstract the BIOS and provide services to the generated tests; in the lab the same flow was then used to generate fresh power-management firmware tuning scenarios ^[11]. The account reports retargeting and its application without quantifying speed-up, bug count or effort savings.

Limits. A published industrial trial from 2019 recorded six real-world observations ^[11]. One of them (that inputs and outputs for activities were not available yet) de-

scribes a 2019 toolchain and should be checked against current offerings. The other five observations concern the approach:

1. Generic actions and activities can be complex to create and may require procedural description beyond the declarative model.
2. PSS tests prefer complete control of the system: a bare-metal environment with statically allocated and scheduled tests.
3. That environment needs a bare-metal library, and building one needs collaboration across disciplines.
4. Randomization should be carefully thought through, because test *generation* time has the potential to increase exponentially. Section 9.3's solver cost reappears at scenario scale, before simulation even starts.
5. Constraint solver errors can be terse, and over-constraints are harder to diagnose across an inferred graph than within a class.

Two more come from the standard rather than from that deck. The value concentrates in scenario composition rather than in checking: expected-result computation, memory modeling and allocation policies may each need outside help, and that help comes as external models, software libraries or algorithmic code in other languages (PSS 3.0 §1.4) [13]. The standard permits them; it does not supply them, and specifying results checking inside the model remains a stated goal rather than a delivered construct. Test generation leaves the oracle, the mechanism that decides whether an observed response is correct, to the verification environment (Chapter 3). Realization code requires work for each platform, even though the model is portable. PSS reuses intent across platforms and automates multi-engine scenario composition beyond Section 9.5's layering, at the cost of a modeling language, bare-metal runtime and expertise in debugging generators.

IN PRACTICE

An example stimulus mix for a settled block combines fixed directed tests (bring-up, one compliance run, three smoke tests) with a constrained-random generator carrying most cycles, a growing set of test-layer constraints targeting named coverage bins, and a short list of scenario classes for conjunctions random stimulus cannot reach. Directed tests run on every commit, random tests nightly across seeds, and targeted tests when coverage reports identify gaps.

After a year, forty accumulated constraint files can obscure which enforce legality and which contain stale targeting rules; each block should therefore declare *legality* or *policy* in a comment. Constraining away a random failure produces Section 9.3's week-4 error, so constraint additions deserve the same review as RTL changes.

PITFALLS

1. **Constraining the DUT's job into the stimulus.** Symptom: tests pass, a block of coverage cells never fills, and the offending constraint reads like a quotation

from the specification. Cure: ask of every constraint whether it restricts the *input* the design must accept or the *output* it must produce; only the first belongs in the generator.

2. **“Invalid” used as a stimulus filter.** Symptom: the error-response rows of the plan have no owner. Cure: retain the legality constraints the interface enforces, label the rest as policy, and disable policy constraints in the error-test build.
3. **Unchecked `randomize()`.** Symptom: a test passes suspiciously fast and the log shows the same transaction repeatedly. Cure: always check the return value; treat failure as fatal, not as a warning.
4. **A `dist` that does not cover the legal range.** Symptom: a bin outside the listed ranges never fills. Cure: values absent from a distribution list are forbidden, so reserve a range for the remainder.
5. **Shaping raw fields instead of measured quantities.** Symptom: weights added, report unmoved. Cure: write the distribution over the same expression the coverpoint samples.
6. **Seeds as the answer to every hole.** Symptom: the nightly seed count rises and coverage does not. Cure: persistently low coverage after many runs calls for reviewing the distribution and constraints before adding seeds.

SUMMARY

- Directed tests suit bring-up, mandated compliance sequences and smoke tests; they stop scaling in the low hundreds and find only anticipated bugs.
- Constrained-random generation replaces enumeration with description: constraints are executable legality, and the same constraint can generate, monitor and assume.
- The constraint solver guarantees uniformity over the constraint model’s *legal combinations*, independently of the coverage model. Legality and distribution are distinct: `dist` and `solve...before` change probabilities without changing the legal set, while `soft` allows defaults to be overridden; the resulting distribution requires measurement.
- Satisfiable over-constraints can silently exclude the feature under test. A persistently empty *block* of cells calls for checking the generator before attributing the gap to a difficult corner.
- A run specifies a seed, source revision and tool version; random stability requires appending new objects, threads and draws after existing ones.
- Layering exists because constraints cannot cross generators: to relate two streams, randomize one object containing both.
- Portable stimulus adds scenario-space description and retargeting across simulation, emulation and silicon, at the cost of a modeling language, bare-metal runtime and its own solver behavior.

EXERCISES

Exercise 9.1 (calculation). Take the implication over a one-bit and a thirty-two-bit random variable that §9.3 uses to introduce `solve ... before`. Derive the size of its legal set and the probability that the wide variable takes the value the implication forces, under the uniformity the constraint solver guarantees. Then derive the same probability once the ordering directive is added, and account for the word `slightly` in the figure §9.3 reports for it.

Exercise 9.2 (calculation). Take the row length and the destination offset that §9.3 uses in its first caveat to plan row DMA-F06a, namely a 4104-byte row at offset 4088. Derive the two pieces the burst splitter produces and the further split of the larger piece, giving the beat counts on the DMA's data path. State which premise of the six-cell exclusion the example lifts, and state what §9.3 then requires be recorded on the plan row.

Exercise 9.3 (judgement). A team adds distribution weights to the descriptor generator's destination-address field to make straddling rows frequent, and the coverage report does not move. Decide why the weights had no effect, naming the expression §9.3 shapes instead and the reason that choice matters. State the two limits §9.3 records for the shaped constraint even where it works as intended.

Exercise 9.4 (judgement). An engineer inserts one `$urandom` call at the head of an existing sequence, and the previous night's failures stop reproducing from their recorded seeds. Decide which property of §9.4 the insertion broke, giving the mechanism that makes seeds reusable at all, and state where the call had to go instead. Name the two other kinds of addition the same rule governs, and the scope one manual seeding call can pin.

Exercise 9.5 (artifact). Read the merge report below, taken from the DMA campaign of §9.3 after the week-4 constraint went in. State which of the three diagnostic checks of §9.3 the shape of this report already answers, and what it answers. State what check 2 finds for the axis at fault, and state why the response §9.3 records for this report, an owner and a due date, is the wrong one.

```
cg_2d_4kb, nightly merge, no test failures
  stride-sign axis      all bins hit
  cp_boundary_dist
    straddle           not hit
    remaining bins     hit
  cp_len               all bins hit
  cp_other_ch          all bins hit
```

Exercise 9.6 (judgement). A team reports that the portable-stimulus contention action of §9.6 ran its three branches without overlapping them, and proposes shrinking the channel pool so that two of the branches must share. Decide what the report and the proposal each mean under the rules §9.6 states for that model. Name the construct that would permit an entirely sequential order, and state what §9.6 says pool capacity does to generation.

FURTHER READING

From the corpus: [4] on constraints, solvers and over-constraint diagnosis; [1] on the directed-versus-coverage-driven trade-off and generator architecture; [3] on atomic/scenario/multi-stream layering and seed discipline; [6] clause 18 for the normative randomization semantics; [5] on the language-design origins of constraint-oriented stimulus; [8] on why hand-written distribution directives do not scale; [13] for the portable-stimulus model, with [12] and [11] for industrial application and its limits; [2] for the 2024 state of automated directed-test generation.

Not in the corpus: the *SystemVerilog for Verification* texts cover randomization idioms at length, and Accellera publishes a PSS user guide alongside the standard.

REFERENCES

1. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. **P10** A. Jayasena and P. Mishra, "Directed Test Generation for Hardware Validation: A Survey," ACM Computing Surveys, vol. 56, no. 5, Article 132, 2024.
3. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
4. **B6** J. Yuan, C. Pixley, and A. Aziz, "Constraint-Based Verification," Springer, 2006.
5. **P18** Y. Hollander, M. Morley, and A. Noy, "The e Language: A Fresh Separation of Concerns," Proc. Technology of Object-Oriented Languages and Systems (TOOLS 38), Zurich, Switzerland, pp. 41–50, 2001.
6. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
7. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
8. **D10** M. Teplitsky, A. Metodi, and R. Azaria, "Coverage Driven Distribution of Constrained Random Stimuli," Proc. DVCon U.S., 2015.
9. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
10. **S21** USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.
11. **D29** P. Gupta and M. Vax, "Test driving Portable Stimulus at AMD," Proc. DVCon U.S., 2019.
12. **D5** Accellera Portable Stimulus Working Group, "Portable Stimulus Standard Update: PSS in the Real World," Proc. DVCon U.S., 2022.
13. **S1** Accellera Systems Initiative, "Portable Test and Stimulus Standard, Version 3.0," Accellera, August 2024.

10. UVM as a Methodology

UVM addresses testbench reuse and integration; in the reference SoC example, two engineers each spend a month building a block-level testbench. One verifies the crossbar with five managers, four subordinates, 32-bit address, 64-bit data and ID width 4. The other verifies the DMA engine, whose two DMA channel backends drive two of those five manager ports. A DMA channel is an independent transfer engine inside the DMA, with a state of its own and a share of the AXI backend, the logic that issues the engine's transfers as traffic on the bus. The AXI channels belong to the bus protocol, and a DMA channel is not one of them. Both engineers write AXI4 drivers that convert transaction objects to handshakes, monitors that reverse the conversion, and checks of response legality. Their code is incompatible: transaction fields differ, one uses a mailbox and the other a queue, and messages use `ERROR:` versus `*E`. A mailbox is SystemVerilog's own construct for passing objects between concurrent processes. The two testbenches also end differently: one on a `$finish` buried inside a task, one on a counter of beats, the single data transfers that make up a burst.

Integration requires an environment containing both, with the DMA engine's channel-1 backend driving crossbar manager port 3. In this example, integration takes roughly three weeks of rewriting despite both engineers having working AXI4 drivers.

UVM standardizes support for this reuse (IEEE Std 1800.2-2020, *IEEE Standard for Universal Verification Methodology* ^[1]); the following sections relate the library's structural decisions to integration effort.

CHAPTER MAP

§10.1 explains why reuse requires a standard. §10.2 defines the agent, environment and test, their roles and the limits of what each knows. §10.3 covers the factory and configuration database, §10.4 phasing, §10.5 sequences and §10.6 the register layer, including how indirection supports reuse and increases debugging cost. §10.7 covers severity and message identifiers as failure criteria and keys for regression triage. §10.8 follows block-to-system reuse through dead stimulus, relocated configuration paths, reassigned objections and expired block-local expectations. §10.9 assigns checking strategy, coverage, debuggability and runtime to the project. §10.10 covers alternatives when UVM is unsuitable.

10.1 The problem: reuse across projects, teams and companies

Verification has become a project that spans internal and external teams, and the discontinuity created by multiple incompatible methodologies among those teams limits productivity. The purpose of standardization is to improve interoperability, re-

duce the cost of repurchasing and rewriting intellectual property for each new project or tool, and make it easier to reuse verification components ^[1]. A verification component is a reusable testbench-side model of one interface, with a driver to stimulate it and a monitor to observe it.

The three orders of reuse become progressively harder. *First-order* reuse is one verification environment reused across many testcases of one project. *Second-order* reuse is components of that environment reused in a system-level environment. *Third-order* reuse is those same components reused in a different environment, for a different design ^[2]. The opening example fails at second-order reuse; block-level teams should budget for that integration.

For any of the three to work, a component must be correct and *configurable*: able to exhibit the behavior a block-level environment needs and, without modification, the behavior a system-level environment needs ^[2]. Configurability makes reuse possible, and most of the UVM library provides it.

UVM also provides a public vocabulary. Making a manager agent passive at the top level instantiates only its monitor, with no driver or sequencer, as specified in the Accellera *Universal Verification Methodology (UVM) 1.2 User's Guide*, 2015 ^[3]. Section 10.2 defines the agent and its passive mode, together with the monitor, the driver and the sequencer named here. In-house vocabulary requires explanation to a new hire or external partner and reconciliation with a purchased verification component. The user guide presents its recommended architecture as one viewpoint among possible structures ^[3]; the standard defines the *library and its meanings* without mandating that structure.

The library's occasional inconsistency reflects its standardization history. UVM was the first verification methodology to become an IEEE standard, and the standard's introduction traces its lineage to the Open Verification Methodology (a merger of three earlier methodologies), VMM, and transaction-level modeling derived from SystemC work. The library provides more than one mechanism for configuration: factory overrides select types, while the configuration database supplies values. Chapter 6 introduced the SystemVerilog language standard the library sits on, the one document that holds the design constructs, the testbench classes and the assertions together. The current UVM standard, IEEE 1800.2-2020, revises the 2017 edition, defining APIs and a base class library atop IEEE 1800 SystemVerilog for library implementers, implementers of supporting tools and users ^[1]. As a conformance document, it specifies required library behavior without prescribing a good testbench. Confusing conformance with testbench quality creates expectations UVM cannot meet; section 10.9 develops the distinction.

In the 2024 industry study UVM is the predominant standard for creating IC/ASIC testbenches ^[4] and the leading standard for FPGA testbenches, with the VHDL-oriented OSVVM and UVVM gaining traction since tracking began in 2018 and Python-based approaches newly measured ^[5]. For an AXI4 interface on a RISC-V SoC, a UVM agent provides suppliers, partners and new hires with a standardized artifact to integrate.

10.2 The component taxonomy and why it exists

Chapter 8 argued that the reusable unit of verification is *one interface*: the protocol is the boundary that survives integration, since AXI4 (AMBA® AXI and ACE Protocol Specification, Arm IHI 0022H.c, 2021 [6]) does not change when the crossbar is dropped into the SoC. UVM implements this boundary with a base class and one mode flag. `uvm_agent` groups a sequencer, a driver and a monitor for one interface, and `get_is_active()` returns the mode the configuration database set for it, defaulting to active [1]. Active mode is where the agent emulates a device and drives design signals, and it requires a driver and a sequencer; passive mode disables their creation and leaves the monitor. The rule of thumb: one active agent per device that must be emulated, one passive agent per RTL device that must be watched [3]. This flag enables second-order reuse without rewriting the agent.

```
class axi_agent extends uvm_agent;
  `uvm_component_utils(axi_agent)

  axi_agent_cfg  cfg;
  axi_sequencer  seqr;
  axi_driver     drv;
  axi_monitor    mon;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon = axi_monitor::type_id::create("mon", this);
    if (get_is_active() == UVM_ACTIVE) begin
      seqr = axi_sequencer::type_id::create("seqr", this);
      drv = axi_driver::type_id::create("drv", this);
    end
  endfunction

  virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    if (get_is_active() == UVM_ACTIVE)
      drv.seq_item_port.connect(seqr.seq_item_export);
  endfunction
endclass
```

Sub-components are built in `build_phase`, not in the constructor, because a virtual function can be overridden without restriction while a constructor cannot. They are created through `type_id::create` rather than `new`, which is what makes section 10.3's overrides possible. And the driver-to-sequencer link is made in `connect_phase`, after all building is finished [3].

The environment composes interrelated agents, scoreboards or other environments into a hierarchical component without adding new behavior. A scoreboard is the structure holding what the design is expected to produce, consulted when an output is observed (Chapter 8). The environment's own configuration fields are structural ones, of the same kind as `is_active`: how many manager agents, how many subor-

dinate agents, whether a bus-level monitor exists ^[3]. Figure 10.1 applies that layering to the crossbar: what the test owns, what the environment owns, and which components actually reach the design.

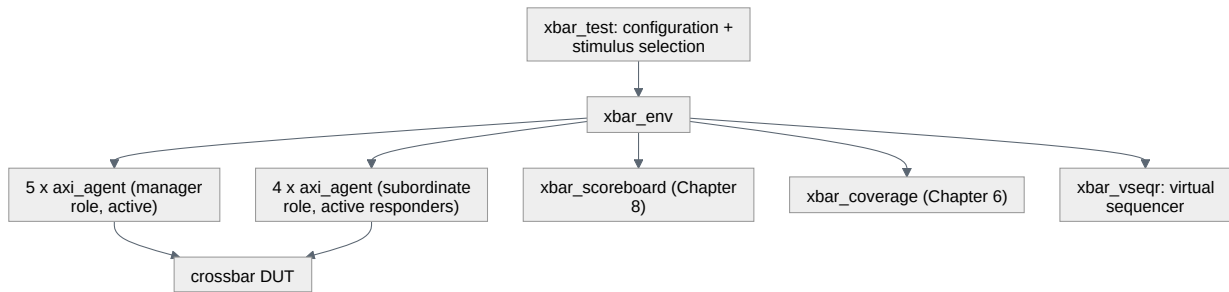


Figure 10.1 — The crossbar testbench as a UVM component hierarchy: the test carries configuration and stimulus selection, and the environment below it composes nine agents, five in the manager role and four responding, with the scoreboard, the coverage collector and the virtual sequencer. Only the agents reach the design; every layer above them is composition rather than new behaviour. Notice that the scoreboard and the coverage collector hang off the environment and not off the test, because whatever a test instantiates for itself is absent the next time the environment is reused.

Scenario. The crossbar connects five manager ports (core-I, core-D, the two DMA channels, debug) to four subordinate ports (SRAM, boot ROM, APB bridge, neural accelerator). **Approach.** One `axi_agent` type is instantiated nine times: five initiators and four responders, the agents that answer requests initiated by the design. **Why.** Nine instances of one class exercise the same driver and monitor nine ways per regression; in this example, the crossbar’s own tests find an agent protocol bug rather than leaving it for the DMA team to find three months later. It also means the ID-routing coverage model of Chapter 6 has exactly one place to sample from.

Above the environment sits the test, whose job is narrow: instantiate the top-level environment, configure it through factory overrides or the configuration database, and apply stimulus by invoking sequences. The recommended pattern is one base test that instantiates and wires the environment, with every real test extending it to configure differently or select different sequences ^[3]. A test that instantiates components of its own has taken structure out of the environment, and that structure will not be there when the environment is reused.

The monitor broadcasts observed transactions on an analysis port and may process them itself or delegate to components connected to that port ^[3]. An analysis port is a broadcast to any number of subscribers, none of which may block the monitor or change what it receives. UVM leaves consumers of the analysis port to the project: the scoreboard is Chapter 8’s subject, and the coverage collector implements Chapter 6’s model; section 10.9 develops this boundary.

10.3 The factory and the configuration database

The structure in section 10.2 requires replaceable component types. If an environment hard-codes `axi_driver`, a test needing different driver behavior must edit the environment, preventing unmodified reuse.

Before UVM, an aspect-oriented verification language of the late 1990s solved this problem in the language itself: Verity's `le` let independent groups add features to an existing environment by extending already-compiled types from separate files; its designers noted both that the object-oriented alternative, the Abstract Factory pattern, was cumbersome, because of the profusion of subclasses it required, and that the underlying difficulty is *influencing what class gets created* [7]. SystemVerilog lacks aspect weaving, so UVM implements that factory and asks developers to route every construction through it.

The mechanism is an override, by type or by instance. Components requested from the factory are resolved at construction time: the factory first searches for an instance override matching the object's full instance name, then for a type-wide override, and failing both creates the requested type; instance overrides take precedence over type overrides [1]. A replacement must be polymorphically compatible, including having the same TLM interface handles [3]. TLM is the transaction-level modeling named above: components pass whole transactions over standard interface handles rather than signals. Because resolution happens when the child is constructed, an override takes effect only if the parent registers it before building its children, which is why environment overrides belong in the test.

```
class xbar_stall_test extends xbar_base_test;
  `uvm_component_utils(xbar_stall_test)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    // Replace the driver of manager port 3 only; the other eight agents
    // keep the standard driver. Done before super.build_phase() builds env.
    set_inst_override_by_type("env.mgr_agent[3].drv",
                              axi_driver::get_type(),
                              axi_stall_driver::get_type());

    super.build_phase(phase);
  endfunction
endclass
```

That test provokes the crossbar's canonical bug, the write-data interleaving at a subordinate port under back-pressure, without touching one line of the environment, the agent or the driver it extends. A canonical bug is the book's recurring defect for one block, returned to whenever that block is the example. A type override would instead make all nine agents stall, applying a general policy rather than targeting the instance in this scenario.

Scenario. The USB controller is a dual-role part (USB 3.2 Revision 1.1 [8]): it must be verified as a device and as a host. **Approach.** One agent, one monitor, one sequence item; two drivers, selected by a type override in two base tests. **Why.** Both roles share the pin-level protocol and transaction abstraction; only the initiative differs. The override shares the monitor, coverage model and register model between the two campaigns, implementing section 10.1's third-order reuse within one project. For the flash controller's error-injection hooks, an ECC-corrupting driver overrides the clean driver in data-integrity tests without downstream changes.

The configuration database lets an integrator configure an environment without knowing its implementation or hook-up scheme [3], using typed values set against hierarchical instance-path patterns:

```
uvm_config_db#(virtual axi_if)::set(this, "env.mgr_agent[3].*", "vif", mgr_vif);
uvm_config_db#(int)::set(this, "env", "num_mgr_agents", 5);
// The subordinate agents answer the crossbar's requests under testbench
// control, so by Chapter 8's criterion they are drivers, not observers.
uvm_config_db#(uvm_active_passive_enum)::set(this, "env.sub_agent*", "is_active", UVM_
ACTIVE);
```

All three are written from the test, where `this` is the top; a component deeper in the tree sets only relative to itself, which is why paths written here break when the environment moves down a level, at SoC level, where `is_active` becomes `UVM_PASSIVE` for all nine agents (section 10.8).

Two conventional monitor parameters, `checks_enable` and `coverage_enable`, should be adopted throughout testbenches because tools and colleagues expect them: integrators can disable checking or coverage without edits [3], avoiding every violation being reported twice when a bound assertion module also checks the interface (Chapter 11).

Both mechanisms replace a compile-time reference with a runtime lookup, so both defer their failures to runtime and report them somewhere other than where the mistake was made. A typo in a `set` path pattern silently leaves the default unchanged; in this example, a null virtual interface exposes the failure a hundred microseconds later. The key `vif` written into the configuration database above carries a virtual interface, which is a reference to an interface instance and the way class-based code reaches the design's signals. An override registered after `super.build_phase` is ignored. Moving an environment one level down invalidates an instance path without a diagnostic. Indirection enables reuse but makes UVM harder to debug than the flat testbench it replaces. The standard override- and configuration-audit facilities should be primary debug tools, and the configuration surface should have one owner, as a module's port list does.

10.4 Phasing: why a testbench needs a state machine

Every simulation performs the same overall sequence of steps to reach completion. Formalizing those steps rather than inventing a synchronization scheme per environment lets different block-level environments be combined into a system-level one, and lets different tests intervene at the right moment without violating the sequence [2].

The standard splits phases into two families. The *common* phases are executed together by every component in the testbench, which is what makes them a synchronization mechanism; the *run-time* phases execute on a schedule that runs concurrently with the run phase. Every common phase before the run phase executes at simulation time zero [1].

Three phases establish the basic execution order:

- `build_phase` is a **top-down** function phase [1]. A parent must run before its children exist because it decides how many agents to create and whether they are active or passive.
- `connect_phase` is a **bottom-up** function phase [1]. Bottom-up because wiring can only happen once both endpoints have been constructed, so the deepest, most local connections are made first.
- `run_phase` is a task phase concurrent with the run-time schedule from `pre_reset` through `post_shutdown`; it starts a global timeout whose expiry is fatal [1], failing a hung testbench before it occupies a license overnight.

Objections determine test completion separately from phase ordering. A component or sequence raises a phase objection at the start of an activity that must complete before the phase may stop, and drops it at the end; a task-based phase persists only while an objection is raised, and it ends once every component has dropped its own [1]. An active manager agent must finish its reads and writes, whereas a reactive subordinate merely serves arriving requests and should therefore not object [3].

Scenario. A crossbar test runs traffic from all five manager agents while the four subordinate agents respond. **Approach.** Each manager sequence objects while it runs; the subordinate agents never object; the environment sets a small drain time. **Why.** With no objections at all the run phase would end at time zero. With the subordinate agents objecting, the test would never end, because a reactive responder is never finished. Without drain time, the final manager objection drops while the last write response still crosses the crossbar, so the scoreboard misses it; the “all objections dropped” condition is temporary. Two mechanisms close that window: a drain time set once at the environment or test level, or a `phase_ready_to_end` implementation that re-raises while a transaction is in flight; the drain also supports vertical reuse, because a subsystem developer declares the drain that subsystem needs, and that drain still applies when that subsystem’s environment becomes one of six such environments inside a larger one [3]. Vertical reuse is second-order reuse: the components of a block-level environment carried up into a system-level one.

Reading a sibling's handle in `build_phase` returns null because bottom-up wiring has not happened; the code belongs in `connect_phase` [3].

10.5 Sequences and the sequencer-driver handshake

A sequence is an object containing behavior for generating stimulus, deliberately outside the component hierarchy. Components are structural and persist for the whole simulation; sequences are transient (created, used, and garbage collected when dereferenced), and one may exist for a single transaction or for the whole run. Structural stimulus would bind a scenario to the reusable environment, because such stimulus is built into the component hierarchy itself. Sequences compose instead: a parent invokes children, each is bound to a sequencer when it starts, and many may be bound to the same sequencer [3].

The driver's `seq_item_port` connects to the sequencer's `seq_item_export`; the sequencer produces items, the driver consumes them and optionally returns responses, and the component containing both makes the connection [3]. The handshake uses a pull because only the driver knows when the interface is ready: it synchronizes to the bus and requests the next item.

Because the driver handles one item at a time while several sequences may be running, the sequencer maintains a queue of pending requests and chooses one each time the driver asks. The choice is the arbitration policy; the available modes are FIFO, weighted, random, strict FIFO, strict random, and a user-defined hook. The default is plain FIFO, which grants in arrival order; only the two strict modes grant the highest-priority request first, while the weighted and random modes grant randomly, the weighted one by weight [9]. A test requiring deterministic priority handling needs a strict mode. Two further controls exist for scenarios that need exclusivity: `grab/ungrab`, which give one sequence exclusive use of the sequencer (the canonical interrupt-service pattern), and plain prioritization, which is less disruptive [3]. Prioritization works through the arbitration modes above instead of taking the sequencer away from the other sequences.

Virtual sequences coordinate several interfaces at once. A virtual sequencer holds references to its sub-sequencers and executes sequences on them; it has no data item of its own and drives nothing directly. Already written and debugged block-level sequence libraries thus compose system-level scenarios [3].

```
class xbar_w_interleave_vseq extends uvm_sequence;
  `uvm_object_utils(xbar_w_interleave_vseq)
  `uvm_declare_p_sequencer(xbar_vseqr)

  function new(string name = "xbar_w_interleave_vseq");
    super.new(name);
  endfunction

  virtual task body();
    axi_wr_burst_seq wr_a, wr_b;
    axi_backpressure_seq bp;
    wr_a = axi_wr_burst_seq::type_id::create("wr_a");
```

```

wr_b = axi_wr_burst_seq::type_id::create("wr_b");
bp   = axi_backpressure_seq::type_id::create("bp");
fork
  wr_a.start(p_sequencer.mgr_seqr[1], this); // core-D -> SRAM, AWID 3
  wr_b.start(p_sequencer.mgr_seqr[2], this); // DMA ch0 -> SRAM, AWID 3 too
  bp.start(p_sequencer.sub_seqr[0], this);  // SRAM port stalls WREADY
join
endtask
endclass

```

Passing `this` as the second argument makes the three streams *children* of the virtual sequence rather than three unrelated roots: the parent's `pre_do`, `mid_do` and `post_do` hooks fire during execution, and each child inherits its priority instead of the root default of 100 ^[1]. Those three hooks are methods a sequence can implement to act at fixed points around each child it starts.

The crossbar's canonical write-data interleaving bug at a subordinate port under back-pressure (Arm IHI 0022H.c §A5.2.2 ^[6]) requires a virtual sequence coordinating two managers and a subordinate doing three things concurrently. No single-interface sequence can state it. Two of those three carry the hazard: concurrent writes to one subordinate, and a stall on its write-data channel. The shared `AWID` tests a separate issue: the crossbar widens IDs on the way out, so the two writes reach the subordinate under *different* IDs. A scoreboard keyed on the subordinate side breaks on that widening (Chapter 8), while the interleaving check does not depend on the distinction.

Chapter 9 covers sequence contents: constraints, distributions and layered scenario descriptions. UVM provides only stimulus lifetime, binding and arbitration.

10.6 The register layer

Registers connect protocol accesses to behavior specific to the design. The register layer builds a high-level, object-oriented model of the memory-mapped registers and memories, abstracting reads and writes so that environments and tests migrate from block to system level without modification. The register abstraction layer (RAL), as the model is known, mirrors the design hierarchy: blocks contain registers, register files and memories, while registers contain fields ^[3].

Mirrored versus desired. The model maintains a *mirror* of what it believes each register currently holds. The mirror is not guaranteed correct, because the model's only information is the accesses it has seen: if the design changes a field on its own, by setting a status bit or incrementing a counter, the mirror goes stale. After every read it is updated from the observed data; after every write it is predicted from the field's access policy, such as read-only or write-one-to-clear ^[3]. The *desired* value is a separate quantity: it equals the mirrored value until `set` changes it in the model alone, in zero time, and `update` then pushes it to the design, issuing bus cycles only for the registers that need updating ^[1], so configuring a 40-register block costs only the required writes rather than forty.

A mirror can predict registers the design does not update, but cannot judge whether an updated value is correct; it therefore cannot serve as a scoreboard [3]. A model and design can agree on a value both got wrong.

Front door and back door. A front-door access drives real bus cycles through the real path; a back-door access reaches the simulation constructs directly through a hierarchical path in the design. Back-door read and write *mimic* the front-door side effects: a back-door read sets clear-on-read bits to zero exactly as a physical read would, and a back-door write leaves read-only bits alone, whereas `peek` samples the register without modifying it and `poke` deposits a value as is, bypassing the behavior entirely [1]. Independent paths provide redundancy, a second independent route to the same value. When both directions share one path, a bug introduced on the way in is undone on the way out, so a reversed data bus is invisible to any write-then-read check that never leaves the front door. Identifying and maintaining hierarchical paths is the main difficulty of back-door access [3]; they break every time the design hierarchy is refactored.

Worked example: the peripherals' status register. The UART's receive-overflow flag is a one-bit, write-one-to-clear, volatile field: hardware sets it, software clears it by writing a one.

```
``systemverilog class uart_status_reg extends uvm_reg; rand uvm_reg_field oe; //
RX overrun error

function new(string name = "uart_status_reg"); super.new(name, 32, 0); endfunction

virtual function void build(); oe = uvm_reg_field::type_id::create("oe"); // parent
size lsb access volatile reset has_rst rand indiv oe.configure(this, 1, 3, "W1C", 1,
1'b0, 1, 0, 0); endfunction endclass ``
```

`build()` here is the generator's convention, not a UVM phase: the enclosing block calls it after creating the register, then calls the register's own `configure` to attach it to an address map, left out here to keep the fragment on the field.

Two arguments govern volatility and access behavior. `volatile` set to one declares that the mirrored value cannot be trusted, because the field can change in a way the model has no way to observe; the definition is borrowed from the IP packaging standard (IEEE Std 1685-2022, *IEEE Standard for IP-XACT*), where a volatile element gives no guarantee about what a read returns after a write or on the second of two consecutive reads [1, 10]. The access policy "W1C" means a written one clears the matching bit and a read has no effect. The `volatile` setting is a statement about the mirror: comparing a volatile field against the mirror without a fresh read is not a check, because nothing entitles the model to believe the value it last saw. The "W1C" policy is a statement about prediction: if the real design *also* clears on read, the register is modeled incorrectly, the mirror diverges on every read, and the model rather than the design gets blamed. That policy belongs to a family of short access-policy codes, each of which fixes what a read and a write do to a field and what the model may predict from them. The `rand` qualifier on the declaration is the generator's habit: `is_rand` is passed as zero, and for a predefined access policy outside the six the class reference lists ("RW" , "WRC" , "WRS" , "WO" , "W1" , "W

01", which exclude "W1C"), the library ignores `is_rand` and turns the field's randomization off in any case ^[9]. The two agree: the argument records intent, while the access policy enforces it regardless of that argument; changing only the argument would not re-enable randomization.

In the canonical UART bug, the overrun flag never clears if software reads status during a stop-bit error cycle; the two access paths disagree: a front-door read followed by the clearing write leaves the model believing the flag is down, while a back-door peek shows it still up. A two-line register test using "read front door, peek back door, compare" exposes the disagreement that causes a firmware lockup.

That test, with the field of the register above: one front-door read through the bus, one back-door peek through the hierarchical path, and the comparison that separates them.

```
// The two-path check of the paragraph above, on the register class shown
// there: one front-door read, one back-door peek, one comparison.
class uart_status_2path_seq extends uvm_reg_sequence;
  `uvm_object_utils(uart_status_2path_seq)

  uart_status_reg status; // the register model, with its oe field

  function new(string name = "uart_status_2path_seq");
    super.new(name);
  endfunction

  virtual task body();
    uvm_status_e s;
    uvm_reg_data_t front, back, predicted;
    int unsigned oe_pos;

    oe_pos = status.oe.get_lsb_pos();

    // What software does: read the status word, then clear the flag by
    // writing back the one it just read. Both accesses drive real bus
    // cycles, here through the APB bridge.
    status.read(s, front, UVM_FRONTDOOR, .parent(this));
    status.write(s, front, UVM_FRONTDOOR, .parent(this));

    // What the model believes after that write, from the W1C policy alone.
    // A field's mirrored value is right justified, so its bit 0 is the
    // flag, and it is captured before the peek, which predicts the mirror
    // in its turn.
    predicted = status.oe.get_mirrored_value();

    // What the design holds: peek reaches the field through its
    // hierarchical path and leaves it as it is, so the two paths share
    // nothing that a bug on one of them could undo on the other.
    status.peek(s, back, .parent(this));

    if (back[oe_pos] != predicted[0])
      `uvm_error("UART_OE", $sformatf(
        "overrun flag: the model predicts %0b after the clearing \
write, the design holds %0b", predicted[0], back[oe_pos]))
  endtask
endclass
```

Where the model comes from. Given the number of registers in a real design and the number of details in configuring these classes, the specialization is normally done by a model generator working from a specification, which yields an up-to-date, correct-by-construction model; generators are explicitly outside the library's scope [3]. That gap is filled by machine-readable register description. Accellera's *System-RDL 2.0 Register Description Language*, 2018, provides a machine-readable single source for every view, from specification through model generation and verification to maintenance and documentation; its stated motivation is tedious, error-prone change propagation when many people replicate information in many places. Field behavior is expressed directly as language properties, including clear-on-read and write-one-to-clear [11], making the `configure` call correct by construction rather than transcription. The IP packaging standard plays the complementary role: an XML schema for IP meta-data used in development, implementation and verification, in which each target interface carries a memory map of address blocks, registers and register files [10].

Scenario. The Ethernet MAC's time-sensitive networking queues expose several hundred per-queue gate-control and shaper registers, and the specification is still moving. **Approach.** The model should be generated. One description generates the design's decode logic, the driver's C header and the register model; a field width changes in all three at once. **Why.** This prevents silent version skew: model, firmware header and design each encode a different map revision, with the mismatch debugged three times by three teams.

Once the model exists, the library supplies predefined register sequences for hardware reset, bit-bashing, front-door/back-door access checks, memory walking-ones and checks of declared hierarchical paths [1]. Bit-bashing writes and reads back the bits of a register in turn, checking that each bit behaves as the access policy declared for its field requires. The user guide recommends starting with the hardware-reset test because it debugs the model, environment, transactors and design together at the lowest cost [3]. A transactor is the component that converts between pins and transactions, so that everything above it speaks transactions.

Implicit prediction attaches the model to a bus sequencer and updates the mirror after each operation issued by the model; it is simple but misses accesses it did not originate. *Explicit* prediction adds a predictor per interface, fed by the bus monitor, which looks up the register at every observed address and updates its mirror; it is more complex, but it captures all register activity, including accesses the model did not originate [1]. *Passive* integration connects the monitor only: the model tracks and checks but cannot drive [3]. On the reference SoC, once firmware running on the core writes the peripherals through the crossbar and the APB bridge, an implicitly predicted model is wrong within microseconds, because those writes never went through it.

10.7 Reporting as infrastructure

The reporting classes issue reports with consistent formatting and filter them by verbosity; reports travel from the component through a report handler to a central report server. The reporting classes also allow the action taken and the file written to be configured by severity, by message ID, or by severity and message ID together. The default actions encode a policy: informational and warning messages are displayed, an error is displayed and *counted*, a fatal message is displayed and exits [1].

Error counting lets a component fail a test without knowing how pass and fail are decided, which lets messages from components reused from different sources be displayed and controlled consistently without modifying the reused component itself [2]. In the SoC environment, the crossbar agent and a purchased memory-controller component share one verbosity setting, and one error from either fails the run.

Message IDs should form a schema. The regression triage of Chapter 12 groups failures by ID; an agent that reports every protocol violation under the ID `AXI4` gives the triage script exactly one bucket, while an agent reporting `AXI4_WSTRB`, `AXI4_ORDER` and `AXI4_4KB` gives it a first-pass classifier.

10.8 Reuse in practice: the same agent, one level up

The crossbar environment of section 10.2 is now instantiated inside the SoC environment, with the real core, the real DMA engine and the real debug module (RISC-V Debug Specification v1.0 [12]) driving the manager ports and the real memories answering. Every one of the nine agents must therefore only observe, using section 10.2's passive mode [3]. One configuration field implements this change, but code assuming the environment drives the interface breaks.

- **Stimulus becomes dead code.** The manager sequences have nothing to drive. Checking in the driver disappears with it, so checking belongs downstream of the monitor.
- **Configuration paths move.** `env.mgr_agent[3].*` is now `soc_env.xbar_env.mgr_agent[3].*`. Every absolute path from the old top-level test silently becomes invalid. Each component should set configuration only relative to itself.
- **Objections change owner.** The manager agents no longer object, so a SoC test that inherited its end-of-test from them ends at time zero; the drain time that suited one crossbar is short when a response crosses two more layers, which is why the drain is declared per subsystem [3].
- **The virtual sequencer holds null handles.** `xbar_vseqr` points at sequencers that a passive agent never built, so the virtual sequencer must be guarded or not built at all.
- **Block-local expectations expire.** The scoreboard assumed every transaction on a manager port was one the environment generated. Now they come from firm-

ware, and the predictor needs a different source of truth (Chapter 8). The register model has the same problem and the same fix: explicit prediction.

The unchanged interface preserves the agent, while code assuming a driving *role* fails. A configurable component meets block-level and system-level needs without modification ^[2]; every failure above exposes a component meeting only the block-level half.

10.9 What UVM does not solve

UVM defines APIs and a base class library for modular verification components that can be scaled and reused ^[1]. Four responsibilities remain outside the library, all four affecting project success.

The checking strategy. The monitor broadcasts, but UVM leaves consumers and their conclusions unspecified; it only observes that different methodologies exist for scoreboards and reference models ^[3]. A reference model is an independent executable of the specification, taken as golden, run on the same stimulus and compared with the design. The protocol specification and verification plan identify write-data ordering under back-pressure as the crossbar property to check; UVM does not select it. The plan specifies both verification process and supporting testbench infrastructure ^[13]; the framework supplies the infrastructure, while the project still has to define the process.

The quality of the coverage model. UVM decides where a coverage collector plugs in, not what it counts. Wrong coverage axes make 100% uninformative (Chapter 6); the neural accelerator’s canonical prefetcher bug escaped 100% code coverage entirely, which a methodology change could not remedy.

Debuggability. Section 10.3 describes how indirection converts compile-time errors into silent runtime failures. An illustrative comparison assumes that a hand-written testbench of a thousand lines is easier to debug than a UVM environment of a thousand lines, while at ten thousand lines the comparison reverses.

Performance. Per-item object allocation, dynamic dispatch and a report server on the message path consume runtime, and dynamic dispatch means that each call finds the method body of the object’s actual type while the simulation runs rather than at compile time. For the runtime comparison, the assumed overhead is negligible on a block-level run of ten thousand cycles but material in a nightly regression (Chapter 12). At the boundary with emulation, transactor code must be synthesizable and a UVM component is not (Chapter 17).

UVM provides reusable testbench infrastructure; the project still decides what to check and establishes that those checks ran.

10.10 When not to use UVM

Before the first transaction, UVM incurs a fixed setup cost: a sequence item, an agent, a configuration object, factory registration and a phasing discipline. The following three situations illustrate where that setup cost may outweigh the reuse benefit.

Small blocks with one narrow interface. A timer on the APB subsystem, a two-flop synchronizer, a small FIFO: one interface, no second consumer, forty lines of directed stimulus. For the small-block example, a module-based testbench with bound assertions (Chapter 11) and one covergroup is assumed to need less setup than UVM and to remain easier to read a year later.

Formal-first blocks. Arbiters, round-robin schedulers, handshake converters, and the crossbar's own ordering rules. When the specification is a set of properties and the state space is small enough to be searched exhaustively (Chapter 14), proof establishes what constrained-random stimulus only samples. The environment should cover what formal cannot reach, including long traffic mixes, integration and firmware-driven scenarios, while proof covers the protocol rules.

Prototype and bring-up testbenches. Bring-up is the phase in which environment and DUT first exchange checked transactions. A bring-up testbench may need to deliver a waveform tomorrow without becoming an asset for the next project. The bring-up testbench needs maintenance only if the project retains it for further use.

VHDL-centered and Python-centered flows have their own methodologies, both gaining ground in the 2024 FPGA study [\[5\]](#); methodology should follow the team's verification language.

When an interface will be verified more than once, in a second test, hierarchy level or project, the expected reuse should be weighed against UVM's setup and debugging costs. If none of the three applies, reuse does not justify the investment.

IN PRACTICE

For standard protocols, teams should obtain an existing verification component and develop the configuration, sequences, scoreboards and coverage models specific to their project. A house layer between the library and its environments can encode local conventions through a base test reading command-line arguments, a standard configuration object and a logging convention. That layer should be reviewed first on a new project because it silently constrains everything above it. Some projects publish the conventions that govern that layer: lowRISC publishes a SystemVerilog coding style guide written for design-verification code [\[14\]](#), and the OpenHW Group publishes SystemVerilog and UVM coding guidelines for its CORE-V verification code, a text it adopted from an outside author and republished under the project's license [\[15\]](#).

Continuous integration should regenerate register models on every commit from the description shared with decode logic and firmware headers; weekly regeneration can

leave a model outdated for six days. Configuration failures include a `set` that never matched or an override registered too late.

PITFALLS

1. **Sub-components built in the constructor.** Symptom: a factory override has no effect. Cure: build in `build_phase`, construct with `type_id::create`.
2. **Absolute configuration paths written from the top-level test.** Symptom: the environment works standalone and silently misconfigures after integration. Cure: every component sets configuration relative to itself.
3. **Sequence priorities with the default arbitration mode.** Symptom: the interrupt sequence waits behind mainline traffic. Cure: select a strict arbitration mode, since only the two strict modes grant the highest-priority request first [9].
4. **Checking implemented inside the driver.** Symptom: all protocol checking disappears when the agent is made passive. Cure: check downstream of the monitor's analysis port.
5. **Implicit register prediction in an environment where firmware also writes registers.** Symptom: the mirror diverges silently and the model is blamed for design bugs it did not see. Cure: explicit prediction with a predictor per bus interface [3].
6. **Drain time increased until the test passes.** Symptom: a green regression that hides a scoreboard which stopped draining. Cure: re-raise the objection while a transaction is genuinely in flight [3].

SUMMARY

- UVM exists to make verification components reusable across testcases, across levels of hierarchy and across companies; its stated purpose is interoperability and the removal of the cost of rewriting verification IP for every new project [1].
- The agent is the reusable unit because an *interface* is the stable boundary; the active/passive switch is what makes block-to-system reuse mechanical [3].
- The factory and configuration database enable reuse through indirection, which increases debugging time.
- Phasing is a synchronization contract that lets independently developed environments combine; objections, not phases, decide when a test ends [2, 3].
- Sequences are deliberately outside the component hierarchy, keeping stimulus separate from structure; virtual sequences express scenarios involving multiple interfaces [3].
- A register mirror is not a scoreboard, and the model should be generated from a machine-readable register description (Accellera SystemRDL 2.0) rather than transcribed [3, 11].
- The framework supplies no verification plan and leaves checks, coverage models and risk priorities to the project.

EXERCISES

Exercise 10.1 (calculation). The crossbar environment of §10.2 instantiates one agent class nine times, five in the manager role and four responding, and §10.3 configures every one of them active. Compute the number of sequencers, drivers and monitors the environment builds at block level, and the number it builds once §10.8 makes every agent passive. Then compute how many driver instances the instance override written in §10.3 replaces at block level, how many a type override of the same pair of types would replace there, and how many each of the two replaces after the agents become passive. Name the property of factory resolution that §10.3 states, and which makes the last pair of answers zero.

Exercise 10.2 (judgement). An interrupt-service sequence is started on a manager sequencer of §10.5 while three mainline traffic sequences are already running on it, is given a higher priority than any of them, and is still granted last. Name the arbitration mode the sequencer is using and the property of that mode that produces the delay, and name the two modes §10.5 identifies as the ones that grant the highest-priority request first. Rank the two further controls §10.5 offers for this scenario by how much each disturbs the other sequences, and give the argument for the ranking. State also the priority carried by a sequence started as a child of a virtual sequence, in place of the root default of 100.

Exercise 10.3 (calculation). §10.6 configures a block of forty registers through the model instead of through forty writes, because `set` moves the desired value in the model alone and `update` afterwards issues bus cycles only where they are needed. Compute the number of front-door write cycles `update` issues in each of the two extreme cases that rule admits, and name the comparison that fixes the count between them. Then take one field of that block declared volatile and state which of those write cycles the model can wrongly suppress, naming the property of the mirror that lets it.

Exercise 10.4 (judgement). A block-level UART environment predicts its register model implicitly and closes each test by comparing the overrun field against the mirror without a fresh read, and that environment is now instantiated inside the SoC environment of §10.8, where firmware running on the core writes the peripherals through the crossbar and the APB bridge. Name the prediction mode the register model needs at the higher level and the component that mode adds, and give the two distinct defects in the block-level check, pointing to the section that carries each. Name the reason §10.6 gives for refusing the mirror the role of scoreboard. State also what §10.5 says a scoreboard keyed on the subordinate side of the crossbar loses when two managers write under one identifier, and which of the crossbar's checks is unaffected by that loss.

Exercise 10.5 (artifact). Read the field descriptor below, which records the receive-overflow field of the peripherals' status register of §10.6 with its access policy left empty. Supply the policy §10.6 fixes for that field, and give the two separate statements §10.6 reads out of the descriptor, one about the mirror and one about prediction. Name the argument the library ignores for that policy, give the reason, and name the two-path register test §10.6 builds to expose the canonical UART bug.

```

uvm_reg_field:      oe (receive-overflow error)
  parent:           this
  size:             1
  lsb:              3
  access:
  volatile:         1
  reset:            1'b0
  has_reset:        1
  is_rand:          0
  individually_accessible: 0

```

FURTHER READING

From the corpus. The standard [\[1\]](#) defines phases, factory semantics, reporting and register classes; its introduction summarizes UVM's origins. The user guide [\[3\]](#) is the companion explaining the recommended structure; the class reference [\[9\]](#) pins down access policies and arbitration modes. For the reasoning *behind* the structure, the methodology ancestor [\[2\]](#) explains the orders of reuse and why messaging and simulation-step formalization are reuse mechanisms; the testbench-writing companion [\[13\]](#) frames the verification plan as the specification the framework serves. The lineage of extension-as-configuration is in the Verisity e-language paper [\[7\]](#). For register generation, the register description language [\[11\]](#) covers behavior, while the IP packaging standard [\[10\]](#) covers the address map. Adoption data is in the 2024 trend reports [\[4, 5\]](#). The *UVM Cookbook* published by Siemens EDA's Verification Academy supplies worked patterns for the components this chapter names: it describes the driver as the component that turns a sequence item into pin-level activity across a virtual interface, while a monitor only observes and puts no value on any signal of the interface, so it keeps running when its agent is configured passive [\[16\]](#). The corpus also holds open-source design-verification style guides from RISC-V hardware projects, cited in *In practice*, which complement vendor material.

REFERENCES

1. [S9](#) IEEE, "IEEE Std 1800.2-2020, IEEE Standard for Universal Verification Methodology Language Reference Manual (Revision of IEEE Std 1800.2-2017)," IEEE, 2020.
2. [B1](#) J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
3. [S11](#) Accellera Systems Initiative, "Universal Verification Methodology (UVM) 1.2 User's Guide," Accellera, October 2015.
4. [R2](#) H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.

5. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
6. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
7. **P18** Y. Hollander, M. Morley, and A. Noy, "The e Language: A Fresh Separation of Concerns," Proc. Technology of Object-Oriented Languages and Systems (TOOLS 38), Zurich, Switzerland, pp. 41–50, 2001.
8. **S21** USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.
9. **S4** Accellera Systems Initiative, "Universal Verification Methodology (UVM) 1.2 Class Reference," Accellera, June 2014.
10. **S7** IEEE, "IEEE Std 1685-2022, IEEE Standard for IP-XACT, Standard Structure for Packaging, Integrating, and Reusing IP within Tool Flows (Revision of IEEE Std 1685-2014)," IEEE, 2022.
11. **S2** Accellera Systems Initiative, "SystemRDL 2.0 Register Description Language," Accellera, January 2018.
12. **S16** T. Newsome and P. Donahue, "The RISC-V Debug Specification, Version 1.0 (Ratified), Revised 2025-02-21," RISC-V International, February 2025.
13. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
14. **O9** lowRISC, "lowRISC SystemVerilog Coding Style Guide for Design Verification," lowRISC/style-guides, DVCodingStyle.md, commit 9c15ff5dce23eef969e00ab3715153967419eadd, accessed 2026-09-02.
15. **O3** OpenHW Group, "CORE-V-VERIF Coding Guidelines for SystemVerilog/UVM code," openhwgroup/core-v-verif, docs/CodingStyleGuidelines.md, commit f3b1f971e0e6b94deae46d279cc50ca390785369, accessed 2026-09-02.
16. **B8** Functional Verification Methodology Team, "UVM Cookbook," Siemens EDA Verification Academy, 2021 (revised 31 January 2022).

11. Assertion-Based Verification

Assertion-based verification expresses design rules as executable properties; this chapter explains their semantics and placement, how to assess their quality, and their use in simulation and formal proof. The opening example concerns a rule decidable from seven signals on one interface, which means those signals carry everything a check needs in order to judge it. In this example, the scoreboard reports a mismatch on the reference SoC: seed 4471 of the nightly regression copies 3 KB through a DMA channel into the neural accelerator’s weight buffer, with eleven bytes at the wrong offset. The scoreboard is the structure that holds the data expected on an output and is consulted when that output is observed. A DMA channel is an independent transfer engine with its own registers and its own state, and the reference SoC’s DMA has two of them, sharing one AXI backend. Two engineers spend a day and a half tracing the waveform backward because the scoreboard reports at transfer completion, some nine hundred cycles after the damage, requiring the entire interval to be retraced. The cause spans four cycles. The DMA offers an address with `AWVALID` on the crossbar port, and the subordinate delays acceptance. While that acceptance is pending, arbitration inside the DMA withdraws the address and offers a *different* one, and all of this happens before `AWREADY` returns. The subordinate had already latched the first one.

The failure is easy to detect but difficult to locate. The broken rule occupies one sentence of the bus specification: an offered address must remain offered and unchanged until accepted, and a check on the interface can detect the violation in the cycle it occurs. The missing check was an assertion stating that rule so a tool could evaluate it at the port.

The same assertion text can be sampled by a simulator and proved by a formal engine in Chapter 14 without changing a character.

CHAPTER MAP

§11.1 defines assertions as executable specifications and explains the precision required to write them. §11.2 develops SystemVerilog Assertions (SVA): sampled values and evaluation attempts, sequences and implication, clocking and `disable iff`, and concurrent versus immediate assertions. §11.3 explains the three claims `assert`, `assume` and `cover` make about one property and their meanings in simulation and proof. §11.4 covers placement and ownership, inline in RTL or in a `bind-ed` checker. §11.5 works through the crossbar’s AXI4 contract, the machinery each clause needs and the limits of passing checks. §11.6 covers implementation invariants absent from specifications, especially agreement between redundant state. §11.7 examines assertion quality through vacuity, density and inability to fail. §11.8 explains how to interpret proof results when assumptions stop being checked, covers search rather than count, and liveness becomes decidable.

11.1 Assertions as executable specifications

An assertion states a property of the design in positive form: what must hold, so that its falsehood indicates an error. An assertion is *declarative*: it specifies what to check and leaves the checking mechanism to the tool ^[1]. The declarative form distinguishes an assertion from a hand-written procedural checker, and it lets a single property of three lines serve a simulator, an emulator and a model checker without being rewritten for any of them. An emulator is a purpose-built machine that runs the design compiled onto a hardware fabric, and a model checker is a proof engine, the tool that decides a property over every legal execution instead of sampling it on one run.

The opening example uses the specification rule *the payload must remain stable while VALID is asserted and READY is low*.

```
// Wrong: compares this cycle's payload against the previous cycle's.
property p_aw_stable_wrong;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && !awready) |-> $stable({awid, awaddr, awlen, awsize, awburst});
endproperty
```

`$stable` tests whether a value is unchanged from the previous clock tick: at the observed stall, this property checks backward whether the payload was the same one tick ago. The obligation is forward, and it needs the non-overlapping implication, the operator that starts the consequent one tick after the antecedent rather than in the same tick:

```
// Right: the payload offered now must still be there at the next tick.
property p_aw_stable;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && !awready) |=> awvalid && $stable({awid, awaddr, awlen, awsize, awburst});
;
endproperty
a_aw_stable : assert property (p_aw_stable);
```

The two versions differ by a cycle and one added term; the two edits serve different purposes. The cycle shift repairs a known mistranslation: “remain stable” in protocol prose means “hold the same value into the next clock cycle”, and a bare `$stable` is a documented weakness of automated specification-to-property translation. A manager drives a fresh address in the same cycle in which it raises `AWVALID`. If `AWREADY` is low in that cycle, then the antecedent is true, `$stable` compares against the idle cycle before and is false, and the property fires on perfectly legal traffic. The property misses the opening example’s withdrawal: when `AWVALID` drops, its antecedent becomes false and nothing is checked. The added `awvalid` in the consequent imposes a second rule, distinct from the first: an offer must not be withdrawn before acceptance. The documented stability fix is the cycle shift alone ^[2]; combining two obligations in one property is this book’s explicit choice. [Figure 11.1](#) shows both defects.

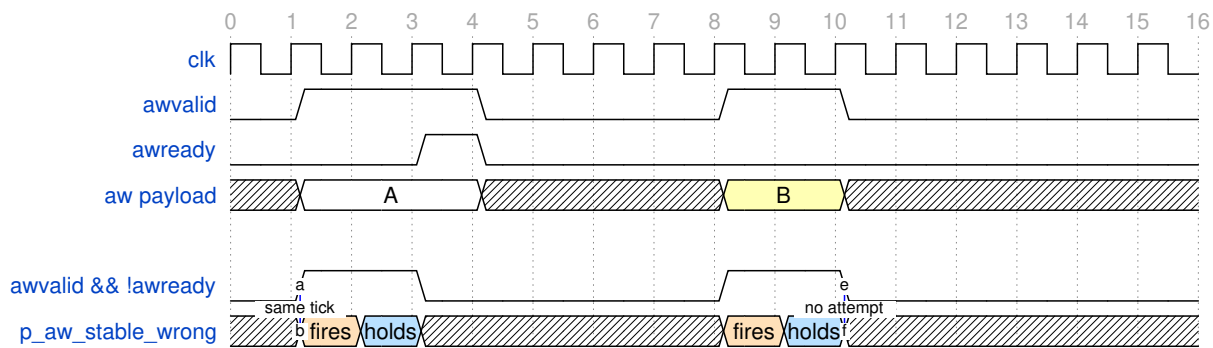


Figure 11.1 — Backward `$stable`: at cycle 1 a fresh address differs from the prior idle value and fires; at cycle 10 withdrawal leaves the antecedent false and nothing is checked.

Writing one requires naming the clock, the exact cycle relation, the condition under which the obligation starts, the reset behavior and the polarity of every term; each question must be answered from the specification or, when it is silent, from its owner. Chapter 5 treats an incomplete specification as a defect to expose; assertion writing exposes it because an unanswered question prevents completion of the property.

Because the language is declarative, debugging one is harder than debugging RTL: there is no statement to step through, only a verdict. Complex properties should be expected to be wrong on the first attempt, reviewed as carefully as design code, and drawn from a library for routine claims: “these two signals are mutually exclusive”, “a request is granted within N cycles”, “this state machine is never stuck” [1].

11.2 The anatomy of SVA

SVA semantics rest on four layers.

The Boolean layer uses ordinary SystemVerilog expressions over *sampled* values. A concurrent assertion is evaluated only at clock ticks, and the value it uses for a variable is that variable’s value in the Preponed region: the value the variable held at the beginning of the time step that contains the clock edge, taken before the edge performs any update (IEEE Std 1800-2023, *IEEE Standard for SystemVerilog*) [1, 3]. An assertion therefore cannot race the RTL, cannot see a combinational glitch, and cannot be raced by the choice between blocking and non-blocking assignments. That choice still changes the design’s behavior, and hence the value sampled at the next tick. Like a waveform reader, the assertion uses the value *before* the edge, so a property that looks off by one cycle in a debugger usually is not.

The sequence layer describes patterns over time, like a regular expression whose alphabet is Boolean expressions and whose step is a clock tick: `a ##1 b` is *b* one tick after *a*, `a ##[1:4] b` allows a window, `s [*3]` repeats. Sequences are why a temporal rule is three lines of SVA instead of the hand-written state machine protocol checking needed before the language existed. [Figure 11.2](#) places the three on one timeline.

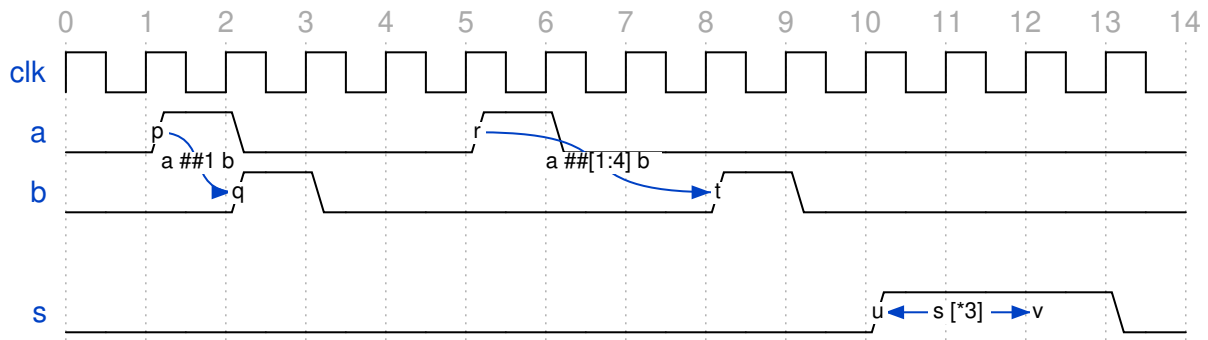


Figure 11.2 — Three sequence forms: `a ##1 b` matches at cycle 2, `a ##[1:4] b` three ticks after `a` at cycle 5, and `s [*3]` from cycle 10.

The **property layer** turns sequences into claims, chiefly through implication. `antecedent |-> consequent` starts evaluating the consequent in the same cycle the antecedent completes; `|=>` starts it in the next. Two facts matter for Section 11.7: a false antecedent makes an implication succeed *without checking anything*, and antecedent selection changes what a pass means.

The **statement layer** supplies the four directives covered next: `assert`, `assume`, `cover` and `restrict`.

A concurrent assertion starts a fresh **evaluation attempt** at every tick of its leading clock; each reaches its own success or failure while overlapping others ^[1]. A five-cycle window permits up to five attempts in flight, so a violation under sustained traffic produces a burst of failures rather than one.

`disable iff` guards reset: while its expression is true, it discards any attempt in flight as *disabled*, neither pass nor failure. It is asynchronous, monitored at every time step rather than only at clock ticks, as asynchronous reset requires; this also exposes it to zero-width glitches unless explicitly sampled. There may be at most one per assertion, immediately after the clocking event, guarding the whole property ^[1]. Nearly every real property should carry one: reset violates almost every design rule, and repeated reset-time failures encourage disabling the suite. Figure 11.3 shows a discarded attempt.

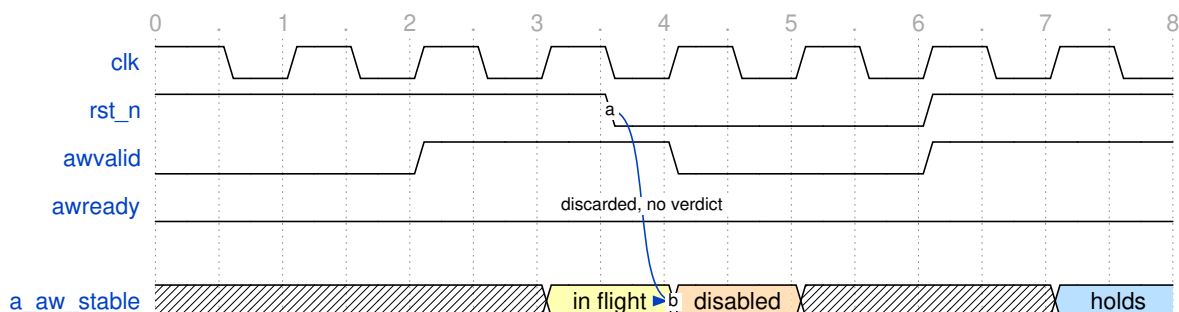


Figure 11.3 — Reset falls between cycles 3 and 4, discarding the in-flight attempt rather than judging it; the attempt after release at cycle 6 holds.

Immediate assertions are procedural: they test a non-temporal expression when control flow reaches them, exactly like an `if`, and `x, z` or `0` counts as a failure. **Con-**

current assertions are the clocked, temporal ones [3]. If the claim is “this is true at this point in this code”, it is immediate; if it involves a clock or spans time, it is concurrent.

```
// Immediate: a sanity check at the point where the value is computed.
always_comb begin
    a_imm_cnt_sane : assert (cnt <= 5'd16)
        else $error("count %0d exceeds depth", cnt);
end

// Concurrent: a rule about the design's behaviour over clock ticks.
a_conc_no_overflow : assert property (@(posedge clk) disable iff (!rst_n)
    (cnt == 5'd16) |-> (!push || pop));
c_cnt_full : cover property (@(posedge clk) disable iff (!rst_n)
    cnt == 5'd16);
```

The immediate assertion describes the surrounding code, while the concurrent one describes the design and is checked on every test that runs, regardless of whether anyone remembers it. Its rule is the one Section 11.6 writes as `a_no_overflow`, there taking the push as the antecedent instead of the full condition; both reduce to *no push while full unless a pop retires in the same cycle*. The `cover` beside it applies Section 11.7’s discipline from the first example onwards: an overflow rule on a FIFO that never fills has never been tested. Figure 11.4 reads the same-cycle exception off a trace.

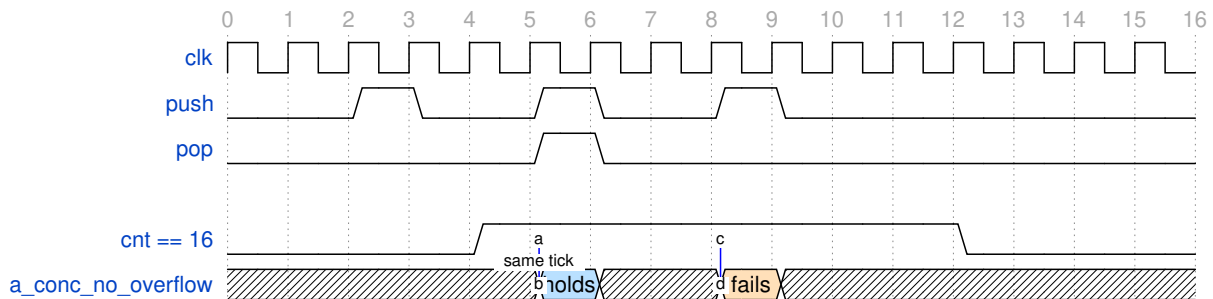


Figure 11.4 — Same-tick consequent: push plus pop at cycle 5 holds at full, lone push at cycle 8 fails, and push at cycle 2 starts no attempt.

11.3 Assert, assume, cover: three claims about one property

The same property text can be attached to a design in four ways expressing verification intent: the three in the heading act in simulation, while a fourth is read only by a proof engine. `assert` makes the property an obligation on the design; `assume` makes it a constraint on the environment; `cover` monitors it for coverage; `restrict` constrains formal computation only, and simulators do not check it [1, 3]. The engine determines what each directive does; Table 11.1 compares the four for both engines, with the readings diverging at `assume`.

Table 11.1 — What the four property directives mean to each of the two engines that read them: the same property text attached as `assert`, `assume`, `cover` or `restrict`, with the simulator’s reading in the middle column and the proof engine’s on the right. The `assume` row is the one to read twice. In simulation an assumption is checked like an asser-

tion; in formal it is not checked at all, it deletes from consideration every trace that violates it. An assumption written stronger than reality therefore deletes real traces too, and the proof result then describes a design that does not exist.

Directive	In simulation	In formal verification
<code>assert</code>	fails if violated on this trace	proved, or a counterexample produced, under all stated assumptions
<code>assume</code>	checked like an assertion: a violation means the environment misbehaved	constrains the legal traces; its own correctness is <i>not</i> checked
<code>cover</code>	reports and counts each match on this trace	asks whether a matching trace exists, and returns a witness
<code>restrict</code>	ignored	constrains traces, like <code>assume</code>

In simulation `assert` and `assume` are effectively synonyms: both are checked, and the keyword only identifies responsibility. In formal, assumptions are unchecked and delete traces; assertions are proved only over the remaining traces [1]. An assumption stronger than the real environment excludes legal traces from the proof. This asymmetry requires reviewing the directive alongside the property and motivates the assumption hygiene in Chapter 14.

When a module’s interface obligations explicitly state its inputs’ rules and its outputs’ guarantees, integration mistakes turn into assumption violations on the first system-level run, and each violation appears at the boundary where the mismatch occurs [1]. The same rule is an `assume` when verifying the DMA in isolation and an `assert` when verifying the crossbar that feeds it.

11.4 Where assertions live

Assertion placement and authorship are two distinct decisions.

Assertions over internal signals require detailed knowledge of the design’s internal structure, so the designer is best placed to write them; assertions over external interfaces treat the block as a black box, derive from the interface specification rather than the implementation, and are normally written by verification engineers [4]. Responsibility follows the source document rather than seniority: the designer uses the micro-architecture, the verifier the specification.

The designer’s assertions belong inline in the RTL, next to the logic they constrain, because they must be maintained with that code. An assertion explains the logic in formal and natural language, as a comment would, and also reports when that explanation ceases to hold [4].

The verifier’s assertions must not touch the RTL at all. SystemVerilog’s `bind` construct instantiates a module, interface, program or checker into a target scope (every instance of a module, or one named instance) without modifying the target’s source [3]. Interface assertions are attached that way, or placed inside the `interface` that carries the signals so they become part of its specification. Ownership becomes unambiguous: the RTL file is the designer’s, and the checker bound into the design is verification’s. A parameterized checker module is bound once to each of the cross-

bar's nine ports instead of copied nine times, and can be compiled out of a gate-level run under a macro [4].

One inline hazard illustrates Chapter 2's independence argument: an assertion written by the same person, in the same hour, from the same reading as the RTL encodes the same misunderstanding and never fires. Keeping implementation details from tainting the check requires explicit discipline [4]. The division of labor mitigates this: inline assertions state *implementation* invariants absent from specifications; bound assertions state *specification* obligations that no implementation should define.

11.5 Protocol checking: the interface as a contract

Protocol rules come from a public specification written by someone else, which provides an independent oracle, the mechanism that decides whether an observed response is correct. Protocol rules are local and decidable from one interface, so they are cheap to evaluate and within reach of a proof. Downstream diagnosis is expensive because a protocol error corrupts state whose effects surface much later elsewhere.

The distinction is between assertions on a *local* interface protocol and assertions on *end-to-end* behavior that relates one interface to another. The local ones are typically well suited to formal proof; the end-to-end ones usually span too much of the design for a proof and are often easier to express in the testbench. A check requiring extensive data structures or algorithms across much of the design belongs in a monitor and scoreboard (see Chapter 8) [4]. Siemens EDA's Verification Academy describes a monitor as doing three things: it reaches the design's signals through a virtual interface; it holds the code that decides when a run of signal activity amounts to a protocol pattern; and it broadcasts each recognized pattern as a transaction to whoever has subscribed [5]. A virtual interface is the reference through which testbench code reaches the signals of an interface instance.

Worked example: the contract on a crossbar port

The reference SoC's crossbar has five manager ports and four subordinate ports. Each port carries an AXI4 contract (*AMBA® AXI and ACE Protocol Specification*, Arm IHI 0022H.c, 2021 [6]), and three of its clauses need visibly different amounts of machinery. The three are Arm IHI 0022H.c §A3.2.1 (handshake process), §A5.2.2 (write data ordering) and §A5.1 (AXI transaction identifiers), the last of which requires a Subordinate to reflect a Manager's ID on the corresponding BID or RID response.

Clause one: handshake stability. This is `p_aw_stable` from Section 11.1: while a request is offered and not yet accepted, it stays offered and unchanged. It needs no memory; the rule relates two adjacent cycles and SVA expresses it directly. Figure 11.5 shows both outcomes.

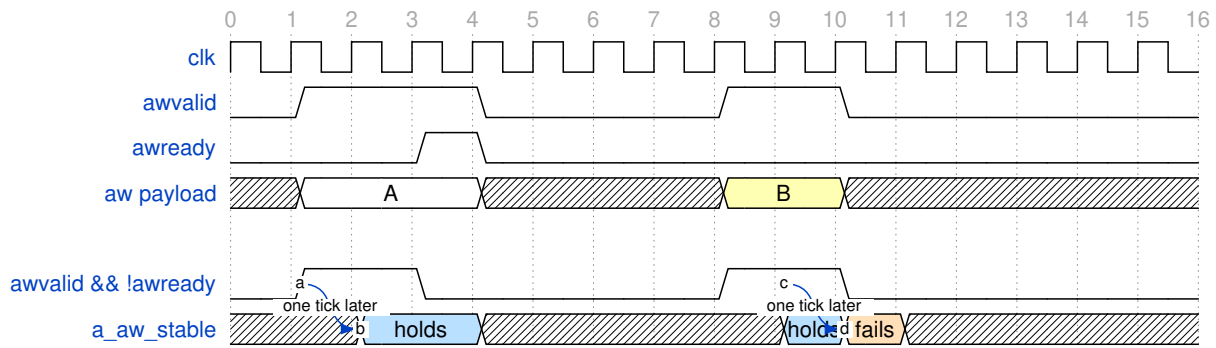


Figure 11.5 — Forward obligation: offers at cycles 1, 2 and 8 hold one tick later with the same payload; the cycle-9 attempt fails on withdrawal at cycle 10.

Clause two: one burst's write data at a time. AXI4 deprecated write-data interleaving, which AXI3 had permitted (Arm IHI 0022H.c §A5.2.2 [6]): a manager must issue its write data in address order; because no way is left to tag an interleaved beat, that rule means a manager finishes the data beats of one burst before starting another's; and an interconnect that merges writes from several managers must forward their data in the order in which it forwarded their addresses. The second obligation belongs to the crossbar in Section 11.5. AXI3's `WID` is gone, leaving no write-data ID to check; a counter and queue must reconstruct each burst and reconcile its beat count with the length promised by its opening address.

Clause three: a response only for work in flight. A write response may be returned only for an ID with an outstanding address, with no more responses than accepted addresses; this requires per-ID bookkeeping. Figure 11.6 shows the bookkeeping.

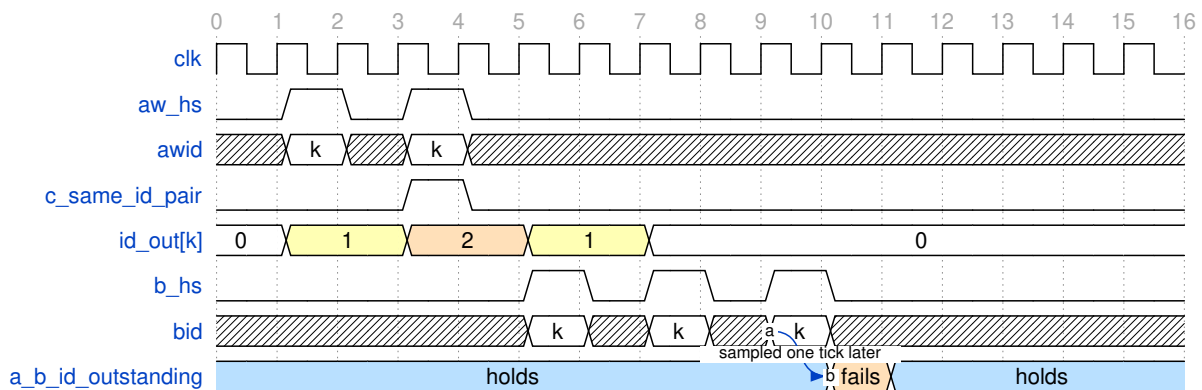


Figure 11.6 — Per-ID bookkeeping: addresses at cycles 1 and 3 leave two writes under one ID; responses at 5 and 7 retire them, and response 9 has none to retire.

Clauses two and three require a *checker*: a small module with its own interface model, bound to the port. A property dependent on an implicit scoreboard is not reusable, so the checker's state must be explicit in its file.

```
module xbar_port_checker #(
    parameter int unsigned IDW = 4, // AXI ID width on this port
    parameter int unsigned AW = 32 // address width
) (
```

```

input logic      clk, rst_n,
input logic      awvalid, awready, wvalid, wready, wlast, bvalid, bready,
input logic [IDW-1:0] awid, bid,
input logic [AW-1:0] awaddr,
input logic [7:0]  awlen,
input logic [2:0]  awsize,
input logic [1:0]  awburst
);
wire aw_hs = awvalid && awready;
wire w_hs  = wvalid  && wready;
wire b_hs  = bvalid  && bready;

// The checker's own model of the port.
logic [7:0]  aw_len_q [$]; // AWLEN of each accepted address, in issue order
int unsigned id_out [1<<IDW]; // writes outstanding, per ID
logic [8:0]  w_beats; // beats so far in the burst now on W (max 256)
logic        len_err, id_err;

always @(posedge clk) begin
    len_err = 1'b0;
    id_err  = 1'b0;
    if (!rst_n) begin
        aw_len_q.delete();
        foreach (id_out[i]) id_out[i] = 0;
        w_beats = '0;
    end else begin
        if (aw_hs) begin
            aw_len_q.push_back(awlen);
            id_out[awid]++;
        end
        if (w_hs) begin
            if (!wlast) w_beats++;
            else begin
                if (aw_len_q.size() == 0) len_err = 1'b1; // data with no address
                else begin
                    if (w_beats != aw_len_q[0]) len_err = 1'b1; // wrong beat count
                    void'(aw_len_q.pop_front());
                end
            end
            w_beats = '0;
        end
        if (b_hs) begin
            if (id_out[bid] == 0) id_err = 1'b1; // response nobody asked for
            else id_out[bid]--;
        end
    end
end

a_aw_stable : assert property (@(posedge clk) disable iff (!rst_n)
    (awvalid && !awready) | => awvalid && $stable({awid, awaddr, awlen, awsize,
awburst}));

a_w_burst_len : assert property (@(posedge clk) disable iff (!rst_n) !len_err);

a_b_id_outstanding : assert property (@(posedge clk) disable iff (!rst_n) !id_err);

// The covers that keep the assertions honest.
c_aw_stall : cover property (@(posedge clk) disable iff (!rst_n)
    awvalid && !awready);
c_w_overlap : cover property (@(posedge clk) disable iff (!rst_n)
    aw_hs && (aw_len_q.size() != 0));
c_same_id_pair : cover property (@(posedge clk) disable iff (!rst_n)
    aw_hs && (id_out[awid] != 0));

```

```

c_max_burst      : cover property (@(posedge clk) disable iff (!rst_n)
                    aw_hs && (awlen == 8'd255));
endmodule

```

Every variable in the checker is written by this one block and read nowhere else except by the assertions, which sample in the Preponed region. The blocking style is safe here because every variable is owned by that one block, not because of the choice of assignment operator. Within a tick the statements run in the order written and the queue methods mutate immediately, so an address pushed at this edge is visible to the `pop_front` at the same edge, which makes a single-beat burst whose address and data arrive together check correctly. `w_beats` is deliberately one bit wider than `AWLEN`, so a manager that runs past its `WLAST` cannot wrap around onto a legal count. Because `len_err` and `id_err` are registered, an assertion samples them at the tick *after* the one that set them: the failure is reported one cycle behind the beat that caused it, with the transaction still on screen. Figure 11.7 shows that one-cycle offset on a burst that runs long.

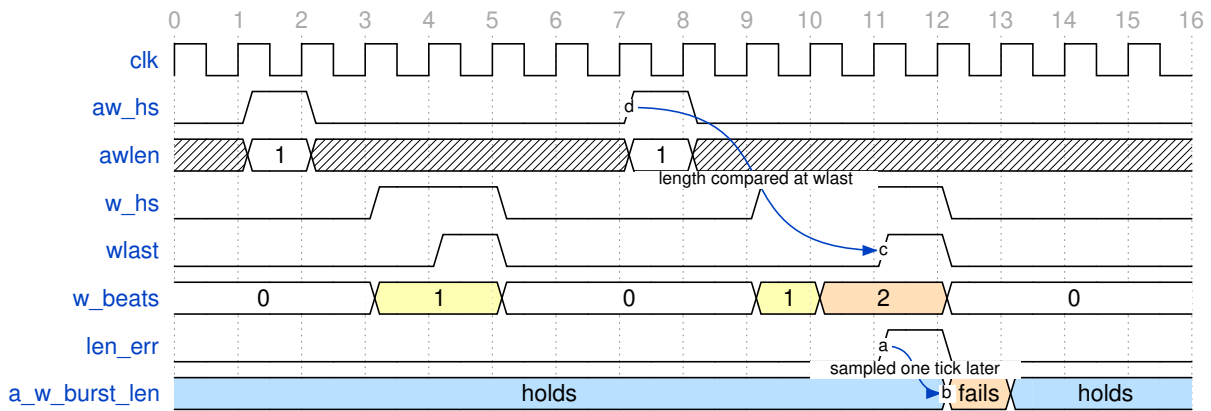


Figure 11.7 — Beats at cycles 3 and 4 match queued length; a third beat at cycle 11 sets `len_err` at `wlast`, reported at cycle 12.

Two instantiation lines attach the checker to all nine ports of the crossbar:

```

bind xbar_manager_port      xbar_port_checker #(.IDW(4)) u_chk (.*) ;
bind xbar_subordinate_port  xbar_port_checker #(.IDW(7)) u_chk (.*) ;

```

The five manager-side instances check the five managers. The four subordinate-side instances check the crossbar, which plays the manager role as far as a subordinate is concerned; this distinction determines how to interpret coverage below. The design source is untouched, and that file is what Chapter 14 hands to a proof engine. The two `IDW` overrides account for interconnect ID widening. An interconnect widens the ID it forwards, so a manager port carries the 4-bit ID the manager issued and a subordinate port carries a 7-bit one (Chapter 8 derives the three extra bits). Arm IHI 0022H.c §A5.2.3 [6] assigns the widening to the interconnect rather than the checker: “The ID identifier at a Subordinate interface is wider than the ID identifier at a Manager interface.” `.*` connects only signals of the same name *and equivalent type*, so a checker at its default width fails elaboration on the subordinate side instead of

silently misconnecting. Detecting this requires elaboration: `lint` is a structural check on the design description, which reports questionable constructs a compiler accepts, and `lint` alone does not resolve `bind`.

Two limits of a passing check. The four `cover` statements detect two different weaknesses.

The first is vacuity in the strict sense. `a_aw_stable` fires only when `awvalid &&!awready`, an offer that had to wait. If the subordinate models in the testbench accept every address immediately, that antecedent is never true, every attempt succeeds trivially, and the assertion reports a perfect record on a rule it never evaluated. `c_aw_stall` is exactly that antecedent, promoted to a coverage question: an empty `c_aw_stall` says the stability check has not run. Figure 11.8 contrasts a run without a stall against the first stall on the port.

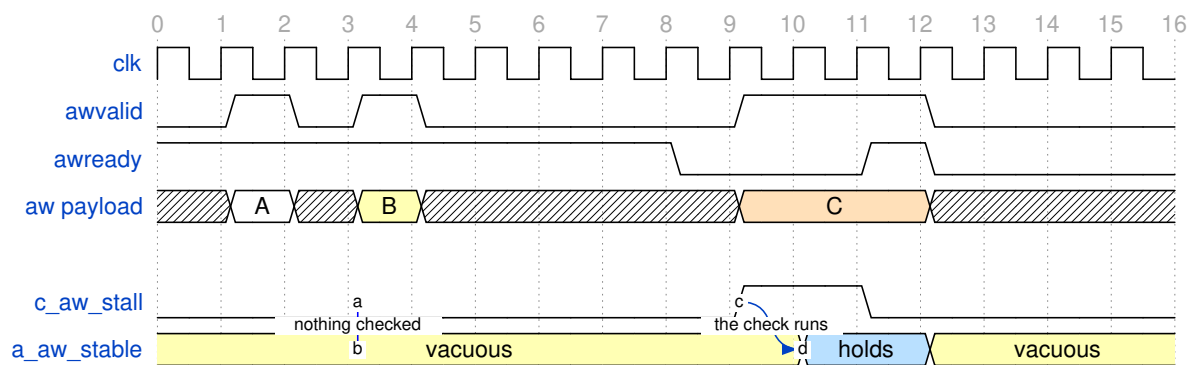


Figure 11.8 — Immediate acceptance leaves attempts vacuous and `c_aw_stall` empty; only the cycle-9-to-10 stall runs the check.

In the second case, the assertion evaluates but misses its intended hazard. In this example, an early sequence library issues one outstanding write per manager at a time, a reasonable initial choice that remains unnoticed once it works. The library belongs to bring-up, the phase in which the environment and the DUT first exchange checked transactions. `a_w_burst_len` still evaluates on every run and would still catch a short or long burst; what it never gets to see is the interleaving case it was written for, because on a manager port `aw_len_q` never holds two entries. After six weeks of nightly regression at 100% pass in this example, the hazard remains untested. `c_w_overlap` is the witness, and it reads zero. `c_max_burst` checks for acceptance of an address requesting the 256-beat maximum; without such a request, the beat-counting arithmetic is not exercised at that burst length.

The covers must be interpreted at their binding point. `c_same_id_pair` fires when an address is accepted whose ID already has a write outstanding *on that port*. On a manager-side instance it concerns one manager: a sequence library that cycles a fresh ID per transaction leaves the cover empty, while one that pins a single ID per manager fills it on every second outstanding write. On the four subordinate-side instances, this cover cannot detect two managers colliding on an ID. An interconnect that merges traffic *widens* the ID it forwards, appending enough bits to identify the source port so that no manager need know which IDs the others use; Chapter 8 does the arithmetic, four bits in and seven out. Two managers presenting `AWID` 3 reach

the subordinate as two distinct IDs, so same-ID at a subordinate port still means *one manager*. The widening that rennumbers those IDs leaves the crossbar with an ordering obligation: it must forward the writes that one manager issues under one ID in the order in which that manager issued them, and it must return their responses in that same order. This ordering is the obligation the cover witnesses there.

The other cover witnesses the crossbar bug's precondition. The canonical crossbar bug of this book is a violation by neither manager: each finishes one burst's beats before starting the next, exactly as clause two requires. The violation occurs on the subordinate side, where the crossbar must obey AXI: its write arbiter releases a burst's grant when the subordinate stalls the first data beat, so beats from two managers' bursts interleave on the write-data channel, and AXI requires the write-data channel's data to follow address order. On a subordinate-side instance, `c_w_overlap` witnesses *a second write address accepted while an earlier burst remains unfinished on the write-data channel*. This is the precondition; `a_w_burst_len` detects the failure because beats from two bursts counted as one match the queued head length only by coincidence. `aw_len_q` is keyed by arrival order alone, which is why an ID-based reading of this hazard finds nothing to check. `c_w_overlap` reads zero on these four instances too: the same sequence library leaves a different condition unmet. Filling it on a manager port takes one manager with two writes outstanding; filling it here takes two managers with writes in flight at one subordinate. A bring-up library exercising one manager against one subordinate, then the next pair, arranges neither condition. Figure 11.9 shows the precondition and the failure it produces on one subordinate-side trace.

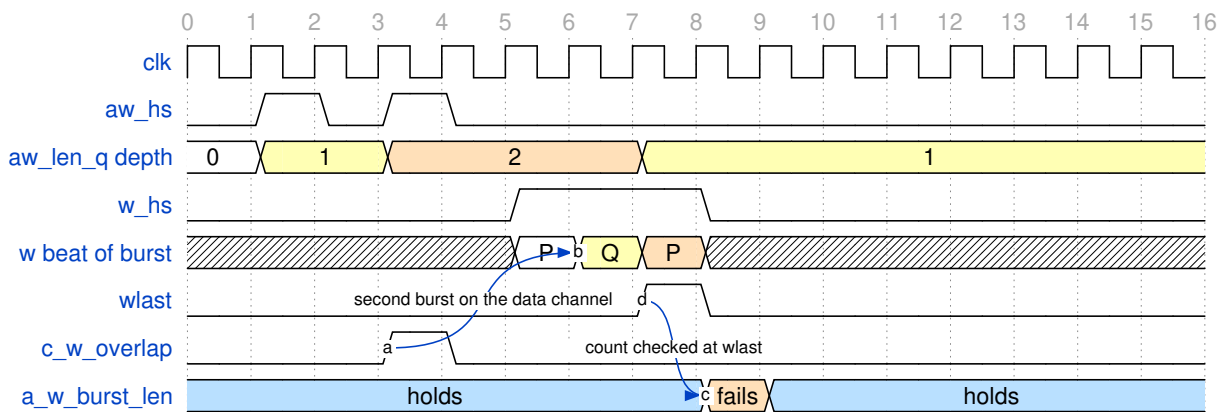


Figure 11.9 — On a subordinate port, address 3 fills `c_w_overlap`; an interleaved beat at 6 makes `a_w_burst_len` fail at 8.

An antecedent cover quantifies whether the condition occurred. When a cover reads zero, two explanations require action: either no test drives the design into the situation, so the stimulus needs work, or the design can never reach it at all, in which case either the property is misformulated or something in the design prevents an expected state [1]. Section 11.7 turns this diagnosis into a general rule for writing assertions.

Scenario (elsewhere in the cast). A 10G Ethernet MAC with time-sensitive-networking queues has a timing obligation: the reference design bounds the cycles from the start-of-frame delimiter to the frame’s capture timestamp, and one hop exceeding that bound silently consumes the network’s synchronization budget. Time-sensitive networking is the set of Ethernet extensions that give a chosen stream a bounded delivery delay. **Approach.** One assertion, and the cover that says whether a frame delimiter was ever seen at all:

```
systemverilog a_ts_latency : assert property (@(posedge clk) disable iff (!rst_n)
$rose(sfd_seen) |-> ##[1:2] ts_captured); c_sfd : cover property (@(posedge clk)
disable iff (!rst_n) $rose(sfd_seen));
```

Why. `$rose` is true at a tick where the sampled value went from 0 to 1, so the obligation starts once per frame, and the window is the reference design’s specified timing budget, rather than a value inferred from the implementation. Deriving the obligation from the protocol specification keeps the check independent of the implementation. Figure 11.10 shows one frame inside the window and one past it.

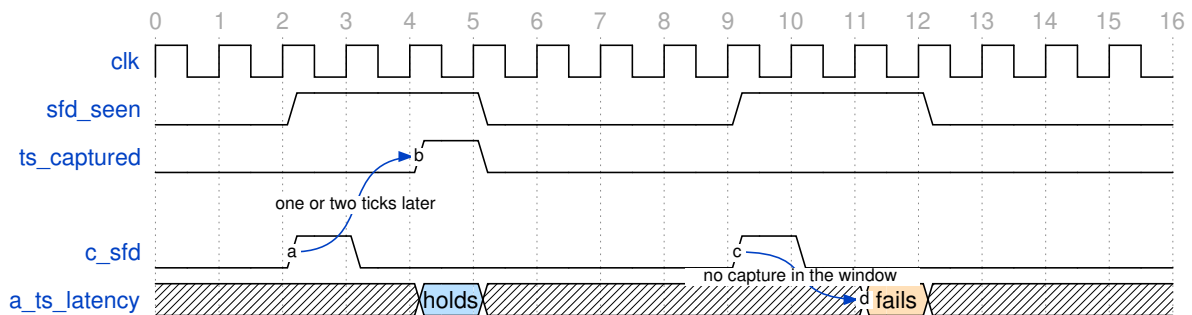


Figure 11.10 — Delimiter 2 captures at 4 inside the window; delimiter 9 has no capture by 11 and fails.

11.6 White-box assertions and internal invariants

Interface assertions check what the outside world is entitled to expect. Internal assertions check invariants that the implementation implies and the specification does not state, invariants that the designer knows and would otherwise record in comments. The standard catalogue covers state-machine transitions, memory access correctness, queue overflow and underflow, arithmetic overflow, and signal stability, though the complete list depends on the methodology ^[1]. Figure 11.11 reads the state-machine invariant off a transfer.

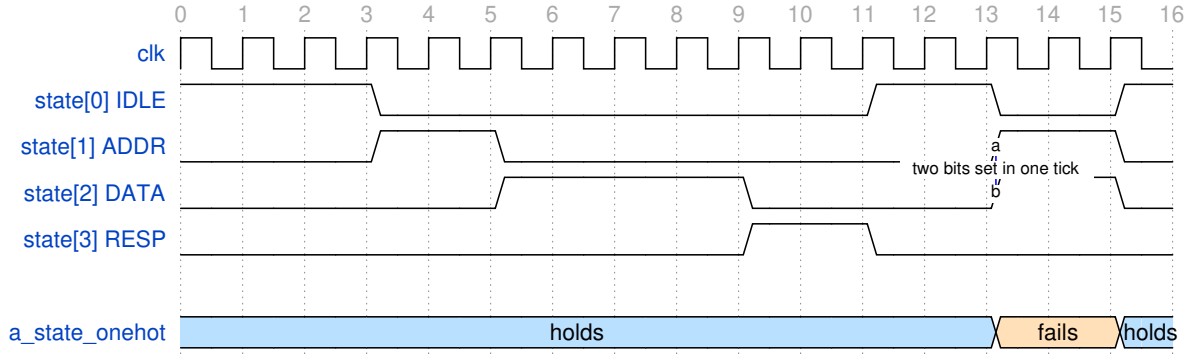


Figure 11.11 — One bit is set through cycle 12; two at 13 and 14 fail the invariant; one returns at 15.

The DMA backend’s datapath FIFO has 16 entries of 64 bits. The example checks four structural invariants, none derivable from the DMA’s specification:

```

module dma_backend_invariants #(parameter int unsigned DEPTH = 16) (
  input logic          clk, rst_n,
  input logic          push, pop,
  // free-running binary pointers on a power-of-two depth: an index plus a
  // wrap bit, so their difference tells full from empty
  input logic [$clog2(DEPTH+1)-1:0] level,
  input logic [$clog2(DEPTH):0] wr_ptr, rd_ptr,
  input logic [3:0] state // one-hot: IDLE, ADDR, DATA, RESP
);
a_no_overflow : assert property (@(posedge clk) disable iff (!rst_n)
    push |-> (level < DEPTH) || pop);
a_no_underflow : assert property (@(posedge clk) disable iff (!rst_n)
    pop |-> (level != 0));
a_level_tracks : assert property (@(posedge clk) disable iff (!rst_n)
    level == (wr_ptr - rd_ptr));
a_state_onehot : assert property (@(posedge clk) disable iff (!rst_n)
    $onehot(state));

c_full : cover property (@(posedge clk) disable iff (!rst_n)
    level == DEPTH);
c_full_bypass : cover property (@(posedge clk) disable iff (!rst_n)
    (level == DEPTH) && push && pop);
endmodule

```

Of the four invariants, `a_level_tracks` checks agreement between redundant state. The other three restate rules a careful designer already coded (`$onehot` is true when exactly one bit of its argument is set), while this one asserts *redundancy*: two independently maintained representations of the same fact, the occupancy counter and the pointer difference, must agree. Redundant-state invariants detect corruption affecting one representation while leaving the other unchanged, producing plausible-looking but incorrect data, six hundred cycles from its cause in this example. The equality holds because both operands are binary free-running pointers of the declared width, so their difference wraps modulo the pointer range. A gray-coded FIFO needs the synchronized binary shadow instead, because its pointers cross a CDC, a clock-domain crossing whose two clocks have no fixed phase relationship. Figure 11.12 shows the counter moving without the pointers.

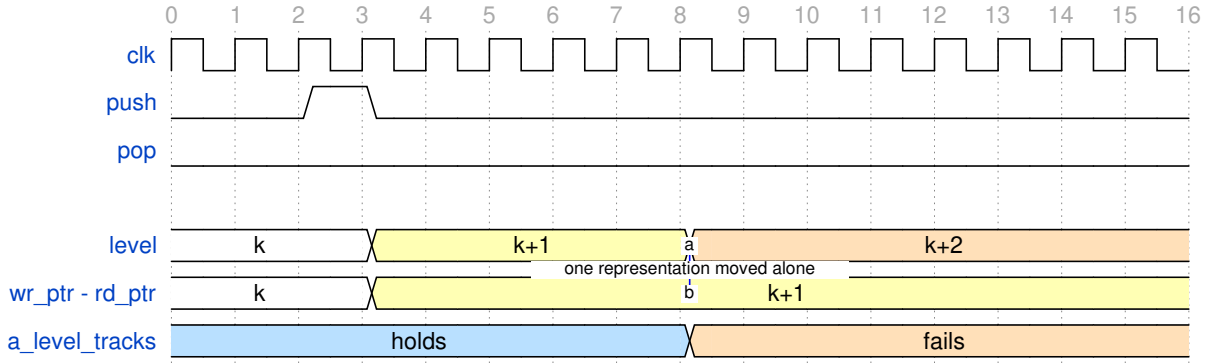


Figure 11.12 — Two representations of one fact: the push at cycle 2 moves both the counter and the pointer difference, and the counter alone advances at cycle 8, where the equality fails with no push and no pop to justify it.

The two `cover` statements apply Section 11.5: `push` and `pop` occur on any run moving data, so coverage must establish whether the FIFO reaches full. Of the two, `c_full_bypass` specifically witnesses `a_no_overflow`'s `|| pop` exception: a push accepted at full because a pop retires in the same cycle. A zero count means that exception remains unexercised and untested. The asymmetry with `a_no_underflow` is deliberate: this FIFO forwards a pop that arrives alongside a push at full, but does not forward a push to a pop at empty. Figure 11.13 shows the exception and the two failures on one trace.

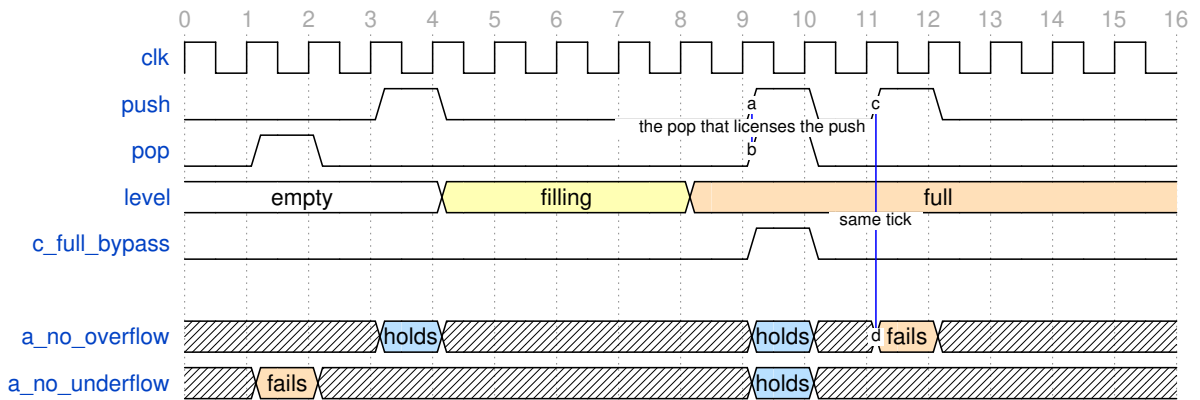


Figure 11.13 — The two level guards: the pop at cycle 1 finds the FIFO empty and fails, the push at cycle 9 is licensed by the pop in the same tick and fills `c_full_bypass`, and the push at cycle 11 arrives alone and fails.

Internal assertions improve observability, which is the basis of the partial-oracle role Chapter 3 assigns them: a partial oracle checks one projection of correctness rather than correctness itself. A bug is caught beside the code that produced it, so the failure message identifies the responsible lines rather than a downstream symptom ^[1]; the opening example's day and a half becomes a single annotated cycle.

The property Chapter 14 will prove

One DMA backend rule is an interface obligation originating in the bus specification, but its violation comes entirely from the arithmetic of the burst splitter, the DMA stage that splits each transfer into bursts that the AXI address rules allow. Chapter 2 used that rule to show what an adversarial reading of a specification produces, and

Chapter 3 used it as a worked case of what a partial oracle does and does not establish:

```
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && awready && (awburst == 2'b01)) |->
      (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

In simulation this property is *sampled*: evaluated on the bursts the tests happened to generate, silent about the ones they did not. Chapter 14 submits the identical text to a proof engine, which establishes that no legal input sequence can produce a crossing burst or returns a counterexample. Section 11.8 examines this change in the question asked of an unchanged property. Section 11.7’s rule applies to this implication, whose antecedent, an accepted INCR write address, is ordinary traffic. What needs covering is the relation between a burst and the 4 KB boundary, the address line that AXI forbids a burst to cross. That relation is why Chapter 5’s plan gives this rule a row paired with a coverage model rather than with a single `cover` line. Figure 11.14 shows what sampling leaves out.

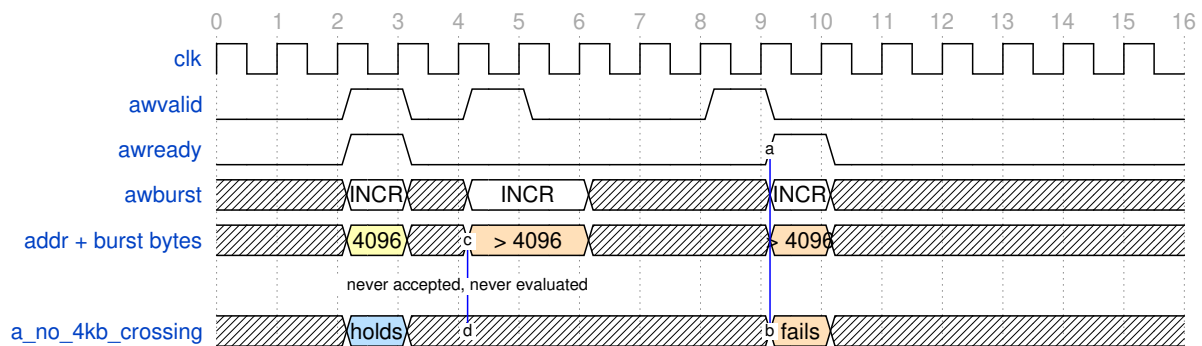


Figure 11.14 — Accepted address 2 ends at 4096 and holds; crossing address 4–5 is unaccepted and unjudged; acceptance 9 fails.

Elsewhere in the cast. Security rules are usually invariants over internal state rather than interface behavior, so internal assertions also express security properties. In the crypto engine, two assertions constrain key handling:

```
a_key_quiet      : assert property (@(posedge clk) disable iff (!rst_n)
    key_load |-> engine_idle);
a_no_ct_unauth   : assert property (@(posedge clk) disable iff (!rst_n)
    ct_valid |-> key_authenticated);

c_key_load       : cover property (@(posedge clk) disable iff (!rst_n) key_load);
c_ct_valid       : cover property (@(posedge clk) disable iff (!rst_n) ct_valid);
```

The first forbids a key change under an in-flight operation, which is how key material bleeds between contexts. The second forbids *presenting* ciphertext while the key is unauthenticated; it says nothing about the key used to compute that ciphertext. If authentication completes between the computation and the rise of `ct_valid`, the as-

sertion passes. Provenance requires a tag traveling with the key material, a different invariant over different state. Both are single-cycle claims over a small cone of logic; in this example, proof is assumed to complete in seconds when they hold. Neither is checkable from the engine's data interface, and both carry a cover on their antecedent because an assertion on key loading remains unexercised on a run that never loads a key. Figure 11.15 shows a trace on which the second assertion passes over a ciphertext computed before authentication.

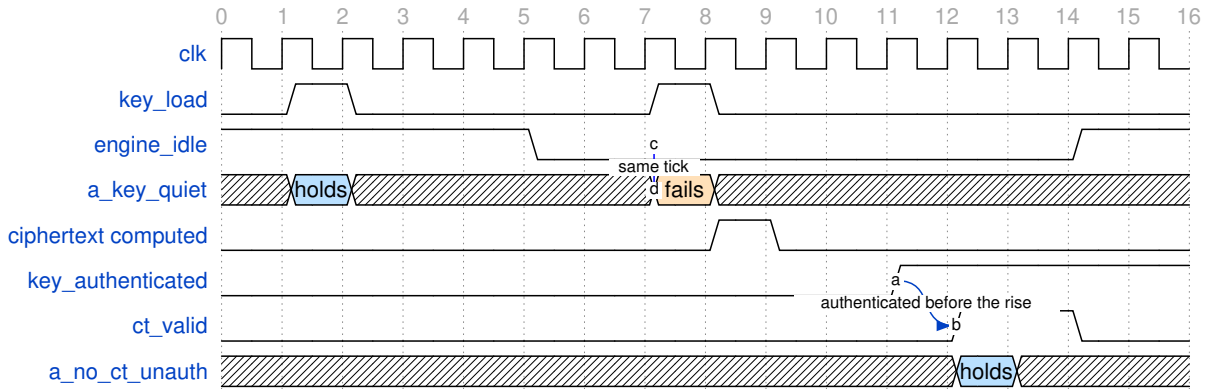


Figure 11.15 — Key load 1 holds idle; load 7 fails in flight. Ciphertext computed at 8 is presented after authentication, so the second rule holds.

11.7 Assertion quality

Assertion-suite quality depends on how much of it could ever have failed rather than its size; three defects defeat that purpose.

It never triggers. A property $a \rightarrow b$ succeeds whenever a is false, establishing nothing: IEEE 1800-2023 §16.14.8 calls this vacuous and interprets it as not matching the user's intent, neither pass nor failure [3]. A run in which an assertion has only vacuous successes indicates one of two situations: either the tests never drive the design into the situation the antecedent describes, or the design can never reach it at all, in which case the property is misformulated or something in the design prevents an expected state [1]. **Every implication should have a cover on its antecedent**, written with the property and reviewed in the same review. For ordinary traffic, such as `push` on a FIFO pushing all day, a bare antecedent cover is insufficient. The rule should then be satisfied in one of two ways: by a witness that *implies* the antecedent and isolates the interesting case, as `c_full_bypass` does, or by the coverage model that Chapter 5's plan pairs with `p_no_4kb_crossing`. Chapter 6 reports these covers as assertion coverage; here they are required during property writing, before measurement.

Rule formulation also determines vacuity. $ok \rightarrow !err$ and $err \rightarrow !ok$ are logically equivalent but pass vacuously under opposite conditions: the first when `ok` is false, the second when `err` is false [1]. Antecedent selection therefore determines what a passing run means. The antecedent should be the rare, deliberately constructed condition, making a non-vacuous pass evidence and an empty cover an action item. Figure 11.16 shows the two passing vacuously on opposite stretches of one trace.

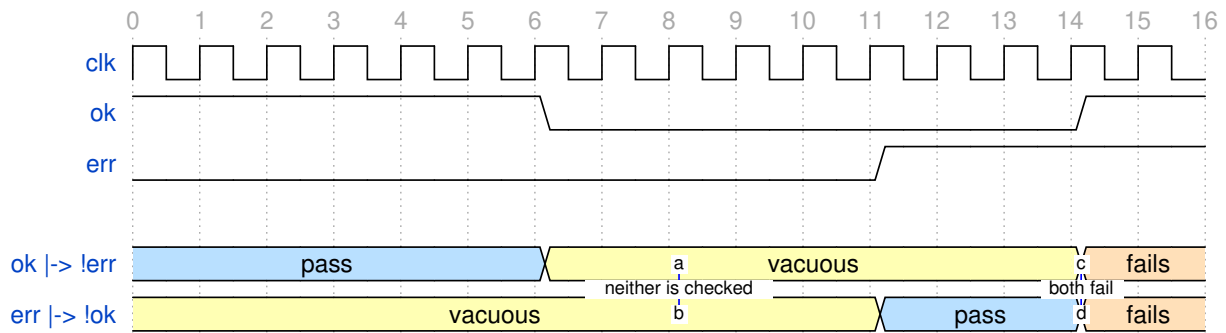


Figure 11.16 — Two equivalent formulations on one trace: `ok |-> !err` passes on the stretch to cycle 5 and is vacuous from cycle 6, `err |-> !ok` is vacuous to cycle 10 and passes from cycle 11, and both fail from cycle 14.

It restates the design. An assertion derived from the implementation agrees with the implementation, including where the implementation is wrong; keeping the code from tainting the check is an explicit obligation on whoever writes an inline assertion [4]. In review, a property paraphrasing the `always` block six lines above it, with the same signals in the same order, reveals this defect. The remedy is Chapter 2’s approach: derive the property from the specification’s *prohibition* rather than the implementation mechanism.

It cannot distinguish anything. Each assertion review should identify one RTL mutation that the assertion catches and code coverage does not. The mutation is a small deliberate change to the design code, introduced to see whether a check notices it. Without such a mutation, the assertion adds runtime cost without distinguishing behavior. Counts of signals touched or assertions relative to code size are legitimate structural indicators of effort [1]; they should not become goals. A suite of two hundred properties that restate a register decode, with none on the ordering rules, is worse than a suite of twenty placed where the risk analysis said the design was novel.

11.8 The same text, two engines

One of the stated goals of SVA is a common semantic meaning, so that a single property can drive different classes of design and verification tool [3]. That is why the checker in Section 11.5 is a deliverable rather than a simulation artifact. Three semantic changes must be understood to interpret proof results correctly.

Assumptions stop being checks. The first of the three is Section 11.3’s asymmetry: assumptions are checked in simulation but never in formal, where they delete traces. Simulation setup builds the environment; formal setup states assumptions, and a missing assumption produces spurious failures rather than a silent gap [1].

Covers stop counting and start searching. A `cover` in simulation reports how many times the scenario occurred on this trace; in formal it asks whether the scenario is reachable at all and returns a witness if it is [1]. An empty cover means either “no test produced it” or “no legal execution can produce it”: two different conclusions, only the second about the design.

Liveness becomes decidable. The property `req |-> ##[1:$] gnt` requires the eventual grant of a request, and it cannot fail in simulation: its attempt remains open until the run ends, and because many open attempts degrade performance, simulation properties should bound open-ended intervals with other operators [4]. A proof engine decides the same text. This rule can be written in the common language but answered only by proof; the usual simulation compromise is a bounded restatement (`##[1:16]`), not equivalent to the liveness rule but falsifiable in a finite run. Figure 11.17 shows the two forms on the same pair of requests.

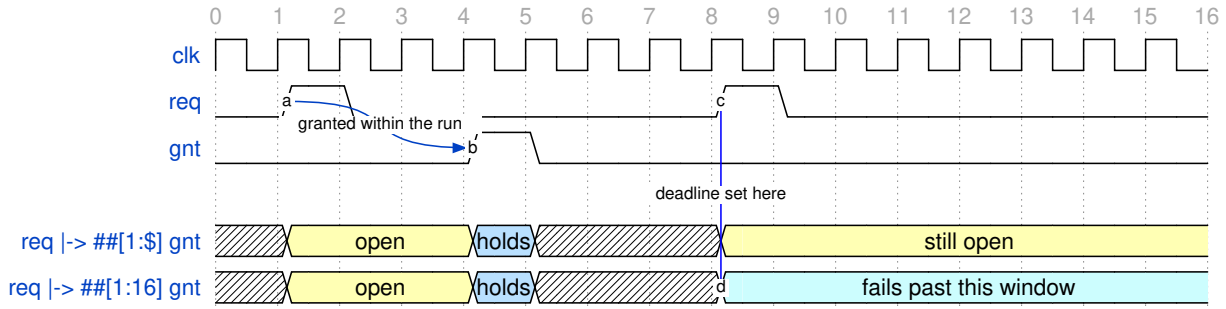


Figure 11.17 — Both forms on one pair of requests: the request at cycle 1 is granted at cycle 4 and both hold, while the request at cycle 8 leaves the unbounded attempt open to the end of the run and gives the bounded form a deadline.

The coherent hub illustrates the distinction. The hub is the block that keeps several caches in agreement about the copies they hold of one memory line. Its invariant requires that at most one cache may hold a line in the Modified state, the state of a line whose only current copy sits in that one cache and may be written there:

```
a_single_writer : assert property (@(posedge clk) disable iff (!rst_n)
    $onehot0(modified_vec));
```

Every system-level simulation of the hub samples this invariant, and Chapter 14 proves it over the hub’s canonical protocol abstraction of two caches and one address: `modified_vec` is the per-line vector of Modified holders, and `$onehot0` is true when at most one of its bits is set. The two engines use the same text; only proof establishes the invariant for the stated protocol abstraction. Figure 11.18 shows the tick at which the invariant fails.

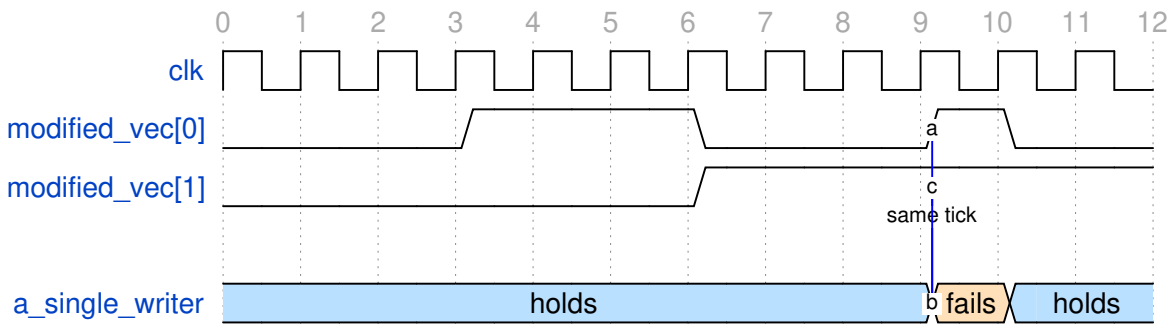


Figure 11.18 — The invariant across two caches: at most one bit of `modified_vec` is set from cycle 0 to cycle 8, both are set at cycle 9, where the assertion fails, and one holder remains at cycle 10.

A 2024 framework for producing SVA from a full natural-language specification reported 89% of its 56 generated properties correct both syntactically and functionally on a single serial-bus design; *correct* there means that the property passed a formal proof against a known-good RTL, and the score was taken after human engineers had removed the generations that referenced signals the tool could not map [7]. Under this definition, a property with an unreachable antecedent also passes proof on bug-free RTL and would score as correct, the issue in Section 11.7. The result does not yet establish a trend, and its correctness criterion does not establish that a property checks the intended behavior. Section 11.1 illustrates the translation failure: protocol documents address experienced engineers and are ambiguous by machine standards, so automated translation chooses an intuitive operator with the wrong cycle relation [2]. The corpus also reports an earlier, narrower result on designs available to download and re-run: the taped-out open-source Ariane (CVA6) core and the OpenPiton manycore framework. In those two hierarchies, 236 liveness and safety properties generated from 110 lines of interface annotation ran across seven forward-progress-critical modules. The properties proved out on the page-table walker and the translation-lookaside buffer after roughly 30 minutes of human effort spent defining the correct transactions; the same generated set found real bugs in the memory-management unit and in an OpenPiton network-on-chip buffer, and both of those modules then proved in full [8]. Transaction annotations are far less ambiguous than protocol prose, which makes the generated properties checkable. Generation should serve as a drafting aid, with its output receiving the same review as hand-written properties.

IN PRACTICE

Two streams of assertion development should meet at integration. Inline assertions should accompany RTL commits and first be used in the designer's unit runs. Protocol checkers should be written from the interface specification during planning and bound at bring-up; in this example, most of their bug-finding occurs in the first two weeks. Chapter 5's plan fixed the `p_no_4kb_crossing` binding point, the backend's AXI write-address channel, before the checker code was written.

If a noisy assertion is disabled with a plusarg and left disabled, failure counts must be audited against the enabled list rather than the file. A plusarg is a switch the simulator takes on its command line, read by the code as the run starts. Assertions in gate-level or emulation runs have to be compiled out or restructured, which is why the macro guards exist. Emulation is the run of a design compiled onto a hardware execution fabric, under stimulus a host supplies. A bound checker with state is software and needs review for its own bugs; if its first week finds more defects in the checker than the design, that alone does not invalidate it.

PITFALLS

1. **No `disable iff`.** Symptom: repeated failures during the first twenty cycles of every test teach users to ignore the suite. Cure: guard every concurrent property with the reset condition, once, immediately after the clocking event.
2. **The assertion that was never in a position to fail.** Symptom: 100% pass rate beside an empty antecedent cover. Cure: write the cover with the property, and treat an empty one as a stimulus bug until proven otherwise.
3. **An implication pointing the wrong way in time.** Symptom: the same timing mistake has two manifestations, failure on legal traffic or silence on the intended scenario. Cure: check the cycle relation explicitly (`| ->` is the same tick, `| =>` the next) and confirm on a waveform which direction `$stable` checks.
4. **The property that paraphrases the RTL.** Symptom: assertion and logic use the same signals in the same order, written in the same hour. Cure: derive from the specification, and state the prohibition rather than the mechanism.
5. **Unbounded eventuality in simulation.** Symptom: a liveness property that has never failed and never will, plus a slow regression from thousands of open attempts. Cure: bound the window for simulation, send the unbounded form to formal.
6. **Assertion count as a goal.** Symptom: a metric rising steadily with no movement in the bug curve, the plot of bug discoveries over time. Cure: measure where the assertions are against where the plan said the risk was.

SUMMARY

- An assertion declaratively expresses a specification in the precise form a tool can evaluate ^[1]. Writing one exposes ambiguity in a specification sentence.
- Concurrent assertions are clocked and use sampled values from before the edge, so they cannot race the design; `disable iff` discards in-flight attempts asynchronously and belongs on essentially every property ^[1, 3].
- `assert`, `assume` and `cover` are three intents over one property text. In simulation the first two are the same check; in formal an assumption is never checked and instead deletes traces ^[1].
- Interface assertions come from the specification and attach with `bind`, keeping verification code out of the design; internal invariants come from the micro-architecture and belong inline ^[3, 4].
- White-box redundancy assertions check agreement between two representations of one fact, detecting silent corruption that affects only one.
- For an implication, an antecedent cover distinguishes exercised checks from vacuous successes ^[1, 3].
- The same text is sampled by a simulator and proved by an engine, but assumptions, covers and liveness all change meaning between them (Chapters 14 and 16).

EXERCISES

Exercise 11.1 (judgement). The crossbar returns a write response to the DMA engine only for an identifier that has an address outstanding, which is clause three of the port contract in §11.5. Name the directive that carries that clause in a block-level environment for the DMA engine and the directive that carries the same text in a block-level environment for the crossbar, and give the reason §11.3 attaches to stating a module’s input rules and output guarantees. State what each of the two directives means to a proof engine, and name what §11.3 says an assumption written stronger than the real environment does to the result of a proof.

Exercise 11.2 (calculation). The port checker of §11.5 increments `w_beats` only on write-data beats that are not the last, and its `c_max_burst` cover looks for the acceptance of an address requesting the 256-beat maximum. Compute the value of the length field on that address and the value `w_beats` holds at the tick the burst’s last beat is accepted, then compute the same two values for a burst of one beat. Show that both cases satisfy the checker’s comparison against the queued head length, and state the identity between the two quantities that makes the comparison work without an adjustment for the encoding. Give the reason §11.5 states for declaring the counter wider than the length field.

Exercise 11.3 (artifact). Read the record below of one bound instance of the port checker of §11.5, taken after several weeks of nightly regression, with the identifier-width parameter left empty. Supply the value the crossbar’s subordinate side requires, state what elaboration does if the parameter keeps its default instead, and name the property of the connection that produces that outcome. Then read the four cover results: name the assertion that each empty cover leaves untested and what each of the two empty covers would have witnessed, give the two explanations §11.5 admits for a cover reading zero, and state which of the two the chapter assigns to these.

```
bind target:      xbar_subordinate_port
checker module:   xbar_port_checker
parameter IDW:
parameter AW:    32
port connection:  .*
a_aw_stable:     no failure on any run
a_w_burst_len:   no failure on any run
a_b_id_outstanding: no failure on any run
c_aw_stall:      nonzero
c_w_overlap:     zero
c_same_id_pair:  nonzero
c_max_burst:     zero
```

Exercise 11.4 (judgement). A review is handed two properties over the same two signals, `ok |-> !err` and `err |-> !ok`, each with a cover on its antecedent. Name the condition under which each of the two passes vacuously, state the rule §11.7 gives for choosing between two logically equivalent formulations, and apply that rule to select one of the two on the premise that `err` is the rare, deliberately constructed condition. State also what §11.7 requires in place of a bare antecedent cover when

the antecedent is ordinary traffic, and name the cover of §11.6 the chapter offers as the model of the stronger witness.

Exercise 11.5 (calculation). §11.8 reports a 2024 framework that produced SVA from a full natural-language specification and scored 89% of its 56 generated properties correct in both syntax and function on one serial-bus design. Compute the number of properties in each of the two outcomes, rounded to whole properties. Name the filtering step §11.8 records between the raw generation and the scored set, give the definition of correct the score uses, and name the class of property that definition admits despite the objection of §11.7.

FURTHER READING

Within the corpus, [1] treats SVA semantics: assertion kinds and evaluation attempts, sampled values and reset, assumptions and coverage as first-class directives, checkers and checker binding, with the fullest available discussion of vacuity. IEEE 1800-2023 [3] is definitive on the four directives, on sampling, on non-vacuous evaluation and on `bind`. For methodology, the assertion chapter of [4] covers authorship, inline versus bound placement, reusable checker packaging and coding rules for simulation performance; [9, 10] place assertions among the other oracles. On automated property generation, [7] reports the state of specification-to-SVA generation as of 2024 and [2] documents why protocol documents in particular defeat it; [8] is the corpus's account of generated liveness and safety properties run against taped-out open-source silicon.

Outside the corpus: H. Foster, A. Krolnik and D. Lacey, *Assertion-Based Design*, is the standard book-length treatment of assertion methodology; the Accellera Open Verification Library remains a readable model of how a reusable checker library is packaged and parameterized.

REFERENCES

1. **B4** E. Cerny, S. Dudani, J. Havlicek, and D. Korchemny, "SVA: The Power of Assertions in SystemVerilog," 2nd ed., Springer, 2015.
2. **P12** Y.-A. Shih, A. Lin, A. Gupta, and S. Malik, "FLAG: Formal and LLM-assisted SVA Generation for Formal Specifications of On-Chip Communication Protocols," arXiv preprint arXiv:2504.17226v1, April 2025.
3. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
4. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
5. **B8** Functional Verification Methodology Team, "UVM Cookbook," Siemens EDA Verification Academy, 2021 (revised 31 January 2022).
6. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
7. **P11** W. Fang, M. Li, M. Li, Z. Yan, S. Liu, H. Zhang, and Z. Xie, "AssertLLM: Generating and Evaluating Hardware Verification Assertions from Design Specifications via Multi-LLMs," arXiv preprint arXiv:2402.00386v4, February 2026.

8. **P15** M. Orenes-Vera, A. Manocha, D. Wentzlaff, and M. Martonosi, "AutoSVA: Democratizing Formal Verification of RTL Module Interactions," Proc. Design Automation Conference (DAC '21), 2021.
9. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
10. **B6** J. Yuan, C. Pixley, and A. Aziz, "Constraint-Based Verification," Springer, 2006.

12. Regression Engineering

Regression engineering turns commits into evidence through repeatable runs and failure triage. In the reference SoC's month nine example, a nightly regression of 900 runs reports 863 passed and 37 failed. Three engineers spend the morning accounting for all 37 failures. Twenty-nine failures report one bug: a scoreboard change that was merged the previous afternoon mispredicts boot-ROM response ordering. The scoreboard is the testbench structure that holds what the design should have produced and compares it with what the design did. Bring-up filed the same misprediction in week three (see Chapter 4); a refactor reintroduced it, and the suite caught it this time instead of an engineer watching waveforms. Four are a real design bug in the neural accelerator's weight prefetcher. Two come from a test that has failed about once a week for two months, now called "the flaky one". One hit the farm's disk quota. And one genuine DMA burst-splitting failure is closed as a duplicate of the flaky test because the logs look similar and the triage window is closing.

The verification techniques were sound: the scoreboard design was right and the merged change broke it. The stimulus was right, and so was the coverage model, the multi-dimensional space that the plan requires the tests to reach. What failed was the process that turns commits into evidence. One fact was reported twenty-nine times, a real find stayed hidden behind a defect that the team had chosen to ignore because the two failures shared one signature, and triage consumed most of a working day of three engineers. Chapter 7 explains how to read regression metrics; this chapter develops the engineering that produces them. [Figure 12.1](#) draws that process, from the commit to a triaged finding.

CHAPTER MAP

§12.1 defines regression tiers and admission rules. §12.2 organizes the (test, seed) matrix for reproducibility, while §12.3 covers per-commit gating and its cost. §12.4 reduces hundreds of nightly failures without losing distinct defects; §12.5 explains why flaky tests cost more than failing ones. §12.6 fits a growing suite into a fixed night and states the tradeoffs.

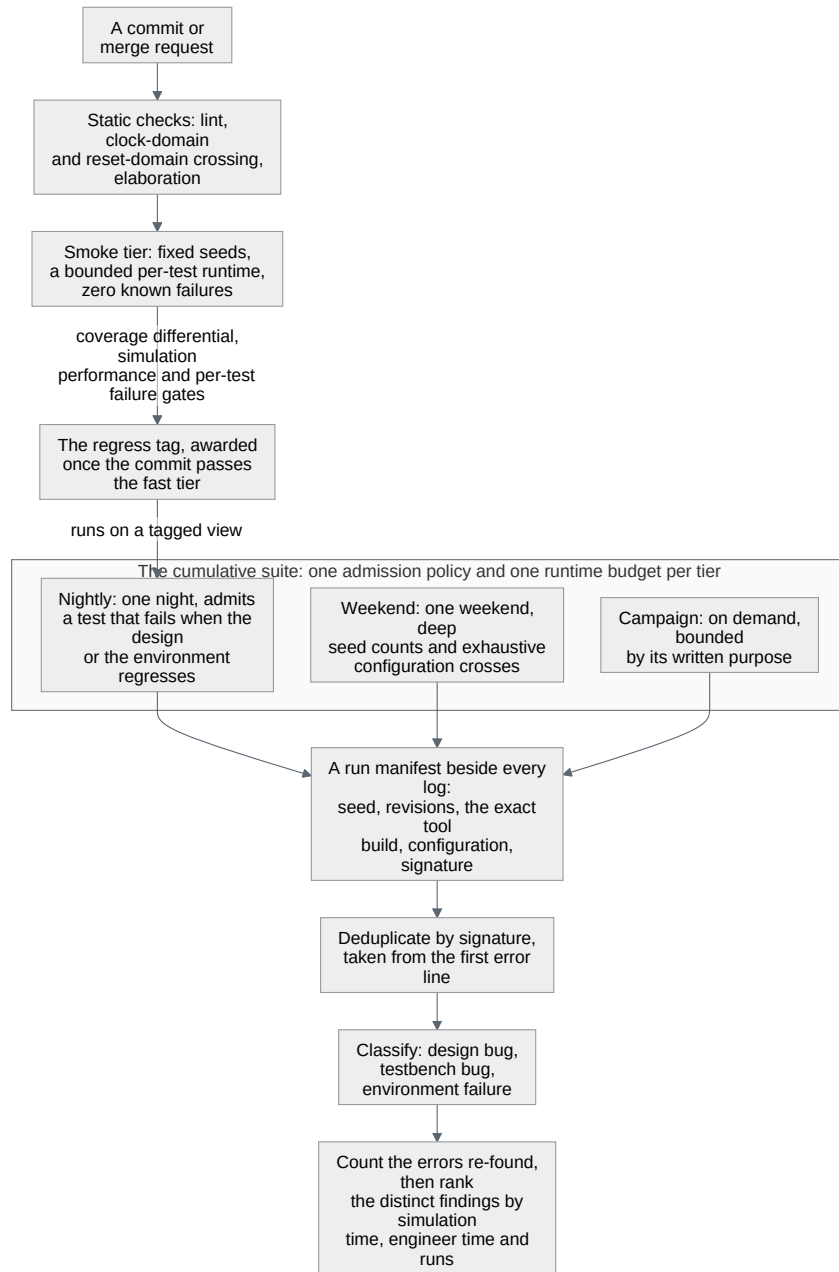


Figure 12.1 — The path a commit takes to become evidence: static checks and a fixed-seed smoke tier gate it, the tagged revision reaches the scheduled tiers and the campaigns, and every run leaves a manifest that triage reduces before anyone debugs.

12.1 Regression suites and admission policies

A **regression suite** checks that today’s design retains yesterday’s verified functionality: a bug fix routinely breaks it, and rerunning the previous checks catches the regression ^[1]. The suite is *cumulative*, so it grows monotonically unless explicitly limited.

Chapter 4 established the cadence and Chapter 7 how to read the resulting metrics. Admission must fit a fixed nightly window. A tier is an admission policy with a runtime budget, rather than a schedule. Table 12.1 gives four policies with their triggers, budgets and admission rules.

Table 12.1 — The four tiers of a regression suite: what launches each one, the wall-clock budget that it has to fit inside, and the rule that a test must satisfy to be admitted to it. The last column carries the argument, because a tier is an admission policy with a runtime budget attached rather than a schedule. The campaign row is the tier that teams keep re-inventing without a name: a purpose-built run owned by someone and answerable to no clock. §12.6 is what happens when the nightly budget stops being enough.

Tier	Trigger	Wall-clock budget	Admission rule
Smoke	every commit / merge request	minutes	proves the build is alive; fixed seeds; bounded runtime; zero known failures
Nightly	scheduled, daily	one night	discriminates: fails when the design or environment regresses
Weekend	scheduled, weekly	one weekend	everything long, deep-seeded, or configuration-crossed
Campaign	on demand	no wall-clock budget; bounded by its written purpose	purpose-built runs that belong on no clock

A published industrial hardware CI setup groups its jobs along almost exactly these lines, down to the extended-duration *stability* configuration pinned to a specific commit hash ^[2]. Job names differ between companies, but this published arrangement matches the tier structure.

The three scheduled tiers have distinct admission rules. **Smoke admission** requires fixed seeds, a hard per-test runtime ceiling and zero known failures: randomization sometimes makes the gate unreliable, while a permanent failure prevents discrimination. **Nightly admission** requires a test that would fail on a plausible design or environment regression; this separates basic-functionality directed tests run nightly from the remainder held for weekends ^[1]. **Weekend admission** covers deep seed counts, exhaustive rather than sampled configuration crosses, and expensive verification engines. A configuration cross is one combination of the design’s configuration options, and a verification engine is one of the tools that run a check (see Chapter 16). Gate-level simulation is the standard case: running the full RTL suite at gate level is prohibitive in labor or schedule terms, so only a minority runs at gate level, almost guaranteeing some missed X-optimism issues ^[3]; selecting that minority is a risk decision (see Chapter 19). Section 12.5 defines X-optimism and gives its regression symptom.

Scenario. In this example, an HBM controller’s PHY training state machine is its main source of late bugs; one training-and-retraining soak covers cold boot, thermal excursion, retrain and refresh collision during retrain, taking four hours across six configurations. **Approach.** All six go to the weekend tier; the nightly keeps a twelve-minute *training smoke* that reaches initial calibration and stops. **Why.** The nightly tier’s budget is the scarce resource and its job is discrimination rather than depth: if calibration itself broke on Tuesday, the team needs to know on Tuesday, while everything past calibration can wait for Sunday without changing anyone’s actions on Wednesday.

The fourth tier gives purpose-built work an explicit place in the suite. A **campaign** is a purpose-built run independent of a schedule: launched for a reason, owned by someone, producing a written result, then stopping. A flash controller’s error-injection campaign sweeps bit-error patterns and burst lengths against the low-density parity-check (LDPC) pipeline across simulated wear states: ten thousand runs over a weekend, once per RTL milestone (see Chapter 4). The LDPC pipeline is the controller’s error-correcting logic, and a wear state is the condition of the flash cells after a given amount of use. Each run is checked by the data-integrity oracle, the mechanism that decides whether an observed response is correct. Scheduling this campaign nightly would consume each night; leaving it outside the tiers makes execution depend on someone remembering it.

12.2 The (test, seed) matrix

A constrained-random test defines a family of experiments indexed by seed, rather than one experiment, since constrained-random means stimulus generated automatically under constraints. Such stimulus is legal but unlikely. A regression runs a **(test, seed) matrix**: tests form rows, seeds form columns, and each cell has its own outcome. Almost every pathology discussed here concerns the matrix rather than an individual test.

Repeated sampling. The commonest failure of constrained-random discipline is to keep the default seed until a simulation runs error-free: the same seed generates the same input sequence, leaving different conditions unverified ^[1]. Passing seed 1 establishes one path through the constraint space; sampling the *distribution* requires repeated draws.

Seed allocation. On the reference SoC, a DMA test drives each transfer through a transfer descriptor, the record that software writes to program one DMA transfer. The test’s four axes are those of `cg_2d_4kb` (Chapter 5): stride sign, distance to the 4 KB boundary, burst length and activity on the other of the DMA’s two channels. A stride is the signed address step from the start of one row to the start of the next, and the 4 KB boundary is the address line that AXI forbids a burst to cross. A DMA channel is an independent transfer engine inside the DMA, with a state of its own and a share of the AXI backend. Such a test has a large space to sample, while a UART framing test with three legal configurations does not, and its seventh seed buys almost nothing. Seed budgets should be allocated proportionally to *unclosed* coverage: groups short of target get seeds; closed, stable groups get one seed as a change detector.

Reproducibility is mandatory and difficult; Chapter 9 explains why and defines a run. Regression engineering adds a supporting artifact and two habits: display the seed at every run’s start and include it in every output pathname, since two runs of the same code with different seeds would otherwise overwrite each other’s results ^[1].

Chapter 7 established that every data point must carry its provenance for the trend lines to mean anything ^[4]. That artifact is the **run manifest**, emitted by every run next to its log.

```
run_id:      nightly-20260312-0417
test:       dma_2d_stride_stress
seed:       3417755291
tier:       nightly
rtl_rev:    9f21ac4           # design repo, tagged 'regress'
tb_rev:    c07e13b           # verification repo
tool:      sim-2025.3-p2      # patch level, not just major version
config:    DMA_CH=2, SRAM_KB=512, XBAR=5x4
plusargs:  +UVM_TIMEOUT=12000000,NO +cov_categories=dma,axi
host:      farm-node-118 (linux-x86_64, 64 GB)
result:    FAIL
signature: SCB_DATA_MISMATCH:dma_scoreboard:beat_compare
cpu_seconds: 641
```

An omitted manifest field can cost a day of debugging. The `tool` field needs more than the *major* version, as Section 12.5 demonstrates. `signature` makes Section 12.4 possible; `cpu_seconds` makes Section 12.6 possible.

Random stability makes the manifest sufficient within a limited scope.

Chapter 9 develops the mechanism and its scope, and Section 12.5 states what it guarantees and where that guarantee stops. The regression-level consequence is that random stability says nothing about changes to the design, to scheduling freedom, or to the simulator build, which is where reproducibility breaks. One inexpensive countermeasure is one *fixed* seed run nightly on a handful of tests alongside fresh seeds. Fresh seeds measure the design; the fixed seed measures the regression machinery, because if its result changes while its manifest does not, something outside the manifest changed.

12.3 Continuous integration for hardware

A hardware test takes minutes to hours versus milliseconds for a software unit test, so running everything on every commit does not transfer to hardware CI. The transferable *principle* is to automatically integrate design changes, run simulations and checks, and deliver verified descriptions or bitstreams, so errors are caught early and design quality stays consistent ^[2]. What must be engineered is the tier that runs per commit.

Static checks precede simulation: one published pipeline groups linting and structural checks (static analysis, clock-domain and reset-domain crossing, elaboration) in a separate job category ahead of simulation ^[2]. Static analysis is inexpensive and deterministic, and it catches defects that simulation catches slowly or not at all; these properties make it suitable for the entry gate, where lint and clock-domain-crossing (CDC) checks run before any simulation (see Chapter 13).

Before automated pipelines, regressions ran on a tagged view: engineers applied a `regress` tag to design and testbench revisions once they trusted basic functionality,

because the newest file might contain unverified code or fail to compile, wasting the regression ^[1]. Per-commit CI retains that goal but awards the tag after the commit passes the fast tier.

Three further inexpensive gates cover coverage, performance and per-test failures, and the first of them reads bins, the counters in which coverage is measured. **A coverage gate** uses a differential between two regressions to identify bins that lost coverage, and a CI run can be failed automatically when constraint work introduces such a regression ^[5], including changes that make the suite faster and greener by checking less. **A performance gate** makes simulation performance a criterion for revision-control check-in, reducing system simulation time ^[4]; a commit that halves the design's cycles-per-second affects the schedule, but that effect remains invisible unless something measures it. **A per-test failure threshold** is the third gate, and one published pipeline sets it on emulation jobs (see Chapter 17): each job carries a configurable failure threshold, and labels selectively disable job categories across the pipeline ^[2]. Since not every test is a hard gate, a work-in-progress branch needs an explicit, recorded exception rather than an engineer silently skipping the tier.

The fast tier need not run a fixed list: a dependency map ties each RTL directory to the testbenches that exercise it, so a commit that touches only the APB peripherals does not trigger the neural accelerator's suite; research goes further by predicting a test's failure probability and assembling a test set from RTL changes ^[6].

Two influences on gate policy. Approaching RTL freeze (see Chapter 4), the fast tier shifts from catching mistakes early to proving a fix broke nothing, so the gate gets *stricter* and exceptions *narrower*: a disable label reasonable in month four leaves a gap in the evidence in month eleven. The social policy is practitioner judgment, unmeasured in this book's corpus of sources: a broken fast tier should be reverted within a stated window rather than fixed in place, because a revert costs one commit while a broken gate affects everyone downstream. Responsibility should follow the commit rather than seniority, making ownership impersonal, and a rota rather than a volunteer should monitor the pipeline.

The fast-tier budget is a design constraint, like timing. In the flagship SoC example, the budget is twelve minutes for lint, CDC and a 60-run smoke suite; when that grew to fifteen, two tests moved to the daily tier rather than accepting the slip, because a gate that exceeds human waiting time invites bypass. One 2026 project reports 4,536 pipelines over three weeks, with successful durations from 6.4 minutes to 2.2 days on average for the largest ^[2].

12.4 Failure triage at scale

The opening example has 37 failures, with one fact reported 29 times. Triage turns failures into distinct findings; at scale, data reduction precedes debugging.

Deduplication by signature is the first reduction. Daily failure analysis begins with two questions: whether failures are new and whether similar failures are the same as one seen before; first-failure analysis does not always answer them because

classification can be *over-generic*, mapping many bugs into one group, or *over-specific*, splitting one bug across many [7]. First-failure analysis is the classification that a failure receives on its first appearance, and it is what those two questions are put to. Both classification errors are expensive: over-generic buckets conceal real finds, as the opening example shows, while over-specific buckets defeat reduction, since 37 failures in 34 buckets require essentially as much work as 37 separate failures. A signature therefore needs a specification: it must derive from the failure rather than the test, exclude everything that varies run to run, and use the *first* error, since later errors are usually consequences.

```
signature := <ERROR_CLASS>:<COMPONENT>:<CHECK>
           from the FIRST error line, with these stripped:
           - timestamps, simulation time, run ids, seeds
           - hierarchical paths below the component boundary
           - numeric operands (addresses, IDs, data values)
example:   SCB_DATA_MISMATCH:dma_scoreboard:beat_compare
counter-ex: UVM_ERROR @ 12480ns [dma_sb] exp=32'hDEAD_BEEF got=32'hDEAD_BEE0
           (over-specific: every seed produces a distinct signature)
```

The seed is stripped from the signature and kept in the manifest. Chapter 9’s rule that the seed travels with the failure is about the failure *report*; the deduplication key must exclude it because a key carrying the seed makes every failure unique, creating over-specific classification.

Failure clustering can be learned from the assignments engineers already make: when an engineer assigns a failed run to a cluster, the system extracts that run’s properties as cluster characteristics and proposes that cluster for similar runs, a capability that one vendor describes as in progress [7]. Results remain incomplete: in one study reported by a 2023 survey, 616 features from simulation-log metadata and message lines gave clustering scores well short of ideal, while supervised root-cause classification reached about 91% accuracy with a random-forest model [8]. The proposed division of labor is that automation supplies the bucket, a human confirms it, and that confirmation becomes training data.

The second reduction classifies three kinds of failure: design bugs, testbench bugs and environment failures, whose ratio is a health metric. Testbench bugs require the same bug-tracking discipline as design bugs, because the verification environment is itself a design (see Chapter 4). Environment failures, such as disk quota, license starvation or a dead farm node, are the only category that may be discarded without debugging; even they must be *counted*, because a rising count reveals an infrastructure problem that resembles flakiness.

The third reduction avoids debugging the same failure twice. A regression should track errors *re-found*, measuring wasted simulation [4]: in this example, re-discovering three known-open bugs consumes 40% of the night, so that 40% supplies no new evidence. Known-failure filters should use signatures and expire when the bug closes. The re-found count shows when known-failure filters have gone stale.

Debug economics. Debug dominates a verification engineer’s time (see Chapter 7); this chapter develops the *per-failure* accounting needed to manage that time and pri-

oritize the work. Bug closure should record three quantities, simulation time, engineer time and runs consumed, so triage can rank work rather than process an undifferentiated queue. Because instrumentation itself costs simulation resources, a regression can run with most metrics disabled and then re-run only the failing tests, each with the metric categories that its own failure type needs [4]. Three practices follow in one vendor’s regression-management flow: prioritize the highest-impact failure modes, identify the *shortest* test reproducing each, and rerun automatically to generate the debug data the engineer will need without making them ask [7].

A **triage rota** should assign one engineer per week to the morning queue, responsible for classification, deduplication and assignment, and for filing what the buckets reveal about the regression machinery; this role does not include debugging. Combining debugging and triage makes them compete for the same engineer’s time, so debugging can displace triage.

12.5 Flakiness and regression reliability

A **flaky test** changes outcome between runs with identical manifests: test, seed, revisions, tool and configuration. Its outcome no longer provides reliable passing or failing evidence. The term is borrowed from software testing, where the problem has a name and a decade of study.

The judgment here is that *a flaky test costs more than a failing one*. A failing test signals a problem to investigate, whereas repeated flaky failures teach teams to ignore failures elsewhere, undermining regression evidence. The opening DMA failure was closed as a duplicate of “the flaky one”, consistent with the response that repeated regression results had taught the team.

The sources below document recurring causes of nondeterminism in the environment and the design.

- **Sampling and scheduling races.** Synchronous interface signals must be sampled and driven through a clocking block (IEEE Std 1800-2023, *IEEE Standard for SystemVerilog* [9]), which avoids races between design and verification environment and lets one environment work against RTL and gate-level models unchanged. Active-region sampling allows races: when the design’s update and monitor’s sample share a time step, their relative order is unspecified. The active region is the part of a simulation time step in which the design’s updates and the testbench’s own statements are evaluated. Two *testbench* processes resuming on the same edge require a distinct ordering solution, developed in the worked example. The same family includes clock edges at time zero and environment instantiation, where a local variable prevents initialization races [10].
- **Uninitialized state.** X-optimism is simulation behavior in which a deterministic 0 or 1 propagates where silicon would have an unknown; it can mask RTL design bugs, though not all of it is unwanted, since asynchronous reset works *because* of it [3] (see Chapter 13). Its regression-level symptom is a result that depends on what a register happened to hold.

- **Time-dependent checks.** A check written against wall-clock-like quantities rather than design events is nondeterministic by construction. For the Ethernet MAC's time-sensitive networking (TSN) queues, a timestamp-precision check derives its tolerance from the *host's* measured latency instead of the design's own clock. That check passes on a quiet farm and fails on a busy one.
- **Provenance drift.** Random stability guarantees an unaffected instance the same sequence for the same seed even when other instances change ^[1], but its scope is randomization-related *source* changes; it says nothing about a different simulator build. A manifest recording the tool's major version and not its patch level permits flakes.
- **Infrastructure collisions and runaways.** If two simulations of the same code run with different seeds and write to the same output filename, the first run's results are lost: the run surfaces as one that “failed for no reason” and passes when rerun alone. A simulation waiting for an event that never occurs runs until 64-bit simulation time reaches its maximum, effectively forever; every simulation therefore needs a time bomb, used only for abnormal termination so success remains distinguishable from deadlock ^[1]. A watchdog, required in every system-level environment and advisable at lower levels too, must likewise halt a run when no progress is observed, since such runs block the tests behind them ^[10].

Instability should be measured. One published flow measures stability for tests it cannot parallelize: an operating-system boot cannot be split across machines, so it is executed eight times specifically to establish that test's stability ^[2]. A **repeat-N stability check** should precede admission of any test to a gating tier. A confirmed flake should enter **quarantine**, a named list on which it is still run and reported but does not gate, with an owner and a date. Quarantine requires resolution; an untouched list preserves the instability.

Worked example: hunting a one-in-four-hundred race

Over twelve weeks, sixty nights of the reference SoC's suite, `SCB_DATA_MISMATCH:dma_s coreboard:beat_compare` appeared 35 times, always on a different seed. Each morning it was closed as “the flaky one”. The suite's 900 nightly runs include 260 that instantiate the DMA environment, so the field rate is 35 failures in 15,600 DMA runs: about one in 450.

Step 1: signature refinement. The first check tests whether the 35 failures have one cause. The signature was over-generic ^[7], and three of the 35 failures carried distinct first-error lines that a stricter signature separated out. Two were a real, already-fixed bug in the burst splitter, the DMA stage that splits each transfer into bursts that the AXI address rules allow. One was a farm node that died mid-run. Thirty-two shared a first error and component, reducing thirty-five cases to one investigation at a field rate near one in 490.

Step 2: campaign statistics. One in 490 is difficult to measure at 260 runs a night. A weekend campaign ran 4,000 fresh seeds of the DMA test set: $4,000 \times 11$ minutes = 733 CPU-hours, or 30.6 hours of wall clock on the reference SoC's 24 slots, and produced 9 failures, one in 444.

Step 3: manifest correction. Rerunning the 9 failing manifests reproduced 6 but not the other 3. With the same seed and the same revisions, those three should have reproduced as well. The farm actually ran two simulator patch levels, while the manifest recorded only the major version. Pinning `tool` to the exact build made all 9 reproduce deterministically. This diagnosed provenance rather than the race; without it, subsequent experiments would have been unrepeatable and the team would have attributed intermittent failures vaguely to the design.

Step 4: observation-path bisection. Reproducibility exposed two parts of the race. The response monitor sampled the DMA backend's response channel from an `always @(posedge clk)` block rather than through a clocking block; it also *popped* the expected item and compared it immediately, in the same region where the command monitor pushed it. Processes sensitive to one event execute in arbitrary order [9]; the simulator chose which of the two ran first, explaining the disagreement between two patch levels. The race mattered only when a response arrived in the same cycle as the previous descriptor's last beat, the last of the data transfers that make up an AXI burst. That arrival requires the 16-entry backend FIFO to sit exactly at a boundary, a coincidence the random descriptor stream produces about once in four hundred and something runs.

```
// before: samples in the active region, and pops the expected item there too,
// so the pop and the command monitor's push are ordered by the simulator
always @(posedge clk)
    if (rvalid && rready)
        compare(exp_q.pop_front(), sample_response(rdata, rresp, rid));

// after: sample through the clocking block, and only enqueue – the scoreboard
// owns the comparison, so the order is stated rather than inherited
clocking mon_cb @(posedge clk);
    default input #1step;
    input rvalid, rready, rdata, rresp, rid;
endclocking
...
@(mon_cb);
if (mon_cb.rvalid && mon_cb.rready)
    obs_q.push_back(sample_response(mon_cb.rdata, mon_cb.rresp, mon_cb.rid));
```

Two changes each address one half of the race. Every field the monitor records now comes from `mon_cb`, and a `#1step` input skew samples in the preponed region, the value present before the clock transition [9], so capture no longer depends on design activity at that edge. The monitor enqueues instead of popping, and the scoreboard compares in its own process.

The clocking block defines sampling time but does not order two testbench processes. Moving *both* monitors behind clocking blocks on the same clock makes them resume together without defined relative order, reintroducing the bug. Enqueuing for comparison in a separate process closes the race; the clocking block makes the sampled evidence reliable.

Step 5: post-fix rate bound. Re-running the 4,000-seed campaign produced zero failures. Zero events in 4,000 trials bounds the true rate below roughly 1 in 1,300

with 95% confidence. The campaign therefore establishes a bound on the rate, below 1 in 1,300, without proving a zero rate or a fix.

The race required a small fix, but twelve weeks of repeated signatures trained the team to close DMA scoreboard mismatches unread, including at least one genuine failure.

12.6 Compute economics

The physical compute budget constrains regression decisions. Additional cores increase throughput, but redundant simulations consume simulator resources and lengthen turnaround without improving coverage ^[11].

Two published results quantify the scale. In a 2023 study, a commercial signal-processing unit required six months of simulation of nearly two million constrained-random tests on almost a thousand machines and EDA licenses, at about two hours per test ^[12]; four million CPU-hours, approximately what a thousand machines produce running continuously for six months. The farm was saturated throughout the project, leaving no capacity for more runs. In a 2026 report, distributing a 1,738-test suite across an eight-board FPGA-emulation cluster cut runtime from about 5.2 hours to about 53 minutes: a 6× rather than 8× speed-up, with near-linear gains only for the independent categories and none for tests that cannot be divided ^[2]. This also demonstrates a parallelism limit when boards constrain capacity.

Compute cost has both a hardware term and a license term, and the two do not scale together. A published per-job cost model for a two-socket simulation server records what happens as simulations fill a socket: cache bandwidth drops and the cache miss rate rises, until cores stall waiting for data and processor utilization declines; the per-job cost rises once license cost eventually dominates hardware cost, and where the license dwarfs the server, packing many jobs onto one socket generally saves nothing ^[13].

A limited budget requires run selection.

Ranking. Ranking chooses tests by their contribution to coverage improvement, producing a compressed suite of specific (test, seed) pairs and eliminating redundant runs ^[11]. The idea predates the tooling: coverage tools grade data sources by absolute contribution or rate of contribution over time, and the selected simulations *together with their initial seed values* form the regression suite, so running the most efficient first reaches the same coverage rating in less time ^[10]; the highest-contributing seeds should run first, with remaining time filled by fresh random seeds ^[1]. Ranking adjusts run *order* and *frequency* as well as membership ^[4].

In 2022 industrial results (posted in 2024), an automotive microprocessor IP suite of 26 tests at 10 seeds each (260 runs, 12 CPU-hours) compressed to 8 runs and 1.1 CPU-hours at 100% of original coverage. An industrial mixed-signal SoC suite of 5,124 runs across seven configurations compressed to 1,204, a factor of 4.25, again regaining full coverage ^[11]. Both are Infineon Technologies' own projects; the paper reports three in all and compares two methods, classical seed ranking and Cadence

Xcelium ML; that tool learns the correlation between randomization points and achieved coverage in earlier regressions and then emits an optimized pattern set; the automotive microprocessor IP is an ASIL-D safety product implemented in VHDL [11]. The last of those facts names a safety rank: ASIL-D, the most demanding of the four automotive safety integrity levels. The paper identifies its projects by class rather than by part number, so that is as far as the attribution goes.

Ranking limits. A ranked regression selects existing test-and-seed pairs and simulates no new scenarios, so it is not recommended for exploring the random space or exposing new bugs [11]. The compression must be regenerated whenever design or testbench changes significantly.

Test selection. The complementary technique filters tests *before* simulating them, on the hypothesis that dissimilar tests hit dissimilar coverage events. The 2023 signal-processing unit of the first result above belongs to the same class as the radar front-end, one of the example devices this book returns to. On that unit, neural-network novelty selectors beat random selection in every configuration; the best saved 49.37% of simulation to reach 99.5% coverage, with computation negligible against simulation avoided [12]. Machine-learning-generated regressions remain random: in the same 2022 results, optimized regressions regained 99% or more of original coverage at roughly 3× compression, with some configurations exceeding 100% by hitting previously missed bins [11]. Ranking trades exploration for speed, while novelty selection seeks speed through exploration.

Scheduling by predicted reachability. A third technique predicts, for each RTL instance, which tests are likely to reach it, then allocates the next regression batch from those predictions rather than spreading seeds uniformly across the suite. Across three designs, one of them the open-source IBEX core, its authors report reaching constrained-random saturation 32% sooner and on 40% fewer resources, with the average coverage improvement itself under three points [14]. Saturation here is the stimulus saturation of Chapter 7: the state in which more seeds mostly re-confirm what a fixed constraint set has already reached.

Two smaller controls complement selection: pruning *coverage collection* rather than tests trades a bounded loss of measurement for compute (see Chapter 7), and nightly runtime should itself be a metric with an owner, so a spike in a block's regression time is detected before its cause becomes difficult to reconstruct [4].

Worked example: 40 runs, 900 runs, and a suite that no longer fits

Week 6: 40 runs. Forty directed tests at one seed each and a mean two minutes take 80 CPU-minutes on one workstation over lunch, with one engineer reading every log. In this example, the engineer uses no tiers, signatures or manifests; none is needed at this scale, where the chapter's techniques would add overhead.

Month 9: 900 runs. The suite that was 40 tests is now 180 distinct tests: 120 constrained-random at 7 seeds each (840 runs) plus 60 directed at one seed (60 runs). At a mean 11 minutes per run, the nightly costs 165 CPU-hours, or 6.9 hours on 24 concurrent simulator licenses within a 10-hour night. Smoke is 22 fixed-seed runs of

at most 90 seconds, one wave across the same 24 slots: under nine minutes end to end, most of it spent compiling. At 40 seeds per random test, the weekend costs 4,800 runs plus the 60 directed: 891 CPU-hours, or 37.1 hours on the same 24 slots within 48 hours. The suite fits, but growth consumes the headroom in four months.

The flagship SoC exceeds the nightly budget. The reference SoC's DMA suite carries forward directly, with the same discipline of splitting every burst at the 4 KB boundary and the same three-stage architecture of register frontend, burst splitter and AXI backend. The register and peripheral suites carry forward too, and the plan adds what eight DMA channels, a coherent host cluster and a mesh NoC demand. A NoC is the on-chip packet network that takes the place of a bus once too many blocks share one path. The nightly *candidate* list is 6,400 runs (Table 12.2):

Table 12.2 — The flagship SoC's nightly candidate list before any re-tiering: 6,400 runs in three groups, each with that group's mean run time and the CPU-hours it costs. The total is the number that does not work — 4,409 CPU-hours on the farm's 240 concurrent simulator licences overruns the night, and that overrun is the gap the re-tiering, the ranking and the pruning have to close.

Component	Runs	Mean	CPU-hours
System-level (OS boot, coherent traffic soak)	340	3.5 h	1,190
Constrained-random block/subsystem (600 tests × 8 seeds)	4,800	30 min	2,400
Directed and configuration runs	1,260	39 min	819
Total	6,400	41 min	4,409

The farm permits 240 concurrent simulator licenses, so 4,409 CPU-hours takes 18.4 hours. Against a 10-hour night, the 8.4-hour overrun delays the summary until 11:00, disrupting the triage rota's day. The following three decisions are applied in this order:

1. **Tier reassignment.** Three hundred of the 340 system-level runs move to the weekend; 40 stay as a nightly system smoke. Cost falls by 1,050 to **3,359 CPU-hours** (14.0 h wall), still above budget.
2. **Random-block ranking.** The 4,800 random runs compress to 1,150, a factor of 4.17 consistent with the 4.25 reported in 2022 for an industrial mixed-signal SoC [11], requiring 575 CPU-hours. Cost falls by 1,825 to **1,534 CPU-hours** (6.4 h wall).
3. **Directed-set pruning.** One hundred and eighty directed runs are retired under the hygiene rules below, leaving 1,080 runs at 702 CPU-hours. Cost falls to **1,417 CPU-hours**, or 5.9 hours, with four hours of headroom for the subsequent growth assumed in this example.

The weekend absorbs 300 system runs (1,050 CPU-hours), the *unranked* random set at 20 seeds instead of 8 (12,000 runs, 6,000 CPU-hours), and the **retained** directed set (702); the 180 retirees leave both tiers for an on-demand parole list. Total 7,752 CPU-hours, 32.3 hours on 240 slots inside a 48-hour window.

The nightly tier is ranked; the weekend remains unranked. A ranked regression reproduces known coverage without exploration, so using it as the *only* regres-

sion leaves new scenarios untested despite a green dashboard ^[11]. Chapter 7 calls the result the flat-curve pathology: a green regression beside a coverage curve that has stopped moving. Ranking fits the nightly budget, while the unranked weekend restores exploration; the ranking must be regenerated whenever the design or test-bench moves significantly, so it remains a compression of the current design rather than an outdated version.

Regression hygiene over time

Decision 3 needs retirement rules: without retirement rules, a suite keeps growing because adding tests is rewarded while deletion is treated as lost verification and a risk of removing a future bug detector. Three categories have explicit detection rules.

Tests whose reason has expired. Many tests target a specific bug, whose closure indicates when the test no longer needs to run ^[4]; closure therefore triggers regression maintenance. Because a bug test protects against recurrence, closure must record a decision to retain it permanently, demote it to weekends or retire it, rather than automatically deleting it.

Tests that duplicate other tests. Regressions contain redundant simulations that add no coverage ^[11]; successive rankings expose this at no extra cost when they drop every seed of a test because the remaining suite covers its contribution. Measuring that duplication does not release runtime until the redundant tests are removed from scheduled runs.

Tests without a justification. Every test should discharge a plan row, a coverage group or a closed bug (see Chapter 5). A plan row is the line of the verification plan that names a feature and the evidence that closes it. A test that discharges no plan row, adds no unique coverage and finds no bug in six months is a retirement *candidate*, and it requires a one-line review because wrongful deletion has asymmetric costs.

```
RETIRE  dma_desc_legacy_mode          (60 runs/night, 11 CPU-h)
Reason:  the legacy descriptor format it exercises was removed from the RTL at
         rev 9f21ac4; no vplan row now depends on it
Unique coverage contributed:  0 bins over last 12 rankings
Bugs found:  1, closed 2025-11-04, fix protected by a_no_4kb_crossing
Reviewed by: DMA owner + verification lead  Decision: retire; 2-month parole on-
demand
```

Two further costs require control. Functional metrics require architecture, implementation and maintenance, plus simulation resources ^[4], so unread coverage groups waste resources as unread tests do. Regression *data* grows faster than tests, so tracking snapshots should be retained and raw results discarded (see Chapter 4). In the morning queue, disk exhaustion appears as a failure just as a design defect does.

IN PRACTICE

Configurations add a dimension that multiplies the (test, seed) matrix: one industrial mixed-signal SoC suite in 2022 spanned seven configurations distinguished by the presence of a CPU, a verification IP and a particular flash type ^[11]. A verification IP is a ready-made checking component for a standard protocol, typically bought rather than written. Infrastructure ownership should name a person, since shared responsibility can leave ownership contested or unassigned. The first ranking exercise can expose substantial duplication and be read as criticism of test authors; this should be understood as a consequence of incremental growth, one bug at a time.

PITFALLS

1. **Reduced checking after a constraint change.** A pass-rate rise after a constraint change may reflect reduced checking; coverage differentials should gate CI alongside pass/fail ^[5].
 2. **The manifest that omits the tool build.** Recording only the major version makes a class of flakes undiagnosable and can make them resemble design bugs for weeks.
 3. **Over-generic signatures.** One bucket containing many distinct defects can conceal a real find ^[7]; such a bucket requires a more discriminating signature.
 4. **Ranking the only regression.** A compressed suite reproduces known coverage without new exploration; an unranked tier and regeneration after design changes are therefore required ^[11].
 5. **The time bomb as normal termination.** Routine watchdog termination makes success and deadlock indistinguishable ^[1], preventing the regression from discriminating between them.
 6. **Triage without a rota.** Without a rota, dependence on one engineer leaves triage uncovered during that engineer's week off.
-

SUMMARY

- Regression tiers require engineering as admission policies with runtime budgets.
- The unit of work is the (test, seed) pair. One seed samples a distribution once ^[1]; seed budgets should follow unclosed coverage.
- Reproducibility must be *easy*, beyond merely possible ^[10], through a per-run manifest of seed, revisions, exact tool build and configuration. An incomplete manifest is a defect in its own right.
- Hardware per-commit gating needs a fast tier preceded by static checks ^[2], with coverage-differential and simulation-performance gates too ^[4, 5].
- Triage is data reduction before it is debugging: signatures neither over-generic nor over-specific ^[7], three-way classification, and a measured count of errors re-found ^[4].

- Flakiness harms team judgment by teaching engineers to ignore failures. Flake investigations need statistics and an explicit account of what a clean rerun establishes.
- Ranking and selection reduce runtime, but a ranked regression explores nothing new ^[11], requiring an unranked tier.

EXERCISES

Exercise 12.1 (judgement). A team proposes admitting a new test to the smoke tier: it draws a fresh seed on every run, finishes well inside the tier’s budget, and fails on a minority of runs for a cause nobody has identified. Name which of the tier’s three admission rules the proposal breaks, give the reason §12.1 attaches to each of those, and name the rule the test does satisfy. Name the check §12.5 requires before any test enters a gating tier, and name the list a confirmed flake belongs on together with the two attributes §12.5 requires each entry of that list to carry.

Exercise 12.2 (artifact). Read the run manifest below, emitted beside the log of a failing run of the DMA test of §12.5, with one field left empty. Name the empty field, state the class of failure its absence permits, and name the step of §12.5’s worked example that found the same omission and what that step established. Name also the field of this manifest that the deduplication key of §12.4 must exclude, give the classification error that including it would produce, and state where §12.4 keeps that field instead.

```
run_id:      nightly-20260312-0417
test:       dma_2d_stride_stress
seed:       3417755291
tier:       nightly
rtl_rev:    9f21ac4
tb_rev:     c07e13b
tool:
config:     DMA_CH=2, SRAM_KB=512, XBAR=5x4
host:       farm-node-118 (linux-x86_64, 64 GB)
result:     FAIL
signature:  SCB_DATA_MISMATCH:dma_scoreboard:beat_compare
cpu_seconds: 641
```

Exercise 12.3 (calculation). After the signature refinement of §12.5’s worked example, a weekend campaign measures a field rate of one failure in 444 runs, from 9 failures in 4,000 fresh seeds. Compute the number of DMA runs the nightly tier would have to accumulate to produce those 9 failures at that rate, and convert the result into nights at the tier’s own DMA run count. Compute the campaign’s wall-clock time on the reference SoC’s slot count, and state what the comparison of the two figures establishes about measuring a rate of this size on the nightly tier.

Exercise 12.4 (calculation). The month-9 example of §12.6 runs 180 distinct tests as 120 constrained-random tests at 7 seeds each plus 60 directed tests at one seed, at a mean 11 minutes a run, on 24 concurrent simulator licenses inside a 10-hour night. Compute the run count, the CPU-hours and the wall-clock time of that nightly,

then compute the CPU-hours the night still admits at the same slot count. Compute the ratio of the weekend's 891 CPU-hours to the nightly's figure, and state what §12.6 says growth does to the headroom.

Exercise 12.5 (judgement). A proposal keeps the ranked flagship nightly tier of §12.6 and retires the unranked weekend, on the argument that ranking regained the coverage at a quarter of the random set's cost. Name the evidence a ranked-only regression cannot produce and the property of the weekend tier that restores it, and name the pathology of Chapter 7 the resulting dashboard would display. Name the maintenance obligation §12.6 attaches to a ranking and the event that triggers it. State which of the flagship's two scheduled tiers §12.6 leaves unranked and the seed count it runs the random set at there.

FURTHER READING

Corpus sources cover the following topics. ^[1] §7, *Simulation Management*, covers seed management and random stability (p. 364), then regressions (pp. 365–369): the nightly/weekend split, seed ranking, time bombs and tagged views. ^[10] supplies the reproducibility requirement, watchdog rule and environment-side race rules responsible for most flakes; ^[9] defines scheduler guarantees. ^[11] is the reference for ranking and machine-learning regression compression, with quantified industrial results and an explicit account of compression costs; one of its authors works for the vendor of the evaluated tool. ^[12] supplies both the farm-scale calibration and novelty-selection results. ^[7] and ^[4] describe regression management, failure clustering, test ranking and metrics; both are vendor accounts without population data. ^[5] covers coverage-based CI gating and coverage-collection pruning, with measurements. ^[2], an arXiv preprint, is this corpus's most recent hardware CI tiering account; ^[6] reviews machine learning in simulation-based verification, including test selection; ^[8] surveys automated failure clustering and root-cause classification.

Outside the corpus, software CI literature applies once the unit-test/simulation cost asymmetry is accounted for; software-testing literature names flakiness and provides a decade of study.

REFERENCES

1. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. **P25** S. Ahmed, A. Kropotov, R. I. Genovese, B. Homs, E. Merino, F. Urbani, et al., "Verification and Validation (V&V)-in-the-Loop for RISC-V Design: The Holistic Vision of BZL," arXiv preprint arXiv:2604.27013v1, April 2026.
3. **D23** S. Zhao, S. Yan, and Y. Feng, "Practical Approach Using a Formal App to Detect X-Optimism-Related RTL Bugs," Proc. DVCon U.S., 2014.
4. **D20** A. Meyer and H. Foster, "Metrics in SoC Verification: Not just for coverage anymore," Proc. DVCon U.S., 2012.
5. **D22** S. Hristozkov, J. Pallister, and R. Porter, "No Country For Old Men – A Modern Take on Metrics Driven Verification," Proc. DVCon Europe, 2021.
6. **P8** C. Bennett and K. Eder, "Review of Machine Learning for Micro-Electronic Design Verification," arXiv preprint arXiv:2503.11687v1, March 2025.

7. **D26** D. Zhang, "Smarter Verification Management with vManager Platform," Proc. DVCon China, 2021.
8. **D35** D. Yu, H. Foster, and T. Fitzpatrick, "A Survey of Machine Learning Applications in Functional Verification," Proc. DVCon U.S., 2023.
9. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
10. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
11. **P7** D. N. Gadde, S. Simon, D. Lettnin, and T. Ziller, "Improving Simulation Regression Efficiency using a Machine Learning-based Method in Design Verification," Proc. DVCon Europe, 2022.
12. **P5** X. Zheng, K. Eder, and T. Blackmore, "Using Neural Networks for Novelty-based Test Selection to Accelerate Functional Coverage Closure," Proc. IEEE International Conference on Artificial Intelligence Testing (AITest), Athens, Greece, pp. 114–121, 2023.
13. **D44** D. Lacey and E. Powell, "Creating the Optimal Regression Farm Infrastructure That Meets All Your Team's Simulation Requirements," Proc. DVCon U.S., 2018.
14. **D45** L. Ferretti, C. Behera, S. T. Bandlamudi, N. Athreyas, V. Narayan, and S. Mittal, "A Scalable Gray-Box Instance-Based Reachability Predictor for Automated DV Regression Scheduling," Proc. DVCon U.S., 2025.

PART IV

Static and Formal

Proving instead of sampling: from the static entry gate to model checking, the industrialised formal applications, and the hybrid flows that make one campaign of both.

13. Static Verification

Static verification checks design structure before simulation. The reference SoC's wake-up defect illustrates the scope of that check and is also a failure that laboratory attempts have not reproduced. In this example, one board in forty hangs on wake-up from deep sleep, roughly once a day, always when the always-on domain returns control to the switchable system domain. RTL regression passes eleven thousand seeds over every planned wake path, and throughout that regression no error comes from the scoreboard, the structure that holds the value each output should take and is consulted whenever an output is observed.

The defect is invisible at the simulation's level of abstraction. The event unit encodes the reason for a wake-up in a three-bit cause code, produced on the 32 kHz always-on clock and consumed on the 200 MHz system clock. The sparse encoding has four legal codes across eight code points; each of the three bits crosses through its own two-flop synchronizer. The three synchronizers, one per bit, individually satisfy lint checks. But when the code changes from `3'b010`, a GPIO wake, to `3'b100`, a comparator wake, two of the three bits change at once, and on the crossing edge each destination flop resolves independently. For exactly one system cycle the destination can therefore observe `3'b110`, a reserved code the source never produced. The decoder has a branch for each legal cause and none for the rest, so for that cycle it holds its previous decode. The wake sequencer is the hardware block that dispatches the wake steps once a cause is decoded. As the cause code changes, the wake sequencer latches the stale decode and dispatches nothing new, so the request-and-acknowledge handshake never completes.

The source flops change at one simulated instant and the destination samples exactly that value one instant later; metastability requires a flip-flop with real timing. Demonstrating this class of bug in simulation needs a gate-level model with timing, which does not exist yet at RTL sign-off, and the same bugs are hard to reproduce in the lab once silicon arrives ^[1].

Structural analysis could find this in seconds, before a testbench exists, by traversing the elaborated design and checking every clock-domain crossing for protection against independent resolution of its bits. It requires no stimulus, no seed and no reference model, the independent executable of the specification that a dynamic check compares the design against. This chapter covers that analysis, the first gate in Part IV.

CHAPTER MAP

§13.1 defines static analysis's benefits before simulation and its limits. §13.2 covers lint findings, report noise and rule-set policy. §13.3 develops clock-domain crossing, metastability, mean time between failures and synchronizer structures; §13.4 distinguishes structural and protocol checks. §13.5 explains reset-domain crossing as a

separate discipline and its findings in single-clock designs. §13.6 covers optimistic and pessimistic X-propagation and the costs of gate-level, simulator and formal approaches.

13.1 The entry gate

Static verification is analysis of the design description itself. It requires no stimulus and no statement of the expected output; the expectations are built into the checker [2].

Chapter 3 framed dynamic verification as bounded by controllability and observability. Static analysis drives nothing, so no condition is unreachable and no coverage question arises; it observes no behavior, so it cannot report what the design computes. It examines the shape of the code and the netlist that code implies, exhaustively, reporting every instance of a structural pattern it was taught to recognize.

It runs the first day RTL compiles, weeks before Chapter 4's bring-up environment drives its first checked transaction, allowing authors to catch bugs the afternoon they write them, when correction costs least [2]. It is deterministic, with no seed or run-to-run variation on unchanged source, so it can serve as a gate (see Chapter 12) and an audited sign-off row with its own waiver file (see Chapter 7). Static analysis also reaches classes of defect that simulation reaches slowly or never; the wake-up example illustrates why static clock-domain-crossing tools were needed earlier than gate-level timing simulation could address these defects [1].

A static tool detects problems in code structure rather than algorithms or data flow, just as a spell checker detects misspellings but misses *with* substituted for *width* [2]. Every *structural* check in this chapter is a check on form, and a design that passes all of them can still compute entirely the wrong result. Sections 13.4 and 13.6 add the functional half, and it takes an engine that evaluates behavior.

Checks fast and deterministic enough for every commit belong inside the static gate; slower checks or those needing more setup belong on a nightly or campaign schedule. Figure 13.1 places each check relative to the gate.

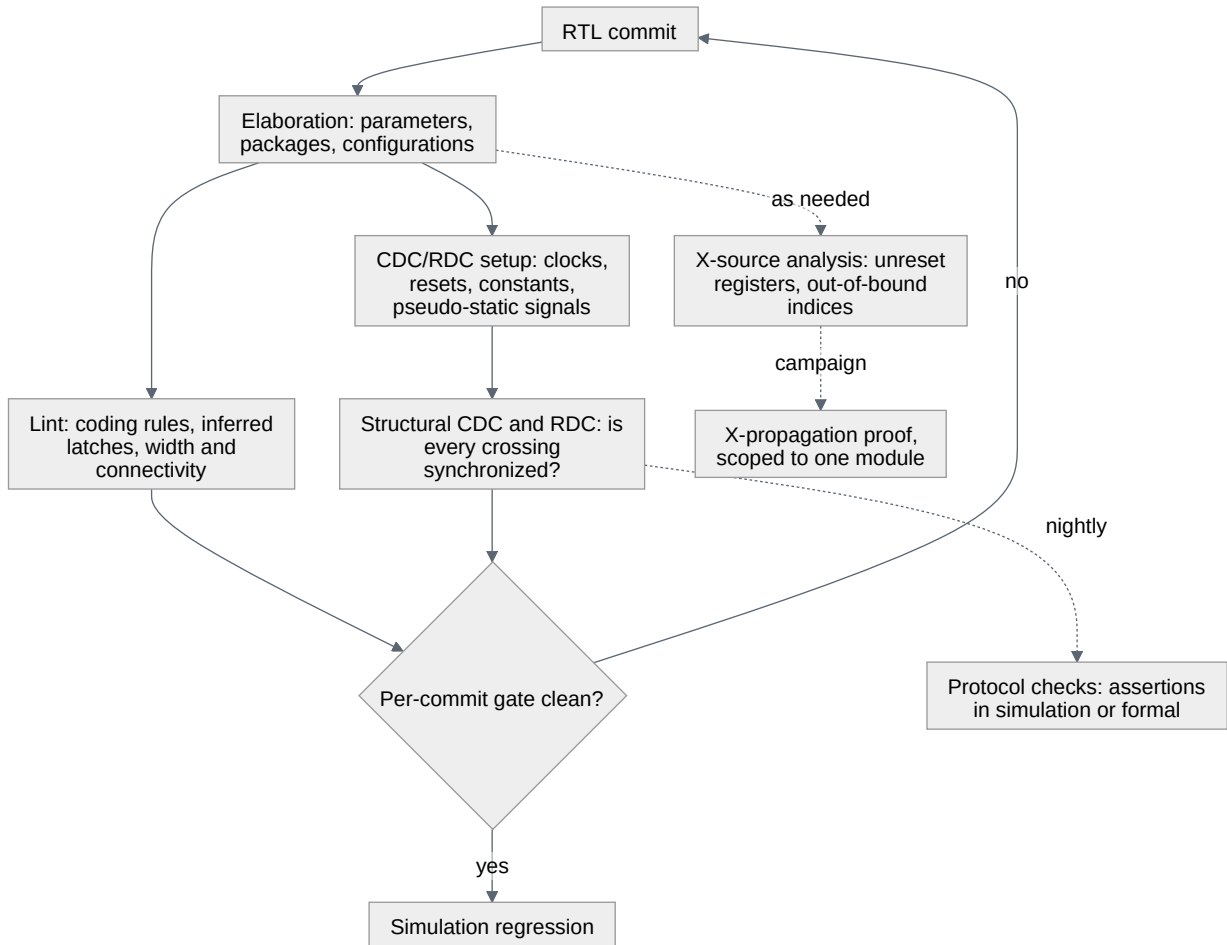


Figure 13.1 — Where each static check sits relative to the per-commit gate: elaboration first, then lint and structural CDC and RDC inside the gate, whose two exits are the simulation regression and the commit sent back to the RTL. The dotted edges are the checks deliberately left outside it — protocol checks nightly, X-source analysis as needed, an X-propagation proof as a scoped campaign — because a gate that also carries the slow checks stops being fast enough to wait for. What the diagram does not show is the setup behind its CDC and RDC node, which is where the engineering lives and where §13.3 shows what one unreviewed line can hide.

CDC setup requirements

A structural CDC check requires a setup that describes design intent. The setup must identify clocks and their asynchronous relationships, inputs constant in the analyzed mode, pseudo-static signals (changed only while the receiver is in reset or disabled), and gray-coded buses [3].

Timing constraints also declare clocks and exceptions, but their file should not be re-used directly for CDC setup. The clock groups a timing tool needs are not the clock groups a CDC tool needs, and a path waived for timing analysis is not a path that may be waived for CDC analysis [3]. A timing false path means *do not schedule this*; a CDC false path means *there is no crossing here at all*, a stronger and usually incorrect claim.

The reference SoC has three clock sources, system at 200 MHz, peripheral at 50 MHz and always-on RTC at 32 kHz, but “three clocks” does not establish “three clock domains”. A clock domain is the set of flip-flops driven by one clock or by

clocks derived from a common source [3]. Had the 50 MHz peripheral clock been the system clock divided by four, the two would be edge-aligned and share a source, and declaring that boundary asynchronous would bury the report in hundreds of crossings that are not crossings. The peripheral clock has its own source, so the boundary carries a real crossing on every register interface in the APB subsystem, and declaring it synchronous would silently delete the entire check. Nothing in the RTL distinguishes the two cases. The CDC setup must record the clock relationships specified in the clocking specification.

Scenario. The reference SoC’s integration team wants a per-commit static gate. **Approach.** The plan should fix the gate’s contents once, limiting them to production-parameter elaboration, lint at “must-fix” severity only, and structural CDC and RDC using a checked-in setup file. Everything slower sits outside it: protocol assertions nightly, the X-propagation proof as a scoped campaign. The setup file should live beside RTL, receive the same review, and name the assumed mode for every constant or pseudo-static declaration. **Why.** Checks scheduled nightly or as campaigns should remain outside the per-commit gate because of their runtime; Chapter 12 discusses the consequences of gate latency. The setup file records design intent; Section 13.3 shows what one unreviewed line can hide.

13.2 Lint: from noise to policy

Lint began as a C utility that reported questionable constructs the compiler accepted: mismatched argument types, a value used before it was set and a silently ignored return value [2].

What it finds falls into four families. *Synthesis-intent mismatches* include combinational blocks that do not describe combinational logic; the language says an `always_comb` procedure should be flagged if latched behavior can be inferred from its body [4]. *Width and type errors*: a truncated assignment, a signed-unsigned comparison, an expression one bit wide where the author meant a vector. *Connectivity and structure*: floating input ports, dangling wires, a variable with no driver, a variable with several.

And *language constructs whose semantics differ from what the author probably meant*. For example, `casex` treats both `x` and `z` as do-not-care in any bit, whether that bit sits in one of the case items or in the case expression (IEEE Std 1800-2023, *IEEE Standard for SystemVerilog*) [4]; a runtime unknown on the selector therefore matches the first item whose remaining bits agree, selecting an unintended branch.

The wake-up decoder is the point in the example where lint could have reported a problem:

```
module wake_decode (
    input  logic [2:0] cause,
    output logic [1:0] branch
);
    always_comb begin
```

```

case (cause)                // sparse encoding, no default
  3'd0 : branch = 2'd0;      // no wake pending
  3'd1 : branch = 2'd1;      // timer
  3'd2 : branch = 2'd2;      // GPIO
  3'd4 : branch = 2'd3;      // comparator
endcase
end
endmodule

```

Four of the eight values of `cause` are decoded. For the other four (`3'd3`, `3'd5`, `3'd6`, `3'd7`), no assignment executes, so `branch` retains its value, implying latched behavior in an explicitly combinational procedure and a latch in the netlist. The linter reports a synthesis-intent mismatch and missing `default` on the day the file is written. It does not know that `cause` crosses a clock boundary, that its bits can transiently disagree, or that `3'd6` is the transient the lab is hitting. Its report is correct and structural, several levels of abstraction below the bug.

Sources of lint noise

To avoid missing real problems, lint reports potential problems where none exist; after a week chasing them, engineers stop reading the output and eventually abandon linting ^[2]. This limitation is intrinsic to deciding from structure alone whether a construct is mistaken, so a better tool cannot eliminate it; two factors amplify it.

Scale amplifies findings in the GPU, one of the book's example designs. Four shader clusters are four instances of the same RTL, and a tile-based architecture replicates the per-tile datapath inside each. One questionable construct in a shared file, such as an unused parameter or a deliberate width truncation in a rate-matching stage, is reported once per instance. An issue count of eleven thousand across the shader complex may be forty distinct constructs, so any policy phrased as a count ("drive lint issues below 500") measures elaboration depth rather than code quality. Lint findings should be counted by distinct rule violation and source line.

The second multiplier is inherited code. A purchased or reused block has its own conventions and accepted deviations, so local rules produce thousands of findings the receiving team lacks authority to fix.

Making the rule set a policy

Three disciplines make lint a gate; only the first concerns the tool.

Rule severity is classified before the first run. Three classes are enough: violations that block the commit, violations waivable by a named owner with a written reason, and findings that are informational and never gate anything. The checked-in classification is a *policy* document stating what the team considers a defect, as Chapter 5 requires. The case Section 13.6 returns to is a Broadcom power-management controller; it had passed extensive block-level UVM constrained-random simulation and SoC-level directed tests; its lint analysis had been run, but the designer had waived its errors and warnings; the block nonetheless still held an unreset flip-flop, which a formal run found before tape-out ^[5]. Nothing in that account says lint would have caught the flip-flop; the discovery is not a lint result. A designer's waiver

alone leaves insufficient evidence for later audit; the owner must be *named* and the reason *written*.

Waivers can be generated rather than maintained. At Intel, a configurable IP ships in many SoC configurations, and its lint and clock-domain-crossing waiver files were re-written as templates parameterized on the IP's top-level parameters, so that each configuration derives its own; the reported generation effort per collateral falls from days to ten minutes, with a day of review still budgeted, and the paper names regeneration as necessary to avoid false waivers carried into a configuration they were not written for [6]. A collateral is a file that a flow generates and keeps beside the RTL, as those waiver files are.

Filtering preserves the check. Suppressing a check in the source or on the command line leaves it suppressed for an unpredictable length of time and hides problems arriving with later files, whereas a filtered report still contains the finding and the filter itself can be audited. Naming conventions should make filtering mechanical: if only intentional latches use a reserved suffix, inferred-latch findings on those signals are expected and every other one is a defect [2]. This is Chapter 7's rule about waiver files, one level down.

Lint runs during code development. A first run on a finished block produces a report large enough to feel like a setback, arriving when its author has already moved on. Running lint during writing lets the person who still holds the context judge each finding [2].

Scenario. The GPU team inherits a shader complex from a previous project and its lint report has 11,000 findings. **Approach.** One report snapshot should become the baseline, with every finding marked inherited; the gate should check only the *delta*, failing commits that introduce findings absent from the baseline. In parallel, findings should be classified by rule, with two or three rule classes tied to real failure modes (inferred latches, casex on a decoded selector) scheduled for retirement against the baseline over the next quarter. **Why.** In this example, the delta gate stops new debt on day one at zero cleanup cost. Absolute closure of inherited code requires a separate project with an owner and schedule. Retiring rule *classes* rather than instances also means each cleanup commit has a stated failure mode behind it, which is what makes it reviewable.

Design code has published coding guidelines and tool policies built on them; a DV-Con Europe paper observes that verification code has no such ready-made policies and that teams therefore derive their own rules from UVM convention; the same paper builds a testbench linter from open-source parts to enforce the rules a team derives that way; that linter is written in Python over a SystemVerilog parser, checks the naming of classes, interfaces and covergroups (the containers in which coverage is declared), requires class methods to be declared extern, and is wired into a continuous-integration flow [7]. The extern keyword leaves only a prototype in the class body and moves the implementation outside the class. That paper reports a method and a rule set, not a measured result.

13.3 Clock-domain crossing

A clock-domain crossing is a signal generated by logic on one clock and sampled by logic on another, where the two clocks have no fixed phase relationship. The fundamental problem is metastability. When the signal changes close enough to the destination edge to violate that flip-flop's setup or hold time, the flop enters an unstable state; it will resolve to a one or a zero after some delay, but which one cannot be predicted [8].

Metastability has two distinct consequences. First, metastability itself is unobservable in RTL simulation, gate-level simulation or formal analysis because it belongs to a physical flip-flop rather than its model. What it produces in the real device is a family of *effects*: uncertainty about when a value changes, a pulse that the device filters away although simulation showed it arriving, the reverse case of a pulse that the device passes on although simulation showed it being filtered, and several distinct timing scenarios where synchronized paths reconverge. Second, it cannot be eliminated, only given time to decay; that is the entire function of a synchronizer, which isolates the potentially metastable net so its unresolved state is not propagated into the destination domain, with a mean time between failures of many years [8].

Settling time and failure rate

The failure rate is set by the probability that a metastable condition has *not* decayed to a stable legal value in time for the next sampling event, and the mean time between failures that follows from that rate is given by

$$\text{MTBF} = e^{(T_s/T_c)} / (T_w \cdot F_c \cdot F_d)$$

where T_s is the settling window available before the next sample, T_c the flip-flop's settling time constant, T_w the window of susceptibility around the destination edge, F_c the destination clock frequency and F_d the rate at which the crossing signal changes [8].

Two terms form the exponent, the settling window T_s divided by T_c , while only T_w , F_c and F_d are linear. The first factor is the time between the first flop's output settling and the second flop sampling it, which allows a two-flop synchronizer to work; this window is eroded by a slow route, by a scan multiplexer on the second flop's data path or by clock skew that samples the second flop early. Because the window stands in the exponent, each of those erosions has an effect disproportionate to its own size. The second factor is flop choice: a metastability-hardened first-stage cell damps metastability faster (smaller T_c) and narrows the susceptibility window. The expression does *not* predict how often metastability occurs. Metastability is relatively frequent, whereas sampling and propagating an unresolved value constitutes a catastrophic, extremely rare failure [8]. Metastability causes failure when settling exceeds the available window or no synchronizer exists.

The structures, and what each one assumes

Four common structures have different preconditions.

The **two-flop synchronizer** is the fundamental element for a single asynchronous signal and the building block of everything more complex. Three of its requirements can be checked on the RTL: there must be no combinational logic on the crossing path that could be falsely sampled, the crossing signal must never carry a pulse too narrow for the destination to sample, and the data must be stable across more than two active destination edges so that nothing is filtered away ^[8].

The **gray-coded parallel synchronizer** extends this to a bus with one two-flop synchronizer per bit. It is safe only if exactly one bit changes at a time, because then at most one bit can go metastable and it resolves either to the old value or the new one, both legal codes. With a binary bus, the same one-synchronizer-per-bit structure generates codes that the source was not driving, both illegal codes and legal codes from the wrong moment ^[8]. In the wake-up example, the reference SoC's event unit uses one synchronizer per bit although two bits change together. Had the cause bus been declared gray-coded ^[3], the structural check would have accepted one synchronizer per bit and reported nothing; one wrong, unreviewed setup line would have let the wake-up defect pass a clean CDC run.

Fixing metastability and fixing data coherency are independent problems. For one bit, either resolved value is legal, so resolving metastability suffices. Several bits also require coherency, so their resolutions must preserve a consistent value ^[3].

The **handshake synchronizer** handles multi-bit data of any encoding without passing the data through a synchronizer. Only the request and acknowledge controls are synchronized; the data bus is held stable in the source domain until the acknowledge comes back, and the destination captures the data while it is guaranteed not to be changing ^[8]. It costs several round-trip latencies per transfer, which is why it is the answer for configuration and status rather than for streaming.

The **asynchronous FIFO** is the streaming answer: gray-coded read and write pointers crossing in both directions, with the payload passing through dual-ported memory rather than through any synchronizer. It provides bandwidth and throttling, absorbing intermittent arrival-rate peaks instead of stalling the source on every item ^[3].

The reference SoC's three domains

The reference SoC illustrates how frequency ratio affects structure selection.

Between the always-on RTC domain at 32 kHz and the system domain at 200 MHz the ratio is 6,250 to one: one 32 kHz period is 31.25 μ s, one system period is 5 ns. From slow to fast, an RTC-clock level is stable for 6,250 system cycles, so no pulse can be filtered and a two-flop synchronizer per bit suffices, provided the bits need not agree. From fast to slow, a system-domain pulse must exceed two destination periods ^[9]: 62.5 μ s, or 12,500 system cycles. Downward crossings on this SoC should use handshakes or toggles; a level pulse in that direction is a defect regardless of simulation results. A toggle carries each event as a change of level, which crosses

through a two-flop synchronizer, and the destination turns each change back into a pulse of its own.

The 200 MHz to 50 MHz boundary has a four-to-one ratio, so pulse width is rarely an issue in either direction, but every APB register interface crosses it, making it the widest boundary. Coherency requires that a multi-bit status field polled by firmware must not be readable in a state it never held.

Scenario. The sensor hub, an always-on fusion block with a 32 kHz island and a DSP clock-gated off between bursts, must transfer a configuration word from the island to the DSP. **Approach.** A handshake should generate the request in the island and require acknowledgment before the word may change; neither a gray-coded parallel synchronizer nor a pulse meets the requirement. **Why.** The “more than two active destination edges” rule is a duration only when the destination clock is running. The DSP clock is stopped for most of the island’s awake time, making the required level duration unbounded and inexpressible in source cycles. Handshake correctness is independent of destination frequency because the source holds data until the destination acknowledges receipt on its own schedule.

13.4 Structural checks and protocol checks

CDC verification requires structural and functional checks.

The structural check analyzes the elaborated design, identifies clock domains and every crossing, flags illegal path logic, and derives required properties by classifying each crossing’s synchronizer protocol: an individual asynchronous bit, a handshake or an asynchronous FIFO [8]. What it decides is *structure*: whether a recognized synchronizer stands where one is required. It does not analyze functionality, so valid structure passes [3]. A valid crossing also requires correct function, checked separately.

An assertion (see Chapter 11), evaluated by a simulator or proof engine, checks function. The properties are the preconditions of the structure the classifier found: minimum pulse width and data stability for a two-flop crossing, one-bit-at-a-time for a gray-coded bus, request-acknowledge sequencing and source-side data stability for a handshake. The standard requires a synchronizer-input pulse wide enough for reliable destination-clock sampling, with this obligation discharged by formal or dynamic verification [9].

Worked example: the pulse-width property, twice

Sampling the source signal on the destination clock and requiring stability gives this candidate pulse-width property:


```
// Wrong: samples the crossing signal on the clock that cannot see the problem.
property p_min_pulse_wrong;
  @(posedge dst_clk) $changed(src_sig) | => $stable(src_sig)[*2];
endproperty
```

At a destination edge, `$changed` is true when the sampled value differs from the previous sampled value; the non-overlapping implication then requires the value to be unchanged at each of the next two destination edges. A sampled value is the one the signal held just before a clock edge, which is what the assertion sees.

A source pulse narrower than a destination period can fall entirely between two destination edges. Then `$changed` remains false and the antecedent never fires, so the assertion passes without checking the crossing; silicon sampling at a different phase may catch the pulse and go metastable [3].

Sampling on the fastest system clock exposes the pulse; the stability requirement must be scaled to two destination periods:

```
// Right: sampled fast enough to see a pulse the destination could miss.
// N = ceil( 2 * dst_clk period / fast_clk period )
module cdc_pulse_chk #(parameter int N = 2) (
  input logic fast_clk,
  input logic rst_n,
  input logic src_sig
);
  property p_min_pulse;
    @(posedge fast_clk) disable iff (!rst_n)
      $changed(src_sig) | => $stable(src_sig)[*N];
  endproperty
  a_min_pulse : assert property (p_min_pulse);
  c_min_pulse : cover property (@(posedge fast_clk) disable iff (!rst_n)
    $changed(src_sig));
endmodule
```

`N` is the number of fast-clock ticks that span two destination periods, rounded up [3]. For the downward crossing discouraged in Section 13.3, using the 200 MHz system clock as the fast clock and 32 kHz RTC as the destination gives `N` = 12,500. That one number expresses the level-pulse cost and motivates a handshake in this direction. If no clock in the design is fast enough, the property is bound to a virtual clock, one created for the property alone rather than existing in the design.

A CDC protocol property specifies *time*, making its sampling clock part of the claim. And the wrong version fails in the way Chapter 11 warned about: it passes vacuously, and only the cover on its antecedent would have exposed it.

CDC reconvergence: signals that recombine after crossing

CDC verification must establish correct operation with synchronizers under every clock relationship, including each synchronizer's timing uncertainty and paths reconverging after more than one synchronizer [8].

Two failure modes arise. The first is loss of data coherency: several bits crossing through independent synchronizers, resolving independently, and recombining into a value the source was not driving. A gray code or handshake eliminates this wake-up failure by construction. The second is glitching: several separately synchronized signals reconverging into combinational logic, where a one-cycle difference in when each arrives produces a momentary transition on the result that neither source ever intended [3].

Glitching can evade review because each crossing is individually correct. For example, the peripheral domain raises two status flags, `fifo_empty` and `xfer_done`, each properly synchronized into the system domain, where a state machine advances on `fifo_empty && xfer_done`. Both flags are legal at all times in the source. But the two synchronizers resolve independently, so the conjunction can be wrong in two opposite ways. It can be asserted for one system cycle at a moment when the source never had both flags true; in the reverse case it can be *missed* for one cycle at a moment when the source did have both true. Nothing in either crossing is structurally wrong. The bug is in the CDC reconvergence, and the cure is architectural: synchronize one signal and qualify the other with it, or carry both across a single handshake, so that only one crossing decision is ever made.

Modeling timing uncertainty

A synchronizer output can arrive one cycle early, one cycle late or on time; receiving logic must tolerate all three, but conventional simulation reproduces exactly one and cannot test that tolerance. Metastability injection instruments each crossing with a model that randomly advances, delays or passes through the sampled value.

Four criteria established in 2012 distinguish accurate injection models from cheaper models that fail these requirements. Delay must be *random* among the three legitimate outcomes: early, late and on time. Injection must be *independent* per receiving register, since two registers sampling the same value can resolve differently, which a model that merely jitters a primary clock cannot express. It must be *accurate*, injecting only when the data changes, is sampled, and violates setup or hold; injecting at random moments manufactures failures silicon cannot produce. And it must be *complete*, covering registers with and without synchronizers, because a crossing can go metastable whether or not anyone synchronized it. Violations invent behavior: replacing every two-flop synchronizer with a three-flop cell that randomly advances or delays can produce values skewed by two cycles, a sequence impossible in silicon [10]. The accurate models are slow, which is why this runs on an emulator rather than in the nightly regression (see Chapter 17).

13.5 Reset-domain crossing

A flip-flop with an asynchronous reset has a third timing requirement beside setup and hold: recovery time, the interval between reset release and the next clock edge. Violating it is no different in kind from violating setup or hold: the flop can go metastable [3]. Asynchronous reset timing splits into two problems, and only the second is what the industry means by RDC.

The first is **reset de-assertion**. A reset generated in one place and released asynchronously with respect to a destination clock will, sooner or later, have one of its releases fall inside that flop's recovery window. The standard cure is the reset synchronizer: assert asynchronously so the design is safe the instant reset arrives, release synchronously so no flop sees the release inside its window. Whether a given reset has one is a purely structural question, which is why static analysis is sufficient to answer it [3].

The second is **reset assertion**, and this is reset-domain crossing proper. Asynchronous reset clears a register output independently of clock edges; the clear-input-to-output path is not normally timing-closed. If another register, *in a different reset domain and therefore not being reset*, samples that output, it samples a signal changing at an arbitrary time and can go metastable [3].

RDC concerns reset domains. Both registers above can be driven by the same clock. A design with exactly one clock has no clock-domain crossing at all and can still be full of reset-domain crossings; a clean CDC report says nothing about them. RDC developed after CDC as resets multiplied in practice: power gating gave each domain a reset, per-IP soft resets became standard firmware controls, and debug and watchdog resets added further domains. The cross-industry standardization effort followed: its first draft covered CDC alone, and RDC entered the scope at the very next release, nine months later [3].

Scenario. The sensor hub's always-on island runs entirely on the 32 kHz clock. It has two resets: a power-on reset covering the whole island, and a soft reset that firmware pulses to restart the fusion pipeline after a sensor is reconfigured. The island's CDC report is empty because it has one clock and no internal crossings. In the lab, reconfiguring a sensor occasionally produces a spurious wake interrupt.

Approach. RDC analysis should declare the two resets as separate domains. Every path from a fusion-pipeline register into the wake-decision logic is reported: the pipeline sits in the soft-reset domain, the wake decision in the power-on domain, and the wake-decision registers are live and clocking while the soft reset asserts.

Why. When firmware pulses the soft reset, the pipeline's `sample_valid` output clears asynchronously, at an instant unrelated to the 32 kHz edge. Sampling exactly then, the wake-decision register can resolve either way or late. The failure occurs despite individually correct registers and resets because the crossing is between reset domains within the same clock domain.

Three architectural fixes require a design decision.

Ordered reset assertions. If the destination reset is always already asserted when the source reset asserts, the destination register is sampling nothing and the crossing is harmless. This replaces a structural obligation with sequencing; the Accellera *Standard for IP Abstraction for Clock and Reset Domain Crossing Integration*, Version 1.0, 2026, therefore defines a reset assertion sequence (the order in which multiple resets shall assert) and a reset group (resets between which no RDC violation exists) [9].

Path qualification. A control can block the crossing while source reset is active, so the destination samples a held value. The qualifying control must itself be synchronous with the destination domain [3]; an asynchronous qualifier transfers the problem to another signal.

Source reset synchronized to the destination clock. Then the source register's output changes only on a clock edge and the crossing becomes an ordinary synchronous path. Of the three options, source-reset synchronization on the sensor hub's always-on island uses the continuously running 32 kHz clock.

An assertion should check reset ordering even when the controller appears correct:

```
module rdc_reset_order_chk (
    input logic clk,
    input logic rst_pipe_n,    // fusion pipeline soft reset, active low
    input logic rst_por_n     // always-on island power-on reset, active low
);
    // The pipeline reset may only assert while the island reset is already
    // asserted. `===` states the intent on its face; `==` would behave
    // identically here, since a property reads an unknown expression as false.
    property p_reset_order;
        @(posedge clk) $fell(rst_pipe_n) |-> (rst_por_n === 1'b0);
    endproperty
    a_reset_order : assert property (p_reset_order);
    // No `disable iff` here either, for the same reason.
    c_reset_order : cover property (@(posedge clk) $fell(rst_pipe_n));
endmodule
```

Three details are deliberate. The overlapping implication requires `rst_por_n` to be *already* low at the tick that first samples `rst_pipe_n` low; `|=>` would allow the pipeline reset to lead the island reset by a cycle, violating the intended order. A reset-ordering property that switches itself off during reset has nothing left to check, so there is no `disable iff`, the reset guard that discards any attempt in flight while its expression is true. And sampling on `clk` makes this a sampled approximation of an asynchronous event; the same obligation is also written against the source reset's own falling edge [3].

A team instantiating a purchased block cannot see its internal reset structure and should not have to; what it needs is a declaration: which ports carry a reset-domain control, with what blocking polarity, which resets share a group, in what order they must assert. Historically each vendor's tool produced that collateral in its own format and the models were not interchangeable [3]. A cross-vendor abstraction format for CDC and RDC collateral reached version 1.0 in March 2026, requiring conforming tools to read and write it and covering those attributes, the declared

facts about a block’s boundary [9]. The working group that produced that format grew to 154 members from 24 companies over the three years between the working group’s launch in January 2023 and the final release, with a first CDC draft appearing that October; among those companies were Arm, AMD, Apple, Google, Huawei, IBM, Infineon, Intel, Marvell, Microsoft, NVIDIA, NXP, Qualcomm, Renesas, Robert Bosch, Samsung and ST Microelectronics, alongside the major EDA vendors [3]. The standard covers “aspects of clock domain crossing and reset domain crossing that are outside the expressiveness of typical HDL semantics” [9]. This participation shows that industry considered the problem real and shared enough to standardize an answer; it establishes no tool’s implementation quality.

13.6 X-propagation

SystemVerilog carries four values: 0, 1, the unknown `x` and the high-impedance `z` [4]. Silicon carries two, so simulation translates between a four-valued model and a two-valued device, losing information in both directions. The last entry-gate check addresses results that appear correct despite this loss.

The two X-propagation errors

X-optimism, defined in Chapter 12 as propagating a definite 0 or 1 where silicon would hold an unknown, follows from language semantics rather than a simulator defect. An `if` takes the true branch only when its condition is a nonzero *known* value; the false branch is taken when the condition is 0, `x` or `z`. An unpredictable condition therefore produces a definite, unintended decision, and simulation continues. `case`, `posedge` and `negedge` behave the same way [5]. As Chapter 12 notes, asynchronous reset in RTL simulation works *because* of these semantics, so making the simulator pessimistic everywhere is not a valid remedy.

X-pessimism is the opposite error and the more visible one. With `a` unknown, both `a & ~a` and `a | ~a` evaluate to `x` in simulation, where silicon gives a definite 0 and a definite 1. Worse, the arithmetic operators are all-or-nothing: if any operand bit is unknown the whole result is unknown, so `3'b000 + 3'b01x` is `3'bxxx` where `3'b01x` would have been accurate. Pessimism does not usually hide design bugs; it fills waveforms with unknowns that are expensive to trace and mostly artifacts [5].

These language semantics let RTL simulation pass tests that silicon fails and fail tests that silicon passes, for reasons independent of the design.

Where the unknowns come from

Four sources account for most of it: registers with no reset, out-of-bound array or bit-select accesses, explicit `x` assignments written as don’t-cares or as error indications, and the corruption of a powered-down domain under a power-intent description [5]. The last belongs to Chapter 19. The first two sources can be found statically at the entry gate. A power-intent description states, outside the design description itself, which parts of the design switch their supply together, how they are isolated and retained, and which supply states are legal.

Scenario. The video codec’s deblocking filter indexes a per-row descriptor array sized for the tile geometry of the profiles it originally shipped with. A new profile adds one row class; at the largest frame size the index reaches one entry past the end, for the last row of the frame only. The out-of-bound read returns `x`, and the filter-mode case that consumes it falls through to its default branch. The regression passes: frames complete and every tested clip’s checksum matches, but clips for the new profile are absent. **Approach.** Structural property analysis should check out-of-bound indexing across the block before a single new test is written. It needs no testbench, only compilable RTL, and it reports the index and the condition under which it exceeds the array. **Why.** A 2014 case study describes this sequence: an audio block extended from sixteen channels to seventeen, an index one past the end of a request vector, an unknown that X-optimism converted into a state machine that always returned to idle, a full passing regression, and a system that hung in the lab whenever the new channel was used [5].

Scenario. The crypto engine’s key-valid flag is a control-register bit left unreset to save resources. The negative test “no ciphertext is produced before a key is loaded” passes on every seed. **Approach.** Formal reset analysis should precede acceptance of the test result. It takes the reset vector and reports every register still unknown when the reset period ends, which is where this flag will appear. **Why.** In simulation, the flag powers up unknown, so `if (key_valid)` takes the else branch and the engine refuses encryption; X-optimism makes the test pass. In silicon the flop powers up to a definite value, which on some corners and some dice is one, and the engine encrypts with whatever the key registers happen to hold. The remedy is reset: control, clock and reset registers in particular should be reset because savings in area, routing or timing do not justify this class of bug [5].

The three answers, and what each costs

Gate level. X-optimism largely disappears there, because `if`, `case` and the edge constructs have by then been synthesized into gates and flip-flop primitives that do not exhibit it. The cost is not only runtime: X-pessimism becomes *more* prevalent at gate level, and the resulting unknowns have to be cleared by depositing or forcing known values, iteration after iteration, which is what makes the productivity poor rather than merely slow [5]. As Chapter 12 explains, only a minority of the suite is regressed at gate level, so this approach arrives late with limited coverage.

A simulator’s X-propagation mode. Simulators can be told to merge the results of both 0 and 1 in place of an unknown when executing `if`, `case` and the edge constructs, preserving an unknown at the point of decision [5]. The performance cost motivates a named regression variant rather than a default; in practice, one 2026 project lists `xprop` beside DMA and JTAG variants in the same UVM regression environment [11].

Formal X-propagation. A formal application proves or refutes instrumented assertions that ask whether an unknown can reach chosen targets: clocks and resets, primary outputs, or `if` and `case` conditions. A bounded proof is a result stating only that no violation exists within a stated number of cycles. The guarantee is about the

near future; despite its name it is not a proof for every cycle. Applied to a whole IP with a two-hour budget, that formal application returned bounded proofs and no violations despite a known X-optimism bug, illustrating its capacity limit [5].

The approach starts by identifying X sources. Formal reset analysis takes the reset vector and reports which registers are still unknown when the reset period ends, catching unreset flops that no structural check can detect, such as those left unreset by a polarity mistake or by a clock not running while synchronous reset asserts. In the reported approach, structural property analysis detects out-of-bound indexing at lower cost than propagation proof. Only when every source has been found and judged intended or not is the proof scoped to the module where a given source is produced and consumed, small enough to converge [5]. On a Broadcom power-management controller for a quad-core application processor, formal reset analysis reported four flip-flops still unknown in the first clock cycle after reset deassertion, and every assertion failure the X-propagation app then produced traced back to X-optimism on just one of them, the very flip-flop the team had already suspected [5].

IN PRACTICE

Static verification depends on the continued validity of its setup assumptions. For example, CDC declarations that an input is constant, a bus is gray-coded or a crossing is a false path can remain unreviewed after the design changes, though each was valid when written. Six months later in this example, a clean report still relies on assumptions last examined by someone who has left the project. Setup review should therefore accompany RTL review at the same milestones, the phase gates that a project passes by demonstration rather than by estimate.

Ownership should be assigned explicitly. Designers should run lint and own its findings, since they concern their code and cost least to judge on the day of writing. An integration or methodology team should own CDC and RDC setup because it specifies chip-wide clocking and reset architecture beyond a block author's scope. Verification engineers should ensure that results constitute evidence: reviewed waiver files (see Chapter 7), protocol assertions run with non-vacuous attempts (see Chapter 11), and gate contents matching the plan.

If crossings into blocks whose internals are still black boxes dominate a full-SoC structural report, boundary models should be used to classify those crossings. The reference SoC's three domains give three boundaries to classify by hand, and ten follow from the five domains of its successor, the flagship SoC. The exercises ask for the rule that turns a domain count into a boundary count. The two-level industrial flow closes each block separately and exports an abstraction model of its boundary alone, including clocks, resets and internally synchronized ports; the SoC run consumes these models instead of RTL [3]. These models expose more than opaque boxes but are valid only for their generating parameters and configuration signals; a block instantiated twice in different modes needs two models. The flagship needs one model per operating point, because its clock ratios change with DVFS, the run-time trading of supply voltage and clock frequency to save energy.

Lint should run every commit; structural CDC and RDC should run every commit once setup is stable and should also run before that, while the setup is being established; protocol assertions belong in nightly and weekend tiers, and metastability injection on the emulator. Formal X-propagation is a campaign launched for a stated reason, scoped to a module and producing a written result.

PITFALLS

1. **A setup file that makes the check pass.** Symptom: a suspiciously short report, hundreds of crossings on a synchronous boundary, or unexplained removal of crossings from a shrinking report. Cure: the CDC setup should be separate from timing constraints because they answer different questions [3]; a CDC false path asserts *no crossing exists here* and requires a named owner and written argument, as does a coverage exclusion.
2. **Declaring CDC done when the structural report is clean.** Symptom: every crossing has a recognized synchronizer and the design still fails in the lab. Cure: protocol properties (Section 13.4) must check the other half.
3. **A crossing property sampled on the destination clock.** Symptom: a pulse-width assertion that has never fired on any seed. Cure: sample on the fastest clock in the system, or a virtual one, and scale the stability count accordingly.
4. **Reading a lint issue count as a quality metric.** Symptom: a number that moves when someone changes an instantiation count. Cure: count distinct rule violations at their source lines, and gate on the delta against a baseline for inherited code.
5. **Leaving control registers unreset because “it does not matter which value it starts at”.** Symptom: a test that passes for the wrong reason and a state machine that behaves differently in silicon. Cure: reset control, clock and reset registers; the area saved does not pay for the class of bug it hides [5].
6. **Treating RDC as covered by CDC.** Symptom: a single-clock block with a clean CDC report and intermittent failures around soft reset. Cure: run RDC with the reset domains declared, and check that the design’s resets have a defined assertion order.

SUMMARY

- Static verification needs no stimulus and no expected output; it is exhaustive over structure and silent about function [2].
- False reports undermine lint’s value. The output should be filtered without disabling the check, naming should encode intent so the filter can be mechanical, and lint should run while code is written [2].
- Metastability cannot be removed, only given time to decay; two MTBF terms, the settling window and the flop’s settling time constant, form the exponent, so even a small reduction in the window has an exponential effect [8].

- One bit requires metastability resolution, while several require both resolution and coherency; a gray code, handshake or asynchronous FIFO provides this where three individually correct synchronizers do not [3].
- The structural check asks whether a synchronizer is present; the protocol check asks whether its preconditions hold. Both are required, and the second is an assertion problem (see Chapter 11).
- RDC is not a special case of CDC: its domains are reset domains, so a design with a single clock and no clock-domain crossing at all can be full of reset-domain crossings [3].
- RTL simulation is optimistic about unknowns at every decision and pessimistic about them in every expression, so it can pass tests that silicon fails and fail tests that silicon passes [5]. Source analysis should address unreset registers and out-of-bound indices before propagation analysis.

EXERCISES

Exercise 13.1 (calculation). The pulse-width property of §13.4 scales its stability requirement to a count of fast-clock ticks, and §13.4 evaluates that count for the crossing from the system domain down to the always-on domain. Compute the same count for the crossing from the system domain down to the peripheral domain, taking the system clock as the fast clock and reading the clock frequencies from §13.3. Compare the result with the count §13.4 reports for the always-on destination, and state what the two counts together say about a level crossing in the downward direction on this SoC.

Exercise 13.2 (calculation). The practice section of §13.6 gives two counts of boundaries to classify by hand, one for the reference SoC and one for the flagship, each following from the number of clock domains on that design and from nothing else about it. Derive the rule that turns a domain count into a boundary count, and apply it to a design carrying one further asynchronous domain beyond the flagship's. State the condition of §13.1 under which a pair of clocks contributes no boundary at all, and give the effect on the count just computed when that condition holds for one pair of the six.

Exercise 13.3 (judgement). On the sensor hub's always-on island, a register that a firmware soft reset clears is sampled by a register the power-on reset covers, and both run on the island's own clock. An engineer proposes to block that crossing with a qualifying control generated by the soft-reset controller itself, asserted before the soft reset and released after it. Decide which of the three architectural fixes of §13.5 the proposal is, state the requirement of §13.5 it fails, and name the fix §13.5 records for this island together with the property of the island's clock that makes it available.

Exercise 13.4 (judgement). A team finds a control-register bit left unreset and proposes to settle its consequences by running the formal X-propagation application over the whole block with a fixed time budget. Decide what §13.6 licenses that run to establish and what it does not, using the whole-IP result §13.6 reports. Give the order §13.6 puts the source-finding steps and the proof in, with the reason each earlier

step costs less than the proof it precedes, and name which of the two X-propagation errors makes an unreset control bit dangerous rather than merely noisy.

Exercise 13.5 (artifact). Read the setup excerpt below against the setup requirements of §13.1 and the synchronizer structures of §13.3. Name the two declarations that would make the structural check report less than the design contains, and for each state which crossings leave the report. Name the field left blank, and state what a review of this file cannot establish while it stays blank.

```
clock      clk_sys      200 MHz    source: system PLL
clock      clk_periph   50 MHz     source: clk_sys divided by four
clock      clk_rtc      32 kHz     source: always-on oscillator
asynchronous { clk_sys, clk_rtc }
constant   test_mode = 0          assumed mode:
gray_coded evt_cause[2:0]         owner: event unit
pseudo_static cfg_prescale[7:0]   changed while: receiver disabled
```

FURTHER READING

Within the corpus, [3] treats CDC and RDC together: synchronizer schemes, setup constraints and their origins, structural analysis, assertion clocking traps and hierarchical block-to-SoC analysis. [9] specifies CDC and RDC collateral attributes; its RDC clause covers reset groups, assertion sequences and blocking controls, and the standard sets SVA requirements for a closed-box crossing. [8] covers synchronizers *after* RTL sign-off: the sub-cycle constraint on the metastable net, MTBF degradation from scan insertion on the second flip-flop, and parallel-synchronizer failure from under-constrained bits. [10] catalogs metastability injection models. On unknowns, [5] carries the definitions, the four X sources, the X-source-driven scoping and two industrial case studies. [2] gives a short account of lint's capabilities and limits; [4] defines four-valued semantics, `casex` and `casez`, and the checks the language asks tools to perform for `always_comb` and `always_ff`. The 2024 industry data on asynchronous clock domains is in [1].

Outside the corpus: C. Cummings' SNUG papers on clock-domain-crossing design and on asynchronous FIFO design are the long-standing practitioner references for synchronizer and FIFO construction; M. Litterick, *Pragmatic Simulation-Based Verification of Clock Domain Crossing Signals and Jitter using SystemVerilog Assertions*, is the simulation-side companion to [8]; and M. Turpin, *The Dangers of Living with an X* (SNUG Boston, 2003) with S. Sutherland, *I'm Still In Love With My X!* (DVCon, 2013) are the two short papers to read next on X semantics and coding style.

REFERENCES

1. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
2. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.

3. **D36** D. Mills, B. Gascoyne, C. Choppali Sudarshan, D. Olopade, and D. K. Gupta, "Breakthrough in CDC-RDC Verification Defining a Standard for Interoperable Abstract Model," Proc. DVCon U.S., 2026.
4. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
5. **D23** S. Zhao, S. Yan, and Y. Feng, "Practical Approach Using a Formal App to Detect X-Optimism-Related RTL Bugs," Proc. DVCon U.S., 2014.
6. **D39** G. Sridhar, V. V. Sateeshm, and M. Krishna, "Automation of Waiver and Design Collateral Generation for Scalable IPs," Proc. DVCon India, 2019.
7. **D40** S. Venkataramanan, D. Palaniappan, and S. P. Singh, "Testbench Linting - Open-Source Way," Proc. DVCon Europe, 2023.
8. **D14** M. Litterick, "Full Flow Clock Domain Crossing - From Source to Si," Proc. DVCon U.S., 2016.
9. **S12** Accellera Systems Initiative, "Standard for IP Abstraction for Clock and Reset Domain Crossing Integration, Version 1.0," Accellera, March 2026.
10. **D34** A. Hari, S. Krishnamurthy, A. Jain, and Y. Badaya, "Verification of Clock Domain Crossing Jitter and Metastability Tolerance using Emulation," Proc. DVCon U.S., 2012.
11. **P25** S. Ahmed, A. Kropotov, R. I. Genovese, B. Homs, E. Merino, F. Urbani, et al., "Verification and Validation (V&V)-in-the-Loop for RISC-V Design: The Holistic Vision of BZL," arXiv preprint arXiv:2604.27013v1, April 2026.

14. Formal Property Verification

Formal property verification checks whether a design can violate stated properties; the reference SoC's crossbar illustrates its scope after six weeks of nightly regressions with no failures. A week before sign-off review in this example, the bound protocol checker from Chapter 11 has never fired on any of the nine ports. One unexamined report value should have stopped review: on the four subordinate-side instances, the cover `c_w_overlap` counts the times a second write address was accepted while an earlier burst was still unfinished on the write-data channel, and it reads zero. Chapter 11 identified that cover as the precondition of exactly one hazard, and in six weeks no test has produced it.

An engineer submits the unchanged checker file to a proof engine at one subordinate port. Overnight, the tool returns an eleven-cycle counterexample: two managers have writes in flight to the same subordinate, and that subordinate back-pressures write data while the crossbar interleaves their data beats into one stream. A beat is one data transfer of a burst. Beats from two bursts get counted as one, and the count no longer matches the length promised by the address at the head of the queue. The miscount is the book's canonical crossbar bug, which no test in six weeks came within reach of finding.

With the property unchanged, a simulator checks for violations in its runs, while a proof engine checks whether any violation is possible; on blocks of the right size, the second question is answerable in a night.

The same run returned three other results: two properties **proven**, one **undetermined**, and one proven only because an unreviewed assumption forbade the traffic that would have violated it. The chapter examines these four results, including the misleading success under an unreviewed assumption that excludes the violating traffic.

CHAPTER MAP

§14.1 defines proof over a model, assumptions and sometimes a depth: three qualifiers, each of which can invalidate the claim. §14.2 examines over-constrained proofs and assumption checking. §14.3 develops bounded bug hunting by unrolling, unbounded proof by induction and the interpretation of proof depth. §14.4 examines abstraction for convergence, sound reductions and lost bug classes; §14.5 orders the responses to an undetermined property. §14.6 covers sign-off evidence, coverage without stimulus and formal's limits.

14.1 Proof and its scope

A synchronous design is a finite transition system: a set of state variables, a set of initial states, and a relation saying which states can follow which. Model checking

builds that finite model and checks whether the desired properties hold in it. The states reachable from the designated initial states are computed by a fixed-point analysis, the *reachability analysis*: the tool adds the states one transition away from those it already has, and repeats until a pass adds nothing new. The property is checked against the resulting set of reachable states, and an error trace is produced if it fails. It is an exhaustive state-space search which, because the state space is finite, is guaranteed to terminate [1]. Formal verification aims to establish that a design works correctly for **all** allowed inputs and **all** reachable states, by mathematical reasoning rather than by sampling [2].

A green regression establishes that none of the generated traces failed, a statement about the testbench; a proof establishes that no design trace fails.

Four outcomes. Complete methods distinguish two results: a property passes or fails. Capacity limits also make incomplete methods common, with three outcomes: passed, failed or unknown; even complete methods report unknown on timeout or memory exhaustion [3]. So a formal report has a vocabulary a regression report does not:

- **Proven:** no reachable state or legal input sequence violates the property. The claim holds subject to the three qualifiers below.
- **Falsified:** the tool produces a *counterexample* (CEX): a sequence of input stimuli leading to the failure, shown with the relevant internal signals and often with explicit don't-care values where the trace does not care what an input did [3]. The trace shows incorrect behavior and can be read much like simulation output to identify the cause [2]; unlike a regression failure, it has no irrelevant history, because the engine had no reason to generate any.
- **Bounded proof:** no violation up to n cycles. The guarantee covers only the near future; §14.3 explains how to interpret the bound.
- **Undetermined** (inconclusive, unknown): the tool gave up. §14.5 describes the next steps.

A **cover** behaves symmetrically. If the scenario can be hit, the tool returns a **witness**: a stimulus sequence reaching the coverage point while satisfying every stated assumption. Otherwise, the tool reports the point **unreachable under the specified assumptions**; the tool may also exhaust memory or time without hitting the coverage point, a distinct third outcome that can resemble unreachability [3]. Chapter 11 states in one line that an empty simulation cover means no test reached it, whereas an unreachable formal cover means nothing can reach it; only the second states something about the design.

The three qualifiers

A proof describes a model under assumptions, sometimes to a depth; these qualifications limit what it establishes about the chip.

The model. The engine reasons about the RTL as elaborated, not about silicon, and a proof at a high level of abstraction does not establish that the lower-level details are correct [2]. A proof over RTL says nothing about the synthesized netlist (Chapter

15's equivalence checking closes that gap), nothing about timing, and nothing about the firmware that programs the block. It also says nothing about unmodeled details: if a submodule was blackboxed or a memory replaced, the proof is over the replacement. A submodule is blackboxed when its contents are taken out of the model and its outputs are left free to take any value.

The assumptions. Verification generally requires assumptions about the environment the device operates in; if the real environment violates them, the device may fail despite successful verification. Formal methods do force these assumptions to be written down [2]. §14.2 examines them.

The depth. If the result is bounded, the property is guaranteed only within the bound [4].

The statement “*the crossbar has been formally verified*” is therefore insufficient. For such a claim to mean anything it must come with a description of what properties were verified and at what level of detail the design was modeled [2]. An explicit form is: *these five properties, on this RTL revision, under these five assumptions, proven unbounded except one which holds to depth 78*. This chapter's sign-off table and assumption register contain exactly those figures, making the statement auditable.

The property Chapter 11 handed over

The property introduced at the end of Chapter 11 remains unchanged:

```
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && awready && (awburst == 2'b01)) |->
      (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

In simulation this is sampled: it is evaluated on the bursts the tests happened to emit, and it is silent about the ones they did not. Bound to the DMA backend's write-address channel, the identical property lets a proof engine search for any legal programming sequence that emits a burst across the 4 KB boundary, the address line that AXI forbids a burst to cross.

The three qualifiers define the claim available at review. The *model* is the burst splitter and the AXI backend as elaborated; it contains the splitting arithmetic where the canonical bug lives, and it does not contain the register frontend if it was black-boxed. The burst splitter is the DMA stage that splits each transfer into bursts that the AXI address rules allow. The canonical bug is the book’s recurring defect for that block: a row mis-split at the 4 KB boundary when the step between rows is negative. A transfer descriptor is the record that programs one DMA transfer, and its stride is the signed address step from the start of one row to the start of the next. The *assumptions* constrain descriptor arrival: assuming a positive stride deletes the negative-stride bug from the model rather than proving it absent. The *depth* matters when the run was bounded: it must be long enough for a descriptor to be programmed, split into legal bursts and emitted. A proof bound that never reaches the first accepted address proves the property without reaching the relevant behavior.

14.2 Properties and assumptions

Constraints and assertions are formal, unambiguous statements about design behavior: an assertion names a property to verify; a constraint names a condition the verification requires; and an assertion in one verification becomes a constraint in another [1]. Chapter 9 used the same object as a generator, Chapter 11 as an assertion, and this chapter as an assumption: one artifact with three roles.

In simulation an `assume` is checked exactly like an assertion; in formal it is never checked, and instead deletes traces from the model, so that the assertions are proved over the traces that survive (see Chapter 11).

Over-constraints and vacuous proofs

An assumption stronger than reality makes the engine prove properties of a model that excludes real behavior. The assertion then holds trivially, or vacuously, which may suggest complete checking when essentially nothing is verified [3]. The assertions here are SystemVerilog’s, specified in IEEE Std 1800-2023, *IEEE Standard for SystemVerilog* [5]; the vacuous evaluation is defined at IEEE 1800-2023 §16.14.8 “Nonvacuous evaluations”, which reads such a success as not matching the user intent and therefore as neither a pass nor a failure. Over-constraints stall simulation progress and lead to vacuous formal proofs [1].

A *missing* assumption lets the model do things the real environment never would, so the engine finds a violation that cannot happen: a spurious counterexample. In this example, investigating it costs an afternoon, but the failure is visible. An *extra* assumption produces no diagnostic; it increases reported successes.

Over-constraint has two forms, only one of which produces an obvious symptom.

The empty model. Two contradictory assumptions, or one conflicting with design behavior, leave no states: for example, requiring a signal to remain stable while an `always` block toggles it every cycle. An empty model may suggest that every assertion should fail, but all of them pass vacuously instead: the proof assumes that all as-

sumptions hold, and in an empty model that hypothesis is false. A free variable is a signal that the proof engine may set to any value at any time, rather than a signal that the design drives. Many tools detect basic cases; a low-cost manual check is that **every assumption should have a design input or a free variable in its support**, otherwise it is likely to conflict with design behavior rather than constrain the environment ^[3]. An assumption's support is the set of signals its text mentions.

Satisfiable narrowing. This one can reach sign-off without an obvious symptom. The assumptions are consistent, the model is far from empty, most properties still prove for the right reasons, while one clause has removed the exact traffic the interesting property was written for. Chapter 9 named this shape on the stimulus side: over-constraining whose quiet form stays satisfiable and silently deletes the feature under test. Formal aggravates this problem: simulation exposes a deleted scenario as a coverage hole because coverage measures what ran. In formal there is no stimulus to measure, so the deletion leaves no trace anywhere but in the assumption text. Overconstraining is usually not obvious and cannot always be discovered by automatic tools, which is why validating the assumptions is a task in its own right ^[3].

Scenario (the crypto engine). The inline AES-GCM engine on the memory path carries the two properties Chapter 11 wrote for it: a key may not be loaded while an operation is in flight, and ciphertext may not be presented under an unauthenticated key. Neither proof converges because the key ladder plus the GCM datapath contains too much state, so an engineer encodes what the specification's boot sequence says: key material arrives only after authentication has completed. The key ladder is the logic that derives the engine's keys.

```
systemverilog m_auth_before_load : assume property (@(posedge clk) disable iff (!
rst_n) $rose(key_load) |-> auth_done);
```

Both properties now prove in minutes. **Approach.** Both results require a re-run with the assumption deleted. This one assumption deletes all the traces in which unauthenticated key material enters the engine, which is the class that `a_no_ct_unauth` checks. **Why.** A functional proof assumes a cooperative neighbor, with a buggy neighbor as the worst case. A security proof faces an adversary who knows the assumption list, so each assumption is a concession that must come with a reason why the attacker cannot make that assumption false. Here no such reason exists: an attacker able to drive `key_load` is the threat itself. The boot ROM's proof should discharge this obligation; the engine's proof cannot assume it without that discharge. [Figure 14.1](#) draws the deletion on two traces.

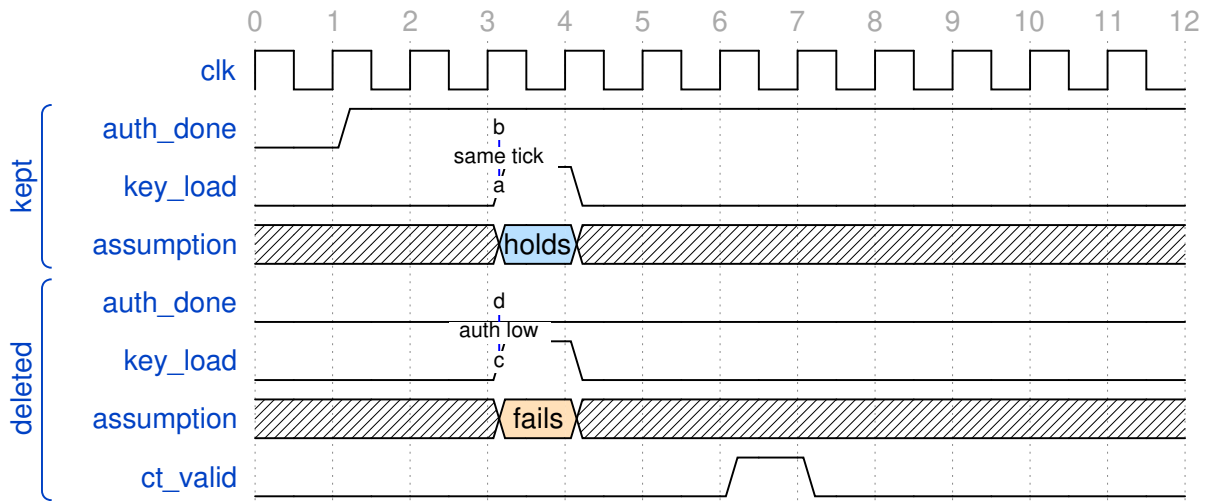


Figure 14.1 — One assumption drawn on two traces: the kept trace rises `key_load` while `auth_done` holds, the deleted trace rises it unauthenticated, and the deletion removes every trace in which ciphertext follows an unauthenticated key.

Discharging assumptions instead of trusting them

Because an assertion in one verification is a constraint in another, an assumption on a block's inputs should be **proven as an assertion on whatever drives those inputs** [3]. That discipline is the assume-guarantee paradigm, under which properties of a system can be deduced from properties of its components without ever model-checking the complete model [2]; the complete model would not converge.

The crossbar reuses Chapter 11's file with one addition. Chapter 11's bound checker carries its own model of an interface in counters and queues, and its five manager-side instances state the AXI manager rules that the crossbar is entitled to expect from the core and the DMA. These AXI4 rules come from the *AMBA® AXI and ACE Protocol Specification*, Arm IHI 0022H.c [6]. When the crossbar is proven, those same rules become its assumptions; when the DMA engine is proven, the same text is an obligation on the DMA's own output. One file carries two directives, selected by a parameter:

```
module xbar_port_checker #(
    parameter bit          ASSUME_MODE = 1'b0,    // 1 = constrain, 0 = check
    parameter int unsigned IDW = 4,
    parameter int unsigned AW = 32
) (
    input logic            clk, rst_n,
    input logic            awvalid, awready, wvalid, wready, wlast, bvalid, bready,
    input logic [IDW-1:0] awid, bid,
    input logic [AW-1:0]  awaddr,
    input logic [7:0]     awlen,
    input logic [2:0]     awsize,
    input logic [1:0]     awburst
);
    // Chapter 11's interface model and its other properties are omitted here;
    // the ASSUME_MODE parameter and the generate below are this chapter's
    // one addition to that file.
    property p_aw_stable;
        @(posedge clk) disable iff (!rst_n)
```

```

    (awvalid && !awready) | => awvalid &&
      $stable({awid, awaddr, awlen, awsize, awburst});
  endproperty

  if (ASSUME_MODE) begin : g_env
    m_aw_stable : assume property (p_aw_stable);
  end else begin : g_dut
    a_aw_stable : assert property (p_aw_stable);
  end
endmodule

```

The parameter records the side of the boundary under proof and locates responsibility for guaranteeing the rules.

One change is needed first, to the interface model rather than a property. Chapter 11’s interface model stores accepted addresses in an unbounded queue, `logic [7:0] aw_len_q [$]`, which simulation supports but a proof engine cannot encode: §14.1 guarantees termination *because the state space is finite*. The queue must be capped at the port’s outstanding-write capacity. The cap is not an assumed limit: it is a fact about the crossbar, proven on its own and handed back as a lemma (§14.5), and it reaches the sign-off package with a discharge like any other.

Sometimes an assumption cannot be discharged this way: the driving block may be too complex to prove, the signals may be driven from several blocks that would have to be pulled in together, or the driver may be third-party IP without an available model. Then the assumptions should at least be checked in simulation, and checked again in simulation of the larger model the block is integrated into [3]. For that reason the crossbar’s assumptions are also compiled into the SoC-level regression as ordinary assertions. An assumption checked in no engine remains unsupported.

An **assumption register** records this evidence and is reviewed like a waiver list (Chapter 7). One row per assumption, four columns: an ID, the text, who owns the behavior it describes, and how it is discharged, including what breaks if it is wrong. §14.6 puts it in the package.

14.3 Bounded and unbounded proof

Two engines produce almost every result; identifying the engine distinguishes an unbounded guarantee from bounded checking comparable to a night’s simulation.

Bounded model checking *unrolls* the sequential design: for each cycle of an interval of length k it makes a fresh copy of the combinational logic, and it feeds each copy’s next-state outputs into the following copy’s state inputs. The copies together form one combinational circuit that holds the whole finite-time behavior of the interval. That circuit and the negation of the property go to a satisfiability solver, which searches for a counterexample within the interval; if none is found, k is increased until a bug appears, the problem becomes intractable, or a preset upper bound is reached [4]. The effect is equivalent to exhaustive simulation with the same cycle bound [1].

Two consequences follow. First, bounded model checking is far more efficient at falsifying a property than at proving one, which is why bounded model checking is used mostly to find counterexamples rather than to establish correctness [4]. Second, a bounded run’s success is not a proof. Bounded methods check that there is no violation with a counterexample shorter than n cycles from an initial state (which need not be the reset state), so they do not guarantee that the assertion is correct, only that it cannot be violated *too soon*. Tools report “the assertion has not been violated up to bound 50 clock cycles”, not “the assertion passed” [3]; the verification report should preserve that wording.

If the bound is smaller than the **sequential depth** of the system (the longest shortest-path from an initial state to any reachable state), part of the reachable state space is never considered and the proof holds only for the interval. Past the sequential depth, the bounded result is a proof of a safety property; an unbounded eventuality needs more than that, because a counterexample to it is a loop rather than a finite path. A safety property asks that something bad never happen. An eventuality asks that something happen at some time, with no cycle count attached. Reaching the sequential depth is usually unavailable: extending the interval that far is infeasible both because the satisfiability solver’s cost grows and because computing that depth is itself hard [4].

Induction is the other route to an unbounded answer, and bounded model checking extends to unbounded model checking precisely through it [1]. Induction unrolls from an *arbitrary* state characterized by a predicate P , instead of fixing the initial state at reset. The argument has two conditions: every reset state satisfies P , and every interval that starts in a state satisfying P both upholds the property and ends in a state satisfying P . When both hold, P is an invariant and the property holds for arbitrarily long time frames rather than just the examined interval, with a computation no longer than a bounded one. The first condition prevents an empty argument: $P = \text{false}$ is also closed under transitions but establishes nothing.

The new failure mode can cost weeks of work. Such proofs are conservative: they never report a property to hold when it does not. But if P over-approximates the reachable state space, the engine starts intervals in states the design can never occupy and produces **spurious counterexamples** from them. The remedy is a *strengthened invariant*: additional assertions, found by inspecting the design or automatically, that rule those impossible starting states out [4].

The first counterexample check is whether the design could occupy the state at cycle zero. The design can never occupy a state in which a FIFO’s occupancy counter reads 7 while its two pointers are equal, a one-hot state vector has two bits set, or an outstanding-transaction count reads 3 on a port that holds at most 2. Each observation becomes a helper assertion, proven independently and then supplied as an invariant. Chapter 11’s redundant-state invariants thus serve a second purpose: the facts written to catch silent simulation corruption also support the proof engine. Figure 14.2 draws the FIFO case against a trace from reset.

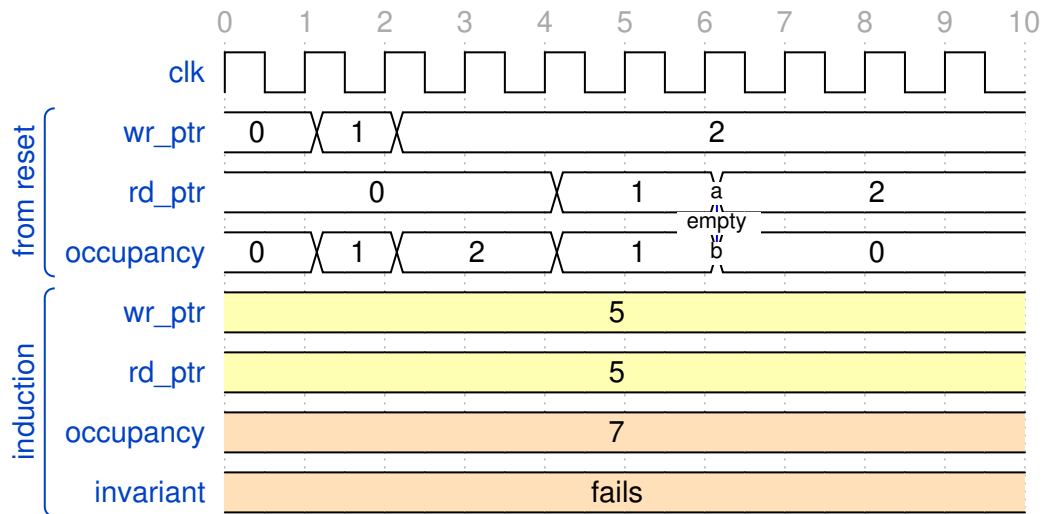


Figure 14.2 — Induction starts an interval at an arbitrary state, so cycle 0 can hold an occupancy of 7 with the two pointers equal, which no trace from reset reaches; the helper invariant, proven on its own, deletes that start state.

Reading a depth number

Because a bound is only meaningful against the length of the behavior under investigation, the number in the report has to be compared with an independent arithmetic calculation. For a pipelined design, a bound equal to or a little larger than pipeline depth is usually enough [3], but most properties of interest do not follow the pipeline structure.

The example uses one run on the DMA engine’s backend, bounded at 40 cycles, and two of Chapter 11’s properties that sit on it: one from the internal-invariant module, one from the port checker bound to the backend’s own AXI manager port.

- `a_no_overflow` guards a datapath FIFO of 16 entries. Filling it takes at least sixteen pushes with no intervening pop, so the interesting state is reachable well inside forty cycles, and the paired `c_full` cover confirms it with a witness, making 40 a generous bound.
- `a_w_burst_len` reconciles the beat count with the address’s promised length, but only at `WLAST`. The engine’s maximum burst is 256 beats, so a maximum-length burst’s `WLAST` cannot arrive until its 256th data beat has been handshaken. Inside a 40-cycle interval no burst longer than about forty beats can have completed at all. The property is proven for short bursts and completely silent about long ones.

These two properties therefore obtain different evidence from the same block, run and bound. One cycle can be decisive: the crossbar’s interleaving counterexample in this chapter’s opening is eleven cycles long, so a run stopped at ten would have reported *not violated up to 10 cycles*, misleading if filed as proof. Figure 14.3 draws that counterexample beat by beat.

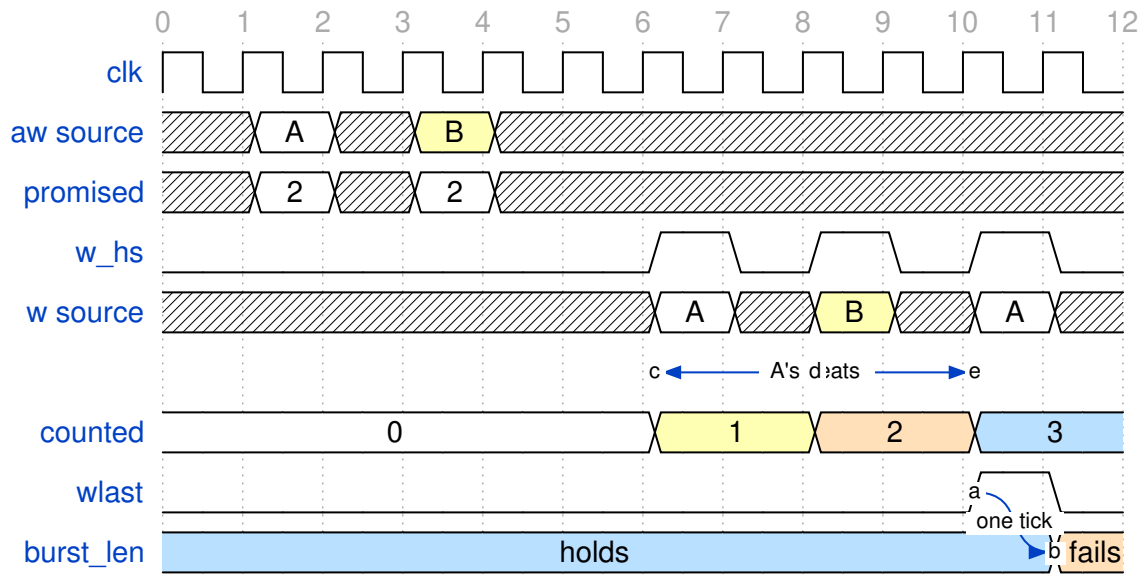


Figure 14.3 — The counterexample a proof engine returns: the crossbar interleaves a write-data beat from manager B between manager A's two beats, so the port checker counts three beats against the two the address promised, and `a_w_burst_len` fails at cycle 11.

A low-cost check adds one directive per property: **the antecedent or guarded state should be covered under the same bound before accepting a bounded pass**. Every related cover point must land below the bound to validate that it is not optimistic [7]; without a witness, the bounded proof has not reached the relevant behavior.

Scenario (the sensor hub). The always-on sensor-fusion block lives on the 32 kHz island and wakes a DSP running at system speed. The obligation under proof is a wake-path latency: from an asynchronous sensor event, the DSP must have its first sample within a fixed number of *slow* ticks. Measured in fast-clock cycles, a handful of 32 kHz ticks is tens of thousands, and no bounded run reaches the interesting behavior. **Approach.** Rather than increase depth, the proof should use the slow-domain state machine, reducing the fast domain to its handshake interface and clocking the property on the slow clock so a tick is its natural unit. **Why.** Depth depends on the measurement clock, so the slow-clock model measures this wake-path obligation in slow ticks. The record must state that synchronizer and metastability behavior at the boundary is outside the reduced model and remains subject to static clock-domain-crossing analysis (Chapter 13).

14.4 Abstraction and excluded behavior

For a proof to complete, the model must be small enough for the tool to finish within the CPU and memory available; this usually means verifying at a relatively abstract level, or verifying relatively small subsystems separately, and doing it with an expert's insight into both the design and the engine [2]. Abstraction is therefore fundamental to formal verification.

Every reduction changes the model’s possible behaviors; the direction determines which results may still be trusted:

- **Over-approximation** adds behaviors. The abstract model is simpler than the original and allows more traces; if the assertion is proven on it, it also holds on the original. If the assertion fails, the counterexample may be *spurious*, impossible in the real design. Over-approximation is **sound**: a success is always accurate [3].
- **Under-approximation** removes behaviors. A failure is always real, so it is **safe**, but a pass means nothing, which is why tools that use it say “not violated up to bound 50 clock cycles” rather than “passed” [3].

The practical hazard of over-approximation is human rather than logical. Classifying each counterexample manually is difficult when many are spurious; in this example, skimming after twenty impossible traces misses a real twenty-first trace. Reduction gains capacity at the cost of analyzing spurious failures [3].

Manual reductions and their implications

Tools automatically prune most irrelevant logic: a property’s cone of influence is the part of the design that can affect its value, and anything outside that cone is excluded. Manual reduction is needed only for signals that affect the property or the assumptions indirectly, which is precisely why it takes detailed knowledge of the design [3]. Three manual reductions cover most needs, with different implications:

- **A free signal** (*cutpoint*) is disconnected from its fan-in and can take any value at any time. Over-approximation: sound, not safe. A cutpoint is appropriate when a property must hold for *every* value of some input, because a property proven with a signal free is thereby proven for the correct values too, so the cut strengthens the result [8]. Blackboxing a submodule is the same lever applied to all its outputs at once.
- **An input set to a constant** under-approximates: safe, not sound. Useful for making a data path tractable: to verify propagation through a queue it can be enough to watch a single bit and tie the rest to zero.
- **An internal signal set to a value** may both forbid original behaviors and add new ones, so it is neither safe nor sound [3]. Each legitimate use needs a recorded justification.

For memories and wide arithmetic there is a fourth move: retain symbolic reasoning on the control logic and treat memory accesses or arithmetic operators as uninterpreted: a black box whose only property is that equal inputs give equal outputs [1].

A tile-based mobile GPU’s memory-ordering rules specify which writes a shader cluster may observe out of order; enumerating its megabytes of unified memory is infeasible. Two reductions make it unnecessary and they are justified differently: one reduction is on the values the memory holds, the other on the addresses it is indexed by. For the values, the rule does not depend on any stored value, so the memory can be uninterpreted without losing anything the rule concerns. The address reduction makes the model tractable and needs a separate argument: the rule is a relation over a *pair* of locations, so a model carrying two symbolic addresses (a write to one

ordered against a write to the other, as seen by a second cluster) states it exactly, while every other location stays free. Two addresses are minimal: one deletes the property because no second write remains to order against. The reduction must be recorded when made: an ordering violation that appears only across a third address or a third cluster goes to system-level simulation.

The canonical proof abstraction: two caches, one address

The coherent hub illustrates a state space too large to bound: the product of every line's MESI state in every cache, for every address. MESI names the four states a cache can hold a line in: Modified, Exclusive, Shared and Invalid. Chapter 11 introduced one invariant for proof: at most one cache may hold a line Modified.

```
a_single_writer : assert property (@(posedge clk) disable iff (!rst_n)
    $onehot0(modified_vec));
```

The proof uses two caches and one address, sufficient because the invariant is per line. The reduction preserves four behaviors: the directory's transition rules for a line; every race between two requesters for it; the ordering of invalidations against data responses; and starvation of one requester by the other, since livelock occurs on a single contended line. The reduced model retains these coherence behaviors; the exclusions below need separate coverage.

A snoop is a request the hub sends to a cache about a line. Arm IHI 0050, the *AMBA CHI Architecture Specification*, Issue H (2025), gives a coherent interconnect a seven-state cache model; it separates the snoops from the data responses onto four link-layer channels in its Table B13.1; and it specifies the cache state transitions twice over, once at a requester and once at a snooped cache, as a table per transaction type [9]. The reduction keeps all of that for the one line it models; what it drops is the product across lines, caches and addresses.

The excluded behavior must be recorded during reduction, since reconstructing it three months later is unreliable:

- **Anything needing a second address.** The classic coherence deadlock, two agents acquiring two lines in opposite orders, is excluded by construction.
- **Anything needing capacity.** With one address there are no conflict evictions, so a writeback triggered by replacement rather than by a snoop is outside the model.
- **Anything needing a third sharer,** including the directory's own sharer-vector encoding and its overflow behavior.

Reducing the hub's many caches to two is **not** sound in the sense defined above. Freeing a signal and setting an input to a constant each have a defined direction of error; deleting requesters has none in general. Reducing the requester count neither removes behaviors nor adds them: it is a different parameterization of the original, with a narrower sharer vector and arbitration over a smaller set. It is therefore neither an under-approximation nor an over-approximation, and neither implication

direction is available. The reduction is a modeling decision justified by a *symmetry argument* (that the hub treats its requesters interchangeably), and that argument is a claim about the RTL rather than a theorem. If the arbitration inside the directory is fixed-priority by requester index, it is simply false, and the two-cache proof will never show the lowest-priority requester starving behind the others. The claim must be checked in the code before use and recorded in §14.2’s assumption register with the other unproven premises. An abstraction argument is an assumption in every sense except the syntactic one.

Entirely avoiding lost bugs would require an abstraction that kept everything, which would not converge. Each deleted bug class must be named when removed and assigned another detection method. On the hub, the two-line deadlock goes to a separate, differently-abstracted proof; the eviction path goes to system-level simulation with a cache-thrashing traffic profile; the sharer-vector overflow goes to a directed test, a stimulus whose content and order are written out by hand. Each becomes a plan row reading *not covered by proof P-COH-2; covered by ...* (Chapter 5’s traceability, and Chapter 16’s division of labor). A proof’s value as evidence depends on explicitly listing its deletions.

Writing the model can itself be a review of the specification. A published Isabelle/HOL formalization of CXL.cache, the inter-device coherence protocol of the Compute Express Link standard, was built from the standard’s English prose and used to prove a single-writer-multiple-reader property over that model: an invariant of nearly eight hundred conjuncts against sixty-eight transition rules, and so more than fifty thousand lemmas to discharge. Each conjunct is a separate fact that the invariant requires to hold. Its authors report that the more useful output was a list of places where the prose was unclear, ambiguous or inaccurate, nearly all of which the CXL consortium has tentatively accepted for adoption ^[10].

14.5 Convergence: the undetermined property

A result is inconclusive when the tool times out, runs out of memory or uses an algorithm incomplete by construction. Getting a conclusive answer means refining the verification model: a more aggressive or more effective abstraction, a smaller model, auxiliary assertions (**lemmas**, which may be easier to prove and, once proven, can be supplied as assumptions for the original property) or simply different tool options ^[3].

The order below differs from this list and can reduce a fortnight’s work to an afternoon in this example.

1. Property validity. Deep bounded checking should precede invariant development. An undetermined property is sometimes a false one whose counterexample is simply further out than the last attempted bound, and, in this example, finding it costs one overnight job instead of a week of strengthening.

2. Property formulation. In simulation an inefficient assertion costs runtime; in formal it can be fatal, because a session with an inefficient assertion may return no

conclusive result at all, and where one text must serve both engines, the form efficient for formal should win ^[3]. Unbounded eventualities, sprawling antecedents and wide equality chains are the usual offenders. Rewriting costs little relative to the subsequent steps.

3. Separate assertions. Asserting five things together combines five proofs under one worst case. Separating them lets most close; the report can identify the one unresolved property alongside four successes.

4. Proven lemmas. This is §14.3's invariant strengthening used deliberately rather than reactively: facts about internal state believed by the engineer but not known to the engine (a queue's occupancy matches its pointers, a state vector is one-hot, an outstanding counter never exceeds the port's capacity) are each proven independently and supplied to the difficult proof as assumptions. Only these assumptions need no further discharge, because they have already been proven.

5. Case splitting. After showing configuration-register values partition the space, the property is proven separately for each value; the plan row must record N results.

6. Abstraction is the last model-changing step because it incurs §14.4's exclusions.

7. A bounded result is acceptable with depth recorded and its antecedent covered under the same bound. **8. Simulation closure.** The plan row is handed to simulation and closed there. Closing the row in simulation is a legitimate outcome; refusing it can consume a whole schedule on one property. Four steps change the problem and restart the process on a smaller version: rewriting, lemmas, case splitting and abstraction. [Figure 14.4](#) connects these choices to the result of each attempt.

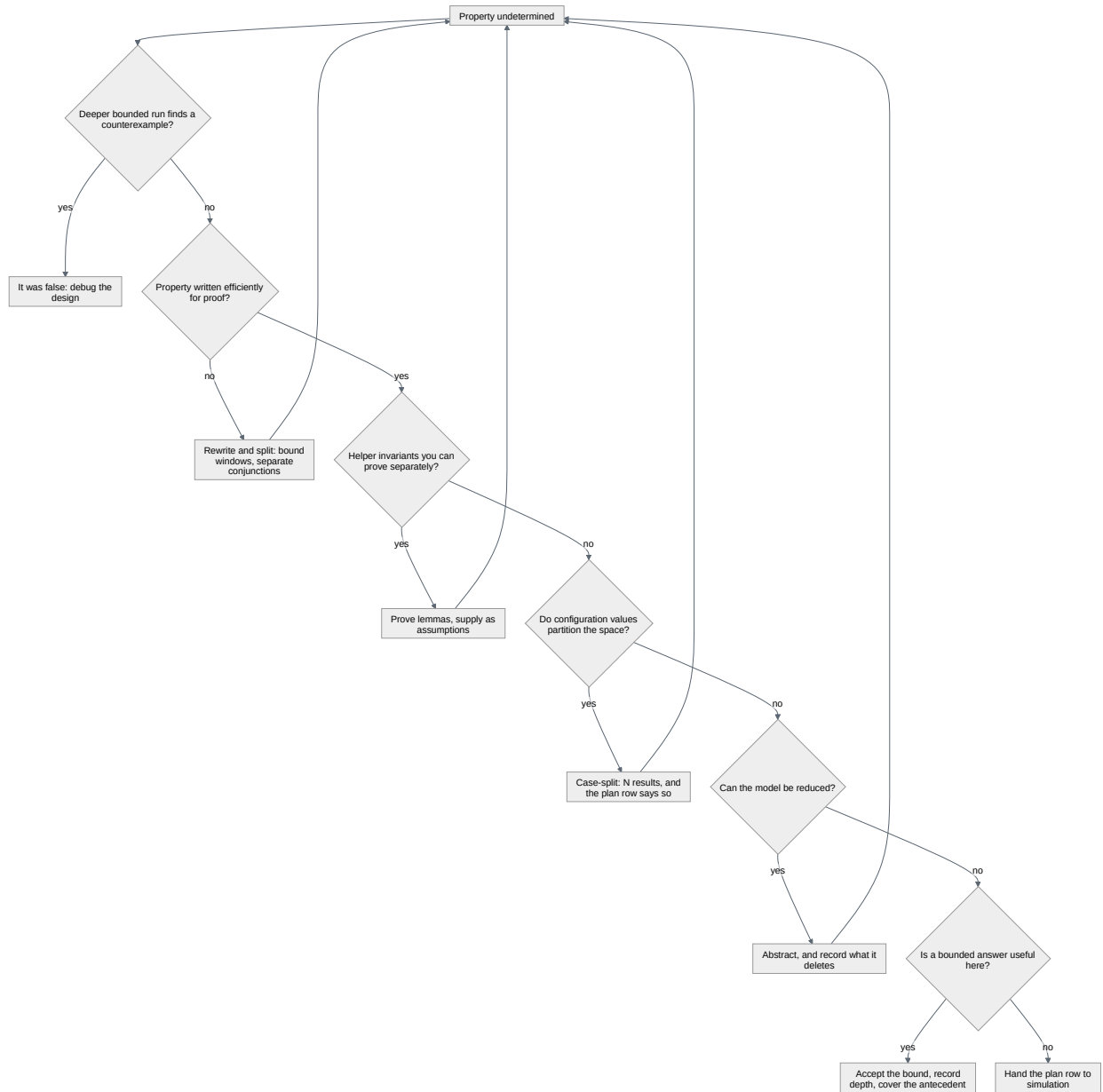


Figure 14.4 — The order in which to work an undetermined property, drawn as a decision tree: ask whether the property is false before strengthening anything, then whether it was written for a proof engine, then split it, then lemmas, then a case split, then abstraction (whose bill §14.4 states), and then a bounded answer; handing the plan row to simulation is the last exit, not a defeat. Four moves loop back to the top: rewrite, lemmas, case split and abstraction. They change the problem instead of answering it and hand a smaller problem back to the same tree, which is why the tree is drawn in the order in which experience tries the moves, not in the order in which a menu would list them.

Tool options also affect convergence. The strategy by which a property-directed-reachability engine generalizes its proof obligations determined which instances the engine could solve at all, measured across seven hundred and thirty hardware model-checking competition instances at a one-hour limit; each strategy solved instances the others missed, which supports running several methods in parallel [11].

When formal is the right instrument

Formal has two resource models; conflating them risks over-investment or dismissal after one unsuccessful week.

Lightweight formal asks only for adequate input assumptions, the most difficult and effort-consuming step; running the tool then resembles running a simulation. Early runs with small bounds remove obvious bugs before a simulation environment exists, saving time at the start of the schedule [3]. This flow is within a verification engineer's competence; the specialist requirement belongs to exhaustive formal. A published test deliberately inverted the staffing: at Intel, on a live graphics project, property verification was pushed out of the central formal group to the *design* engineers; 85 of those engineers were trained inside two weeks and then supported by onsite consultancy for a further four; and the drive found 82 interesting bugs across twelve weeks, even though it started only after a major dynamic-verification milestone had already completed; the simulation environments were already stable and reporting no failures [12]. A milestone is a phase gate defined as a checkable demonstration rather than an estimate of the project's state.

Exhaustive formal is a full-time job: model reduction and pruning, hand-written abstract models for parts of the design, checking that the specification is complete, iterative refinement, algorithm tuning. That cost is worth paying only where correctness is critical and simulation unreliable: arithmetic units such as multipliers and dividers, arbiters, coherency managers, branch prediction logic, critical controllers, bus protocols [3]. Reference SoC targets include crossbar ordering and routing, the DMA's burst-splitting arithmetic and the event unit's interrupt priority resolution. Chapter 10 reached the same conclusion from stimulus: when the specification is a property set, constrained-random generation samples what proof establishes.

Equivalence-checking measurements show the same split. A reference model is an independent executable of the specification, run on the same stimulus as the design and taken as golden. One DVCon paper from Arm and Mentor reports proving floating-point units of an Arm CPU design sequentially equivalent to their C reference models and publishes the human cost: two days of proof development for the adder against two months each for the multiplier, the divider and the square root [13]. Another, from Intel's graphics organization, runs the same C-versus-RTL comparison before the design is complete and with the full-coverage claim deliberately dropped, and reports eighteen bugs across the C++ model and the RTL in under a week's effort [14].

Scenario (the video codec). A team with proof capacity proposes a critical, hard-to-verify H.265-class encoder as a formal target. **Approach.** The encoder should be divided, with only one half assigned to property verification. The rate-distortion decision path is data-dependent control over a wide bit-exact datapath whose specification is a reference model. There is no compact property that says "this is the correct macroblock", and any attempt to write one reproduces the reference model in SVA, the SystemVerilog assertion notation that a simulator and a proof engine both read. That half belongs to golden-model comparison in simulation, and its

arithmetic blocks to the equivalence checking of Chapter 15. The other half contains formal targets: a frame is never emitted before its last slice completes, the line buffer is never read past what has been written, a bitstream packet's length field always matches the bytes that follow. **Why.** In this encoder, the bounded-state rules are property-verification targets; the data-transformation path is checked against its reference model.

14.6 Formal sign-off

At sign-off, a proof supports a plan row that must specify its evidence. A formal test-bench is ready for sign-off when it can answer three questions: completeness of end-to-end checkers, absence of unintentional over-constraints and attainment of required proof depth by all checkers [7]. These restate §14.1's three qualifiers, model, assumptions and depth, from the reviewer's perspective.

Coverage when there is no stimulus

Simulation coverage measures what tests exercised. Proof has no tests; formal coverage has four meanings, each answering a different question [15].

Cone-of-influence coverage takes the union of the cones of all asserted properties and reports the design space outside it. This inexpensive structural measure identifies logic without any check; areas outside every cone contain verification holes. The response should be more properties or a reviewed waiver naming the plan item covering that logic. It cannot establish closure: proving an assertion does not verify its whole cone.

Input stimuli coverage ignores the assertions and measures what the *constraints* let the engine reach, reporting code that is uncoverable only because of them; this turns §14.2's over-constraint hazard into a number. In the reported case, one line was uncoverable because a single setup command in a tool script held an enable low. It had been proven nowhere and checked in no simulation, and it would have survived any number of manual reviews [15].

Bounded proof coverage measures how much of a property's cone is reachable within the current bound, and supplies the stopping rule §14.3 left as judgment. If a bounded proof covers very little of its cone, the bound is not good enough to sign off on; and when two days of engine time between bound 10 and bound 11 adds no newly covered condition, the proof should use abstraction instead of further depth increases [15].

Proof core coverage provides the most precise structural measure at the highest cost. Engines abstract internally, so the logic actually used to establish a proof (the proof core) can be much smaller than the cone, and a bug outside the core but inside the cone will not be caught by that assertion even though it is fully proven [15]. Practitioners warn that such vendor-specific metrics are smarter than simulation code coverage and correspondingly harder to interpret [8]. The analysis should target a handful of highest-value properties rather than the entire suite.

Grading the property set by breaking the design

All four models are structural: they say which logic was involved, not whether a check would have noticed a change to it. **Mutation** provides an empirical complement: a deliberate design defect should make some assertion fail [8]. It is Chapter 8's known-bad variant and Chapter 11's mutation question, applied to a proof.

At a DAC challenge in 2015, attendees inserted 73 functional bugs into a crossbar with an existing formal testbench; every one triggered one or more checkers. Two further changes failed nothing: one was a loop bound rewritten to save area, which had no functional impact at all; the other had been inserted specifically to evade the methodology, an evasion that the authors classify as a non-bug on the grounds that a determined adversary can always do that [7]. Mutation grades the property set against designer bugs rather than adversarial changes. Chapter 23's threat model is a different instrument.

Completeness and its cost

Chapter 3 introduced a stronger criterion for verification gaps. It asks whether the properties **determine the outputs**. To exclude verification gaps it is enough to establish that, for every input trace, exactly one output trace satisfies the conjunction of the properties. In other words, it is enough to establish that the engineer pinned down the output value at every point in time. If not, the verification must be improved [16].

Real interfaces require a refinement of this otherwise excessive criterion. Protocol specifications routinely leave signals free outside their valid windows, and those values may legitimately change with an area or power optimization, so a verification that checks them is checking the wrong thing. The permitted freedom is made explicit through *determination conditions* (assumptions saying which input traces count as the same, and commitments saying which output traces count as the same), so completeness is judged only where the specification actually commits. A bug that crosses a sign-off boundary undetected is called an escape. Across a published list of commercial projects using complete verification and its less mature predecessors, almost all the verified circuits functioned properly after fabrication, and in the few projects where a fault was overlooked the escape was traced to human failure in applying the methodology: wrong constraints that masked errors, or an inaccurate description of expected output behavior that accepted a real fault [16]. Chapter 3 identifies the cost: a different property-writing method, a dedicated methodology and effort concentrated on selected modules.

Verification of this kind, with the completeness check run as an automatic step, was in regular productive use at Siemens and at Infineon from 1999 onwards; the published project list's only named part is the Infineon TriCore2, a superscalar 32-bit automotive processor, alongside AHB master, slave, bridge and multilayer bus interfaces and an AXI interface, and on those projects experienced engineers verified 2,000 to 4,000 lines of RTL per person-month, with a recorded best of 8,000 [16]. The author reports that almost all the circuits verified this way worked after fabrication, and that the escapes which did occur came from a misapplied methodology rather than from a criterion too weak to catch them.

Sign-off package contents

The reviewer needs the following evidence:

1. **The property list with per-property results:** proven, falsified-and-fixed, or bounded with its depth stated as a number.
2. **The assumption register** of §14.2, with each row's discharge status: proven on the driving block, checked in simulation, or accepted with a written argument.
3. **The required proof depth** for every bounded result, with its derivation. The calculation combines latency analysis, micro-architectural analysis (how many transactions must pass to exercise the arbiter), corner-case analysis, plus margin. On an 8×8 crossbar it came to 13 cycles: one cycle of design latency, an input sequence long enough to push eight same-priority requests through the arbiter (the last of them visible at the target at cycle 11), plus one cycle for back-pressure and one for a safety net. Because the engine reports the *shortest* path to failure, any input sequence beyond that depth reaches only scenarios already covered inside it [7], provided the corner-case enumeration behind the depth is complete, so review must examine the enumeration as well as the arithmetic. Two cross-checks make the number defensible: every related cover witnessed below the bound, and a bound exceeding the longest counterexample the project has ever seen on that block [8].
4. **Formal coverage evidence** for whichever of the four kinds were run, with waivers for uncovered logic.
5. **The abstraction deletion list** from §14.4, each entry pointing at the plan row that covers it elsewhere.
6. **The provenance:** RTL revision, property-file revision, tool build and engine settings. The proof must be reproducible.

Worked example: the crossbar's formal row in the sign-off package

Table 14.1 records the crossbar properties and the evidence each contributes to sign-off.

Table 14.1 — The crossbar’s formal results in the shape a sign-off package presents them: one row per property binding, with the verdict, the proof depth where the result is bounded, the assumptions the row leans on by ID, and a note. The columns are chosen so that nothing can hide behind a green tick — the third row records a property that was false and fixed before it was proven, the fourth is the only bounded result and therefore owes a derivation of its depth, and the fifth was proven only under an assumption withdrawn at review, so the row leaves formal for a directed simulation sequence.

Property (binding)	Result	Depth	Assumptions	Note
<code>a_aw_stable</code> (4 subordinate-side)	proven	unbounded	A2	the crossbar’s own stability toward each subordinate
<code>a_b_id_outstanding</code> (4 subordinate-side)	proven	unbounded	A2, A4	
<code>a_w_burst_len</code> (4 subordinate-side)	falsified → fixed → proven	unbounded	A2, A4	CEX 11 cycles; bug #217 (Chapter 4), this chapter’s opening
<code>a_grant_within_64</code> (arbitration, per manager port)	bounded pass	78	A1, A2, A3	window and depth derived below; antecedent cover at 12
<code>a_b_order</code> (response ordering, per manager port)	proven under A5; undetermined without it	—	A2, A5	A5 withdrawn at review; row handed to simulation as a directed multi-ID sequence

Table 14.2 records the environmental premises on which those results depend.

Table 14.2 — The assumption register those results lean on: the five assumptions by ID, the party that owns each one, and how each was discharged. The discharge column is what the proofs are worth — A1 is discharged three different ways across the four subordinates, A4 was proven on the crossbar itself and supplied back as a lemma, A3 rests on simulation alone, and A5 was added during debug to make a property converge, was never reviewed, and is withdrawn at sign-off.

ID	Assumption	Owner	Discharge
A1	every subordinate accepts an address within 16 cycles — a project constraint, not a protocol requirement	subordinate owners	proven as an assertion on the SRAM subordinate; checked in SoC simulation for the APB bridge (<i>AMBA® APB Protocol Specification</i> , Arm IHI 0024, Issue E, 2023, chapter 3, “Transfers”) ^[17] and the neural accelerator; the boot ROM’s port is always ready, discharged by construction
A2	the five managers obey the AXI manager rules (Arm IHI 0022H.c §A3.2.1 “Handshake process”, §A3.4.1 “Address structure”, §A5.2.2 “Write data ordering”) ^[6]	DV	the same bound checker with <code>ASSUME_MODE = 1</code> on the manager ports; the identical text is <i>asserted</i> on the DMA and core in their own proofs
A3	no manager stalls its own responses indefinitely	DV	simulation only — if false, the response buffers fill and A1 fails behind them, so <code>a_grant_within_64</code> does not hold; accepted, revisit if arbitration changes

ID	Assumption	Owner	Discharge
A4	at most one write address outstanding per manager at each subordinate port	design	proven on the crossbar itself , then supplied back as a lemma; it bounds the checker’s address queue to five entries. Two managers with one write each still overlap, so the opening’s bug stays in the model
A5	each manager keeps at most one write outstanding per ID	DV	none — added during debug to make <code>a_b_order</code> converge, never reviewed. Withdrawn at sign-off

Row four, the only bounded result, needs the derivation required by item 3. *The obligation* distinguishes two readings of “grant” by A1: an offered manager-port write address must be granted by the target subordinate’s arbiter within 64 cycles. The address handshake that follows is a separate obligation, bounded by A1, and not what this property claims. *The contention model* makes the row falsifiable: all five managers may want one subordinate at once, the arbiter is round-robin and sees a request in the cycle it is offered, and a grant is held until the subordinate accepts the address, sixteen cycles at worst, by A1. *The window* is $4 \times 16 = 64$: each of the other four managers is served at most once ahead of this one and holds the target address port for at most sixteen cycles. *The depth* is derived from the window: the antecedent (five requests pending at one subordinate) is first witnessed by its cover at cycle 12, so the shortest counterexample this property admits is 76 cycles long, and the two cycles of margin above, one for back-pressure and one for a safety net, put the run at 78. There is no latency term, because the obligation ends at the grant, not at the subordinate’s port. Chapter 11’s unbounded rule states that a request is eventually granted, which a proof engine decides; sign-off records the bounded restatement so a reviewer can recompute it.

A1 makes the wait finite only in this row. Rows two and three check pure safety, an unsolicited response and an incorrect beat count; subordinate acceptance speed cannot affect their truth, so they exclude A1. What they need is A4, the lemma that keeps the checker’s queue finite. Assumptions must be derived per row; copying the column gives unsupported evidence.

A3 and A5 explain the second table’s importance at review. A3 was a known concession accepted with an argument. A5 was added mid-debug for convergence and never reviewed; with one response per ID in flight, no pair remained to order, so `a_b_order` proved a rule over traffic A5 had deleted. As Chapter 7 states, full proof under over-constraints leaves a verification gap, so review must examine assumptions rather than checkmarks.

IN PRACTICE

An organization must distinguish formal’s two streams. Designers should run lightweight proofs on their own blocks in the first weeks after RTL compilation, before a simulation environment exists. A much smaller group should run exhaustive proofs on selected blocks. On IC/ASIC projects, formal property checking adoption grew at a 5.8 percent compound annual growth rate (CAGR) through 2024, while automatic formal applications, Chapter 15’s packaged checks requiring no property writing,

grew at 8.7 percent [18]; FPGA projects had a combined 7.5 percent CAGR [19]. The half requiring no property-writing expert grew faster; the other half still depends on that expertise.

With nightly RTL changes, a six-week-old proof describes an obsolete design; formal should therefore run periodically in continuous integration like a regression tier (Chapter 12), treating newly undetermined properties as failures. An assumption added on a Friday for debug convergence can remain until sign-off; the assumption register and named owner should prevent this accumulation. One industrial effort found that an environment assumption blocking proof could be eliminated by adding a small amount of arbitration logic, concluding that designing for provability yields shorter proofs and simpler designs [2]. Design review should address provability while changes remain inexpensive.

PITFALLS

1. **Reading a bounded pass as a proof.** Symptom: the report says “proven” where the tool said “not violated up to N cycles”. Cure: every report should include depth and its derivation for the block.
2. **An assumption removing the property’s precondition.** Symptom: a property that would not converge for days proves in seconds shortly after an assumption was added. Cure: delete that assumption and re-run; if the property collapses, the assumption was doing the proving.
3. **Waiving a counterexample.** Symptom: “we know our managers never do that”, and the failure is dismissed. Cure: the waiver must become an explicit, reviewed assumption; tools report one counterexample rather than all, and several bugs can violate the same assertion, so dismissing the failure without an explicit assumption leaves the second hidden behind the first [8].
4. **Abstracting without a deletion list.** Symptom: a proof that now converges, and no record of what left the model. Cure: the deletion list should be written during reduction, with another coverage destination for each behavior.
5. **Spurious counterexamples masking real failures.** Symptom: an over-approximated model produces impossible traces and the team starts dismissing failures by reflex. Cure: the invariant should be strengthened until traces are real; twenty spurious traces do not make the twenty-first one spurious.
6. **Proof effort misaligned with risk.** Symptom: a growing property count concentrated on decode and register logic, with the ordering and arbitration rules untouched. Cure: proof coverage should be compared with the plan’s risk priorities.

SUMMARY

- A proof is a statement about a model, under assumptions, sometimes to a depth. “Formally verified” without those three qualifiers is not a claim [2].

- A formal report distinguishes four outcomes rather than two; a bounded pass, the one most often mistranslated, establishes only that the property cannot be violated too soon [3].
- Assertions and constraints are one artifact with two directions. In formal an assumption is never checked; it deletes traces, and an over-constrained model produces vacuous proofs that look exactly like real ones [1].
- A missing assumption produces a spurious counterexample, while an extra one produces no warning. This asymmetry makes the assumption list the sign-off review target.
- Abstraction is fundamental to formal verification. Over-approximation preserves proofs and can fabricate failures, under-approximation preserves failures and cannot prove anything [3].
- Convergence work has a cheapest-first order: check the property is true, check it was written for a proof engine, split it, prove lemmas, case-split, abstract, accept a bound, stop.
- Formal coverage measures four aspects: cone of influence, input stimuli, bounded proof and proof core; none establishes whether a check would have caught a change. Only mutation does [8, 15].

EXERCISES

Exercise 14.1 (calculation). §14.3 puts one bounded run on the DMA engine's backend and attaches two of Chapter 11's properties to it, at the single bound the section states. Using the beat count §14.3 gives for the engine's maximum burst, give a lower bound on the number of cycles in which that run could reach the guarded event of `a_w_burst_len` on a maximum-length burst. Then express the part of the burst-length range that the run does reach as a fraction of that maximum. Compute the number of pushes that reaches the guarded state of `a_no_overflow` from the FIFO depth §14.3 gives, and state which of the two properties satisfies the low-cost check §14.3 attaches to a bounded pass.

Exercise 14.2 (calculation). The worked example of §14.6 builds the depth of its only bounded row from a contention window, a witnessed antecedent cover and a margin. Recompute the window and the depth on the supposition that the acceptance bound stated by assumption A1 is halved, holding the manager-port count, the arbitration policy and the witnessed cover cycle as §14.6 states them. State which term of the derivation is unchanged by that supposition and why, and give the reason §14.6 puts no latency term in this row.

Exercise 14.3 (judgement). A reviewer is offered the coherent hub's two-cache, one-address proof with the argument that a reduction can only add behaviors, so a success on the reduced model carries to the full hub. Decide what §14.4 licenses in that argument and what it does not, keeping the address reduction and the requester-count reduction apart. Name the premise §14.4 says the requester reduction rests on, state where §14.4 requires that premise to be recorded, and give the arbitration implementation that makes it false.

Exercise 14.4 (judgement). A team offers proof core coverage, run over every property in its suite, as evidence that the property set would notice a change to the crossbar's arbitration logic. Decide what the four coverage kinds of §14.6 establish as a class and what none of them establishes, and name the instrument §14.6 assigns to the question the team is actually asking. State the scoping §14.6 gives for proof core analysis, with the two reasons it gives for that scoping.

Exercise 14.5 (artifact). Read the two-run record below, kept for one response-ordering property on a manager port of the crossbar. State what the cover result establishes about the assumption added between the two runs, and name the form of over-constraint of §14.2 that the record exhibits. Name the low-cost manual check of §14.2 that this assumption passes despite the cover result, and give the step of §14.5 the record shows was skipped. Give the discharge status and the disposition §14.6 records for an assumption in this condition.

```
property          a_b_order (response ordering, manager port)
run 1             undetermined after eight hours
change           assumption m_one_outstanding_per_id added
run 2            proven, unbounded, elapsed 00:00:41
cover c_b_pair    two responses of the same ID in flight: no witness
support of m_one_outstanding_per_id  awvalid, awid (design inputs)
review of m_one_outstanding_per_id    none
```

FURTHER READING

Within the corpus, ^[2] surveys formal verification and the required content of verification claims, while ^[3] Chapter 20 gives practical definitions of counterexamples and witnesses, complete and incomplete methods, over- and under-approximation, pruning, lightweight and exhaustive flows, and assume-guarantee. ^[1] places model checking and bounded model checking among formal techniques and explains constraint/assertion duality; ^[4] treats bounded and unbounded proof, sequential depth and spurious counterexamples from arbitrary initial states in induction; ^[16] presents the corpus's strongest completeness argument. For industrial practice, ^[7] is a sign-off methodology worked end to end on a crossbar, with a required-proof-depth derivation and a mutation experiment; ^[15] defines the four formal coverage models; ^[8] tours complexity management (cutpoints, black-boxing, counter abstraction) and the traps around waivers and coverage metrics.

Outside the corpus: E. Seligman, T. Schubert and M. V. Achutha Kiran Kumar, *Formal Verification: An Essential Toolkit for Modern VLSI Design*, is the standard book-length introduction to the practice this chapter describes, and E. M. Clarke, O. Grumberg and D. Peled, *Model Checking*, remains the canonical text for the algorithms deliberately left inside the tool here.

REFERENCES

1. **B6** J. Yuan, C. Pixley, and A. Aziz, "Constraint-Based Verification," Springer, 2006.

2. **P20** C. Kern and M. R. Greenstreet, "Formal Verification in Hardware Design: A Survey," *ACM Transactions on Design Automation of Electronic Systems*, vol. 4, no. 2, pp. 123-193, 1999.
3. **B4** E. Cerny, S. Dudani, J. Havlicek, and D. Korchemny, "SVA: The Power of Assertions in SystemVerilog," 2nd ed., Springer, 2015.
4. **T1** M. Schwarz, "Formal Hardware/Firmware Co-Verification of Optimized Embedded Systems," Ph.D. dissertation (Dr.-Ing.), Technische Universität Kaiserslautern, Germany, 2021.
5. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
6. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
7. **D31** I. Tripathi, A. Saxena, A. Verma, and P. Aggarwal, "The Process and Proof for Formal Sign-off: A Live Case Study," *Proc. DVCon U.S.*, 2016.
8. **D13** J. Bromley and J. Sprott, "Formal Verification in the Real World," *Proc. DVCon U.S.*, 2018.
9. **S26** Arm Limited, "AMBA CHI Architecture Specification, ARM IHI 0050, Issue H," Arm Limited, September 2025.
10. **P27** C. Tan, A. F. Donaldson, and J. Wickerson, "Formalising CXL Cache Coherence," in *Proc. 30th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25)*, 2025, pp. 437-450.
11. **P26** T. Seufert, F. Winterer, C. Scholl, K. Scheibler, T. Paxian, and B. Becker, "Everything You Always Wanted to Know About Generalization of Proof Obligations in PDR," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 4, pp. 1351-1364, 2023.
12. **D18** M. V. Achutha Kiran Kumar, E. Seligman, A. Gupta, Ss Bindumadhava, and A. Bharadwaj, "Making Formal Property Verification Mainstream: An Intel Graphics Experience," *Proc. DVCon U.S.*, 2017.
13. **D37** T. W. Pouarz and V. Agrawal, "Efficient and Exhaustive Floating Point Verification Using Sequential Equivalence Checking," *Proc. DVCon U.S.*, 2017.
14. **D38** M. Achutha KiranKumar V, D. Puri, and S. Dharade, "Bringing DataPath Formal to Designers' Footsteps," *Proc. DVCon India*, 2019.
15. **D11** X. Feng, X. Chen, and A. Muchandikar, "Coverage Models for Formal Verification," *Proc. DVCon U.S.*, 2017.
16. **T2** J. Bormann, "Complete Functional Verification," Ph.D. dissertation, University of Kaiserslautern, Germany, 2009 (English translation of the German dissertation 'Vollständige funktionale Verifikation', April 2009).
17. **S19** Arm Limited, "AMBA APB Protocol Specification, ARM IHI 0024, Issue E (ID022823)," Arm Limited, February 2023.
18. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
19. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.

15. Formal Apps and Equivalence

Formal apps supply properties for predefined checks, while equivalence flows check transformations left unchecked by property checking. The reference SoC's top level illustrates the integration problem: two thousand lines of instantiation and wiring contain one mistake. The neural accelerator's error interrupt arrives at the event unit on input 5; the firmware header, generated from the architecture document, says input 6. Both blocks are correct, so every block-level environment passes. The system regression also passes, because no test both provokes an accelerator error and enables that particular interrupt. The error path is exercised by the accelerator's own tests, which read the status register. The interrupt map is exercised by the event unit's tests, which inject events directly. The mistake surfaces in the lab, as a hang.

The intent is a table of a few hundred rows specifying source, destination and condition; the implementation is structural wiring. Comparison is mechanical, whereas simulation needs one test per table row, written out by hand and checking a single wire without exhausting its conditions. Such a test is called a directed test, and it does the same thing on every run.

In this example, the team's one formal specialist is three weeks into the crossbar's ordering proof, choosing abstractions and defending assumptions (see Chapter 14). The specialist is unavailable but unnecessary here because the tool generates properties from a table.

CHAPTER MAP

§15.1 explains why the pre-packaged formal *app* succeeded where general property checking stalled: the tool supplies properties, so a team supplies only the design and a configuration. §15.5 examines the one app that extracts its properties from the design itself. §15.2 covers connectivity proofs from integration tables. §15.3 discharges a register map against its machine-readable description and states which gap register checks leave open. §15.4 replaces a coverage waiver's argument with an unreachability proof whose trustworthiness depends on its constraints. §15.6 states what logic and sequential equivalence checking each prove and assume, and why logic equivalence checking won full industrial acceptance while property checking was still developing. §15.7 explains why arithmetic datapaths need their own flow.

15.1 The app: formal as a product

Chapter 14's flow requires someone to read the specification, decide what to prove, write properties, state and defend assumptions, work toward convergence and interpret the result. Only one of those steps is the engine's. The rest is specification work,

which is why formal property checking spent two decades as a high-effort specialist activity requiring skills and expertise most project teams did not have [1].

A formal app supplies the properties, changing the cost of the flow. The user supplies the design and a configuration; the tool supplies everything else. This structure applies to every app in the catalog. An assertion is a declarative statement of a property the design must hold; a tool evaluates it, and its falsehood indicates an error. An X-propagation app reads the RTL, analyzes the design, and automatically writes assertions that check for X values reaching targets such as clocks, resets, control signals and output ports; the engineer's remaining job is to judge whether each proved occurrence was expected [2]. The tool already contained the specification, so no user wrote properties or stated assumptions.

Between 2012 and 2014, adoption of automatic formal applications grew by 62 percent against incremental growth for property checking, because these are narrowly focused solutions that do not require specialized skills to adopt [1]. A decade later, the gap persisted. Across IC/ASIC projects from 2016 to 2024, property checking adoption grew at 5.8 percent compound annually and automatic formal applications at 8.7 percent, a shift toward tools that make formal accessible to non-experts [3]. FPGA projects moved the same way over the same window, formal technologies together growing at 7.5 percent compound annually [4].

The app catalog spans the whole flow rather than one phase [5] (Table 15.1):

Table 15.1 — The catalogue of formal apps: each app, the claim it discharges, and — the column that decides whether it pays — the machine-readable source its intent comes from. Read the right-hand column as the price list, because the properties come free only where that source already exists for some other reason; where it does not, creating and curating it is a project's cost rather than a tool's. The first four rows are this chapter's business and the last two belong to the static gate.

App	What it proves	Where its intent comes from
Connectivity	Every declared connection exists, and no undeclared one does	An integration table (§15.2)
Register access	Reset values, access policies, decode	A register description (§15.3)
Unreachability	A coverage target admits no legal trace	The coverage database (§15.4)
Auto-generated properties	Interface and structural invariants extracted from the RTL	The RTL itself (§15.5)
X propagation	No unknown reaches a control point	The RTL's reset and initialisation structure (Chapter 13)
Clock-domain crossing	Synchroniser structure and protocol correctness	The clock and reset declarations (Chapter 13)

The first four are this chapter's business; the last two belong to Chapter 13's static gate. The fourth also connects to Chapter 14 by drafting what a specialist would otherwise write, and it is the one row whose economics do not resemble the others', so §15.5 gives it a section of its own.

An app relocates specification work. The properties come free only because the intent already exists somewhere machine-readable, and where that source does not ex-

ist, creating and curating it is the app’s real cost, borne by the project rather than the tool. Connectivity planning must account for intent curation when estimating hours saved on property writing.

An app uses Chapter 14’s engine, so a bounded result from an app is exactly as bounded as one from a hand-written proof. Run on a small module chosen with hindsight, an X-propagation app found its bug in minutes; run on the whole IP under a two-hour limit, the same app reported no violations and achieved only bounded proofs [2]. Free property writing does not reduce design size.

A safety app checked an ECC block at NXP Semiconductors using SECDED (single-error correction, double-error detection) around an 8K memory. When the safety app was run on the whole block, the fault campaign did not converge in even 12 hours, so the memory was blackboxed and its input ports promoted to observation points, the places where a fault becomes visible to the outside world. The authors characterize this abstraction explicitly because it can produce false negatives on the safe classification but not false positives. A prime fault is the representative of a class of faults with the same observable behavior. With that abstraction, the campaign ran in stages: it cut 3,606 injected faults to 2,489 prime faults; it proved 763 of those structurally unobservable; it then proved a further 232 formally unobservable, and after that another 16; and it ended by identifying 512 faults that corrupt the stored data without ever raising the block’s error signals [6]. The app supplied the fault model and machinery for pruning and classification, but a human supplied the abstraction needed for termination.

15.2 Connectivity checking: proving the wiring

Integration assembles individually correct blocks, so all its simple wiring errors survive block-level verification.

Each connection is a claim about a short path: this source reaches this destination, after this many cycles, when this condition holds. The cone of influence is the logic a proof has to consider, and here it is a wire and perhaps a multiplexer, so proofs converge in seconds, thousands run in one job, and each answer is a proof rather than a sample. The reference SoC’s interrupt map (several dozen sources into the event unit, some conditional on configuration) is discharged completely in a single run, where a simulation argument needs one directed test per source per condition and still leaves coverage unresolved.

From one table row, the app generates:

```
// Intent row: src = nn_err_irq, dst = evt_in[6],
//             enable = cfg_irq_en[6], latency = 1 cycle.
property p_conn_nn_err;
    @(posedge clk) disable iff (!rst_n)
        cfg_irq_en[6] | => (evt_in[6] == $past(nn_err_irq));
endproperty
a_conn_nn_err : assert property (p_conn_nn_err);
```

The antecedent is the enabling condition sampled now; the consequent fires one tick later and compares the destination against the source *as it was when the enable was true*, expressing a one-cycle registered connection through `$past`. It says nothing about cycles where `cfg_irq_en[6]` is low; an unconditional claim would be wrong because the connection is not always live. Two hundred rows of the connectivity table, generated and discharged in one run, is the entire flow.

One of the two hundred is the row above, and on the reference SoC it does not prove: the wiring reaches input 5. The proof needs no test provoking an accelerator error while the interrupt is enabled, whether or not one ever would. The generated text also lacks a cover on `cfg_irq_en[6]`. An intent row does not discharge Chapter 11's rule that every implication ships with a cover on its antecedent. This row fails, so its own gap is moot; on the other 199, nothing in the generated text asks whether the enable is reachable, and a successful result does not distinguish a proved connection from an enable no legal trace can set.

A positive connectivity table leaves the equally important requirement for forbidden connections unspecified. The reference SoC's debug fabric supplies negative rows: the JTAG-reachable abstract-access path may reach memory and the core's register file, but must not reach the event unit's retention registers. The abstract-access path is the debug fabric's own route for reading and writing memory and registers, and retention registers are the ones that keep their contents while the block around them is powered down. A no-connection check is cheaper because it asks structurally whether any path exists in the cone, without invoking a proof engine.

Scenario. The crypto engine sits inline on the memory path, with a key ladder that derives working keys from fuses. The security requirement is that a working key reaches the AES core and nothing else: not a readable register, not the trace buffer, not a scan chain. **Approach.** The job should include positive rows from key register to cipher core per round and the especially important negative rows: no path from any key-ladder output to any bus-readable address, the debug data register, or a scan-observable point. They should run in the same connectivity job at every integration build. **Why.** This structural security check catches a debug hook, a test observation point or a "temporary" mux added during bring-up, the phase in which environment and DUT first exchange checked transactions. A no-connection result is *structural*: no path exists. It does not say the key never influences an observable value along a path that does exist, which is an information-flow question and belongs to Chapter 23. Structural non-adjacency alone does not prove non-leakage.

Checking every path of a multi-port interconnect individually has poor runtime and, by symmetry, minimal verification value. The crossbar's five manager ports against its four subordinate ports make twenty combinations, and far more once each combination is qualified by burst type and identifier. With *symbolic* port indices, an arbitrary manager M and subordinate S are constrained only to remain stable, and the single path from M to S is checked. Because the proof must hold for every value the arbitrary indices could take, one property covers all of them. Symbolic checking is

not always faster: 162 seconds against 65 for all exhaustive per-path proofs together [5], so runtime needs measurement. It keeps the connectivity argument constant-sized as the interconnect grows.

15.3 Register checks: the map against its description

Chapter 10 built a register model and used it to drive and predict accesses. That model was generated from the subject of this section: a register description language (Accellera SystemRDL 2.0 Register Description Language, Version 2.0, 2018), one file from which every view is generated [7]. The views so generated include RTL decode and field logic, the verification model, firmware header and documentation; maintaining them separately allows drift.

The register app proves that RTL implements the description. From that description it generates and proves reset, access-policy and functional checks for every field [5], plus the implementation-level properties the description implies but does not state. For each register the app proves this much: the reset value is what the description says, for every reset; the decode maps exactly one address to it; a read returns what the policy says; and a write does what the policy says *for every value written*, not for the values a test happened to pick. It also proves that reserved fields ignore writes and that unimplemented addresses behave as declared.

A conventional register test walks the map front-door, register by register and pattern by pattern, through write-read-verify sweeps, reset checks and bit-bash, driving real bus cycles rather than reaching the registers by hierarchical path. In a bit-bash the test writes and reads back each bit of a register in turn. Its cost scales linearly with the map, so a large map's register test takes hours. The app proves the same statements over all values and orderings in one job, at RTL rather than through the bus, so it converges even for registers a bus-level test reaches only through a long configuration sequence.

The canonical status register from Chapter 10 carries the UART's RX overrun flag:

```
class uart_status_reg extends uvm_reg;
  rand uvm_reg_field oe;    // RX overrun error

  function new(string name = "uart_status_reg");
    super.new(name, 32, 0);
  endfunction

  virtual function void build();
    oe = uvm_reg_field::type_id::create("oe");
    //      parent size lsb access volatile reset has_rst rand indiv
    oe.configure(this, 1, 3, "W1C", 1, 1'b0, 1, 0, 0);
  endfunction
endclass
```

Two of those arguments determine how a register app checks this field. The policy "W1C" (write one to clear) is a statement about every possible write value and both possible current values, of which a random register test exercises a handful. And `vol`

`atile` records that hardware can change the field without software's involvement, which is why the check must read *a software write of 1 clears the field unless hardware sets it in the same cycle, not the field is 0 after the write*. A register description language makes this explicit: any hardware-writable field is inherently volatile, because verification and test need to know ^[7], so the generator derives the model's `volatile` argument from the field's hardware access rather than from anything anyone wrote down. Ignoring the flag generates a check that fails on correct hardware, creating an incentive to disable it.

Chapter 3 identifies the common-mode limit. The app proves *RTL against description*, and both the register model and the RTL check come from the same file, so an error in the description is invisible to both: a common-mode error, encoded once and read twice. Nothing here reviews whether bit 3 was the right bit. That review is a human reading the description against the architecture document, while the app removes the other checks, leaving only the part that needs a person.

Scenario. The GPU's four shader clusters carry identical register banks with performance counters, cache-invalidate triggers and per-cluster clock-gate overrides, instantiated four times with a parameterized base address. During a late change, cluster 3's bank is hand-edited rather than regenerated, and one counter's reset value is left at the previous revision's. **Approach.** All four banks should be regenerated from one description, then checked by the register app across the whole map: all four instances and every field. **Why.** Replication permits copy-paste errors, and a per-cluster register test finds this only if someone remembered to parameterize the test as well as the design, otherwise reproducing the same omission. The app enumerates from the description, so it checks cluster 3 whether or not anyone thought about cluster 3. In this example, four thousand fields take a long afternoon of bus traffic but a short formal job.

15.4 Unreachability: a proof instead of a waiver

Chapter 6's coverage closure distinguishes stimulus gaps from unreachable targets. Some holes are genuine stimulus gaps requiring closure. Other holes are targets no legal execution reaches, and they warrant an exclusion in Chapter 7's written form: the hole, reason, risk argument, owner and expiry. For an unreachability exclusion, the reason is the engineer's claim that a state is unreachable.

That claim is a reachability question about the design, and reachability is what a model checker computes. The unreachability app takes the coverage targets and asks, for each, whether any legal trace reaches it; the unreachable ones come back with a proof, and this saves the manual analysis they would otherwise consume ^[5]. The waiver's claim becomes a database result. Dead RTL is excluded from coverage metrics this way, manually or with formal tools; the exclusion is necessary for a meaningful coverage percentage ^[8].

Reachability depends on the constraints, and a wrong constraint makes reachable behavior look impossible. An input constraint in a tool setup script (`enable_cnt == 0`)

made a counter's increment line uncoverable; the line was reported as unreachable, and under that constraint it really was unreachable, though the constraint was only an assumption, one that no engineer had reviewed and that did not reflect the design's real environment ^[8]. An unreachability verdict inherits every assumption in the environment, including ones written in a scripting language by someone who has left the project. The constraint set should be evidence, reviewed as seriously as the waiver it replaces.

The second limitation is structural: the app works on the *design* (RTL lines, branches, FSM states, cover properties bound into design scopes), so a functional coverage model computed by the testbench is invisible to it. The canonical DMA coverage model is `cg_2d_4kb` from Chapter 5. Its cross `x_worst` pairs every value of each axis with every value of the others, and has **72** combinations: 3 stride signs $\times$ 4 boundary relations $\times$ 3 burst lengths $\times$ 2 states of the other channel. A stride is the signed address step from the start of one row to the start of the next. The other channel is the DMA's second transfer engine, independent of the first, with a state of its own and a share of the AXI backend. **Six** of those combinations, the whole `straddle  $\times$  max` column, are excluded: three stride signs by two other-channel states. They are excluded by *policy*, which Chapter 9 derives, rather than by a design property, leaving **66** cells to close.

The app cannot see that cross because `txn.bnd_rel` lives in the monitor. A machine verdict requires a design-level restatement in terms of the burst splitter, the DMA stage that splits each transfer into bursts that the AXI address rules allow. The design's own code calls the burst splitter by an older name, the midend legalizer, and the identifiers below keep that name:

```
// Design-level restatement: can the midend ever emit a full-length
// burst out of a transaction it had to split at a 4 KB frontier?
// Signals are the midend's own outputs, not the monitor's per-row relation.
// midend_did_split is per emitted burst: set when this burst's parent
// row had to be split at a 4 KB frontier. midend_burst_len is a beat
// count, like Chapter 5's cp_len – not an AXI AWLEN, which is beats-1.
c_split_max_len : cover property (@(posedge clk) disable iff (!rst_n)
    midend_txn_done && midend_did_split && (midend_burst_len == 9'd256));
```

The comment's encoding note is the protocol's own: *AMBA® AXI and ACE Protocol Specification*, Arm IHI 0022H.c, §A3.4.1, where `Burst_Length = AxLEN[7:0] + 1` ^[9].

The engine answers in seconds: the cover is **reachable**, with a trace showing how. Nothing in the design bounds a 2D row at one maximum burst; a 2D row is the contiguous run of bytes that makes up one line of a two-dimensional transfer. The bound that empties the column lives in the constrained-random generator (legal but unlikely stimulus, generated automatically under constraints), and a design-level cover does not inherit that bound.

The two questions are related but distinct. The design-level cover asks about the burst splitter's own split flag and burst length; the six-cell figure is an argument about the monitor's per-row `bnd_rel` axis and about how that axis pairs with a post-split length sample. The exclusion remains valid as a stimulus claim, not a design claim: true while the generator is unchanged, false when it widens.

The app therefore cannot discharge this exclusion; the limitation is structural. To return *unreachable* the tool would have to be told about the generator's budget, and that budget is not a property of the design: Chapter 9 labels its bounds engineering choices about solve time rather than statements about the DUT. Assuming that budget would make the verdict inherit it, recreating the first limitation. A disposition is the decision recorded on an open item, and the one recorded here remains a waiver whose expiry is an edit to the constraint file; that edit must count as verification evidence.

The third limitation concerns some targets unreachable only in the current configuration. A parameter value can make a whole class of coverage impossible (a narrow bus cannot carry a wide transfer) and waiving that coverage by hand is laborious, while aggregating coverage across parameterizations is a weak substitute [5]. Conditional unreachability should be generated: deriving targets and exclusions from the same configuration regenerates both on a parameter change. Hand-maintained exclusions can become stale when design parameters change.

15.5 Auto-generated properties and specification review

The fourth catalog row reverses the intent-source relationship of the first three. The app extracts implementation-detail properties from RTL [5]: interface handshakes, one-hot encodings and FIFO occupancy bounds, the structural invariants a designer would state if asked. No table, no description, no configuration beyond a module name. The third column identifies the cost advantage: the design already supplies the intent source.

Every other app finds bugs by comparing the design with external intent from a wiring table, register description or coverage model. Here, intent and implementation coincide by construction. A golden reference is an independent statement of intended behavior, trusted as the standard that the design is checked against. The limitation is structural rather than a matter of tool maturity: automatic property generation divides into two families, dynamic mining from simulation traces and static analysis of a specification; the mining family generates and evaluates its properties on the same RTL, and it has no golden reference, so flaws in the RTL yield flawed properties, and mining has no independent way to detect them [10].

A mined property derives from the behavior of its source RTL, which is why it proves on that RTL, and why it would prove in the same way if it had been mined from RTL with the opposite behavior. Its bug-finding power against today's RTL is zero, and no amount of proof depth changes that. Its regression value is to make current design behavior an obligation against which subsequent changes are checked.

Each of three hundred generated properties requires review for intended behavior, since truth is guaranteed by construction; this remains specification work for the specialist the app was supposed to replace. An app that emits verdicts removes that work. An app emitting drafts relocates that work to project review, as §15.1 explains for intent curation; generation remains free. Running the app requires a review budget.

Scenario. The reference SoC’s inherited UART is silicon-proven and four years old; its specification is a two-page register table plus the RTL. Bring-up needs a change detector around it before anyone touches the clock divider. **Approach.** The generation app should run and its several hundred extracted properties be retained as change detectors, never as statements of what the block should do. **Why.** The RX overrun flag never clears when software reads status in the same cycle a stop-bit error arrives; mining makes this RTL behavior an obligation, so fixing the bug triggers a regression failure. A generated set encodes the design’s defects with exactly the fidelity it encodes its features. For a block whose specification is lost, this remains useful provided the team distinguishes change detection from intended behavior and never cites the pass rate as verification evidence.

15.6 Logic and sequential equivalence checking

Every earlier technique compares a design against a *specification*, whereas equivalence checking compares it against another design. The following flow checks whether a transformation preserves functionality.

Chapter 2’s reconvergence model explains the flow. Synthesis transforms RTL into a netlist; equivalence checking is an independent second path from the same origin, and the two reconverge at a mathematical proof that the transformation preserved functionality ^[11]. The question has a yes-or-no answer, requiring no stimulus or coverage and avoiding the *oracle* problem, the difficulty of deciding whether an observed response is correct. Figure 15.1 follows that second path along the chain of transformations, and the branch a broken state-point mapping takes.

Logic equivalence checking (LEC) is the combinational form used throughout the flow. It compares two structurally similar models, most often RTL against the netlist synthesized from it, or netlist against netlist after a post-processing step such as scan-chain insertion, clock-tree synthesis or a manual edit ^[11]. It is an automatic activity requiring minimal human interaction, and its scalability to industrial-size circuits won it full industrial acceptance at a time when property checking was still developing ^[12].

LEC assumes matched state points rather than reasoning about sequences. It matches *state points* (the registers of both models) one-to-one, then proves that for every matched pair and every output the combinational logic computing them is Boolean-equivalent. Given two sequential circuits with the same state encoding, combinational equivalence of their next-state and output functions is sufficient for their equivalence ^[13]. LEC is fast because Boolean equivalence of two similar cones

is a well-solved problem, and exhaustive because it never enumerates a trace. When the state points do not match, it becomes a mapping problem: it reports unmapped registers rather than a verdict, and it compares only the subset it could pair. In a restructuring example, the first hour goes on mapping; renaming or restructuring registers breaks LEC until their correspondence is supplied.

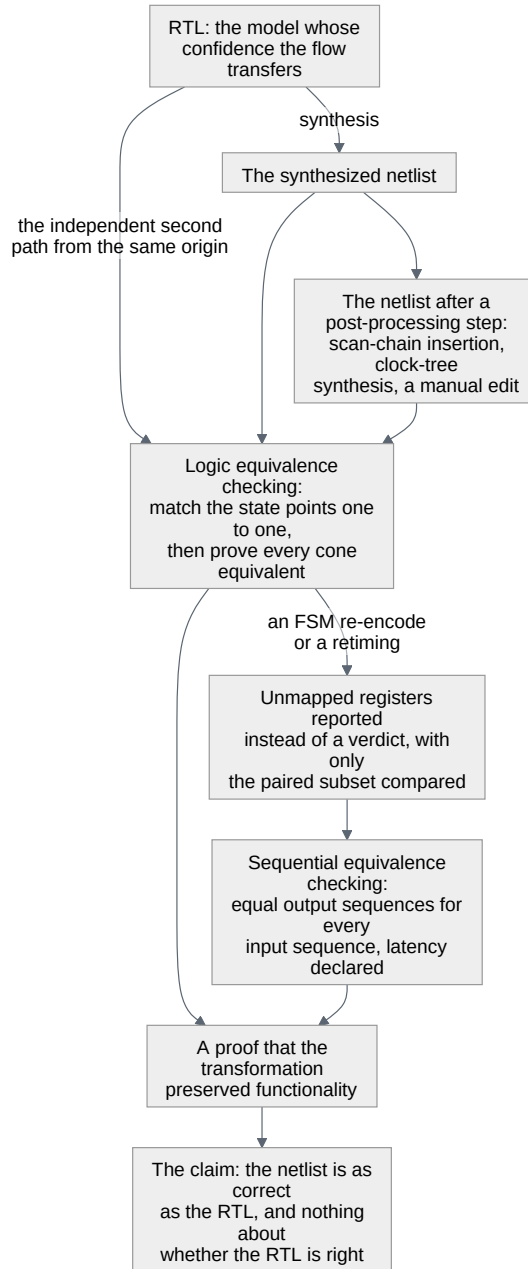


Figure 15.1 — Equivalence checking as a second path from the same origin: every transformation of the design owes one proof, a broken state-point mapping is what sends a step to the sequential flow, and the claim stays relative to the RTL.

Synthesis tools are large software systems depending on the correctness of algorithms and library information, and history shows such systems to be error-prone. An example is a design at Digital Equipment Corporation synthesized incorrectly because it used a synthetic arithmetic operator whose documentation stated correctness was not guaranteed above 48 bits; the synthesis tool could not read that docu-

mentation. Equivalence checking found it quickly; gate-level simulation would have found it with great difficulty, if at all [11].

An equivalence checker is also a large software system: one study randomly generated SystemC programs built by construction to be equivalent to trivial RTL, tested them against three major commercial C-to-RTL checkers, and uncovered sixteen distinct defects [14]. A C-to-RTL checker is a different class of tool from the RTL-to-netlist checkers above: it compares a model written in C or SystemC against the RTL. The paper withholds tool names under license, so the result concerns the class rather than a ranking.

Sequential equivalence checking (SEC) removes the matched-state-point assumption at greater computational cost. Two machines are equivalent when every input sequence produces the same output sequence; deciding that means computing the reachable states of a product machine that runs both designs in lock-step on the same inputs and reports whether their outputs agree [13]. Computing those reachable states is a model-checking problem, with model checking's costs, so SEC is aimed at bounded, well-chosen transformations rather than whole designs; it covers the cases that LEC cannot handle, rearchitected finite state machines and restructured data pipelines, and on those cases modern equivalence checkers handle sequential differences between two RTL models [11]. The following two cases concern clock gating and retiming.

Clock gating. The power tool inserts gates on the neural accelerator's 16-lane MAC datapath, and the gated design must remain sequentially equivalent to the ungated one, producing the same outputs at the same times while consuming less energy. If the gating condition is wrong by one cycle, the datapath holds a stale value that surfaces only on a specific weight-stream boundary. SEC against the pre-gating RTL answers it once, for all inputs.

Retiming. The radar front-end's FFT pipeline misses timing at the fast corner, and the fix moves a stage boundary: same arithmetic, one register deeper in one path. The function did not change and the register mapping entirely did, so LEC will not map the state points, and full functional re-verification of the DSP chain is days of simulation. SEC with a declared latency offset proves the two pipelines produce the same output sequence, and the change ships as a timing fix rather than a functional risk.

Table 15.2 compares both flows with the next section's datapath flow, with a row for each assumption that determines applicability.

Table 15.2 — The three equivalence flows side by side, one column each: what pair of models the flow compares, what it assumes before it starts, what it proves, and what it is blind to. Read the last row across all three columns — every one of them is blind to whether the reference itself is right, because equivalence checking is a relative argument that transfers your confidence onto another model exactly, and transfers your errors with the same fidelity.

	LEC	SEC	Datapath (§15.7)
Compares	RTL $\leftrightarrow$ netlist, netlist $\leftrightarrow$ netlist	RTL $\leftrightarrow$ RTL across a sequential change	Algorithmic reference $\leftrightarrow$ RTL

	LEC	SEC	Datapath (§15.7)
Assumes	Matched state points, same encoding	A declared input/output correspondence and latency relation	A declared operand and result mapping, bit-exact
Proves	Boolean equivalence of every cone	Equal output sequences for every input sequence	Equal result for every operand value
Blind to	Whether the reference itself is right	Whether the reference itself is right	Whether the reference itself is right

This limit applies to all three flows. Equivalence checking is a *relative* argument: it transfers confidence in one model onto another exactly and without loss, and it transfers errors with the same fidelity. It establishes that the netlist is as correct as the RTL, which is the flow’s required claim; it does not establish RTL correctness.

A second limit concerns the tool itself. Seven of the confirmed defects are cases where the tool reported an equivalence that did not hold, which the authors describe as the kind that could have let an incorrect design be signed off [14]. A proof engine answering *equivalent* wrongly produces no failure symptom: the flow reports success, the netlist ships, and the argument that the transformation preserved functionality was never made. Confidence transfer also depends on the tool performing it.

15.7 Datapath equivalence at word level

A design’s state space is the set of all configurations its state elements can take. For control logic, simulation’s coverage argument is a reasonable approximation: the interesting behavior lives in state sequences, the state space has structure, and a suitable coverage model samples it. For arithmetic, meaningful sampling by a coverage model is infeasible. The video codec’s inverse transform illustrates the input space. An 8×8 residual block is 64 coefficients; at 16 bits each, the input space of a *single* invocation is 2^{1024} values. No coverage model meaningfully samples that space, even with a billion random blocks. The failure mode is scattered across the input space: a rounding term applied at the wrong stage, a saturation clipping one bit late, an intermediate width one bit short; each is wrong for a sparse, structured subset of operands that random stimulus meets with probability near zero. The codec’s compression standard specifies the inverse transform as exact integer arithmetic precisely so encoder and decoder cannot drift, which makes a one-bit mismatch in one coefficient a conformance failure rather than a quality issue.

An arithmetic block’s specification is a mathematical relation between operands and result, which can be written directly. The formal verification of floating-point hardware after Chapter 1’s Pentium division episode established this pattern: a multiplier’s top-level specification is a formalization of the arithmetic standard, and it states a relation between the input operands and the result in terms of arithmetic on integers. The significand is the digit string of a floating-point number, the part that the exponent scales; the specification reads the significand and exponent vectors as integer values rather than as bit vectors. Proofs at that level are decomposed: over

120 properties covering all arithmetic operations of one industrial floating-point unit, each a property about a subunit (a loop invariant where the algorithm is iterative), with the irrelevant parts extracted away per property ^[13]. A loop invariant is a statement that holds on every iteration of the loop. Two requirements follow: word-level reasoning and decomposition, because a monolithic claim does not converge.

The industrial flow that grew from this compares an algorithmic reference (typically C or C++, bit-exact, often the code the architects used to develop the algorithm) against the RTL, with the operand and result correspondence declared by the user. In that description of the flow, the clause about the user declaring the operand and result correspondence is practitioner judgment: this book's corpus does not document it, and no source is cited for that clause.

The corpus explains why this needs a *separate* flow from §15.6's checkers. A general equivalence checker relates an RTL model to a netlist or another RTL model because they are structurally very similar; a model written at a genuinely different level of abstraction uses a completely different modeling approach and is very difficult to correlate mathematically with the RTL ^[11]. Datapath tools address this difficulty only by narrowing scope until the correlation is mechanical: a pure function, a declared operand mapping, a declared latency, no side effects, no protocol. A C model containing a state machine again requires hand-written properties.

Two published cases put that flow at opposite ends of its cost range. A reference model is an independent executable of the specification, taken as golden and run on the same operands as the design. Arm and Mentor Graphics proved floating-point operations of Arm CPU designs sequentially equivalent to C reference models, and published the effort per operation. Proof development took two days for FADD, and two months for each of FMUL, FDIV and FSQRT; the proof runtimes ranged from forty minutes to twenty hours. The figure that motivated the work was one hundred and fifty days of CPU time to simulate one operation with two half-precision operands exhaustively, and what the proofs bought carries the same scope: exhaustive simulation was eliminated for half-precision operations, not generally ^[15].

Intel's graphics organization reports this comparison used for early bug finding. The comparison started before the design and the C++ model were finished, as an exercise aimed at early bugs rather than at the complete coverage a sign-off claim requires, and it produced eighteen bugs in under a week, in the C++ model and the RTL together, with the split between them not stated ^[16]. A plan must distinguish the counterexample budget from the evidence needed for a proof.

Scenario. The radar front-end's constant-false-alarm-rate (CFAR) detector accumulates FFT magnitudes in a fixed-point format the algorithm team chose from a floating-point precision study. Datapath verification proves the RTL bit-exact against the team's fixed-point C model, on every operand. **Approach.** After shipment, the fixed-point C model should also be compared with the floating-point model over the operating envelope, with a declared error bound. **Why.** Bit-exactness completely establishes implementation of the fixed-point model, but provides no evidence of that model's adequacy. A detector that saturates on a strong near-

target loses a weak far one, and every stage of the equivalence argument passes while it happens. This outcome is Chapter 3’s common-mode error: complete agreement between RTL and reference leaves the correctness of both untested.

IN PRACTICE

In an adoption example, connectivity is absent from the first vplan; a wrong wire prompts an integration check that takes two days and finds nine more, so the next project plans a connectivity table. Adoption should start with apps whose machine-readable intent already serves another purpose, such as the register description generating firmware headers or the interrupt map generating documentation.

A plan row is the line of the verification plan that carries one feature: its attributes, a pass/fail criterion, a method and one closure metric, the evidence that decides when the feature counts as covered. The plan gains two of them in Chapter 5’s DMA format (Table 15.3):

Table 15.3 — The two verification-plan rows a connectivity app earns at SoC top level: the feature and its attributes, the pass/fail criterion, the method, and the metric that closes the row. There are two rather than one because “the connections exist” and “the forbidden ones do not” are two kinds of evidence with no single metric between them; note what the closure column has to spell out, since a connectivity property that ran out of budget is not a discharged row.

ID	Feature & attributes	Pass/fail criterion	Method	Closure metric
SOC-F01	Interrupt map: every row of the integration table — source, destination, enabling condition, latency — implemented as declared	Each declared connection holds for every legal value of its enabling condition	Connectivity app, SoC top level	Every generated property proven; none bounded, none waived
SOC-N01	No path from any key-ladder output to a bus-readable address, the debug data register, or a scan-observable point	No structural path exists in the cone	Connectivity app, no-connection mode	Every negative row proven; each exception dispositioned per Chapter 7

Two rows rather than one are needed, as when Chapter 5 split DMA-N01: declared connections and forbidden paths are two kinds of evidence, and no single metric evaluates both. *None bounded* means a property that exhausted its budget leaves the row open; the default report does not distinguish these two outcomes.

The following examples identify planning risks. In one example, the intent spreadsheet belongs to its last editor; the first formal run finds dozens of table errors rather than design errors, useful because software also uses the table, although day one looks unsuccessful. When physical design or synthesis owns equivalence checking, the verification lead may lack visibility of the latest netlist’s LEC result; the plan should provide it, since a failing LEC is a functional regression regardless of ownership. Every app produces items to disposition, so Chapter 7’s waiver discipline applies unchanged.

PITFALLS

1. **Unbudgeted app costs.** Symptom: connectivity gets one sprint but the intent table takes three weeks; or three hundred generated properties arrive without a review budget. Cure: the estimate must cover intent preparation and output disposition, excluding tool setup.
2. **Bounded results counted as complete.** Symptom: a run reports no violations, at a runtime limit, on a design that is too large. Cure: every generated property's proof status requires checking; "bounded, no counterexample" remains unfinished work.
3. **Unreviewed unreachability constraints.** Symptom: unreachable targets accompanied by a setup script unreviewed since project start. Cure: constraint review as exclusion evidence and over-constraint analysis must precede acceptance of a dead hole.
4. **Register-map intent left unchecked.** Symptom: a field at the wrong offset in silicon, with a green formal report behind it. Cure: one human review must compare description and architecture document, because the app proves conformance rather than correctness.
5. **LEC mapping failure after restructuring.** Symptom: an equivalence run that will not map state points after an FSM re-encode or a retiming, and a team debugging the mapping instead of the change. Cure: the sequential case requires SEC with a declared latency relation.
6. **Bit-exactness mistaken for correctness.** Symptom: a datapath proven identical to a reference that is itself wrong, or precision-inadequate. Cure: the reference requires separate verification against the standard or a higher-precision model with a stated error bound.

SUMMARY

- A formal app supplies the properties while the user supplies the design and a configuration, moving the specification work that made property checking a specialist activity into the tool ^[1, 2]. It disappears only where the intent already exists in machine-readable form; where it does not, curating that source is the flow's real cost. With design-derived intent, specification work remains: a mined property has no golden reference ^[10] and proves by construction, providing change detection rather than verification evidence.
- Connectivity checking proves declared connections; its equally important negative rows check forbidden paths more cheaply through structural analysis.
- Register checks discharge RTL against a register description exhaustively, over all values and orderings, and close nothing at all between the description and the architect's intent.
- Formal replaces a coverage waiver's argument with a result that inherits every environmental assumption, including those hidden in setup scripts ^[8].

- LEC assumes matched state points and proves Boolean equivalence cone by cone; SEC drops that assumption, becomes a model-checking problem, and earns its cost on retiming and clock gating [11, 13].
- Equivalence is a relative argument that transfers confidence and errors from one model to another exactly, without establishing whether the reference is correct.
- Simulation coverage cannot meaningfully sample arithmetic input spaces, which call for mathematical specifications and word-level proofs [13].

EXERCISES

Exercise 15.1 (calculation). §15.7 states the input space of a single invocation of the codec’s inverse transform as a power of two. Derive that exponent from the residual block geometry and the coefficient width the same section gives, showing both factors. Express the count of random blocks §15.7 mentions as the nearest power of two as well, and state what the comparison of the two exponents settles about a coverage argument for this block and what it leaves for the flow §15.7 prescribes instead.

Exercise 15.2 (calculation). §15.2 compares per-path connectivity proofs on the crossbar with a single proof over symbolic port indices, and gives a runtime for each. Compute the number of paths the exhaustive job proves, from the crossbar’s manager-port and subordinate-port counts, and compute the ratio of the two runtimes §15.2 reports. State the property of the symbolic argument that §15.2 offers in place of a runtime advantage, and name the qualification that pushes the exhaustive path count past the figure just computed.

Exercise 15.3 (judgement). A team presents a clean register-app report as evidence that the reference SoC’s register map matches the architecture document, and proposes to disable one field check that keeps failing on hardware the team has reviewed and believes correct. Decide what the app of §15.3 discharges and what it leaves open, naming the limit §15.3 identifies and the one review that stays a person’s work. Decide what makes that field check fail, using the two arguments of the canonical status register §15.3 discusses, and give the reading of the field’s access policy that a correct check has to carry.

Exercise 15.4 (judgement). A team asks the unreachability app for a verdict on the six excluded cells of the canonical DMA coverage model’s worst-case cross, and proposes to hand the app the generator’s row-size budget so that the verdict returns unreachable. Decide what §15.4 licenses, naming the premise the exclusion actually rests on and the limitation of §15.4 that the proposal would recreate. State what the design-level restatement of §15.4 returns instead and why, and give the disposition §15.4 leaves the exclusion with together with the event that ends it.

Exercise 15.5 (artifact). Read the record below, kept for one row of the reference SoC’s connectivity table and reported against the plan row `S0C-F01` of the chapter’s practice section. Name the blank field and give the wording that plan row requires in it. Decide whether the result as reported closes the row, with the reason the practice

section gives. Name the directive the generated property lacks, and state what a passing result cannot distinguish without it.

```
intent row      src = uart_err_irq, dst = evt_in[9],
                enable = cfg_irq_en[9], latency = 1 cycle
property
result          p_conn_uart_err (generated)
                no counterexample, bounded at depth 20
antecedent cover none generated
plan row        SOC-F01
closure metric
```

FURTHER READING

Within the corpus, [1] is the origin point for the automatic-formal-application category and its adoption argument, with current figures in [3, 4]. [5] surveys the app catalog and production hurdles, including symmetry and specimen-value techniques for constant-sized interconnect checks; [2] documents one app's generation, proof and bounded-result limitations on a real tape-out. For coverage-side formal, [8] defines coverage models and presents an over-constraint case study relevant before any first exclusion. On generated properties, [10] states the limitation that governs the RTL-mined variety and surveys where the alternatives take their intent from; Chapter 11 covers the specification-to-property direction. On equivalence, [11] gives the practitioner's framing and industrial anecdotes, while [12] places equivalence checking in the design flow and explains why it scaled first; [13] covers state-machine equivalence via product machines, combinational equivalence under matched state encodings, and the hierarchical decomposition that made RTL-versus-transistor checking practical on a commercial microprocessor. [7] specifies register descriptions as a single generated source. [14] examines the checkers themselves and reports twelve vendor-confirmed defects.

Outside the corpus: E. Seligman, T. Schubert and M. V. Achutha Kiran Kumar, *Formal Verification: An Essential Toolkit for Modern VLSI Design*, is the standard practitioner treatment of formal apps and their deployment. Equivalence-checking *algorithms* are the thin spot: relative to their industrial importance the published verification literature covers them lightly, and within the corpus [13] is the closest thing to a treatment.

REFERENCES

1. **P17** H. D. Foster, "Trends in Functional Verification: A 2014 Industry Study," Proc. 52nd Design Automation Conference (DAC '15), San Francisco, CA, 2015.
2. **D23** S. Zhao, S. Yan, and Y. Feng, "Practical Approach Using a Formal App to Detect X-Optimism-Related RTL Bugs," Proc. DVCon U.S., 2014.
3. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
4. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
5. **D13** J. Bromley and J. Sprott, "Formal Verification in the Real World," Proc. DVCon U.S., 2018.

6. **D12** N. Ahuja, M. Agarwal, and S. Jana, "Formal Assisted Fault Campaign for ISO26262 Certification," Proc. DVCon India, 2019.
7. **S2** Accellera Systems Initiative, "SystemRDL 2.0 Register Description Language," Accellera, January 2018.
8. **D11** X. Feng, X. Chen, and A. Muchandikar, "Coverage Models for Formal Verification," Proc. DVCon U.S., 2017.
9. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
10. **P11** W. Fang, M. Li, M. Li, Z. Yan, S. Liu, H. Zhang, and Z. Xie, "AssertLLM: Generating and Evaluating Hardware Verification Assertions from Design Specifications via Multi-LLMs," arXiv preprint arXiv:2402.00386v4, February 2026.
11. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
12. **B3** V. Bertacco, "Scalable Hardware Verification with Symbolic Simulation," Springer, 2006.
13. **P20** C. Kern and M. R. Greenstreet, "Formal Verification in Hardware Design: A Survey," ACM Transactions on Design Automation of Electronic Systems, vol. 4, no. 2, pp. 123-193, 1999.
14. **D46** M. Pardalos, A. F. Donaldson, E. Morini, L. Pozzi, and J. Wickerson, "Who checks the checkers? Automatically finding bugs in C-to-RTL formal equivalence checkers," Proc. DVCon Europe, Munich, 2024.
15. **D37** T. W. Pouarz and V. Agrawal, "Efficient and Exhaustive Floating Point Verification Using Sequential Equivalence Checking," Proc. DVCon U.S., 2017.
16. **D38** M. Achutha KiranKumar V, D. Puri, and S. Dharade, "Bringing DataPath Formal to Designers' Footsteps," Proc. DVCon India, 2019.

16. Hybrid Flows

Hybrid flows coordinate formal and simulation evidence within one verification campaign. The flagship host-cluster example covers a quarter of verification by two engineers who do not read each other's reports.

The formal report records nine properties on the directory's transition rules, proven unbounded on a model cut down to two caches and one address. The directory is the record of which caches hold each line. The same report records an assumption register of five rows, and four of those rows are discharged on a neighboring block. That assumption register is the reviewed table of every assumption, the party accountable for it and the evidence that discharges it. Among the nine is the invariant that at most one cache may hold a line Modified (see Chapter 14).

The simulation report records four hundred runs of random traffic across four application cores, a functional cross standing at 71% over three axes (the MESI state, the requester and the concurrent-snoop), a scoreboard that has been silent for three weeks, and the note that the remaining holes are all in the eviction column. The MESI state of a cached line is Modified, Exclusive, Shared or Invalid.

The lead's dashboard merges both databases into one number that is numerically valid but makes no defined claim: its denominator is unknown at review. The two engineers disagree on whether the single-writer proof fills the cross bins addressing the same condition. The single-writer proof is the proof that at most one cache may hold a line Modified, the condition those cross bins also address.

The problem is the flow: two engines run as two projects, joined only at the dashboard despite sharing one design, one plan and one sign-off. A unified campaign assigns work by problem structure rather than tool ownership, maintains one definition of legal traffic across both engines, and reports exactly what the evidence establishes.

CHAPTER MAP

§16.1 assigns each plan row by state-space shape, corner or volume behavior, witness cost and observability. §16.2 follows one interface constraint through stimulus, checker and formal assumption, with two detectors preventing those three from drifting. §16.3 explains the difficulty of combining simulation coverage and proofs, the interchange database's scope, and what a unified report may and may not claim. §16.4 gives the engine-selection procedure, together with two cases it has to settle: the deliberate use of both engines on one feature, and proofs that cost less but cannot discharge a plan row. §16.5 specifies the artifacts exchanged in both directions and their required accompanying records.

16.1 Dividing the work

Engine selection applies to individual plan rows. A plan row is one line of the verification plan: a feature, its attributes, a pass/fail criterion, a method and the evidence that closes it. Blocks are heterogeneous: Chapter 14 divided a video codec by function, while Chapter 15's apps discharge integration questions independently of how the blocks on either side were verified. Chapter 5 provides the method field for each row; the criteria below determine its value. Figure 16.1 places those criteria between the row and the two engines, with the artifacts that later cross between them.

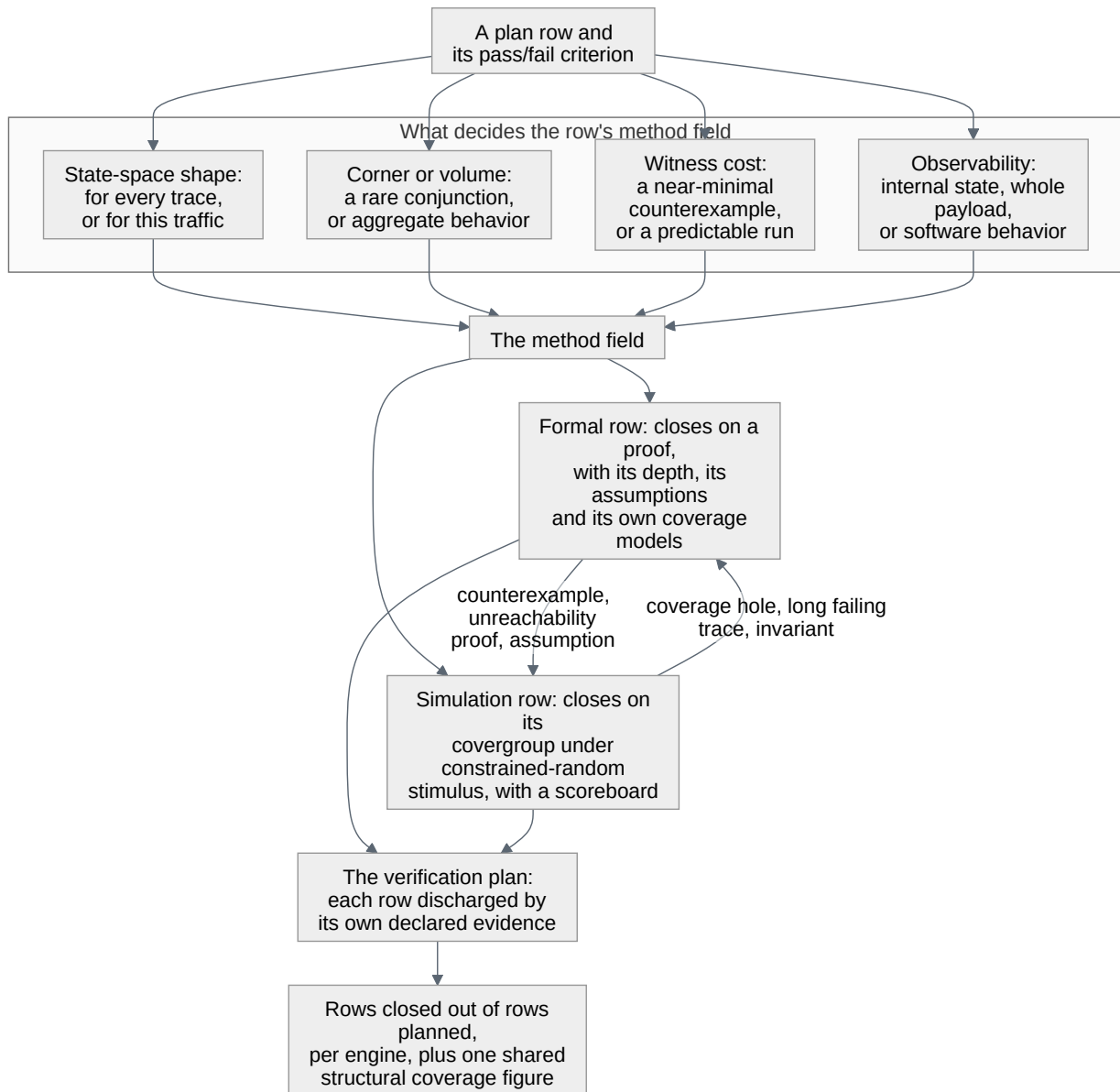


Figure 16.1 — Engine assignment belongs to the plan row rather than to the block: four criteria set the row's method field, each engine closes its own rows on its own evidence, and the plan, rather than a merged database, is where the two report together.

State-space shape. Chapter 14 states in one line that formal excels at rules over bounded state and fails at data transformations. The row's pass/fail criterion ex-

presses the distinction through its quantifiers. A criterion that begins *for every* (every burst stays inside one 4 KB region, every grant follows a request, at most one cache holds this line Modified) is a claim over all traces, and an engine that samples can only ever provide evidence for it. A criterion that begins *for this traffic* (the destination buffer matches the source under a mixed read/write profile, the encoder's output decodes bit-exactly on this stream) is a claim about a transformation, and its oracle is another executable (Chapter 8). The first kind belongs to formal, with simulation as corroboration; the second belongs to simulation because formal requires the reference model to be rewritten in temporal logic, the notation for what must hold across cycles rather than at one instant.

Corner or volume. A *corner* is a conjunction of individually rare conditions. Simulation cost depends on the product of probabilities, regardless of seed count; formal searches independently of the conjunction's rarity. One published deployment records a case in which an arbiter starved a requester only under a conjunction of three input conditions that the authors record as next to impossible to construct in simulation: the first condition is that the output FIFO had filled, so that both ports were unavailable; *and* the second is that all three clients then asserted request continuously; *and* the third is that the two ports subsequently became available on alternate cycles ^[1]. A *volume* criterion concerns aggregate behavior over many transactions: sustained throughput, queue occupancy under load, fairness across a frame or energy per operation. Such a statistical criterion cannot be discharged by proof. Formal verification generalizes and abstracts design behavior, which is why it complements a technique that can only ever stimulate a small portion of the design and may therefore miss errors surfacing under rare conditions ^[2]; it is also why it cannot determine how often something happens.

Scenario (the GPU). Two rows of the tile-based mobile GPU's plan, on the same interface between a shader cluster and unified memory. Row one: *no shader cluster may observe a write reordered past a barrier it issued*. Row two: *at 90% memory-bus utilization, the 99th-percentile shader stall waiting on a tile fetch stays under N cycles*. **Approach.** Row one goes to formal using Chapter 14's two-symbolic-address memory model: stored values do not affect the rule, so memory can be uninterpreted; two addresses are the minimum needed to express ordering. Row two uses simulation with a traffic profile reproducing a real frame and closes on a distribution rather than a bin. **Why.** The first requires a specific interleaving of a barrier, a write and another cluster's read; under random stimulus its probability is the product of three small numbers. The second is a volume: an all-traces proof instead returns an existing worst-case stall exceeding N, occurring once in a billion frames in this example, which does not answer the row's question. Neither engine could discharge the other row on this interface.

Witness cost in both directions. A counterexample arrives near-minimal and carries no irrelevant history (Chapter 14); a simulation failure arrives at the end of a run with everything that preceded it attached, incurring Chapter 8's detection distance at debug time. For a result of *no violation*, simulation's cost is predictable, whereas formal's is not. A few minutes of simulation gives a good estimate of clock

cycles per second for the rest of the run, whereas if an assertion is slow to prove it is almost impossible to predict how much longer it will take [3]. A lead may put a simulation row on the critical path and estimate it; a formal row there is an unbounded commitment unless the plan says in advance what happens when it does not converge; accordingly, Chapter 14's convergence ladder ends with *stop, and hand the row to simulation* as a planned outcome. The convergence ladder is the ordered list of moves that chapter prescribes for a property that has not converged.

Where the criterion is observable. Criteria statable over internal state, such as a queue's occupancy against its pointers or a one-hot encoding, are cheap in both engines, which is why Chapter 11's redundant-state invariants also serve as formal lemmas. A lemma is a fact about internal state, proven on its own and then given to a harder proof as an assumption that needs no further discharge. Criteria over a whole transaction's payload are expressible for a proof engine, which rarely finds them tractable. Criteria over software behavior belong to neither engine at this level (Chapter 18).

These four properties change engine assignments across generations. On the reference SoC's 5×4 crossbar, the ordering and arbitration rules are small enough that end-to-end proofs close and simulation's job is data transport at volume. On the flagship's 4×4 mesh, the same class of obligation (end-to-end ordering between two endpoints) spans sixteen routers, virtual channels and XY routing, and does not close as one proof; deadlock-freedom goes to formal on an abstracted model, ordering goes to formal on a symbolic endpoint pair, and everything about congestion, latency distribution and virtual-channel occupancy goes to simulation and emulation (Chapter 17). Virtual channels are the separate queues that share one physical link between two routers, and XY routing moves a packet along the mesh's X dimension first and along Y second.

16.2 Assumption and constraint duality, as a flow

Chapter 9 named the duality: one constraint generates legal traffic on a block's input, checks legality on the neighbor's output, and is assumed in formal. Chapter 11 wrote it as an assertion, Chapter 14 as an assumption. Operationally, stimulus legality resides in a `constraint` block inside a randomizable class, whereas the checker and assumption are temporal properties. The constraint and the property are not one statement in two syntaxes: the constraint is a statement about a generated object, while the property is a statement about a signal history. Because of that difference, no tool diffs the constraint against the property, and no compiler flags their divergence. Divergence can remain undetected until sign-off or indefinitely.

The drift is asymmetric

Drift has two directions, only one directly visible.

The generator is narrower than the assumption. The formal environment permits traffic the tests never send. The proof soundly covers a superset of the simulated traffic. The unsent traffic is visible as a simulation coverage hole, handled by Chapter 6’s classification.

The assumption is stronger than the generator’s legal set. The proof holds over a subset of the traffic the tests actually send: so simulation is exercising behavior that the proof never covered, while both reports are green. Neither side is empty, and no coverage hole, failure or artifact records the gap. Chapter 14’s silent over-constraint arises here in a particular way: the assumption was accurate for an *older* environment and stimulus has since outgrown it, rather than having been too strong for the design’s environment from the start.

Two detectors

Because the languages differ, the constraint cannot be written once. One of its three forms must serve as the source, and a mechanical test then checks the other two against that source. The source is the *assumption*, whose failure is silent.

Detector one: assumptions compiled as simulation assertions. A published sign-off methodology includes this among the techniques for establishing that a formal environment has no unintentional over-constraints: instantiating the constraints in simulation, alongside cross-proof with neighboring blocks ^[4]. A cross-proof is a proof on the neighboring block, where the condition this block assumes becomes a property the neighbor has to satisfy. That technique uses Chapter 14’s `ASSUME_MODE` parameter in reverse. In formal, the parameterized checker constrains an input. In the block’s own simulation, the identical text is *asserted*, and because the signals it watches are driven by the testbench, that instance is checking the generator rather than the design.

The sensor hub’s wake path provides a minimal instance. The always-on island raises `wake_req` and the DSP domain answers with `wake_ack`, both sampled on the 32 kHz clock (Chapter 14 proved the island’s wake latency on this clock, avoiding the proof depth required on the fast one):

```
// Sensor hub wake path: the always-on island requests, the DSP domain answers.
module wake_handshake_checker #(parameter bit ASSUME_MODE = 1'b0) (
    input logic clk_slow, rst_n,
    input logic wake_req, wake_ack
);
    // The requester may not withdraw a request that has not been acknowledged.
    property p_req_holds_until_ack;
        @(posedge clk_slow) disable iff (!rst_n)
            (wake_req && !wake_ack) | => wake_req;
    endproperty

    // Has the environment ever come near the rule's edge?
    c_req_waited : cover property (@(posedge clk_slow) disable iff (!rst_n)
        wake_req && !wake_ack);

    if (ASSUME_MODE) begin : g_env
        m_req_holds_until_ack : assume property (p_req_holds_until_ack);
    end else begin : g_chk
```

```

    a_req_holds_until_ack : assert property (p_req_holds_until_ack);
end
endmodule

```

The property requires only that an unacknowledged request survive into the next cycle; it specifies nothing after `wake_ack` rises and never constrains `wake_ack`. This complete rule suffices for both uses. When the DSP's wake controller is proved, `wake_req` is an input and the instance is built with `ASSUME_MODE = 1`. When the island is proved, `wake_req` is an output and the same text is asserted. And in the DSP's block-level simulation the assert-mode instance is bound into the DSP's scope, where `wake_req` comes from the testbench:

```

bind dsp_wake_ctrl wake_handshake_checker #(.ASSUME_MODE(1'b0)) u_wake_chk (
    .clk_slow(clk_32k), .rst_n(rst_n_32k),
    .wake_req(wake_req), .wake_ack(wake_ack)
);

```

If a sequence ever drops an unacknowledged request, that assertion fires in simulation and the message names the assumption that would have been silently violated in formal. A nightly run thus exposes the generator's drift outside the proof environment before sign-off.

Detector two: coverage of the assumption's boundary. Detector one catches a generator going *outside* the assumption, but cannot detect stimulus staying so far inside that the assumption remains untested. `c_req_waited` checks whether any run produced an unacknowledged request. If it reads zero, `a_req_holds_until_ack` has never evaluated non-vacuously, and the claim that simulation corroborates the formal environment is empty. An assertion evaluates vacuously when its antecedent never becomes true, so nothing is checked. Detector two takes Chapter 11's rule, that every implication ships with a cover on its antecedent, and applies it to a third artifact. Applied to an assumption, the rule reads: **every assumption ships with a cover on the condition it restricts, and an assumption whose boundary is never approached in simulation is untested.** Chapter 11's caveat travels with it: where the restricted condition is ordinary traffic, a bare cover proves nothing, and the rule is discharged instead by a witness that implies the condition and isolates the interesting case.

Together the two detectors monitor Chapter 14's reviewed assumption register. Each row of the register gains a column: the run in which its assertion form last passed, and the run in which its boundary cover was last hit.

Scenario (the crypto engine). Chapter 14 identified for the inline AES-GCM engine a dangerous assumption (that key material arrives only after authentication completes) and the verdict that the obligation belonged on the boot ROM's proof rather than on the engine's. **Approach.** The transfer requires three bookkeeping steps. The assumption becomes an assertion in the boot ROM's own formal run, where `key_load` is an output and the claim is discharged. The same text, in assert mode, is bound into the SoC-level simulation environment, so every firmware-driv-

en boot sequence in the nightly regression re-checks it against real traffic. Its boundary cover, a key load attempted at all, belongs on the same row: an obligation unapproached by tests provides no evidence. The engine's assumption register row keeps the text and gains a discharge pointer. **Why.** Proving this condition on the engine's driver and re-checking it in system simulation substantiates the engine's assumption. An artifact from a formal convergence problem thus becomes a permanent member of a simulation regression tier (Chapter 12).

16.3 Coverage unification

Combining the two engines' different result types can overstate the evidence.

A simulation bin counts occurrences of a situation in the executed runs. A proof holds a claim: *no legal trace violates this rule*. These are two different quantities, concerning the testbench and the design respectively, as Chapter 14 distinguishes. Their average has a denominator combining two incompatible units.

The interchange standard's data model

The Accellera Unified Coverage Interoperability Standard (UCIS), Version 1.0, 2012, defines a single repository holding coverage data from all verification processes (simulation, static checking, functional formal verification, sequential equivalence checking, emulation) together with a standard access layer, so that data produced by many producers is semantically interpretable by many consumers. Its target flow has two stages: the simulator or the formal tool collects primary data in memory and exports it on completion, and merge applications later combine those primary databases, recording their construction history in the database itself [5].

Formal-run data makes Chapter 14's three qualifiers machine-readable: the model a proof ran on, the assumptions it ran under, and the depth it reached if it was bounded. An assertion's result is one of a small enumeration: no result, failure, proof, vacuous, inconclusive, assumption, or a merge conflict (UCIS 1.0 §8.19.3). One number travels with that result, a bounded proof's depth or a counterexample's length, recorded together with the name of the clock it is measured in; that number is called the *radius* (UCIS 1.0 §8.19.4–§8.19.5). Alongside the result, the database records the assumptions used, the restrictions, the initialization sequence, initial-value abstractions and any cutpoints inserted to achieve the proof [5]. A cutpoint is a signal cut from its fan-in so that a proof may give it any value. The schema carries the same qualifiers that §14.6's sign-off package lists, rather than leaving them to a report template, so that the proof result can be interpreted with its conditions.

Three kinds of merge are distinguished (UCIS 1.0 §2.1.2), and only two of them, the temporal and the spatial, are mechanical. A *temporal* merge combines runs of the same process on the same design. A *spatial* merge combines coverage from different parts of a design. A *heterogeneous* merge combines data from different verification processes; the standard cautions that items are more often semantically different than equivalent, so they should not be automatically merged, and an analysis tool im-

plementing a merge should create a *new* item semantically tagged as merged data. The merging algorithm itself is left unspecified (UCIS 1.0 §4.3.1) [5]. A tool may build a combined view, but may not present it as a primary measurement.

A metric is the rule assigning observed values to bins, and different metrics over the same data produce very different percentages: an eight-bit variable that took the values 17 and 96 scores $2/256$ under a metric with one bin per value and $2/8$ under an eight-bin auto-binning metric. Collection loses information that cannot be recovered afterwards [5]. Percentages depend on models rather than designs, as Chapter 6 explains for a single simulation model; combining two engines with different partitions compounds the problem. Table 16.1 separates the meanings that a combined report must preserve.

What a unified report may and may not claim

Table 16.1 — What a cross-engine coverage report may and may not claim, sorted into three dispositions: quantities that genuinely add because both engines report hits against the same structural denominator, results that travel only as tagged items carrying their qualifiers, and pairs that must never be summed at all. The left column is where a hybrid flow earns its keep; the right column is where confidence gets manufactured, because writing a proof’s verdict into a functional bin looks like progress and blinds two metrics at once.

Merges in one unit	Travels as a tagged item, with its qualifiers	Must never be summed
Code coverage: lines, branches and conditions — formal reachability and simulation both report hits against the same structural denominator	An assertion’s result plus its radius, assumptions, abstractions and cutpoints	Functional coverage bins against proofs — a hit count and a proof answer different questions
Assertion coverage: the same property’s pass and vacuity counts from simulation runs	An unreachability verdict on a coverage target, with the constraint set that produced it	Two percentages whose metrics partition the space differently
Plan-row status: each row closed by <i>its</i> declared metric, whichever engine produced it	A merged view, provided it is recorded as a derived item and labelled as one	A proof’s “100%” into a functional bin the tests never filled

The left column combines structural coverage from both engines. Formal reachability reports line, branch and condition coverage in simulation’s units, so the results add. In the published sign-off case study of a multicast crossbar parameterized to eight clients and eight targets, coverage analysis run from the completed formal testbench alone reported two results: all 288 lines and all 88 conditions were covered, and the deepest cover point was at depth 5, comfortably inside that project’s derived required proof depth, a margin the authors take as evidence that the depth is not optimistic [4]. The required proof depth is the depth that a bounded result must reach, derived from the block’s latency and corner cases with a margin, and recomputable by a reviewer. A second deployment mandated line, statement and branch coverage at the end of every formal exercise and found real holes doing it [1]. Both use structural coverage to check that properties and constraints reach the logic, without treating it as functional evidence. Structural coverage of this kind differs from Chapter 14’s four formal coverage models because it uses simulation’s

units. The four models also quantify gaps in what the properties cover, and one of them measures the cone of influence, the logic that a proof has to consider in order to decide a property. Oracle Labs ran cone-of-influence coverage across every register of its own design and found 18%, or 1,375 registers, outside every assertion's cone of influence [6]. No proof had failed or become invalid, yet a fifth of the design's state lay outside the property set's scope and needed assignment to the other engine.

Writing a proof verdict into a functional bin destroys two measurements while appearing to improve coverage. The bin records stimulus activity, and overwriting it prevents two later checks: Chapter 7's coverage position can no longer corroborate a flat bug curve, and Chapter 12's coverage differential can no longer fail a commit that reduces the suite's scope. A flat bug curve is a plot of bug discoveries over time that has stopped rising, and the coverage differential is the bin-level comparison of two regressions. The resulting number adds no coverage evidence and disables both metrics.

The reconciliation

The verification plan provides the unification layer; a better database merge algorithm cannot replace it. Chapter 5's one-row-one-metric rule specifies that a row is discharged by its own declared evidence, and a row discharged by a proof closes on the proof, with its depth, its assumption list and its own formal coverage evidence attached (§14.6), while a row discharged by a covergroup closes on the covergroup. A proven row needs no functional **bin** because the proof discharges what the bin sampled, but still requires the formal coverage models; a proof alone never closes a row. The two kinds of closure require no averaging because they are not fractions of the same whole. The dashboard reports rows closed out of rows planned, per engine and in total, with an explicit denominator. The consequences of unlinked plans are documented: the Verification Academy's *Coverage Cookbook* describes projects whose plan is written early and then left with its rows unlinked to coverage elements, so the regression runs with the coverage feedback switched off and the tape-out decision rests on a judgment of sufficiency in place of coverage data [7].

The coverage database retains two functions it performs better than the plan, the first being structural coverage. The second mechanism means a coverage item can be flagged as *formally unreachable* with respect to a particular test; the classification distinguishes a don't-care from a *predicted-unreachable* point so that a warning is raised if a predicted-unreachable point is later covered [5]. Chapter 15's unreachability proof justifies a waiver; the flag makes that justification continuously checkable. If a later RTL change, relaxed constraint or new test reaches the excluded point, the regression warns rather than retaining a file exclusion valid only for the old design.

lowRISC's OpenTitan project publishes a design-verification methodology requiring an annotation on every exclusion that explains the reasoning for the waiver, and a category tag drawn from a fixed set: unreachable under design constraints, a non-RTL simulation construct, an unsupported feature, an item covered in another

bench, or one deemed low risk, with the last three requiring a written explanation and a designer signing the exclusion off in a pull-request review [8].

That vocabulary governs the waiver. A second published classification precedes that vocabulary by one step, addressing a different question; the two classifications apply in sequence. The Verification Academy's *Coverage Cookbook* sorts why an uncovered item exists at all: input stimulus that was never generated, a bug in the design or the testbench that blocks the stimulus from reaching the code, or code unused in a given configuration and expected to be unreachable in normal operation. Only the third class is directed toward exclusion by the Cookbook, which observes that formal tools can be used to identify unreachable code and generate the exclusion files automatically [7]. The waiver vocabulary starts where that sorting ends, on the third class alone: a hole in the first two owes a test or a fix.

Scenario (the flagship's host cluster). The lead rebuilds the opening report row by row. The directory transition rows close on proofs, each recorded as unbounded on the two-cache one-address abstraction. The rows that abstraction deleted are simulation rows: the two-address deadlock; conflict eviction and capacity writeback; the third sharer and the sharer-vector overflow. A two-address deadlock is one that needs two addresses at once, a capacity writeback is the writeback a replacement triggers rather than a snoop, and a sharer-vector overflow is more sharers than the directory's vector can encode. Conflict eviction and capacity writeback are exactly the eviction column where the opening report's functional cross was empty. The cross is then re-scoped to the rows simulation owns (a reviewed coverage-model change with its justification recorded (Chapter 6)) and the structural coverage from both engines merges into one line/branch figure. **Approach.** The report gives two closure numbers and one merged structural number, never one blended percentage. **Why.** The issue was the unexplained origin of the missing 29%, rather than the 71% coverage value itself. Assigning each hole to a row and each row to an engine turns the report into a work list.

16.4 Choosing an engine, row by row

In a plan review, a lead can apply the following procedure per row, in the order the criteria break ties.

1. **Quantification over traces or a traffic profile.** (§16.1.) Profile criteria belong to simulation even when expressible as elegant properties.
2. **Corner or volume behavior.** (§16.1.) Rare conjunctions belong to formal even in large blocks, because the abstraction needed for convergence usually preserves the conjunction.
3. **The environment's ability to construct a witness.** If the block-level testbench cannot produce the scenario at all (Chapter 6's bin that needs corrupted authentication tags, or a coverage cross that needs contention no block bench can generate) stimulus tuning cannot suffice; the row needs a testbench capability the block bench does not have (Chapter 6), formal, or a higher verification level.

4. **Required model content.** If the property needs the whole datapath, the memory and the firmware that programs them, the abstraction cost of §14.4 will be large and the deletion list long. That cost needs estimating before commitment.
5. **Sign-off ownership.** Sometimes the authority lies outside the project.
6. **Non-convergence fallback.** The plan review should define it before week nine.

Item five concerns sign-off authority rather than completeness.

Scenario (the USB controller). The dual-role USB 3.x controller’s link-training and low-power rows, covering U0 through U3 transitions, suit formal: a bounded-state machine with published transition rules (*Universal Serial Bus 3.2 Specification*, Revision 1.1, sections 7.5.6 to 7.5.9) ^[9]. U0 through U3 are the link power states the USB specification defines, from the fully active link down to its deepest low-power state. In this example, proof finds real bugs at low cost and with short runtimes. **Approach.** Proof should run without closing these rows. Chapter 5 sends those rows to the compliance suite defined by the USB compliance methodology, whose item list is their closure metric; the proof enters the package as corroborating evidence and as a bug-finding instrument available months before the suite can run. A closure metric is the evidence that a plan row names as what closes it, here the compliance suite’s item list. **Why.** The cheaper proof is stronger evidence than the suite for the rules it covers, but remains insufficient for sign-off. On compliance rows, the authority to declare completion is external to the project. Substituting the project’s own evidence therefore changes the plan, beyond choosing a method.

Complementary methods on the same feature

Assigning two methods addresses the engines’ different failure modes.

One industrial deployment, Intel’s live graphics project, began formal *after* a major dynamic-verification milestone, with stable, running simulation environments, and reported 82 interesting bugs over twelve weeks. That dynamic-verification milestone is a phase-gating event, and what it requires is a demonstration that can be checked rather than an estimate of the project’s state. In the authors’ own taxonomy, the easy finds were things like a state machine attempting two arcs at once, which in simulation would have surfaced as an X-propagation problem and been hard to trace; the medium ones were counter overflows and the arbiter starvation of §16.1; the hard ones needed a deep internal counter value coincident with a specific input sequence. The authors note that even the easiest of these would have been difficult for the running simulation environment to find ^[1]. Chapter 14’s opening reports the corresponding case on this book’s own crossbar: six weeks of green nightly regression, the hazard’s precondition cover reading zero, and an eleven-cycle counterexample overnight.

Deliberate double assignment follows Chapter 5’s risk-decision format: two rows, two metrics. For the DMA’s splitting of transfers into legal bursts, the coverage row closes on a covergroup under constrained-random stimulus with a scoreboard, while

the separate proof row closes on the boundary property proven on the backend standalone. The splitting belongs to the burst splitter, the DMA stage that splits each transfer into bursts that the AXI address rules allow. The boundary that the proof row names is the 4 KB boundary, the address line that AXI forbids a burst to cross. The two rows require distinct evidence; neither substitutes for the other.

Coherence at two scales

The standalone coherent hub and flagship host cluster share a protocol family at two scales, with engine assignments changing by scale rather than technique.

On the hub, block-level, the two-caches-one-address abstraction covers a large fraction of the coherence obligations worth proving (the directory's transition rules, the races between two requesters, the ordering of invalidations against data responses, and starvation on a contended line) and Chapter 14 enumerated what it deletes. Simulation's job at that level is small: the deleted classes, plus data integrity through the datapath.

In the flagship, the same proofs run on the same abstraction, and everything they delete gets harder rather than easier. Conflict eviction now involves a four-slice L2 and a real replacement policy; the third and fourth sharers are real cores; and a new class appears that the hub never had. The new class is a DMA transfer that the burst splitter has split wrongly, whose second half writes a line that a core holds Modified. The class is the flagship descendant of this book's canonical DMA bug, invisible until the writeback and reachable only under a running software workload. The canonical bug is the book's recurring defect: a transfer split wrongly at the 4 KB boundary when the step between rows is negative, and only while the DMA's other channel is active. The proof does not lose value at scale; the simulation half gains share. Between the hub and flagship, formal's rows remain roughly stable while simulation's multiply.

16.5 The handoff

Hybrid flows are made of artifacts crossing between two engineers. Each artifact needs its prescribed form and accompanying records for the handoff to remain traceable.

Formal to simulation.

A counterexample becomes a directed test. The trace is near-minimal, so the test derived from it is short, and it is exactly the stimulus a random generator was demonstrably not producing. That test is a directed test, a test whose content and order are written out by hand so that every run does the same thing. It joins a regression tier (Chapter 12) so the fix stays fixed. What must travel with it: the property it violated, so the test's failure condition is the assertion rather than a hand-written expectation, and a cover on the scenario, so a later constraint change that stops reaching it is visible. A GPU memory-ordering counterexample contains five cycles of barrier, write and cross-cluster read that frame-driven traffic never aligned. In this example, a dir-

ected test takes an afternoon and remains in regression indefinitely; debugging alone takes a week and leaves no retained test.

An unreachability proof becomes a waiver's justification. Chapter 15 established the verdict and its main caveat, that reachability is computed relative to the constraints. What §16.3 adds is where the verdict then lives: not only in the waiver's reason field but as a flag on the coverage item, with the classification that raises a warning if the point is ever covered later. The constraint set must accompany it as evidence, reviewed as seriously as the waiver it replaces.

The OpenHW Group publishes, for a released RISC-V core, a code-coverage waiver list whose summary sheet gives each waiver category a written reason and a column for the issue that tracks it ^[10], and an uncovered-coverage table whose columns are the cause, the action to waive or to add a test, the future action on the RTL, and the risk of leaving the item uncovered on a four-level scale ^[11].

An assumption becomes a checker. This §16.2 mechanism runs every night, and among the three transfers from formal to simulation listed above (the counterexample, the unreachability proof and the assumption) it is the one that provides continuous checking. It must carry the discharged assumption-register row and the boundary cover.

Simulation to formal.

A coverage hole becomes a reachability question. A hole resisting three targeted tests is a candidate. A design-level cover property lets formal answer; Chapter 15 shows this restatement and warns that a design-level engine cannot see a monitor-side functional axis, an axis whose value the testbench's monitor computes rather than one the design's own signals carry. Two of the three outcomes are decisive: a witness gives the stimulus engineer a recipe, provided the generator's own constraints permit that trace; an unreachable verdict justifies exclusion by proof. An inconclusive result leaves the hole unresolved, information needed before spending a week on it.

A long failing trace becomes a property. When a simulation failure is reproducible but the trace is thousands of cycles, formulating the violated rule as a property and running a bounded proof on the block often returns a counterexample an order of magnitude shorter. What must travel with it: the seed and the run manifest of the original failure, because the short trace is a different execution and someone must confirm it is the same bug. A run manifest is the record of everything needed to repeat one run, from its seed to the exact tool build.

A redundant-state invariant becomes a lemma. Chapter 11's invariants are the facts a proof engine has to be given if it is to stop generating impossible starting states (Chapter 14), and their existing, accepted formulations can be handed over without being rewritten.

IN PRACTICE

The two flows' differing cadences need coordination. Chapter 14 recommends a separate formal regression tier; hybrid-row evidence is only as fresh as the slower of the two flows contributing to it. One published deployment kept its proofs and scripts in a repository and re-ran them on a fixed interval, reporting the trend to the owner [1].

The assumption register needs one owner, including when the two engineers report to different leads. For a purchased dashboard capable of printing a blended percentage, the project should define its headline number before allowing the tool's default to determine it. Available structural coverage from formal runs should be explicitly requested and enabled. Every handoff in §16.5 requires someone who can read both report types; exactly one such person creates a project bottleneck.

PITFALLS

1. **Assigning engines to blocks instead of rows.** Symptom: "the crossbar is a formal block" and its data-transport rows have no owner. Cure: each row should specify its method, allowing both engines within a block.
2. **A proof written into a functional bin.** Symptom: coverage jumps when a proof closes. Cure: the proof should close the plan row while the bin continues measuring stimulus, preserving the differential and fresh-seed metrics. The fresh-seed metrics count failures per unit of newly seeded stimulus.
3. **A blended percentage with an undefined denominator.** Symptom: the review cannot establish what fraction the number represents. Cure: the report should give rows closed per engine plus one shared structural figure.
4. **An assumption unapproached by tests.** Symptom: its assertion form passes in every simulation, while its boundary cover reads zero. Cure: the empty cover should count as a failure, because a rule the environment never approaches is untested in both engines.
5. **A counterexample debugged and not captured.** Symptom: the bug is fixed, the trace is in a chat thread, and no regression member exists. Cure: every counterexample leading to an RTL change should leave a directed test and cover in regression.
6. **A stale unreachability exclusion.** Symptom: a coverage exclusion justified by a proof against an RTL revision three months old. Cure: a predicted-unreachable database flag should warn on later hits, and the proof should rerun at the consuming regression's cadence.

SUMMARY

- Engine assignment belongs to each plan row, according to criterion quantification, corner or volume behavior, witness cost and observability.
- Simulation's cost when nothing fails is predictable; formal's is not [3]. The plan should account for this scheduling difference before week nine.

- One interface rule has three forms; an assumption stronger than the generator's legal set creates silent drift, leaving both reports green without an artifact recording the gap.
- Two detectors address the gap: assumptions compiled as simulation assertions [4], and a cover on each assumption's boundary.
- A hit count and a proof answer different questions, and the interchange standard says so in its own terms (UCIS 1.0 §2.1.2): heterogeneous data should not be automatically merged, and a merged result should be recorded as a new, tagged item [5].
- Unification belongs in the verification plan: rows closed out of rows planned has a defined denominator, whereas a blended percentage does not. The database retains two cross-engine jobs: structural coverage in one unit, and the formally-unreachable flag warning when a supposedly unreachable point is covered [5].
- A deliberate double assignment has value: formal deployed on stable, already-simulated designs found bugs the running environment could not reach [1].

EXERCISES

Exercise 16.1 (judgement). The two engineers of the chapter's opening disagree over whether the single-writer proof fills the functional cross bins that address the same condition. Decide what the proof entitles the report to do with those bins, and name the disposition of §16.3's three-way sorting that the pairing falls under. State what the plan row closes on instead, and which two further measurements the report loses if the bins are marked filled.

Exercise 16.2 (calculation). §16.3 scores an eight-bit variable that took the values 17 and 96 under two metrics, one with a bin per value and one with eight automatic bins, reporting 2/256 and 2/8. Compute both scores as percentages and the factor between them. Identify the property of the two metrics that produces that factor, and state what the pair establishes about a percentage combined from two engines whose metrics partition the space differently.

Exercise 16.3 (artifact). Read the assumption-register row below, kept in the form §16.2 asks of a reviewed register. Decide what the row establishes about the assumption and what it leaves unestablished. Name the detector of §16.2 that the row's last two fields belong to, and the pitfall of the chapter's list that the row's state matches.

Assumption:	the requester may not withdraw a wake request that has not been acknowledged
Assumed in:	the DSP wake controller's proof, request an input
Assertion form bound in:	the DSP's block-level simulation, where the request is driven by the testbench
Assertion last passed:	last night's regression, and every regression before it
Boundary cover:	a wake request standing while the acknowledge is low
Boundary cover last hit:	(never)

Exercise 16.4 (calculation). The formal report of the chapter’s opening carries an assumption register of five rows, four of them discharged on a neighboring block. Compute how many rows remain undischarged, and how many run dates the register holds once every row carries both of the records §16.2 names. State which of §16.2’s two drift directions leaves no artifact of the gap behind, and what §16.5 requires to travel with a row discharged on a neighboring block.

Exercise 16.5 (artifact). Read the plan row below, brought to a review of the flagship’s mesh NoC, the on-chip packet network that takes the place of a bus once too many blocks share one path. Decide which criterion of §16.1 the row’s method contradicts, and state what a proof can and cannot return for a criterion of that shape. Name the item of §16.4’s ordered procedure that the row’s empty field belongs to, and the point in the schedule by which §16.4 asks for it.

Row:	NOC-P07
Criterion:	under sustained traffic, the endpoint-to-endpoint latency distribution stays inside the stated envelope
Method:	formal, end-to-end property on a symbolic endpoint pair
Closure metric:	proof, unbounded
Non-convergence fallback:	(empty)

Exercise 16.6 (judgement). A coverage hole that resisted three targeted tests has been proved unreachable, and the team proposes to close it with a waiver whose reason field cites the proof. Decide what §16.5 requires to accompany the verdict, and where §16.3 puts the verdict besides that reason field. State what keeps the record valid as the design changes, and the pitfall of the chapter’s list that the team thereby avoids.

FURTHER READING

Within the corpus, ^[5] is the primary source for §16.3’s account of the interchange standard: the unified-repository model, the formal result enumeration and radius, the temporal/spatial/heterogeneous merge distinction with its caution about automatic merging, the metric-dependence of percentages, and the formally-unreachable API with its predicted-unreachable classification. ^[4] works a formal sign-off end to end and is the source for instantiating constraints in simulation as an over-constraint detector, and for structural coverage measured from a formal testbench. ^[1] describes formal alongside established simulation, with a bug taxonomy by difficulty showing what a second engine finds on already-verified logic. ^[3] is the practitioner’s tour of formal in production, and the source for the predictability asymmetry between the two engines. ^[2] places simulation and formal on one spectrum and develops symbolic simulation as a hybrid engine, a third point between the two treated here.

Outside the corpus: E. Seligman, T. Schubert and M. V. Achutha Kiran Kumar, *Formal Verification: An Essential Toolkit for Modern VLSI Design*, treats the deployment questions of §16.4 at book length; and H. Foster, A. Krolnik and D. Lacey, *Assertion-Based Design*, is a treatment of the property written once and read by both engines.

REFERENCES

1. **D18** M. V. Achutha Kiran Kumar, E. Seligman, A. Gupta, Ss Bindumadhava, and A. Bharadwaj, "Making Formal Property Verification Mainstream: An Intel Graphics Experience," Proc. DVCon U.S., 2017.
2. **B3** V. Bertacco, "Scalable Hardware Verification with Symbolic Simulation," Springer, 2006.
3. **D13** J. Bromley and J. Sprott, "Formal Verification in the Real World," Proc. DVCon U.S., 2018.
4. **D31** I. Tripathi, A. Saxena, A. Verma, and P. Aggarwal, "The Process and Proof for Formal Sign-off: A Live Case Study," Proc. DVCon U.S., 2016.
5. **S3** Accellera Systems Initiative, "Unified Coverage Interoperability Standard (UCIS), Version 1.0," Accellera, June 2012.
6. **D11** X. Feng, X. Chen, and A. Muchandikar, "Coverage Models for Formal Verification," Proc. DVCon U.S., 2017.
7. **B7** Verification Methodology Team, "Coverage Cookbook," Mentor Graphics Verification Academy, 2013 (revised 26 April 2019).
8. **O8** lowRISC, "Design Verification Methodology within OpenTitan," lowRISC/opentitan, doc/contributing/dv/methodology/README.md, commit 4e5c99db524e99425cb78ca6f2714e5287fa800c, accessed 2026-09-02.
9. **S21** USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.
10. **O5** OpenHW Group, "CV32E40Pv2 Waiver List, RTL v1.8.3," openhwgroup/programs, Project-Descriptions-and-Plans/CV32E40Pv2/Milestone-data/RTL_v1.8.3/CV32E40Pv2_waiver_list.xlsx, commit f7968ac5a0e6438549057fe45311abf6244b9a3b, accessed 2026-09-02.
11. **O6** OpenHW Group, "CV32E40Pv2 Uncovered Code Coverage Explanation, RTL v1.8.3," openhwgroup/programs, Project-Descriptions-and-Plans/CV32E40Pv2/Milestone-data/RTL_v1.8.3/CV32E40Pv2_uncovered_coverage_explanation.xlsx, commit f7968ac5a0e6438549057fe45311abf6244b9a3b, accessed 2026-09-02.

PART V

Beyond RTL Simulation

What to do when the simulator runs out of cycles: acceleration and emulation, prototypes running real firmware, verification after synthesis, and the first encounter with real silicon.

17. Acceleration and Emulation

Hardware-assisted verification addresses workloads too long for RTL simulation, with speed and visibility traded against each other. A milestone is a phase-gating event. It is defined as a demonstration that can be checked, not as an estimate of the state the work has reached. The flagship SoC's full-chip milestone illustrates the scope: one acceptance criterion, written eighteen months earlier and never disputed, requires four application cores to boot the operating system to a shell prompt, the mesh NoC to carry real traffic, LPDDR4 PHY training to finish, and the DMA to move data under the host cluster's coherence protocol. This agreed criterion marks the first product-like behavior.

The reference SoC had an equivalent gate: the core boots, the DMA moves a buffer and the UART prints. That gate was a bare-metal image of a few tens of kilobytes and a couple of million cycles, comfortably inside the eleven-minute mean of that project's nightly runs (see Chapter 12). The flagship's boot is an operating system: page tables, drivers, an interrupt controller, a filesystem: several seconds of real execution, which at any application-class clock is billions of cycles. The simulation farm cannot deliver that boot before tape-out regardless of farm size, because a boot is *one run* and serial: two hundred and forty simulator licenses shorten a suite without shortening a single test.

Without hardware-assisted verification, the project first encounters the operating system in the lab. Buying hardware-assisted verification and meeting the operating system first in the lab are both expensive choices; mistaking a slow testbench for an unsimulatable workload can misdirect the decision between buying the hardware and meeting the operating system first in the lab. This chapter distinguishes those two problems and the tradeoffs of running a design a thousand times faster while observing almost none of it.

CHAPTER MAP

§17.1 quantifies simulation's cycle limit. §17.2 compares compilation, visibility and debug ergonomics across simulators, emulators and prototypes, including their workflow costs. §17.3 explains the two use models, in-circuit and virtual, their benefits and limits, and the shift toward virtual operation. §17.4 derives how the transaction boundary determines throughput and limits batching gains. §17.5 covers planning for a shared, scheduled capital resource and diagnoses misframed simulation problems.

17.1 Simulation's cycle limit

RTL simulation runs roughly a billion times slower than the clock of the system it models. An activity taking a few seconds of execution on the target silicon (booting

an operating system is the canonical example) takes several years on an RTL simulator, which precludes running software on top of an RTL model at all and confines RTL simulation to short directed or random tests. When mapped onto a reconfigurable fabric or a purpose-built emulator the same RTL runs about a hundred to a thousand times faster than the simulator, which puts that operating-system boot back inside a few hours ^[1].

These are order-of-magnitude claims: the speed-up band spans a factor of ten, and the position within it depends on design size, on how much of the system is instantiated, and on how much instrumentation is compiled in. The slowdown also spans a band: this book uses tens of thousands at the low end and a billion at the high end. The high end is the cited figure, measured against a full application-class system's target clock. The low end is this book's own measurement. The reference SoC's equivalent boot gate ran a few tens of thousands of times slower than its silicon rather than a billion, and that gate was a couple of million cycles, comfortably inside an eleven-minute nightly mean. Across the stated ratios, an application-class boot requires *years* of one simulation and *hours* of one emulation run, and neither of those fits a nightly window.

Simulation serializes design activity. An event-driven simulator serializes the design's parallel activity: every signal change becomes a software event, and every process sensitive to that signal is re-evaluated in sequence. A million gates switching in the same design cycle are a million pieces of work for one processor. For two decades the industry absorbed this through increasing processor frequency; then frequency scaling plateaued, and simulation-based techniques stopped keeping pace with design complexity, a gap most visible on large designs that instantiate both software and embedded processor core models; and that gap is why acceleration techniques are now essential ^[2]. An individual RTL simulation does not usefully exploit multiple cores ^[3]. Compute available to one run has been flat for years.

Regression and instrumentation multiply the cost. The arithmetic covers a single execution of a single workload, whereas verification requires multiple executions. Chapter 12's flagship nightly *candidate* list totals 6,400 simulations and 4,409 CPU-hours, already requiring re-tiering, ranking and pruning to fit a night. Re-tiering moves a test into a different regression tier. Instrumentation also has a cost: in a Mentor Graphics team's published measurement, introducing accurate metastability injection models into a design degrades simulation performance by three to four times, which is why metastability-tolerance verification cannot live in a nightly regression on a large design ^[4] (see Chapter 13).

The parallelism fallacy. A farm's width multiplies *runs*; it does not shorten a *run*. Licenses address a 4,800-run random suite. For one workload whose length is measured in billions of cycles, licenses are irrelevant: cycle n of a boot depends on cycle $n-1$, so a single run has no parallel fraction to exploit and additional farm width cannot shorten it. Before considering capital equipment, planning should distinguish the number of runs from the length of one run.

Workloads requiring long histories. Simulation cannot reach a bug that requires a long history, one whose events remain correlated and self-consistent; these workloads derive their value from *length* rather than from randomness.

Scenario. A video codec’s rate-control loop adapts quantization to a rolling estimate of channel occupancy. A reference model is an independent executable of the specification, taken as golden and run on the same stimulus. In simulation the team checks a few macroblocks per run against the bit-exact reference model, and everything matches. In the lab, encoded streams drift out of the target bitrate after a scene change followed by a long static passage. **Approach.** The scenario is put on the emulator: several hundred frames of a real sequence, driven through the same reference-model comparison, run end to end. **Why.** The control loop retains hundreds of frames of memory, so only length exposes the bug: its shortest effective stimulus exceeds simulation’s reach, and additional random seeds cannot find it as a data corner case.

The same shape covers three further workloads. An operating-system boot is one. A GPU driver is another, because its interesting failures live in queue and residency management across many submissions rather than in any single shader; residency management is the tracking of what stays resident in memory. The third is a mesh NoC, whose latency distributions and virtual-channel occupancy only become meaningful statistics over sustained traffic (see Chapter 16). Virtual-channel occupancy is how full those channels run.

Comparison across generations. The two generations retain the same approach for the unchanged verification problems. Table 17.1 sets out the unchanged contracts and the work added by scale.

Table 17.1 — One verification problem per row, with the approach each generation of the example SoCs takes to it. The rows marked *unchanged* are the surprise: block-level constrained-random work carries forward intact to the larger design, and what the second generation adds is not a harder version of an old problem but a new class of workload — one long, coherent, software-driven execution — that no amount of simulation capacity delivers.

Problem	Reference SoC	Flagship SoC
DMA bursts split at the 4 KB boundary, 2D transfers, negative stride	Constrained-random simulation, multi-seed	<i>Unchanged</i> — constrained-random simulation, more seeds, eight channels
Register and peripheral suites	Simulation	<i>Unchanged</i> — carried forward
Boot to a working system	Bare-metal image, a couple of million cycles, one nightly run	Operating system; no simulation answer at any farm size
Interconnect congestion behaviour	Crossbar arbitration, proved and simulated at block level	Mesh NoC latency and occupancy statistics under sustained load

Chapter 12 showed the flagship’s DMA suite carrying forward directly: a suite of 4,800 independent runs is the workload a license farm is good at, and moving it to a machine that runs one image at a time would be a mistake (Section 17.5 does that arithmetic). The second generation adds a *new class* of workload (one long, coher-

ent, software-driven execution) that the first generation never had and that no amount of simulation capacity delivers. Emulation addresses this specific workload; applying it to independent regressions can leave an idle multi-million-dollar machine and an unfinished suite.

17.2 Three platforms and their costs

In emulation, the design is *compiled*, synthesized and partitioned onto hardware execution fabric, then runs under host-provided stimulus. The fabric defines three deployment categories. The first is prototyping, which uses one or more commodity FPGAs. The second is an emulator proper, a vendor-built system with its own compilation, transactor and debug infrastructure. The third is hybrid co-emulation, a split in which a virtual platform on the host runs the well-modeled parts while an emulator runs selected RTL at cycle accuracy. The two sides are joined by transaction-level co-modeling channels, which carry whole transactions between the host and the emulator. A virtual platform is a software model of a whole system, fast enough to run real firmware long before stable logic exists (see Chapter 18). Prototypes and emulation systems alike operate at MHz-scale effective speeds on large SoCs, against a simulator rate that is a hundred to a thousand times slower (Section 17.1) [5]. A design capacity on the order of a billion gates is within reach of an emulator [4], and in the 2024 industry study emulation adoption rises significantly for designs above one billion gates [2].

The three costs beyond speed are compared in Table 17.2, below each platform's benefits.

Table 17.2 — A simulator, an emulator and an FPGA prototyping platform compared row by row: throughput and capacity are what the hardware platforms buy, and the rows beneath them are what each hands over to get them. The pair that reorganises how people work is build time against changing what you observe — on these platforms a signal that was not brought out before the build cannot be looked at without another build. §17.5 adds the cost in money.

	Simulator	Emulator	Prototyping platform
Throughput	baseline	10^2 – $10^3\times$ the simulator	the fastest pre-silicon platform
Capacity	bounded by host memory and patience	order of a billion gates	device capacity; above it, multi-FPGA partitioning
Build time	minutes	hours to days for a complex design	hours for a bitstream
Visibility	any internal signal, at any time	triggers, selective trace, save/restore	a few thousand signals, chosen <i>before</i> the build
Changing what you observe	edit and re-run	re-arm a trigger; recompile if the signal was never brought out	recompile the bitstream
Parallel width	licences (hundreds)	capacity $\div$ design size	capacity $\div$ design size, replicated per board

Visibility. In a simulator, virtually any internal design signal is observable at any time. On a reconfigurable fabric (the fastest of the pre-silicon platforms) observability is restricted to a few thousand internal signals, and they must be selected before the bitstream is generated; reconfiguring observability means recompiling, which might take several hours [1]. Emulators soften this with controlled observability mechanisms (triggers, selective trace and save/restore), but those mechanisms move the constraint rather than remove it: capturing every internal signal at full speed over billions of cycles is infeasible, so a subset of signals or trigger conditions is chosen a priori, and a failure arising from an unmonitored interaction requires the same long compile [5].

Turnaround shapes the workflow. Compile times of hours to days for complex designs, worsened by instrumentation, are reported for large emulation flows [5]. In simulation an engineer can edit, rebuild in minutes and have an answer the same morning: the reference SoC's smoke tier is under nine minutes end to end, most of it build (see Chapter 12), so it is feasible to revise a mistaken observation choice, ten times a day. An emulator may allow *one* build per day, sometimes fewer. In this example, every observation omitted before the build costs a day.

Emulation engineers should instrument for the questions they expect, compile in assertions and monitors rather than planning to add them later, and treat signal visibility as a review decision made before debug. Once built, an image is reused across many hours or days of execution, regressions and campaigns, so the build is paid once and divided by everything that runs on it [5]. A build supporting one run bears the full build cost; across two hundred runs, that cost is amortized over the group. Section 17.5 turns that into a decision rule.

Debug ergonomics. Per-beat logging and unrestricted waveform dumping conflict with emulation's throughput and trace limits, and a beat is one data transfer inside an AXI burst. A `$display` on every bus beat forces a host interaction per cycle and destroys throughput (Section 17.4). Waveform dumping is bounded by a finite trace buffer, not by disk. The replacement has two parts. Assertions compiled into the platform, as part of the model, report violations immediately, either by logging them or by stopping the run at the violation. Trigger-based debug runs large numbers of cycles under trigger state machines built to recognize complex sequences of events. Placing an assertion near a suspected cause makes a long run debuggable, since a failure on an accelerated platform routinely surfaces far from its cause and much later, as injected metastability does [4]. Chapter 11 explains that an assertion is *declarative*: it says what to check and leaves the mechanism to the tool, which is why the same three lines serve a simulator and an emulator alike. An assertion needs no conversation with the host, so it is the one form of checking that crosses the boundary intact and keeps *detection distance* short (see Chapter 8) where nothing else can.

Scenario. A mobile GPU's driver-level bring-up needs many thousands of frame submissions to shake out queue and residency management, but the application cores that run the driver are mature, well-modeled IP with no open verification

questions. **Approach.** Hybrid co-emulation: the cores stay as fast host-side models, the GPU's four shader clusters and its memory path go on the emulator, and the two exchange transactions. **Why.** Three benefits follow: the RTL footprint on the scarce machine shrinks, mature models are used for the mature IP, and cycle accuracy is preserved exactly where the open questions are [5]. The constraint is that the boundary between virtual and RTL is now a modeling assumption the team must be able to defend: any bug whose mechanism straddles it is invisible by construction.

The book uses stricter terminology than the industry. An emulator runs a design *model*, so its use is pre-silicon verification. An FPGA prototype on a bench with real peripherals attached is closer to what this book calls **validation** (checking on real hardware rather than on a model), which is why prototyping and the software work that prototyping enables get their own treatment (see Chapter 18), and why post-silicon gets a treatment of its own (see Chapter 20). This distinguishes evidence about a design from evidence about a board.

17.3 Use models: in-circuit and virtual

Exactly two use models provide stimulus to the emulated design. In **in-circuit emulation** (ICE), real physical peripherals drive the emulated design: a host, a link partner or test equipment. In the **virtual** or **targetless** model, there is no physical target: the environment is a synthesized testbench on the platform plus software models on the host, and the design is driven through transactors.

The two ends of ICE run at different speeds. The design executes at MHz; the real link runs at its specified rate, typically orders of magnitude faster, with timeouts. A *speed adapter* bridges them, supplied alongside protocol transactors for hardware interfaces such as USB, Ethernet and PCIe [3]. It buffers, throttles and re-times traffic to keep the physical link within specification while the design runs slowly. The design therefore receives real *content* at adapter-generated *timing*.

ICE capabilities. Three capabilities follow: a real driver stack produces ordering quirks and buggy scenarios beyond constrained-random stimulus; third-party interoperability needs no model of the other party; and instrument-only compliance procedures can run.

Scenario. A USB 3.x dual-role controller's link-training and low-power rows (the U0 through U3 transitions) were assigned in the vplan to the USB compliance suite, with the suite's own item list as the closure metric (see Chapter 5). **Approach.** ICE: the emulated controller is connected through a speed adapter to the physical compliance instrument, and the suite is executed as the compliance methodology defines it; the tests are specified not by the USB specification itself (*Universal Serial Bus 3.2 Specification*, Revision 1.1) [6] but by a separate compliance methodology document. **Why.** There is no file of vectors to replay: the suite is a procedure driven by an instrument, with electrical and timing components as well as protocol ones, and reimplementing it as a virtual transactor would mean produ-

cing an independent reimplementation of the specification that is being checked. **Scope of row closure.** The speed adapter sits between the design's pins and the instrument, so what closes here is the *protocol-state* half of each row. Timing at the pins is the adapter's, not the design's, and those rows stay open until silicon (see Chapter 20). A row that records "compliance suite passed" without that split is claiming evidence it does not have.

ICE limits drove the shift toward virtual operation. ICE loses repeatability: a real link partner's failure has no seed or replay. Chapter 12's apparatus assumes that a run is a function of its recorded inputs: a run manifest records those inputs, duplicate failures are folded together by failure signature, and a repeat-N stability check runs before a test may serve as a gate. A failure that cannot be reproduced cannot enter triage, the work of turning failures into facts before any of them is debugged. Scheduling is constrained by a physical, local, singular rig, one bench and one set of cables, conflicting with Section 17.5's economics. Unattended operation is lost because someone must plug in the device. Save/restore is also lost because the physical environment cannot be rewound.

Virtual emulation controls model time. In a virtual environment built on the Standard Co-Emulation Modeling Interface (SCE-MI), an interface that emulation vendors support, the platform provides full simulation interoperability: it generates clocks in the simulation style *on the emulator*; and it tracks model simulation time inside the emulator accurately enough to measure how far apart in model time a transmit clock edge and a receive clock edge fall [4]. That interface is a published Accellera standard, the SCE-MI Reference Manual, Version 2.4, whose time-access section specifies how the software side obtains the current simulation time and the precision it is expressed in [7]. When the platform owns model time, execution can stop, resume and replay, and an event can be placed at an exactly chosen instant. Such control enables deterministic reruns, controlled clock relationships, injection at a specified cycle, and a debug environment that behaves like a simulator's.

Virtual emulation environments offer almost all of the simulation-style debug features (interactive run control against model time, waveform display, full or selectively traced waveforms, source browsing), and virtual emulation is repeatable; that repeatability is what makes an elimination strategy possible, in which the engineer disables a subset of the injection points, reruns the identical scenario, and sees whether the failure persists [4]. The same environment can serve both simulation and acceleration when it is built to the standard [3], further supporting the migration.

Scenario. A radar front-end's FFT and constant-false-alarm-rate (CFAR) pipelines are being verified for fixed-point precision: every intermediate result must match a bit-exact reference across long chirp sequences, in three arithmetic configurations. Attaching the real RF front-end in ICE is technically possible. **Approach.** One real chirp sequence should be captured in the lab, its digitized samples stored and replayed from the host through a virtual transactor: the same file, three times, once per configuration. **Why.** A bit-exact comparison requires the *same* input twice. A live front-end delivers a different realization of noise on every run, so a mismatch

between two configurations cannot be attributed to either of them, and a failure cannot be reproduced for debug. ICE's environmental realism has no value for this arithmetic property; the one useful contribution, a realistic signal, survives as a recording.

ICE is appropriate when the environment cannot be modeled; virtual operation is appropriate otherwise. A compliance instrument, a third-party host and a driver stack the team does not own are environments that justify accepting irreproducibility. Noise, traffic patterns and workloads are recordable, indefinitely replayable *data*, so they do not justify irreproducibility.

17.4 The transactor boundary

Chapter 8 introduced the transactor as an abstraction device: it encapsulates every physical-level operation of one interface in one place, so that everything above it speaks transactions. In acceleration, it becomes a physical boundary. Below it, the design is synthesized onto the platform and runs at the platform's clock rate; above it, code runs on a workstation. Between them is a link with latency, and the placement of that boundary, rather than the platform's headline frequency, determines throughput.

Signal-level acceleration creates a bottleneck. In a conventional environment, bus-functional behavior resides in the UVM (Universal Verification Methodology) driver and monitor, which drive pins through a virtual interface. Accelerating only the DUT puts the design on-platform but leaves the protocol in the UVM driver and monitor classes on the host, which forces every single pin event across the link. This is *signal-level acceleration only*, and it is the bottleneck ^[3]. A workstation thus feeds a billion-gate MHz machine one clock edge at a time.

Chapter 8 states that an environment that reaches into signals from twenty places cannot be moved at all; Chapter 10 states why: a UVM component (IEEE Std 1800.2-2020, *IEEE Standard for Universal Verification Methodology* ^[8]) is not synthesizable, so it cannot cross to the other side. The required boundary belongs at the transaction, as [Figure 17.1](#) shows.

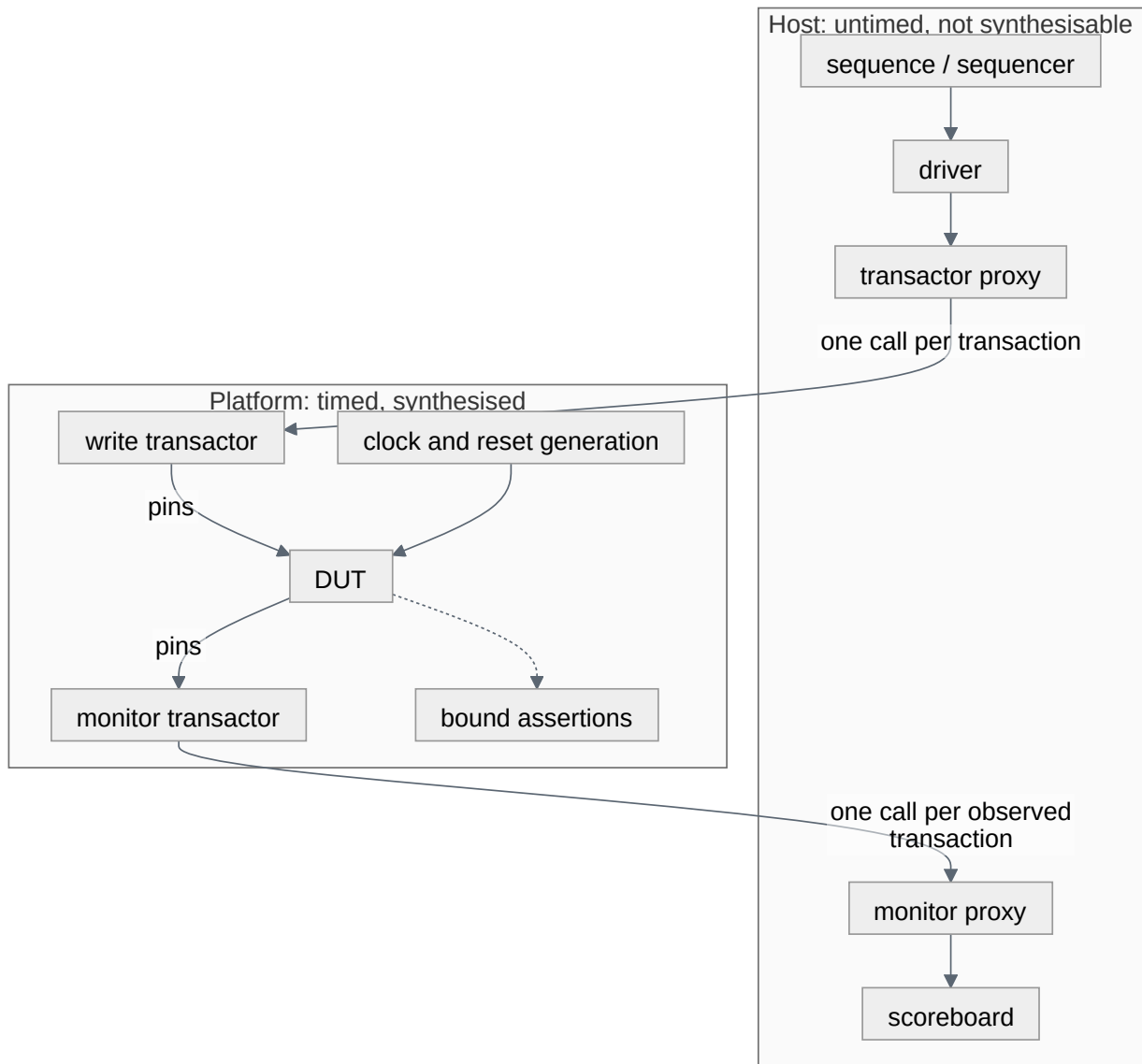


Figure 17.1 — Where the transaction boundary cuts an accelerated environment: the upper box is untimed, not synthesisable and runs on the host, the lower box is timed, synthesised and running on the platform, and the cut carries exactly one call per transaction in each direction. What matters is what has no counterpart above the boundary — clock and reset generation, and the bound assertions, all sit below it, so nothing on the host is consulted per cycle.

Clock and reset generation and assertions are absent from the host, whose code knows no clock edges and is never consulted per cycle; a monitor transactor feeds the scoreboard at transaction rate, while assertions require no host interaction. The scoreboard is the structure that holds the data expected to be received, and it is consulted when an output is observed.

A bus-functional model is a transactor's timed half, the part that converts between transactions and pins. Siemens EDA's Verification Academy publishes a checklist for that partition; the checklist splits each transactor so that its untimed half stays on the host as a proxy and its timed half becomes a bus-functional model on the platform, and it sets out three requirements a testbench must meet before it can be accelerated: the verification code must be entirely untimed; timed and untimed code must live in separate domains; and no reference may cross between them [9].

The arithmetic of the boundary

Let f be the platform's clock rate, let N be the number of design cycles the platform advances between host interactions, and let L be the cost of one host interaction expressed in equivalent design cycles. Each period delivers N cycles of design progress and costs $N + L$, so the effective rate is

$$R(N) = f \cdot N / (N + L).$$

Pin-level driving is the case $N = 1$, giving $f/(1 + L)$. Batching to N cycles per interaction therefore gives a speedup of

$$S(N) = R(N)/R(1) = N (1 + L) / (N + L),$$

with two limiting properties. It **saturates at $1 + L$** regardless of N , so batching cannot exceed the round-trip cost itself. And it reaches exactly half of that ceiling at $N = *L*$. Transaction granularity should therefore be comparable to round-trip cost, and optimization should stop once it is comparable; moving to transactions is not by itself sufficient.

L is large, and it is the parameter least under the team's control: it is a driver call, a link traversal, host scheduling and a return, measured against a clock period that at MHz-scale rates is of the order of a microsecond. For L somewhere in the tens to the thousands of cycles, [Table 17.3](#) gives the fraction of the machine's peak rate the testbench obtains, $N/(N + L)$.

Table 17.3 — The fraction of an accelerated platform's peak rate a testbench actually obtains, $N/(N + L)$: rows are the design cycles N that one host interaction covers, columns three values for the round-trip cost L of that interaction expressed in equivalent design cycles. The top row is pin-level driving, which throws away nearly everything the machine was bought for; the rightmost column is the warning against complacency, since on a costly link even the longest burst the protocol encodes still leaves you at 11% of peak, and the remedy there is a coarser transaction rather than a faster link.

Cycles per host interaction	$L = 20$	$L = 200$	$L = 2000$
1 (pin-level)	4.8%	0.5%	0.05%
16 (a short burst)	44%	7.4%	0.8%
256 (the longest burst AXI encodes)	93%	56%	11%

Pin-level driving throws away between 95% and 99.95% of peak throughput, so accelerating only the DUT produces an expensive simulator. The rightmost column shows that on a platform with a costly link, even the longest burst the protocol can express leaves the testbench at a tenth of peak. That case calls for a *coarser* transaction, such as a transfer descriptor, a page or a frame, rather than a faster link. A transfer descriptor is the record software writes to program one DMA transfer, holding its source and destination addresses and its lengths.

Drawing the boundary in code

The change is small and entirely local: timed protocol behavior moves from the class into a synthesizable transactor module exporting a host-callable task; a host proxy imports it, and the driver's body becomes a single call. Below the boundary, the flag-

ship DMA's write transactor issues one burst using 40-bit physical addresses on a 512-bit channel:

```
// ---- Below the boundary: synthesised onto the platform. One call = one burst.
module wr_xactor #(parameter int ADDR_W = 40, parameter int DATA_W = 512) (
    input logic clk,
    input logic rst_n,
    output logic [ADDR_W-1:0] awaddr,
    output logic [7:0] awlen,
    output logic awvalid,
    input logic awready,
    output logic [DATA_W-1:0] wdata,
    output logic wlast,
    output logic wvalid,
    input logic wready
);
    logic [31:0] lfsr;

    export "DPI-C" task do_write_burst;

    task do_write_burst(input longint unsigned addr,
                       input byte unsigned len,
                       input int unsigned seed);

        lfsr = seed;
        @(posedge clk);
        while (!rst_n) @(posedge clk);
        awaddr <= ADDR_W'(addr);
        awlen <= len;
        awvalid <= 1'b1;
        do @(posedge clk); while (!awready);
        awvalid <= 1'b0;
        for (int i = 0; i <= int'(len); i++) begin
            lfsr = {lfsr[30:0], lfsr[31] ^ lfsr[21] ^ lfsr[1] ^ lfsr[0]};
            wdata <= {(DATA_W/32){lfsr}};
            wlast <= (i == int'(len));
            wvalid <= 1'b1;
            do @(posedge clk); while (!wready);
        end
        wvalid <= 1'b0;
        wlast <= 1'b0;
    endtask
endmodule

// ---- Above the boundary: what the untimed testbench calls, once per burst.
interface wr_xactor_proxy;
    import "DPI-C" context task hdl_do_write_burst(input longint unsigned addr,
                                                  input byte unsigned len,
                                                  input int unsigned seed);
endinterface
```

Four aspects of the code determine acceleration behavior. Three of them restate the standard acceleration guidelines ^[3]; the fourth is the lesson those guidelines leave implicit.

The task is timed and the call is not. Everything that waits on `clk` is inside the module; the proxy above declares no clock and the driver never mentions one. The testbench must synchronize on transactions and events, never on clocks. A driver that waits for an edge has re-created the pin-level case by another route.

One call covers `len + 1 beats`. With `awlen` at its maximum of 255 that is a 256-beat burst (the $N = 256$ row of the table) for a single host interaction. That table row is the protocol's ceiling, not the ceiling of this AXI write data channel: a beat here is 512 bits, which is 64 bytes, so 256 of them would move 16 KB, and that would cross three 4 KB boundaries. The 4 KB boundary is the address boundary that AXI forbids a burst to cross: every burst stays inside one 4 KB address region, so it never spans two subordinates. This DMA inherits a burst splitter, the stage that splits each transfer into bursts that the AXI address rules allow. That splitter caps a legal INCR burst at 64 beats, which is exactly one 4 KB region. So a single call reaches somewhere between the table's second and third rows, and the gap to the third row is not something a longer burst can close. When one legal burst is not granularity enough, the next transaction up is a descriptor, rather than a bigger burst.

The same source serves both engines. The transactor is ordinary SystemVerilog that a simulator will execute directly, so one environment supports simulation and acceleration, and a bug found on either platform is reproducible on the other; this holds for exactly as long as the two remain one source, a condition difficult to sustain (see *In practice*).

The payload is manufactured on the platform. The host passes a 32-bit `seed`; the shift register produces the data. If the host randomizes the payloads and ships a 512-bit beat for every beat, the environment is back to one host interaction per beat, and each of those interactions now carries data volume as well. When a metastability-injection methodology needed an uncorrelated random decision at every clock-domain-crossing point, a stream of random values arriving from the workstation was rejected on two counts: the logic needed to distribute the values, and the runtime needed to transfer them; the choice fell instead on an on-platform shift-register generator whose bits are statically distributed among the injection points ^[4]. Per-cycle environment operations must become hardware, determining how much testbench code needs rewriting.

Testbench costs at the boundary

DPI-C is the SystemVerilog mechanism for calling C from the language and being called back. The platform accepts a superset of RTL, in that implicit state machines, some system tasks, shared variables, hierarchical names and named events are all typically compiled; the common denominator is still *synthesizable plus DPI-C*; and in the other direction the DPI itself is a restricted subset, in which the usable argument types are limited, nesting of imported and exported calls is bounded, and four-state values may be reduced to two-state at the boundary ^[3].

X-dependent checks remain in simulation. The reduction to two-state is documented at the DPI boundary; in practice the prudent assumption is broader, because a fabric of real gates has no third value to propagate. Checks requiring an unknown to reach an output, such as an uninitialized-register detector or a reset-coverage check, should remain in simulation (see Chapters 12 and 13). The porting effort should identify which checks moved and which did not.

Checking splits by where its cost lands. An assertion compiled into the platform costs logic and no interactions, so it scales to billions of cycles. A scoreboard on the host costs one interaction per transaction it must see, so it scales to whatever transaction rate the table above allows. An oracle is the mechanism that decides whether an observed response is correct. One that needs to see *every beat* costs one interaction per beat and does not scale at all, which means it has to be redesigned rather than ported.

Scenario. An inline AES-GCM crypto engine on the memory path is checked in simulation by a host-side scoreboard that compares every ciphertext beat against a reference model. The team wants a multi-hour soak with real firmware driving real traffic through it. **Approach.** The oracle requires restructuring. A synthesizable digest accumulator on the platform hashes the ciphertext stream and the authentication tags; the host reads and compares one digest per checkpoint (say every few thousand blocks) instead of one value per beat. Per-beat comparison remains a short-vector check in the simulation tier. **Why.** Long emulation runs become host-bound when oracle cost scales with beats. On-platform per-cycle work and one host interaction per checkpoint make the soak possible. **What it costs.** Detection distance (see Chapter 8): a digest mismatch says the stream diverged somewhere in the last checkpoint interval, not where. Checkpoint intervals are necessary and bound the debug bisection, the search that narrows a failure down by halving the interval it lies in.

17.5 Economics: one machine, many claimants

Emulation requires capital equipment rather than license seats, complicating its introduction into a verification program. The cost of acquiring and maintaining a full-featured emulator confines it, in practice, to a limited number of large design houses, and even inside those houses the emulator slots are shared across multiple projects and teams: they have to be scheduled and prioritized, and capacity is typically reserved for tape-out-critical functional regressions, so that other campaigns get a subset of high-risk blocks or get deferred to later project stages [5]. A simulation license pool degrades gracefully under contention, whereas emulator access is either granted or unavailable.

One estimate put hardware-assisted verification at USD 760.4 million in 2025, split roughly 61% emulation to 39% prototyping, and projected it to reach USD 3.08 billion by 2035 at 15.0% compound growth; a second and independent estimate puts 2024 at USD 641.5 million and 2030 at USD 1.5 billion, a compound growth of 15.2%; and the whole electronic-system-design industry, for comparison, reported USD 5.6 billion of revenue in a single quarter of 2025 [5]. The book interprets the segment's small size and rapid growth as consistent with a shift from optional to necessary equipment. As a capital purchase, the machine calls for calendar planning, handled by a senior manager in this example; a request for three days belongs in the project plan.

In the 2024 industry study the leading reason projects adopt emulation or prototyping is hardware/software co-design and verification, with post-silicon debug well down the list [2]. When software readiness justifies the machine, verification shares access under the resulting scheduling priorities.

Workload placement by wall-clock cost

The workload contains N runs. On the farm, wall-clock cost is $N \cdot s / W_{\text{sim}}$, where s is per-run simulation time and W_{sim} the license count. On hardware, it is $B + N \cdot r / W_{\text{hw}}$, where B is the build and r the per-run execution time. Hardware parallel width, W_{hw} , is capacity divided by design size, rather than a separately purchased quantity. A block-sized image replicates across boards cheaply, which is why an eight-board cluster can run a suite in parallel (see Chapter 12), while a full-SoC image occupies the machine, and $W_{\text{hw}} = 1$.

The flagship provides two cases.

The OS boot. N is a handful of runs per design revision; s is years (Section 17.1); r is a few hours [1]; B is hours to days. Simulation costs years per run and W_{sim} cannot help, because a license pool shortens suites and not runs. Hardware costs a build plus an afternoon. Break-even is below one run, so the workload requires a slot without further business justification.

The DMA suite. Chapter 12 gives the numbers: 4,800 constrained-random runs at a 30-minute mean, 2,400 CPU-hours, ten hours of wall clock on 240 licenses. The 100–1000× band is the platform’s *raw* rate, before the boundary derating of Section 17.4, which only makes the comparison worse. If the suite is moved, even at the optimistic end of that band, r is under two seconds, but W_{hw} is 1, so 4,800 runs cost about 2.4 hours of machine time, against simulation’s ten hours, a real gain; and the total then also includes B , a build that takes hours to a day. The build B is paid **per design revision**, as is every nightly suite. A build measured in hours and incurred every night yields no gain against ten hours on existing licenses. At the pessimistic end of the band the emulator loses on run time alone.

Hardware is favored when s is enormous and N is small; the farm is favored when N is large and s is moderate. Independent runs favor width over speed, explaining Section 17.1’s comparison.

Admission criteria for emulator time

The admission policy is a tier in Chapter 12’s sense, a budgeted rule for admission, with a different currency. A tier spends fungible CPU-hours, whereas emulation spends nonfungible *machine days and build slots*. Three claims earn a slot, and one rule protects it:

1. **Workloads with no simulation answer:** boot, firmware, long soaks, sustained-traffic statistics on the mesh NoC (see Chapter 16).

2. **Campaigns valued for iteration count under one image** include fault-injection sweeps and adversarial campaigns whose practical effectiveness depends on high iteration counts and repeatability, with build cost amortized across the sweep ^[5]. Full-chip metastability-tolerance verification belongs here too: the accurate injection models are exactly the ones that cost 3-4× in simulation (Section 17.1), which is why they run on hardware (see Chapter 13). That same team reports the degradation as tolerable on a small block and enough to make the traditional flow impractical as designs grow, supporting the move onto hardware ^[4]. Its “often 1000X” figure for emulation over simulation is a general remark with no benchmark behind it, and this book does not use it; the 100-1000× band that this section uses to cost workloads comes from ^[1] instead.
3. **Reproductions of failures found downstream.** Recreating a silicon failure on an acceleration platform gave one industrial team all the data a designer needed for root cause, because the platform has the observability that silicon lacks; the failure could not be migrated as-is, however, since silicon runs more than six orders of magnitude faster and the exerciser that drives the design had to be re-tuned to hit it ^[1]. Describing the scenario portably rather than per-platform makes migration cheap enough to be routine ^[10] (see Chapters 9 and 20).
4. **Farm-suitable workloads are excluded.** The admission policy must enforce this exclusion.

Chapter 5’s vplan row already carries a *method* field, and that field names the engine that discharges the row. An emulation row needs one additional column: **the image it runs on**. Because the image is the scheduling unit, emulation rows are batched by image rather than by priority: a low-priority row that fits an image already being built is nearly free, and a high-priority row needing its own instrumentation costs a build. Images should be planned before rows are assigned; reversing that order can leave top-priority rows across five unscheduled builds in this example.

Workloads better served by simulation

Three diagnostic cases can justify refusing emulator time; in this example, each can be diagnosed in an afternoon.

The testbench is what is slow. Per-item object allocation, dynamic dispatch and a report server on the message path cost nothing on a ten-thousand-cycle block run and a great deal on a long one (see Chapter 10). Dynamic dispatch is the run-time choice of which method a call runs. Profiling should precede machine costing: if the design takes a minority of runtime, the problem and fix are software.

Every run boots from zero. A workload that spends 90% of its cycles reaching the state under test does not need a faster engine; it needs a saved state, or a shorter path to that state.

The stimulus is badly shaped. A system-level soak can substitute for unwritten block-level stimulus that would reach the same coverage directly, at greater expense. Emulator time is, in that case, being bought to avoid rewriting a test, a poor trade because the soak also finds the bug more slowly and localizes it worse.

Section 17.1 distinguishes run count from run length; only the latter justifies emulation.

IN PRACTICE

Monitors and harnesses frequently have to be re-engineered to fit emulator timing and capacity constraints, and that re-engineering produces divergent simulation and emulation environments, with subtle inconsistencies in observed behavior: those inconsistencies complicate debug and make it harder to reproduce an issue across platforms [5]. Maintaining one source for both engines and scheduling the same scenario across platforms should expose divergence before a tape-out escalation, as a standing commitment.

Planning should account for the additional orchestration work described below. Regression management tooling routinely schedules thousands of simulations, analyzes coverage and tracks closure; extending it to schedule long hardware campaigns is non-trivial, because emulator jobs have to be co-optimized against functional runs, and because campaigns requiring coordinated software stacks add orchestration complexity of their own [5]. A planning example can assume three things: a small build team that is distinct from the test writers, a failed build that costs a day and leaves jobs queued behind it, and next week's allocation decided in a meeting that the requesting engineer may not attend. These constraints call for planned images and instrumentation before execution, without arguing against emulation.

PITFALLS

1. **Accelerating the design and leaving the environment alone.** *Symptom:* an extremely expensive machine delivering a few thousand cycles per second. *Cure:* move the timed protocol into synthesizable transactors and batch to a granularity comparable to the round-trip cost (Section 17.4).
2. **A required signal was not brought out.** *Symptom:* every interesting failure ends in “rebuild with more probes”, at a day each. *Cure:* choose observability in a review before the build, and compile assertions in rather than planning to add probes later.
3. **Buying capacity for a farm problem.** *Symptom:* the machine idles while the nightly overruns. *Cure:* distinguish N from s before requisitioning equipment, and profile the run to confirm that design execution occupies the majority of it.
4. **In-circuit by default.** *Symptom:* irreproducible failures and a bench unavailable to other users. *Cure:* virtual unless the environment itself is the unmodelable thing; record and replay anything that is only data.

5. **Porting the oracle instead of restructuring it.** *Symptom:* a per-beat scoreboard pins throughput at transaction rate and the soak never finishes. *Cure:* per-cycle checking on the platform, one host interaction per checkpoint.
6. **Treating a compliance pass as a closed row.** *Symptom:* an electrical requirement marked covered because a suite ran through a speed adapter. *Cure:* split the row: protocol state closes pre-silicon, timing at the pins does not.

SUMMARY

- RTL simulation is roughly a billion times slower than the target clock: seconds on silicon become years in simulation, and hardware-assisted platforms recover a hundred to a thousand times of that ^[1]. The slowdown spans tens of thousands at reference-SoC scale to a billion at full application-class scale, with the same conclusion throughout.
- One distinction governs triage: run count versus run length, because a license farm shortens suites but can never shorten a run.
- Visibility, turnaround and debug ergonomics involve signals and triggers chosen before building, a build rate that may be one a day and sometimes fewer, and checking planned in advance.
- The transactor boundary determines throughput beyond the platform's clock rate. Batching to N cycles per host interaction saturates at $1 + L$ and reaches half that at $N = L$, so granularity should match round-trip cost without further optimization.
- Per-cycle needs, including payload generation, randomization and per-beat checking, require hardware implementation and determine how much testbench code needs rewriting.
- Virtual operation is preferable unless the environment is genuinely unmodelable, because control of model time enables repeatability, replay and simulator-like debug.
- Images should be planned before rows: the image schedules the shared capital resource, while row-priority planning conflicts with the build calendar.

EXERCISES

Exercise 17.1 (calculation). §17.4 gives the fraction of an accelerated platform's peak rate that a testbench obtains as $N/(N + L)$, with N the design cycles one host interaction covers and L that interaction's round-trip cost in equivalent design cycles. Take the round-trip cost $L = 200$ of the batching table's middle column together with the burst splitter of §17.4, which caps one legal INCR burst at 64 beats. Compute the fraction that one legal burst per host interaction obtains, and locate the result among the table's rows. Compute also the ceiling at which the batching speedup saturates and the batch size at which half that ceiling is reached, then state which of the two the burst cap leaves out of reach.

Exercise 17.2 (artifact). Read the plan row below, brought to the review that closes the compliance rows of §17.3. Decide which half of the criterion the recorded evidence closes and which half it leaves open, and name the platform element that forces the split. Name the column §17.5 requires of every row that runs on hardware, left empty here, and state what the scheduling of a shared machine loses without it.

Row:	LNK-L07
Criterion:	the link-training and low-power state transitions follow the published rules, with the specified timing at the pins
Platform:	in-circuit emulation through a speed adapter
Closure metric:	the compliance methodology's item list
Result:	suite passed
Status:	closed
Image:	(empty)

Exercise 17.3 (judgement). A lead proposes moving the reference SoC's smoke tier onto the emulator, so that the tier no longer waits on a simulation build. Decide what the build economics of §17.2 and §17.5 do to the proposal, separating the case in which a build is amortized from the case in which it is not. State the one workload property that would reverse the decision, and the admission criterion of §17.5 that a workload with that property satisfies.

Exercise 17.4 (calculation). §17.5 costs the DMA suite on both platforms: 4,800 constrained-random runs at a 30-minute mean, ten hours of wall clock on 240 licenses, and about 2.4 hours of machine time if the suite is moved. Derive the per-run execution time that the machine figure implies, and show that the wall-clock figure on the farm follows from the run count and the license count. State which term of §17.5's placement model decides against the move, and why the machine's parallel width cannot offset that term.

Exercise 17.5 (artifact). Read the triage record below, filed after a first attempt at a multi-hour soak on the emulator. Decide which of the three classes of checking in §17.4 the oracle belongs to, and state why the observed throughput is a property of the oracle rather than of the platform. Name the restructuring §17.4 prescribes for an oracle of that class, and the cost §17.4 attaches to it.

Workload:	firmware-driven traffic through the inline crypto engine
Platform:	emulator, virtual use model
Oracle:	host-side scoreboard, one comparison per ciphertext beat
Compiled-in assertions:	none
Observed throughput:	the run advances at the rate the host accepts beats
Soak completed:	(no)

Exercise 17.6 (judgement). A plan proposes in-circuit emulation for a bring-up campaign whose environment is a third-party host the team does not model, and whose failures are required to gate a release under Chapter 12's stability check. Decide whether §17.3 licenses the use model for this campaign, and name the two losses of §17.3 that the gating requirement makes decisive. State how the plan should record the row that gates the release, following the scope split §17.3 applies to a row closed by a compliance suite.

FURTHER READING

Within the corpus, ^[1] is the anchor for this chapter's arithmetic (the billion-fold ratio, the hundred-to-thousand-fold recovery, the observability ladder from simulator to re-configurable fabric to silicon, and the bitstream recompile) and its industrial case study is also an account of carrying a failure back from silicon onto an acceleration platform. ^[5] is a recent survey of hardware-assisted verification as a *workflow*: platform taxonomy including hybrid co-emulation, the compile-versus-execution distinction and its amortization, the observability and trace-buffer constraints, the divergence between simulation and emulation environments, and the market figures of Section 17.5; it is framed around security, but its platform and workflow analysis is general. ^[3] gives a step-by-step UVM conversion for acceleration, covering the transactor, exported task, proxy and guidelines, and merits full reading before a first port. ^[4] explains virtual emulation's advantages over ICE and illustrates the general problem of on-platform randomization. ^[2] supplies the 2024 adoption data by design size and the reasons projects give. ^[10] and ^[11] cover the stimulus side: describing scenarios once and retargeting them across simulation, emulation, prototyping and silicon. The corpus also holds the Accellera SCE-MI reference manual for Section 17.4: its use-model sections define the transactor boundary, and its time-access section governs model time ^[7].

Outside the corpus, P. Rashinkar, P. Paterson and L. Singh's *System-on-a-Chip Verification: Methodology and Techniques* merits consultation: its account of emulation and rapid prototyping predates most current tooling and explains the rationale for the use models.

REFERENCES

1. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
2. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
3. **D33** A. Grove, "UVM hardware assisted acceleration with FPGA co-emulation," Proc. DVCon Europe, 2015.
4. **D34** A. Hari, S. Krishnamurthy, A. Jain, and Y. Badaya, "Verification of Clock Domain Crossing Jitter and Metastability Tolerance using Emulation," Proc. DVCon U.S., 2012.
5. **P14** T. Rahman, S. Saha, A. Y. Alhurubi, S. K. Saha, F. Farahmandi, and M. Tehranipoor, "Emulation-based System-on-Chip Security Verification: Challenges and Opportunities," arXiv preprint arXiv:2604.15073v1, April 2026.

6. **S21** USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.
7. **S22** Accellera Systems Initiative, "Standard Co-Emulation Modeling Interface (SCE-MI) Reference Manual, Version 2.4," Accellera Systems Initiative, November 2016.
8. **S9** IEEE, "IEEE Std 1800.2-2020, IEEE Standard for Universal Verification Methodology Language Reference Manual (Revision of IEEE Std 1800.2-2017)," IEEE, 2020.
9. **B8** Functional Verification Methodology Team, "UVM Cookbook," Siemens EDA Verification Academy, 2021 (revised 31 January 2022).
10. **D29** P. Gupta and M. Vax, "Test driving Portable Stimulus at AMD," Proc. DVCon U.S., 2019.
11. **S1** Accellera Systems Initiative, "Portable Test and Stimulus Standard, Version 3.0," Accellera, August 2024.

18. FPGA Prototyping and HW/SW Co-verification

FPGA prototyping supports hardware/software integration and lab debug; this chapter examines how debug methods can reduce the nine days spent locating the flagship SoC's DMA bug. In this book **validation** checks a design on real hardware rather than a model: post-silicon on fabricated parts, *or in the lab on a prototype* (glossary; Chapter 20 states it in full). An FPGA prototype is therefore pre-silicon in *time*, because it runs before any part is fabricated, and validation in *kind*, because by that definition the hardware carrying the design is real. It is the first platform on which real software runs against real hardware, and the first on which bug cost is determined by lab debug ergonomics rather than by simulation's ergonomics.

The flagship SoC example uses four FPGA boards in a rack, a fan tray and a serial console. In this example, nine months before tape-out, it boots an OS on the host cluster's four application cores, mounts a filesystem over the board's Ethernet port, and reaches a shell prompt. The OS, drivers, boot ROM and RTL run together for the first time, successfully for about forty seconds. Then a benchmark that submits work to the compute cluster returns a result whose last cache line is stale. A rerun passes, while forty repetitions of the benchmark fail twice.

The verification team cannot reproduce it: their DMA and coherence tests pass, while the failing workload spans hundreds of millions of cycles, beyond their simulator's reach this quarter. The software team's driver is the same driver that has run for six months against a functional model without a single failure. Available evidence: one console, a few LEDs, and a trace buffer configured for a different question three bitstreams ago. Two teams spend nine days deciding whose bug it is. The flagship's DMA must split every transfer into bursts that the AXI address rules allow, none of them crossing the 4 KB boundary, the address line that AXI forbids a burst to cross so that no burst spans two subordinates. The DMA stage that performs that split is called the burst splitter, and on the flagship it mis-splits a 2D transfer descriptor at that boundary. A transfer descriptor is the record software writes to program one DMA transfer. The flagship therefore inherits the reference SoC's canonical bug, the book's recurring defect. Here the same defect sits one layer deeper: coherence hides the corruption until writeback, because the second half of the split writes a line the host cluster holds Modified. A cache holds a line Modified when it has written to that line and has not yet returned it to memory. The bug itself was straightforward, but locating it took nine days because neither team could see it on the prototype.

CHAPTER MAP

§18.1 compares prototype and emulator speed, capacity, turnaround and visibility to guide workload placement. §18.2 covers what virtual platforms can establish about firmware before stable RTL, and what they structurally cannot. §18.3 locates the hy-

brid boundary and the bugs it deletes. §18.4 explains what real drivers and boot sequences find beyond constrained-random testing, and why their limits make them complementary. §18.5 covers triage across two organizations and defects belonging to the specification.

18.1 The prototype as a different instrument

An FPGA prototype maps the SoC’s RTL onto one or more commodity FPGAs. It is widely used for early hardware/software integration because it gives hardware-speed execution of large RTL blocks, supports real I/O connectivity, and lets software be brought up on a cycle-accurate model of the design ^[1]. *Hardware-speed* means megahertz: platforms of this kind run roughly a hundred to a thousand times faster than an RTL simulator, turning an operating-system boot from an impossibility into a few hours of runtime on a large SoC ^[2]. *Real I/O* connects the console to a real UART, the network to a real PHY and storage to a real device, exercising the actual driver rather than a testbench substitute. *Cycle-accurate* means no abstraction sits between the software and the design it programs.

A 2026 report describes one such flow. Its FPGA validation suite of 1,738 tests spans three classes: integration tests checking the RTL against the FPGA platform itself; bare-metal benchmark and litmus tests; and OS-level tests exercising the driver, the operating system and the software stack; within it, a full Linux boot of a dual-core design clocked at 20 MHz on the board completes in 115 seconds ^[3]. That is two minutes on a design far smaller than the flagship, for a boot that RTL simulation would never finish.

Observability limits

Limited internal visibility defines the prototype more than speed or capacity. Observability on an FPGA is restricted to a few thousand internal signals, which must be selected before the bitstream is generated; reconfiguring that choice means recompiling the bitstream, which may take several hours ^[2]. In simulation, every net is visible for free after the fact, without rerunning anything (Chapter 3). Constrained internal probing and trace capacity make prototyping emphasize system-level execution and functional realism; deep debug is selective or deferred to a higher-visibility environment ^[1].

Deferring debug requires reproducing the failure in simulation or on an emulator, which the opening example’s teams could not do, costing nine days.

Synthesis, partitioning and compilation of a complex design can take many hours or even days, and instrumentation increases that cost: probes are logic competing for the same fabric and routing resources as the design [1]. Adding a probe, rerunning and inspecting takes seconds in simulation but a working day here, with each iteration exposing only the signals selected at its start. Toolchains offer incremental builds, automatic partitioning and signal multiplexing, and static and dynamic probes [4], but visibility is still decided before the run and changes require compilation. Signal multiplexing is the sharing of one trace resource among several internal signals, one group at a time.

Capacity, partitioning and clock ratios

A large SoC does not fit in one device. Partitioning across several FPGAs introduces machinery absent from the design: bridges between partitions, serializer/deserializer links across the board, and synchronization logic to hold the pieces together, increasing communication latency across FPGA boundaries and integration complexity [3]. A NoC is the on-chip packet network that takes the place of a bus once too many blocks share one path. A crossbar or NoC path costs materially more cycles across a partition boundary than within one device, changing arbitration and back-pressure behavior relative to silicon. Performance measured on a partitioned prototype is a lower bound with an unknown gap, never a projection.

The system clock must drop by some factor to close timing in fabric. The peripheral domain drops with the system clock, so the 4:1 ratio between the reference SoC's 200 MHz and 50 MHz domains (Chapter 13) survives: dividing both by the same factor preserves the relationship. The always-on domain's 32 kHz reference is a real board crystal, whose frequency does not scale. The 6,250:1 ratio between those two domains (200 MHz over 32 kHz) therefore becomes 6,250 divided by the system slowdown factor, changing every timing relationship across that boundary.

The canonical wake-cause bug occurs at that boundary: a 3-bit code crosses from the always-on island to the system domain through per-bit synchronizers, briefly exposing reserved code `3'b110` during a legal `3'b010` → `3'b100` transition. Reproduction depends on how many fast-domain sampling edges fall within the slow domain's transition window, and the clock change alters that number. A prototype that never reproduces a crossing bug and one that reproduces it constantly can be the same design at two clock settings. Structural CDC analysis found this bug in seconds (Chapter 13); the prototype is not the instrument for it, neither when it never reproduces the crossing nor when it reproduces it constantly.

RTL requirements for prototyping

Prototypability is a design property that requires planning for RTL constraints and FPGA platform substitutions. The RTL must be technology-independent enough to synthesize for a fabric it was not written for: no vendor-specific memory macros or standard-cell instantiations, no latch-based structures the FPGA has no equivalent for, no manually gated clocks the fabric's clocking resources cannot honor, and no analog. Everything at the chip's physical edge is substituted: bringing a design up on

a board means interfacing the platform's own memory controllers, Ethernet, console UART, host DMA over PCIe, SPI for firmware boot and JTAG for debug, until the result is a complete software-bootable system [3].

The flagship's LPDDR4 controller and PHY cannot be prototyped, although their training state machine is among the riskiest things on the chip: the PHY is analog and its physical timing determines its behavior, so the board replaces them with the platform's own memory controller. The prototype can run the OS but establishes nothing about memory training, a bring-up risk that must be discharged at the analog and mixed-signal boundary (Chapter 21) and in the post-tape-out lab (Chapter 20). The plan must record an explicit prototype-scope exclusion, as Chapter 5 demands for every gap.

Prototype and emulator roles

Both platforms run RTL on hardware, and both reach megahertz-scale effective speeds on large SoCs, with the achievable frequency depending on design size, partitioning, instrumentation and the selected debug configuration [1]. An emulator provides capacity and controlled observability through triggers, selective trace, save and restore, with compile and debug infrastructure sized to the design (Chapter 17). A prototype provides speed and realism, with observability limited to instrumentation inserted before the build. *Emulation* synthesizes and partitions the design, then compiles it onto a hardware execution fabric where it runs under stimulus a host supplies; what that fabric runs is a model of the design rather than a fabricated part, so the work stays pre-silicon. The 2024 study reports sharply rising emulation adoption above a billion gates [5], a capacity threshold where prototype partitioning becomes a substantial project.

An emulator serves as a fast simulator with reduced visibility; a prototype approximates early silicon with more visibility than silicon; the two sit one on each side of the verification/validation line. Work limited by simulation speed belongs on an emulator, while lab work awaiting a chip belongs on the prototype.

Admission criteria for prototype workloads

A prototype workload must satisfy all three conditions:

1. **The workload needs more cycles than simulation can deliver.** Examples include OS boot, full initialization, encoding a thousand video frames and sustaining a network link for minutes. Workloads that fit in simulation should run there, where debug is cheaper and coverage recording already exists.
2. **Its oracle works with limited visibility.** Pass or fail must be decided without a waveform, using an offline bit-exact diff, self-checking firmware or an in-fabric monitor latching violations into a readable register. A workload whose only oracle was "the engineer watched the waveform" has no oracle at all here (Chapter 3).
3. **Its failure is plausibly reproducible in a higher-visibility environment,** because debug will occur there.

Skipping condition 2 leaves no usable oracle, while condition 3 determines whether failures can reach a suitable debug environment. In this example, failing condition 2 yields a red LED and a week of ownership argument, the subject of Section 18.5.

Scenario. The video codec’s encoder passes its block-level environment: a bit-exact golden model checks every macroblock against the reference encoder, and the coverage model closes on prediction modes, quantiser steps and buffer-full conditions. The golden model is the one trusted because it is the specification itself; the coverage model is the multi-dimensional region that its closure conditions and their values mark out. The untested sequence is a thousand consecutive frames of real content, with rate control adapting across scene changes while DDR traffic shifts. In simulation that is weeks. **Approach.** The encoder runs on the prototype, receives a real clip from a file and writes the bitstream to storage; the entire output is then diffed offline against the bitstream the reference encoder produces, in one comparison on a workstation after the run. **Why.** It satisfies all three admission conditions. The workload needs the cycles; its offline bit-exact oracle needs no visibility; and if frame 738 differs, its number and the encoder state that produced it suffice for a simulation testcase using the same input.

A bug escape is a design flaw that survives verification into a later stage, found on the wrong side of the boundary that was meant to catch it. In the FPGA-target market segment, designs ship in fabric, and the lab, hours away, routinely serves as the primary validation vehicle; many teams minimize pre-lab verification to reach that lab quickly, and the strategy worked for simpler devices but is increasingly impractical for modern SoC-class ones; in the 2024 Wilson Research Group FPGA study, only 13 percent of projects in that segment reported zero non-trivial bug escapes into production ^[6]. Using the prototype as the verification instrument gains only speed: it gives up simulation’s control, visibility and reproducibility for an environment that has no control, no visibility and only weak reproducibility.

The same study ran twice before that edition and reported an unchanged result: in 2020, and again in 2022, seventeen percent of all FPGA projects in its sample achieved no non-trivial bug escapes into production ^[7, 8].

18.2 Virtual platforms

A prototype requires synthesizable RTL, a partition that closes and a board, so it arrives months after firmware development needs a target. A **virtual platform** fills that gap: a software model of the whole system that runs the real firmware binary fast enough for development, long before stable RTL.

At the start of a project the only models available are high-level architectural specifications; abstract virtual models of the individual IPs come next, serving primarily as the framework against which software and firmware are developed and checked. These are highly abstract software models that preserve only the basic functionality of the hardware/software interfaces, and what they support is correspondingly *coarse-grained* checking of hardware/software use cases ^[2].

The platform centers on an instruction-accurate model of the core: it executes the target instruction set faithfully and updates architectural state correctly, with no commitment to how many cycles an operation takes. Functional models of the memory map surround the core model, providing a register file per peripheral with the side effects the architecture document describes. That is enough to boot an OS and debug software as if it were running on real hardware, with considerably more controllability and observability than debugging the same code over a JTAG port; this costs execution speed, but may be the only option while the hardware is still under development [9].

Capabilities and structural limits

A virtual platform establishes four things at once: that the initialization sequence sets the registers it means to set, in an order the hardware accepts; that firmware's view of the memory map agrees with the architecture's; that the handlers connect to their intended sources; and that device tree, linker script and boot ROM hand-off agree. The device tree is the data structure that describes the hardware to the operating system at boot, and the linker script is the file that decides where each section of a compiled program sits in memory. On the flagship that is four cores' worth of OS bring-up, a driver stack and a filesystem, debugged with full state inspection, none of it waiting for RTL.

The platform's limits follow directly from preserving only the basic functionality of the hardware/software interfaces. It has no arbitration, because there is nothing to arbitrate. It has no back-pressure, because a model that returns immediately never stalls. It has no latency, so two operations that the design will overlap are sequential in the model. It has no coherence, because a single flat memory has nothing to be coherent with. Bugs involving *two things happening at once* are absent by construction, so the platform reports success without testing them.

Two omissions produce firmware that passes on the model and fails on hardware:

- **Untested timing assumptions.** A poll loop that reads a status register until a bit sets terminates on the first read against a model that completes instantly, leaving the lack of a timeout and memory barrier unnoticed. A `udelay(10)` that was chosen because it felt safe is never tested against a device that takes longer.
- **Untested synchronization.** Firmware that hands a buffer to a DMA engine without a cache maintenance operation works perfectly on a platform with no caches. The omission surfaces on the prototype, where it looks exactly like a hardware coherence bug and is disputed as one.

A second weakness appears when the virtual platform becomes the specification by default. When model and RTL disagree, neither is golden: both were written by people reading the same document, so a disagreement is as likely to be two misreadings as one bug. Where they agree wrongly, a common-mode error remains, an error the two artifacts share, and comparing them cannot bring it to light (Chapter 3). Chapter 15's structural fix generates model register behavior, decode logic and the firmware header from one register description, reducing disagreements to behavior rather than layout.

Scenario. The GPU's driver stack is developed for eight months against a functional model: four shader clusters, a command processor, a unified memory. Submission, context switching, allocation and fault handling all work, and the driver team's suite is green. On the prototype, a compositing workload tears a frame roughly once every few thousand submissions. **Approach.** Diagnosis should begin with assumptions the model could not test, before suspecting the driver. The untested assumption concerns ordering between the command processor's descriptor read and the shader clusters' writes to the same memory: these are serialized by construction in the model but not in the design. **Why.** With no concurrency, the model could neither confirm nor deny the ordering assumption, making this bug unreachable in it. When a prototype failure looks like a race, diagnosis should first examine what the software development platform could not model, because that omission allowed an untested assumption to form.

Arm IHI 0050, the *AMBA CHI Architecture Specification*, Issue H (2025), gives a coherent interconnect a seven-state cache model; the specification then states the cache state transitions twice over, as a table per transaction type: once at a requester and once at a snooped cache ^[10]. A requester is the cache or device that asks the interconnect for a line; a snooped cache is one that the interconnect then questions about a line it may hold. Every one of those rules applies on the prototype but has no counterpart in the driver's development platform, so missing cache maintenance reaches the board looking like a hardware bug.

Approximately timed models

Between instruction-accurate models and RTL, *approximately timed* models add latency estimates, arbitration policies and bus delays; slower than untimed models but far faster than RTL, they support architectural exploration and buffer sizing.

An untimed model offers no timing to trust, whereas an approximately timed model supplies plausible timing numbers that can be quoted and that still differ from the design's. Tuning a firmware queue depth, a batch size or an interrupt coalescing threshold against this timing produces a number that reflects the model rather than the design; review slides should identify those numbers as model estimates. An interrupt coalescing threshold is the number of events that a device waits for before raising one interrupt. An approximately timed model answers *architecture* questions with ratios or trends; it cannot answer *verification* questions requiring a verdict.

18.3 Hybrid co-simulation

Hybrid hardware/software flows run components outside the verification target as fast models and components under verification as RTL, joining the two at a transaction-level boundary.

The industrial form partitions the system across two execution domains: a virtual platform on the host, running high-level or fast instruction-set models, and an engine that executes selected RTL blocks at cycle accuracy; transaction-level co-modeling channels connect those two domains, so software workloads execute quickly while the hardware behavior under verification stays at RTL fidelity [1]. The RTL side may be an emulator (Chapter 17) or, for smaller blocks and shorter workloads, an ordinary simulator; the architecture of the split and its consequences are the same either way, because the single channel that Figure 18.1 shows between the two sides carries transactions, never clock edges.

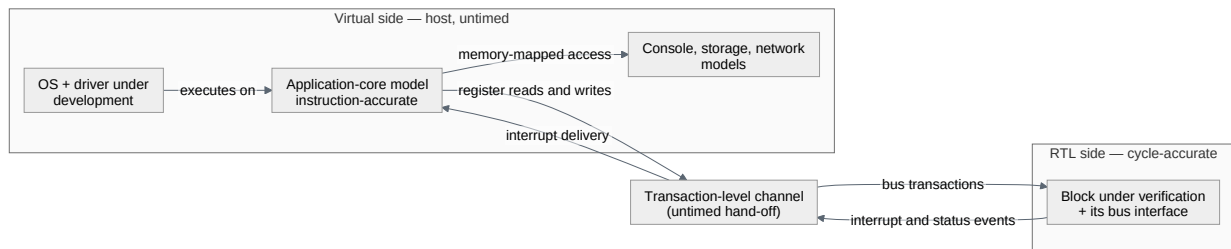


Figure 18.1 — A hybrid co-simulation partition. On the virtual side, the operating system and the driver under development execute untimed on the host, which runs an instruction-accurate model of the application core alongside console, storage and network models. On the RTL side, the block under verification and its bus interface run cycle-accurate. Everything between the two sides crosses one untimed transaction-level channel: register reads and writes go inward, interrupt and status events go outward. That hand-off is what deletes every bug that lives in the timing relationship between the two sides.

Three motivations and partition criteria

Published motivations also define boundary-placement criteria. First, hybrid co-modeling *reduces the RTL footprint* by keeping well-modeled components virtual; second, it exploits *model-maturity asymmetry* through mature virtual models, typically CPUs and standard peripherals, while retaining RTL for subtle or insufficiently modeled behavior; and third, it *balances throughput against fidelity*, accelerating end-to-end workloads while preserving cycle accuracy where timing, ordering and microarchitectural behavior matter [1].

The boundary belongs where three things are true at once:

1. **The boundary already has architectural transaction semantics:** a memory-mapped register interface, bus port or mailbox. Its contract must specify operations because the channel can only carry operations. A boundary across microarchitecture, such as a pipeline, credit counter or handshake, has no transaction to send. *Transaction-level modeling*, or TLM, means moving whole transactions between components over standard interfaces rather than signals. The TLM standard is built on SystemC and forms part of IEEE 1666, which defines Sys-

temC and supplies an interoperability layer for models written above the level of signals. A published analysis of TLM-2.0 reports that its interoperability guarantee reaches only memory-mapped bus protocols; for serial links and interrupt lines, every designer defines a payload of their own, leaving such boundaries dependent on channel definitions local to the project ^[11].

2. **The virtual side is mature**, because every run trusts it without verification throughout execution. If a model written last week to fill a partition gap has no test plan, the platform may not safely rely on that second design.
3. **All timing under verification remains on the RTL side**, as the following section explains.

Bugs excluded by the boundary

The channel between the domains is untimed. A transaction crosses as a hand-off rather than a waveform: the virtual core issues a write and the RTL side executes it in its own time. Between the moment the core issued the write and the moment the block saw it there is a relation, and that relation is an artifact of the co-modeling implementation, not a property of any system that will be built.

Therefore **every bug in the timing relationship between the two sides is unreachable by construction**. If a core and DMA contend for one memory port but only the DMA is RTL, contention is entirely absent. If a driver's write races an interrupt raised in the same cycle, and the driver is virtual, the race cannot happen. This is inherent to hybrid platforms and becomes a defect only if undocumented.

The plan row must state the exclusion as Chapter 5 requires: *“cross-boundary timing between the host cluster and the compute island is out of scope on this platform; covered by block-level full-RTL simulation and by the prototype's system regression.”* Recording it makes it a coverage decision; leaving it undocumented conceals a gap behind a successful result. As the project matures, blocks migrate from virtual to RTL, trading throughput for fidelity under control ^[1]; the boundary therefore needs a schedule and migration plan.

Scenario. The sensor hub is an always-on island with its own 32 kHz domain, a small DSP, and firmware that decides autonomously whether an event is worth waking the rest of the chip for. The verification problem spans the island's RTL, the island's firmware, and the host driver that configures the island and receives its wake events. **Approach.** The small, new island runs as RTL because its wake-decision logic carries the risk; the host cluster, OS and driver remain virtual, joined at the island's register interface and wake-event line. **Why.** The boundary is a real architectural one with transaction semantics, the virtual side is entirely stock, and the timing that matters (DSP pipeline, wake-decision register, interrupt assertion) is on the RTL side. The wake-cause *crossing* into the system domain becomes a channel hand-off, hiding the canonical multi-bit crossing bug: reserved code 3'b110 observed for one cycle during a legal transition; structural CDC analysis retains responsibility for it (Chapter 13). The platform verifies the *policy* but establishes nothing about the *crossing*.

Platform choice at two system scales

On the reference SoC, a hybrid platform is usually the wrong tool. The chip is small enough that full-RTL simulation of the whole thing is affordable, the firmware is a few thousand lines of bare-metal C, and a boot-to-main test runs in minutes. An unnecessary co-modeling channel would discard what full-RTL simulation provides free at this size: timed observation of every interaction between the core, the two DMA channels and memory. A DMA channel is an independent transfer engine with its own descriptor and state; the DMA has two of them, and both share the AXI backend.

On the flagship, full-SoC RTL simulation cannot finish an OS boot; the emulator is shared and scheduled; and the mesh NoC and LPDDR4 subsystem change for most of the project. In this example, a hybrid lets the driver team run real driver code against the real compute cluster in month four rather than month eleven. The NoC stays virtual, its timing is out of scope, and the plan row records both facts.

18.4 Firmware-driven verification

Firmware-driven verification uses the shipping binary's real boot sequence and device driver as stimulus, rather than a testbench imitation.

A constrained-random environment samples the legal space widely to check whether every legal transaction is handled correctly. Firmware checks whether a specific sequence, order, set of values and reset state achieves the software's required effect, answering for exactly one trajectory. The methods examine orthogonal aspects of the same design; firmware-driven verification covers behavior the other method structurally cannot. Hardware/software co-design and verification is the most-cited reason for using emulation or FPGA prototyping, and in 2024, 84 percent of IC/ASIC projects incorporated one or more embedded processors [5].

Defects exposed by firmware

Initialization ordering. Hardware/software interaction is rarely a single access; it is a sequence that must follow rules and respect environmental constraints, and those access constraints and preconditions are frequently *not documented* despite being required by the device hardware or by the bus systems between that hardware and the core [9]. A testbench writer reads the register map and writes accesses to it; a driver writer reads the same map and produces the sequence the software actually needs: power-up order, clock enables, a PLL lock poll, a reset deassertion, and a queue base address written before the queue is enabled and not after. Silicon executes that sequence throughout its life; verification should check whether it is the only sequence tried.

Register access races. Interface changes can introduce races as well as expose latent ones. In one published study, three variants of a driver differing only in their hardware abstraction layer and register layout were checked against each other; in one, a missing side-effect annotation introduced a race on the peripheral's input re-

gister that could make the device raise an interrupt incorrectly, despite passing the simulation-based verification previously applied to that design [9]. The race was not planted: software-side interface optimization introduced it, a change outside the hardware regression's checks.

Error-recovery paths. Real drivers contain recovery logic: timeouts, retries with shorter bursts, channel resets and reinitialization from scratch. A directed test is the kind of stimulus whose content and order are written out by hand, doing the same thing on every run. Directed tests cannot derive the driver's recovery sequence from the register map; an engineer inventing one produces a different sequence. Running the driver under conditions that activate recovery exercises its actual recovery sequence.

Integration wiring. Firmware exposes interrupts on wrong inputs, disagreement between header and decoded addresses, and resets asserted for the wrong domain. In Chapter 15's connectivity example, block environments and system regression passed, but the mismatch surfaced in the lab as a hang. Firmware is the first stimulus that traverses the whole path end to end with the software's own view of the map.

Scenario. The crypto engine sits inline on the memory path and is programmed through a key ladder: a sequence of writes that derives a working key from a root key, each stage gated by the previous one's completion. Its block environment covers every stage transition and every legal key length. On the prototype, firmware re-keying between two secure sessions occasionally produces invalid decryption, roughly once per few hundred re-keys, never reproducibly. **Approach.** Comparing the driver's actual access trace with documented engine preconditions reveals an undocumented case: a write to the next key-slot register while the previous operation is still draining. **Why.** A published peripheral check illustrates the same problem. In a formal check of an open-source SPI peripheral's access preconditions, any write to the device during an active transmission was found capable of producing divergent behavior, and the conclusion was that firmware must implement mechanisms to avoid generating such a pattern [9]; this requirement on software was discovered in the hardware itself, and it is written down neither in the hardware's documentation nor in the software's. Neither engine nor driver is wrong; the interface contract is incomplete, as Section 18.5 discusses.

Defects outside firmware coverage

Firmware follows one path, using the burst lengths the driver chooses, the descriptor shapes the driver builds, the interrupt configuration the product ships, and the buffer alignments the allocator happens to produce. It never issues legal transactions that the driver has no reason to issue, it never applies the back-pressure of a hostile responder, and it never varies stimulus, because variation is not the driver's purpose. A responder is the part of the testbench that answers the requests the design initiates, and the testbench controls what it replies.

Firmware coverage is deep along a narrow trajectory, complementing constrained-random coverage across a wide, shallow space (Chapter 9). Neither substitutes for the other: a firmware-only flow can miss defects exposed by a different driver’s legal transactions, while a random-only flow can miss a deadlock in the shipping initialization sequence even when individual transactions are legal.

At AMD, test intent written in the Portable Test and Stimulus Standard was compiled into UEFI and bare-metal tests, and those tests ran unchanged on an emulated AMD APU and then on post-silicon parts, reaching the hardware through an in-house runtime layer called uDREX, which abstracts the BIOS and starts the test on every enabled thread; on the emulator, the test and uDREX were backdoor-loaded into DRAM; and in the lab, the team used the same descriptions to compose fresh power-management firmware tuning scenarios instead of the fixed set a shipping driver happens to exercise ^[12]. Backdoor-loaded means written straight into memory by the platform, without the bus transfers that a processor would issue to put it there. The slide deck that reports the AMD work gives no bug count, speedup or effort saving; what the deck does record is test generation time that the authors warn has the potential to increase exponentially. A lab engineer or architect can compose a system-level sequence in the afternoon it is needed, supplying variation the driver alone structurally cannot. Chapter 9 gives the language itself its own treatment.

Firmware is not an oracle for the hardware. A driver checks that it achieved its own goal, not that the design behaved correctly. Retry logic can silently absorb a hardware bug, such as a descriptor succeeding only on its second attempt, leaving a passing boot with a defect the software never reports. Firmware-driven runs therefore need their own checkers.

Worked example: a protocol monitor in FPGA fabric

The reference SoC’s DMA carries this book’s canonical protocol property, and the flagship’s DMA inherits that property unchanged, because the flagship’s burst splitter is subject to the same discipline at the 4 KB boundary (Arm IHI 0022H.c §A3.4.1) ^[13].

```
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && awready && (awburst == 2'b01)) |->
      (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

A concurrent assertion is a clocked, temporal assertion, evaluated at clock ticks on the values sampled at those ticks and checked on every run that executes it. Concurrent assertions are a simulation and proof construct; there is no assertion engine in the fabric, and where a toolchain offers synthesizable assertion support, what it can report is bounded by the same visibility limits as everything else. The check must become logic reporting its verdict through a driver-readable register.

```

// Prototype-side transliteration of a_no_4kb_crossing: the same expression,
// evaluated in fabric, latching evidence into registers the driver can read.
module dma_4kb_monitor #(
    parameter int unsigned ADDR_W = 40,    // the flagship's physical address width
    parameter int unsigned CNT_W = 16
) (
    input logic          clk,
    input logic          rst_n,
    // write-address channel of one DMA manager port, observed passively
    input logic          awvalid,
    input logic          awready,
    input logic [ADDR_W-1:0] awaddr,
    input logic [7:0]     awlen,
    input logic [2:0]     awsize,
    input logic [1:0]     awburst,
    // exposed through the prototype's debug register block
    output logic          viol_seen,
    output logic [CNT_W-1:0] viol_count,
    output logic [ADDR_W-1:0] first_addr,
    output logic [7:0]     first_len,
    output logic [2:0]     first_size
);
    logic viol;

    // Identical to the property's consequent, negated: INCR bursts only,
    // 16-bit arithmetic, evaluated on an accepted address handshake.
    always_comb begin
        viol = awvalid && awready && (awburst == 2'b01) &&
            ((16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) > 16'd4096);
    end

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            viol_seen <= 1'b0;
            viol_count <= '0;
            first_addr <= '0;
            first_len <= '0;
            first_size <= '0;
        end else if (viol) begin
            if (!viol_seen) begin
                viol_seen <= 1'b1;
                first_addr <= awaddr;
                first_len <= awlen;
                first_size <= awsize;
            end
            if (viol_count != {CNT_W{1'b1}}) begin
                viol_count <= viol_count + 1'b1;    // saturate rather than wrap
            end
        end
    end
endmodule

```

The waveform is lost: when `viol_seen` comes back set, it provides one address, one length and one size, but no picture of what the interconnect was doing around them. Details of every violation after the first are lost, which is why the counter is needed: `viol_count` distinguishes *the driver is systematically emitting illegal descriptors from one corner case fired once*, while the sticky bit `viol_seen` alone answers a much weaker question. The monitor consumes fabric logic and routing resources shared with the design and debug probes.

The check is evaluated at the instant of the violation, at full speed, on every burst of a workload no simulator will ever reach; detection distance (Chapter 8) between the design’s mistake and the verdict is zero cycles, even though knowledge of the verdict waits until the driver next polls. This satisfies condition 2 of Section 18.1: an oracle that works despite limited visibility. Monitors must be built before the bitstream; adding them afterward requires compilation.

18.5 Bug ownership and triage

A prototype failure presents as a hardware or software defect; in this example, both teams arrive at triage with passing results and no evidence about the other: the hardware team’s block environments, assertions and coverage all pass, while the software team’s driver has run for months against a virtual platform without a failure. Each team’s evidence concerns its own artifact and establishes nothing about their interface, leaving interface bugs unresolved. In this example, neither team’s normal workflow supplies triage evidence, leading to a week of alternating hypotheses.

Arranging that evidence in advance costs far less than reconstructing it afterward. Four things make prototype triage tractable.

1. Repeatable runs. Simulation replay is a command line with a seed, design revision and tool build (Chapter 12). A prototype run has no seed: it has real time, real I/O, a real network peer, and a boot that consumes entropy. The nearest equivalent is a manifest fixing all bitstream and boot dependencies: RTL revision, bitstream hash, partition assignment, clock configuration, firmware and OS image hashes, boot arguments, board identity and input data. Without it, a failure attributed only to Tuesday’s build cannot guide diagnosis.

When deterministic replay is unavailable, repeating the run is what checks whether its result is stable. The 2026 flow of §18.1 encounters this limit: its OS boot is a single indivisible test that cannot be distributed across the boards the rest of the suite parallelizes over, so it is executed eight times instead to establish that the result is stable [3]. This applies Chapter 12’s repeat-*N* stability check where determinism cannot be assumed, to distinguish intermittent failures from mistaken observations.

2. One timeline. Firmware writes a log; in-fabric monitors latch into registers; the trace buffer captures bus activity. If those three record time in three different ways, an engineer establishing whether the driver’s timeout preceded or followed the DMA’s error must guess. A common timestamp source turns three records into one ordered timeline: a free-running counter readable by software and sampled by the monitors at a resolution finer than the events to be ordered. The always-on domain’s 32 kHz reference is not that counter. Adding the counter is cheap before the build and impossible within an already built bitstream.

3. A shared definition of “the same run”. Monitors and harnesses re-engineered to meet a hardware engine’s timing and capacity constraints leave divergent enviro-

onments; that divergence introduces subtle inconsistencies in observed behavior, complicating debug and making an issue harder to reproduce across platforms [1]. Checks on both platforms must be the *same checks*, generated from one source rather than implemented twice. The 4 KB monitor of Section 18.4 and the assertion it was transliterated from belong to one description; maintaining them separately risks divergence over whether a burst was legal.

4. A path back to visibility. A prototype failure becomes useful for debug as a simulation testcase, the escalation used in post-silicon bring-up (Chapter 20), one platform earlier. The manifest identifies the image and the monitor identifies the first offending transaction; someone must then be able to construct a stimulus that reproduces that transaction where every net is visible. Without that path, every prototype bug remains at the worst observability available before the lab, the trade described in Section 18.1.

Specification defects at the hardware/software interface

Triage must evaluate prototype failures against the specification rather than the two implementations, yielding three outcomes rather than two:

- **The specification says X and the hardware does X.** The software is wrong. The driver requires correction.
- **The specification says X and the hardware does Y.** The hardware is wrong. The RTL requires correction and a test that would have caught the defect.
- **The specification does not say.** *Neither team owns it*, because this is a specification defect. Filing it as a firmware bug misclassifies it.

The third case has a structural cause, independent of diligence. Peripheral access constraints follow from internal architecture; a designer who naturally satisfies them without stating them has no prompt to document them. Formal checks of open-source peripherals illustrate the documentation gap: one device’s software reset proved not to be implemented at all, a fact recorded in the HDL source and absent from the documentation [9].

An open-source UART’s baud-rate counters run continuously, and if the rate is reprogrammed from a higher target value to a lower one while the counter sits between the two, the counter is left un-reset until it wraps and the device is unresponsive for up to 2^{18} clock cycles; **software cannot avoid this because nothing in the interface lets it infer the counter’s value** [9]. In this example, “the console stops responding after the driver changes baud rate” looks exactly like a driver bug, costing the firmware team days searching for a missing wait. No correct driver can avoid this hardware defect, which the document omits; the only fix available to software is a workaround that must be specified before it can be written.

CXL.cache, the inter-device coherence protocol of the Compute Express Link standard, was modeled formally from its English prose; modeling identified five places where the text was inconsistent, redundant, inefficient or unclear, and the authors proposed wording for each of the five; the engineers drafting the protocol confirmed that four of those proposals would be incorporated into the next version [14]. This

protocol family differs from the coherent interconnects in the book's examples, and the result concerns no implementation: it identifies questions the document could not answer, the third triage outcome at protocol scale.

Among the root causes of logical and functional flaws in the 2024 study, changing, incorrect or incomplete specifications were a common concern of verification engineers and project managers, alongside design error ^[5].

Scenario. The radar front-end's firmware computes per-channel calibration coefficients at start-up and writes them into the FFT chain's correction registers. On the prototype, detections are consistently off: targets appear at roughly twice the expected amplitude, and the threshold logic of the constant-false-alarm-rate (CFAR) detector behaves accordingly. The DSP team demonstrates the datapath is bit-exact for every input it was given, against its reference model, the independent executable of the specification that is run on the same stimulus and compared with the design. The firmware team demonstrates the coefficients are correct, checked against the calibration procedure. **Approach.** Triage should examine the register description rather than dispute the two implementations. The correction register is 16 bits wide, and the document says only "signed coefficient". The RTL uses Q2.14, with two integer bits and fourteen fractional bits, while firmware computes Q1.15. Both are self-consistent, both match the document, and the values differ by a factor of two. **Why.** This is the third triage outcome. Neither software nor hardware is wrong: the field interpretation was undocumented, and the two teams each supplied the missing half from their own conventions. The remedy requires a register-description entry, not a patch; it generates decode logic, model and firmware header (Chapter 15), alongside a plan row asserting the interpretation to prevent recurrence with the next 16-bit "signed coefficient".

Filing a specification defect as a firmware bug leads to a driver patch while leaving the document wrong for the next driver, derivative and customer. The third outcome belongs in Chapter 5's spec-hole log, which exists for questions the specification cannot answer, plus three artifacts: the document change, a checker enforcing the newly stated rule, and a plan row that owns it. Prototype triage also needs a named owner spanning both teams, on a rotation drawn from both. Without these provisions, the example's ownership dispute costs a week per incident.

IN PRACTICE

If only the two or three prototype builders can operate it, undocumented bitstream bring-up makes the platform unavailable to its intended teams; removing that dependency takes priority. Automation from RTL revision to programmed board should be a deliverable. Contended board access needs a project queue with a preemption policy set before the week it matters; bitstream builds should run overnight in a separate pipeline, outside the per-commit tier (Chapter 12), which excludes multi-hour jobs.

Probes left after one investigation can obscure which connections remain active, so each build needs an instrumentation manifest. Firmware and RTL version drift can produce image mismatches; in this example, they account for half of prototype failure reports, and a day's work on boot-time hash checking eliminates them. A customer demo may compete with DMA debugging in the same week because the prototype also serves as the demo machine; priority is a management decision rather than an engineering one. A block left off the prototype because it could not be mapped, such as the analog PHY or the trimmed memory macro, risks being overlooked until bring-up; it must remain on the plan as an explicit exclusion with a named alternative.

PITFALLS

1. **Reading the prototype as a measuring instrument.** Symptom: coverage goals or sign-off criteria assigned to prototype runs, or latency numbers from a partitioned build quoted in an architecture review. Cure: cross-device links and reduced clocks make those latency figures an upper bound with an unknown gap, and closure belongs where the design is observable and stimulus controllable; the prototype validates integration and software.
2. **Debug instrumentation decided after the first failure.** Symptom: every investigation begins with a multi-hour rebuild to add probes. Cure: instrumentation should address anticipated questions before the build, with fabric budgeted for it.
3. **Firmware developed only against an untimed model.** Symptom: the driver first encounters back-pressure, latency and concurrency on a platform with limited visibility. Cure: driver code should run against RTL early on a hybrid platform, even if only for the block whose timing it depends on.
4. **A hybrid partition whose deletions are undocumented.** Symptom: a passing system campaign hides an unidentified, untested cross-boundary interaction. Cure: every partition needs a written scope exclusion on its plan row, as for any coverage decision.
5. **Prototype runs with no manifest.** Symptom: a failure is attributed to an unidentified old bitstream. Cure: each run needs its RTL revision, bitstream hash, image hashes, clock configuration and board identity recorded, plus repeated indivisible tests to establish stability.
6. **Filing a specification defect as a firmware bug.** Symptom: the driver gets a workaround, the document stays silent, and the next derivative repeats the failure. Cure: triage must use the specification; an unanswered question requires a spec-hole entry, document change and checker, rather than a patch.

SUMMARY

- An emulator is a fast simulator with reduced visibility; a prototype approximates early silicon with more visibility than silicon, but its observability constraints push deep debug elsewhere ^[1].

- Visibility on a prototype is decided before the run: a few thousand signals, chosen ahead of the bitstream, and changing the choice means a recompile measured in hours [2].
- A prototype workload must need the cycles, retain an oracle despite limited visibility and be plausibly reproducible in a higher-visibility environment.
- Virtual platforms preserve the basic functionality of the hardware/software interfaces and support coarse-grained checking of use cases [2], which is why the class of bugs involving two things happening at once is unreachable there.
- A hybrid platform's untimed boundary deletes every bug living in the timing relationship between its two sides. This inherent exclusion becomes a defect only when omitted from the plan row.
- Firmware-driven runs expose initialization order, interface races, recovery paths and integration wiring beyond testbench stimulus; a driver follows one unvarying trajectory and cannot supply a generator's variation. They complement constrained-random stimulus; they do not replace it.
- Hardware/software access preconditions are often required by the device and absent from its documentation [9]. If the specification cannot answer a prototype failure, a driver patch leaves a specification bug that will recur for the next team.

EXERCISES

Exercise 18.1 (calculation). §18.1 keeps three clock domains in view: the reference SoC's 200 MHz system domain, its 50 MHz peripheral domain, and the always-on domain's 32 kHz reference, which is a board crystal. Assume a prototype whose system clock has been divided by a factor of ten to close timing in fabric, with the peripheral domain dropping alongside it. Compute both domain ratios as the prototype presents them, and state which of the two the division preserves and which it changes. State what the change does to the reproduction of the canonical wake-cause bug, and which instrument §18.1 leaves responsible for that bug.

Exercise 18.2 (judgement). A peripheral's virtual model and its RTL disagree on a register side effect, and each team offers its own artifact as the reference. Decide what the disagreement establishes about which artifact is wrong, using the argument §18.2 gives against the model that becomes the specification by default. State the three outcomes §18.5 requires triage to reach instead, and which of them leaves the defect owned by neither team. Name the structural fix §18.2 takes from Chapter 15, and the one class of error that neither the fix nor the comparison can surface.

Exercise 18.3 (artifact). Read the register readback below, taken from the in-fabric monitor of §18.4 after a long firmware-driven run on the prototype. Decide what the readback establishes about the run and what no readback of this form can establish. State which of the two questions §18.4 distinguishes is the one the counter field answers, and the admission condition of §18.1 that the monitor satisfies.

Monitor:	DMA write-address channel, one manager port, observed passively
viol_seen:	set

```

viol_count:    at its maximum value, saturated
first_addr:    the address of the first offending burst
first_len:     the length field of that burst
first_size:    the size field of that burst
Waveform:     (none captured; the trace buffer was armed for another question)

```

Exercise 18.4 (calculation). §18.1 reports a published flow in which a full boot of a dual-core design clocked at 20 MHz on the board completes in 115 seconds. Compute the number of design cycles that boot executes, and compare the result with the length of the failing workload in the chapter’s opening. State which of §18.1’s three admission conditions the comparison settles for a workload of that length, which two it leaves open, and what the opening reports as the cost of failing them.

Exercise 18.5 (artifact). Read the triage record below, filed against a prototype failure on the console path. Decide which of §18.5’s three outcomes the record supports, and state what the filing line gets wrong about the fix available to software. Name the three artifacts §18.5 requires once the outcome is classified correctly, and the log of Chapter 5 that holds the question the document cannot answer.

```

Symptom:       the console stops responding after the driver lowers the
                baud rate, for a bounded interval, and then recovers
Software evidence: the driver follows the documented programming sequence and
                omits no wait the document states
Hardware evidence: the baud-rate counters run continuously, as the source
                shows, and are not reset when the rate is reprogrammed
Document says:  (nothing about reprogramming while a counter sits between
                the old and the new target value)
Filed as:       firmware bug

```

Exercise 18.6 (judgement). A hybrid partition is proposed for the flagship: the compute cluster runs as RTL, the host cluster and its driver stay virtual, and the two are joined at the cluster’s credit-based flow-control interface through a channel model written for the purpose. Decide which of §18.3’s three boundary-placement conditions the proposal fails, and give the reason §18.3 attaches to each failure. Name the boundary §18.3 would accept in its place, and state what the plan row must record once the boundary is placed there.

FURTHER READING

From the corpus. ^[1] covers the platform taxonomy of §18.1 and §18.3: prototyping, ASIC-class emulation and hybrid co-emulation; capacity, speed and visibility trade-offs; compilation versus execution; and environment divergence that hinders cross-platform reproduction. ^[2] places these platforms on the pre-silicon-to-post-silicon observability gradient and explains virtual-model roles and bitstream recompilation costs when visibility changes. ^[3] describes an end-to-end FPGA validation flow: platform integration, multi-device partitioning and a three-class suite spanning integration checks, bare-metal benchmarks and OS-level driver tests. ^[9] treats the hardware/firmware boundary as a formal object and supplies §18.5’s central lesson on undocumented access preconditions. ^[5] and ^[6] give the industry framing. ^[4] works

the acceleration-ready testbench question for readers crossing into Chapter 17's territory.

Outside the corpus. D. Amos, A. Lesea and R. Richter, *FPGA-Based Prototyping Methodology Manual*, treats design-for-prototyping at book length, covering partitioning, clocking and memory substitution. P. Rashinkar, P. Paterson and L. Singh, *System-on-a-Chip Verification: Methodology and Techniques*, contains a chapter-length account of hardware/software co-verification flows. F. Ghenassia (ed.), *Transaction-Level Modeling with SystemC*, describes the modeling styles used to build virtual platforms and hybrid boundaries.

REFERENCES

1. **P14** T. Rahman, S. Saha, A. Y. Alhurubi, S. K. Saha, F. Farahmandi, and M. Tehranipoor, "Emulation-based System-on-Chip Security Verification: Challenges and Opportunities," arXiv preprint arXiv:2604.15073v1, April 2026.
2. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
3. **P25** S. Ahmed, A. Kropotov, R. I. Genovese, B. Homs, E. Merino, F. Urbani, et al., "Verification and Validation (V&V)-in-the-Loop for RISC-V Design: The Holistic Vision of BZL," arXiv preprint arXiv:2604.27013v1, April 2026.
4. **D33** A. Grove, "UVM hardware assisted acceleration with FPGA co-emulation," Proc. DVCon Europe, 2015.
5. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
6. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
7. **R4** H. D. Foster, "2020 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2020.
8. **R6** H. D. Foster, "2022 Wilson Research Group FPGA Functional Verification Trends," Siemens EDA white paper, 2022.
9. **T1** M. Schwarz, "Formal Hardware/Firmware Co-Verification of Optimized Embedded Systems," Ph.D. dissertation (Dr.-Ing.), Technische Universität Kaiserslautern, Germany, 2021.
10. **S26** Arm Limited, "AMBA CHI Architecture Specification, ARM IHI 0050, Issue H," Arm Limited, September 2025.
11. **D43** G. Delbergue, M. Burton, B. Le Gal, and C. Jegou, "Analysis of TLM-2.0 and its Applicability to Non Memory Mapped Interfaces," Proc. DVCon U.S., 2016.
12. **D29** P. Gupta and M. Vax, "Test driving Portable Stimulus at AMD," Proc. DVCon U.S., 2019.
13. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
14. **P27** C. Tan, A. F. Donaldson, and J. Wickerson, "Formalising CXL Cache Coherence," in Proc. 30th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25), 2025, pp. 437-450.

19. Gate-Level, Timing and Power-Aware Verification

Gate-level verification distinguishes failures of the synthesized design from failures of its testbench, while timing and power-aware runs address implementation delays and power intent. The reference SoC illustrates the distinction with two regression results.

The first is a boot test that fails at time zero with unknowns. Nine hours of debug establish three things. The testbench was written against RTL, where no scan-enable pin existed, so it never drove the netlist's scan-enable pin. A scoreboard comparison using `!=` reported success on all-unknown data for two of those hours. A white-box checker on the DMA's internal descriptor pointer can no longer bind, because synthesis dissolved the register it watched; that pointer addresses transfer descriptors, the records that program each DMA transfer. Such a checker is attached by a `bind`, the SystemVerilog construct that instantiates a checker into a design scope without changing the design's source. These findings concern the testbench and establish nothing about the design.

The second result prevents a tape-out escape, a bug that crosses a sign-off boundary undetected. With power intent loaded, a sleep/wake test shows the event unit losing its interrupt-mask configuration after power-down. Chapter 7 treats that loss as its canonical escape, and plain RTL simulation could not represent it because it does not model power loss.

The same engineers obtained both results on the same platform in the same week. Only the second run could have exposed that bug in this flow; the first served only to diagnose the environment. The cost of gate-level simulation makes this distinction a budget decision.

CHAPTER MAP

§19.1 identifies what gate-level simulation finds beyond RTL and which traditional obligations other tools discharge better. §19.2 distinguishes design failures from testbench failures at gate level. §19.3 covers SDF back-annotation, timing-check semantics, static timing analysis's authority on timing, and the contribution of simulation. §19.4 develops formal, checkable power intent and sequencing between domains. §19.5 budgets gate-level and power-aware runs and checks that can start earlier.

19.1 The role of gate-level simulation

Four arguments for gate-level simulation (GLS) differ in strength.

Synthesis correctness. Running functional testbenches on the netlist verifies the synthesis tool, a task that formal equivalence checking and static timing analysis perform better ^[1]. Chapter 15 explains why: equivalence checking exhaustively checks the transformation without stimulus, including synthesis defects such as a wrong result in a wide arithmetic operator that affects a vanishing fraction of inputs and is impractical to find by simulation. Projects should run logic equivalence checking after synthesis, which discharges this justification for GLS.

Timing. Section 19.3 explains why GLS cannot establish timing correctness. Static timing analysis examines every timing arc in the netlist structurally, across process corners, voltage and temperature, and on-chip variation ^[2]; a simulation exercises the paths one test happens to sensitise, at one corner, with one set of annotated delays.

X and reset problems hidden at RTL. Chapter 13 established the mechanism: RTL simulation is optimistic about unknowns at every decision, because `if`, `case` and the edge constructs take a definite branch on an ambiguous condition. In a netlist, gate primitives and flip-flop models replace those constructs without exhibiting their optimism ^[3]. So a design that passed at RTL because an unknown was silently resolved will fail at gate level, at reset, where the unknowns live. The cost is that replacing those constructs also makes pessimism worse: the netlist returns an unknown where the silicon would produce a definite value. The worsened pessimism is Section 19.2's problem.

Structures absent from RTL. Scan chains, test-mode multiplexers, compression logic, self-test wrappers and the debug fabric's boundary are inserted *after* RTL sign-off; no RTL simulation has seen them, so the obligation that every one is inert in functional mode cannot be checked earlier. A two-flop synchronizer is a pair of chained flip-flops in the destination domain, and the first of the pair is given time to settle before the second samples it. A structure inserted for one purpose can degrade another: scan insertion on the *second* flip-flop of the pair adds multiplexer logic on the metastable path, that logic erodes the settling window, and the eroded window degrades mean time between failures; scan insertion on the first flip-flop of the pair has no such effect ^[2]. This finding could not have come from simulation because it describes a structural property of the implemented netlist.

Two sources on demonstration and sign-off

Two sources appear to disagree over the role of GLS.

Metastability is the unstable condition of a flip-flop sampled too near a change, which resolves to a legal value only after an unpredictable delay. One class of metastability bugs is impossible to demonstrate on an RTL model in simulation; reproducing them requires a gate-level model with timing, typically unavailable until late in the flow ^[4]. Yet when back-end processes interfere with synchronizer operation, the answer is *not* gate-level simulation on a back-annotated netlist but structural and timing analysis of the final implementation ^[2].

The sources address different questions. GLS with timing can *exhibit* one timing violation: one path, one corner, one alignment of edges. It cannot *cover* the class, because whether a given alignment occurs in a given run is an accident of the stimulus. Sign-off requires coverage of the class, which structural netlist analysis provides; simulation gives one sample. GLS can therefore demonstrate a failure, but cannot sign off its absence.

A proposed gate-level run should distinguish two purposes: exhibiting a phenomenon nothing else can produce, or discharging a plan row, the line of the verification plan that names a feature, the method that covers it and the evidence that closes it. The first is legitimate and occasionally irreplaceable. For obligations within its scope, static analysis supplies exhaustive evidence; the plan row should identify that scope.

Scenario. The flagship’s LPDDR4 subsystem includes a PHY training state machine that runs at link bring-up, and the team’s plan carries a row for it. The proposal is to run training at gate level with back-annotated delays because it involves timing. **Approach.** The plan row should be split into separate obligations. The functional obligation is to reach a trained state from every starting condition and report failure correctly otherwise; it is verified at RTL, where a thousand seeds are affordable. Static timing analysis establishes that trained delay settings satisfy setup and hold at every corner; simulation cannot improve that coverage. Exactly one obligation remains for gate level: the training sequence on the real netlist with real delays must not violate a timing check that the constraints declared impossible. Section 19.3 returns to those declarations, the timing exceptions that static timing analysis obeys without checking. **Why.** The undivided row would have cost days of gate-level runtime and discharged neither obligation properly: too few seeds to be functional evidence, one corner to be timing evidence. The split costs less and produces two supported arguments instead of one unsupported argument.

The gate-level uses considered here are unknowns at reset, structures present only after synthesis, power intent on a netlist, and checks on timing constraints.

19.2 Reading a gate-level failure

Chapter 13 defines the semantics of unknowns, and Chapter 12 defines the term *optimistic*: RTL simulation is optimistic, resolving an ambiguous condition into a definite branch. Synthesis removes those constructs and their optimism, but worsens pessimism: the netlist evaluates far more expressions, and each of them is all-or-nothing about unknown operands [3]. The first gate-level run can expose unknowns caused by design defects or by the environment. Triage must distinguish these origins before design debug. Triage is the work of turning failures into facts before any of them is debugged: each failure is classified as a design bug, a testbench bug or an environment failure, and assigned to an owner.

Four failure families and their signatures

Unreset state. The tell that identifies this family has three parts: the unknown is present at time zero, it survives reset de-assertion, and it traces back to a flip-flop with no reset pin in the netlist. Formal reset analysis finds this class earlier and more completely, because it reports every register still unknown when the reset period ends (see Chapter 13); a GLS run that discovers one therefore identifies a gap in the entry gate. The entry gate is the set of checks a design passes before it earns a gate-level run.

An unreset register is not necessarily a defect.

Scenario. The radar front-end's FFT pipeline holds several stages of butterfly registers, deliberately left unreset: they are pure datapath, the reset tree would be large, and every one of them is overwritten by the first frame that flows through. Butterfly registers are the pipeline registers that make up those stages, each holding an intermediate result on its way through the pipeline. At gate level the first run floods the whole chain with unknowns for the first frames, and a junior engineer opens a bug. **Approach.** The unreset datapath is not a bug, but that conclusion requires an argument rather than an assumption. The argument has one premise: no control decision consumes any of those registers before the flush completes. The checks establish that the valid chain is reset, the threshold logic of the constant-false-alarm-rate (CFAR) detector reads only post-flush samples, and no comparator output feeds a state machine during the flush window. The plan row should record the argument, with an assertion that no downstream control signal is unknown once the valid chain has been high for the pipeline's depth. **Why.** The unreset datapath is a legitimate area decision and it costs nothing in silicon. Verification must permanently establish that the unknown never reaches a decision; judging waveform X values to be flush data cannot discharge that obligation. The assertion detects later changes that violate the argument.

Pins that exist only in the netlist. The netlist adds pins of its own: scan-enable, test-mode, scan inputs, built-in self-test (BIST) enables and the clock-multiplexer select for at-speed test. An at-speed test runs the netlist at its functional clock speed rather than at a slower test clock. The unknown starts at a top-level input with no RTL counterpart, and its cone of influence is the netlist region that input can reach. This is missing harness infrastructure that could not have been written earlier, rather than a design defect or, strictly, a testbench bug. The harness should be built once as a named functional-mode tie-off module and reviewed as an artifact: tying `test_mode` to the wrong value does not produce an error, but produces a netlist that passes tests while running in a mode excluded from the shipped device.

Environment infrastructure that cannot survive a netlist. White-box checkers depend on a specific implementation and cannot be used in gate-level simulation [1]: synthesis dissolved or renamed the signal, or replicated it into three. The same applies to any hierarchical reference: the path of a checker bound into the design, a register model's back-door path (see Chapter 10), a `force` on an internal net. A back-door access reaches the register's simulation constructs directly by that hierarchical

path, instead of driving real bus cycles through the design. The failure appears in environment messages: an elaboration error, a null object handle, or a back-door read returning unknowns while the front door works. A null object handle is a testbench variable that should point at an object and points at nothing. Chapter 11 gives the standard answer: bound checkers carry macro guards so they can be compiled out of a gate-level run, and the checks that must survive are the ones written against ports.

Pessimism artifacts. The tell is that the unknown appears mid-logic with no source: no unreset register upstream, no undriven pin, just an expression whose operands were partially unknown. The remedy is to clear real sources first, in order, and re-run; each one removed collapses a large fan-out of artifacts.

Comparisons that accept unknown data

The third family includes environments that silently stop checking without failing. A scoreboard's comparison operator decides whether an unknown is a failure or a pass.

```
// `!==' is case inequality: its result is always 1'b0 or 1'b1, so an
// unknown in `act` always reaches the report. `!=` is logical inequality
// -- 1'bx when the relation is ambiguous -- and `if (1'bx)` takes the
// false branch, scoring the transaction as correct. A 2-state variable
// holding that result would convert the x to 0 by a second route.
always_comb begin
  if ($isunknown(act))
    $error("unknown in sampled data: %0b", act);
  else if (act !== exp)
    $error("mismatch: got %0h, expected %0h", act, exp);
end
```

For the logical equality and inequality operators, the result is 1'bx only when unknown or high-impedance bits make the relation *ambiguous*; the case operators include those bits in the comparison and always yield a known result [5]. A value corrupted only in bits that already differ from the expectation is still reported, because the relation is not ambiguous. Two values escape: the fully corrupted one, and the partially corrupted one whose unknowns sit precisely on the bits that would have carried the difference.

The checker can still report an unambiguous comparison, and such reports are no evidence that it rejects the ambiguous ones. The failure is silent: the run passes, coverage is collected, and the plan row is discharged. The testbench should be audited for this defect before its first netlist run. The two-line remedy checks every sampled value explicitly for unknowns before comparison, giving unknown data its own verdict rather than treating it as equal. A sampled value is the value the scoreboard captured at the sampling edge, before any comparison.

The order of operations

A gate-level run has three independent choices: timing annotation, power intent, and netlist stage; enabling them together makes their interaction part of debug. Gate-level bring-up is the phase in which the environment and the design first exchange

checked transactions. It should use a zero-delay netlist without power intent and the suite's shortest reset-and-boot test; nothing is added until that test passes, so each subsequent addition has exactly one suspect. The first clean zero-delay boot belongs in the plan as a prerequisite for the remaining work in this chapter.

19.3 SDF back-annotation and what a timing check asserts

RTL has no delays. A netlist has library delays, but they are estimates computed for an average output load, and in a real netlist every instance of the same cell drives a different number of inputs over wires of different lengths. Back-annotation replaces these estimates with per-instance values from the physical netlist or layout geometry, delivered in a Standard Delay Format file, so each gate instance and pin-to-pin connection carries its own delay ^[1].

What SystemVerilog takes from the file is precisely bounded: specify path delays, specparam values, timing-check constraint values, and interconnect delays, with the values themselves coming from delay-calculation tools that use connectivity, technology and layout geometry ^[5]. Everything else in an SDF file is ignored.

Three annotation failure modes

Missing annotation. Any timing value for which the SDF file provides no value is left unmodified, retaining its pre-annotation value, and the annotator is required only to warn about data it cannot annotate. The annotator that reads the file is a system task, and it takes its scope from one of its own arguments. A call that targets the wrong instance, or a block whose hierarchy differs from the SDF, annotates nothing there and leaves the subsystem on library defaults, frequently zero. The run fails no checks and is fast, but provides no timing evidence for that block. The annotation log records every individual annotation ^[5]; review should compare its annotated-instance count against the netlist's.

Unselected corner. A string argument selects which member of each min/typ/max triple gets annotated, and its default value leaves the choice to the simulator; a separate argument applies scale factors to the triples ^[5]. The operating corner, which determines whether a gate-level run says anything about setup or about hold, is therefore a defaulted parameter of a system task, easily inherited from a script unread for two years. A run with an unselected corner provides no evidence about any corner. The run manifest (see Chapter 12) should name the corner alongside the seed and tool build.

Annotation cost. Netlists contain millions of gates and millions of pin-to-pin connections, each needing annotation, so the process is slow; annotating once at compile time and running many test cases against the same compiled model avoids repeating annotation per test ^[1]. Annotation cost is the reason a gate-level tier is organized as a small number of long runs rather than a large number of short ones, and it interacts badly with per-test randomization.

Timing-check semantics

The system timing checks fall into two groups. One group defines a stability window with respect to a reference signal and checks when the data signal transitions relative to that window: `$setup`, `$hold`, `$setuphold`, `$recovery`, `$removal`, `$recrem`. The other group measures the difference in time between two events: `$skew`, `$timeskew`, `$fullskew`, `$width`, `$period` and `$nochange`, the last of which involves three events rather than two. Evaluation uses two events: a *timestamp* transition records the time, and a *timecheck* transition triggers evaluation; `$setuphold` combines setup and hold windows in a single check [5].

By themselves, these checks only print violations. The mechanism that makes a timing violation *functional* is the notifier:

```
// Abridged excerpt from a library flip-flop model. The rows handling
// unknown inputs are elided; in a real cell they outnumber these, and they
// are where gate-level pessimism comes from.
primitive dff_udp (q, clk, d, notifier);
  output q; reg q;
  input clk, d, notifier;
  table
    // clk d ntf : state : q
    r 0 ? : ? : 0 ;
    r 1 ? : ? : 1 ;
    f ? ? : ? : - ;
    ? * ? : ? : - ;
    ? ? * : ? : x ; // any notifier event drives the output to x
  endtable
endprimitive

module dff_with_timing_check (output wire q, input wire clk, input wire d);
  reg notifier;
  dff_udp u_ff (q, clk, d, notifier);
  specify
    // These limits are the library's defaults; SDF back-annotation
    // overwrites them per instance.
    specparam tSU = 120, tHLD = 40;
    $setuphold(posedge clk, d, tSU, tHLD, notifier);
    (posedge clk => (q += d)) = 90;
  endspecify
endmodule
```

Every timing check can take an optional notifier, a variable the check toggles whenever it detects a violation, so the model can make its behavior a function of violations; the canonical use drives the device's output to `x` [5].

A timing violation injects an unknown into the datapath, which propagates to a functional failure elsewhere, potentially thousands of cycles later. In a run with twelve early timing violations in one block, those violations cause the final scoreboard failure and cannot be treated as benign. Timing-check output should be resolved in time order before any functional mismatch is debugged in a back-annotated run.

Static timing analysis is the authority

Static timing analysis checks all legal timing parameters in a gate-level representation: it analyses the netlist structurally to identify every possible timing arc, it compares those arcs against a timing-constraints file that states the required timing, and it makes that comparison not against one set of parameters but across process variation, minimum and maximum voltage and temperature, and on-chip variation; of those corners, the best case is the one where hold analysis is critical, and the worst case is the one where setup analysis is critical [2].

Gate-level simulation exercises paths one test sensitizes, at one corner, under one annotation. It cannot report margin or cover unsensitized arcs, and cannot establish that a path passes at typical but fails slow unless it runs at both. Using GLS for timing sign-off incurs runtime cost and false confidence.

Checking timing constraints with simulation

The constraints file that STA reads is *manually generated* to match the device specification, and enhanced through synthesis, place-and-route and clock-tree synthesis as the netlist evolves [2]. Every exception, including false paths, multicycle paths and case-analysis settings, is a human claim that STA obeys without checking. A case-analysis setting is a constant that the timing tool is told to assume on a pin. STA stops analyzing a path declared false.

A back-annotated gate-level run makes no such assumption: it applies the real path delay and evaluates the real timing check. If the exception was wrong, the check fires. GLS with SDF thereby checks that constraints describe the implemented design; it does not establish timing sign-off.

Scenario. The neural accelerator’s weight-decode path is declared a multicycle path in the constraints. The argument for the declaration is that the 16-lane MAC datapath consumes a weight only after several cycles of set-up for one convolution job, the accelerator’s unit of work. The declaration is correct for 8-bit weights. At 2-bit weights the decoder produces four weights per fetch and the datapath runs one result per cycle, so the same path is single-cycle. STA reports timing met, because it was told not to look. **Approach.** One gate-level run should exercise the accelerator’s quantization suite at the narrowest precision, back-annotated at the slow corner. \$setuphold violations appear on the decode registers within a few hundred cycles, the notifier drives unknowns into the MAC array, and the scoreboard mismatches. The fix is in the constraints file, not the netlist. **Why.** RTL simulation could not find this because the path has no delay at RTL. STA could not find it either because it accepts the exception as a premise. A mode-dependent exception illustrates this gap between the two methods: gate-level tests should exercise *configurations the constraints made assumptions about*, rather than maximize exercised logic.

19.4 Power-aware verification

Hardware description languages were built to capture what a design computes. They cannot describe how elements are powered, and power gating and multiple supply voltages made power architecture a functional matter rather than a physical one; that shift required a separate description: IEEE Std 1801-2024, *IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems* (UPF) [6]. UPF, the unified power format, is that description: a machine-readable statement of a design's power intent. Power intent must live outside the design source that every tool treats as authoritative, because the same RTL is instantiated into different power architectures by different integrators [7], and because a description of the power architecture has to be readable by synthesis, place-and-route, static checkers and simulators alike.

A **power domain** is a collection of instances treated as a group for power-management purposes, typically sharing a primary supply set. **Isolation** provides defined behaviour for a signal whose driving logic is not active, implemented by an **isolation cell** that passes values normally and clamps its output to a specified value when its control asserts. A **level-shifter** translates a signal from one voltage swing to another. **Retention** is enhanced functionality on selected sequential elements so their values survive the power-down of the primary supply, and a **power state table** states the legal combinations of supply states [6]. The section concerns the interaction of these five elements over time.

Power intent and simulator value resolution

Loading power intent changes the simulator's value resolution. The change has three parts. The first part is that when a domain powers down, every logic node inside it is corrupted to unknown and stays unknown until the domain powers back up; the second is that an isolation cell on an output port stops that unknown reaching powered-on logic by driving the clamp value the power intent specifies; and the third is that a retention register is corrupted like everything else, except that the value saved before entry is restored on exit [3].

Corruption supplies checks without separately written checking logic. Injecting unknowns into a powered-down domain exposes unprotected crossings: unknowns appear there and downstream logic starts misbehaving shortly after the driving domain powers down [7]. A missing isolation cell is diagnosed through its resulting failure rather than a rule.

The standard makes corruption level a declared property rather than a fixed behavior. A **simstate** specifies the operational capability of a supply state. The ladder runs from `NORMAL` (full switching with characterized timing), through `CORRUPT_STATE_ON_CHANGE`, `CORRUPT_STATE_ON_ACTIVITY`, `CORRUPT_ON_CHANGE` and `CORRUPT_ON_ACTIVITY`, to `CORRUPT`, where the supply cannot even hold existing state. The four names in the middle differ on two points: whether corruption takes the stored state alone or every value the logic drives, and whether it follows a change of value or any switching. The predefined power state `ON` carries simstate `NORMAL`; `OFF` and `E`

`RROR` both carry `CORRUPT` (IEEE 1801-2024 §4.8) [6]. Each declared state specifies whether logic still switches, holds its value without switching, or cannot even hold its existing state.

The reference SoC's two power domains

The reference SoC has two power domains: an always-on island with the real-time clock and wake-up logic, and a switchable system domain containing everything else; both use one nominal voltage, requiring no level-shifters. This deliberate two-domain simplification separates isolation, retention and sequencing from voltage; the flagship supplies the multi-voltage example. Every crossing is shown in [Figure 19.1](#).

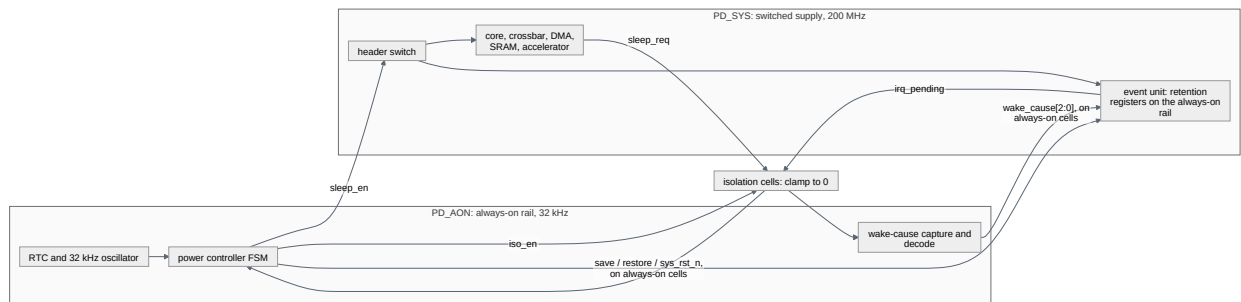


Figure 19.1 — The reference SoC's two power domains and every signal that crosses between them: an always-on island carrying the real-time clock, the wake-cause capture and the power-controller state machine, against a switched domain holding the core, the crossbar, the DMA, the SRAM, the accelerator and the event unit. The crossings are not symmetric. Signals leaving the switched domain pass through isolation cells, because their drivers vanish; signals entering it need no clamp, but they do need a route built entirely of always-on cells, and the isolation enable itself needs the same kind of route.

The crossing requirements differ by direction. Signals leaving the switched domain for the always-on island, including the core's sleep request and event unit's pending-interrupt flag, must pass through isolation cells (IEEE 1801-2024 §4.4.4): their drivers vanish while the island runs and would otherwise feed it unknowns. Signals entering the switched domain from the island (the wake-cause code and the controller's `save`, `restore` and `reset`) need no clamp because their destination is unpowered and its value irrelevant. They need a route built entirely of always-on cells, and the isolation enable needs such a route too, because its own cell must stay powered in order to clamp. A routing tool breaks such a path if it inserts an ordinary buffer: one industrial design was found with ordinary buffers rather than always-on cells inserted on isolation-enable and wake-up signals, corrupting the wake-up path [7]. The event unit's retention cells use the always-on rail, while normal-mode storage uses the switched supply; this is why one survives power-down and the other does not [6].

The sequences below specify the required hardware ordering. [Table 19.1](#) places the shutdown and restoration steps side by side.

Table 19.1 — The power-down and power-up sequences for the reference SoC’s switched domain, one column each, in the order the hardware requires. Three of the steps carry published failures, which is what to read the columns for: isolation must be asserted before the sleep enable and released after it, reset must already be asserted when the supply returns, and reset must be inactive at the restore edge — which is why step 3 of the power-up column has an order inside it.

Power-down	Power-up
1. Quiesce: no outstanding bus transactions	1. Assert the domain reset, while power is still off
2. Gate the domain clock	2. Close the header switch; wait for supply-good
3. Assert <code>save</code> : retention captures state	3. Release reset, then assert <code>restore</code>
4. Assert isolation: outputs clamp	4. Release isolation
5. Open the header switch: power off	5. Un-gate the clock

Three of those steps carry published failures. Isolation must assert before the sleep enable and de-assert after it; reversing either order makes the always-on island sample an unknown throughout the overlap. Reset must already be asserted when the supply returns. In one design, reset was asserted *after* the sleep enable de-asserted, and the register output was corrupted for exactly the window between the supply arriving and the reset landing. And reset must be inactive at the restore edge, which is why step 3 of the power-up column has an order inside it ^[7].

All three failures come from Samsung Electronics’ published checklist of its own low-power practice, which records specific failures rather than deductions or generalizations. Its isolation table alone runs to eleven numbered checks, enumerated as named failures: a missing isolation cell, a redundant isolation cell, a normal isolation cell sitting inside a block that switches off, an isolation cell whose output floats, an isolation control signal of the wrong polarity, an isolation control signal tied off to a constant, and further entries for a cell type or a cell name that disagrees with the declared power intent. The retention checks run the same way, down to a save or restore control taken from the wrong source or tied off to a constant ^[7]. These published failures belong on the checklist §19.4 has been assembling, each as a check against the declared power intent.

Worked example: the event-unit escape and its assertion

The canonical event-unit bug of Chapters 3 and 7 lives on the boundary between steps 3 and 4 of the power-up column.

The event unit’s retention strategy carries a *restore-period condition*: a predicate that must hold throughout the restore window. The condition is “isolation still asserted”: a half-restored register drives invalid data, and only the clamp keeps it out of the always-on island throughout restore. The standard is normative about what happens if the condition lapses: it shall be true throughout the entire restore period (IEEE 1801-2024 §6.53), otherwise the restore operation fails and the retention element is corrupted ^[6].

The reference SoC’s wake path has a fast-wake optimisation. The always-on wake logic drops isolation enable in the cycle a wake event arrives if supply-good is already

asserted, saving 32 kHz cycles of wake latency by bypassing the release state of the controller sequencer. The sequencer is the state machine that walks the power-up column one step at a time. The qualification appears safe because the wake event that starts power-up arrives while the supply is still off and cannot fire the bypass; on every ordinary wake, the sequencer releases isolation itself at step 4. Failure requires a *second* wake event after supply-good, inside the restore window opened at step 3. That one qualifies, so the bypass lowers isolation enable a cycle before the sequencer would. The restore-period condition becomes false mid-restore, corrupting the retention element and destroying the event unit's interrupt-mask configuration. The wake event coincides with isolation release, as Chapter 7's post-mortem recorded.

Plain RTL cannot represent this failure: without power-down, corruption or retention, restore always succeeds. Power-aware simulation can represent it, but still needs stimulus that hits the race. The assertion below states the restore-period condition on the control signals themselves, so it fails as soon as isolation drops inside the restore window, whether or not the corruption is modeled; under formal proof it also needs no stimulus that hits the race.

```
// Bound in the always-on domain. Every signal here lives in the always-on
// domain, so this checker keeps running while the switched domain has no
// power -- which is the whole point.
module pd_sys_sequence_chk (
    input wire clk_aon,      // 32 kHz always-on clock
    input wire rst_aon_n,    // always-on reset, active low
    input wire iso_en,       // 1 = switched-domain outputs are clamped
    input wire sleep_en,     // 1 = the domain's power switch is open
    input wire restore,      // level-sensitive retention restore, active high
    input wire sys_rst_n     // switched-domain reset, active low
);
    // Power may be removed only from an already-isolated domain.
    property p_iso_before_sleep;
        @(posedge clk_aon) disable iff (!rst_aon_n) $rose(sleep_en) |-> iso_en;
    endproperty
    a_iso_before_sleep : assert property (p_iso_before_sleep);

    // Isolation may be released only once power is back.
    property p_iso_after_power;
        @(posedge clk_aon) disable iff (!rst_aon_n) $fell(iso_en) |-> !sleep_en;
    endproperty
    a_iso_after_power : assert property (p_iso_after_power);

    // The clamp must hold for the whole restore window: this is the
    // retention strategy's restore-period condition, stated as an assertion.
    property p_restore_isolated;
        @(posedge clk_aon) disable iff (!rst_aon_n) restore |-> iso_en;
    endproperty
    a_restore_isolated : assert property (p_restore_isolated);

    // Reset must already be asserted when the supply returns.
    property p_reset_before_power;
        @(posedge clk_aon) disable iff (!rst_aon_n) $fell(sleep_en) |-> !sys_rst_n;
    endproperty
    a_reset_before_power : assert property (p_reset_before_power);

    // One cover per antecedent (Chapter 11's rule). Each is a rare, deliberately
```

```

// sequenced event, so an empty cover is a finding rather than a formality.
c_sleep_entry   : cover property (@(posedge clk_aon) disable iff (!rst_aon_n)
                        $rose(sleep_en));
c_iso_release   : cover property (@(posedge clk_aon) disable iff (!rst_aon_n)
                        $fell(iso_en));
c_restore       : cover property (@(posedge clk_aon) disable iff (!rst_aon_n)
                        restore);
c_supply_return : cover property (@(posedge clk_aon) disable iff (!rst_aon_n)
                        $fell(sleep_en));

endmodule

```

The module holds four assertions. All four are overlapping implications on the always-on clock, each of them has a cover on its antecedent, and all of them check only always-on signals. An overlapping implication is an assertion form whose consequent is checked in the same cycle as its antecedent, rather than in the next one. The checker observes only the power controller’s *outputs*, never the switched domain, so it remains active and evaluating while that domain is unpowered. `a_restore_isolated` fails on the canonical escape in the cycle the fast-wake path fires, rather than microseconds later at the next read of the event unit’s configuration.

Because these assertions concern a small always-on finite state machine with a handful of inputs, they are suitable for formal proof (see Chapter 14); proved once, they hold for every wake-event arrival time, precisely the randomization that the escape’s post-mortem identified as missing. And because the assertions are written against controller outputs rather than against a netlist, they run unchanged at RTL, which answers the scheduling question the escape raised: power sequencing does not need to wait for gate level.

State that must not survive power-down

Retention decisions must justify exclusions as well as inclusions.

Scenario. The crypto engine sits inside a switched power domain, and an unwrapped AES key sits in the ordinary registers of its key ladder, the chain that derives a usable key from a stored one. During power-intent capture, the domain’s retention strategy is applied broadly to “retain the configuration registers”, and its filter includes the key registers. **Approach.** Two negative power-intent obligations should be stated as properties rather than review comments. The key registers must *not* be in the retention strategy, so that powering the domain down destroys the key, which is the entire security value of powering it down. The isolation strategy must clamp `key_valid` to 0; clamping to 1 would tell the always-on island that a key is loaded while its domain is unpowered, so the clamp value is a functional decision. **Why.** Tests that check correct encryption after power-up miss this defect because retention leaves the key available. A directed power-aware test catches it by powering down and up, attempting encryption without reloading the key, and requiring failure. The second obligation establishes that isolation clamp values cannot be treated as don’t-cares. Clamping a `busy` output to 1 hangs the interconnect; clamping a request line to 1 launches a transaction into a dead do-

main; clamping an error flag to 0 hides a fault. Each clamp value needs a plan row and rationale explaining how receiving logic uses it throughout power-off.

Always-on routing after layout

Scenario. The sensor hub is a 32 kHz always-on island with its own DSP, continuously fusing sensor data while its host sleeps and waking the host when a fusion threshold trips. Chapter 13 identified a bug earlier on this wake path: a reset-domain crossing *inside* the island caused a spurious wake interrupt, diagnosed statically on RTL. The next path segment fails differently at a later stage, with nearly identical symptoms in the two cases. **Approach.** Power-aware simulation should cover the *post-layout* netlist as well as RTL and the pre-layout netlist. Low-power verification has three stages, RTL, pre-layout netlist and post-layout netlist, each exposing defects the previous stage could not [7]. Here the post-layout run shows the wake request going unknown whenever the host's switched domain is off: the system never wakes, and only a watchdog recovers it. **Why.** Place-and-route routed the wake request through a repeater physically placed inside the host's switched domain, so a signal that starts and ends in always-on logic now passes through a cell with no power. That placement did not exist at RTL, did not exist in the pre-layout netlist, and is invisible to any functional test on either; the signal is logically correct at every stage before the one where it breaks. This failure motivates at least one power-aware post-layout run and structural checks of always-on routing, rather than an assumption that the routing survived layout.

Power-state growth and verification strategy

The reference SoC's two domains yield one switchable domain with two states, on and off, one power-down sequence and one power-up sequence; in this example, three people can review them around a whiteboard in an afternoon.

The flagship declares six power domains (always-on, root of trust, host cluster, compute island, NoC and memory, peripherals); dynamic voltage and frequency scaling (DVFS) provides three operating points (0.6, 1.1 and 1.5 GHz) for the host cluster and compute island, while the NoC stays at 800 MHz. The host cluster and the compute island have three operating points plus off, four states each; the remaining three switchable domains are on or off. Those five switchable domains give $4 \times 4 \times 2 \times 2 \times 2 = 128$ combinations. This is an upper bound, not the legal-state count: the power state table specifies the legal subset, and most of the 128 combinations are illegal by construction. The 128 combinations require checks of legal states and their transitions; sequencing bugs occur in transitions, which far outnumber states.

A published industrial case from Samsung Electronics in 2010 describes a 40-million-gate SoC with 30 power domains, eight of them software-controlled power-gating domains, more than 2,000 lines of power intent and more than 4,000 legal power states; it identifies power-state coverage as an explicit challenge with no satisfactory solution in the tooling of the day ^[7]. The three examples considered here stand at two, 128 and four thousand, and the following strategies can be assessed against them.

For the reference SoC with two domains, the power state table documents the states, and directed tests check the sequences. A directed test is a hand-written one: its content and its order are fixed, so it does the same thing on every run. For the flagship with 128 combinations, the proposed strategy uses the power state table as a machine-readable specification, generating from it one sequencing checker per domain rather than one written by hand, and a coverage model whose axes are power state and transition rather than data. A coverage model is the set of axes along which coverage is measured, with the values each axis may take. For a case with four thousand legal states, a strategy to assess is proving that the power-management finite state machine emits only legal sequences and checking domains against those sequences, rather than covering the product space.

DVFS adds an orthogonal complication that the state count hides. An operating point is a state; a *transition* between operating points is a voltage ramp, and the interesting failures happen during the ramp rather than at either end. A level-shifter corrupts its output when its input voltage leaves its characterized range; this documented behavior was reproduced in simulation by driving the input outside the cell's declared voltage range ^[7]. Chapter 13 noted the companion effect on the clocking side: the flagship's clock ratios move with the operating point, so a CDC abstraction model is valid for one operating point only, since the model is built for one fixed ratio between the crossing clocks.

Power-aware simulation has its own optimism

Chapter 13 described RTL simulation's optimism about unknowns. Power-aware simulation has optimism one abstraction level higher.

Simulation results reflect the implemented hardware only to the extent that the simstate declared for each power state is correct. On the ladder, `CORRUPT_ON_ACTIVITY` sits nearer `CORRUPT` than `CORRUPT_STATE_ON_CHANGE` does, and so corrupts more. Declaring `CORRUPT_ON_ACTIVITY` for silicon that is `CORRUPT_STATE_ON_CHANGE` changes results; this mismatch is conservative, but other mismatches between the declared simstate and the silicon are optimistic, making simulation show working behavior that fails in the implemented design ^[6]. If the power-aware model repeats an architectural misunderstanding, its tests can pass for the wrong reason, as when a reference model repeats a misreading of the specification (see Chapter 3). An overly generous simstate lets a domain's corruption tests pass for the wrong reason.

19.5 Run selection and scheduling

Gate-level capability must be budgeted against runtime.

Runtime calculation

Chapter 12 costed the reference SoC's nightly regression at month nine: 180 distinct tests, 900 runs, a mean of 11 minutes, 165 CPU-hours on 24 concurrent simulator licences, comfortably inside a ten-hour night. The gate-level comparison uses those 180 tests with back-annotated delays.

In this example, the team measures a 40× slowdown. Eleven minutes becomes 7.3 hours; 180 runs cost 1,320 CPU-hours, or 55 wall-clock hours on the same 24 licenses, exceeding both a weekend and a night. Even at 10×, the same 180 runs cost 330 CPU-hours, or 13.8 hours on 24 licenses, exceeding the night; break-even for the ten-hour night is around 7×. The whole plausible slowdown range therefore excludes the full gate-level suite, which is prohibitive in practice for any reasonably sized SoC [3].

The ratio also determines the size of the affordable tier. At 40×, fourteen tests cost 102 CPU-hours; fourteen runs on twenty-four licenses form one wave of 7.3 hours, leaving ten slots free and fitting the weekend tier without displacing other work. The decision is which fourteen tests to run. The example assumes an order-of-magnitude variation in the slowdown ratio with netlist size, annotation scope and how much of the environment survives; that assumption must be checked by measuring the ratio on the project's own design and tool. A DVCon India 2025 poster from Samsung on a complex SoC reports an improvement from reducing that ratio: boot dominates gate-level runtime, so the poster runs boot once with timing checks enabled and disables them for the remaining vectors, and it reports a 32 percent run-time improvement for that one change; it credits its whole set of flow optimizations with a 43 percent reduction in disk space and a 58 percent reduction in head count [8].

Three independent run choices

Three independent choices, timing annotation, power intent and netlist stage, yield the combinations in Table 19.2, answering different questions at very different costs.

Table 19.2 — The useful combinations of the three independent choices that make up a gate-level run — timing annotation, power intent and netlist stage — each paired with the question it answers and the tier it belongs in. The first row is the one teams get wrong most expensively: RTL plus power intent answers the power-sequencing question without a netlist at all, and belongs in the nightly tier from the day the power intent file exists.

Configuration	The question it answers	Where it belongs
RTL + power intent	Does the power sequence work? Is every domain crossing protected?	Nightly, from the day the power intent exists
Zero-delay netlist	Does the design leave reset with no unknowns? Are the test structures inert in functional mode?	Weekend, from first netlist

Configuration	The question it answers	Where it belongs
Zero-delay netlist + power intent	Does the <i>implemented</i> protection logic match the intent?	Weekend, once protection cells are inserted
SDF-annotated netlist, named corner	Do the timing constraints describe the design that was built?	Campaign, per corner
Post-layout netlist + power intent	Are the always-on routes really always-on?	Campaign, once per netlist drop

Power-aware verification starts at RTL and also runs at gate level; it should begin as soon as the power intent file exists. Chapter 7’s escape post-mortem reached exactly this conclusion and wrote it into the sign-off checklist. The gate-level stage adds a later check: only the netlist contains the actual isolation, retention and always-on cells, so the netlist is the only place where the inserted structures can be checked against what was specified; power-aware simulators therefore provide a mode intended for gate-level runs, where multi-voltage cells are physically present rather than inferred from the intent [7].

Selection criteria for fourteen tests

Five criteria rank candidates by priority. They identify more than fourteen candidates, several from the second criterion alone, so the budget determines the cutoff. Selection should follow this order until the runtime budget is exhausted; excluded tests must be recorded with justification.

1. **Reset and boot.** This short run is always required: it exposes unknowns and must pass before any other gate-level run provides evidence.
2. **One test per structure that does not exist at RTL.** Scan-enable inert in functional mode, test-mode multiplexers not selected, debug-fabric entry and exit, clock-gating enables. These structures first become available for simulation at the netlist stage.
3. **Every power state transition, at least once.** Gate-level power transitions check inserted cells; RTL already covered sequencing, so timing annotation is unnecessary.
4. **The configurations the timing constraints assumed something about.** Section 19.3’s multicycle example motivates annotating tests for modes the exceptions were written for or *around*.
5. **Whatever the last three escapes would have needed.** The tier can include tests that would have exposed those escapes.

The constrained-random regression is excluded regardless of budget. Constrained-random is the automatic generation of legal but unlikely stimulus under constraints. Random seeds buy almost nothing per unit cost at gate level because their value depends on scenario volume, which the slowdown removes. Ranking also reproduces existing coverage without exploring anything new (see Chapter 12), so a ranked gate-level tier expensively repeats RTL evidence.

Scenario. The USB controller’s plan carries rows for its link power states, U0 through U3 (*Universal Serial Bus 3.2 Specification*, Revision 1.1, sections 7.5.6 to 7.5.9) [9], and someone proposes running them power-aware at gate level on the grounds that they are “the power tests”. **Approach.** The two state machines should be treated separately: apart from the interaction below, they share only vocabulary. Link power states are a *protocol* obligation: the device negotiates them with a host, the specification mandates transitions and timing, and an RTL compliance suite supplies evidence. The controller’s *power-domain* state is a power-intent obligation about supplies, isolation and retention. They interact at exactly one point: entering the deepest link state permits the integrator to power-gate the controller’s domain, and the controller must then be quiesced, isolated and retained correctly. That interaction is the only plan row that earns a power-aware run, and it needs no timing annotation. **Why.** Conflating the two puts protocol tests on a slower, less effective platform and leaves power intent unchecked because the U-state tests passed. A test needs a gate-level or power-aware slot only if it requires that platform’s unique capabilities; its name cannot justify selection.

IN PRACTICE

The environment repairs in the opening example can have one owner responsible for the resulting testbench changes. The resulting tie-off harnesses, unknown checks and white-box checker macro guards permanently extend the testbench rather than serve as workarounds.

Sampling and driving through clocking blocks lets a single environment work against both RTL and gate-level models (see Chapter 12), and every hierarchical reference is a place where that environment will not work. A clocking block is the declaration that fixes the moment interface signals are sampled or driven, so the testbench cannot race the design it watches. Discovering this at netlist arrival costs twice: rewriting the environment and losing confidence when revised checkers disagree with the originals.

Timing-check waiver files require review alongside the constraints. A waiver is a written record that accepts a specific hole in the evidence, with its reason, its risk argument, its owner and its expiry. A timing violation waived as “this path is a false path” must trace to the constraints file; changes to either file can leave the waiver inconsistent with the constraints. The waiver and constraints files must therefore be reviewed together.

In this scheduling scenario, a late netlist and an extended bring-up of the first clean gate-level boot leave the nominally nightly tier only two chances to run before tape-out. Test selection therefore budgets two clean runs rather than the planned twenty. Test selection is the filter that decides, before anything is simulated, which tests earn those runs (see Chapter 12).

PITFALLS

1. **Running the functional regression at gate level.** Symptom: a weekend tier that never completes, and a slowdown blamed on the farm. Cure: the arithmetic above, once, in writing; equivalence checking discharges the synthesis-correctness argument that motivated it (see Chapter 15).
2. **Debugging a functional mismatch before resolving timing-check output.** Symptom: days spent on a scoreboard error whose cause was a `$setuphold` violation two thousand cycles earlier. Cure: resolve timing violations first, in time order, because the notifier converts each one into a propagating unknown [5].
3. **Unaudited annotation.** Symptom: a suspiciously fast back-annotated run with no violations, or an unidentified min, typ or max corner. Cure: review the annotation log, compare annotated-instance and netlist counts, and record the corner, a defaulted argument [5], beside the seed and tool build in the run manifest.
4. **Treating the power state table as documentation.** Symptom: sequencing checkers written by hand, one domain at a time, drifting from the table. Cure: generate the checkers and the coverage model from the table, so that a table edit breaks the checks rather than silently outdating them.
5. **Deferring power-aware verification until the netlist.** Symptom: a retention or isolation escape found after RTL sign-off, when the fix is an ECO. Cure: run power-aware at RTL from the day the power intent exists; the netlist stage checks the inserted cells, not the sequence.
6. **Retaining by default.** Symptom: a retention strategy whose filter is “the configuration registers”, applied without asking what must *not* survive. Cure: each retention decision needs a plan row; in the crypto example, the key registers must be excluded from retention.

SUMMARY

- Gate-level simulation’s traditional justifications have mostly been taken over by better tools: running the functional testbenches on the netlist verifies the synthesis tool, a job that formal equivalence checking and static timing analysis do better [1]. What survives is unknowns at reset, structures that exist only after synthesis, power intent on a netlist, and a check on the constraints themselves.
- Exhibiting a phenomenon and signing off its absence are different jobs. Simulating the metastability class needs a gate-level model with timing [4], and even then what appears is the timing violation on the synchronizer path: metastability itself is not observable in simulation, and only structural and timing analysis of the implementation covers the class [2].
- Initial gate-level triage must distinguish testbench failures from design failures. Triage by tell: an unknown from an unreset flop, from a pin with no RTL counterpart, from a checker that cannot survive a netlist, or from pessimism with no source. The silent failure is `!=` returning `1'bx` for an ambiguous relation, making

`if (1'bx)` take the false branch [5]; unknowns need explicit checks before comparison.

- Static timing analysis is the authority on timing; gate-level simulation is a sample of it. What GLS uniquely adds is a check on the *timing exceptions*, which STA obeys rather than verifies [2].
- Power-aware simulation checks by *corrupting*: a powered-down domain's nodes go unknown, and an unprotected crossing becomes visible as a failure downstream rather than as a rule violation [3].
- Power sequencing governs domain interactions. Isolation must precede power-down and remain after power-up, reset must precede supply return, and restore conditions must hold throughout the window; a lapsed restore condition corrupts retained state (IEEE 1801-2024 §6.53) [6].
- Power-aware simulation has its own optimism: results are only as accurate as the declared simstate, and an over-generous one produces tests that pass for the wrong reason [6].

EXERCISES

Exercise 19.1 (calculation). Section 19.5 costs the reference SoC's nightly regression at 180 distinct tests, 900 runs, a mean of 11 minutes and 24 concurrent licenses inside a ten-hour night. Compute the slowdown ratio at which those 180 tests, each run once with back-annotated delays, exactly fill that night. State which of the two run counts the derivation must take as its base, and give the ratio the other base would produce. Compare the result with the break-even figure §19.5 reports, and name the two costs §19.5 gives for those same 180 runs that confirm which base is right.

Exercise 19.2 (judgement). The first zero-delay gate-level run of the reference SoC's boot test reports no mismatch, its coverage database fills as it does at RTL, and the scoreboard whose verdict closed every RTL plan row compares sampled data against the expectation with `!=`. The team proposes to record the netlist as clean at reset. Decide what that run establishes about the netlist, and name the audit §19.2 requires before the result counts as evidence. Explain why a long history of reported mismatches from the same scoreboard leaves the objection unanswered.

Exercise 19.3 (artifact). Read the run record below, filed for a back-annotated gate-level run of the reference SoC's crossbar subsystem, against the three annotation failure modes of §19.3. Identify the two entries whose values withhold the evidence the run was launched to produce, and state for each what the run therefore cannot support. Give the two further items §19.3 requires in the run manifest beside the corner.

Netlist stage:	post-synthesis, pre-layout
Timing annotation:	SDF file, applied once at compile time
Annotator call scope:	u_soc.u_xbar_wrap
Min / typ / max member:	
Annotated-instance count:	not recorded; the annotation log was discarded


```

Timing-check violations:      none reported
Wall-clock against the zero-delay run of the same test:  shorter
Verdict recorded on the plan row: hold timing met at the fast corner

```

Exercise 19.4 (judgement). A proposal asks for one back-annotated gate-level run per corner on the netlist's two-flop synchronizers, to sign off the absence of metastability failures, on the argument that only a netlist with timing can exhibit them. Decide what such a campaign can establish and what it cannot, using the distinction §19.1 draws between exhibiting a phenomenon and signing off its absence. Identify the analysis that covers the class instead, and name the inserted structure §19.1 reports as eroding the settling window when it lands on the second flip-flop of the pair rather than on the first.

Exercise 19.5 (artifact). Read the power-intent extract below for the reference SoC's switched domain against §19.4. Decide which of its entries §19.4 requires to be stated as negative properties rather than left as review comments, and give for each the failure the entry as written produces. Identify the one entry that the section's own domain figure already licenses, and name the step of the power-up column of §19.4's sequence table that the restore-control entry contradicts.

```

Power domain:      PD_SYS, the switched supply
Retention strategy: every element matching "the configuration registers",
                  a filter that reaches the crypto engine's key registers
Isolation strategy: clamp value 1 on key_valid
                  clamp value 1 on the crypto engine's busy output
                  clamp value 0 on the event unit's irq_pending
Restore control:   asserted by the power controller one always-on clock
                  cycle before the switched-domain reset is released

```

Exercise 19.6 (calculation). Section 19.4 bounds the flagship's power-state combinations at 128. Recompute that bound for a variant in which the NoC and memory domain takes the same DVFS operating points as the host cluster and the compute island. Show the factor each of the flagship's power domains contributes, including the one that contributes none, and state what the power state table does to the recomputed figure.

FURTHER READING

Within the corpus, ^[6] is normative for everything in Section 19.4: the definitions of domain, isolation, level-shifter and retention, the simstate ladder and its corruption semantics, and the retention strategy's save, restore and power-down conditions with the failure behaviour of each. ^[7] supplies a power-intent qualification checklist, an eleven-item isolation checklist, sequencing rules for isolation and sleep enables, and an industrial case whose domain and state counts demonstrate inspection limits. ^[5] is definitive on back-annotation and timing checks: what an SDF file may carry, what happens to values it omits, the two families of check, notifiers, and the min/typ/max and scale-factor arguments that decide which corner a run represents. ^[2] covers static timing analysis and its corners between RTL sign-off and silicon, synchron-

izers' physical requirements, and why scan insertion on the second flip-flop erodes mean time between failures. [3] carries the gate-level side of the unknown problem and the power-aware corruption semantics. [1] explains the purpose of gate-level simulation and SDF, and why checks usually assigned to GLS belong elsewhere. The 2024 data on asynchronous clock domains, and the class of metastability bugs needing a gate-level model with timing, is in [4].

Outside the corpus: M. Keating, D. Flynn, R. Aitken, A. Gibbons and K. Shi, *Low Power Methodology Manual for System-on-Chip Design* (Springer, 2007) is the standard architectural treatment of the techniques this chapter verifies; S. Jadcherla, J. Bergeron, Y. Inoue and D. Flynn, *Verification Methodology Manual for Low Power* (Synopsys, 2009) is its verification-side companion and the origin of much of the sequencing discipline in Section 19.4. IEEE Std 1497 was the Standard Delay Format specification, referenced but not reproduced by the SystemVerilog standard; IEEE now lists it as inactive-reserved and names its IEC replacement, recorded in the appendix on standards, as where the format is defined today. For static timing analysis as a discipline rather than as a neighbour of simulation, J. Bhasker and R. Chadha, *Static Timing Analysis for Nanometer Designs* (Springer, 2009) is the practitioner reference.

REFERENCES

1. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. **D14** M. Litterick, "Full Flow Clock Domain Crossing - From Source to Si," Proc. DVCon U.S., 2016.
3. **D23** S. Zhao, S. Yan, and Y. Feng, "Practical Approach Using a Formal App to Detect X-Optimism-Related RTL Bugs," Proc. DVCon U.S., 2014.
4. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
5. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
6. **S10** IEEE, "IEEE Std 1801-2024, IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems (Revision of IEEE Std 1801-2018)," IEEE, 2024.
7. **D17** J. Liu, M.-S. Hong, B. H. Lee, J. Choi, H. Won, K.-M. Choi, H. Vardhan, and A. Kher, "Low Power Verification with UPF: Principle and Practice," Proc. DVCon U.S., 2010.
8. **D4** V. Vyas, R. Bondugula, P. Sahoo, S. S. Kashide, and G. Srivastava, "Novel and optimized solution to accelerate gate level simulation for complex SOC," Proc. DVCon India, 2025.
9. **S21** USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.

20. Post-Silicon Validation

Post-silicon checking combines target-speed execution with limited visibility: in the flagship example, about four thousand trace cycles cover a run of order 10^{12} cycles. The two silicon bring-ups below, eleven months apart, show how design scale changes debug.

The reference SoC's first parts arrive on a Tuesday in this example. One goes onto a debug board with a JTAG probe, a scope on the 32 kHz crystal and a serial terminal. By Thursday it powers on, releases reset, boots from ROM and blinks a GPIO. By the following Friday the DMA has moved a 2D tile from SRAM to the neural accelerator and the UART is echoing at 115,200 baud. The bring-up takes two engineers six weeks with one part on a bench. When something misbehaves the engineers halt the core over the debug module and read the DMA's `STATUS` register through the abstract memory access. The abstract memory access is the debug module's way of reading a location in the memory map, used here with the core halted. The whole chip is 512 KB of SRAM, five crossbar managers and a handful of peripherals. Because the chip is that small, the second diagnosis the engineers try is the correct one in this example.

Eleven months later, the first parts of the same company's next generation arrive: the flagship retains the register discipline and scales the same DMA architecture to eight channels. A DMA channel is an independent transfer engine inside the DMA, with a state of its own and a share of the AXI backend. A rack replaces the bench, and the part boots an operating system on four application cores; workloads combine the LPDDR4 subsystem and mesh NoC with video decode against live network traffic. A NoC is the on-chip packet network that takes the place of a bus once too many blocks share one path. One failure costs the program three weeks. Under one particular mixture of decode and traffic, roughly once every forty minutes, the system stops without an exception, error interrupt or kernel log. When the lab halts it, the host cluster is idle, two NoC queues are full, the flagship's DMA reports no error on any channel, and every configuration register reads back exactly what firmware wrote. Forty minutes is 2,400 seconds at any point on the flagship's 0.6–1.5 GHz DVFS band: the interval between failures is wall-clock time and not a count of cycles, so the operating point the part runs at does not stretch or shorten it. DVFS is the dynamic voltage and frequency scaling that trades supply voltage against clock frequency at run time to save energy, and every operating point it adds is one more the lab has to cover.

The first bring-up provided no preparation for the second; closing it depended on a decision taken fourteen months earlier rather than on a debug technique. That decision specified which buses would receive monitors, how deep the trace buffer would be, and whether the monitors would remain reachable while the compute island was power-gated. Lab observability is fixed by design decisions made months before the failure is known.

CHAPTER MAP

§20.1 explains the inversion of the pre-silicon budget: a part runs everything and reveals almost nothing. §20.2 budgets design for debug before its pre-silicon deadline, while §20.3 derives the bring-up sequence from its checks' dependencies. §20.4 develops techniques to shorten localization, the expensive step after detection. §20.5 traces every escape back to changes in pre-silicon verification; §20.6 covers fix decisions when a respin is unavailable.

20.1 Changes from model to physical part

Validation is checking a design on real hardware rather than on a model: post-silicon on fabricated parts, or in the lab on a prototype (Chapter 18's territory). It is *not* the software-engineering sense of “did we build the right thing?”, and it is not an umbrella term covering pre-silicon work. Everything before tape-out (simulation, formal, emulation, gate-level and power-aware work) is **verification**, on a model; validation begins when the object under test is physical. Much of the literature instead says “pre-silicon validation” for pre-silicon verification and quotes cost figures that combine the pre-silicon and post-silicon phases (Chapter 1 flags this scope trap when using those numbers).

Chapter 3 established the three links a bug must traverse to be found (activation, propagation, checking) and showed how controllability and observability bound the first two. Post-silicon completely inverts both budgets ([Table 20.1](#)):

Table 20.1 — What changes when the design becomes a part: cycles available, internal visibility, reachable stimulus, the cost of a fix, and what only a physical part exhibits at all. The first two rows carry the chapter's argument and are meant to be read together — pre-silicon you can see everything and run almost nothing; post-silicon you can run everything and see almost nothing.

	Pre-silicon (RTL simulation)	Post-silicon (a part)
Cycles available	~nothing. An OS boot that takes seconds on silicon takes years on an RTL simulator [1]	everything. Real workloads, real software stacks, indefinitely
Internal state visible	all of it, any signal, any cycle	a few hundred signals, for a few thousand cycles, chosen in advance
Stimulus reachable	anything the interface can express	whatever the running system and its configuration hooks can produce
What a fix costs	recompile	a new silicon stepping
What only this platform sees	nothing physical	power, temperature, electrical margin, real signal integrity, real peripherals

Every technique below uses the surplus of cycles to recover some internal visibility.

Post-silicon tests run at target clock speed, so they reach use cases nothing pre-silicon can: booting a full operating system in seconds, exercising power management and security features across many IPs at once, running under realistic on-field workloads. Because the vehicle is a physical object rather than a model, it also becomes

possible to check non-functional characteristics (power consumption, physical stress, temperature tolerance, electrical noise margin) that no functional model represents at all. As of 2017, compatibility work on a large SoC routinely spanned more than a dozen operating system flavors, over a hundred peripherals and more than five hundred applications ^[1]. This space was never simulable, so its absence pre-silicon was not an oversight.

Three costs follow; the third changes how the work is organized.

Observability. Silicon exposes only what design-for-debug hardware routes to a pin or buffer; Chapter 3 traced that loss of observability stage by stage and Chapter 1 priced it. Section 20.2 is about spending the budget.

Controllability. Silicon control is limited to exactly the configuration options the architecture defined ^[1]. An internal FIFO cannot be forced nearly full; the lab must use the provided registers and traffic, without guaranteed success. A defeature bit, if provided, replaces the testbench's `force` statement.

Fix latency, and therefore error sequentiality. Pre-silicon, debugging is naturally sequential: find a bug, fix it, find the next. In the lab, a hardware fix requires a new **stepping**, a fabrication cycle with corrected masks that Chapter 1 calls a respin, and no program can afford one per bug. When a bug is discovered (often *before* its root cause is known) someone must find a way to work around it so that validation can continue; and the workaround has to eliminate the bug's effect without masking the other bugs still waiting to be found ^[1]. Post-silicon debug therefore runs in parallel, on a shared part, with a growing stack of workarounds underneath it. Section 20.6 develops the consequences for workarounds and fix decisions.

A fourth constraint is not technical: the calendar. Post-silicon runs between first silicon and the decision to start mass production, and that date is fixed years in advance by market forecasts; missing the window can cost the entire investment in the product. The decision is made on post-silicon evidence: the trend and type of bugs found, power and performance results, tolerance to noise and electrical variation, against production quality gates that in 2017 practice required an average defect level below about 50 parts per million ^[1]. This is why lab schedules are measured in weeks, and why every technique below is judged first on how much calendar it saves.

The goal is to narrow an observed failure to an error scenario that can be investigated in a pre-silicon environment ^[1]: a stimulus short enough to simulate, on a design scope small enough to load, with the failure still present. Root-causing then uses pre-silicon visibility. Every technique should be judged by progress toward that hand-off rather than by how much it explains in the lab.

Scenario. The flagship's video codec passes every pre-silicon check: a bit-exact golden reference decodes the same streams the RTL does, and the regression carries a corpus of conformance bitstreams. In the lab, a customer's live stream produces one corrupted macroblock roughly every two hours of playback. A macroblock is one of the fixed-size blocks a decoder reconstructs a frame from. **Ap-**

proach. The lab reduces the failure instead of explaining the corruption: it captures the offending stream to memory, replays it from DRAM instead of from the network, then bisects it to a 40-frame segment that still fails and then to a single frame. It hands that frame plus the decoder’s register configuration to the pre-silicon team, who reproduce it in RTL simulation. **Why.** The existing golden oracle, the bit-exact reference model of Chapter 8, runs on the captured stream on a workstation, so the lab can decide *corrupt or not* without any internal visibility at all. Each bisection step narrows the captured stream while preserving the failure, producing the reproducer needed by the pre-silicon team. The failing frame combined a slice-boundary condition with a reference-list update never paired by the conformance corpus; Section 20.5 examines this coverage escape.

20.2 Design for debug before tape-out

Design for debug (DfD) is on-die instrumentation whose only purpose is to make silicon observable and controllable during validation. The deadline requires that DfD be specified, sized, routed and taped out with everything else, months before the first failure exists. Missing observability is typically discovered only in the lab, as a failure that cannot be root-caused, and adding it then requires a silicon spin, which is what the lab is trying to avoid ^[1]. There is no incremental path in silicon: an FPGA prototype (Chapter 18) at least allows the lab to re-select signals and rebuild the bit-stream at the cost of hours, and silicon offers no equivalent at any price.

DfD allocates a budget under uncertainty. Every observability mechanism ultimately writes into a fixed store or out through a fixed port, which gives one governing inequality: **signals × cycles ≤ buffer**. The flagship has a 64 KB trace buffer. Sixty-four kilobytes is 524,288 bits, so 128 traced signals buys 4,096 cycles of history; 256 signals buys 2,048; 512 signals buys 1,024. This fixed capacity trades observation width against depth, both decided at tape-out. Published industry practice has long used representative buffers of 128 signals × 2,048 cycles; a design with millions of signals typically traces a few hundred for a few thousand cycles ^[1]. A decade of published post-silicon work uses the same order of magnitude through trace buffers that hold roughly a thousand cycles of history, or stretch that history by recording fewer signals ^[2]. Section 20.4 applies this limit to a run 10^{12} cycles long.

The debug port is orders of magnitude slower than the logic it watches (a JTAG-class interface runs at megahertz while the design runs far faster), so continuous export is arithmetically impossible and an internal buffer is forced ^[1]. Triggers and qualified capture manage this small capacity, which cannot grow at runtime. Qualified capture records only the cycles in which a stated condition holds, so the buffer spends its depth only on the cycles that matter.

The instruments provide the following capabilities (Table 20.2):

Table 20.2 — On-die design-for-debug instruments: what each yields, what it costs, and the design stage at which it is fixed. Trace and scan are complementary rather than alternatives — a short film of a few signals against a still photo-

graph of nearly every flop — and no row of the last column falls later than tape-out, which is what makes this a budget allocated before the first failure exists.

Instrument	What it yields	What it costs	Decided by
Scan chains, reused for debug	full internal state at one stopped moment	already present for manufacturing test; needs a debug-usable stop	DfT/DfD architecture
Signal trace into an on-chip buffer	a few hundred signals over a few thousand cycles, continuous	buffer area and power; routing congestion to the buffer	signal selection, at RTL freeze
Processor trace	the instruction flow of a core, compressed	modest per core; bandwidth to transport	core integration
Bus / protocol monitors with triggers	transaction-level history at a chosen interface, on a condition	monitor logic + share of the same buffer	interconnect architecture
Hardware checkers in silicon	a stop close to the bug's origin	area, power, and timing pressure — the reason there are few	design, per block
Irritators and non-functional modes	<i>controllability</i> : forced corner conditions without the stimulus	area and power, plus the risk of a mode nobody ever exercises	design, per block
Debug fabric and access ports	reachability of all the above from outside	routing, and one port per power domain if you want it to survive gating	SoC integration

Two complementary instruments need explicit planning. **Scan chains** were built to find manufacturing defects, and they are a mature, already-paid-for architecture that also gives post-silicon observability of internal state ^[1]; scan gives a full snapshot of nearly every flop at a stopped moment, while trace records a few signals over time. Planning only one of the two leaves the other kind of observation unavailable.

On-chip checkers are the silicon descendants of Chapter 11's assertions; their value for debug is *proximity* to the bug's origin: when a hardware checker fires, the part stops near the origin of the bug, so the trace buffer still holds relevant history and the debug logic contains usable hints. In one large processor program, failures caught by hardware checkers were markedly easier to debug for exactly this reason, while their use stayed limited by area, power and timing cost ^[1]. A silicon checker shortens Chapter 8's *detection distance* by orders of magnitude, but its cost permits only a handful.

Chapter 3 gives a gray-box prescription: add provisions for forcing error and exception conditions that would otherwise be prohibitively slow to provoke ^[3]. In silicon that prescription becomes an **irritator**: logic embedded in the design that triggers microarchitectural events at random, initialized at power-on and firing as the part executes. It is the on-die counterpart of Chapter 7's testbench irritator, and by flushing a pipeline at random, or injecting an invalidation as if it came from another chip, it reaches tough corners without the stimulus that would normally produce them ^[1]. Non-functional modes do the same job statically: a cache mode in which almost

every access forces a cast-out exists solely to stress the level below it. A cast-out is the eviction of a line from a cache to the level below.

Choosing which signals to trace

Research evaluates signal sets through *state restoration*: traced values imply untraced ones, forwards through a gate and backwards through it, so the quality of a signal set is measured by the **state restoration ratio** (restored plus traced states over traced states). On a small worked circuit, tracing 2 signals over 5 cycles yields 10 traced states and 16 restored ones, a ratio of 2.6 ^[1]. A large literature optimises that number by structural analysis, simulation or learning.

There is **no known industrial report of the restoration ratio being used** to select trace signals ^[1]. The ratio takes no account of what the design *does*, so the same block in two products gets the same answer for two different sets of lab needs; it ignores architectural and physical constraints that rule signals out; and it assumes tracing is the only instrument, although trace works alongside scan, monitors and checkers. Selection still rests mainly on designer experience. For each block, trace selection should distinguish the two likeliest causes of the failure most feared.

Two conflicts: power management and security

Both conflicts are structural.

Power management. Debug data has to travel, and its route runs through logic that power management may switch off. A block can be perfectly instrumented and still invisible because the path carrying its signals to the buffer crosses an IP that is powered down at the moment of interest; that IP need have no functional relationship to the one under debug ^[1]. Disabling power management prevents debugging the power sequences themselves. The flagship instead uses a debug access port per power domain, preserving access when the first island is gated.

Scenario. A sensor hub's always-on 32 kHz island fuses accelerometer and pressure samples and wakes the DSP when a fused threshold trips. In the lab, roughly once a day the wake does not happen; the system stays asleep until the next unrelated event. **Approach.** The team wants a trace of the island's threshold logic across the wake decision; however, the DfD route from the always-on island to the trace buffer passes through the DSP island, which is power-gated precisely while the bug is armed. Two things make the debug possible: a debug access port living inside the always-on domain, and a small always-on capture of the four signals that matter (threshold comparator, event FIFO not-empty, wake request, isolation-release acknowledge) sampled at 32 kHz. **Why.** A domain's observability must be powered by that domain. Depending on a neighbor's power state removes visibility when needed; the canonical retention bug, a wake event racing isolation release, occupies exactly this blind spot (Chapter 19).

Security. DfD exposes internal state by construction, and much remains on-field so deployed systems can be diagnosed. It therefore provides a path to adjacent keys,

firmware and debug modes; published system compromises have used post-silicon observability features for access ^[1]. The flagship uses authenticated debug unlock through the root of trust, limiting trace-fabric reach after the lab; cutting DfD for security prolongs validation, increases its difficulty and can make it impossible. Chapter 23 treats the unlock path as an attack surface. The neighboring crypto engine's key ladder and trace fabric need a threat model before tape-out.

In the flagship example, both conflicts are addressed in a specification review a year before the lab exists, by engineers outside the future lab team. **Post-silicon readiness** (DfD architecture, debug software, debug boards, post-silicon test plans) is a pre-silicon workstream that starts at product-functionality planning and runs the whole length of the design cycle ^[1]. Like Chapter 5's verification plan, it needs owners and plan rows with review gates.

20.3 Bring-up: the first hours and days

Chapter 4 used **bring-up** for the pre-silicon phase in which environment and design first exchange checked transactions. Silicon bring-up applies the same idea one platform later, where two disciplines carry over: work should target demonstrations rather than infrastructure and should self-check from the very first transaction. A testbench allows probes to be added when needed; silicon does not, so check ordering must establish the evidence each verdict needs.

The first activity is **power-on debug**, the one stage when none of Section 20.2's machinery is available: if the part does not power on, the on-chip instrumentation is not running either, and visibility into the internals is extremely limited or zero. What remains is a custom debug board offering fine-grained external control, plus brainstorming. The standard method is **defeaturing** to reduce the configuration to a bare-bones system without power management, security or complex firmware boot that can power on reliably, followed by incremental **refeaturing**. A stable power-on recipe typically takes days to a week from the moment silicon reaches the lab; getting the complex features back on can take several weeks, and it gets harder the more configurable the design is ^[1]. Configurability normally helps, but every added mode also enlarges the search for a bootable configuration.

Once the part powers on, one rule orders the checks: **each must be decidable using only what preceding checks established**. Depending on unproven machinery leaves two indistinguishable failure modes: the object checked and the machinery judging it. Power precedes clocks because a PLL cannot lock on a rail that is not up. Clocks precede reset release because a reset synchroniser needs edges. Reset precedes the debug port because the port is logic like everything else. The debug port precedes every check whose oracle is a register read; after that point this is nearly all checks, since reading back a known reset value is the cheapest oracle silicon offers.

A bring-up plan is a verification artifact that needs Chapter 5's treatment: each row states one check, the evidence that closes it, and (the field a pre-silicon vplan does not need) the oracle actually available at that moment. Each plan row must specify

one metric that determines its verdict; “it seems to be running” supplies no checkable verdict.

Worked example: bring-up plan excerpt, the flagship's first week. Ten rows from a plan of about ninety. The third column is the one that makes it a *bring-up* plan.

#	Check	Oracle available here	A pass rules out	Exit criterion
B01	Rails come up in sequence; quiescent current in band	bench instruments only	shorts, missing rail, wrong sequence	all rails in spec, current in band, 5 consecutive power cycles
B02	Always-on PLL locks; divided clock present on a pin	scope, external reference	oscillator, PLL configuration	measured frequency within tolerance
B03	Reset released; JTAG identification register reads back	a constant known from the spec	chain wiring, TAP state machine	value matches, 20 consecutive reads
B04	Debug fabric reaches the always-on domain's registers	reset values from the register spec	debug decode, address map at this port	all reset values match
B05	One host core executes a three-instruction loop from boot ROM, writing a heartbeat register	the heartbeat toggles	core reset, ROM read path, first bus hop	heartbeat observed for 60 s without stall
B06	L2: write-read-verify a walking pattern across all four slices	data comparison	slice enable, L2 address decode	pattern verified in every slice
B07	LPDDR4 PHY training converges; write-read-verify over the channel	data comparison + training status	PHY configuration, board routing, gross SI faults	training converges on 5 cold boots; margin recorded
B08	First timer interrupt taken and acknowledged on one core	a handler-incremented counter	interrupt wiring, exception entry, vectoring	counter advances at the expected rate
B09	DMA channel 0: 1D copy, L2 to L2, one NoC hop	data comparison + completion interrupt	descriptor decode, one NoC route, one coherent port	copy verified; completion interrupt observed
B10	OS boots to a shell on one core, then on four	console output	MMU, caches, coherence, driver stack	shell reached on 20 consecutive boots

Why this order. B05 is the first row whose oracle lives *inside* the part, and deliberately the smallest one available: three instructions, one of them a store, because a heartbeat that stops shows that the host core stopped, whereas a failing OS boot reveals almost nothing. B08 follows B06 rather than preceding it, because an interrupt handler needs a stack and a stack needs memory B06 has already proven. B09 depends on B06 and B08 at once, so it follows both; its failure would otherwise be unattributable. B10 is one row rather than ten because its prerequisites are proven: when the shell fails to appear after B01–B09 pass, the search is confined to the software stack.

On the reference SoC, this plan is a page, the whole order can be improvised on a bench by one engineer, and a wrong guess costs an afternoon: the part has one core, one memory, three clock domains and two power domains, and a person can hold all of it in their head. On the flagship, the same sequence has to be executed across five asynchronous clock domains and six power domains, with one debug access port per power domain and a bring-up team that is not the design team. For the engineer running B07 at 2 a.m., the plan must document why B06 comes first.

Scenario. The flagship’s LPDDR4 PHY training completes on eleven of the first twelve parts and, on the twelfth, converges to a setting that fails a memory test an hour later. **Approach.** The lab does not debug the training algorithm. It runs training a hundred times per part across the temperature range, logs the margin the training reports at each step, and finds that the failing part converges at the edge of the acceptance window while the others sit centred. The fix is a firmware change to the acceptance criterion plus a retrain-on-error path; the RTL is untouched. **Why.** This failure is *only* observable post-silicon, because of physical effects rather than complexity: pre-silicon there is no signal integrity to train against. Every model of the PHY, from RTL to a real-number model of the analog path (Chapter 21), trains against an idealization, and the distribution across parts and temperature does not exist until there are parts. The training state machine’s per-step margin status register makes this debug tractable. That register is design for debug, decided at tape-out; without it the only observables would have been “training passed” and, an hour later, “the memory test failed”.

Two supporting workstreams decide whether the first week goes well, and both are pre-silicon deliverables. **Post-silicon test plans** are typically more elaborate than pre-silicon ones, because they target system-level use cases that could not be exercised before silicon existed; their development starts alongside design planning, before a detailed microarchitecture exists, so the first version is written against architectural specifications and refined as features land ^[1]. That is Chapter 5’s extraction discipline applied a year earlier and one level up.

Debug software needs explicit planning for its half of readiness, and that half has four parts. First is the instrumented system software that replaces a general-purpose OS with something small enough to reason about. Second are the tools that configure triggers and query registers, and third is the transport that gets data off-chip. Fourth are the analysis tools that turn raw signal streams back into transactions. All four parts are written against a design whose features are still changing; in post-silicon work it is not uncommon to root-cause an observed failure to the debug software rather than to the part ^[1]. Chapter 4’s rule that the environment is a design too, whose bugs are tracked like any other, applies here.

On a large processor bring-up reported in 2017, roughly half of all post-silicon bugs were found in the first three months; much of the first two of those months went on hardware stability and screening parts, not on finding logic bugs ^[1]. Planning should accelerate bug-finding readiness; power-on work continuing into month three consumes time needed for Section 20.4.

20.4 Triage and localization costs

A bug announces itself in one of four ways. A system hangs; a hardware checker fires; a customer's stream shows a corrupt macroblock; a **multipass consistency check** reports a mismatch millions, and sometimes billions, of cycles after the error that caused it; that fourth check runs the same test several times and compares architected registers and part of memory at the end of each pass against a saved reference pass ^[1]. **Localization** identifies the activating input sequence and the block containing the bug. The effort of localising bugs from observed system failures dominates the overall cost of post-silicon validation and debug, and a single difficult logic bug has been reported to take days, weeks or months of manual work ^[2].

The governing quantity is **error detection latency**: the time between the moment a test activates a bug and creates an error, and the moment that error becomes an observable failure (a crash, a timeout, a deadlock or a wrong result). For difficult bugs this latency runs to millions or even billions of clock cycles ^[2]. Section 20.2's buffer bounds the visible interval. The flagship example runs for order 10^{12} cycles, while its 128-signal trace buffer holds 4,096. The trace therefore covers roughly one part in 250 million of the execution. Even a perfect trigger capturing the last 4,096 cycles before the hang records the *symptom*; activation occurred millions of cycles earlier, outside that window.

A checker detects a wrong answer, but the trace's four thousand cycles omit the activation in this forty-minute run.

Section 20.1 defines the hand-off: a **pre-silicon reproducer**, with a stimulus short enough to simulate and design scope small enough to load, that still fails.

Stages from failure to resolution

Published industrial practice distinguishes the following stages from failure to fix ^[1] (Table 20.3):

Table 20.3 — The industrial path from a failed test run to a resolved bug, one row per named stage, with the artifact each stage has to hand on. Note the third row: *sighting disposition* means assigning a debug team and a plan, not the recorded fix / waive-as-errata / defer decision, which arrives much later and is §20.6's subject.

Stage	What happens	Exit
Test execution	run it; on failure, the obvious sanity checks — part seated, supplies connected, switches as the test expects	resolved, or a pre-sighting
Pre-sighting analysis	make it repeatable: rerun under varied software, hardware, system and environmental conditions until a stable recipe exists	a sighting
Sighting disposition	assign a debug team; agree a plan to track, work around and address it	an owned plan
Bug resolution	work around it so validation continues; group with related failures; localise; root-cause; prove the fix	a bug , resolved

Sighting disposition means assigning a team and a plan; it is not Chapter 7's **disposition**, the recorded fix / waive-as-errata / defer decision, which arrives much later and is Section 20.6's subject. Chapter 12's **triage** retains its intent but becomes harder in practice: deduplicate by signature, classify and assign, without debugging the same fact twice. Two failures with one root cause can present as a crash in one test and a hang in another, so an aggressive schedule requires correct grouping *before* either is analyzed ^[1]. The first classification distinguishes physical silicon issues from logic errors, a distinction absent from pre-silicon triage. Repeating the test on a different part, or a different core of the same part, provides a discriminator: a failure following the part is a manufacturing or marginality problem; one following the design is a logic error.

Reproduction, and the problem of non-determinism

Most localization methods assume the failure can be made to happen again; post-silicon, that assumption is challenged in two ways.

The first is ordinary system non-determinism: interrupts, I/O activity, interactions between cores, operating-system context switches ^[2]. Returning a complex part to an identical error-free state and re-executing the same stimulus is not something a running system naturally supports. The second is physical and has no pre-silicon analogue. A logical error can be masked by a glitch or a fluctuating supply, and reproducing it may require a particular thermal range; finding a recipe (a tuned combination of physical, functional and non-functional parameters) that brings the bug back is itself a challenge ^[1]. Clock-domain crossing illustrates the problem: a whole class of metastability bugs cannot be demonstrated by RTL simulation and is generally difficult to reproduce and find in the lab (2024) ^[4]. Metastability is the unstable condition of a flip-flop sampled too near a change, which resolves to a legal value only after an unpredictable delay. That difficulty is why Chapter 13 puts structural clock-domain analysis before tape-out rather than trusting the bench.

Silicon speed permits a **weakened standard of reproducibility**: an error that appears reliably once in a few executions still counts as reproducible. A one-in-five failure rate is intolerable in a nightly regression, where the same failure on an identical manifest is Chapter 12's flake; in the lab it is a working recipe, because five runs cost minutes. A manifest is the per-run record of everything needed to repeat a run, down to the exact tool build. Where available, a **cycle-reproducible mode** improves on that weakened standard: rerunning the same test in this special hardware mode produces exactly the same execution ^[1]. Cycle reproducibility is the condition that permits the time-bisection technique below.

Scenario. A radar front-end runs an FMCW chirp chain whose constant-false-alarm-rate (CFAR) detection threshold is calibrated at power-on against a reference target. In the field, roughly one unit in two hundred reports spurious detections after about twenty minutes of operation. Nothing reproduces on the bench.

Approach. The lab characterizes the *recipe* instead of continuing attempts to reproduce the *failure*: parts run in a thermal chamber across the qualification range while the calibration coefficients are logged every chirp frame. The spurious detec-

tions appear only above roughly 85 °C, and only on parts whose initial calibration landed in the lower third of the coefficient distribution; these are two conditions no bench at room temperature produces together. With this recipe the failure reproduces on demand, and the recalibration path's fixed-point saturation bug is localized in a day in this example. **Why.** The example illustrates two lessons. This is a calibration path only real operation exercises: pre-silicon it was verified against a model of the analog chain, and the model used for this pre-silicon check did not represent temperature drift. In this radar example, failure to reproduce results from unvaried recipe parameters rather than a non-deterministic bug. Temperature, supply voltage, part-to-part variation and elapsed time are parameters. Deliberate variation and continuous logging make these parameters experimental inputs.

Method 1: binary search in time

Cycle reproducibility and programmable trace permit bisection. Run the test, terminate the trace at cycle N, look; if the state is still good, run again terminating later; if it is already bad, run again terminating earlier. Each run halves the interval containing the transition from good to bad. Published practice uses the same mechanism: re-running the same test with different trace-array configurations, terminating at different cycle counts, and aggregating the results of several runs into one effective trace far longer than the buffer ^[1]. This *backspacing* reconstructs history from many forward runs.

The cost quantifies Section 20.2's instrumentation trade-off. To narrow a 10^{12} -cycle run down to one 4,096-cycle window takes $\log_2(10^{12} / 4096) \approx 28$ halvings. At forty minutes a run, twenty-eight halvings are about nineteen hours of pure execution, before any time spent reconfiguring the trace between runs. If only one run in three reproduces the failure, the cost triples: multiplying by three gives over two days. The search has two dimensions rather than one: bisection identifies *when*, but the window is only useful if it contains the right signals, so a change of hypothesis part-way through means re-selecting the trace set and repeating steps already completed. A trace buffer twice as deep removes one of twenty-eight halvings, rather than halving the cost. The cost can instead be reduced by a *trigger*: a condition, programmed into a bus monitor, that starts or stops capture on a hypothesized cause rather than on the failure itself, converting a blind search into a single run when the hypothesis holds.

Method 2: binary search in configuration

Configuration bisection should precede trace analysis. A failure with four active cores should be tested with three. A failure with the compute island powered should be tested with power management off. Changing the hardware setup by disabling cores or varying the number of active hardware threads is standard practice for establishing what the failure actually requires ^[1]. Each answer removes a subsystem from the search space at a cost of one run.

Scenario. The GPU's four shader clusters share a unified memory path. In the lab, a compute workload gives wrong results roughly once per thousand dispatches: never with a single cluster, sometimes with two, reliably with four. **Approach.** Configuration bisection first: with clusters {0,1} the failure appears; with {0,2}, {1,3} and {2,3} it does not; with {0,1} and the DMA idle it disappears. Five runs isolate an interaction between two specific GPU clusters and concurrent DMA traffic; their shared unified-memory port implicates the memory path rather than shader cores. Only then does the trace budget get spent, on that port, with a trigger on the arbiter's grant sequence. **Why.** Configuration bisection gives a coarse hypothesis cheaply. It costs one run per question and it answers the question that decides where the expensive instruments point. Tracing before forming that hypothesis can waste a week in this example.

Method 3: reducing detection latency

Test generation can reduce the latency itself. **Quick error detection (QED)** transforms an existing test into one with a much shorter interval between activation and observable failure; the canonical transformation inserts a read immediately after each write, so a write that stores a wrong value is caught at once instead of thousands of cycles later when a subsequent operation reads it ^[1]. The transformations are software-only: duplicate the instruction stream into a reserved half of the register and memory space and compare periodically, or proactively load and check shared locations; nothing is added to the silicon ^[2].

Layering formal analysis on top turns detection into localisation. Bounded model checking is the proof method that works over a fixed interval of cycles: it copies the design's combinational logic once for each cycle of the interval, then asks a satisfiability solver for a violation inside that interval. Over the space of QED-compatible tests it searches for the *shortest* instruction sequence that activates and detects the bug, yielding a minimal reproducer and a short list of candidate blocks at once. On a 500-million-transistor open-source multicore SoC (the OpenSPARC T2) in 2015, this localized all 92 injected bug scenarios in under seven hours, with traces up to six orders of magnitude shorter than the original end-result-checking tests; the motivating deadlock, whose activation occurred millions of cycles before the failure, reduced to a three-instruction trace found in two and a half hours ^[2]. An industrial application narrowed post-silicon localisation automatically to about twenty flip-flops out of a million, and nine instructions out of billions ^[5]. The literature calls this layering of bounded model checking over QED tests **Symbolic QED**. The same flow was deployed industrially on an ASIL-rated Infineon automotive microcontroller core (60 instructions, 1.8K flip-flops, 70K gates, tracked across 16 versions): Symbolic QED found every bug Infineon's own flow had found, plus 7% more that were specification errors, at 2 person-days of effort against 6 person-months for a reused design ^[5]. An ASIL is an automotive safety integrity level, A through D with D the most demanding, assigned to a hazard rather than to the block that addresses it. The core's gate count limits the scope of that comparison.

The method requires a programmable core in the system, a way to run transformed tests, and a formal engine that can hold a reduced model. Where these requirements can be met, QED can produce shorter reproducers, as in the reported cases.

Method 4: transfer to a platform with visibility

Every localization aims at a hand-off, usually to one of two platforms.

Emulation and acceleration (Chapter 17) offer far more visibility than silicon at far less speed; transfer requires tuning: because silicon runs so much faster, a test that fails there cannot simply be replayed on an accelerator; it must first be tuned so that it fails *early*, within a run the slower platform can complete ^[1]. That tuning is itself a localisation exercise, and it is usually where the pre-silicon reproducer is born.

Formal property verification faces a scaling argument here: post-silicon failures are isolated to large subsystem boundaries beyond formal's capacity. The published method avoids taking that subsystem as the proof scope. The method has four steps. It selects the smallest block boundary at which the property to be checked can still be observed. It over-constrains everything that did not participate in the failure, including configuration registers pinned to the values dumped from the failing part. It leaves the signals of interest under-constrained, so the tool can still find manifestations not predicted by the engineer. Finally, instead of abstracting a long boot sequence, it loads the design's state from a simulation waveform at a timestamp after initialization and starts the proof there.

The reported cases quantify the effort. A configuration-register corruption was reproduced in a 25,000-gate scope in three person-days. Temporary over-constraints produced five further manifestations of the same bug, and five person-days proved the fix. Because the environment outlived the emergency, eight more functional bugs turned up when it was reused on later projects. A second case chose a 1.5-million-gate scope over a 4-million-gate one purely so that basic covers would hit, and reproduced its data-corruption bug in about four weeks and three hundred person-hours ^[6]. Both cases are Intel's own report, and the paper identifies its two designs only by function (a bridge block and a scheduler) inside SoCs it does not name ^[6].

Reproducing one post-silicon failure funds the formal environment's setup, leaving a verification asset for the next project. For the plan row exposed by a silicon escape, formal sign-off in the next generation follows the division of labor between simulation and formal discussed in Chapter 16.

20.5 Escapes and the feedback loop

Every bug found post-silicon is, by this book's definition, an **escape**: it crossed a sign-off boundary undetected (Chapter 1). The lab's bug list therefore records weaknesses of the pre-silicon process after sign-off. Upstream changes need explicit priority before the team disperses, when escape analysis risks being left incomplete.

Chapter 7 supplies blameless escape analysis with five written questions: mechanism, why stimulus never created it, why no checker caught it or none could have

seen it, why the metrics said we were done, what changes now. The same questions apply post-silicon, with different emphasis. In the codec example below, the fourth question concerns why the dashboard was green despite a missing coverage cross. The bug shipped despite a closed coverage model, a clean regression and a plan whose every row had been discharged by the evidence it named. A coverage model is the structured space of conditions a project has declared its tests must be seen to reach. Escape analysis includes explaining why the green dashboard did not expose the missing coverage.

Chapter 3’s chain classifies the required upstream changes. Every escape breaks one of three links first, and each break has a different upstream fix (Table 20.4):

Table 20.4 — Classifying a silicon escape backwards, by which of the three links — activation, propagation, checking — broke first, and what each break owes the process that missed it. The classification earns its keep because the three owe different things: a plan row and a coverage cross, a platform change, or a checker plus a test proving the checker fires.

Link that broke	What it means	The upstream change it owes
Activation	the stimulus never created the condition	a new plan row and a coverage cross that would have demanded it (Chapters 5, 6)
Propagation	the condition happened but could not reach any observable — usually because the platform did not model the mechanism	a platform change: a level, an engine, or an earlier scheduling of one (Chapters 13, 19)
Checking	it happened and was visible, and nothing looked	a new checker, plus a test that proves the checker fires (Chapter 5’s companion-row rule)

A fourth failure is a wrong **assumption**. A formal `assume` that does not hold in the real system deletes traces silently (Chapter 11), and no coverage number anywhere reports the deletion. When an escape’s mechanism was inside the deleted set, the upstream change must be a written assumption, promoted into a checked obligation on whoever must guarantee it, rather than a checker or a bin. A bin is one of the counters a coverage model keeps over a set of values (Chapter 6).

Worked example: escape record for the video codec corruption of Section 20.1. Five questions and five written answers identify the required changes.

1. **Mechanism.** A slice boundary falling inside a macroblock row, combined with a reference-list update on the same frame, causes the line-buffer read pointer to advance once too far; the affected macroblock reads a neighbour’s reconstructed samples.
2. **Missing stimulus.** The conformance corpus contains unusual slice geometry and frequent reference-list updates in separate streams. It contains none with both, because the corpus is organized by feature and each stream was authored to exercise one.
3. **Checker capability.** The checker was correct: the bit-exact reference model would have flagged the bug instantly on a stream containing it.

4. **Completion metrics.** The coverage model had a coverpoint for slice geometry and a coverpoint for reference-list update, both at 100%. It had no cross. As Chapter 6 explains, two closed coverpoints do not establish that the cross is closed; a model built one feature at a time inherits the corpus's blind spot.
5. **What changes.** (a) A cross of slice geometry $\times$ reference-list state added to the codec's coverage model, with the excluded cells argued rather than assumed. (b) Two synthesised streams added to the regression, generated to hit the new cells. (c) A plan-row family for *stream-syntax combinations* rather than stream-syntax features, so the next codec's plan cannot be written one axis at a time. (d) For any block with a bit-exact golden model, a standing rule should budget a stream generator rather than a library: with a free oracle, stimulus diversity is the constraint.

Analysis effort: a week in this example. Only (a) and (b) fix this bug. Items (c) and (d) change the next plan; stopping at (a) and (b) leaves the process that created the hole unchanged.

Two practices support this feedback loop.

Internal escape criteria. One large processor program counted any bug found by an operating-system-based test or real application software as an escape from its *validation strategy*, because bare-metal exercisers were supposed to find it first ^[1]. This internal standard is deliberately harsher than customer escape: it holds exercisers accountable for coverage instead of relying on the OS as an unplanned safety net. The program was IBM's POWER8 (12 cores on a chip of over 5 billion transistors) and bare-metal exercisers were its primary post-silicon test vehicle, which makes that internal bar coherent ^[1].

Coverage one platform down. A covergroup is the SystemVerilog container that states one coverage model in code. Synthesizing covergroup monitors into silicon has prohibitive area, timing and power costs. Published practice synthesizes those monitors into the *accelerator* model instead, runs the same exercisers there, and uses the resulting coverage to decide which exerciser shifts deserve scarce silicon time ^[1]. This implements a **unified plan**: one plan source, with each row assigned to the platform that will discharge it, and a common test-template language so a test written for one platform can be narrowed and re-run on another. This enables Section 20.4's hand-off: when a bug appears on silicon, narrowing the same template until it fails in simulation turns the lab failure into a debuggable one.

On the publicly documented processor program, about 1 percent of all bugs were found in post-silicon validation (2017) ^[1]. That one percent consumed weeks per bug in a phase measured in weeks, and any one could have forced a stepping. The pre-silicon process caught ninety-nine out of a hundred, supporting the process in Chapters 5 through 19; the hundredth required weeks of debugging, motivating escape analysis. That program is IBM's POWER8, and the 1% is a statement about one chip's own bug population rather than an industry rate ^[1].

Feedback also crosses generations. In the reference SoC's canonical DMA escape, a 2D transfer with a negative stride was split wrongly at the 4 KB boundary by the

burst splitter, the DMA stage that splits each transfer into bursts that the AXI address rules allow. The 4 KB boundary is the address line every 4 KB that AXI forbids a burst to cross, so no burst ever spans two subordinates. A stride is the signed address step from the start of one row of a 2D transfer to the start of the next. The data was corrupted only when the second DMA channel was active, and that escape produced a plan row, a coverage cross and a regression test. The flagship inherits all three along with the DMA architecture. It also inherits a descendant of the bug: the same faulty split, now under coherence, where the split's second half writes a line the host cluster holds Modified, so the corruption stays invisible until the writeback. A cache holds a line Modified when it has written to that line and has not yet returned it to memory. Inherited assets do not establish inherited coverage: the old cross omits coherence state, and the old test never runs with a cache holding the line dirty. The next generation must re-derive coverage against the new mechanism; rerunning the old suite successfully does not establish that a previously found bug remains covered.

20.6 Workarounds and fix decisions

Section 20.1's error sequentiality changes the order of operations. Pre-silicon, the sequence is *find* → *root-cause* → *fix* → *verify*. Post-silicon, workarounds must precede root-causing and the final fix decision:

work around → localise → root-cause → propose a fix → prove the fix pre-silicon → decide what to do with it.

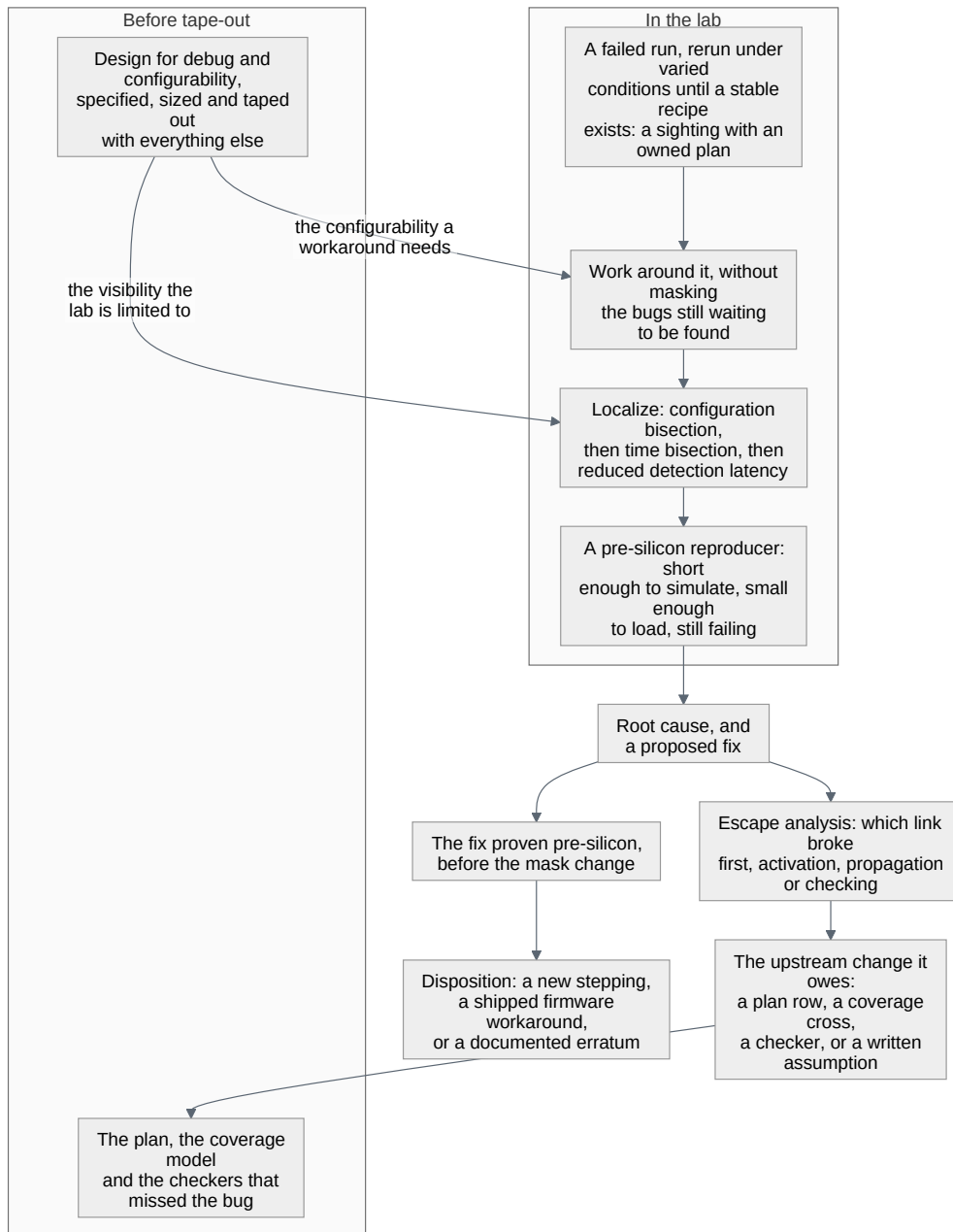


Figure 20.1 — Post-silicon work is a loop rather than a line: what the lab can see and patch was fixed before tape-out, the workaround precedes the diagnosis, and every escape closes on an upstream change to the process that missed it.

The workaround comes first, before anyone knows what the bug is, because no program can afford a silicon stepping per bug and the lab cannot stop while one is investigated. And the workaround carries the constraint Section 20.1 named: it must eliminate the bug's effect *without masking the other bugs still waiting to be found*. A workaround that disables a whole subsystem stops validating it, even when that consequence was unintended. Figure 20.1 places that order in the loop it belongs to, from the provisions made before tape-out to the upstream change an escape owes.

Workaround availability depends on pre-silicon provisions, as observability does in Section 20.2. Mitigation post-tape-out is overwhelmingly a matter of patching or re-configuring behavior through software or firmware; the ability to patch depends dir-

ectly on how much configurability and controllability were built into the design beforehand ^[1]. DfD provides visibility, while configurability enables workarounds. The same architecture review must decide both because neither can be added later.

Disposition after tape-out

Chapter 7 gave a project three ways to dispose of an open bug: fix it before tape-out, waive it as known errata with a named software workaround and a documentation owner, or defer it with an argument. After tape-out the first option is gone in that form, and what replaces it is more expensive. The post-tape-out options differ in both their costs and obligations (Table 20.5):

Table 20.5 — The three forward-looking options for an open bug once the masks exist — a new stepping, a shipped firmware workaround, a documented erratum — with the conditions that favour each, its cost, and the obligation it carries. The second and third rows are a spectrum rather than two alternatives: what separates them is who is obliged to apply the workaround.

Option	When it wins	What it costs	What it obliges
New stepping (revised masks)	no workaround exists, or the workaround’s cost is unacceptable; safety- or security-relevant behaviour; the bug blocks a committed use case	refabrication, requalification, and the calendar — usually the dominant term	the fix proven pre-silicon <i>before</i> the mask change, plus regression of everything the change touches
Firmware or software workaround , shipped	a configuration or code path can avoid the condition reliably	performance or power, permanently; a constraint every future software release must honour	proof that the workaround covers <i>every</i> path to the condition, and an owner for that proof
Errata — documented, not worked around in the product	the condition lies outside the product’s committed use cases, or the customer can reasonably avoid it	support burden and a permanent public record	a named workaround in the document, a documentation owner, and Chapter 7’s recorded decision with a real expiry

The second and third rows form a spectrum: an errata entry usually contains a workaround; the difference is who must apply it. Chapter 7’s five waiver fields apply verbatim to an erratum: the hole and its reason, the risk argument, an owner and an expiry. An erratum lacking a named workaround and owner leaves waiver fields blank and exposes customers to rediscovery.

The finding that follows says what set severity, when severity could be established, and how early a lab in fact assigned it. On one industrial program, bug severity was set by three factors together: the bug’s impact on functional behavior, the *performance cost of working around it* where a workaround existed, and the complexity of the fix; severity on that program could only be determined once the root cause was found and a fix proposed; and yet its lab engineers guessed severity correctly and early from limited data, and scheduled accordingly, so that high-severity bugs were root-caused in about 6 days on average against roughly 10 for the medium ones (2017) ^[1]. Severity cannot be inferred from the symptom alone: a hang looks catastrophic and may be one register bit away from a free firmware workaround, while a

one-bit corruption looks minor and may have no workaround at all; yet an experienced lab triages by expected consequence long before it can prove consequence. This early estimate determines which bugs receive scarce debugging time first.

Pre-silicon fix proof. A fix must be shown to correct the original error and to introduce no new one ^[1], and the environment built in Section 20.4 to reproduce the bug is exactly the instrument for it. The published recommendation is to reproduce every post-silicon control-logic issue in a formal environment and check its fix with formal technology before the design goes for a re-spin ^[6]. The environment already exists and proving the fixed logic costs person-days; omitting that work risks discovering an incomplete fix in the next stepping.

Scenario. The flagship’s inline AES-GCM engine on the memory path has a fault: under a particular back-to-back key-slot switch, one 16-byte block is encrypted with the previous key. The lab finds it in month two on an unexpected workload.

Approach. In this example, triage takes a day: the functional bug follows the design rather than the part and reproduces with a two-transaction sequence once known. Workaround: firmware serialises key-slot switches by draining the pipeline, at a measured cost to throughput on the affected path. The subsequent decision takes a week in this example and belongs outside verification. Because the failure produces ciphertext under the *wrong* key, it is a confidentiality failure, not a performance one; the drain-based workaround is enforceable only in the driver the vendor ships, and any customer writing their own would reintroduce it. Confidentiality is the property that breaks when an unauthorized reader can obtain protected state. The disposition is a stepping, with the firmware workaround shipped in the interim and the erratum published so that customer software cannot inherit the hazard silently. **Why.** Two factors, neither of them symptom severity, determine the decision. First, the consequence class: a workaround that depends on every future software author doing the right thing is not a workaround for a security property (Chapter 23). Second, discoverability: external knowledge of such a defect sharply shortens mitigation time because it can be exploited rather than merely encountered ^[1]. That asymmetry is why security bugs are dispositioned differently from functional ones of identical technical severity.

The disposition decision is not made by the verification team and should not be: it requires information verification does not have about committed customers, launch dates, the qualification plan, the cost of a mask revision, the competitive window. Verification’s responsibility is evidence rather than an opinion about respinning: three findings specify **what the bug does, the necessary and sufficient conditions**, and **the workaround’s coverage and exclusions**. These three findings support the decision. A severity label alone omits the evidence verification is uniquely qualified to supply.

IN PRACTICE

Scarce parts require scheduled bench time. When debug shares silicon with characterization, software development and demonstrations, limited bench time can favor a

technique requiring fewer part-hours despite lower technical effectiveness. If the lab receives a design it did not verify and verification receives a failure description written by someone who has never seen the coverage model, both hand-offs can lose context.

Three omissions require attention. Scheduling escape analysis after launch risks losing it when the team disperses. Without pre-silicon plan ownership, defeature modes and irritators can ship unchecked; a defeature-mode bug can consume a week investigating a fault absent from the functional design. Closing a ticket with a firmware workaround without recording the coverage hole lets the next generation inherit that hole with the IP. A coverage hole is a required behavior or stimulus that nothing has yet been observed to produce.

First-silicon success is the outcome in which the first fabricated parts are already fit for production. In 2024, 14 percent of IC/ASIC projects reached first-silicon success and 75 percent finished behind their original schedule ^[4].

PITFALLS

1. **Sizing DfD from an area budget rather than from debug scenarios.** *Symptom:* “we allocated our usual 20 percent for debug.” *Cure:* DfD sizing should distinguish each block’s most feared failure’s two likeliest causes, then negotiate area against those needs.
2. **Spending the trace budget before bisecting the configuration.** *Symptom:* a week of trace captures and no hypothesis. *Cure:* configuration bisection first; it costs one run per question and it decides where the trace should point.
3. **Treating “cannot reproduce” as a property of the bug.** *Symptom:* an intermittent filed as non-deterministic and parked. *Cure:* the recipe needs deliberate variation of unvaried parameters: temperature and voltage, part and elapsed time, active feature set.
4. **A workaround that masks other bugs.** *Symptom:* a subsystem disabled to unblock the lab, and its bug queue never grows again. *Cure:* each workaround should record what it stops exercising as a validation gap with an owner.
5. **Closing an escape with a fix and nothing else.** *Symptom:* the bug tracker closes; the plan, the coverage model and the checklist are untouched. *Cure:* no post-silicon bug closes without a named upstream change: a plan row, a cross, a checker, or a written assumption.
6. **Dispositioning by symptom.** *Symptom:* a hang is S1 because hangs look bad, a corruption is S2 because it is one bit. *Cure:* severity is impact × workaround cost × fix complexity and remains unknown until root cause and a proposed fix exist; deferring a clock-domain waiver to the lab repeats this error one stage earlier, because a bench does not reproduce metastability.

SUMMARY

- **Validation** is checking real hardware, not a model. Post-silicon completely inverts the budget: execution is unrestricted while visibility reveals almost nothing.
- DfD fixes lab observability before silicon exists: **signals × cycles ≤ buffer** is fixed at tape-out; 64 KB buys 128 signals for 4,096 cycles, with no subsequent widening in the lab.
- Bring-up is a verification artifact. Each check must use only previously established evidence, and each row needs a single exit criterion.
- Detection is cheap; **localization** costs more because error detection latency for difficult bugs runs to millions or billions of cycles against a trace window of a few thousand [2]. Configuration bisection should precede time bisection, and latency reduction should precede searching it.
- Post-silicon debug delivers a **pre-silicon reproducer** rather than an explanation: short enough to simulate, small enough to load, still failing [1].
- Every escape owes an upstream change: a plan row, a coverage cross, a checker, or a written assumption. A fix alone leaves the process that created the hole unchanged.
- Post-tape-out disposition is stepping / shipped workaround / errata, constrained by configurability designed in a year earlier.

EXERCISES

Exercise 20.1 (calculation). Section 20.2 fixes the flagship’s trace buffer at 64 KB, and §20.4 bisects its order- 10^{12} -cycle run down to a single window of the buffer’s depth. Compute the depth the same buffer affords once the trace set widens from 128 signals to 512, and the number of halvings the bisection then needs. Show the extra pure execution time at forty minutes a run, and again on the assumption §20.4 makes that only one run in three reproduces the failure. Reconcile the result with what §20.4 says a trace buffer twice as deep buys.

Exercise 20.2 (artifact). The bring-up row below comes from the flagship’s first-week plan of §20.3, with one field left blank and the row placed immediately after the heartbeat row. Fill the blank field from the plan’s own vocabulary, and name the two earlier rows whose evidence this check consumes together with the reason §20.3 gives for each dependency. State the ordering rule of §20.3 that the placement breaks, and the two failure modes the placement leaves indistinguishable.

Row:	B09
Check:	DMA channel 0, 1D copy, L2 to L2, one NoC hop
Oracle available here:	
A pass rules out:	descriptor decode, one NoC route, one coherent port
Exit criterion:	copy verified; completion interrupt observed
Placed after:	B05

Exercise 20.3 (judgement). The flagship inherits the reference SoC’s DMA architecture together with the plan row, the coverage cross and the regression test that closed the original 2D burst-splitting escape, and the inherited suite is rerun on the flagship and passes. No test in that suite has ever run with the host cluster holding the split’s target line Modified. Decide what the passing suite establishes about coverage of the descendant mechanism §20.5 describes, and name the link of §20.5’s classification table that broke first together with the upstream change that link owes. State what the inherited coverage cross omits.

Exercise 20.4 (judgement). A lab lead asks the verification team for a severity label on a flagship hang and for a recommendation on whether to take a new stepping. Decide what §20.6 obliges the team to supply and what it obliges the team to decline, giving the reason §20.6 states for the decline. Name the three factors that set severity in the industrial program §20.6 cites, and state why the symptom on its own cannot order a hang against a one-bit corruption.

Exercise 20.5 (artifact). Read the two triage entries below against §20.4. Name the stage of §20.4’s pipeline each entry has reached and the exit each one owes. State what the repetition on a second part discriminates, and what it therefore says about the first entry. Decide what §20.4 requires of the second entry before anyone analyzes it, and name the standard of reproducibility §20.4 applies to the first entry’s rate together with the verdict that same rate would receive in a nightly regression.

Entry: LAB-0447	
Test:	video decode against live network traffic
Symptom:	system stops; no exception, no error interrupt
Part and socket:	part A, socket B
Recipe:	one decode-and-traffic mixture, one run in five
Repeated on part C:	same symptom, same mixture
Stage recorded:	pre-sighting
Entry: LAB-0452	
Test:	compute dispatch, four active clusters
Symptom:	wrong result, no hang
Part and socket:	part A, socket B
Recipe:	none yet; not seen a second time
Repeated on part C:	not attempted
Stage recorded:	pre-sighting

FURTHER READING

From the corpus. The post-silicon tutorial ^[1] anchors this chapter, covering the pre-silicon-to-field checking continuum, pre-sighting/sighting vocabulary, trace selection and state restoration, the readiness workstream, and a detailed account of one large processor bring-up. The symbolic quick-error-detection paper ^[2] treats error detection latency and automated localization, with the bug scenarios and trace-length data used here; the accompanying industrial deck ^[5] adds effort comparisons. For post-silicon formal work, ^[6] provides a step-by-step method and two worked industrial cases: over-constrain uninvolved signals, under-constrain participating ones, and load proof initial state from a simulation waveform. The 2024 trend reports provide

context: [4] covers first-silicon success, schedule slip and why metastability is the bug class least likely to be caught on a bench; [7] describes early lab-based checking, a strategy that worked for simpler devices but is increasingly impractical for SoC-class FPGAs.

Outside the corpus. P. Mishra and F. Farahmandi (eds.), *Post-Silicon Validation and Debug* (2019), is the book-length treatment of everything this chapter compresses. For bring-up as an industrial practice, A. Nahir et al., “Post-silicon validation of the IBM POWER8 processor” (DAC 2014), and M. Riley et al., “Debug of the CELL processor: moving the lab into silicon” (ITC 2006), provide published accounts of processor bring-up and debug. F. M. De Paula, A. J. Hu and A. Nahir, “nuTAB-Back-Space” (CAV 2012), is the primary source on reconstructing history backwards from a non-deterministic system, and S.-B. Park, T. Hong and S. Mitra, “Post-silicon bug localization in processors using instruction footprint recording and analysis” (IEEE TCAD, 2009), addresses on-chip localization instrumentation.

REFERENCES

1. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
2. **P9** D. Lin, E. Singh, C. Barrett, and S. Mitra, "A Structured Approach to Post-Silicon Validation and Debug Using Symbolic Quick Error Detection," Proc. IEEE International Test Conference (ITC), Anaheim, CA, pp. 1–10, 2015.
3. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
4. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
5. **D24** S. Mitra, E. Singh, and K. Devarajegowda, "QED & Symbolic QED: Pre-silicon Verification, Post-silicon Validation, Industrial Results," Proc. DVCon Europe, 2019.
6. **D3** A. Jain, A. Gupta, M. V. Achutha Kiran Kumar, Ss Bindumadhava, S. S. Kolar, and S. Gadey NV, "Never too late with formal: Stepwise guide for applying formal verification in post-silicon phase to avoid re-spins," Proc. DVCon U.S., 2022.
7. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.

PART VI

Specialized Domains

The obligations a market imposes: where the digital abstraction meets the continuous world, what changes when a standard is watching, and how to verify against an adversary.

21. Analog and Mixed-Signal Verification

Analog and mixed-signal verification checks continuous quantities within tolerances and defines the digital team’s responsibilities. The reference SoC’s on-die analog front-end comprises a 12-bit SAR ADC, a PLL and a temperature sensor, all controlled by a peripheral-domain register block owned by the digital team. A SAR ADC converts by successive approximation, settling the output code one bit at a time. In this example, the first checker uses the method the team has used for every checker over two years: apply a known input, compute the expected code, compare:

```
assert (adc_code == expected_code);
```

It fails on nearly every run by one code, occasionally two; three engineers spend a week investigating, although there is no bug. One least-significant bit against the front-end’s 1.8 V reference is $1.8 / 4096 \approx 439 \mu\text{V}$. The model, the schematic and the eventual silicon are all expected to disagree by more than 439 μV .

Changing the check to `±2 LSB` makes the regression pass, but the number remains recorded only in that line of code. Six months later, a failure at 3 LSB cannot be classified as a bug, modeling artifact or unlucky seed because the origin of the 2 is unknown. The debugging adjustment now serves as the specification.

The oracle is the mechanism that decides whether an observed response is correct, and the coverage model is the multi-dimensional space of values and conditions that a plan asks a regression to reach. The oracle, the checkers, the coverage model and the regression pass/fail rule must therefore use a band of correctness rather than equality. Under the responsibility split used in this chapter, the digital verification engineer does not verify the ADC itself. The chapter defines the digital engineer’s responsibilities at this boundary. UVM, the Universal Verification Methodology, is the standardized class library from which reusable testbenches are built. The two normative documents behind this chapter are Accellera’s *Verilog-AMS Language Reference Manual* (Accellera Std VAMS-2023) ^[1] and its *Universal Verification Methodology for Mixed-Signal Standard* (UVM-MS), Version 1.0, January 2025 ^[2].

CHAPTER MAP

§21.1 explains how the analog boundary changes the oracle and propagates tolerance-based verdicts through checkers, coverage and regression pass/fail. §21.2 covers what real number modeling captures and discards, and how it made mixed-signal regression practical. §21.3 covers the costs and uses of full AMS co-simulation and selection of a handful of scenarios. §21.4 treats behavioral models as assumptions requiring circuit correlation, renewal and ownership. §21.5 defines digital responsibility for the interface contract, control and calibration sequences, and failure handling.

21.1 Where the digital abstraction stops

Two assumptions underpin the techniques of the preceding twenty chapters and are embedded in their tools: discrete values and discrete time. The first is that a signal takes a value from a small discrete set, so comparison is exact and a bin either contains a value or does not. The second is that time advances in discrete steps, so “the value at the clock edge” is a well-defined thing to sample.

An ADC, a PLL, a voltage regulator and a high-speed PHY satisfy neither assumption. Checking one means measuring amplitude, gain, frequency, phase or similar quantities rather than reading a logic value [3]; these continuous properties cannot be reduced to a bit without discarding the property being checked. PLL lock means remaining within some frequency error of the target long enough to count as settled; it differs from membership in a state-machine state.

The **oracle** fails first: Chapter 3 established that every practical oracle checks only a projection of correctness, and the analog boundary makes that limitation explicit. The check measures whether the output lies inside a numeric band for a specified time under specified conditions. The verdict has three parts where a digital verdict has one:

- **A band.** The band is stated either in absolute terms (± 10 mV) or in relative terms ($\pm 2\%$), and each form has a weakness. A relative tolerance is a fraction of the reading itself, so near a legitimate zero it has almost no width and fails. An absolute tolerance is meaningless over most of a quantity’s range if that range spans three decades, three successive factors of ten.
- **A window.** Because analog outputs settle, a comparison before settling checks a transient; one at an arbitrary time checks whatever the model’s step size produced. Every analog check needs a stated moment or interval at which it is valid.
- **A condition set.** Supply voltage, temperature, process corner, input frequency, load. A tolerance without the conditions under which it holds is not a specification.

Downstream, checkers compare against bands instead of equality. Coverage also changes. Bins are the counters that a coverage model keeps over sets of values. A coverage bin on a real value that names a single point can fail to be hit at all, because floating-point representation has limited precision: $0.1 + 0.2$ yields 0.30000000000000004 , which does not fall in a bin covering $\{0.3\}$. The language standard names that shortfall explicitly as a problem for analog specifications that carry tolerance ranges. The remedy is in the same standard: a tolerance token attached to a single real value defines an implicit range, so $\{1.8 \pm 0.05\}$ and $\{1.8 \pm 2.0\}$ are bins that a real measurement can land in [4].

A digital regression’s verdict is a property of the design; a mixed-signal regression’s verdict is a property of the design *and* of every tolerance somebody typed into a checker. Bring-up is the phase in which the environment and the DUT first exchange checked transactions. Scattered testbench literals leave two questions unanswered during bring-up: the tolerance’s source and its approver.

Tolerances therefore belong in verification-plan closure criteria, rather than being defined in code. A plan row is the line of the verification plan that carries one feature, together with its method, its pass/fail criterion and the evidence that closes it. A coverpoint is an observed variable partitioned into bins, and an assertion is a declarative statement of a property that the design must hold, evaluated by a tool. A digital plan row closes when a coverpoint is hit or an assertion is proven; a mixed-signal plan row closes when a measured quantity sits inside a stated band under stated conditions, with a named source for the band. What the named source buys is traceability, the permanent link from a specification to the plan row, to the check and to the result. That traceability is what analog verification has historically lacked: analog practice has leaned on interactive simulation and visual inspection of waveforms by the same analog team that designed the circuit [3].

Scenario. The reference SoC's PLL must be within 100 ppm of its target after lock. **Approach.** The plan row states the band (100 ppm), the window (measured over 1024 output cycles beginning 20 μ s after the lock request), the conditions (nominal supply, three temperature points, three process corners), and the source of the band: the tightest frequency accuracy any consumer of the system clock demands. The checker reads all four from a package, not from literals. **Why.** At 140 ppm, the row identifies the violated requirement so the team can discuss whether the band or design should change. Without the plan row, the team must establish the source of the 100 ppm requirement before discussing a change.

21.2 Real number modelling

Real number modeling (RNM) describes the block's behavior with real-valued signals in the ordinary event-driven simulator: a voltage is a `real`, a net carries a number rather than a logic value, and the model computes its output directly from its inputs and internal state.

An analog model describes the current-versus-voltage relationship between nodes, and the analog solver finds all node voltages and branch currents by Newton-Raphson iteration with matrix inversion, choosing each timestep from accuracy criteria. A real model has no matrix solution: the output is computed directly, the model itself decides when to perform each internal computation, and there is no continuous-time operation; only sampled, clocked or event-driven updates. There is no analog solver and therefore no convergence failure [5], and the model runs in an event-driven framework that is inherently faster than the analog solvers it replaces [3].

Real number models are written in ordinary HDL. Verilog-AMS provides `wreal` nets; SystemVerilog provides user-defined net types with a resolution function, so a net carrying a structure of voltage and current can define what happens when two things drive it. No standard defines how discrete modeling of analog nets should be done, so published examples are one method among several, rather than a normative pattern [2].

Measured RNM performance

Three measurements, from two independent sources, each against its own baseline (Table 21.1):

Table 21.1 — Three published real number modelling results, each measured against its own baseline: the first row against a SPICE netlist, the other two against Verilog-AMS models of the same blocks. The second column is why the ratios cannot be chained into a ladder: the three results come from different designs, different references and different years. The first row's accuracy figure is the one to notice, because the verdict never depended on what that model traded away.

Design	Compared against	Result	Year
A major analog block with filtering and fault detection	SPICE netlist	24× faster, 0.3% gain inaccuracy at 100 MHz	2011
PLL loop filter	Verilog-AMS	≈90× faster (47 s vs 1 h 8 min 32 s)	2015
Delta-sigma ADC	Verilog-AMS	230× faster (0.4 s vs 92.5 s), ENOB 11.515 vs 11.72	2015

The first row's 0.3% inaccuracy was far better than required, because the tolerances on the checkers were in the range of 5%; the accuracy traded away for the speedup was accuracy the verdict never depended on [3]. A real number model should meet the plan row's tolerance, with no finer precision. The proof-of-concept targeted a current LSI Corporation design, selecting a major analog block driver with filtering and fault detection [3]. The evidence covers one block in one company's design, so the ratio must retain its baseline.

The other two rows compare SystemVerilog real models with Verilog-AMS models of the same blocks, excluding SPICE [5]. The three ratios measure separate things, so multiplying them would produce an unsupported composite claim.

Physics omitted by RNM

Electrical relationships. A real net carries a value, not a Kirchhoff relation between the currents and the voltages meeting at a node. Loading, drive strength, input impedance and current draw are simply absent unless the modeller writes them in, and in SystemVerilog that means a user-defined net type with a resolution function that does the arithmetic explicitly. A shared bias line feeding two loads behaves exactly as with one load in such a model, hiding an entire class of circuit failures.

Continuous time. The model updates on events. Between updates the modelled quantity is a held value, and any transient shorter than the update interval does not exist. A real model shows glitches, ringing, supply-ramp overshoot and settling behavior required by tolerance windows only if the modeler explicitly included them.

Noise, variation and margin. Thermal noise, jitter, offset, mismatch and the spread across process corners are not free consequences of a real model the way they are consequences of a circuit. Each one is a feature somebody wrote, with parameters somebody chose.

RNM remains necessary because the alternative to abstraction here is no simulation; simulating everything is infeasible. Checks dependent on discarded physics must be identified because they check the model rather than the design.

Scenario. The radar front-end's receive chain runs from mixer output through a variable-gain amplifier to the ADC, feeding the FFT and constant-false-alarm-rate (CFAR) detection pipelines. Chapter 17 replayed a recorded chirp sequence through it, because the property under test there was bit-exact arithmetic across three configurations and that demands the identical input every time. **Approach.** For a different property, a real number model of the analog path lets the testbench generate the chirp: sweep the return amplitude across the full input range, inject a controlled noise floor, drive the amplifier into and out of saturation, and run it as a constrained-random regression. **Why.** Detection behavior at low signal-to-noise ratio requires coverage of conditions rather than reproducibility, and a recorded capture set contains exactly the SNR values that happened to be recorded. The two stimulus strategies answer different questions and the project needs both. **The limit.** The model's noise floor is a parameter rather than a measurement; a CFAR threshold tuned against an underestimated noise floor assumes less noise than silicon has. That parameter needs a source, as §21.4 explains.

21.3 AMS co-simulation

When physics matters, AMS co-simulation runs both engines: the digital event-driven simulator coupled to an analog time-domain solver, one simulation with two solvers exchanging values at their shared boundary. This restores the electrical relationships, transients and actual circuit convergence behavior omitted in §21.2. [Figure 21.1](#) draws that boundary and the runs on either side of it.

A net's *discipline* is its declared nature: continuous or discrete. Where a continuous discipline meets a discrete one, connect modules are automatically inserted to join them, selected by the disciplines and directions of the ports involved ^[1]. Whenever the digital side requests an analog value, for example on a periodic sampling clock, the simulator asks the analog solver to interpolate at that digital time point. That request is a *blocking* one: control passes to the analog solver and does not return until the value is available ^[2]. An unsuitable sampling clock repeatedly stalls the digital engine on the system's slowest component.

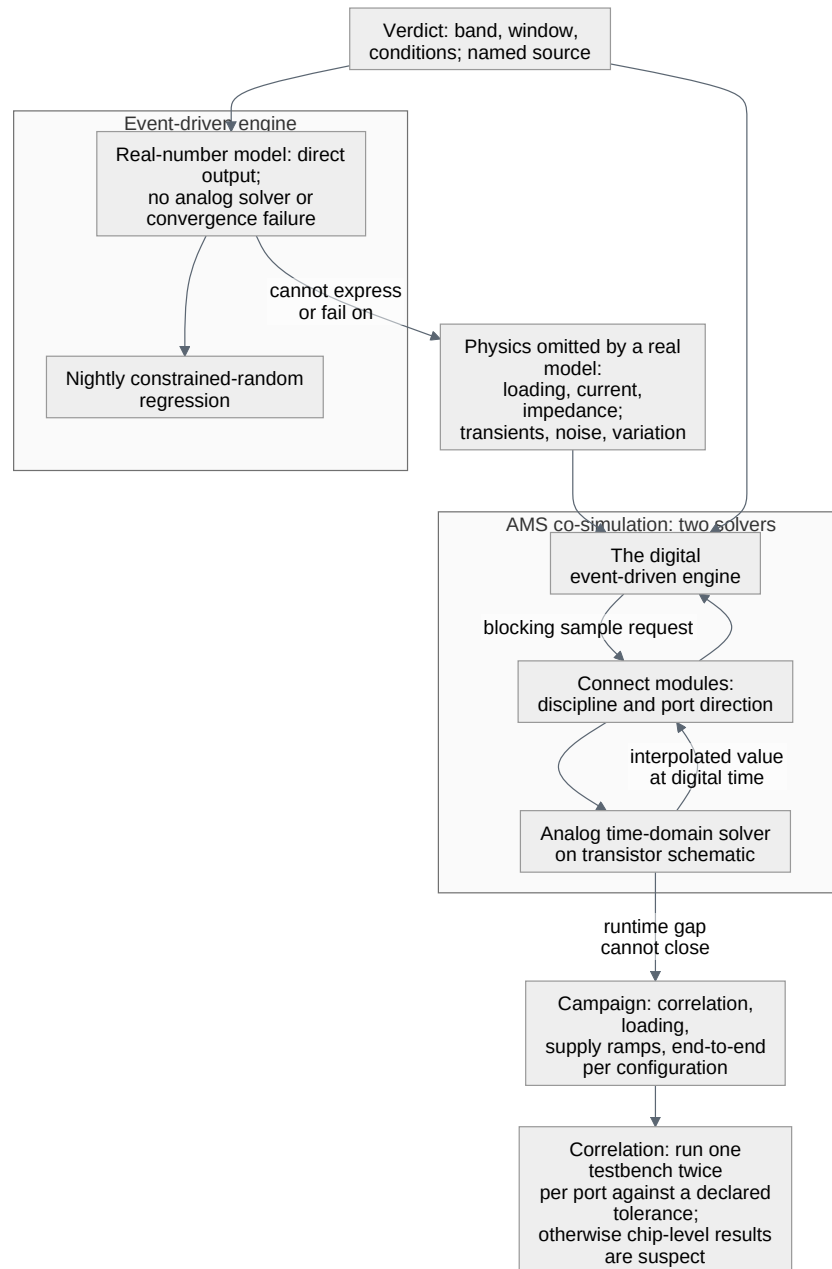


Figure 21.1 — The two engines meet at one boundary: the digital side blocks on each request while the analog solver interpolates, which is why the two-solver run is a campaign reserved for the physics a real number model leaves out.

The cost follows from §21.2’s table. There, a real number model of a delta-sigma converter finishes in 0.4 seconds where the Verilog-AMS version takes 92.5. A delta-sigma converter reaches its resolution by oversampling its input and filtering the result. At anything like that ratio of runtimes, an hour of real-model regression takes multiple days with analog solvers in the loop. A job that has to finish overnight then misses its deadline. The runtime gap cannot be resolved by tuning. AMS co-simulation therefore requires a *campaign*: a purpose-built run with a stated reason and written result that then stops, rather than a regression tier with a nightly budget.

Scenario selection should reserve co-simulation exclusively for physics omitted by real number modeling.

- **Model correlation runs.** §21.4 treats these runs, and they establish the basis for every subsequent real-model result.
- **Failure modes that live in loading, current and impedance.** Precisely the class that a real model cannot express and therefore cannot fail on.
- **Power-up, power-down and supply ramps.** Sequencing across a boundary where the analog side has a genuine DC operating point to find, and where the digital side's reset release depends on that operating point.
- **One end-to-end signal-path scenario per configuration.** Not for bug-finding, but as evidence that the composed system behaves as the real models claim.

Scenario. The sensor hub's always-on island shares one bias reference between its temperature channel and its pressure channel, and firmware may enable them independently. **Approach.** The real model of the bias generator delivers its nominal output regardless of how many channels are enabled, so a real-model regression will never show a problem. One co-simulated scenario enables both channels and checks the reference under the doubled load. **Why.** When the second channel switches on, shared-reference droop shifts the first channel's conversion beyond its stated tolerance; this loading failure is invisible to any abstraction without current. **What it costs.** One scenario, re-run when the bias generator's schematic changes, not nightly.

PHY training verification limits

Chapters 18 and 20 defer LPDDR4 PHY training to this chapter. The AMS boundary can discharge the *algorithm*: that the training state machine converges against a channel model, terminates rather than loops when a step fails, has reachable and correct retry and fallback paths, and respects the protocol's timing obligations throughout. Real models plus a few co-simulated runs cover this substantial algorithmic risk.

It cannot discharge what Chapter 20 names: the *distribution* of training outcomes across parts and temperature. Every pre-silicon channel model trains against an idealization of signal integrity, and the spread separating a part that converges at the center of its acceptance window from one that converges at the edge does not exist until there are parts. The plan carries both rows, with the second closing in the lab; this division should be documented before bring-up.

21.4 Behavioral models and model correlation

The two preceding sections depend on models that make claims about silicon. An uncorrelated model incurs the cost of producing large volumes of passing nightly results quickly for months without evidence of agreement with silicon.

The reference is inverted

In digital work the specification is the reference and the implementation is checked against it. A *golden* model is golden because it is the specification; Chapter 8’s example of a standard supplying the executable model outright reduces checking to comparison. Synthesis then produces the gates, and equivalence checking closes the gap between RTL and gates mechanically.

Analog blocks also have specifications, but the schematic is the reference because it is what gets fabricated: a different sense of golden. There is no analog synthesis, so the schematic is a creative artifact rather than a derived one, and there is no equivalence checking to confirm that a hand-written model matches the schematic that will become silicon. Chip-level verification therefore assumes correlated models, and without them chip-level verification is suspect [6].

Digital verification addresses specification misinterpretation with two independent readings between specification and RTL: one reading is embodied in the RTL the digital designer writes, the other in the checks the digital verification engineer writes against the same specification. Analog specifications are usually interpreted correctly and are quite simple; the problems are creating the implementation and creating the model. Analog therefore needs comparison of two independently created block representations, schematic and model; a testbench assertion does not provide that independent check [6].

Model correlation procedure

Model correlation requires the following procedure and artifact; a review, a glance at two waveforms or a design-document claim is insufficient:

1. **One self-checking testbench**, an environment that computes pass/fail itself rather than relying on human inspection of waveforms, written against the block’s specification.
2. **Run twice**, once against the behavioral model and once against the transistor-level schematic.
3. **Compared per port, against a declared tolerance**. In a published programmable-gain-amplifier example, the specification states that the output voltage of the model and of the schematic must differ by less than 10 mV under the same stimulus, and that the supply current on the supply pin must match to within 1 mA [6].

At the third step, **correlation tolerance must already be part of the specification, per port, before any run**. A criterion chosen after seeing the mismatch cannot establish correlation.

A state space is the set of all the configurations that a design’s state elements can take. Analog’s state space is smaller, which makes this procedure more affordable than in digital verification. Analog blocks generally have little state, and that amount keeps falling as design style pushes digital logic out to the digital designers. For functionality at the analog block level, all states can typically be simulated exhaustively even at transistor level, giving 100% coverage for the digital inputs [6]. This

avoids the combinatorial explosion that forces constrained-random stimulus on digital blocks. The enumerable modes should therefore all be exercised.

Correlation expiry triggers

A correlation result applies to one revision of one schematic and has a limited lifetime. The flow must repeat correlation as schematics and/or specifications are updated [6]. Three further triggers require attention in addition to these updates:

- **A new operating condition.** A model correlated at nominal supply says nothing about behaviour at the corners of that supply. An entire power-management campaign can therefore run against a model that was never correlated outside its nominal box.
- **A parameter change with no schematic change.** Someone retunes a bias current or a trim default. The schematic revision moves; the model, which hard-codes the old value, does not.
- **A model edit made during debug.** A model edit during debug requires re-correlation, including edits made under pressure to make a failing test pass.

Correlation is a **campaign** with named triggers: it runs the transistor-level schematic at §21.3's cost, and its written result remains valid until a trigger fires. The transistor-level runtime limits how often correlation can run; omitting the campaign leaves chip-level regressions without correlation evidence at tape-out.

Model ownership and effort

One consultancy's experience across many mixed-signal chips puts the creation of correlated analog functional models at as much as 80% of the effort involved in getting analog blocks into chip-level verification. That same experience puts the analog verification effort as a whole at typically 20% of the design effort, rising to 30% or 40% for a team new to it. The same source puts the proportion of analog designers skilled at *reading* models at around 20%, with fewer able to write them [6]. These undated experiential figures describe the problem; they are neither survey data nor a basis for a budget.

A proof-of-concept run took about three months of calendar time, half of it verification planning: the two implementation phases took three weeks and two weeks, and creating and checking the real number model of the block took an expert analog modeler two weeks on top of that [3]. That was 2011, one block, one team. This establishes that a real number model is a specialist deliverable requiring multiple weeks rather than an afternoon, but supports no broader conclusion.

The skill distribution determines ownership: an analog designer or specialist modeler writes the model, while digital verification consumes it; the person able to judge correctness differs from the person noticing errors, requiring a shared artifact.

The model register

Chapter 14’s assumption register records one row per assumption, reviewed like a waiver list, because an unwritten, unproven premise remains unproven at sign-off. An analog model also embodies assumptions and requires this treatment. [Table 21.2](#) records the assumptions and correlation evidence for one model.

Table 21.2 — A model register entry laid out one field per row, filled in for the analog front-end’s ADC: what the model abstracts away, what it was correlated against, under which criteria, and at which conditions. The last two fields are what make the register worth maintaining — a date lets a correlation claim visibly go stale, and the consequence field answers the only question a reviewer needs to ask about a model.

Field	Example
Model ID	MDL-AFE-ADC-01
Block	Analog front-end, 12-bit SAR ADC
Abstraction	Real number model, event-driven, no supply-current behaviour
Correlated against	Schematic rev. 7, transistor level
Correlation criteria	Output code within 1% of full scale (≈ 41 LSB); conversion time within 5%
Conditions covered	Nominal supply, 3 temperature points; corners not correlated
Date and owner	Named analog engineer, with date
If wrong	Every ADC-path result in the chip-level regression, and every plan row that closes on an ADC measurement

A date turns “the model is correlated” into a claim that can visibly go stale, while the consequence field identifies which other results would be wrong if the model were wrong at sign-off.

21.5 Digital verification responsibilities

The digital team’s responsibilities at the analog boundary concern its interactions with the block. The responsibility split used here assigns verification of the analog block to the analog team. The analog team owns performance, noise, linearity and corner behavior, using its distinct tools and expertise. A level shifter is the cell that translates a signal from one voltage swing to another. The digital team owns every digital interaction with that block: the RTL that drives calibration, control loops and power-up sequences is a documented source of chip bugs, alongside missing level shifters and unconnected signals ^[6].

These responsibilities form four categories.

The interface contract. Every control line’s meaning, every status line’s timing, the register map, and the wiring. At the analog subsystem level the wires include supply lines, current bias lines, voltage references and digital control lines, and all of them have to be connected correctly; at chip level there are thousands running from

the digital domain into the analog [6]. Correct connection of those wires is a connectivity problem, and Chapter 15 covers its verification. The analog-specific part is only that a swapped or inverted control line here produces a plausible-looking analog output rather than an obvious failure, so the digital-side check is the one that will catch a swapped or inverted line.

The control and calibration sequences. The algorithm that drives the analog block into specification: trim searches, offset cancellation, lock acquisition, gain ranging. Offset cancellation measures the block's output with no input applied and subtracts what it finds; gain ranging selects the amplifier setting that keeps the signal inside the range the converter can represent. This digital logic operates on analog feedback, and its correctness depends on both a state machine and a tolerance.

The failure handling. What the design does when the analog reports out-of-range, fails to converge, or saturates. Chapter 5 calls this class a must-not-happen feature; at the analog boundary, verification requires explicitly directing the model to produce failure conditions.

The tolerances themselves. Not the physics behind them, but their provenance: every band in the plan traceable to a specification, and the model register kept current.

Scenario. The sensor hub's temperature channel runs a trim search at wake-up: the always-on controller steps a trim code, waits for the reading to settle, and keeps the code whose reading is closest to the on-die reference. **Approach.** Verification of the search must cover termination from every starting code, including the two range ends; ties between two codes; and a broken sensor whose reading never settles, distinguishing convergence on a wrong code from reported failure. All cases run against a real number model with injectable offset, drift and non-monotonicity. **Why.** The analog team owns sensor accuracy; the digital team owns finding the right code within bounded time and reporting failure when the search cannot do so. **The trap.** A search that always converges in the model may be relying on monotonic behavior the real sensor does not guarantee; this modeling assumption belongs in the register with its model.

Scenario. The flash controller reads a page and ECC reports too many errors to correct. Its read-retry state machine walks a table of reference-voltage offsets, re-reading at each until the page decodes. **Approach.** The digital team verifies that state machine: the table is applied in order, the retry budget is respected, a mid-retry power loss leaves the array readable, and exhausting the table reports an uncorrectable error rather than silently returning the last attempt. **Why.** NAND reads are threshold-voltage comparisons, so the analog quantity being manipulated is a reference level; but the correctness question is entirely digital. **What is different here.** The oracle is a *probability*, not a band: with ECC downstream, a marginal read is not wrong, it is more likely to need correction. The plan row closes on a bit-error-rate distribution across the retry table, a third closure criterion after equality and tolerance; every mixed-signal plan row should name its criterion.

The radar front-end illustrates the third responsibility, failure handling. Chapter 5 requires every FFT stage to raise its saturation flag on saturation; a second, upstream obligation requires the receive chain to flag input beyond the amplifier's range and the detection pipeline to discard that frame instead of treating clipping as a target. The two flags are different mechanisms, and both belong on the negative list, the plan's record of what the design must never do. Checking the upstream flag requires driving the real number model beyond the amplifier's range; an amplitude sweep covers this region when recorded captures do not.

Worked artifact: tolerances with one owner

The chapter's opening failure was a tolerance invented in a checker. Every band should reside in one package as a named constant with an explanatory comment, referenced by every checker and coverage bin. The constructs in the package below come from IEEE 1800-2023 (the SystemVerilog language standard), and the tolerance tokens in particular need a tool configured for that language version:

```
package afe_spec_pkg;
  // Bands for the analog front-end. One place, one owner, quoted everywhere.
  localparam real V_REF      = 1.8;           // ADC reference, volts
  localparam real V_MIDSCALE = V_REF / 2.0;   // mid-scale of the input span, volts
  localparam real V_BIN_STEP = 0.05;          // coverage bin width, volts
  localparam real V_BIN_TOL  = 0.005;         // mid-scale bin half-width, volts
  localparam real V_CORR_ABS = 0.01 * V_REF;  // model-vs-schematic band, volts: 1%
  // of full scale, from MDL-AFE-ADC-01
endpackage

module afe_probe (input logic sample_done, input bit corr_run);
  import afe_spec_pkg::*;

  real v_in;    // stimulus, applied identically to model and schematic
  real v_md1;   // real number model output, volts
  real v_ckt;   // transistor-level output, volts; driven, and corr_run high, in
                // correlation runs only

  covergroup cg_afe_input @(posedge sample_done);
    cp_v_in : coverpoint v_in {
      type_option.real_interval = V_BIN_STEP;
      bins span[] = {[0.0 : V_REF]};
      bins midscale = {[V_MIDSCALE +/- V_BIN_TOL]};
    }
  endgroup

  cg_afe_input cg_i = new();

  always @(posedge sample_done)
    if (corr_run && !(v_md1 inside {[v_ckt +/- V_CORR_ABS]}))
      $error("AFE model outside its correlation band");
endmodule
```

The example has three relevant details. `type_option.real_interval` sets the width of the automatically created bins, so `span[]` divides the input range into equal intervals rather than demanding exact values. `midscale` uses an absolute tolerance token, and that token's total width is smaller than the real interval, so `midscale` stays a single bin. A measurement near half-scale lands in that bin instead of falling between two

points that floating-point arithmetic will never produce exactly. The correlation check uses the *absolute* token, discharging §21.1's rule: a band taken as a percentage of the measured value shrinks with the signal and collapses near zero, whereas the same percentage of the fixed full-scale reference is a constant band across the whole range. Across all three, every number is a commented `localparam`, and the correlation band names its register row. The `corr_run` qualifier keeps the nightly regression and the correlation campaigns apart in one file, so a nightly run never compares `v_mdl` against an undriven `v_ckt`.

IN PRACTICE

Two verification populations. A nightly regression built entirely on real number models, and correlation campaigns fired by the triggers of §21.4. Combining the populations can make the nightly job exceed its runtime budget; correlation results also need separate records to remain traceable.

Supply randomization and coverage. A fractional-N PLL multiplies its reference frequency by a fractional rather than a whole number. A covergroup is the System-Verilog container that holds one coverage model. A published fractional-N PLL example randomizes a nominal 2.5 V supply across $\pm 7\%$, deliberately beyond the $\pm 5\%$ legal band; its covergroup has three bins: under-voltage below 2.375 V, healthy from 2.375 V to 2.625 V, and over-voltage above [5]. Other designs need their own supply-voltage limits when applying this randomization and coverage method. In that example, twenty runs report whether the supply ever left its healthy band, enabling this check before a supply-related lab failure.

Model creation and register ownership. The analog engineer writes and correlates the model; the verification team owns its row in the register and its expiry. Maintaining the register requires knowledge of the chip-level runs that depend on the model, while judging its physics requires analog modeling expertise.

PITFALLS

1. **A tolerance invented in a checker.** *Symptom:* an unsourced `$\pm 2$  LSB` literal and disputes about marginal failures. *Cure:* every band is a named constant traceable to a plan row and a specification.
2. **Comparing before the output has settled.** *Symptom:* intermittent failures that move when the model's timestep changes. *Cure:* every analog check states the moment or interval at which it is valid, and the check is qualified by it.
3. **Trusting a real number model for something electrical.** *Symptom:* a loading, droop or current-draw bug found in the lab that no simulation could have shown. *Cure:* enumerate which checks depend on discarded physics, and cover those in a co-simulation campaign.
4. **A model correlated once, at nominal, and never again.** *Symptom:* months of green chip-level regressions against a model that stopped matching the schemat-

ic three revisions ago. *Cure*: the model register, with a date, an owner, the conditions covered, and named re-correlation triggers.

5. **Co-simulating on a fast periodic sampling clock.** *Symptom*: a co-simulation far slower than its analog content justifies. *Cure*: each query is a blocking request into the analog solver; sampling should occur on change where the modeling style allows it.
6. **Verifying the analog block instead of the digital control of it.** *Symptom*: effort spent reproducing the analog team's characterisation, while the calibration sequence's failure paths are untested. *Cure*: the boundary of §21.5, written into the plan as an explicit scope statement.

SUMMARY

- At the analog boundary the oracle stops being an equality and becomes a **band, a window, and a condition set**. Checkers, coverage and regression pass/fail all inherit that structure.
- **Real number modeling** represents continuous quantities at digital simulation speed, making mixed-signal regression practical. Checks dependent on its omitted electrical relationships, continuous time and noise require identification.
- Published speedups are real and are **not composable**: they measure different baselines, different designs, and different years. Every ratio must retain its baseline.
- **AMS co-simulation requires dedicated campaigns**, covering model correlation, loading and current failures, supply sequencing, and one end-to-end run per configuration rather than a regression tier.
- **Analog models require correlation** against schematics with per-port tolerances declared in advance, a model register recording date and owner, and re-correlation on named triggers.
- The plan must assign the digital team the **interface contract, control and calibration sequences, failure handling, and tolerance provenance**, while excluding verification of the analog block itself.

EXERCISES

Exercise 21.1 (calculation). The model register of §21.4 states the ADC model's correlation criterion as an output code within 1% of full scale, noted there as about 41 LSB, while the package of §21.5 states the same band as a real constant equal to 0.01 times the reference. Derive the 41 LSB from the converter's resolution, and express it in volts through the 1.8 V reference of the chapter's opening. Show that the two statements of the band agree. State which of the two forms of tolerance distinguished in §21.1 is in use here, and what the other form would do to this band near the bottom of the input range.

Exercise 21.2 (artifact). Read the model register entry below against §21.4. Name the two claims the entry cannot support while its last two fields are blank, and state

what §21.4 says each of those fields is for. Decide which class of run the conditions line disqualifies this model from supporting, and name the correlation-expiry trigger of §21.4 that such a run would fire.

```
Model ID:           MDL-AFE-PLL-02
Block:             Analog front-end, PLL
Abstraction:       real number model, event-driven; no phase noise
Correlated against: schematic rev. 7, transistor level
Correlation criteria: frequency error inside the plan row's band, per port
Conditions covered: nominal supply, one temperature point
Date and owner:
If wrong:
```

Exercise 21.3 (judgement). A plan carries a single row for the flagship's LPDDR4 PHY training, and the proposal is to close it before bring-up with real number models plus a few co-simulated runs. Decide which obligations of that row the AMS boundary can discharge and which one it cannot, and name the platform where the second closes. Give the property of pre-silicon channel models that §21.3 offers as the reason, and state what §21.3 requires of the plan as a consequence.

Exercise 21.4 (calculation). Section 21.2's table reports the delta-sigma converter finishing in 0.4 seconds against 92.5 for the Verilog-AMS model of the same block. Compute the ratio, and use it to price one hour of real-model regression rerun with the analog solver in the loop. State the conclusion §21.3 draws from that figure about how AMS work is scheduled. Give the reason §21.2 states for not applying the same ratio to the other rows of its table.

Exercise 21.5 (artifact). Read the coverage report below against §21.1 and the package of §21.5. Name the cause §21.1 gives for a bin of this shape never being hit, and the remedy the same section takes from the language standard. Write the mid-scale bin as the package of §21.5 writes it, and give the property §21.5 states for why the repaired bin stays one bin. State why further regression runs cannot close this plan row.

```
Coverage report, covergroup cg_afe_input, after twenty regression runs
  coverpoint cp_v_in
    type_option.real_interval = V_BIN_STEP
    bins span[]               every interval of the span sampled
    bins midscale = {V_MIDSCALE} no hits
  plan row status: open, blocked on the midscale bin
```

FURTHER READING

From the corpus: [3] for metric-driven verification applied end to end to an analog block, including the effort data and the coverage-to-plan mapping; [6] for the trust problem, the specification-driven approach to model correlation, and the effort economics; [5] for real-versus-analog modelling, the runtime comparisons, and worked supply-coverage and real-randomisation examples; [2] for the mixed-signal UVM bridge architecture and the push, push-sync and pull coupling patterns; [1] for the

continuous-discrete boundary and connect-module semantics; [4] for real cover-points, tolerance tokens and real-valued randomisation.

Outside the corpus: Kenneth S. Kundert and Olaf Zinke, *The Designer's Guide to Verilog-AMS* (Kluwer Academic Publishers, 2004), for behavioral modeling discipline; and Peter J. Ashenden, Gregory D. Peterson and Darrell A. Teegarden, *The System Designer's Guide to VHDL-AMS* (Morgan Kaufmann, 2002), for the same material from the VHDL side.

REFERENCES

1. **S5** Accellera Systems Initiative, "Verilog-AMS Language Reference Manual, Accellera Std VAMS-2023," Accellera, February 2024.
2. **S13** Accellera Systems Initiative, "Universal Verification Methodology for Mixed-Signal Standard (UVM-MS), Version 1.0," Accellera, January 2025.
3. **D19** N. Khan, Y. Kashai, and H. Fang, "Metric Driven Verification of Mixed-Signal Designs," Proc. DVCon U.S., 2011.
4. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
5. **D30** J. Brennan, T. Ziller, K. Fotouhi, and A. Osman, "The How To's of Advanced Mixed-Signal Verification," Proc. DVCon Europe, 2015.
6. **D27** H. Chang and K. Kundert, "Specification Driven Analog and Mixed-Signal Verification," Proc. DVCon U.S., 2015.

22. Safety Verification

Safety verification establishes three things about a design: whether it detects physical faults, how much of their effects it detects, and what evidence supports that result.

In this example, eleven weeks before the flagship-A's tape-out, conventional evidence is complete: a verification plan with every row discharged, a coverage database at closure, a passing regression, and a bug curve that flattened a month ago. A bug curve is the plot of bug discoveries over time, and it is the main instrument of convergence. The functional-safety audit additionally requires the evidence requested by an assessor from the customer's safety organization.

The architecture specifies single-error-correcting, double-error-detecting protection for the tightly-coupled memory. Double-bit errors are reported to the safety island inside the fault-tolerant time interval. The fault-tolerant time interval is the budgeted span measured from the moment a fault occurs, and a mechanism must detect the fault and react within that span. The audit requires the faults injected into that path, those the mechanism missed, and the rationale for excluding faults from injection.

The team lacks this evidence because it was never requested. The environment has never corrupted a memory bit on purpose; the error-correcting code was verified like every other block, by checking that correct data goes in and correct data comes out. That establishes functional correctness, while the audit requires evidence of silicon-fault detection and its extent.

Safety verification changes what must be built and measured, which evidence must be retained, and especially the final written argument, which must remain usable after its authors leave.

CHAPTER MAP

§22.1 covers the additional obligations imposed by safety standards. §22.2 distinguishes systematic faults, the subject of the rest of this book, from random hardware faults. §22.2 then connects failure modes and effects to safety mechanisms and required diagnostic coverage. §22.3 covers fault injection, justified fault-list pruning, and the distinction between undetected and non-dangerous faults. §22.4 covers mechanism verification, including the obligation not to fire spuriously; §22.5 covers tool qualification and audit evidence.

22.1 Obligations imposed by safety standards

In 2024, 44 percent of IC/ASIC projects reported working to at least one safety-critical development standard or guideline, and among projects that do, the median spends between a quarter and half of total project time on functional-safety activities

[1]. On FPGA projects the adoption figure is higher still, 63 percent in the same year [2].

ISO 26262 dominates automotive functional safety and defines it as the absence of unreasonable risk from hazards caused by malfunctioning behavior of electrical and electronic systems [3]. That definition, and the rest of the vocabulary this chapter uses, belongs to Part 1 of the series: ISO 26262-1:2018, *Road vehicles — Functional safety — Part 1: Vocabulary*. DO-254 (RTCA, Inc., *Design Assurance Guidance for Airborne Electronic Hardware*, published by EUROCAE as ED-80) is the corresponding regime for airborne electronic hardware, and it appears in the same 2024 survey among the standards that teams comply with [1]. The concrete examples here are automotive; the obligations below apply equally to both regimes.

Process obligations. The standard prescribes how work is organized, not only what is produced: activities have prescribed inputs and outputs, reviews have prescribed independence, and changes have prescribed impact analysis. A controlled work product changes when a team deletes an obsolete test, upgrades a tool, or alters the scope of a coverage model. A coverage model is the multi-dimensional space of situations that a plan requires the stimulus to reach. Every such change requires a record of the decision maker and rationale.

Evidence obligations. Every claim requires an inspectable artifact: a passing regression becomes evidence only when its state is retained with the design revision, tool configuration and raw results, so later review can establish what ran and what it returned.

Traceability obligations. Every safety requirement is linked forward to the thing that discharges it, and every test back to the requirement that justifies it. Chapter 5 requires permanent IDs linking specification, feature, test and result. Safety makes this traceability mandatory, including the backward link from each test to its requirement. A test that traces to no requirement is wasteful and leaves an unexplained artifact in a controlled repository, which an assessor will ask about.

The argument becomes a deliverable. On an ordinary project, the rationale connecting evidence to the shipping decision can remain with the sign-off participants and be lost when they leave. On a safety project, that reasoning is written into a safety case for review and delivery; the evidence supports its claims, and its gaps require explicit arguments. Verification engineers produce results that serve as the premises of somebody else's safety argument. The argument draws a claim from each premise, and for each premise they must be able to show that the premise supports that claim.

Among the biggest functional-safety challenges reported by 2024 IC/ASIC participants, safety architecture and design and safety requirement definition, management and traceability lead, ahead of fault list generation and injection [1].

Allocation of integrity targets

An **ASIL** (automotive safety integrity level, A through D, with D most demanding) applies to a hazard rather than a block. It is arrived at by assessing the risk of a poten-

tial *hazard*, weighing three judgments: how severe the harm would be, how often the vehicle is exposed to the situation, and how well a driver could control the outcome [4]. A headlamp that fails on a lit motorway and one that fails on an unlit country road are the same electronics and different hazards [3].

The integrity level passes down from the hazard to the safety goal, then to system requirements, requirements on the part and requirements on the block; it reaches the block engineer as a number in a requirements document. The block engineer inherits that target and the mechanisms chosen to meet it.

Scenario. The reference SoC has no error-correcting code anywhere. Its fast, four-banked 512 KB tightly-coupled SRAM has no protection: a particle strike that flips a weight-buffer bit produces a wrong answer without an error indication. That was a defensible choice for a low-power edge part whose worst failure is a wrong inference. **Approach.** For an automotive application, the architecture becomes a derivative, the flagship-A, rather than adding detection to the first-generation part: SECDED protects the tightly-coupled memory, L2 slices and LPDDR path, alongside a dual-core delayed-lockstep safety island and monitored reset and clock. **Why.** Detection belongs in the architecture; verification cannot add it. No fault campaign can improve the reference SoC's diagnostic coverage because it has no mechanism whose coverage could be measured. This first judgment call precedes verification: if failure analysis says a failure mode needs covering and no mechanism covers it, the finding concerns the design, and raising it later makes it more expensive to address.

The flagship-A's safety island is built to ASIL D within an ASIL B system. Mixing integrity levels inside one design is ASIL decomposition, the subject of ISO 26262-9:2018, *Road vehicles — Functional safety — Part 9: Automotive safety integrity level (ASIL)-oriented and safety-oriented analyses*. Different parts of one chip carry different targets, and the boundary between them must itself be verified: a fault in the ASIL B region must not defeat the ASIL D region's mechanisms. This requirement explains the safety island's separation. One published account documents an ASIL-rated custom automotive microcontroller core at Infineon: it has 60 instructions, 1.8K flip-flops and 70K gates; it was verified across 16 versions over 5 years of feature updates, architecture upgrades and bug fixes; and it ships in products as different as a 3D camera, an airbag and a pressure monitor [5]. An ASIL-rated core can be small; its obligations, rather than its gate count, make verification demanding.

22.2 Failure analysis as a verification input

A **systematic fault** is a design bug: a mistake made during development, present in every part ever manufactured, and always permanent. A **random hardware fault** is a physical defect arising during manufacturing or operation, permanent or transient: a latch-up, a marginal via, an aging transistor or a particle strike [6].

Almost everything in Chapters 1 through 21 is aimed at systematic faults. Constrained-random stimulus, formal proof, coverage closure, regression engineering,

gate-level simulation, post-silicon bring-up: all of it exists to find design bugs before they ship. A safety standard demands that work too, with additional process, but also addresses physical faults: a design can be perfectly correct and still cause a fatal failure when a particle flips a bit in its configuration register.

Random faults cannot be verified away: physical defects occur at some rate regardless of verification. Safety verification measures how effectively a mechanism detects and safely handles that defect population.

FMEDA as the campaign input

A failure mode, effects and diagnostic analysis (**FMEDA**) produces the verification work list. The part of the series that carries the hardware metrics an FMEDA feeds is ISO 26262-5:2018, *Road vehicles — Functional safety — Part 5: Product development at the hardware level*, and the semiconductor-specific guidance on applying the series to an integrated circuit is ISO 26262-11:2018, *Road vehicles — Functional safety — Part 11: Guidelines on application of ISO 26262 to semiconductors*, which is new in 2018. The FMEDA works element by element, asking for each how it can fail, what the consequence would be, how likely and severe that consequence is, and which safety mechanism handles it [6]. A typical answer names a mechanism from a small, stable family: error-correcting code on a memory, a dual-core lockstep pair (two hardware cores running the same instruction stream, watched by a comparator), a software test library, or a triply redundant structure with a majority voter.

First, **the FMEDA precedes verification evidence**. A safety engineer produces it early, from design data and technology data, with an estimated diagnostic coverage for each mechanism: the fraction of that failure mode’s faults that the mechanism is claimed to catch. Fault injection is what converts that estimate into a measurement [6]. If measured coverage falls below the estimate, the architecture’s metrics (ISO 26262-5:2018) change; if they change far enough, the part misses its target. A late fault campaign can therefore be the last chance to discover an optimistic architectural assumption.

Second, **FMEDA failure-mode granularity determines the fault list**. A failure mode written as “the FIFO indicates its flags incorrectly, so data is overwritten or lost, and the mechanism is redundant flag logic” specifies exactly where to inject and which signal is the alarm [6]. One written as “the block malfunctions” identifies neither and cannot be measured. Granularity should be challenged early, while changes are cheap, rather than leaving revision to the last four weeks.

Required fault classes

Fault campaigns exist to sort a design’s faults into classes the standard recognises (ISO 26262-5:2018, with the defined terms themselves in ISO 26262-1:2018). These names have technical meanings that differ from ordinary English [3]:

- A **safe fault** cannot violate the safety goal because it cannot affect safety-related logic at all or its effect is tolerated.

- A **single-point fault** violates the safety goal and no safety mechanism is present to catch it. This is the class that must be nearly empty.
- A **residual fault** violates the safety goal in a place where a mechanism is present, but this particular fault is outside what the mechanism covers.
- A **latent fault** cannot violate the safety goal on its own, and is neither detected by a mechanism nor perceived ^[6]. A fault counts as perceived when the driver notices its effect although no mechanism has raised an alarm. The fault remains latent until a second fault combines with it. For example, a dead lockstep comparator causes no problem until the two cores disagree and the mismatch goes undetected.
- A **multi-point fault** is a combination that violates the safety goal, and is classified by whether the mechanism detects it, whether it is merely perceived, or whether it stays latent ^[6].

These classes feed the architectural metrics: the **single-point fault metric** (SPFM), the **latent fault metric** (LFM), and the probabilistic metric for random hardware failures (PMHF). Each carries a threshold per integrity level, tabulated in ISO 26262-5. The values reported for ASIL D are SPFM at or above 99 percent, LFM at or above 90 percent, and PMHF below 10 FIT; for ASIL C, SPFM at or above 97 percent and LFM at or above 80 percent; for ASIL B, SPFM at or above 90 percent and LFM at or above 60 percent, with PMHF below 100 FIT at both of those levels ^[4]. Those reported values come from that paper, not from the standard, and the standard is outside this book's corpus. FIT is the unit in which those PMHF thresholds are expressed: a failure rate, measured in failures per billion hours of operation. The standard also specifies the formulas behind those metrics: structural analysis and formal proof establish the fraction of safe faults, while fault injection measures the diagnostic coverage terms that determine whether the metric meets its target ^[6].

Fault counts and failure-rate weights

A campaign counts faults: it injects a number of them and reports how many were detected. Dividing the detected count by the injected count is tempting, but the quotient must be applied to the failure-rate weights before it contributes to the architectural metric.

Architectural metrics are ratios of **failure rates**, rather than fault counts. Each failure mode of each design element carries a base failure rate computed from design data and technology data; the metric formulas combine those rates through the safe, residual and multi-point fractions into a single number for the element and then for the part ^[6]. The campaign's detected fraction is an input applied to a weighted quantity; it is not itself that quantity.

Two campaigns with identical fault counts can produce different metrics: a fault list that oversamples a large but low-rate structure while undersampling a small critical one creates a bias that no coverage report shows. The fault list should be agreed with the safety engineer per failure mode, with results kept separate by mode through reporting. A single aggregate percentage across a whole block cannot be combined with anything the metric formulas use.

Scenario. The brake controller MCU is small enough to enumerate fully: a lock-step core pair, its ECC-protected memories, a watchdog, and a safety island holding the comparator and the alarm aggregation. Its FMEDA fits in one spreadsheet, and every failure mode maps to a mechanism a verification engineer can name.

Approach. The verification lead walks the FMEDA row by row with the safety engineer *before* writing any injection infrastructure, and produces one artifact per row: where faults are injected, which signal is the observation point (where a violation of the safety goal becomes visible), and which signal is the detection point (where the mechanism raises its alarm). **Why.** In this example, that review takes two days and removes almost all late surprises. On the flagship-A, the same review spans multiple weeks and a dozen sub-teams, motivating a full-time safety architect to keep the FMEDA and campaign definition aligned. In these examples, the technique remains the same as the review expands from the brake controller MCU to the flagship-A; coordination increases because the larger review involves multiple sub-teams.

22.3 Fault injection and fault simulation

A workload runs twice: once on the unmodified design, the *good machine*, and once on a second copy that carries one hypothetical fault, the *faulty machine*. At designated strobe times, designated signals are compared; a difference between the two runs means the fault reached that signal [6]. A strobe time is an instant scheduled in advance for that comparison, and the list of those instants fixes what a campaign measures as much as the list of compared signals does.

Two kinds of designated signal have distinct roles:

- **Observation points** are where the failure becomes visible to the outside world: the data output that would now be wrong. A fault seen here is *observed*.
- **Detection points** are where the safety mechanism raises its alarm: the error flag, the comparator mismatch, the interrupt to the safety island. A fault seen here is *detected*.

In the published ECC campaign this chapter draws on, the observation point is the read-data port and the detection points are the single-error, multi-error and any-error signals on the block's boundary [4]. These choices define the measurement.

Error injection, introduced in Chapter 2 and used in Chapters 3, 8 and 10, drives erroneous *stimulus* at an interface to check that the design detects and recovers. **Fault injection** corrupts internal design state, such as a net or flip-flop, to model a

physical defect no interface can produce. The two share infrastructure and answer different questions, and a safety assessor means only the second.

Four fault outcomes

The two axes yield four verdicts. A fault neither observed nor detected did nothing visible in this run. A fault observed and detected is one the mechanism caught: it corrupted something, and the alarm went off. A fault detected but not observed triggered an alarm without corruption; it is noisy but not dangerous. And a fault **observed but not detected** corrupted the design's output while the mechanism stayed silent [4]. This dangerous class is the campaign's target.

A fault that is not detected is not the same as a fault that is not dangerous. A fault never observed in a campaign has two possible explanations that its results cannot distinguish: either it is safe and cannot affect the safety goal under any stimulus, or it is dangerous and the workload failed to excite it [6]. Both produce an identical database row marked "not observed".

Classifying every unexcited fault as safe can inflate the reported metrics arbitrarily; an assessor requires an argument for that classification.

The two forms of evidence answer different questions:

- **Structural and formal arguments answer "under any stimulus".** The cone of influence of an observation point is the netlist region that can reach it. A fault outside the cone of influence of every observation point cannot reach an observation point under any workload whatsoever; this is a statically established property of the netlist, and it scales from block to system [4]. A proof engine is a formal tool that settles a question over every legal stimulus at once. When such an engine shows that a fault can never cause a difference at the observation point, that fault is likewise untestable by construction, not merely unstimulated [6]. These faults are safe and may be excluded.
- **Simulation answers "under this stimulus".** A workload that fails to excite a fault provides no evidence about its safety; the fault must be resolved through better stimulus, proof or a recorded engineering judgment with a named owner.

Pruning is defensible when these two forms of evidence remain distinct, and indefensible when they are conflated.

Fault-list size and justified reduction

Every net, port and sequential element is a fault site with at least two permanent models (held at zero and held at one), plus transient models. Permanent defects are conventionally injected as stuck-at faults, transient upsets as a single event upset: a bit flipped once and then left to propagate or die [3]. On a large block the raw count runs to millions.

The reduction sequence is standard, and each step answers a specific question (Figure 22.1):

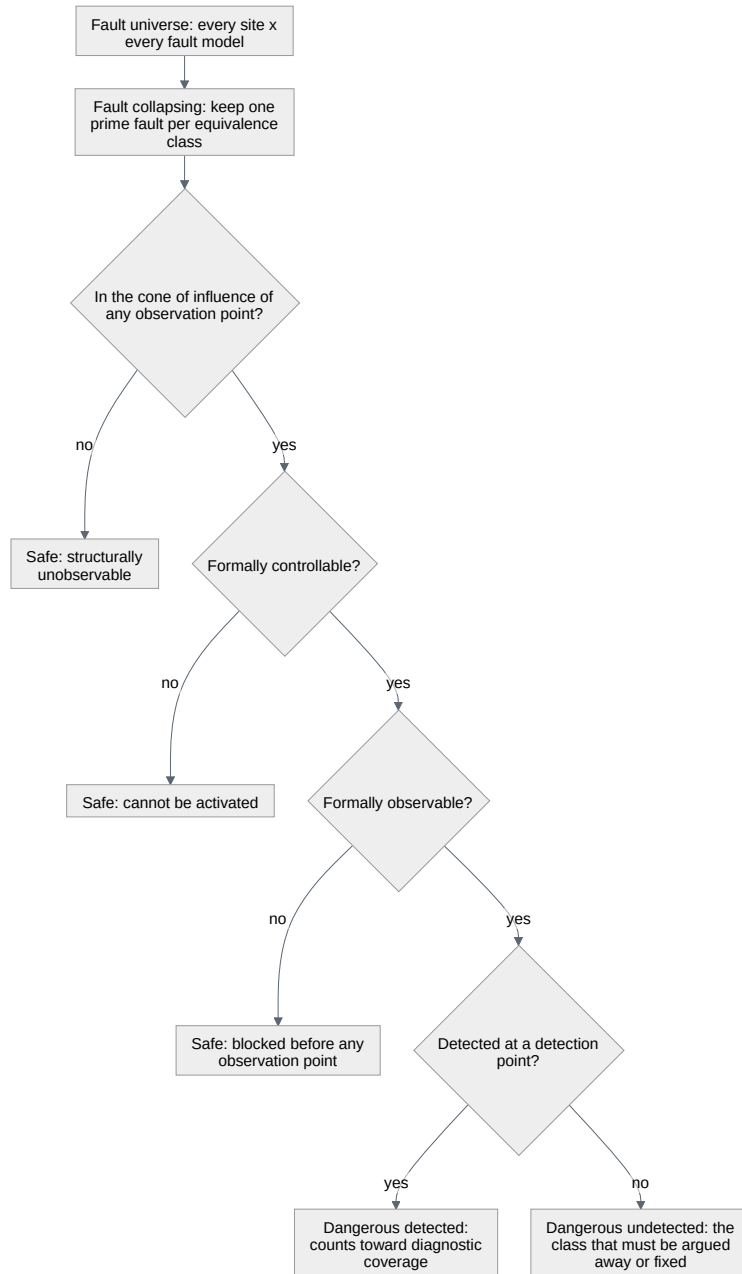


Figure 22.1 — The reduction sequence that takes a fault list from every site crossed with every fault model down to the faults a campaign actually simulates: collapsing, then structural pruning, then formal controllability and observability analysis. Each diamond is a question whose answer holds under any stimulus, and the order is deliberate — cheap arguments first, so the expensive engine only sees what the cheap ones could not settle.

Collapsing first sorts faults into equivalence classes, a *prime* fault represents each class, a *collapsed* fault is one producing the same observable behavior as its prime, and only primes are simulated [6]. **Structural pruning** follows, removing faults outside the cone of influence. Formal controllability and observability analysis then removes faults that cannot be activated or cannot propagate; controllability is the ability to steer a design into a given internal condition using only the interfaces that a testbench can drive. The faults left after these reductions are then simulated.

A design with faults at ports, flip-flops and wires starts from 3,606 faults; collapsing removes 973 and testability analysis another 144, leaving 2,489 to analyse. Structural-

al cone-of-influence analysis marks 763 of those unobservable, leaving 1,726. Formal observability analysis then clears a further 248 (232 in one phase and 16 in a second), leaving 1,478 faults that reach the read-data port [4]. Testability analysis is the second reduction in the counts just given, and the published account names it as a step of its own. It runs before the cone-of-influence pruning rather than as one of the steps the figure above sets out.

The reduction from 3,606 to 1,478 rests on explicit arguments. A collapsed fault is dropped because an equivalent prime fault stays in the list to represent it; every other fault is dropped by an argument that holds under any stimulus whatsoever. The order reduces cost by applying cheap arguments first, so the expensive engine sees only what the cheaper methods could not settle.

Analysis of the campaign plateau

Simulation-based campaigns follow a characteristic curve. Progress is close to linear at first, then flattens, and eventually additional runs classify almost nothing. The remaining unobserved faults require analysis and cannot be assumed safe. Manual review must establish, for every unobserved fault, that design structure rather than an incomplete environment prevented observation; this costs engineer-days to engineer-weeks [4].

Formal proof answers whether a fault can affect the safety goal under any stimulus, so it can convert a not-observed fault into a proven-safe fault, a classification simulation can never establish. The published campaign classified the whole block formally, although it was only almost small enough for convergence. The 8K memory on its periphery would not converge in twelve hours until two transformations were applied in turn: the memory was black-boxed, with its input ports promoted to observation points; and the black box was then abstracted to a pair of back-to-back flip-flops under a constraint fixing the read/write ordering [4].

Promoting the memory inputs to observation points can only make a fault look more observable than it is; this direction of error makes the abstraction usable, because a fault that the abstraction calls safe is genuinely safe, in the sense of the safe-fault class of ISO 26262-5:2018 [4]. An abstraction that can classify a dangerous fault as safe invalidates the evidence. Chapter 14's discipline on sound abstraction therefore applies to this audited evidence.

Detection analysis ran on the faults that structural cone-of-influence analysis left, and that population is larger than the one that the reduction finally left reaching the read-data port. The campaign's final classification divides the 1,726 faults that entered it into four classes: 225 neither observed nor detected, 23 detected without being observed, 966 both observed and detected, and 512 observed but not detected, faults that corrupt read data while the ECC error signals stay quiet [4]. The campaign isolates those 512 dangerous-undetected faults. These results concern an NXP Semiconductors ECC block with single-error correction and double-error detection around an 8K memory [4]. The 512 belong to that block, and a different memory geometry behind a different mechanism would produce a different number.

Scenario. The radar front-end senses for an automotive safety function through a fixed-point DSP chain: FFT pipelines feeding a constant-false-alarm-rate detector. An initial campaign injects stuck-at faults across the FFT datapath and strobes the raw spectrum at the FFT output, reporting low diagnostic coverage: a great many faults change that output without an alarm. **Approach.** A single-bit fault deep in a twiddle-factor multiplier perturbs a bin's magnitude by a fraction of a least-significant bit. The detector's threshold sits far above that perturbation, so the detection list comes out identical: the fault changes the raw spectrum and changes nothing at the interface that the safety requirement is written against. The observation point belongs at the safety requirement's interface, the detection list handed to the fusion stage, rather than at the spectrum. **Why.** The observation point specifies where a fault violates the safety goal. Setting the observation point at the boundary named by the requirement is correct; moving it outward afterwards to improve disappointing numbers is unjustified pruning. The distinction requires a written rationale, agreed before the campaign runs and countersigned by the safety engineer. A perturbation that hides under the threshold on one frame need not hide on the next, so the margin argument must hold across the operating envelope. The margin is therefore a stimulus obligation, not grounds for a simple exclusion; Chapter 21 covers the radar chain's numerical and mixed-signal behavior, while this chapter covers its safety argument.

Time is part of the verdict

Timing adds one axis: whether detection meets the required interval. A safety mechanism must detect and react *within a budgeted time interval measured from the moment the fault occurs*, and must be running continually while the device operates; the fault-tolerant time interval and the fault reaction time interval are both things a fault injection campaign exists to establish [3]. Of the two, the fault-tolerant time interval is the one measured from the moment the fault occurs, and that interval covers detection and reaction together. The fault reaction time interval is the budgeted time that the mechanism has to act once it has detected. The campaign distinguishes detection in time from late detection, which counts as undetected despite the alarm log.

The simulation cannot stop at the first difference; it has to keep running long enough to see whether the alarm arrives inside the interval, which is why fault simulators expose a separate time budget for what happens after the first divergence [3]. That budget must follow the FMEDA timing requirement rather than run-time convenience.

22.4 Safety mechanisms and their verification

A failure analysis will typically name error-correcting code, a dual-core lockstep pair with a comparator, a software test library, or hardware redundancy with a majority voter [6]; with watchdogs and built-in self-test run at startup or periodically, this list covers most mechanisms encountered.

Hardware redundancy means duplicated or triplicated silicon: two cores computing the same thing, or three register copies voted bit by bit. That is not the sense of Chapter 2, where redundancy is a second engineer’s independent reading of the specification. **Lockstep** here means the hardware pair defined above, distinct from Chapter 3’s instruction-set simulator stepping with RTL as a reference model.

A safety mechanism is itself a design with bugs. It was written by the same people, under the same schedule, from the same specification as the logic it protects. Its inactivity in the nominal case can draw verification attention toward the functional path, while its dependencies on the rest of the design can delay integration. These conditions risk leaving a safety mechanism with the least verification on the part, even though the metrics assume it works.

Each mechanism therefore has three verification obligations.

Detection when required. The campaign infrastructure answers the first of the three questions as a by-product.

Detection within the required interval. The budgeted interval is part of the requirement: a mechanism that reports a double-bit error long after the corrupted data has been consumed has not provided useful detection.

Silence during healthy operation. Absence of spurious alarms is also a verification obligation. False positives are a reliability defect in their own right. They can be worse than the fault a mechanism guards against, if they fire on healthy parts in the field more often than that fault would occur. A spurious trip takes the system directly to a safe state rather than degrading gracefully: in a vehicle, a warning lamp, limp mode or dealership visit. Verification that asks only the first question can miss spurious trips that cause returns from otherwise healthy parts.

The two checks differ in scope. Injecting a fault through a script demonstrates whether the mechanism fires. Proving it stays silent means running the design’s *entire legal operating space* with the alarm never asserting, and that obligation has no natural stopping point. It is the shape of any must-not-happen requirement (Chapter 5) and is discharged the same way: an always-on assertion in every regression run, plus a demonstration that the assertion can actually fail.

Scenario. The sensor hub is an always-on part: a 32 kHz island and a DSP, fusing accelerometer and gyroscope streams while the system it serves sleeps. Its safety derivative adds a clock monitor comparing the system clock with the always-on reference and alarming if it stops, slows or runs fast, on the theory that a stopped clock is the one failure the rest of the chip cannot report about itself. **Approach.** The fault campaign stops, halves and doubles the clock, checking the alarm in each case. Healthy transitions also require verification. The watched domain legitimately changes frequency during a low-power transition, legitimately stops during power-down, and legitimately restarts with a ramp; every one is a healthy event that looks exactly like the failure the monitor exists to catch. The obligation is a cross of every legal power and clock transition against the monitor’s alarm, with the alarm asserted as an error. **Why.** This cross catches false alarms on working parts whenever they enter a low-power state, a bug especially hard to find late be-

cause it cannot reproduce on a bench that never exercises low-power sequences. Chapter 19’s power-state work and this cross are the same stimulus written for two purposes, which is a good argument for building it once and sampling it twice.

Scenario. The GPU renders the instrument cluster and its telltales, warning symbols whose absence is itself a hazard. The safety mechanism is frozen-frame detection: logic that watches the display pipeline and raises an alarm if the rendered content stops changing, on the theory that a hung GPU showing a stale frame is indistinguishable, to the driver, from a working one showing good news. **Approach.** Stalling the pipeline and checking the alarm readily verifies detection. Verifying silence is harder because a correct instrument cluster is legitimately static most of the time. A parked vehicle with no faults displays an unchanging image for as long as the ignition is on. Simple frame comparison cannot serve this purpose; the renderer must update an injected counter or pattern every frame, providing a known-varying region so that “unchanged” unambiguously describes the pipeline rather than ambiguously describing content. **Why.** The alarm condition must be false exactly when the design is healthy; a heuristic merely correlated with health cannot meet this requirement. A heuristic under verification should be referred back to the architect early.

A worked artifact: the non-firing obligation

The third obligation has this assertion form on the flagship-A’s ECC path. The mechanism corrects single-bit errors and flags them for the error counters, and reports double-bit errors to the safety island as a fault. The obligation is that neither flag asserts while nothing is wrong.

```
// In the checker bound to the ECC wrapper on the flagship-A's memory.
// inject_active is the campaign's own control: high only while a fault
// is deliberately present. Every other cycle is a claim about health.
property p_no_spurious_ecc_alarm;
  @(posedge clk) disable iff (!rst_n)
    !inject_active |-> !(ecc_err_single || ecc_err_double);
endproperty
a_no_spurious_ecc_alarm : assert property (p_no_spurious_ecc_alarm);

// The mechanism must also be alive, not merely quiet: a mechanism that
// can no longer fire is a latent fault, and silence is what it looks like.
c_ecc_double_reported : cover property (
  @(posedge clk) disable iff (!rst_n)
    inject_active && ecc_err_double
);
```

The assertion alone cannot establish health: an ECC wrapper whose error outputs never assert because a design bug tied them off or a fault killed the mechanism passes `a_no_spurious_ecc_alarm` on every run of every test indefinitely. A dead mechanism is silent and is a latent fault under the classification above. The cover directive checks that silence accompanies a working mechanism. It applies Chapter 8’s known-bad-variant argument and Chapter 6’s treatment of vacuous passes to the mechanism itself rather than to a checker.

`inject_active` preserves the assertion's use during campaigns. Without that qualifier, the assertion fires on every fault campaign run, prompting the team to disable it during campaigns; it then protects nothing during the runs where the ECC path is most exercised. Gating it on injection control keeps it active in campaign runs whenever no corruption is being injected.

Verifying a lockstep pair

For two cores and one comparator, injecting a fault into one core and checking for a mismatch is insufficient. A design with a serious defect can pass that test.

What makes a lockstep pair effective is that the two cores fail *independently*. On the flagship-A the pair is delayed: the shadow core runs some cycles behind, so a disturbance hitting both at the same instant hits them at different points in their execution and produces a mismatch instead of two identical wrong answers. That delay, and the separation behind it, is the mechanism. Verification must establish three things. The first is whether the delay pipeline preserves comparison across every stall, flush and interrupt. The second is whether the two cores share any resource whose single failure would corrupt both identically, such as a clock buffer, a power rail, a memory port or a reset synchronizer. A reset synchronizer is the structure that asserts reset asynchronously and releases the reset on a clock edge, so that no flop meets the release inside its recovery window. The third is whether something else catches the comparator's own failure. That last question is the latent-fault question again, and its usual answer is a periodic self-test that provokes a mismatch on purpose and confirms the alarm.

None of that is found by injecting a fault into one core. Dependent failure analysis instead asks what a single physical event could do to both halves at once; verification engineers contribute to this separate discipline rather than owning it. They must supply the netlist's actual shared-resource list.

22.5 Tool qualification and the evidence trail

A certification body needs the simulator's identity because verification evidence is tool output. Safety-case artifacts, including coverage databases, fault classifications and proof results, come from software that could, in principle, be wrong. If the tool is wrong the evidence is wrong, and the argument built on it is unsound while looking exactly as sound as before.

Tool qualification is the response (ISO 26262-8:2018, *Road vehicles — Functional safety — Part 8: Supporting processes*). Tool qualification asks two questions. The first is how much damage the tool could do by malfunctioning, either by introducing an error into a safety-related work product or by failing to detect one already there. The second is how likely something else in the flow is to catch that malfunction. Greater potential damage and less independent checking require more supporting evidence, supplied through a vendor qualification package, a demonstrated history of use, or the project's own confirmation testing, a suite that the project runs on the tool itself, to check its output on the use cases that the evidence relies on.

Qualification is scoped to a tool version, a set of use cases and often a configuration (ISO 26262-8:2018). This scope has three operational consequences.

- **Version pinning becomes a safety obligation.** The exact tool build must be retained with the evidence. Chapter 12's run manifest (seed, design revision, test-bench revision, exact tool build, configuration, host) becomes the unit of the evidence trail.
- **A mid-project upgrade requires renewed justification.** A required vendor fix raises the question of which evidence from the old version must be re-established, beyond whether the fix installs. Without advance planning, this obligation may emerge in month nine.
- **Qualification applies only within its use cases.** A simulator qualified for the use case being exercised supports evidence through that qualification; the same simulator driving a flow outside its qualified use case rests on trust in the tool alone.

When qualification is unavailable or too narrow, a second engine should independently check the same property, so a tool error must occur twice and identically to survive.

Flow qualification

A second, separate question concerns whether the verification environment can find the bugs it claims to seek. Safety assessment asks for evidence of flow robustness: deliberate systematic-fault injection at architecture, system and register-transfer level measures verification completeness through the faults the environment detects [6].

This is the industrial form of the known-bad variant from Chapter 8: instead of keeping one deliberately broken copy of the design in the repository, the flow builds a systematic population of small mutations, runs each against the environment and classifies each as detected or not. Undetected mutations identify places where a real bug of the same shape would also have gone unnoticed, pointing to a missing checker or an unsampled signal rather than to a missing test.

Tool qualification asks whether the software can be trusted; flow qualification asks whether the environment can fail. An assessor requires both arguments, and neither substitutes for the other.

Traceability through the audit

The traceability chain a safety project is asked for runs further in both directions than the one Chapter 5 builds, and it must remain navigable years after everyone has moved on:

vehicle-level hazard → safety goal → safety requirement → the design feature that implements it → the failure modes that feature can exhibit → the safety mechanism that covers them → the plan row that discharges the mechanism's verification → the specific runs that produced the evidence → the archived results → the report that quotes them.

That chain crosses three parts of the series: Part 3, which holds the concept phase and the hazard analysis that assigns the ASIL; ISO 26262-4:2018, *Road vehicles — Functional safety — Part 4: Product development at the system level*; and ISO 26262-5:2018 at the hardware level.

Two traceability links require particular attention. First, failure modes and plan rows can drift after design changes because different people maintain the FMEDA and verification plan in different tools. Second, reports based on summaries can lose their link to archived results when summaries outlive raw data. Each plan row should carry its failure-mode identifier, and every number quoted in a report must be reproducible from a retained manifest.

Here is what a plan row with a certification obligation looks like in this book's five-column format, on the flagship-A's ECC-protected memory (Table 22.1):

Table 22.1 — Three plan rows, in this book's five-column format, for one safety mechanism on the flagship-A's ECC-protected memory: that the mechanism fires within its budgeted interval, that it raises no alarm on healthy traffic, and that the classification's residue is argued rather than assumed. Three rows because there are three obligations, each of which an assessor will ask to see discharged on its own.

ID	Feature & attributes	Pass/fail criterion	Method	Closure metric
FA-SM04a	SECEDED ECC on the flagship-A's tightly-coupled memory detects double-bit errors. FMEDA failure mode TCDM-FM03. Attributes: error type {single-bit, double-bit, address decode} × access {read, write} × requester {cluster core, DMA} × concurrent bank activity {idle, contended}	Every injected double-bit error raises <code>ecc_err_double</code> to the safety island within the fault reaction time budgeted for TCDM-FM03	Fault injection campaign, permanent and transient models, on the gate-level netlist	Fault classification report for TCDM-FM03 complete: every fault in a class, dangerous-undetected list itemised and dispositioned
FA-SM04b	The mechanism raises no alarm on healthy traffic, including low-power entry and exit	<code>a_no_spurious_ecc_alarm</code> holds in every run in which injection is inactive	Constrained-random sim (always-on check) + power-aware sim	Check enabled and passing in 100% of regression runs
FA-SM04c	The classification's residue is argued, not assumed	Every fault left not observed carries either a structural or formal argument that it cannot violate the safety goal, or a named engineering judgment	Formal observability analysis, then review	Zero faults in state not observed, unargued at sign-off

Three plan rows serve three obligations under Chapter 5's one-row-one-metric rule, which applies at least as strictly to safety: each plan row must be discharged separately for assessment. The cover directive from the artifact above needs no row of its own: row FA-SM04a's campaign hits it the first time an injected double-bit error is detected, and a campaign in which it is never hit has already failed that row. Row FA-SM04c addresses the plateau by requiring an argument for faults left unobserved; it does not promise that every fault is safe, but prohibits silently assuming safety.

Any remaining hole requires Chapter 7's waiver with the same five fields: the hole, reason, risk argument, owner and expiry. Safety changes one aspect of that record: its signatory. The verification lead owns the statement's accuracy; the safety engineer owns residual risk and the decision to accept it.

Engineering and audit documentation

Engineering work includes fault campaigns, observation-point and detection-point definitions, abstractions enabling formal convergence, verification of every mechanism's false positives, lockstep dependent failure analysis, and arguments separating proven-safe from merely unstimulated faults. This work finds real defects outside the remit of ordinary functional verification, the practice that establishes before fabrication that a design implements its specification. The same work also requires precise statements of what was measured and against what, a discipline applicable to other verification work.

Audit documentation includes retyping the same information into a third tool because the traceability database cannot import from the plan; review records whose value lies in the signature rather than the review; templates whose inapplicable sections cannot be deleted; and periodic reissue of documents whose content has not changed. These tasks make evidence auditable, meeting a documentation requirement. When repetitive steps have not been automated, engineers carry out that work manually; the automation plan should distinguish these steps from engineering activities such as fault campaigns.

Resentment toward both documentation and fault campaigns can undermine safety work, while a team that treats all documentation as engineering risks burnout. When the schedule slips, the plan should distinguish engineering work that needs protection from documentation that needs automation. Generating traceability artifacts from the plan and run database avoids maintaining them by hand. The funding plan should identify this as a tooling project and avoid postponing it until the second program.

IN PRACTICE

Campaign infrastructure should start before the FMEDA is final. The schedule should allow weeks to build the injection harness, strobe definitions, good-machine reference run and results database, which barely depend on the faults eventually injected. Waiting for a signed FMEDA risks putting the campaign on the critical path.

The netlist campaign requires a separate budget. Fault models describe physical structures, so a gate-level campaign matches failure-rate data, but runs far slower than register-transfer simulation and becomes available late. An early register-transfer campaign should check mechanism logic and architectural assumptions, followed by a netlist campaign for evidence. The budget must therefore cover two campaigns rather than one.

Workload selection requires agreement. Diagnostic coverage is measured under a workload. In this example, leaving half the design idle leaves half its faults unobserved, on two assumptions: that the idle portion contains the corresponding share of fault sites, and that the workload does not excite those faults. The workload must activate the mechanisms yet remain short enough for thousands of faulty-machine runs within the compute budget; its written engineering rationale is one of the first items an assessor reads.

Fault campaigns should remain outside nightly regression. A fault campaign follows Chapter 12's definition: launched for a stated reason, assigned an owner, producing a written result and then stopping. The nightly suite carries the mechanisms' always-on checks instead.

PITFALLS

1. **Counting faults where the metric weighs failure rates.** *Symptom:* one aggregate diagnostic-coverage percentage for a whole block, which the safety engineer cannot use. *Cure:* results must be classified and reported per failure mode so the FMEDA can apply base failure rates; architectural metrics must never be computed by dividing fault counts.
2. **Un-excited faults recorded as safe.** *Symptom:* diagnostic coverage that improved sharply in the last two weeks with no change to the design or the workload. *Cure:* a fault is safe only when a structural or formal argument shows it cannot violate the safety goal under any stimulus; everything else is unresolved and needs a name on it.
3. **Verifying only that the mechanism fires.** *Symptom:* a green campaign, and field returns from healthy parts. *Cure:* an always-on assertion that the alarm stays silent outside injection, exercised across every legal power, clock and reset transition, plus a cover proving the alarm can still fire.
4. **A mechanism that has silently died.** *Symptom:* the non-firing assertion passes on every run for a year. *Cure:* every silence assertion requires a cover directive on the alarm's positive case, and the mechanism's self-test must run in the gating tier.
5. **Moving the observation point to improve the number.** *Symptom:* a classification that changed after someone "corrected the strobe list". *Cure:* the observation point must follow the safety requirement, with the safety engineer's counter-signature; changing it requires a written rationale.
6. **Tool versions treated as environment rather than evidence.** *Symptom:* the build that produced last quarter's fault report cannot be identified. *Cure:* the run

manifest is the evidence unit; the build must be pinned, the manifest archived with results, and upgrades treated as controlled work-product changes.

SUMMARY

- Safety verification does not replace functional verification; it adds a second target. Design bugs are systematic faults and always permanent; random hardware faults are physical defects arising in manufacturing or operation, permanent or transient, and cannot be verified away; they can only be detected and handled [6].
- The FMEDA fixes failure modes, injection points, alarms and required diagnostic coverage; it must be reviewed early enough to negotiate granularity.
- The deliverable is fault classification: safe, single-point, residual, latent and multi-point are defined classes with per-level metric thresholds, with SPFM at or above 99 percent and LFM at or above 90 percent for the most demanding level [4].
- Unobserved faults remain unresolved. Structural and formal arguments answer *under any stimulus* and license exclusion; simulation answers *under this stimulus* and does not by itself justify classifying an unobserved fault as safe. The residue left when the campaign plateaus is analysed, argued or fixed.
- Safety mechanisms risk receiving the least verification even though the metrics assume they work. Verification must establish detection, its timeliness and silence on a healthy design.
- Evidence is tool output, making the tool part of the evidence. Qualification is scoped to a version and a use case, making version pinning a safety obligation and a mid-project upgrade a re-argument.
- Chapter 7's sign-off asks whether evidence justifies shipping; safety makes that reasoning a written deliverable that must withstand review years later by someone absent from sign-off.

EXERCISES

Exercise 22.1 (calculation). Take the fault-list reduction §22.3 reports for the published campaign. Compute the number of faults the reduction removed in total, and split that total between the faults dropped because an equivalent prime fault stays in the list to represent them and the faults dropped by an argument that holds under any stimulus. State the reason §22.3 gives for applying the reductions in the order it does.

Exercise 22.2 (calculation). Take the campaign's final classification in §22.3 together with the count of faults the reduction left reaching the read-data port. Compute which two of the four classes sum to that count, and compute the difference between the classification's own total and it. Name the reduction step whose count that difference matches, and state what §22.3 requires before a fault left unobserved may be classified as safe.

Exercise 22.3 (judgement). A campaign report reaches the safety engineer as a single diagnostic-coverage percentage for a whole block, obtained by dividing the faults the mechanism detected by the faults injected. Decide what the FMEDA can do with that percentage, and give the property of the architectural metrics stated in §22.2 that makes it unusable. Name the reporting granularity §22.2 requires instead, and state how two campaigns with identical fault counts can arrive at different metrics.

Exercise 22.4 (judgement). Two faults in a campaign's database carry the same row, marked not observed. One of them lies outside the cone of influence of every observation point. The other was never excited by the workload the campaign ran. Decide which of the two may be removed from the fault list, name the property of the evidence that separates them in §22.3's own terms, and name the three dispositions §22.3 leaves open for the fault that stays.

Exercise 22.5 (artifact). Read the plan row below, given field by field for the clock monitor on the sensor hub's safety derivative. Name the field left empty and state what §22.5 says the traceability chain loses without it. Name the two obligations of §22.4 that the pass/fail criterion leaves untested, and, for the one §22.4 discharges with a cross, name the stimulus that cross requires.

ID:	FA-SM07a
Feature & attributes:	The clock monitor on the sensor hub's safety derivative alarms when the watched system clock stops, slows or runs fast.
FMEDA failure mode:	
Pass/fail criterion:	The alarm asserts in each of the three injected clock disturbances: stopped, halved, doubled.
Method:	Fault injection campaign on the clock monitor
Closure metric:	Three injected disturbances, three alarms

FURTHER READING

From the corpus. [6] provides an overview of how functional safety verification is structured around ISO 26262: the systematic-versus-random split, the FMEDA process and its columns, the fault classification tree, the metric formulas and their failure-rate weighting, the seven-step fault campaign flow, and on the evidence side, mutation-based qualification of the verification flow itself for Part 8 and 9 assessments. [4] is the case study this chapter's numbers come from: a formal-assisted fault campaign on a single-error-correcting, double-error-detecting block, with the full reduction chain from the raw fault list to the dangerous-undetected residue, and a candid account of the manual-analysis cost that motivates the approach. [3] covers the fault-injection side in more detail: permanent and transient fault models, failure and detection strobes, static pruning by cone of influence, the mapping from analysis statuses to the standard's fault classes, and the timing dimension of detection. The industry adoption and effort figures are from the 2024 Wilson Research Group studies [1, 2]. Chapter 14 covers the abstraction discipline formal fault classification depends on; Chapter 12 covers the run manifest the evidence trail is built from; Chapter 21 takes up the analog and mixed-signal side of the radar chain.

Not in the corpus. The standards themselves are the primary references and should be read directly rather than paraphrased: the ISO 26262 series, *Road vehicles — Functional safety*, particularly Part 3 on the concept phase, where the hazard analysis and risk assessment that assigns an ASIL is defined; ISO 26262-5:2018, *Product development at the hardware level*, which carries the architectural metrics and their target values; ISO 26262-8:2018 on supporting processes including tool qualification; ISO 26262-9:2018 on integrity-level-oriented analyses; and ISO 26262-11:2018 on the application of the standard to semiconductors. Those descriptions are of each part's scope; only Part 5's title is quoted. For airborne electronic hardware, RTCA DO-254 / EUROCAE ED-80, *Design Assurance Guidance for Airborne Electronic Hardware*. The IEC 61508 series is the general industrial functional-safety standard from which several domain standards derive, and the one most IC/ASIC projects reported working to in the 2024 survey [1].

REFERENCES

1. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
2. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
3. **D2** Sessa Sai Kumar C V, A. Mouallem, and J. Mazzawi, "Fault Injection Analysis for Automotive Safety and Security," Proc. DVCon Europe, 2022.
4. **D12** N. Ahuja, M. Agarwal, and S. Jana, "Formal Assisted Fault Campaign for ISO26262 Certification," Proc. DVCon India, 2019.
5. **D24** S. Mitra, E. Singh, and K. Devarajegowda, "QED & Symbolic QED: Pre-silicon Verification, Post-silicon Validation, Industrial Results," Proc. DVCon Europe, 2019.
6. **D32** J. Richter, "Unified Functional Safety Verification Platform for ISO 26262-Compliant Automotive," Proc. DVCon Europe, 2019.

23. Security Verification

Security verification establishes claims against a named adversary, beyond functional conformance to a specification. The flagship SoC's root of trust is signed off twice through functional and security reviews.

The first, functional review passes. The inline AES-GCM engine on the memory path has behind it a reference model, an independent executable of the specification taken as golden and driven by the same stimulus. It also has a scoreboard, the structure that holds the value expected at each output and is consulted when that output is observed. The scoreboard has compared ciphertext beat by beat for nine weeks; a beat is a single data transfer inside an AXI burst. Every plan row is discharged; a plan row is the line of the verification plan that carries one feature, its pass/fail criterion, its method and the evidence that closes it. Chapter 15's connectivity job has run at every integration build, with both positive and negative connectivity rows. That job proves the security requirement: no structural path from any key-ladder output to a bus-readable address, to the debug data register, or to a scan-observable point; §23.4 returns to these three destinations, and to what a structural result leaves open. A connectivity job means checking each row of an integration table to decide whether a structural path runs from a named source to a named destination. The bug curve is flat and the fresh-seed yield is near zero: bug discoveries plotted over time have leveled off, and new random seeds are no longer finding bugs. The block meets every completion criterion from Chapter 7.

The second, security review asks what an attacker wants, what they can do, and where those two meet; the verification plan omits these questions. In this example, the review identifies unplanned obligations within an hour. Nothing in the plan says what the engine's completion latency may and may not depend on. It also omits what happens to a working key when the key ladder's domain goes down during an operation. The plan also leaves one question unspecified: whether an open debug port can reach the key ladder even though no structural connection joins the two. The trusted authenticated unlock path is what opens that port. The plan names no adversary, so it gave no reason to ask these questions.

The functional plan has not failed: it establishes completely that the engine implements the specification. Security asks whether the engine can be made to do something unspecified; no quantity of functional evidence answers that question.

The gap can be wide even for a fully functional block. A published information-flow study took an AES encryption block from a public repository and checked one rule against it: the key must not flow to the data output. The block was unmodified and fully functional, and it encrypted correctly across thousands of test vectors. The rule fails because the key, exclusive-ORed with the data, reaches the output pin as plaintext ^[1]. Functional tests that check only the specified encryption behavior do not check this confidentiality rule.

CHAPTER MAP

§23.1 explains why the adversary sets the scope and prevents closure through functional coverage. §23.2 develops a versioned threat model from assets, adversary capabilities and trust boundaries, turning its rows into checkable claims. §23.3 covers confidentiality and integrity as information-flow properties, the suitability of formal methods and their limits. §23.4 examines information-flow evidence and the effect of its exception clauses. §23.5 distinguishes side channels below RTL abstraction from claims supported by pre-silicon evidence; §23.6 separates verifiable properties from those that can only be argued.

23.1 The adversary defines the scope

Every technique in the preceding twenty-two chapters shares one premise: there is a specification, and the job is to establish that the design implements it. The plan enumerates its features (Chapter 5), the coverage model partitions its input space (Chapter 6), and the oracle encodes its expected response (Chapter 3). Even the negative work is specification-derived: a *must-not-happen* feature is a requirement the specification states, and an `illegal_bins` declaration is a rule someone wrote down.

Security verification has no such anchor. Its property is “the design cannot be made to do Y”, where an adversary chooses Y after reading the specification, the datasheet and possibly the RTL, specifically seeking unspecified behaviors. These differences have four structural consequences for verification.

Negative properties require evidence beyond positive behavior. “This key never reaches that bus” is a claim about the complement of the design’s intended behaviour. However much of that behavior is demonstrated, demonstrating intended behavior cannot establish a claim about its complement; this is what the opening AES result shows, and it is why a block can pass every vector and still fail the one confidentiality rule. Chapter 3’s distinction between specified behavior and everything else defines this subject; that distinction is usually a footnote about the oracle’s projection, the part of correctness that the oracle checks rather than correctness itself.

Rarity does not reduce adversarial risk. Risk-driven verification (Chapter 3) allocates effort by likelihood times cost, and likelihood is computed against a stimulus distribution the team controls: a constrained-random generator draws from a distribution, so a scenario needing four improbable conditions at once is improbable. An adversary instead optimizes within a budget: in this example, a month spent arranging exactly the four conditions that the generator would only stumble into is enough to make them repeatable on demand. Thus unlikeliness, central to functional prioritization, provides no defense within the threat model’s scope.

The failure is often functionally silent. A confidentiality breach need not corrupt any output, and an integrity violation may hold only briefly before something masks it, so detecting one usually means correlating several kinds of evidence rather than watching for a wrong value ^[2]. In Chapter 3’s terms, the bug is *activated* and it *propagates* to an observable point, and activation and propagation together consti-

tute the failure rather than its detection; nothing *checks*, because no checker was written for a correct value.

Coverage does not close. There is no enumeration of everything an attacker might try, hence no denominator or percentage. Functional metrics (line, toggle, FSM) correlate only loosely with the attacker’s search space; the lack of a settled, tool-friendly security-coverage definition is among the area’s most consistently reported gaps [2]. This is not a temporary tooling shortfall: it is what happens when the space being covered is defined by an opponent rather than by a document.

Evidence and residual risk

Security verification produces evidence for *named* claims against a *named* adversary, without a completeness argument. The output has four parts: a threat model, a set of properties discharged against it, a set of properties *not* discharged with their reasons, and a residual-risk statement someone signs. That statement matches Chapter 7’s sign-off with one difference: the residual is not merely unquantified, it is unquantifiable, because an unenumerated space cannot be counted.

When the space cannot be bounded, the analysis must be explicit about which part it examines, and each examination must be exhaustive within its stated scope.

Architecture review is manual and rarely reaches the implementation as built. Red-team exercises are essential for the finished product and find nothing before silicon exists, and their results are hard to measure. A red-team exercise is one in which an authorized team attacks the product as an adversary would. Formal methods belong inside a larger strategy and do not scale to a large design running hardware and software together. A directed test is a piece of stimulus whose content and order are written out by hand, and it does the same thing on every run. Directed tests take substantial effort to write and mostly re-confirm vulnerabilities already known [1]. A plan relying on any one of these techniques alone retains that technique’s stated limitations.

Verification after the threat model

Everything downstream of the threat model is verification: properties, assumptions, stimulus, coverage, waivers, sign-off. Once someone has written “the working key must not be observable at the debug port under adversary capability C3”, establishing that claim uses the methods developed in the preceding twenty-two chapters.

The threat model should be the first engineering artifact, recording the specialist inputs on assets, adversary capabilities and trust boundaries.

23.2 The threat model as the plan’s root

A threat model has three interdependent parts.

Assets are the things worth protecting. Asset identification comes first in the literature of the discipline, because it fixes what must be protected, and until that is

settled no threat, property or test can be defined at all; where it is left late, the fixes land in the far more expensive post-silicon stage [3]. On the flagship, cryptographic assets include the device root key that is burned into the OTP fuses, the working keys that the ladder derives from it, and the entropy that the TRNG produces. Assets omitted from the functional plan require particular attention. The boot measurement is an asset. It is the hash of each boot stage that the root of trust records before it runs that stage. The configuration of the firewall and the address decode is a single asset, and what makes it one is that it enforces every other protection: if an adversary can rewrite the enforcement, every other protection can be bypassed. An asset need not be secret: the flash controller's anti-rollback counter is public by design, and its protection depends on no adversary being able to *decrement* it, so it needs an integrity property rather than a confidentiality one. Classifying assets by which of the two they need prevents encryption from becoming the sole security response.

Adversary capabilities say what the opponent can do. Only an enumerable set can be covered to completion: capabilities form such a set, whereas attacks form an open set. The model should therefore list capabilities. The flagship's threat model uses four capability tiers:

- **C1:** unprivileged software running on the host cluster: it can issue loads and stores, program what it is allowed to program, and time itself.
- **C2:** privileged software on the host cluster: everything in C1, plus configuration of the interconnect, the memory management unit, the power controller and the debug fabric.
- **C3:** physical access to the package pins: probing the debug interface, measuring supply current and electromagnetic emission, perturbing clock and supply.
- **C4:** invasive physical access, up to decapsulation and direct probing of the die.

The flagship declares C4 **out of scope**, and the reason is written down rather than assumed: countering it is a packaging, sensor and countermeasure-design problem whose evidence does not come from RTL verification, so including it would create plan rows that no pre-silicon activity could ever discharge. An out-of-scope declaration needs a reason, and the reason is what makes the escape risk visible; an escape is a bug that crosses a sign-off boundary undetected, found on the side that was supposed to catch it.

Trust boundaries are the surfaces where the capability tier changes. They make the first two lists checkable: a property is always "asset A does not cross boundary B under capability C". Modern SoCs make this hard because they stack heterogeneous IP, complex interconnect and several privilege boundaries into one attack surface that is large and awkward to analyze [3]; this requires explicit boundaries. [Figure 23.1](#) places the flagship components relative to those boundaries.

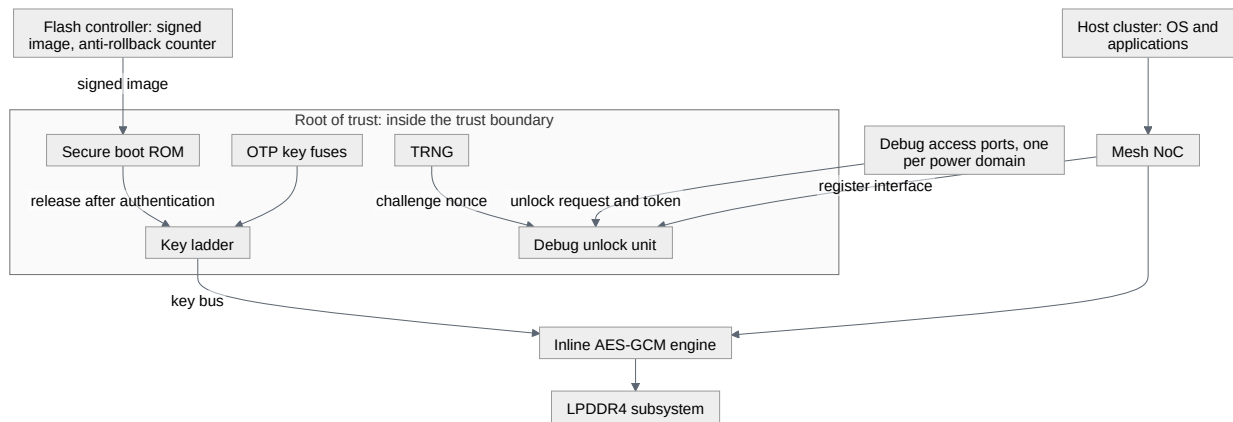


Figure 23.1 — The flagship’s root of trust, asserting one thing: which blocks sit inside the trust boundary — fuses, key ladder, secure boot ROM, TRNG, debug unlock unit — while everything drawn outside it is reachable, in some mode, by an adversary at C2 or above. The inline AES-GCM engine is outside the box deliberately: it holds key material and sits on a path carrying untrusted traffic, and boxing such a block on one side would tell a lie about it.

The inline engine is deliberately outside the box because it is the one component that straddles the boundary: it holds key material from the ladder and it sits on a memory path that carries untrusted traffic and terminates in untrusted DRAM. A published diagram of the Rambus CryptoManager RT-360 shows a RISC-V CPU beside an OTP management core, a key transport core, a TRNG management core, an SRAM interface behind a memory protection unit, and AES, hash and public-key engines on a bus fabric; those are blocks on a diagram, while the failure classes that a security-verification vendor names against that part are architectural rather than cryptographic: assets mismanaged during a secure boot, a memory protection unit misconfigured, keys or configuration registers not cleared, debug modes left enabled [1]. No measurement travels with that list and none is claimed for it here; what the list establishes is that those surfaces belong to a shipping part rather than to this book’s example.

The rows

The three lists form the table whose obligations the rest of the chapter addresses. The table has six rows, and each of them names an asset, a capability, a boundary and its obligation (Table 23.1):

Table 23.1 — The flagship’s threat model as rows: each crosses one asset with the adversary capability it must withstand and the boundary where that capability meets it, then states the obligation. Read the last column first. Three rows name a single technique, two name a pair, and TM-06 names none — which is the table doing its job rather than a defect in it.

ID	Asset	Capability	Boundary	Obligation	Discharged by
TM-01	Device root key (fuses)	C1	Root-of-trust register interface	No sequence of register accesses makes fuse content observable outside the ladder	Information flow, RTL (§23.4)

ID	Asset	Capability	Boundary	Obligation	Discharged by
TM-02	Working key	C2	Memory path through the engine	Key material influences no value on the memory path except through the cipher	Information flow + Chapter 15's negative connectivity rows
TM-03	Working key	C3	Debug access port into the root-of-trust domain	A debug request addressing the key region is never granted, unlocked or not	Formal, on the always-on control logic (§23.3)
TM-04	Working key	C2	Power-domain edge of the root of trust	Powering the domain down destroys the key; leaving the unlocked state requests a wipe	Formal + power-aware simulation (Chapter 19)
TM-05	Anti-rollback counter	C2	Flash controller register interface	No command sequence decreases the counter	Formal, on the counter's control logic
TM-06	Working key	C3	Engine's physical emissions	No key material is recoverable from the engine's power signature	Argued, not verified — see §23.5

Three rows name a single evidence-producing technique, two name a pair, and TM-06 names none because it cannot be discharged at RTL. Including only closable rows would derive the threat model from available tools.

Threat-model versioning and review

The threat model should follow Chapter 5's treatment of the verification plan: a versioned document, under review, with traceability from each threat-model row to the evidence that discharges it, and with a named owner. That traceability identifies the rows whose evidence must be reviewed after a design change. lowRISC's OpenTitan project publishes its threat model as a separate standing document, self-described as lightweight: the document lists in-scope assets, attacker profiles, attack surfaces and attack methods, and it states that it exists to produce security requirements and implementation guidelines [4].

Design changes can invalidate rows almost invisibly. Adding a user-readable performance counter moves a boundary. Giving a new DMA channel a convenient address range also moves a boundary; a DMA channel is an independent transfer engine inside the DMA, with a state of its own and a share of the AXI backend. During bring-up someone adds the "temporary" multiplexer Chapter 15 warns about, and the negative connectivity row that discharged TM-02 is now false while still showing green in yesterday's report. Detecting these changes requires the discipline Chapter 7 applies to waivers: every row should carry an owner and an expiry, and the review that re-reads the threat model should be scheduled.

A finding at a module boundary is conditional on integration. A published module-scoped study of an accelerator's configuration port establishes that the module, taken on its own, applies no local privilege check and therefore depends on trusted integration or upstream filtering; the study explicitly declines to claim that any particular deployment is exploitable; and whether the weakness becomes a system-level one depends on integration and on whether an untrusted requester can reach that

path at all [3]. “This block does not enforce X” and “this SoC is vulnerable to Y” require different evidence; conflating them can prevent a report from being acted on. The study takes the `NV_NVDLA_csb_master` IP of NVIDIA’s NVDLA at its public `nvd1av1` snapshot, and enumerates the port’s assets against named weakness identifiers as the first step of a security plan [3]. The Accellera Security Annotation for Electronic Design Integration Standard, Rev 1.0, standardises the producer’s side of that hand-off: it defines data objects through which an IP developer declares a block’s attack points and associates known security weaknesses with the block, so that the integrator who inherits the risk can select different IP, add mitigations, or record the risk as out of scope [5].

23.3 Security properties and how they differ

Two property types account for most obligations, both concerning *flow* rather than values. Integrity is violated when an unauthorized agent (software or physical) can modify the state of a target register or memory: untrusted sources must not flow to trusted state. Confidentiality is violated when an unauthorised agent can read the state of a target register or memory: secret sources must not flow to observable destinations [1]. Every threat-model row concerns one or the other, supporting the asset classification.

Chapter 11’s canonical 4 KB-boundary property (Arm IHI 0022H.c §A3.4.1) [6] is a claim about *values at a point*: given an accepted INCR write address, this arithmetic relation holds. The 4 KB boundary is the address line that AXI forbids a burst to cross. That property can be decided by looking at one execution at that point. A confidentiality property instead quantifies over *pairs* of executions: when the secret changes, nothing the adversary can see may change. By Chapter 11’s definition, a concurrent assertion evaluates sampled values from one execution at every clock tick. This mismatch between pairs of executions and sampled values cannot be resolved by writing a more elaborate assertion sequence, which is why information-flow analysis exists as a separate technique (§23.4).

Formal evidence for invariants

Formal methods suit properties expressible as invariants because they can prove that a violation never occurs along any path. No amount of simulation establishes that negative claim. Formal methods provide exhaustive checking within their scope: a successful proof excludes all violations of the property for the design as modeled, subject to the declared assumptions; in security verification, formal methods work most effectively on local properties that can be stated precisely, including access-control invariants, correct privilege transitions, and protocol safety [2]. Three of the six rows use formal for at least part of their evidence; TM-03 relies on it alone. A countermeasure primitive is a hardware block that implements one countermeasure. Binding an assertion attaches it to the design instance that it checks. In a published framework, lowRISC’s OpenTitan project embeds in each countermeasure primitive an assertion that the primitive’s error indicator is followed within a few cycles by a fatal alert, proves that assertion with formal property verification, and adds a second

check that fires when an instance of the primitive was built with the assertion left unbound ^[7]. A fatal alert is an alert that the chip cannot recover from without a reset. Formal property verification is the method that decides by mathematical reasoning whether a design can violate a stated property, rather than sampling its behavior with stimulus.

Scenario (the debug unlock path). TM-03 and TM-04 both concern the working key and both cross a boundary the flagship’s own instrumentation creates: post-silicon observability provides designed-in access to the state beside it (keys, firmware, debug modes) much of it still present on-field, and published system compromises have exploited those features ^[8]. Observability is the amount of internal design state that is visible while a design is debugged. The unlock unit that guards the debug port is a small always-on finite state machine. **Approach.** Three assertions and three covers are written for that unit, and all of them read always-on signals only.

```
``systemverilog // Root-of-trust debug port: obligations TM-03 and TM-04. // All
signals below are in the always-on domain. a_unlock_needs_token : assert property
(@(posedge clk) disable iff (!rst_n) $rose(dbg_unlock) |-> token_verified);

a_key_region_locked : assert property (@(posedge clk) disable iff (!rst_n) (dbg_req
&& dbg_addr_in_key_region) |-> !dbg_gnt);

a_relock_wipes : assert property (@(posedge clk) disable iff (!rst_n)
$fell(dbg_unlock) |=> wipe_req);

c_unlock_granted : cover property (@(posedge clk) disable iff (!rst_n)
$rose(dbg_unlock));

c_key_region_probed : cover property (@(posedge clk) disable iff (!rst_n) dbg_req
&& dbg_addr_in_key_region);

c_relock_seen : cover property (@(posedge clk) disable iff (!rst_n)
$fell(dbg_unlock)); ``
```

Why. None of the six looks back more than one cycle or forward more than one, and all read a handful of always-on signals; in this example, a proof engine settles them in seconds for every arrival time of an unlock request, covering timing variation that no directed test could supply.

`a_unlock_needs_token` constrains one cycle: the one on which `dbg_unlock` rises, and only to the extent that `token_verified` was asserted then. It says nothing about *which* token, and nothing about whether the verification that produced that signal is itself sound. Chapter 11’s reading of `a_no_ct_unauth` applies here to another asset. An invariant names a state, and provenance is a different invariant over different state, one that names where the state came from. The obligation to establish that the verifier is sound belongs on the verifier, to be discharged there (Chapter 14).

`a_key_region_locked` deliberately does not mention `dbg_unlock`. Guarding a security property with the very state the adversary is working to reach would make it vacuous exactly where it is needed: it would pass by never being evaluated at all. It con-

strains the *grant* rather than the request. The adversary controls the request; only the grant belongs to the design, so constraining an adversary-controlled signal cannot establish a design guarantee.

`a_relock_wipes` asserts that a wipe is *requested* one cycle after the unlocked state is left. It does not assert that the wipe completes: it asserts the request, which is all TM-04 asks of it. The other half, outright key destruction on power-down, is established by the second technique, Chapter 19’s power-aware test, so the row needs two techniques.

The three covers are not optional. An assertion about denial proves nothing on a run that never made the request, an assertion about relocking proves nothing on a run that never relocked, and `c_key_region_probed` demands a request that neither a functional environment nor a cooperative requester would produce. Chapter 9’s constraints encode what the interface makes *legal* and what the testbench steers toward, and neither covers traffic whose only purpose is to be refused. An adversary is under no such restraint. An unchanged functional environment can omit the entire threat model: in a month-long example, the security campaign stays green while checking nothing.

Formal capacity limits

Chapter 14 shows the limit: the crypto engine’s two key-handling properties do not converge, because the key ladder plus the GCM datapath is more state than the proof engine can handle. Formal at SoC scale runs into state explosion, heavy abstraction effort, and the difficulty of modelling realistic software and environment assumptions; it is indispensable for targeted reasoning and rarely sufficient on its own for a full system [2].

Required evidence and available capacity determine how the remaining rows are assigned. TM-03 and TM-05 are small control logic and stay in formal, and so does the first half of TM-04. TM-01 and TM-02 are statements about a datapath’s dependence on a secret, so they go to information flow. Anything that only manifests after a firmware stack has booted goes to a platform that can boot one. TM-06 has no verification platform here, as §23.5 explains.

Chapter 14’s crypto-engine scenario also establishes a rule for assumptions. In a functional proof an assumption is a promise about a cooperative neighbour; in a security proof the environment is the adversary, so every assumption is a concession. Operationally, the assumption list must be a column of the threat model, and each entry must name either the capability tier that cannot produce the excluded behavior, or the property elsewhere that discharges it. An assumption meeting neither condition leaves a gap despite a passing proof.

23.4 Information flow and its verification

Information-flow tracking attaches labels to data (secret or public, trusted or untrusted) and follows how those labels propagate through the design, raising a violation

when a sensitive value reaches an untrusted sink or an untrusted input influences a control signal ^[2]. The label, usually called taint, tracks *dependence* rather than *value*. Anything whose value could differ because the secret differs is tainted, whether it holds a copy of the secret, a function of it, or merely a bit that was set because of it.

Writing the four parts of a flow rule accounts for most of the work (Table 23.2):

Table 23.2 — The four parts of an information-flow rule, with the crypto engine’s confidentiality rule written out against each. The fourth part is where the argument lives: without an exception the rule fails immediately and correctly, because a cipher’s output depends on its key, and with one the rule is exactly as strong as the exception is narrow.

Part	What it names	On the crypto engine
Source set	the asset, as the signals that hold it	every bit of the key ladder’s working-key output
Sink set	what the adversary can observe	bus-readable registers, the debug data register, the trace fabric’s input
Activation	when tracking starts	from the cycle a key is loaded, rather than from reset
Exception	the flow that is permitted	the cipher core’s ciphertext output

Tools express these differently, and the rule languages are proprietary; the shape is what transfers. The published vocabulary covers the same four ideas: a no-flow relation between a source and a destination, a **when** clause that starts tracking on a condition, a disjunct that excuses the flow while some condition holds, and an *ignoring* clause that declares one destination secure and stops the analysis there ^[1]. The disjunct is the exception among those four parts, and the **when** clause is the activation condition.

The scope of exceptions

A confidentiality rule for the engine reads, in words: *the working key must not flow to any output, ignoring the cipher’s ciphertext output*. Without the exception, the rule fails immediately and correctly because ciphertext depends on the key. With the exception, the rule becomes checkable, and the narrower the exception is, the stronger the resulting rule.

This adds a review hazard to Chapter 14’s over-constraint problem, where constraints exclude legal behavior and the proof then says nothing about what they excluded. Reviewers should challenge over-constraining assumptions even when they resemble modeling shortcuts. An exception can instead resemble domain knowledge, since ciphertext does depend on the key, and pass review as competence rather than a concession. Widening it by one signal declares a whole port secure. As with assumptions, every exception requires a row, with a written argument for why the flow it permits is safe, and with the same expiry and owner as any waiver (Chapter 7). “The ciphertext output is declassified because the cipher is trusted” is an argument. “We added **busy** to the ignore list because the rule kept failing” records an unresolved defect.

Structural connectivity and non-leakage

Chapter 15 proved the negative connectivity rows for this exact key ladder (no structural path to a bus-readable address, to the debug data register, or to a scan-observable point) and was explicit that this is a *structural* result: no path exists. The key bus from the ladder to the cipher core is a path that exists **by design**. Information flow asks whether anything reachable downstream of that path depends on the key other than through the cipher’s intended transformation. No cone-of-influence check can answer that question, because the cone must exist; in a flow analysis the cone of influence is the logic through which a source can reach a sink.

In the opening AES result, key and datapath must be structurally adjacent, yet a value carrying the key survives to the output pin. A passing connectivity report establishes non-adjacency, so reading it as non-leakage misidentifies what it established.

Static and dynamic tracking limits

Static analysis builds dependence graphs from the RTL or the netlist and reasons about possible flows without running anything, which makes it an early screen and an over-approximation, a model that adds behaviors not present in the real design: it reports flows the design will never actually perform. Dynamic tracking propagates labels at runtime, so it sees the paths an execution took and catches trigger-dependent behavior, at the cost of instrumentation and runtime. Both struggle to model implicit flows and microarchitectural effects accurately: over-approximation produces false positives, while aggressive pruning loses real violations [2]. An implicit flow is one that reaches its destination through a control decision rather than along a data path.

In this example, four hundred alerts, nearly all benign, lead to the report being ignored within two weeks; loosening the policy then starts deleting real findings. One rule per threat-model row attaches each alert to an asset, capability and boundary, so triage checks whether the flow crosses TM-02’s boundary rather than making an unspecified judgment; triage is the step that turns failures into facts before any of them is debugged, deduplicating them by signature and classifying them before they are assigned. The rows also provide a permanent record for benign flows, so their dispositions can be reused at subsequent releases; a disposition is the decision recorded on an open item, to fix it, to waive it as errata or to defer it with a stated justification.

Scenario (the GPU). Four shader clusters share a unified memory, and contexts belonging to different applications time-slice it. The asset is one context’s data; the sink is anything the next context can read; the boundary is the context switch. **Approach.** The flow rule needs an activation condition: it should track everything written by context A, starting at the switch into A, and require that none reaches anything readable after the switch out of A. It should be paired with a structural inventory of every piece of state surviving a switch: register-file banks, line buffers, scratchpad, cache ways and descriptor queues. **Why.** The failure is *residue*

rather than a wire: the next context overwrites state before reading it, hiding it from a functional test, while taint tracking detects the dependence immediately. The activation condition makes this rule expressible. Without it, the rule forbids context A's data from ever reaching memory, contradicting the intended memory writes.

Source covers check whether a run activated the source required by the rule, for example by putting a secret in the source register, switching context with live data resident, or loading a key; sink covers do not establish source activation. A flow rule evaluated over a run in which no source was ever tainted gives Chapter 6's vacuous pass: the report is green even though nothing was tracked, and this difference remains invisible unless someone covered the source.

23.5 Side channels

An RTL model represents only logic values at clock ticks, so physical side-channel properties can fall below its abstraction. The RTL model does not model supply current at all, even approximately; an unknown value is treated optimistically in RTL, and supply current is not treated at all. Optimistic treatment means that the simulator settles such a value to a definite one where silicon would hold an unknown. Charge drawn by the part is inexpressible in RTL, regardless of stimulus, coverage or proof depth.

Side channels must first be classified by whether the leaking quantity exists in the model, since that is what determines whether verification is possible.

Timing is represented in RTL. A cycle count is a signal-level fact: if the number of cycles between an operation's start and its completion depends on key material, that dependence is present in the RTL and can be reasoned about there. Two shapes handle it. Where the specification says the operation has a fixed latency, the claim is single-execution and mechanical:

```
// LAT is the engine's declared operation latency, in cycles.
a_fixed_latency : assert property (@(posedge clk) disable iff (!rst_n)
                        $rose(op_start) |-> ##LAT op_done);

c_op_start      : cover property (@(posedge clk) disable iff (!rst_n)
                        $rose(op_start));
```

Where the latency legitimately varies (with message length or back-pressure from the memory path) `a_fixed_latency` is false and the claim becomes a flow rule instead: no key bit may flow to the completion signal. Thus `op_done` is a sink for §23.4: a timing side channel means execution latency depends on secret data or privileged state ^[2], and its analysis is a listed information-flow application ^[1]. A timing channel is therefore a confidentiality property with a control-signal sink rather than a data bus.

Power and electromagnetic emission are absent from RTL. Pre-silicon work can reason over *proxies*: switching activity, performance counters and timing distributions exported from a run and post-processed as stand-ins for power and timing behavior, and reasoning over those proxies is enough to rank structures by risk and to compare a countermeasure against its absence [2]. Switching activity correlates with dynamic power but is not a power measurement: it carries none of the analog detail that a measurement carries, and a countermeasure that improves it has demonstrated improvement only in that proxy. This abstraction provides no leakage verdict; implying one falsely closes an unresolved question.

This is the same structural problem Chapter 21 has with analog behavior (a property in the continuous world while the model is discrete) and the same resolution applies: the quantity must be modeled explicitly at some cost and fidelity, or the question must move to a platform where the quantity is physically present. Here the second platform is fabricated silicon with measurement equipment attached, placing the closing evidence in Chapter 20's scope and on Chapter 20's calendar.

The capability tier is a design decision

A design choice can invalidate the threat model without changing any security block. Physical side channels and fault injection are C3 capabilities, and nominally they require access to the package; fault injection is the deliberate forcing of a fault into internal design state to see what the design then does. But software-controllable mechanisms for scaling voltage and frequency and for monitoring power consumption are ordinary features of modern chipsets, and they let an attacker mount fault-injection and side-channel attacks without physical access to the device [9]. The flagship has DVFS with three operating points on the host cluster and the compute island, and something has to select among them; DVFS is the run-time trading of supply voltage against clock frequency that saves energy. If that selector is reachable from C1 or C2, a whole row's capability tier has changed, and every property justified by "the adversary would need physical access" is now unsupported.

A verification engineer can examine which side of the trust boundary contains the selector's enable without analyzing the cryptography or power analysis.

In fault injection the adversary perturbs rather than observes, deliberately inducing conditions a functional model would classify as impossible: glitched supplies and clocks, and faults engineered to skip security-critical instructions [9]. The machinery for injecting faults and measuring what survives is Chapter 22's, built there for safety; what security adds is that the fault distribution is chosen by an opponent rather than by physics, so a campaign weighted by natural failure rates is answering a different question. The tooling transfers, but the weighting must change.

Evidence for TM-06

TM-06 requires that no key material be recoverable from the engine's power signature against a C3 adversary. The timing claim, no key bit reaching `op_done`, has a named sink and should receive its own TM-07 row when found in review, preserving the distinct boundary governed by TM-06. The power claim has no signal to name at

all, so it requires an argument with three pieces of explicit evidence: a named countermeasure recorded in the design review; a proxy comparison showing the countermeasure moves the proxy in the expected direction, with its own limitations written down; and a post-silicon leakage assessment on the plan with a date and an owner. This applies Chapter 7's conditional sign-off: present evidence, named future evidence and a signed residual risk. A security review must distinguish this conditional row from a discharged row such as TM-03.

23.6 Where security verification lives in the flow

Early asset identification and boundary definition reduce security costs before verification begins. In this example, an hour in architecture definition identifies assets and boundaries before the interconnect topology is fixed, while the address decode is still an open decision. Late asset identification delays protection until only expensive, awkward options remain. Chapter 15's integration table should include negative rows from the first build, so a temporary multiplexer fails the job the day it is added instead of surviving to tape-out. In published design guidance, lowRISC's OpenTitan project opens its secure hardware design guidelines by asking designers to identify the module's sensitive and privileged operations and to document them, so later design and verification reviews can take them as focus areas or as coverage points [10].

Table 23.3 assigns each question to the platform that can answer it, independently of the project's equipment.

Table 23.3 — Six security questions, each routed to the platform on which it can be answered, from a proof over a small control block to measurement on a fabricated part. The third column is what keeps this from being a menu: the answers are not interchangeable, an adjacency result and a flow result answer different questions, and the last row's question exists nowhere but in silicon.

Question	Where it is answered	What the answer is worth
Does this small control block enforce an invariant?	formal	exhaustive within the scope and the assumptions
Is there any structural path from this asset to that sink?	connectivity check (Chapter 15)	cheap, and about adjacency only
Can this asset influence that sink at all?	static information flow	an early screen, over-approximating: expect false positives
Does it influence it along the paths a real workload takes?	dynamic information flow, in simulation or on an emulator	grounded in execution, silent about paths not taken
Does the policy still hold once firmware boots and drivers run?	emulation	realistic and long, with limited visibility
Does the fabricated part leak through a physical channel?	silicon and measurement	the only place the question exists

Firmware-dependent properties

Some security behaviour has no meaning below the system level. Four kinds of behavior all need a running software stack, interrupts and peripheral concurrency, and these four also need executions long enough to reach them, which emulation supplies and simulation does not; those four are privilege transitions, the interaction of a DMA engine with a memory protection configuration, a debug unlock sequence driven by real boot firmware, and a driver handing a buffer across a boundary [2]. Emulation is the compiling of a design onto a hardware execution fabric, where it runs under stimulus that a host supplies; a model is still running, so the work stays pre-silicon.

Property-guided monitor generation connects this work to §23.3. The property is first proved formally where tractable, for example an access-control invariant in a small subsystem; the same property is then translated into a synthesizable checker and run on the emulator under full firmware workloads, checking the rule two ways through proof and a search for sequences that stress it [2]. `a_key_region_locked` is a candidate. In this example it is proved on the unlock unit in seconds, and it is suitable for carrying into a boot campaign because a proof cannot establish whether real firmware ever puts the system into the state the property protects.

Emulator capacity is normally reserved for tape-out-critical functional regressions, and security workloads do not usually hold equal standing in flows built for coverage closure, so campaigns get scoped down to a few high-risk blocks or pushed to a later stage [2]. Coverage closure is the work of bringing every item of the plan to covered-or-consciously-excluded status. A security campaign is a **campaign** in Chapter 12's exact sense (launched for a stated reason, owned by someone, producing a written result, then stopping) and it must compete for slots explicitly, with a named block list and a named window.

Security monitors and adversarial harnesses often have to be re-engineered to fit an emulator's timing and capacity, which leaves the simulation and emulation environments different and makes a failure seen on one hard to reproduce on the other [2].

Adversarial stimulus by IP type

Stimulus aimed at driving a design into an unsafe state poses a different generation problem from checking functional correctness; its value depends almost entirely on three things: the quality of generated sequences, security oracles that define violations, and whether an interesting failure can be reproduced and triaged at all [2]. Reproduction and triage therefore need an explicit budget before a finding can support action.

The benefit also varies substantially across IPs.

Scenario (the video codec). A decoder presents an attack surface through an input bitstream arriving from the network or a file, which is attacker-supplied by construction; its control flow is data-dependent by design because it processes a variable-length entropy-coded format. An entropy-coded format is a compressed

one in which symbols have codes of different lengths. **Approach.** Two changes are made to the block’s existing environment. First, the oracle changes. Chapter 3’s definitional oracle, the standard’s bit-exact reference decoder, cannot serve this purpose, because on a malformed stream the reference has no defined output to compare against. The oracle for hostile input is instead a set of invariants that must hold whatever arrives: every line-buffer and DDR write stays inside its allocated region, the decoder makes forward progress or raises an error within a bounded number of cycles, and no malformed stream reaches a state only a firmware reset can leave. Second, input generation mutates well-formed streams instead of constraining a transaction class. **Why.** Mutation-based generation is suitable here because the input is a structured byte stream that a mutation leaves *almost* legal, and almost-legal input is the region that matters for hostile-input testing. For a bus-interface campaign in which most mutations produce traffic the protocol forbids outright, those requests are rejected and yield no useful evidence about deeper states.

Verifiable properties and security arguments

Verifiability requires named signals. A verifiable security property can be expressed entirely as signals in an available model: a source set, a sink set, an activation condition and an exception if it is a flow rule; an antecedent and a consequent if it is an invariant. TM-03 passes on the second form, the invariant with an antecedent and a consequent: `dbg_req`, `dbg_addr_in_key_region`, `dbg_gnt`. The timing claim that §23.5 sets beside the power claim passes on the first form: `op_start` and `op_done` are signals, and “no key bit reaches the second” is an ordinary flow rule. TM-06 fails this test because its physical quantity has no model representation or signal list, so the model cannot settle it. Properties with represented signals but questionable exceptions require particular review attention.

A verification engineer without a security background can contribute through design questions. Attack and cryptographic expertise are not required for these questions. Three persistent checks concern who can write a register, what happens to state across a transition, and exactly what an exception permits. Every one of those is a question about design behaviour, and none of them requires knowing how a power analysis works. Specialists supply the threat model; the subsequent work uses the verification methods developed in this book.

IN PRACTICE

In 2024, 83 percent of IC/ASIC projects incorporated security features (hardware modules holding encryption keys, content-protection keys, passwords, biometric references) and those features add new requirements and complexity to the verification process ^[11]. Feature ownership must include staffing for verification against an adversary.

When a late threat-model finding cannot be implemented after RTL freeze, its disposition needs a waiver argument with a deadline. The threat model should be treated as a month-one deliverable with the same standing as the verification plan, so that its rows can still change the design.

Two further issues concern tooling and defect classification. Where specialist information-flow tooling uses proprietary rule languages, licenses are scarce, and knowledge is concentrated in one or two people, one of those people leaving can disrupt the capability. Rules should therefore be recorded against threat-model rows in a form a successor can read, independently of tool artifacts. And security bugs do not fit the severity ladder, the scale on which functional defects are ranked from cosmetic to blocking. Chapter 20's asymmetry is the reason: once a defect of this kind is known outside the company, the time available to mitigate collapses, because it can be exploited rather than merely encountered. A parallel classification and separate, access-controlled tracker address this asymmetry despite the inconvenience.

PITFALLS

1. **Reusing the functional environment unchanged.** *Symptom:* the security regression is green from the first night and stays green. *Cure:* an adversary is bound neither by the protocol nor by what the testbench chooses to generate; the environment must be able to emit the forbidden request, and Chapter 11's rule applies here in full: every implication ships with a cover on its antecedent, the positive obligations as much as the denials.
2. **Reading a connectivity result as non-leakage.** *Symptom:* "we proved the key cannot reach the bus", on the strength of a no-connection job. *Cure:* that result is about adjacency; the flow question concerns paths that exist by design, and those are the paths the key actually uses.
3. **An exception clause that grew.** *Symptom:* the ignore list has six entries and no comments. *Cure:* each exception needs one written argument, an owner and an expiry, reviewed like a waiver; a widened exception is indistinguishable from a passing property.
4. **A threat model written backwards from the tools.** *Symptom:* every row closes. *Cure:* reviewers must determine whether the model was scoped to evidence already available from the tools; such a model hides residual risk instead of retiring it, while the row that cannot close particularly needs a named plan.
5. **An undischarged security assumption.** *Symptom:* a proof converges after an `assume` that the boot sequence is followed. *Cure:* every assumption names the capability tier that cannot produce the excluded behaviour, or the property elsewhere that establishes it.
6. **Reporting a module-scope finding as a system claim.** *Symptom:* "this accelerator is vulnerable." *Cure:* reachability and integration require explicit statements; a block enforcing nothing locally may be perfectly safe behind a filter, or may not be, and that difference is evidence somebody has to produce.

SUMMARY

- A **threat model** scopes security verification through assets, adversary capabilities and trust boundaries; a specification alone does not. It turns an otherwise unanswerable security question into checkable claims and explicit gaps.
 - The properties are negative and the failures are often functionally silent: the design computes the right answer and leaks anyway ^[2]. No quantity of functional pass evidence accumulates into a security result.
 - There is no settled definition of a security-coverage percentage. The deliverable is named claims, named gaps, and a residual risk somebody signs.
 - Rarity provides no defense within the threat model's scope. A constrained-random generator samples a distribution; an adversary optimizes, deliberately arranging and repeating an improbable conjunction.
 - A no-connection result proves **non-adjacency**. Only an information-flow argument addresses the paths that exist by design, and the result depends on how narrowly its **exception clauses** permit flow, which is why each one needs a written argument.
 - Formal proves negative properties but reaches capacity limits where assets are densest. Small control logic belongs in proofs, datapath dependence in information flow, and firmware-dependent rows on a platform that can boot firmware.
 - Side-channel methods depend on whether the leaking quantity exists in the model. Timing provides a sink; power and emission are absent, so pre-silicon work ranks and compares countermeasures without a leakage verdict.
-

EXERCISES

Exercise 23.1 (calculation). Count the rows of the threat-model table in §23.2 written against each of the four adversary capability tiers, and count the rows whose evidence column names formal. Check that second count against the statement §23.3 makes about it independently of the table. Name the tier that carries no row at all and the reason §23.2 records for its absence.

Exercise 23.2 (judgement). A lead reports TM-02 discharged on the strength of the negative connectivity job Chapter 15 ran on this key ladder, which shows no structural path from the ladder to a bus-readable address, to the debug data register or to a scan-observable point. Decide what that report establishes about TM-02's obligation and name the path it cannot speak about. Name the published result the chapter opens with that shows a structurally adjacent key surviving to an output pin, and state which of the two techniques TM-02's evidence column names is still outstanding.

Exercise 23.3 (judgement). The flagship's operating-point selector turns out to be enabled through a register window that unprivileged software on the host cluster can write. Two rows of the threat-model table in §23.2 are written against physical access to the package. Decide which of the two the selector's reachability unseats and why the other survives, using the reading §23.3 gives of its own grant-side asser-

tion. Name the check §23.5 leaves to a verification engineer here, the one that needs no knowledge of power analysis.

Exercise 23.4 (artifact). Read the information-flow rule below, written for the inline engine against the four parts §23.4 names. Name the part left empty and, from the shared-memory graphics scenario §23.4 works, state what a rule missing that part forbids. Decide which of the two entries in the exception list §23.4 licenses as an argument and which of them records an unresolved defect, and name what §23.4 requires of every exception before a review can accept it.

```
Rule:          no-flow (working key -> engine outputs)
Source set:    every bit of the key ladder's working-key output
Sink set:      bus-readable registers; the debug data register;
               the trace fabric's input
Activation:
Exception:     the cipher core's ciphertext output;
               the engine's busy flag
```

Exercise 23.5 (judgement). A security reviewer proposes recording the timing claim, that no key bit reaches the engine's completion signal, inside TM-06 rather than opening a row of its own for it. Decide whether the two claims belong in one row, and name the property §23.6 requires of a verifiable security property that separates them. Name the platform on which each of the two claims can be answered.

FURTHER READING

From the corpus. ^[2] is the anchor for this chapter and a recent survey of the area as a *workflow*: the technique taxonomy and its bottlenecks, static versus dynamic information-flow tracking, the campaign-driver and observability dimensions, the coverage gap, the emulator-contention and environment-divergence problems, and formal-assisted monitor generation. It uses “validation” as an umbrella term where this book says verification. ^[3] organizes the same territory by workflow stage (asset identification, threat modeling, test-plan generation, simulation and formal artifacts, countermeasures) and its worked module-scoped analysis is a model of how to state a finding without overclaiming it. ^[1] briefly states confidentiality and integrity as flow relations, with the rule shapes including the exception clause, and it is the source of this chapter's opening result. ^[9] extends fault campaigns into adversarial territory and supplies the weakness taxonomy for glitching, instruction skips and software-reachable voltage and frequency control. ^[8] explains why design-for-debug is itself an attack surface and why security defects are dispositioned differently from functional ones. ^[11] gives the 2024 adoption figure.

Outside the corpus. MITRE's Common Weakness Enumeration, and in particular its hardware design view, is the standard vocabulary for naming a weakness class, and a threat-model row that cites a CWE identifier is easier for a security reviewer to check than one that does not. For the artifact this chapter says to write first, the IEEE P3164 working group's *Asset Identification for Electronic Design IP* supplies a principled vocabulary for assets; Accellera's SA-EDI standard, its published counter-

part, is held and is cited above. For the channel this chapter can only rank: P. Kocher, J. Jaffe and B. Jun, "Differential Power Analysis" (CRYPTO '99), and S. Mangard, E. Oswald and T. Popp, *Power Analysis Attacks* (Springer, 2007), with G. Goodwill et al., "A testing methodology for side-channel resistance validation" (NIST, 2011) for the measurement side. The OpenTitan project publishes an open-source root of trust together with its security specifications, of which this chapter holds and cites the threat model, the countermeasure verification framework and the design guidelines; reading a real one end to end shows what a reviewed threat model looks like.

REFERENCES

1. **D28** J. Oberg, "System-Level Security Verification Starts with the Hardware Root of Trust," Proc. DVCon U.S., 2019.
2. **P14** T. Rahman, S. Saha, A. Y. Alhurubi, S. K. Saha, F. Farahmandi, and M. Tehranipoor, "Emulation-based System-on-Chip Security Verification: Challenges and Opportunities," arXiv preprint arXiv:2604.15073v1, April 2026.
3. **P13** K. T. Hasan, M. A. Hasan, N. Alam, M. T. Islam, U. Das, and F. Farahmandi, "AI-Assisted Hardware Security Verification: A Survey and AI Accelerator Case Study," Proc. IEEE VLSI Test Symposium (VTS), Napa, CA, pp. 1-7, 2026.
4. **O15** lowRISC, "Lightweight Threat Model," lowRISC/opentitan, doc/security/threat_model/README.md, commit 4e5c99db524e99425cb78ca6f2714e5287fa800c, accessed 2026-09-02.
5. **S23** Accellera Systems Initiative, "Security Annotation for Electronic Design Integration Standard, Rev 1.0," Accellera Systems Initiative, July 2021.
6. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
7. **O13** lowRISC, "Security Countermeasure Verification Framework," lowRISC/opentitan, doc/contributing/dv/sec_cm_dv_framework/README.md, commit 4e5c99db524e99425cb78ca6f2714e5287fa800c, accessed 2026-09-02.
8. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
9. **D2** Sesha Sai Kumar C V, A. Mouallem, and J. Mazzawi, "Fault Injection Analysis for Automotive Safety and Security," Proc. DVCon Europe, 2022.
10. **O14** lowRISC, "Secure Hardware Design Guidelines," lowRISC/opentitan, doc/security/implementation_guidelines/hardware/README.md, commit 4e5c99db524e99425cb78ca6f2714e5287fa800c, accessed 2026-09-02.
11. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.

PART VII

The Human and the Machine

The people, the process, and the machine: how verification teams are built, what automation is actually delivering, and where the discipline is going.

24. Teams, Process and Ramp-up

Verification teams need ownership of design outcomes. This chapter examines roles, reviews, progress reporting, onboarding and the limits of process through an escape in week 41 of the reference SoC. A system-level test running a quantized network on the neural accelerator, with both DMA channels streaming, produces one wrong output tensor in roughly four hundred inferences. A DMA channel is an independent transfer engine inside the DMA, with a state of its own and a share of the AXI backend. Two days of debug find the mechanism: the accelerator’s weight prefetcher wraps its buffer pointer one entry early. It does so when two conditions hold at once: the DMA is contending for memory, and the weight stream ends in the middle of a beat, a single data transfer of an AXI burst. It reproduces only at 2-bit weight precision, only with a degenerate tensor geometry, and only when the other DMA channel is moving traffic. A tensor geometry is degenerate when the tensor is one element wide, the narrowest of the width cases Chapter 6 defines. The block signed off in week 33 at 100% statement and branch coverage.

The post-mortem involved two people. The accelerator’s verifier treated memory contention as a system-level concern: her environment drove its ports directly, as planned, and she closed all her quantization-corner rows. The SoC integration verifier treated quantization corners as a block-level concern: he owned connectivity and system traffic, with no visibility into weight-buffer geometry. The DMA verifier was not invited, because the DMA was not at fault. Everyone fulfilled their assigned responsibilities. Every artifact in the project had a named owner: the block environment, the SoC environment, the DMA environment, the coverage model, the vplan, which is the verification plan. The escape falsified an unowned outcome: demonstrable accelerator correctness while another component competes for memory.

Markers identify the claims that the project’s document corpus supports, following the citation convention the front matter sets out; other interpretations are attributed explicitly.

CHAPTER MAP

§24.1 compares artifact ownership (“owns the testbench”) with outcome ownership; §24.2 examines each review’s purpose, productive mechanics and failure modes. §24.3 applies Chapter 4’s maturity stages to evidence-based status reporting and stage inflation. §24.4 covers designers’ transferable skills, counterproductive reflexes and a weekly ramp-up; §24.5 examines the benefits of checklists, templates and coding standards, and when process substitutes for judgment.

24.1 Roles and outcome ownership

The **block verifier** answers for a block's features, from the extraction that reads them out of the specification through to sign-off. The **methodology owner** serves the other verifiers by owning the base environment, register generator flow, regression infrastructure and the coverage merge that combines the coverage of many runs into one result. The **integration verifier** answers for behavior that exists only once parts are connected. The **lead** reads trends and negotiates. Designers, architects and firmware engineers are outside the verification staff, but all produce verification evidence.

When ownership is defined by the *artifact*, an owner answers for the block environment, SoC environment or coverage model. Artifact ownership is administratively convenient because it maps onto repository directories, but it caused the opening escape.

Ownership by **outcome** instead makes the owner answer for whether the accelerator is demonstrably correct under every condition the system can present to it. The two formulations disagree about who is at fault when the environment is excellent and the answer is still unknown. Artifact ownership is discharged by the artifact being good; outcome ownership is discharged only by evidence about the design.

Writing RTL is only a means to producing a working design, so the entire design team, beyond verification alone, must contribute to the verification plan and answer for its completeness ^[1]. For verification roles, "my rows are closed" therefore establishes artifact completion without establishing the outcome claimed about the design.

Ownership boundaries. This section examines three seams in ownership.

The first is **block-to-system**. At integration, each block generally arrives from its own team, and the SoC verification team must debug and analyze the assembled system without knowledge of the IP internals; each block is a black box to that team, which has neither the time nor the background to understand the inner workings of several complex IPs at once ^[2]. This is a structural limitation rather than a staffing failure, so asking the integration verifier to learn every block cannot close the gap. The gap closes by naming an owner for each *cross-block outcome*, such as "the accelerator under memory contention" or "the flagship's DMA under a coherent writeback". A writeback is the return to memory of a line that a cache has modified, and on the flagship the corruption from a wrongly split DMA transfer stays invisible until that return. That owner sits on the block side, where the internals are understood, and is obliged to state what system conditions the block environment did *not* present.

The second, **design-to-verification**, crosses overlapping responsibilities. Design engineers spent, on average, 49% of their time on verification activities in 2024 ^[3]. Here, white-box assertions, smoke tests and debugging alongside the verifier illustrate activities within that half of a designer's working life. A white-box assertion is a declarative statement about the design's internal signals rather than about its inter-

face behavior, evaluated by a tool, and its falsehood reports an error. Where the boundary is drawn as though the designer hands over an artifact and departs, design-side checks end up written but never regressed, because they belong to no suite.

The third is **features without owners**. Plan rows may depend on external evidence: a protocol checked by purchased verification IP, a corner covered by emulation (the design run on a hardware execution fabric onto which it has been compiled, before there is silicon), or a mode “the software team exercises”. A plan row is a single line of the verification plan: the feature, the criterion that decides it, and the method that supplies the evidence. Chapter 5 has those assumptions about external evidence reviewed face to face with the designers, and Chapter 14 puts the ones a formal proof rests on into a written register of assumptions, with an owner and a discharge column. The discharge column exposes assumptions assigned to a sentence instead of a person; each needs a named owner or deletion.

Scenario. The video codec’s checking rests on a bit-exact C model. The design team first wrote it to explore entropy-coding trade-offs; it is faster and better tested than anything verification would have produced in the time. So verification adopts it as its checking reference and compares the RTL against it frame by frame. Artifact ownership avoids duplicated work: design owns the model and verification owns the comparison. **Approach.** A transfer function is a testbench model that reproduces the design’s data transformation in order to predict its output, and not the signal-processing function of the same name. A reference model is an independent executable of the specification, taken as golden, and a golden model is a reference trusted by construction, because it is the specification itself at its own level of abstraction. The model should remain a transfer function written by the design team, without being treated as a golden model, while verification owns the *outcome*: “the codec’s output is what the standard requires”. The verification owner should discharge it with three kinds of evidence the C model cannot produce: conformance streams from the standards body, decoded against both the C model and RTL; an independent reader for the syntax layer, written from the specification; and a written record of every place the C model was tuned to match the RTL rather than the specification. **Why.** A model written by the design team encodes the design team’s reading of the specification, so it is not the independent second interpretation a reference model has to be (Chapter 3). Comparing RTL against it reconverges on that reading, and the comparison can stay green on a misread clause forever. Adopting the sound artifact transferred an outcome without reassigning its owner.

One test separates the two kinds of ownership, and it is the author’s read rather than a published rule: ask an owner to state what would have to be true for their answer to be *wrong*. An artifact owner describes a bug in the artifact; an outcome owner describes a condition of the design they have not yet reached. A project unable to obtain the second answer for a feature lacks an outcome owner for it.

24.2 Review purposes and mechanics

Chapter 4 puts reviews on the calendar and names who sits in each; Chapter 5 details the plan review's internal procedure, with weakness analysis row by row, and Chapter 7 details the sign-off review's evidence package. This section addresses how reviews work as institutions and how they fail. Their purposes differ, so confusing them undermines the review.

The **plan review** exists to test *completeness*, a property no single perspective can check. That is why its roster deliberately reaches outside the project team, and why experienced design, verification and software engineers are all named as contributors: the reviews are designed to establish that the identified functional verification requirements are complete ^[4]. Greater expertise can make completeness harder to question when reviewers remain within what they already know.

The **code review** tests *fidelity between intent and implementation* in the testbench rather than the design. Its object is source code produced by one engineer and read by one or more peers, and its declared purpose is to find problems no automated tool can: discrepancies between intent and implementation, and qualitative properties like maintainability and the relevance of comments and assertions. The goal is explicitly not to ridicule the author ^[1].

A second reason for testbench code review concerns temporary debug structures. A testbench routinely acquires temporary structures that bypass large sections of a testcase in order to speed up the debugging of a critical part. Nothing fails when those structures are left in, and the environment then no longer runs the tests it claims to contain. What catches a bypassed testcase is redundancy, a second and independent route to the same fact: peer review of the testbench against the plan, and review of the simulation log to confirm the run followed the specification it claims to execute ^[1]. The mechanical counterpart is Chapter 8's known-bad variant, a copy of the design with a defect put into it on purpose so that the environment can be seen to catch it; the review is the human one.

The **coverage-model review** tests whether the *measurement* is valid, and it needs its own slot because a coverage model reflects the plan and there is no automated way to establish that either is complete. A functional coverage model should therefore be put through the same development and review process as the plan itself ^[4]. The block's designer must be in the room because whether a cell is reachable *by construction* is a question about the RTL. A cell of a coverage cross is one combination of values, one from each of the variables the cross joins. Without this review, a passing coverage result can certify stimulus without establishing behavior.

The **sign-off review** exists to *judge assembled evidence and accept what is left over*; Chapters 4 and 7 work its content. The sign-off review fails when it arrives with no room to act on what it finds: by then, a discovered hole is a risk to be accepted rather than a defect to be fixed, which is why the three earlier reviews have to function. [Figure 24.1](#) sets the four in that order, with what each one hands to the next.

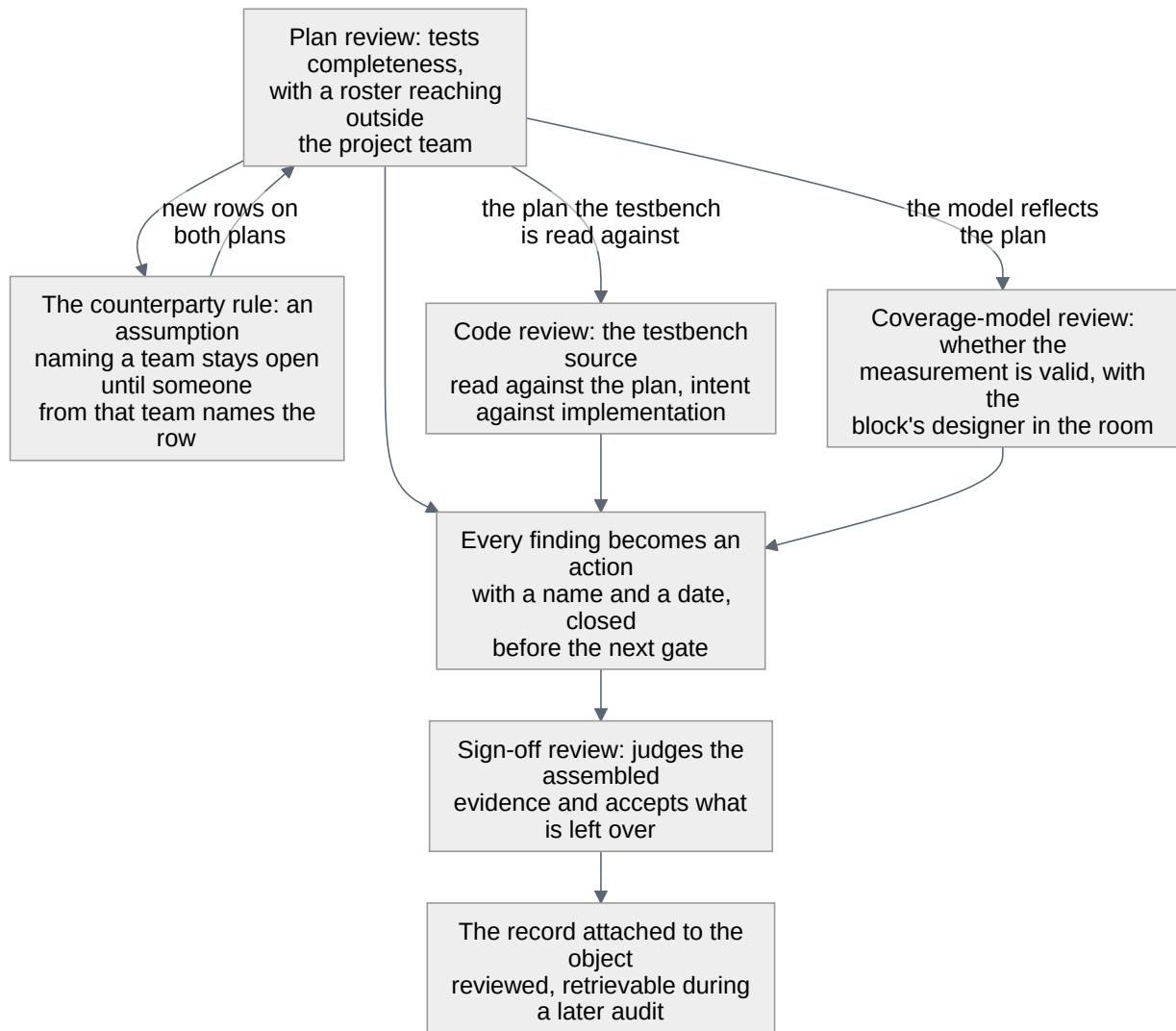


Figure 24.1 — Four reviews with four different questions, in the order that makes the last one possible: completeness, then fidelity of the testbench to the plan, then validity of the measurement, then a verdict on what the evidence leaves open.

Productive review mechanics. Four mechanics, of which only the last is drawn from the corpus; the first three are the author’s read, offered because they are cheap and their absence is diagnostic.

The artifact should circulate *before* the meeting, which should open by asking whether people read it; a review where the author narrates the document aloud functions as a presentation, with reviewers reduced to an audience. Someone other than the author should chair, so the author can answer questions instead of managing the clock. Every finding should become an action with a name and date, closed before the next gate rather than at it. The record should attach to the object reviewed; links directly to plan objects make history retrievable during audits and quality reviews [5] rather than isolated in meeting notes.

Beyond those four, one further mechanic concerns tone rather than procedure: one practitioner tutorial demands clear justification of how each specification point is verified, accepts that the justification may be complex and needs thoughtful review, and rejects tolerating or encouraging hand-waving [6].

Review failure modes. When every attendee has a stake in plan adequacy, plan review can yield a signature date instead of identified gaps. Code review fails when it enforces style that tools should check, without examining whether the environment implements the plan. The coverage-model review degenerates when the only attendees are the person who wrote the model and the lead who wants it closed. Under schedule pressure, any review held *after* the decision it should inform can only record findings as risks.

A second failure mode, difficult to see within the meeting, concerns abstraction. Where a review spans two levels of abstraction, such as a formal property set against a coverage model, the mismatch makes the review process much harder; a detailed examination of individual items is often required, and the time for it has to be allowed for [6]. Allocating one hour to a review needing a half-day examination leaves it without a judgment.

Scenario. The crossbar’s coverage-model review, week 9. The proposed model includes a cross intended to stress ID handling at a subordinate port: two managers presenting the *same* manager-side ID with writes in flight to one subordinate. The designer explains that the interconnect appends source-identifying bits on the way to the subordinate. It takes 3 bits to distinguish the 5 manager ports, and the interconnect appends those bits to the 4-bit manager ID. Adding those source-identifying bits to that manager-side width makes the subordinate-side ID 7 bits wide, so two managers that present one manager-side ID arrive with *different* IDs. The cell can never be hit, by construction. **Approach.** The row is *replaced* rather than deleted: the canonical write-interleaving hazard is the interleaving of the write data of two bursts at one subordinate port. The hazard’s real precondition is a second write address accepted while an earlier burst is still unfinished on the write-data channel, and that precondition is what the cross must sample. The minutes record the replacement and its reason to prevent the original being proposed again in the next generation. **Why.** An unreachable cover point remains at zero without reporting a failure, and it absorbs stimulus effort during closure, when time is shortest. It stops absorbing that effort only when an engineer excludes it, and by then the engineer no longer remembers whether the point was impossible or merely unreachable. The intended hazard, meanwhile, remains unmodeled. The model compiles and the bin is legal; identifying its unreachability here requires knowledge of interconnect ID handling.

This second scenario develops Chapter 4’s single-line warning.

Scenario. The coherent hub’s plan review. The hub’s rows cover MESI state transitions, directory occupancy, and livelock and starvation cases. The MESI states are the four states a cache line can take. A coherent hub keeps the cached copies of a line consistent across the caches that share it, and its directory is the record of which caches hold each line. Directory occupancy is how many of the directory’s entries are in use. The assumptions section contains one line: “*IO-coherent port behaviour under snoop-in is covered by the accelerator teams.*” Two of the three accelerator plans contain the mirror-image sentence: “*coherency is handled by the*

hub.” Neither side wrote a row. A snoop is a request the hub sends to a cache about a line, and a snoop-in is one that arrives at an accelerator’s IO-coherent port. **Approach.** The reviewer’s rule, the **counterparty rule**, is that an assumption naming a team is not closed until someone from that team has read it *in this review* and named the row that discharges it. Here that produces two new rows on the hub’s plan (snoop-in during an in-flight accelerator write; snoop-in to a line the accelerator has just released) and one on the plan of each accelerator team that wrote the mirror sentence. **Why.** The individually reasonable assumptions jointly leave the feature unchecked. Unspoken and undocumented assumptions are how design features fall through the cracks, which is why every assumption has to be reviewed with the designers face to face, usually over several rounds before alignment is reached ^[5]. The counterparty rule extends this discipline by requiring the named party to verify the assumption during review.

24.3 Maturity stages as a shared language

Chapter 4 defines the maturity stages (bring-up complete, verification mature, verified) and their written, demonstrable exit checklists. Here they structure three types of team conversation.

Progress reporting. Without stages, progress reports offer unauditable percentages and confidence; Chapter 4 explains why outsiders cannot read a coverage trajectory. A verification effort whose plan carries too little detail communicates poorly with other teams, leaves them confused about what has actually been verified, costs the verification team their trust, and pushes everyone toward proxies for progress, such as coverage percentages and test counts ^[7]. Stages replace the proxy with an artifact: a stage claim is auditable in an hour by someone who does not work on the block, because every line of a stage checklist is a demonstration, not an estimate.

Readiness for consuming teams. The consuming team’s question needs a *comparable* answer across blocks written by different people, and that is what stages supply. Stage 2 is verification mature, and it means the same thing for the crossbar and the DMA and the core, which is what makes a table of blocks against stages meaningful. The record then works as a shared contract rather than a per-team document: when a management platform holds one plan centrally and keeps every team’s view of it in sync automatically, that single plan becomes the verification contract across sites, and the document-merge exercise that otherwise substitutes for agreement disappears ^[8].

Requests for help. On the author’s read, stages justify their cost because they let a team ask for a specific intervention, and a percentage gives a team no way to ask for one. A block reporting “stage 2, four checklist lines open, two of them blocked on the formal proof of the ordering rules” is asking for a specific intervention; a block reporting “85%” is asking for nothing, because there is nothing in the number a lead can act on. Because stages describe states rather than verdicts, teams can say *not yet* without saying *we are behind*; inability to say “not yet” can instead produce months of “nearly”.

Scenario. The sensor hub arrives from a partner team, delivered as a pre-verified block for integration into the reference SoC’s always-on island. The delivery note says “verification mature”. The integration lead requests the ticked stage-2 checklist without disputing the claim. It comes back with the stimulus, checker and regression lines ticked and three lines blank: retention behaviour across the hub’s ultra-low-power modes, asynchronous wake events crossing into the system domain, and the functional coverage model for low-power mode transitions. **Approach.** The block is accepted at stage 2 *for the functions the checklist covers*, and the three open lines become rows on the SoC plan with the partner team named as owner and a date. Integration proceeds, with low-power sequencing tests gated on those rows. **Why.** “Verification mature” was unqualified rather than false; the checklist gave both teams evidence they could interpret alike without accusations or an appeal to trust.

Stage inflation differs from ordinary optimism: a stage arrives *without* its checklist. A stage claimed rather than demonstrated becomes a status word, which can be assigned by whoever wants it assigned, generally someone outside the team. Chapter 4 names the process countermeasure; the conversational one belongs to every engineer, not just leads. Every stage report must include its open lines: “stage 2, three lines open” specifies what “stage 2” alone omits.

lowRISC’s OpenTitan project publishes both halves of the arrangement this section describes: the checklist that defines a stage, and the filled-in copy that claims one. The project’s generic signoff checklist sets out what a block must show to move stage, in OpenTitan’s own numbering of the stages rather than this book’s: twenty-two items to reach its V1 stage, twenty-two more for V2 and thirteen for V3 ^[9]; the same checklist appears filled in per block, and one block’s copy records every item as done, not started, not applicable or waived, carrying a written reason beside each of its three waivers ^[10].

Chapter 4 cautions that stages describe *verification* maturity rather than design cleanliness. Stage 2 with fifty open bugs reports functioning bug-finding machinery, which is what stage 2 claims.

24.4 Onboarding: designer → verifier

Verification engineers often arrive from design, or oscillate between the two roles over a career, and the resulting verification mindset is very different from a design mindset: the verification mindset is usually reached only through years of experience, mentoring and lessons learned through mistakes ^[11]. That characterization belongs to Verilab, a verification consultancy that frames the mindset as a set of concrete choices rather than an attitude, the first being whether the environment’s duty is to replicate the real world or to stress the design beyond it ^[11]. Two implications govern ramp-up. The slow transition requires more than a two-day tool course, and a mindset that is learned by being taught requires more than a wiki.

Chapter 2 describes the verification mindset. This section covers existing skills, reflexes to unlearn and a deliberate twelve-week ramp-up.

Transferable design skills

Ramp-up should use the newcomer's existing design skills. Fast RTL reading helps designers localize failures, place white-box assertions at the right signals and assess proposed coverage exclusions. Designers have micro-architectural intuition about where designs are fragile, including which counters wrap, which FIFOs have a subtle almost-full condition and which handshakes have a reset hole; that intuition guides bug searches in ways the specification cannot. They know how the block was *built*, and often which parts the design team itself was uneasy about. Their standing with the design team helps when identifying a failure as a design bug.

One industry taxonomy identifies an organizational asset through its absence: under the reuse gap it lists an IP domain knowledge gap, and it lists the failure to apply at one level the knowledge developed about a block at another level ^[12]. That taxonomy is the work of engineers at Oracle Labs (the company's research organization), presented in 2015: it sorts the verification gap by tractability (inherent, transient and self-induced) and names its contributors as the complexity, skills, schedule, quality, metrics, reuse and integrity gaps; the self-induced part is the part a team can act on ^[12]. A designer moving into verification of the same block family is, on the author's read, the cheapest available closure of that gap, provided the ramp-up asks them to use the knowledge rather than to forget it.

Counterproductive design reflexes

These reflexes were appropriate in design and are not deficiencies; several make a testbench *look* healthier while reducing its checking.

- **Explaining a failure instead of reproducing it.** A designer first hypothesizes a failure mechanism because they own the block that produced it. A verifier's first obligation is a failure that can be replayed on demand from the seed and manifest (Chapter 12). Hypothesis-first produces a diagnosis with no artifact behind it, and often enough the diagnosis turns out to describe a different bug from the one that fired. On the author's read, the correction repeated most often during a designer's first months is that a failure must reproduce before it is considered understood.
- **Interoperating rather than stressing.** The design reflex is to build the most robust component possible; applied to a verification component, that reflex inverts the purpose, because the more tolerant the testbench, the less it checks. A verification component is the reusable testbench-side model of one interface, built both to stimulate that interface and to observe it. Clock snooping illustrates the problem: the testbench uses the design's internal clock instead of generating its own, and engineers coming from a design mindset adopt it because it yields fewer failing tests and faster environment bring-up, at the cost of masking clocking bugs ^[11].

- **Treating “invalid” as a stimulus filter.** “A transfer with that parameter is invalid, so there is no need to test it” is usually true about intent and irrelevant to verification: what matters is whether the scenario can occur, and if it can, the recovery from it is a feature ^[11]. Chapter 2 develops this rule; newcomers should state it at their first plan review.
- **Building a testbench that recovers.** A design must survive an error and carry on. Once a testbench catches an error, effort spent making it recover and continue checking is misplaced ^[11]. Newcomers build elaborate error recovery into a monitor because that is what a good component does. A monitor is the observing part of a verification component.
- **Debugging in waveforms.** Designers use waveforms because almost all design state is visible there. Zero time means that no simulated time passes while a component does its work. Verification components operate in zero time and dynamic data structures, so debugging requires logs rather than waveforms; a testbench that cannot be debugged from its log has a messaging problem rather than a tooling problem ^[11].
- **Staying zoomed in.** Design work is naturally scoped to a module. Verification requires switching deliberately between detailed tasks, such as reading the spec, writing the plan, coding and debugging, and broader decisions: which features to focus on, how to expose bugs, and where effort buys the most checking and coverage. Verification also continues past tape-out, so what is done before, after, and not at all has to be prioritized consciously ^[11].

A structured ramp-up

The ramp-up structure is the author’s; its essential components come from the sources below.

Time for experimentation and early revisions. Engineers need time to experiment and gain experience because poor early decisions frozen by schedule pressure can be counterproductive ^[6]. The project should allow revision when a newcomer recognizes an incorrect first architectural choice.

Explicit mentoring and escalation. Formal property verification is the use of mathematical reasoning to decide whether a design can violate a stated property, rather than a sampling of behavior with stimulus. One published industrial deployment took it mainstream across a large graphics design organization, and that deployment built a two-tier support structure rather than relying on a central team of experts. It identified a **champion** per cluster, the first point of contact and local expert for that group’s questions, with escalation to a central expertise team when a query exceeded the champion, and those champions continued to own the activity across projects ^[13]. The organization is Intel: 85 engineers, design engineers rather than formal specialists, were trained in a span of two weeks and then supported by onsite consultancy for another four weeks; the drive found 82 interesting bugs in twelve weeks, and it did so despite beginning only after a major simulation phase had already been completed; the paper emphasizes that the tools had become deployable outside a central formal team ^[13]. The counts just given are the figures the

paper opens with; its later pages give slightly different totals in more than one place, which is the reason to hold the opening ones rather than an average of all three sets of figures the paper prints. Mentoring thus requires a named local person with an escalation path and continuity. The same tutorial that asks for experimentation time also asks for teamwork and discussion, because this work is not intuitive and a single engineer can easily become stalled, and for identifying internal expertise and making sure it is *accessible* [6].

Learning from worked material. The same deployment paired its training with self-help material aimed squarely at novices: frequently asked questions, cheat sheets per application, and worked step-by-step exercises, all reachable without asking a person [13]. A curated library does the same job for code: novices learn to write assertions and coverage points by reading existing verified checkers and adapting them to their own problem [4]. For ramp-up, newcomers should read known-good artifacts alongside the documentation about them. That material supplemented dedicated training and the champion network. Self-help material should address recurring questions while keeping the mentor available for discussion.

Existing onboarding artifacts. Two existing artifacts can support onboarding. Enabling new teammates rapidly is a stated benefit of having a verification plan [7], the document that says what the effort is *for*. And the house comment standard should already assume its reader is a junior engineer who knows the language but not the design, and who must maintain and modify it without the original designer's help [1]. Comments meeting that standard already support onboarding.

Worked example: the DMA ramp-up ladder. Four assignments, in order, each with an exit criterion stated as a demonstration rather than a duration. The sequence progresses through *reproducing*, *extending*, *owning* and *breaking*: newcomers must debug existing infrastructure before writing any.

Ramp-up: <name>, joining DMA verification from RTL design
Mentor: <name> | Escalation: methodology owner | Review: fortnightly

[1] Reproduce (weeks 1-2)

Task: replay 3 archived failing seeds from their run manifests;
write one bug report per failure at MECHANISM level.
Done when: all 3 reproduce on demand, and the mentor cannot tell
the new reports from the originals.
Purpose: kills the explain-before-reproduce reflex.

[2] Extend a checker (weeks 2-4)

Task: add the per-channel error-report cross-check to the
scoreboard - predict from stimulus which transfers must
fail; treat the DMA's own error report as an OUTPUT to be
checked, never as ground truth.
Done when: peer code review passed; the check fires on a
deliberately broken error-report path (Ch. 8's known-bad-
variant discipline) and is silent on 200 clean seeds.
Purpose: independence of judgment, at the smallest scale.

[3] Own an outcome (weeks 4-8)

Task: DMA-F06a (2D bursts split at the 4 KB boundary) end to end:
implement cg_2d_4kb, drive it to closure, and defend the
exclusion at the coverage-closure review.

Done when: the 6 (., straddle, max, .) cells are recorded as a
 WAIVER naming the generator's c_solver_budget as the reason
 - not written off as geometry - with owner and expiry.
 Purpose: the difference between impossible and merely decided.

[4] Break it on purpose (weeks 8-12)

Task: the MENTOR injects 3 undisclosed bugs into a private RTL
 copy; run the full block regression; report which were
 caught, by what, and how long each took to localize.

Done when: written result reviewed with the mentor; any bug the
 environment missed becomes a plan row.

Purpose: the environment is a design, and it is under test too.

Assignment 3 deliberately allocates four weeks to judgment rather than technique. The six excluded cells of the canonical 2D coverage cross are the cells in which a straddling 2D row also reaches the maximum length. The ladder writes those cells in the four-part form of Chapter 5's cross, where a dot stands for any value of its axis. A 2D row is a single line of a two-dimensional transfer, a contiguous run of bytes. Those cells are dead because the stimulus generator's constraint solver budget caps a 2D row at 2048 bytes. Dividing that cap by the width of the 64-bit path in bytes gives 256 beats, so a straddling 2D row splits into pieces and none of the pieces can itself be 256 beats. Chapter 5 states that same budget directly in beats where it defines that cross. That is a team decision about solve time, not a property of the design, and the entire lesson is that the newcomer must be able to tell those two apart and file the first kind as a waiver that expires. Assignment 4 applies Chapter 8's known-bad-variant discipline, ending with an environment measurement instead of an unaudited new component.

Two things this ladder deliberately does not do. The ladder does not begin with a course on the methodology library: investigating a failure gives the newcomer a concrete use for that material. The ladder does not begin with new code: its first artifact is a bug report that communicates the reproduced failure to the rest of the project.

Rotating engineers through both roles makes Chapter 2's liaison position a permanent capability rather than a hiring accident. This is the author's interpretation without a measured result, offered as the cheapest experiment on this list.

24.5 Process quality and limits

Process records conventions that prevent repeated mistakes; without it, teams re-derive them each project. This section considers three instruments.

Templates buy consistency across authors. A plan template is crucial because many authors contribute to a large project's single guide, and the template gives each author the same prescriptive path through the same creation process ^[5]. With a shared template, ten plans can be read by one reviewer without ten context switches. The logic scales down to bug reports and up to sign-off packages.

Coding standards require agreement on which details matter. One published guideline hierarchy separates rules from recommendations: a rule must be followed, because breaking it costs the productivity the methodology exists to provide; a re-

commendation is one whose *detail is often explicitly unimportant*, and the example given is a naming convention; such a recommendation can therefore be customized, although adherence within a team is strongly recommended so that the implementation stays consistent and portable [4]. For recommendations whose detail is unimportant, such as the naming convention above, consistent adherence provides the value. Comments are written for a reader who knows the language but not the design, and who must maintain it without its original designer's help [1]. Writing for that reader imposes a testable requirement.

Reusable libraries and checklists lower entry barriers and retain past knowledge. In the mainstream-adoption deployment of Section 24.4, the RTL was reviewed up front and cookie-cutter property templates were produced for recurring modules, including arbiters, state machines and standard interfaces, and packaged so that a novice met a filled-in starting point rather than a blank file [13]. Chapter 7 describes sign-off checklist lines as records of escapes identified in past post-mortems.

The same deployment reports that practices absent from mainstream deliverables will not be exercised despite their benefits, so it introduced check-points analogous to the schedule events the project already had [13]. This supports integrating process into schedules without establishing its quality.

Process costs. Adherence and discipline impose continuing costs [5]. Wholesale adoption of a full methodology may simply be unaffordable for a project with real people and schedules, an argument for adopting individual elements incrementally rather than adopting none [4]. A house-built flow adds to the cost: the team owns and must maintain the infrastructure, and switching methodology introduces a learning curve for experienced engineers with limited exposure to the new environment [14]. Not using best-known methods is itself estimated to cost on the order of 20% of productivity [12].

What a checklist cannot do

Checklist quality depends on distinguishing auditable claims from judgments. Section 24.3's stage checklists make claims auditable; they cannot make judgments, for four reasons.

It cannot decide whether its own line is true. "Functional coverage model implemented and reporting in nightly regression" is objectively true or false; anyone can go and look. "Coverage is adequate for this feature" records only someone's willingness to make that judgment. The rule that follows: every line of a gating checklist must name an artifact that a skeptic could regenerate, and a line that cannot be written that way belongs in a review discussion instead. As one practitioner tutorial states about establishing completeness, tools can supply some quality metrics but there is no substitute for engineers thinking it through [6].

It cannot see what is missing from the plan. Every line audits against a question somebody already thought to ask. The neural accelerator’s escape in this chapter’s opening would have passed a stage-3 checklist unblemished, because no line asked about the block under memory contention. Chapter 7’s escape analysis extends checklists with *new rows* for previously uncovered failures.

It cannot ensure impartial review. A tick records a judgment for later recall, but cannot counter a desire to pass the gate; the checklist merely formats that decision. The listed contributors to why teams under-verify include fear-driven, miscommunication-driven and ulterior-motive-driven verification, alongside a lack of introspection; that self-induced portion is the part a team can address by asking whether it is creating the gap itself ^[12].

Individual productivity and mechanical consistency. Individual productivity across engineers ranges up to tenfold ^[12]. Process can standardize how a bug is filed, what a plan contains and when a regression runs; those conventions do not determine which corner an engineer pursues, whether a failing seed is minimized or shelved, or whether an odd waveform is investigated at 5 p.m. on a Friday.

Artifact production can displace the assessment of progress, as in the following workshop example. One published workshop describes the weakness as *too much detail too soon*: a good-faith effort becomes labor-intensive because every item on the feature list must be planned, every change requires a detailed plan update and every finished task requires another update, with granularity so fine that tasks are measured in hours; the result communicates worse because it becomes hard to tell what is finished and what is not ^[7].

Scenario. The radar front-end’s fixed-point precision sign-off. The house checklist contains “*quantization error characterized against the reference model across the operating range*”, inherited from an earlier project and ticked on every project since. The engineer ticks it: the characterization ran, the report exists, the maximum observed error is recorded. The review omits whether that error is *acceptable* to the downstream CFAR stage, the constant-false-alarm-rate detector whose false-alarm rate is the customer’s requirement. **Approach.** The obligation should split in two. One line stays mechanical and auditable: the characterization ran across the declared range, and here is the report. The second obligation belongs in a named review item rather than a checklist line, with the DSP architect present, asking whether the measured error meets a tolerance written down *before* the run. **Why.** Combining the two obligations conceals an unmade judgment. The mechanical half was always true; the acceptance half lacked an owner, a threshold and any record of consideration.

IN PRACTICE

The following recommendations address staffing and maintenance constraints.

- **Staffing against outcomes.** Assigning integration verification to whoever is available can leave the outcome map fitted to existing staffing. Teams should list outcomes first, staff against them and track unowned outcomes as open issues.
- **Working and decision reviews.** For twelve people reviewing a plan, a shared reading can be unproductive. Two meetings can separate the work: a small review produces findings, then a wider meeting examines them and closes actions, avoiding one meeting that achieves neither.
- **Continuity after champion promotion.** Published champions retain ownership across projects ^[13]; promotion to a lead role risks removing local expertise if it remains solely with that person. Every champion's answers should enter the self-help material so the knowledge survives reorganization.
- **Mentoring time as a deliverable.** In this example, a twelve-week ramp-up uses perhaps two weeks of a senior engineer already on the critical path. That time should be an explicit deliverable; otherwise, in month four, the team may discover that newcomers have received only work that requires no explanation.
- **Industry talent constraints.** The 2024 industry study raises concern about the talent gap in design and verification engineering as one of its headline themes ^[3]. A team unable to train newcomers limits recruitment to engineers who already have the required skills.
- **Mechanical enforcement of coding standards.** A linter or continuous-integration job should enforce standards; without it, a standard can decay within two projects in this example, leaving new hires to learn conventions through rejected patches.

PITFALLS

1. **Ownership by artifact.** *Symptom:* after an escape, everyone can prove they did their job. *Cure:* each owner should answer for a design outcome, and every assumption should name a person.
2. **The review with no pre-read.** *Symptom:* the author narrates the document while the reviewers meet it for the first time. *Cure:* the artifact should circulate in advance; the meeting should check who read it and be rescheduled if participants have not.
3. **Onboarding by documentation.** *Symptom:* "read the wiki and shout if you're stuck." *Cure:* a named local expert with a named escalation path ^[13], plus an assignment ladder whose first deliverable is a reproduced failure.
4. **The stage as a status word.** *Symptom:* a stage reported without its open checklist lines, and assigned by whoever needs it. *Cure:* every stage report must include its open lines, and transitions require review.

5. **Judgment recorded without review.** *Symptom:* a checklist line containing the word *adequate*, *sufficient* or *acceptable*. *Cure:* the auditable half stays on the checklist; judgment becomes a named review item with a threshold agreed in advance.
6. **The single accessible expert.** *Symptom:* one person answers every hard question, and their queue is the project's real critical path. *Cure:* champions per team with an escalation path, and every answer written into the self-help material [6, 13].

SUMMARY

- Roles should assign responsibility for outcomes. A good testbench discharges artifact ownership; outcome ownership is discharged only by evidence that the feature is demonstrably correct, and only the second form of ownership sustains accountability after an escape.
- The three seams where ownership fails are block-to-system, design-to-verification, and features whose owner is an assumption. The first is structural: at integration each block is a black box to the team that must debug the system [2].
- Reviews test completeness (plan), intent-implementation fidelity (code) and measurement validity (coverage model), with distinct failure modes. Peer review of the testbench is the redundancy that catches the bypassed testcase on which nothing fails [1]. Review records linked to the plan objects under review are what keep a review's history retrievable [5].
- Maturity stages change status conversations from confidence to evidence, and make "not yet" sayable. Stage reports must include open checklist lines.
- Designers bring RTL fluency, micro-architectural instinct and credibility; the reflexes of interoperating instead of stressing, filtering "invalid" stimulus and building recovering testbenches all make environments look healthier while checking less [11]. The book adds one reflex: explaining a failure before reproducing it.
- Mentoring and scaffolding support a sequence of reproducing, extending, owning and breaking. The first deliverable should be a bug report before any new infrastructure.
- Process provides consistency, lower entry barriers and scheduled deliverables; practices absent from the deliverable list are not exercised [13]. It cannot supply judgment, identify plan omissions or ensure impartial review; individual productivity ranges up to tenfold [12].

EXERCISES

Exercise 24.1 (calculation). Take the interconnect's identifier arithmetic from the coverage-model review scenario in §24.2. Compute, from the number of manager ports the crossbar serves, the number of source-identifying bits the interconnect appends toward a subordinate, and from that the subordinate-side identifier width. State what the result does to the cross the model proposed, and give the reason the minutes record a replacement rather than a deletion.

Exercise 24.2 (calculation). Take assignment three of the ramp-up ladder in §24.4. Compute the number of beats a maximum-length 2D row occupies on the path the assignment names, from the constraint solver budget in bytes and the width of that path. Decide whether the excluded cells belong in a plain exclusion or in a waiver, and name the three things the assignment's exit criterion requires of the record.

Exercise 24.3 (judgement). Two owners of one feature are asked what would have to be true for their answer to be wrong. The first replies that a monitor in the block environment could be sampling the wrong signal. The second replies that the accelerator has never run while another requester competed for memory. Decide which of the two holds outcome ownership under the test §24.1 gives, name what §24.1 says a project unable to obtain the second kind of answer lacks, and name which of §24.1's three ownership seams the chapter's opening escape fell through.

Exercise 24.4 (judgement). A team proposes two lines for a gating checklist: that every waiver on the block carries an owner and an expiry date, and that the coverage model is adequate for the block's risk. Decide which of the two belongs on the checklist, applying the rule §24.5 gives for a gating line. Name where the other obligation belongs instead, and the two things §24.5 requires of it there.

Exercise 24.5 (artifact). Read the review record below, filed after a plan review of the neural accelerator. Name the four productive review mechanics of §24.2 the record breaks, and the line of the record that breaks each. State what §24.2's counterparty rule requires before the assumption line can be recorded as closed.

Review:	Neural accelerator verification plan, rev. C
Chair:	the plan's author
Pre-read:	plan circulated at the start of the meeting
Attendees:	accelerator verifier, accelerator designer, accelerator verification lead
Assumptions carried forward:	"Behavior under memory contention from the other requesters is covered by the SoC integration team."
Findings:	(1) no row for weight-buffer geometry at the lowest supported weight precision (2) quantization-corner rows do not cross with concurrent traffic from another requester
Actions:	both findings to be addressed before sign-off
Filed:	meeting notes, verification team wiki

FURTHER READING

From the corpus. ^[11] addresses the design-to-verification transition and supplies most of Section 24.4's reflexes; Chapter 2 develops its mindset. ^[13] describes mainstream adoption in a large organization through champions, escalation, self-help scaffolding and cookie-cutter libraries, and reports that practices absent from mainstream deliverables are not exercised. ^[6]'s planning-and-completion slides cover experimentation time, teamwork against stalling, accessible internal expertise and rejection of hand-waving in reviews. ^[1] carries Section 24.2's code-review material (Chapters 2 and 3) and the comment standard aimed at a junior maintainer (Appendix A); ^[4] supplies the requirements-review roster, the guideline hierarchy behind Section 24.5, the novice's route in through existing verified checkers, and the rule that a coverage model is reviewed like a plan. ^[5] and ^[7] are the planning workshops behind the template, assumption-review and over-detail material; ^[8] shows the plan working as a contract across distributed teams. ^[2] is the source for the integration team's black-box problem, ^[12] for the skills, quality, reuse and integrity gaps, and ^[3] for the 2024 workforce and talent-gap data. ^[9] is the stage checklist of Section 24.3 as lowRISC's OpenTitan project publishes it, an item list per stage; ^[15] defines what the stage letters mean and ^[10] is one block's copy filled in. The OpenHW Group's TRL-5 sign-off checklist for a RISC-V core ^[16] is the same kind of artifact from another project, its rows carrying a sign-off criterion and an exceptions column. The letters TRL stand for technology readiness level, and that checklist is the one for the fifth level.

Not in the corpus (pointers, not evidence). Weinberg, *The Psychology of Computer Programming* (1971), on why independent review works on people rather than only on code. Wile, Goss & Roesner, *Comprehensive Functional Verification* (Morgan Kaufmann, 2005), treats the verification engineer's role and its ramp at industrial scale. Fagan's 1976 *IBM Systems Journal* paper on design and code inspections is the origin of the formal software inspection, whose separation of roles and defect-recording discipline Section 24.2's mechanics informally reconstruct.

REFERENCES

1. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. **D20** A. Meyer and H. Foster, "Metrics in SoC Verification: Not just for coverage anymore," Proc. DVCon U.S., 2012.
3. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
4. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
5. **D9** B. Ehlers, C. Vargas, and P. Carzola, "Best Practices in Verification Planning," Proc. DVCon U.S., 2013.
6. **D13** J. Bromley and J. Sprott, "Formal Verification in the Real World," Proc. DVCon U.S., 2018.
7. **D7** P. Marriott, J. Vance, and J. McNeal, "Proven Strategies for Better Verification Planning," DVCon 2022 Workshop, 2022.

8. **D26** D. Zhang, "Smarter Verification Management with vManager Platform," Proc. DVCon China, 2021.
9. **O10** lowRISC, "Signoff Checklist," lowRISC/opentitan, doc/project_governance/checklist/README.md, commit 4e5c99db524e99425cb78ca6f2714e5287fa800c, accessed 2026-09-02.
10. **O12** lowRISC, "AES Checklist," lowRISC/opentitan, hw/ip/aes/doc/checklist.md, commit 4e5c99db524e99425cb78ca6f2714e5287fa800c, accessed 2026-09-02.
11. **D21** J. Montesano and M. Litterick, "Verification Mind Games: how to think like a verifier," DVCon, 2014.
12. **D16** R. Narayan and T. Symons, "I created the Verification Gap," Proc. DVCon U.S., 2015.
13. **D18** M. V. Achutha Kiran Kumar, E. Seligman, A. Gupta, Ss Bindumadhava, and A. Bharadwaj, "Making Formal Property Verification Mainstream: An Intel Graphics Experience," Proc. DVCon U.S., 2017.
14. **D22** S. Hristozkov, J. Pallister, and R. Porter, "No Country For Old Men – A Modern Take on Metrics Driven Verification," Proc. DVCon Europe, 2021.
15. **O11** lowRISC, "OpenTitan Hardware Development Stages," lowRISC/opentitan, doc/project_governance/development_stages.md, commit 4e5c99db524e99425cb78ca6f2714e5287fa800c, accessed 2026-09-02.
16. **O7** OpenHW Group, "OpenHW Group TRL-5 Checklist for CORE-V RTL IP (CV32E40P)," openhwgroup/programs, Project-Descriptions-and-Plans/CV32E40Pv2/Milestone-data/RTL_v1.8.3/OpenHWGroup_TRL5_for_COREV_RTL_Cheklist-CV32E40P.xlsx, commit f7968ac5a0e6438549057fe45311abf6244b9a3b, accessed 2026-09-02.

25. AI and ML in Verification

AI and ML in verification require evidence suited to their task. This chapter compares a failure classifier with generated checkers, in two illustrative proposals that reach the same verification lead.

The first proposal, received in the same week as the second, comes from the team that owns the video codec's nightly regression. They have trained a classifier on eighteen months of failure logs; it sorts each morning's failures into buckets and names a probable cause for each. On a held-back month of history it agreed with the human triage engineer 88% of the time. The proposal is to put it in front of the triage rota.

The second comes from a pilot on the crypto engine. A language model was given the engine's specification chapter and produced forty SystemVerilog assertions in about twenty minutes. Thirty-eight compile, and thirty-six pass a formal proof against the current RTL. The proposal is to adopt the flow for the remaining blocks.

Both proposals report real numbers, but only the classifier's comparison with human triage provides evidence about the proposed use.

The first is measured against a baseline the team owns: yesterday's triage decisions, made by a person, written down, and available to disagree with. It can be re-measured next quarter on next quarter's failures, and if agreement falls the team will see it fall. The second measures agreement between a generated property and the design it was generated to check. An assertion must not be judged by agreement with the RTL, for the reasons developed in a section of Chapter 11; a property whose antecedent this design never satisfies passes on it without ever having been evaluated. An antecedent is the condition that must hold before the property's consequent is checked. Thirty-six passing proofs establish compilation and consistency, but they do not establish intent, the only thing a specification-derived property exists to capture.

Separating what has been *demonstrated* from what is *claimed*, and dating everything, manages two risks for this chapter: that it becomes obsolete, and that an engineer who was promised more than the evidence supports reads cynically and stops. A vendor demonstration needs a stated benchmark and success criterion before its result can be assessed. A published result on a benchmark is evidence for that benchmark under its definition of success. An industrial deployment result should state its scope and caveats.

A 2025 review of machine learning for micro-electronic design verification observes that although these techniques have been proposed since the late 1990s, many never reached mainstream adoption; the review gives three reasons, and all three concern evidence rather than capability: withheld code and data make results hard to reproduce; evaluations use simple designs against random stimulus and little else; and research rarely asks whether a method generalizes past the one application tested ^[1].

CHAPTER MAP

§25.1 identifies machine-learning techniques with a track record in verification, what each one optimizes and what it costs to keep working once deployed. §25.2 examines capability claims about generated verification artifacts: what was measured, on what artifact, against what baseline, and what the paper meant by *correct*. §25.3 explains why a generated artifact is plausible by construction and why that makes a wrong one more expensive than none. §25.4 sets out what reviewing machine-written code requires beyond reviewing human-written code, and names the decision points where a person must remain responsible. §25.5 describes the records needed to defend sign-off resting partly on generated evidence two years later, and criteria for what to adopt, pilot or defer.

25.1 Classical machine learning in verification

Three applications have an established track record: steering stimulus toward uncovered behavior, sorting regression failures, and selecting tests. Their published results span two decades rather than two years. Each technique optimizes a narrower objective than the verification goal, and each one decays.

Coverage-directed generation objectives

Chapter 6 places learning-based coverage-directed generation in the automation spectrum: every technique learns and inverts the mapping from generator directives to achieved coverage, optimizing a derivative of the verification goal. This chapter asks when it transfers.

In 2003 a Bayesian network was trained to direct stimulus at the storage control element of a commercial mainframe. An attribute is a dimension of a behavior with a set of values the stimulus has to reach, and a coverage model is the multi-dimensional region those attributes and their values define. The storage control element's coverage model had five attributes with a cross product of 13,888 combinations, of which 1,968 were legal coverage tasks. Training used 160 test-cases, each taking more than thirty minutes to run, and covered 1,745 of them; 223 remained. The trained network covered all 223 within 250 further test-cases, where an expert-written directive file covered two-thirds of them after more than 400 [2].

The rare use of a human expert's tuned directive file rather than uniform random stimulus gives the comparison its value. The 2025 review quoted above notes that random is the most common baseline in this literature and cautions that beating random says nothing about whether a technique generalises [1]. The cost follows from the training figures above: more than eighty hours of simulation before the model produced its first useful directive, on top of building a coverage model precise enough to learn from.

Nine years later, a review of the whole literature found that none of the approaches surveyed could consistently cover all tasks in a coverage model; the same review also named an adoption barrier that remains relevant: the average verification en-

gineer lacks expertise in these techniques and cannot tune them, so a method needing customization will not be supported, and an industrial approach must produce results engineers can easily interpret [3].

Chapter 6 records the 2023 deep reinforcement-learning result on a compression encoder, where an agent reached 133 of 136 hard functional bins; uniform-random stimulus plateaued at 28. Closure required merging the coverage of two separate training runs, because neither alone hit every bin: a 500-episode run reached 133, a 750-episode run reached 127, and the second run's bins were not a subset of the first's [4]. Reaching hard bins did not provide closure, so a plan promising closure would have been wrong.

A 2025 survey describes a 2024 system given the coverage report and the design under test and asked to propose stimulus; it beats random testing on simple and medium designs and degrades above roughly sixty-four states, which the survey reads as an argument for combining language models with structural approaches rather than replacing them [5].

The GPU's four shader clusters and scheduler are a suitable target: its coverage model combines occupancy, divergence and memory-request pattern in bins whose manual enumeration is undesirable. Occupancy is how many thread groups are in flight, and divergence is how far the threads of one group take different paths through the kernel. The fit is good because the space is large, the legal directives are many, and a bin is cheap to check. A key ladder is the sequence of register writes that turns a root key into a working key, and no stage unlocks until the one before it has finished. The crypto engine's key ladder is a poor fit: in this example its few interesting states are reachable in an afternoon by a directed test, a stimulus written out by hand that does the same thing on every run. They are also defined by a security argument that no coverage bin expresses.

Failure triage and clustering

Chapter 12 owns the mechanism: signatures, buckets, the rota, and the three reductions that make a morning's failures tractable. Machine learning contributes two distinct capabilities.

A 2023 survey reports two parts of a 2018 study that used one feature set: 616 features from semi-structured simulation logs. The survey's subject is machine learning in functional verification, the practice that establishes before fabrication that a design implements its specification. Unsupervised clustering of failures scored 0.543 and 0.593 on adjusted mutual information across three algorithms, against an ideal of 1.0. Supervised classification of the root cause, on the same feature set, reached 90.7% accuracy and an F1 score of 0.913 with a random forest [6].

Supervised classification is strong and needs a labelled history: somebody must already have looked at hundreds of failures and written down what caused each. Unsupervised grouping needs no labels and is weak. Classification therefore works best where a team already has years of curated triage decisions, and worst on a new project or a block's first tape-out, where triage is most difficult. The survey separates

three tasks (cluster failures by cause, classify the cause, suggest the fix) and reports that most research addressed the first two, with no results on the third [6].

That survey also reports a typical machine-learning project spending up to 70% of the team's effort on preparing data [6]. On the video codec's nightly, preparing data requires actual work: golden-model mismatches dominate the failure population, and their useful label is a frame region and a coding tool, neither present in the log unless somebody puts it there. Chapter 12's observation that the triage engineer's confirmation is the training data makes labeling affordable, and only if that confirmation is recorded in a form a model can read.

Regression compression and test selection

Chapter 12 carries the published figures for ranked regressions and novelty-based selection with their baselines. This section adds only what each optimises, because that is what decides where it belongs.

A ranking model optimises time-to-first-failure over a known suite: it reorders existing test-and-seed pairs so the runs most likely to fail go first. A selection model optimises coverage per simulation-second, discarding runs whose contribution it predicts is redundant. Neither optimizes discovery of new behavior: the industrial ranking study states that ranked regression simulates no new scenarios and is not recommended for random-space exploration or exposing new bugs [7].

A ranked or compressed regression quickly and cheaply detects whether a commit breaks behavior the suite already checks. Replacing the full random regression with a compressed one converts the main source of new bugs into a regression check, sacrificing bug finding. Deployment should retain both a compressed suite per commit and a slower full sweep with fresh seeds, whose yield Chapter 7 tracks separately.

The stale model

These techniques require retraining as their design or failure data change.

A model trained on one design's coverage response, one quarter's failure logs, or one release's regression outcomes assumes those patterns persist. The 2025 review explains that an example once novel becomes unremarkable after more examples have been seen, so the model needs regular retraining to stay relevant; once deployed, the learner also shapes the examples it will later be retrained on, which can suppress exploration and make performance *decrease* over time. The same review observes that research usually frames verification as a one-time learning problem while industrial designs change continuously, and warns that if substantial effort goes into crafting fitness or reward functions, the technique has merely shifted effort from writing tests to tuning models [1].

Retraining is a scheduled task with a named owner and a cost: in this example, a triage classifier 90% accurate at tape-out and 60% accurate two derivatives later is worse than none, because the rota has stopped checking it. The classifier needs a measured agreement rate, re-measured on a fixed cadence against human decisions and published where the people relying on it can see the number.

A stimulus model that goes stale misdirects generation: coverage per run deteriorates, and the coverage report reveals the change. A generated checker that goes stale becomes a bad oracle, the mechanism that decides whether an observed response is correct, and no report announces that the oracle is wrong. That difference is why the rest of this chapter is mostly about generated checkers.

25.2 Evidence for large language models

Three capabilities are claimed for language models in verification: generating assertions and properties, scaffolding testbench code, and drafting verification plans. The corpus supports statements about all three, with differences in strength that matter more than the headline figures. Each claim below states its year, measured artifact and success criterion.

Assertions and properties from a specification

Chapter 11 records the headline result: a 2024 framework that reads a full natural-language specification and emits SystemVerilog assertions reported 89% of 56 generated properties correct both syntactically and functionally on a single serial-bus design, where *correct* means the property passed a formal proof against known-good RTL, after human engineers had removed the generations referencing signals the tool could not map [8].

An ablation is the experiment that removes one part of the framework to measure what that part contributes. In the ablation, the underlying general-purpose model alone received the same specification files and produced 71 candidates: 23 syntactically correct, 6 passing proof, or 8% under the measure that gave the full framework 89% [8]. The engineering around the model does most of the work: the specification is split, signal names are mapped, and the right passage is retrieved per property; that engineering does not transfer for free to a specification in a different house style.

Of the three property classes the framework generates, connectivity properties could be produced only for control signals, because data-signal connectivity depends on internal RTL structure the specification does not describe; function properties could be produced only where the specification detailed register behavior [8]. The generator reaches only a subset of the specification without identifying the plan rows it leaves untouched, and a plan row is a line of the verification plan, naming a feature, its attributes, a pass criterion, a method and the evidence that closes it.

A 2025 study targets a different artifact class, complicating comparison. Its subject is protocol formal specifications, documents with timing diagrams, and its flow has

two stages. The first generates candidate properties from a grammar and checks each against the timing diagram with a SAT solver, where SAT is the question whether an assignment exists that makes a Boolean formula true. The second uses a language model to filter the survivors against the textual description. The SAT stage retains between 5% and 22% of candidates in every case, quantifying how much mechanical filtering can precede human review. Across six open-source on-chip protocols, extracted properties covered 53 of 58 manually derived targets for every protocol but one, which is excluded from that count [9].

The paper also tests the specification-driven framework on one of its protocols; the two results cannot form a trend. They differ in task and artifact class. The specification-driven framework was built to consume a full design specification and was measured on one design under its own pass criterion; the two-stage flow targets a protocol formal specification with timing diagrams, and the comparison re-ran the specification-driven framework itself under two underlying models. On that protocol the two-stage flow covered all 13 manually derived properties with no syntax errors; under one of the two models the specification-driven framework produced 38 properties, of which 3 were judged correct and 2 of the 13 targets covered, with 16 syntax errors and 11 references to signals that do not exist [9]. The comparison establishes no ratio of superiority between frameworks. A framework built for one artifact class does not transfer to another, and the eleven non-existent signals depend on the model rather than the framework: under the other model, the framework produces none.

The study's taxonomy gives reviewers a vocabulary for outcomes: a generated property may be *correct* (unique or redundant), *incorrect*, *unsure*, or *hallucinated*, meaning invented rather than drawn from the set it was given to filter. One protocol's transaction section produced four hallucinated properties [9].

The authors state that the method depends on unambiguous timing diagrams; on complex protocols, the target set does not cover the entire formal specification; models remain poor at reading figures, often getting a signal's value in a given cycle wrong; experienced engineers are still better at inferring properties where the document has no text to work from [9].

Agentic formal verification flows

The corpus contains one industrial report of an end-to-end agentic formal flow, from 2025. Its pipeline has six steps: a lead agent drafts a verification plan in natural language; a dispatcher delegates assertion generation; critic agents evaluate the results; properties are proven in a commercial model checker; counterexamples are analysed; and formal coverage is assessed for sign-off. The flow does not guarantee a result every run and works most of the time, with overall efficacy around 40% [10].

The study's first-named indicator is a *success rate*, defined as the number of runs completing end-to-end formal verification out of the total runs [10]; this measures whether the pipeline finished without establishing whether its properties were the right ones.

A vacuous pass is the success of a property whose antecedent never holds. The authors list vacuous passes in formal properties among the failure modes they observed, alongside syntactically incorrect or semantically irrelevant assertions, assertions referencing signals absent from the specification, and truncation of large RTL against the model’s context limit ^[10].

The benchmark comprises fifteen designs in three difficulty tiers; the *advanced* tier contains a Booth multiplier, pipelined adder, small RISC-V core, PID controller and round-robin arbiter ^[10]. These are block-scale exercises, and a 40% figure should be read as 40% on designs of that size. The report is Infineon’s and the flow is called Saarthi; its benchmark designs are not Infineon products, distinguishing a company reporting a capability from a company reporting a deployment ^[10].

The result is also a function of which model was used, by an amount that should govern how it is quoted. The same flow was run with three general-purpose language models. On the easiest tier they completed 17.78%, 43.19% and 48.77% of runs end to end; on the hardest, 4.33%, 9.00% and 41.43%. The two models that sat within six points of each other on the easiest tier were the two that completed the largest shares of runs there, and on the hardest they scored 9.00% and 41.43% ^[10]. The result characterizes the technique paired with one model, and Section 25.5 makes that pairing part of the record.

When two agents loop on generating and correcting an assertion without converging, the flow stops them at a threshold of five iterations and asks a human for the correct assertion ^[10], providing a designed hand-off point.

Debug and root cause

Debug consumes verification time: Chapter 7 puts debug-time share among the five instruments of its project dashboard, and it reports that verification engineers spend more of their time on debug than on any other activity. Counterexample debugging is therefore a target for reducing that time. A 2025 system converts the failure trace into a causal graph, scans the graph for suspicious nodes, explores them agentially into a causal narrative, and proposes an RTL fix. On a benchmark of 38 real debugging challenges with human-verified ground-truth fixes, the full system scored 0.956 on best-hypothesis quality, and its proposed fixes resolved the failure 71.1% of the time on the first attempt and 86.8% within five ^[11]. The system is NVIDIA’s and is called FVDebug. Those two figures were earned on the benchmark; the paper’s other case studies involve proprietary designs with masked details, so the pair supports no claim about what the tool does on a production part ^[11].

Hypothesis quality is scored by a language model comparing the hypothesis against the ground truth; Pass@k is measured mechanically, by applying the proposed fix to the RTL and re-running the proof in a model checker ^[11].

The paper’s baseline table shows that the two metrics do not track each other. One unstructured baseline scored 0.783 on best-hypothesis quality and resolved 65.8% of failures within five attempts; another scored 0.474, the table’s worst hypothesis quality, yet resolved 81.6% ^[11]. A model-judged score measures how convincing an

explanation reads, without predicting whether the fix works; a study reporting only that score establishes plausibility.

On two counterexamples from a larger open-source application-class processor, best-hypothesis quality falls to 0.713 and no Pass@k is reported at all; one of those two counterexamples is a load-store unit failure spanning 1,918 lines of RTL across seven files, and the authors state that automated fix generation for such multi-line issues remains future work ^[11]. The result shows the causal-graph machinery navigating a graph of 170 nodes, but says nothing yet about fixes at that scale. That processor is CVA6, and both failures were first found by AutoSVA; this half of the paper is checkable because the RTL and counterexamples are public ^[11].

Evidence gaps

For testbench scaffolding and verification-plan drafting, the capabilities most often demonstrated and least often measured, this corpus contains no study scoring the artifact itself; plan-drafting flows are graded only by downstream outcomes ^[10, 12]. The 2023 survey of machine learning in functional verification found that code summarization in design verification had not been reported in any literature at that point, and that no research had been published on coding assistance for this domain ^[6].

A 2025 survey of language models in electronic design automation names structural obstacles: models struggle with long inputs in which one passage depends on another far away, the long-range dependencies that both testbenches and specifications contain; hardware description languages and tool-control scripts are scarce in pre-training data compared with mainstream software languages, limiting the fluency that makes models useful elsewhere. That survey concludes that scalability on modern designs and the role of human oversight in verification remain the significant hurdles ^[5].

As practitioner judgment rather than a cited finding, generated scaffolding pays where a mechanical check independent of the generator can confirm it. A register block whose fields are already machine-readable can have its checker generated and then diffed against the field description; the diff provides evidence independent of how the file was produced. A constraint block encoding a rule about which transfers are legal cannot be checked that way and requires human review.

The evidence table

Each row of [Table 25.1](#) pairs one reported capability with the conditions that produced it. The fourth column should be read first because it states the success criterion.

Table 25.1 — What language models have been reported to do in this field, each capability paired with the artifact it was measured on, the year, and the study’s own definition of success. The fourth column is the one to read first. Two

rows describe one system twice on purpose, because one of its numbers was awarded by a language model and the other by a model checker.

Capability	Reported result and year	Measured on	What counted as success	Stated limits
Assertions from a full specification document	89% of 56 properties correct, 2024	One serial-bus design, 23 signals	Passing a formal proof against known-good RTL	Generations referencing unmappable signals were removed by human engineers before scoring; connectivity properties limited to control signals [8]
The same task without the surrounding framework	6 of 71 candidates passed, 2024	The same design and specification	The same proof criterion	Reported by the framework's own authors as the baseline for their contribution [8]
Properties from a protocol specification with timing diagrams	53 of 58 target properties covered, 2025	Five of six open-source on-chip protocols; the sixth is outside that count	Covering a manually derived target property, after a SAT check and a model-based filter	The SAT stage retains 5–22% of candidates; on the complex protocols the target set is not the whole formal specification [9]
End-to-end agentic formal verification	17.78–48.77% of runs completed on the easiest tier, 4.33–41.43% on the hardest, 2025	15 block-scale designs, three difficulty tiers, three models	A run that completed the flow end to end, not a property that was correct	Authors summarise overall efficacy at around 40%; observed failure modes include vacuous passes, irrelevant assertions and context truncation on large RTL [10]
Root-cause hypotheses for a counterexample	0.956 best-hypothesis quality, 2025	38 debugging challenges with human-verified fixes, one model	Similarity to the ground-truth cause, scored by a language model	The score is model-judged, and does not order configurations the same way the mechanical measure does [11]
Fixes for a counterexample	71.1% first attempt, 86.8% within five, 2025	The same 38 challenges, the same model	The fix applied to the RTL and the proof re-run	Not reported on the two application-class processor failures; the authors call fix generation at that scale future work [11]
Testbench scaffolding and plan drafting	No measurement of artifact quality in this corpus	—	—	The corpus flows that draft a plan are graded only by downstream outcomes [10, 12]

Two rows of the evidence table describe the same paper's system twice, on purpose. A combined row, *root-cause analysis, 0.956 quality and 71.1% fix rate*, would be accurate in both halves but misleading as a whole, because it would hide that one number was awarded by a language model and the other by a model checker.

25.3 Plausible by construction

A generated artifact is plausible by construction because of its objective: a model trained to continue text produces the continuation most consistent with the text it receives, and is built to emit a property that reads like one written by a competent engineer. Its objective is plausibility, with correctness a frequent by-product.

The crypto engine example pairs one specification sentence with the generated property:

```
// From: "While the engine is holding a request that the memory path has
//       not accepted, the key slot selector must not change."
property p_keysel_stable_gen;
  @(posedge clk) disable iff (!rst_n)
    (req_valid && !req_ready) |-> $stable(key_sel);
endproperty
a_keysel_stable_gen : assert property (p_keysel_stable_gen);
```

To a reviewer, the signals are the right ones, the guard is present, the handshake condition matches the specification sentence, and the operator is the one the word *stable* suggests. Yet one missing delay makes it wrong in two directions. Figure 25.1 draws both.

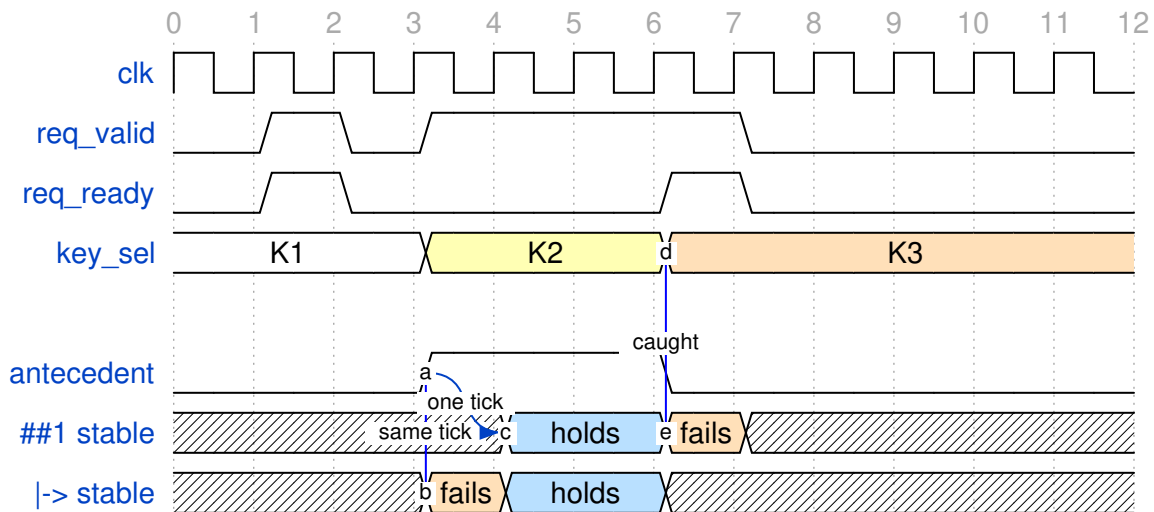


Figure 25.1 — One missing delay, wrong in two directions: `|-> $stable` fails at cycle 3 where the fresh request brings a new selector, and says nothing at cycle 6 where the selector changes as the request is accepted, which `|-> ##1 $stable` catches.

`$stable` compares the sampled value at the current tick against the one at the previous tick; the sampled value is the value the assertion sees, the one held just before the clock edge. The first held cycle is, here, the cycle in which the request rises, and the property checks its stability requirement in that same cycle. Since `$stable` looks one tick back, the property therefore demands that `key_sel` already had its current value one cycle *earlier*, before the request existed. A design that legally presents a fresh request with a new key slot in the cycle immediately after a previous transfer completes will fail a rule the specification never stated. On the last held cycle the

property says nothing about `key_sel` in the following cycle, the one in which the request is finally accepted and where a change would do damage. The obligation the sentence expresses lands one tick later than the property places it:

```
property p_keysel_stable;
  @(posedge clk) disable iff (!rst_n)
    (req_valid && !req_ready) |-> ##1 $stable(key_sel);
endproperty
a_keysel_stable : assert property (p_keysel_stable);
c_keysel_held    : cover property (@(posedge clk) disable iff (!rst_n)
                                   (req_valid && !req_ready)[*2]);
```

The cover is a strengthened form of the antecedent: it asks for a two-cycle hold rather than one, because a hold of a single cycle exercises only the accepting beat, the one cycle in which the transfer is taken. The longer hold is what shows the property constraining `key_sel` while the request is still waiting. Figure 25.2 draws a hold of each length.

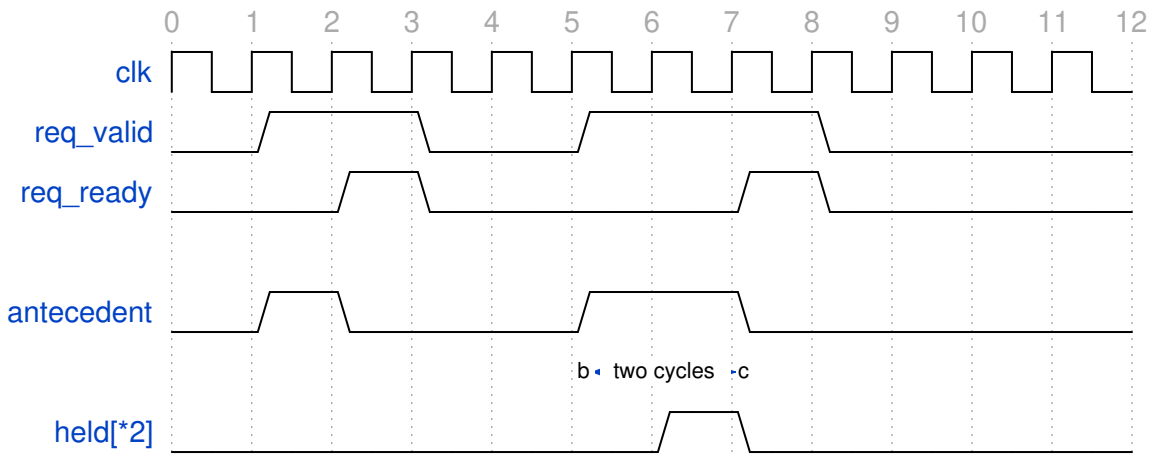


Figure 25.2 — The strengthened cover on two requests: the first is accepted after a single held cycle and matches nothing, the second is held for two cycles and gives `c_keysel_held` its witness at cycle 6, while the selector is still waiting.

A 2025 study of property generation from protocol documents opens with the same defect on a generic valid/ready interface, a stability requirement without its one-cycle delay, and notes that earlier machine-learning work made exactly that mistake [9].

The first version’s proof result depends entirely on whether this pipeline holds `key_sel` steady for a cycle before a request rises. If a register upstream loads the selector one cycle early, the wrong property is proven, on RTL that is behaving correctly, and under a pass criterion of *proved against known-good RTL* it is scored correct and enters the suite. The proof verdict concerns today’s RTL without establishing the property’s correctness. Figure 25.3 draws that pipeline.

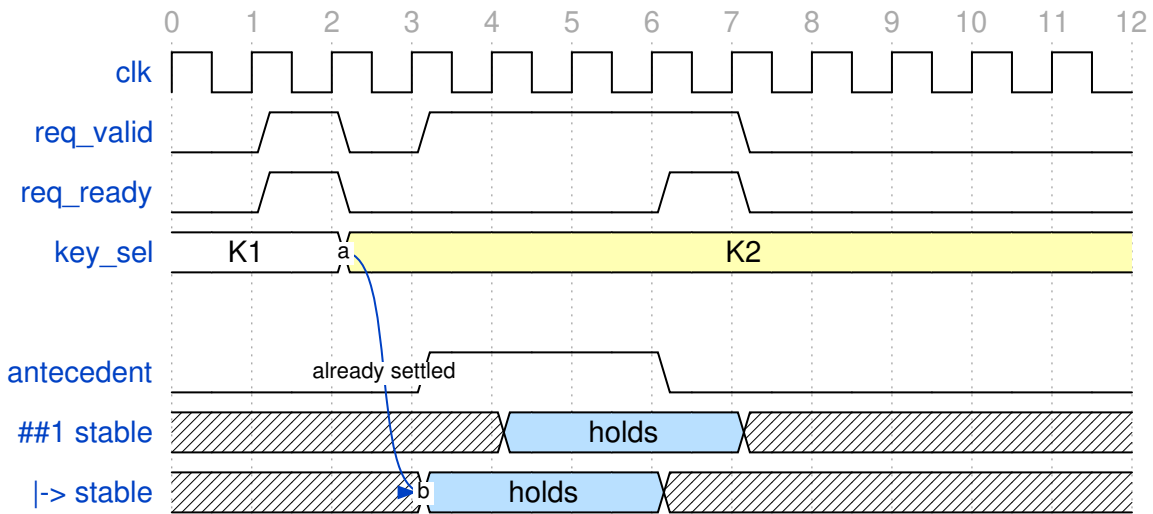


Figure 25.3 — The same generated property on a pipeline that loads the selector one cycle before the request rises: both forms hold, the proof passes against correct RTL, and the missing delay leaves no trace in the verdict.

Chapter 11 already names three ways an assertion can be worthless: it never triggers, it restates the design, or it cannot distinguish a correct design from an incorrect one. A generator produces all three fluently, and restating the design is structural rather than accidental. Where an assertion is mined from the design’s own traces, generation and evaluation both use the same RTL with no independent reference, so an RTL flaw becomes an assertion flaw that the method has no way to detect; a method reading the specification *and* the RTL together inherits the same weakness [8]. A property derived from the design cannot find a bug in that design; it can only restate whatever the design does, bug included.

A fourth failure mode belongs to generated artifacts specifically, and it changes how they must be reviewed. A human writing forty properties makes idiosyncratic mistakes, such as a typo or a misread sentence, uncorrelated across the batch; sampling five of the forty is therefore defensible because the sample estimates a rate. A generator’s errors are systematic: one misreading of one recurring pattern, *while X holds, Y must not change*, reproduces identically in every derived property, so a sample of five either catches or misses the entire class. Such sampling provides no estimate of an error rate. This review distinction is practitioner judgment, not a published result.

The cost of a plausible wrong artifact extends beyond finding it to its effects on the surrounding verification suite. Engineers trust a passing regression because they believe its checkers were written by someone who understood the design. In the illustrated review, one apparently careful property encoding a wrong rule can erode trust in the suite; each subsequent one can erode it further. In that review, a reviewer whose skepticism is exhausted can come to skim all the properties rather than reject the generated ones. The gates precede admission so that a wrong property does not acquire the apparent authority of an accepted check; an empty plan slot remains an explicit gap.

25.4 The human gates

Human involvement requires explicit decisions and responsibilities. The flow must specify where people decide, what they decide and the consequences of leaving a decision unowned. [Figure 25.4](#) separates four human decision points from checks a machine can settle alone. Each has a question, an owner, the evidence that owner needs, and the specific damage of leaving it unowned.

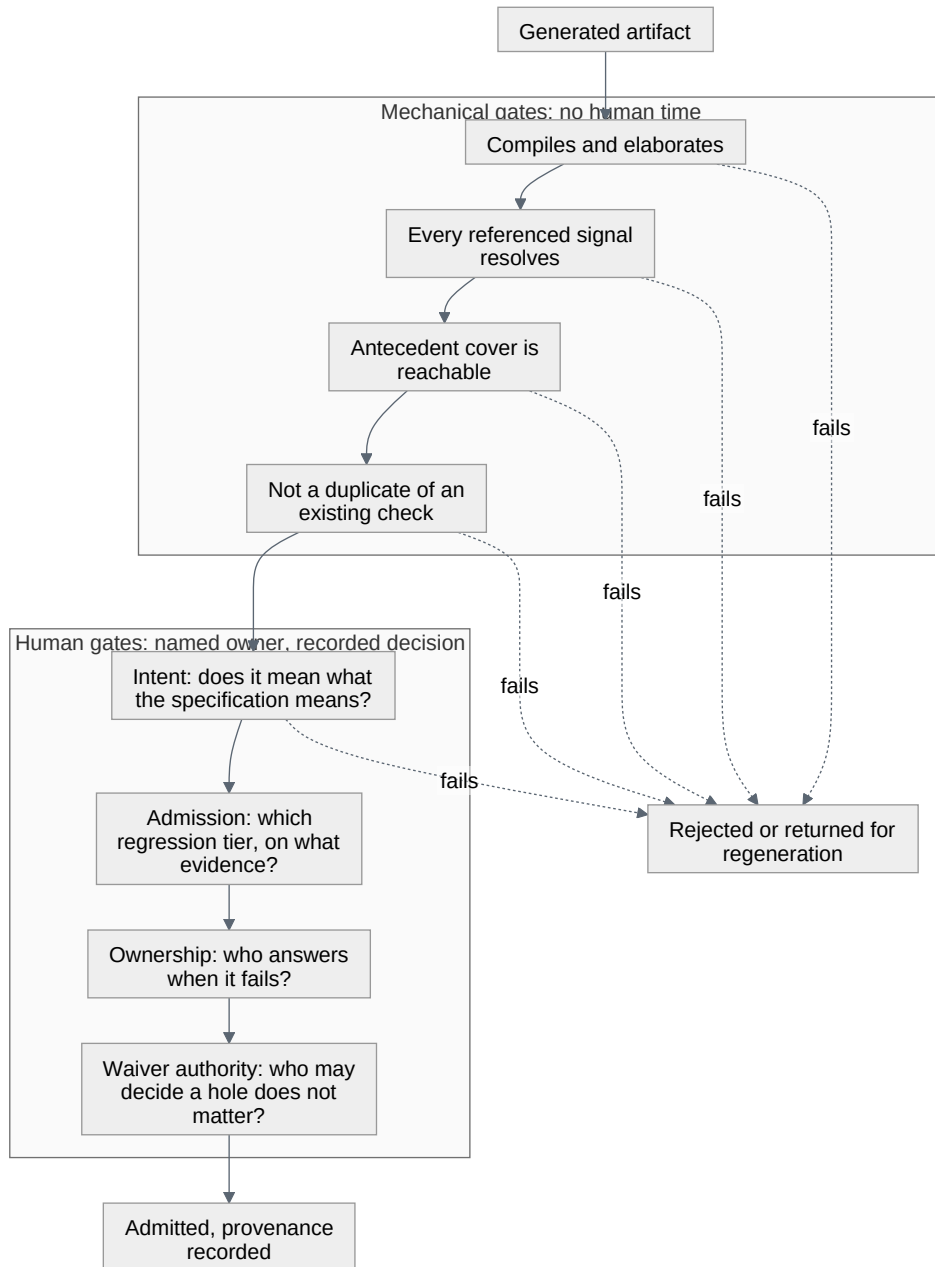


Figure 25.4 — The gates a generated artifact passes before it is admitted to a suite: four mechanical checks that cost no human attention, then four human gates, each with a named owner and a recorded decision. A failure at any mechanical gate, or at the intent gate, sends the artifact back rejected or for regeneration. The ordering is what makes the human gates affordable, since no artifact reaches a person until the machine has finished with it.

Every mechanical check precedes human review, so only artifacts surviving all such checks consume reviewer attention, making the human gates affordable.

Gate 1: intent. *The artifact must express the specification's meaning.* The owner is whoever owns the plan row the artifact closes, rather than whoever ran the generator, and Chapter 5's traceability makes the assignment unambiguous. They need two things: the specification sentence the artifact came from beside the artifact, and a demonstration that the artifact can fail, through the cover on its antecedent, a mutation, or a deliberately broken variant. A passing proof is not evidence at this gate; Section 25.3 is entirely about why. Left unowned, a wrong rule enters the suite and immediately inherits the suite's authority.

Gate 2: admission. *The gate establishes the regression tier and promotion criteria.* Chapter 12 puts a repeat-N stability check on any test entering a gating tier, to establish that it does not fail spuriously. Generated checks need this discipline to detect failures on behavior that is not a bug, distinct from random failures. A generated property earns a gating tier by running clean across a window of known-good history, and that window is the evidence. Without an owner, the first false failure on a Monday morning can prompt engineers to bypass the check with a plusarg.

Gate 3: ownership on breakage. *The engineer who answers a failure at 07:40 must be named in the file at merge, before the first failure.* If no engineer has established a generated component's purpose, the on-call engineer may lack someone prepared to explain it under time pressure. In that situation, the engineer can read the code but cannot recall what it was *for*, because no time was spent deciding. The accepting engineer must be able to explain the artifact without regenerating it, and the record must confirm that they did so.

Gate 4: exclusion and waiver authority. *The gate identifies who may decide a hole does not matter.* A coverage exclusion proposed by a model is still a waiver in Chapter 7's sense, carrying the same five fields: the hole, the reason, the risk argument, an owner, an expiry. Nothing about its origin changes who signs it. A coverage metric that treats excluding a bin as equivalent to covering it gives an optimizing system an incentive to favor exclusion.

A fifth place where a person belongs is a cap rather than a gate. Two automated stages iterating against each other through generation, criticism and regeneration need a cap, such as the five-iteration threshold above. A 2026 survey of agentic design automation names *deadlooping*, oscillation between states without convergence, as the failure a cap prevents; it also identifies context loss, where an agent optimizes a local sub-goal after forgetting a global constraint, and tool hallucination [13]. A tool hallucination is an invented tool call or an invented result from one. Without a cap, a loop consumes budget and returns output without clearly signaling failure.

Additional checks for generated artifacts

Review of machine-written verification code differs in kind from ordinary code review: three checks carry weight they do not carry when the author is a person.

- **Every referenced signal exists and is the one intended.** Generated properties can name signals absent from the design: the comparison in Section 25.2 counted eleven such references in a batch of thirty-eight ^[9], and the specification-driven framework needed human engineers to remove unmappable generations before it could be scored at all ^[8]. Elaboration catches non-existent signals but cannot catch a *wrong-but-existing* signal among similarly named ports.
- **The artifact is in a position to fail.** Vacuous passes are a named, observed failure mode of generated properties in industrial use ^[10], which promotes Chapter 11's cover-with-every-implication rule from good habit to admission criterion.
- **Complete batch review.** Because a generator's errors are correlated, a review covers every artifact in a batch or rejects the batch.

Throughput is limited by how fast artifacts can be read, regardless of how fast they can be generated. In this example, a flow producing two hundred properties in an afternoon has not saved a week unless someone reviews two hundred properties; without that review, it produces a passing regression with an unknown number of wrong rules. Adoption requires a strong mechanical filter and batches small enough for complete review; these are flow design constraints.

25.5 Governance

Governance records support the escape analysis of Chapter 7, triggered in this example by a field failure two years after tape-out; an escape analysis is the blameless post-mortem of an escape that feeds changes into the next plan and checklist. The sign-off package points to a property as the evidence that a particular behavior was covered. A sign-off review is the formal review that judges evidence against the plan and accepts residual risk, and it must then establish where that property came from. If the only answer is that a model wrote it and someone approved it, reviewers cannot re-derive the property, tell whether it was checked against the specification or against the RTL, or establish whether its source specification is the revision that shipped; governance records keep that review possible.

The provenance record

Every generated artifact needs a record in the repository, rather than a wiki, with these fields:

- **What produced it.** The system, its version, and the identity and version of the model underneath. Section 25.2 showed the same flow varying from 4.33% to 41.43% end-to-end success on one benchmark tier depending on which model was underneath ^[10]. Model identity is a parameter of the result.

- **What it was given.** The exact specification document, revision and section, together with the prompt or template version. Without the revision, a later reviewer cannot tell whether the property predates the change that made it wrong.
- **The sampling settings, and how many runs were kept.** See below.
- **Which mechanical checks it passed, and their versions.** Compilation, signal resolution, antecedent reachability, duplicate detection.
- **Who accepted it at each human gate, and against what text.** A name for each artifact and the specification sentence used in review; a batch signature is insufficient.
- **Whether it was edited after generation, and by whom.** Unless manual edits are recorded, a hand-edited generated property has a misleading history when the generator is re-run on the next revision.

The last two fields must be retained for defensible sign-off. Machine output records audit the machine; acceptance records audit the human decision under review.

Reproducibility of generated artifacts

Chapter 9 makes the seed part of a run’s identity and is precise about what it does and does not reproduce. Generated artifacts sit outside that discipline entirely, and the vocabulary invites a false analogy. A language model’s sampling parameters are not a seed in Chapter 9’s sense: identical inputs can yield different outputs across runs, so one research group issues the same prompt three times and takes the union of the property sets to make the result robust enough to report ^[9]. Sampling temperature is the setting that governs how varied a model’s output is. An industrial report finds that effective sampling temperature varies by design, with wrong settings producing overly deterministic or overly random output ^[10].

The record should state the sampling settings and how many runs were kept: “we generated this” is insufficient for reproduction, while “we took the union of three runs at these settings” is at least describable. Regeneration cannot re-check an artifact: a second run agreeing with the first provides no corroboration, since both come from the same process with the same blind spots. The artifact is what shipped, so it is the artifact that must be versioned, not the procedure that produced it.

Data governance

Industrial obstacles extend beyond accuracy. A 2023 survey groups industrial concerns under three headings: model generalization, model scalability and data governance; the third means that an engineer who cannot send data outside her organization and has no machine-learning infrastructure inside it cannot use a model requiring training on her data [6]. The constraint has sharpened rather than softened. A 2026 survey of agentic design automation names data privacy and intellectual-property exposure in cloud-based inference as a first-order trust problem, and lists the mitigations: local deployment, federated approaches, and differential privacy [13]. A federated approach trains a model across several sites without moving their data; differential privacy adds calibrated noise so that no single record can be recovered from the result.

For a verification lead, this procurement question requires an engineering answer. Before a pilot starts, the verification lead must establish which specifications, RTL, failure logs and coverage databases leave the organization. The verification lead must obtain that account in writing from whoever owns the design's confidentiality, the duty to keep its specifications and RTL inside the organization, rather than from the team running the pilot.

Adoption criteria

Three criteria can be assessed in a fifteen-minute meeting in this example.

The criterion is a check independent of what produced the artifact. Examples are a diff against a machine-readable source, a proof against a reference the artifact was not derived from, or an applied fix followed by re-running the failure. If the only evidence is the generator's confidence, or a proof passed against the very design the artifact came from, there is no check.

The team owns a baseline and can re-measure it on its own data. Yesterday's triage decisions, last quarter's regression runtime, the current suite's coverage. The team's own last quarter supplies a baseline that a vendor benchmark cannot.

Acceptance may require an intent judgment that no run can settle. If so, the flow runs no faster than that judgment can be made, and the time for that judgment should be budgeted before the pilot rather than discovered during it.

Immediate adoption is appropriate where the first two criteria hold and the third does not. Regression ranking and selection qualify: the team owns the runtime baseline and the coverage database used for checking, subject to Section 25.1's constraint that a compressed suite detects changes and the full fresh-seed sweep must survive alongside it. Supervised triage classification qualifies where a labelled failure history already exists, with a published agreement rate re-measured on a schedule. Generated scaffolding qualifies wherever a mechanical diff against a machine-readable source decides correctness.

A pilot in a non-gating tier suits property generation from a specification on a block whose specification is unusually precise, with reviewing capacity budgeted first and a team-owned statistic, the fraction surviving the intent gate, recorded from

the first batch. Counterexample debug assistance also suits a pilot: its output is a hypothesis a human was going to form anyway, and its proposed fix can be applied and the proof re-run, providing a check the assistant does not control [11].

Deferral is appropriate for flows whose sign-off artifact is their own report. The 2026 survey reports industrial tape-out by reinforcement-learning systems, while no language-model-driven autonomous agent has yet demonstrated complete, human-free tape-out of an industrial chip [13]. The survey calls reading high module-level benchmark pass rates as system-level evidence the *unit-test fallacy* [13]. Every headline figure in Section 25.2 concerns a block, protocol or curated benchmark; none concerns an SoC.

The survey places proposals on a scale from tool-assisted design through prediction-oriented copilots to agentic orchestration, where an agent drives the task and a human monitors, and finally autonomy, where a human supplies only intent; its near-term expectation from 2026 is strong assistants under human oversight rather than autonomous flows [13]. Proposals should state their level on this scale and budget for that same level.

The artifact must arrive with a mechanically checkable argument for itself; the survey calls this proof-carrying and pairs it with a reasoning trace linking the intent assigned to an agent to the action it took, so an engineer audits the decision rather than the output [13]. A generated property arriving with its antecedent cover and the specification sentence it was derived from is a small instance of exactly that, available today without new tooling.

Assessing changes in the evidence

Every capability figure in this chapter is dated in the text, and the dates run from 2003 to 2026. The classical results have been stable for two decades and will most likely still hold in five years: what a coverage-directed generator optimises, and that models decay, are structural facts rather than technology facts. The language-model figures are the perishable ones.

The figures will age unevenly; their expected rise is insufficient as a signal of progress. Three changes matter: whether results are reported on designs larger than a block; whether the pass criterion distinguishes correct rules from plausible ones rather than accepting proof against known-good RTL; and whether the mechanically checked metric is reported alongside the model-judged one rather than instead of it. Without these changes, a higher percentage establishes only improvement on the same benchmark.

IN PRACTICE

The team should select one statistic of continued effectiveness (human-triage agreement, coverage retained after compression, or fraction passing the intent gate) and re-measure it on a fixed cadence on the bug-curve dashboard. The dashboard measure should inform decisions to retire an unchecked technique.

When an external demonstration brings enthusiasm and a deadline, the team should respond with a pilot and review budget rather than a debate about the technology. Manual edits invalidate provenance unless the *edited by* field records them. A changed coverage model can leave an unowned reward function ineffective. If the first generated batch exceeds its estimated review effort because reviewers are learning what this generator gets wrong, that useful effort should be recorded to budget the second batch.

PITFALLS

1. **Reading a pass rate without its pass criterion.** Symptom: a proposal quotes a percentage and cannot say, when asked, what the study counted as correct. Cure: the definition, whether one sentence or longer, should be read before interpreting the number.
2. **Chaining results from different baselines.** Symptom: a slide composing one paper's speedup with another's accuracy into a single trend. Cure: each figure must retain its benchmark, baseline and underlying model; results must remain separate.
3. **Judging a generated property by whether it proves.** Symptom: "thirty-six of forty passed formal" offered as evidence of quality. Cure: proof against the source design establishes consistency without intent; admission requires an antecedent cover and specification sentence.
4. **Sampling a generated batch.** Symptom: "we spot-checked five and they were fine", or two hundred generated properties without a complete review. Cure: the batch must be reviewed or rejected because correlated errors make sampling uninformative; reviewing capacity must be budgeted before the pilot because it sets throughput.
5. **Replacing the random sweep with a compressed one.** Symptom: regression runtime halves and the bug discovery rate follows unnoticed. Cure: the compressed suite should detect changes, while the full fresh-seed sweep finds bugs.
6. **The model without retraining.** Symptom: a triage classifier that was excellent at tape-out and is silently wrong two derivatives later. Cure: an owner, a cadence, and a published agreement rate that anyone can see falling.

SUMMARY

- Classical machine learning has real results in coverage-directed generation, failure classification and regression selection, spanning two decades; each optimises something narrower than the goal, and each needs re-measuring once deployed [1, 6].
- The corpus's headline property-generation result, 89% of 56 properties on one design in 2024, defines *correct* as passing formal proof against known-good RTL after manual removal of unmappable generations; that definition determines what the number means [8].

- Generation optimizes plausibility, with correctness a by-product. A property derived from the design cannot disagree with the design ^[8].
- For the same system, model-judged quality scores and mechanically checked outcomes do not rank alternatives the same way; a study reporting only the former establishes how convincing its output reads ^[11].
- Human gates assign owners for intent, tier admission, ownership on breakage and waiver authority; all mechanical checks precede them, reserving human attention for decisions only a human can make.
- A generator's errors are correlated across a batch, invalidating sampling as a review strategy and making reviewing capacity the limit on adoption.
- Defensible sign-off two years later requires records of human acceptance and the text used in review; machine output records alone are insufficient.
- As of a 2026 survey of agentic design automation, no language-model-driven autonomous agent had demonstrated complete, human-free tape-out of an industrial chip ^[13]; every capability figure in this chapter concerns a block, protocol or curated benchmark rather than an SoC.

EXERCISES

Exercise 25.1 (calculation). §25.2 reports 89% of 56 generated properties correct for a framework that reads a full specification document, and 6 of 71 candidates passing the same proof criterion for the underlying model alone. Compute the property count behind each of the two rates. State which of the two was scored only after human engineers had removed the generations referencing signals the tool could not map. Explain what that removal does to a comparison of the two rates, using the success criterion the fourth column of §25.2's evidence table records.

Exercise 25.2 (judgement). The chapter opens with two proposals reaching one verification lead in the same week. The first offers a failure classifier for the video codec's nightly regression, which agreed with the human triage engineer 88% of the time on a held-back month of history. The second offers a specification-to-assertion flow piloted on the crypto engine, most of whose generated properties passed a formal proof against the current RTL. Decide which of the two reports evidence about the use it proposes, and give the property of the first measurement that the second one lacks. State what the passing proofs do establish, pointing to the section that explains why a proof verdict settles consistency rather than intent.

Exercise 25.3 (judgement). A verification lead applies the adoption criteria of §25.5 to three proposals for the reference SoC. The first ranks and selects runs from the existing nightly regression; the second generates properties from the crypto engine's specification; the third classifies the video codec's nightly failures by root cause. Decide for each proposal whether §25.5 places it in immediate adoption or in a pilot within a non-gating tier, and give the criterion that decides it. State the condition §25.1 shows to be unmet for the third proposal, together with the work that condition demands first.

Exercise 25.4 (artifact). Read the provenance record below against the field list of §25.5. Two of its fields carry no value. State what a sign-off review two years later can no longer establish without each of those two. Identify the filled entry that the field list of §25.5 rejects as insufficient, and give the reason that section states for the rejection.

Artifact:	a_keysel_stable_gen, crypto engine key-slot stability
Produced by:	in-house generation flow, flow version recorded
Model underneath:	
Given:	crypto engine specification, section on key selection
Specification revision:	
Sampling settings:	the union of three runs at recorded settings
Mechanical checks:	compiles; every signal resolves; antecedent cover reachable; not a duplicate of an existing check
Accepted at human gates:	one batch signature for the whole batch
Edited after generation:	recorded, one manual edit by the accepting engineer
Proof result:	proved against the current RTL

Exercise 25.5 (artifact). The record below documents how one generated batch entered a gating regression tier on the crypto engine. State why §25.3 denies that the recorded review supports the recorded conclusion, and give the property of a generator’s errors that decides the point. Identify the two admission conditions of §25.4 that this record leaves unmet, and state the consequence §25.4 attaches to each.

Batch:	generated properties, crypto engine key ladder
Mechanical gates:	all compile; all signals resolve; three antecedent covers unreachable, returned for regeneration; two duplicates of existing checks rejected
Review performed:	five of the surviving properties read in full, all five judged correct
Recorded conclusion:	the batch reads clean and is admitted as a whole
Admission tier:	gating regression, from the first merge
Stability window:	not run
Responder on breakage:	

Exercise 25.6 (calculation). §25.2 reports one agentic formal flow run under three general-purpose language models, completing 17.78%, 43.19% and 48.77% of runs end to end on the easiest benchmark tier and 4.33%, 9.00% and 41.43% on the hardest. Two of the three models sit within six points of each other on the easiest tier. Compute their separation on the hardest tier, and compute the ratio of the highest hardest-tier figure to the lowest. State what those two results permit a proposal to quote as the flow’s success rate, using the sentence of §25.2 that names what such a figure characterizes.

FURTHER READING

Within the corpus, ^[6] introduces classical machine learning in functional verification as of 2023, covering applications and the clustering-versus-classification asymmetry, with a section on industrial obstacles; ^[1] is its companion, a 2025 review of why so

little of the literature reached deployment. For coverage-directed generation, [2] gives the founding result and its costs; [3] reviews the field at its 2012 maturity; [14] broadly surveys directed test generation in 2024 and refers readers to [3] for machine learning. On generated properties, [8] and [9] should be read together in that order to compare their definitions of success; [10] is the corpus's one industrial agentic-formal deployment report and details its failure modes; [11] exemplifies how results should be reported, pairing a model-judged metric with a mechanically checked one. For the wider landscape, [5] surveys language models across design automation as of 2025 and [13] supplies the autonomy scale, the failure taxonomy and the governance vocabulary used in Section 25.5. [12] applies the same evidentiary discipline to security verification and complements Chapter 23.

Section 25.2's table supplies the reading criteria: benchmark, pass criterion and baseline must precede interpretation of the percentage. A paper must state all three to support an actionable conclusion.

REFERENCES

1. **P8** C. Bennett and K. Eder, "Review of Machine Learning for Micro-Electronic Design Verification," arXiv preprint arXiv:2503.11687v1, March 2025.
2. **P16** S. Fine and A. Ziv, "Coverage Directed Test Generation for Functional Verification using Bayesian Networks," Proc. 40th Design Automation Conference (DAC '03), Anaheim, CA, pp. 286-291, June 2003.
3. **P19** C. Ioannides and K. I. Eder, "Coverage-Directed Test Generation Automated by Machine Learning - A Review," ACM Transactions on Design Automation of Electronic Systems, vol. 17, no. 1, Article 7, 2012.
4. **D1** E. Ohana, "Closing Functional Coverage With Deep Reinforcement Learning: A Compression Encoder Example," Proc. DVCon U.S., 2023.
5. **P24** J. Pan, G. Zhou, C.-C. Chang, I. Jacobson, J. Hu, and Y. Chen, "A Survey of Research in Large Language Models for Electronic Design Automation," arXiv preprint arXiv:2501.09655v1, January 2025.
6. **D35** D. Yu, H. Foster, and T. Fitzpatrick, "A Survey of Machine Learning Applications in Functional Verification," Proc. DVCon U.S., 2023.
7. **P7** D. N. Gadde, S. Simon, D. Lettnin, and T. Ziller, "Improving Simulation Regression Efficiency using a Machine Learning-based Method in Design Verification," Proc. DVCon Europe, 2022.
8. **P11** W. Fang, M. Li, M. Li, Z. Yan, S. Liu, H. Zhang, and Z. Xie, "AssertLLM: Generating and Evaluating Hardware Verification Assertions from Design Specifications via Multi-LLMs," arXiv preprint arXiv:2402.00386v4, February 2026.
9. **P12** Y.-A. Shih, A. Lin, A. Gupta, and S. Malik, "FLAG: Formal and LLM-assisted SVA Generation for Formal Specifications of On-Chip Communication Protocols," arXiv preprint arXiv:2504.17226v1, April 2025.
10. **D25** A. Kumar, D. N. Gadde, K. K. Radhakrishna, and D. Lettnin, "Saarthi: The First AI Formal Verification Engineer," Proc. DVCon, 2025.
11. **P22** Y. Bai, G. Bany Hamad, C.-T. Ho, S. Suhaib, and H. Ren, "FVDebug: An LLM-Driven Debugging Assistant for Automated Root Cause Analysis of Formal Verification Failures," arXiv preprint arXiv:2510.15906v1, September 2025.
12. **P13** K. T. Hasan, M. A. Hasan, N. Alam, M. T. Islam, U. Das, and F. Farahmandi, "AI-Assisted Hardware Security Verification: A Survey and AI Accelerator Case Study," Proc. IEEE VLSI Test Symposium (VTS), Napa, CA, pp. 1-7, 2026.

13. **P23** Z. Zang, Y. Song, A. Wang, B. W.-K. Ling, Q. Sun, Z. Lei, F. Yang, C. Zhuo, and J. Luo, "The Dawn of Agentic EDA: A Survey of Autonomous Digital Chip Design," arXiv preprint arXiv: 2512.23189v2, February 2026.
14. **P10** A. Jayasena and P. Mishra, "Directed Test Generation for Hardware Validation: A Survey," ACM Computing Surveys, vol. 56, no. 5, Article 132, 2024.

26. The Road Ahead

Verification's limiting constraint is choosing what to check, as the next generation of the book's DMA architecture illustrates when it returns to tape-out. The flagship SoC's DMA has eight channels and inherits the reference SoC's register frontend, burst splitter and AXI backend, together with its discipline of splitting bursts at the 4 KB boundary, with 40-bit physical addressing and descriptor chaining. The register frontend is the DMA's first stage, the register interface that software programs. A DMA channel is an independent transfer engine with a state of its own and a share of the AXI backend. The burst splitter is the DMA stage that splits each transfer into bursts that the AXI address rules allow, and the 4 KB boundary is the address line that AXI forbids a burst to cross. The 4 KB boundary is now enforced by a formal property rather than only by a scoreboard check that depends on receiving the right transfer descriptor, the record that software writes to program one DMA transfer. Descriptor chaining is the arrangement in which each transfer descriptor points to the next one and the DMA follows the chain without software. The stimulus is a scenario space described once and retargeted from simulation to the prototype, rather than a constraint block written for one environment; §26.2 names that practice. Regressions gate on every commit and coverage merges across platforms; machine time is no longer disputed by this block's team.

The bug that arrives in week 31 is SOC-1284's descendant. The burst splitter again mis-splits a negative-stride 2D row across the 4 KB boundary: wider addresses and a different module retain the arithmetic mistake, but the second half now writes a line the host cluster holds Modified. The host cluster is the flagship's cluster of application-class cores. A 2D row is a single line of a two-dimensional transfer, a contiguous run of bytes, and the stride is the signed address step from the start of one row to the start of the next. A cache holds a line Modified when it has written to that line and has not yet returned it to memory. The scoreboard that caught the original sees nothing, because the corrupted bytes never reach memory during the check; they surface at a writeback, later, somewhere else. The team finds it eventually, from a firmware symptom on the prototype: weeks of debug on a platform where the corrupted line is not visible, for a bug whose ancestor a scoreboard caught in one night.

In SOC-1284, two hundred and fourteen directed tests passed and a constrained-random run reached the combination: a negative stride, a crossing of the 4 KB boundary and a second DMA channel active at the same time. A directed test is a piece of stimulus whose content and order are written out by hand, and constrained-random is the automatic generation of legal but unlikely stimulus under constraints. Escape analysis recorded that the functional coverage model lacked a cross of those three dimensions, leaving the hole invisible to the team's metrics; a cross is the coverage item that counts the runs in which two or more conditions hold together. Escape analysis is the blameless structured post-mortem of an escape. A coverage model is the multi-dimensional region that attributes and their values mark out, and an attribute is a single dimension of a behavior together with the set of values it can take. In the

descendant, no engine was wrong: the sound proof covered the *previous* generation's backend but entered the new plan as a plan row already discharged. A plan row is the line of the verification plan that carries one feature, its method and the evidence that closes it, and it is discharged when that evidence exists and is linked to it. No plan row stated that a data-integrity check crossing into a coherent domain is a different claim requiring different evidence. Both outcomes were constrained by knowing what to ask, rather than by compute, stimulus or checking.

CHAPTER MAP

§26.1 traces the constraint's migration across four generations of tooling, the evidence for each stage and its dependence on verification level. §26.2 examines the benefits and limits of shift-left and shift-right and their combined pressure on the middle of the flow. §26.3 distinguishes properties that survive better tooling from those that only seem permanent. §26.4 describes verification as a profession and four weaknesses it retains.

26.1 The bottleneck moves

This section's historical argument is the book's interpretation, not a conclusion of the cited studies. The sources supply the evidence for each stage, and each measurement belongs to that stage: figures taken in different years, from different populations and with different instruments cannot form a trend line, just as two speed-ups measured against different baselines cannot be multiplied. The stages share a pattern without a common quantitative scale.

A binding constraint is the activity to which a doubled budget would go. Verification's binding constraint has occupied four positions in turn.

Cycles. The founding constraint was raw simulation throughput. RTL simulation runs approximately a billion times slower than the target clock at full application-class scale, the high end of the band this book declares, and that ratio is why an operating-system boot takes years in a simulator for seconds of silicon execution; it is also why hardware-assisted platforms exist to recover a hundred to a thousand times of that slow-down ^[1] (Chapter 17 works the economics). This constraint has not disappeared; it has been *relocated by level*. For a block environment the size of the reference SoC's DMA, cycles have long ceased to bind because the nightly suite fits in a night. They still bind firmware runs on the flagship: the 2024 industry data records that plateauing processor frequency scaling leaves simulation-based techniques struggling with growing complexity, making acceleration essential for many designs ^[2].

Stimulus. Affordable cycles made stimulus selection the next constraint. Hand-enumerated tests stop scaling in the low hundreds (Chapter 9); replacing enumeration with description lets a constraint solver generate legal but unlikely traffic by the thousand, an approach that arrived with the constraint-based verification languages of the late 1990s and became the industry default. The reported limits identify the

level at which stimulus generation ceased to bind. In 2014, constrained-random adoption appeared to be leveling off, with scaling offered as the reason: the technique works well at IP block or subsystem level but does not scale to entire SoC integration [3]. The difficulty at that level is deciding *what* to generate rather than generating stimulus. A decade later, constrained-random simulation was the one exception to general stabilization of simulation-technique adoption [2]; that continued growth is consistent with a technique that has solved its problem and is still reaching teams that have not used it.

Checking. A testbench must evaluate the traffic it generates. The self-checking discipline addressed this constraint by computing the verdict rather than inspecting waveforms (Chapter 3), architecting the environment so that stimulus, driving, monitoring, checking and coverage are five separable responsibilities (Chapter 8), and using assertions to carry rules local enough to state (Chapter 11). The adoption data shows this in standardization: SystemVerilog Assertions is the predominant assertion language and UVM the predominant IC/ASIC testbench methodology [2]. The video codec illustrates the remaining limits. A standards body that publishes a bit-exact reference decoder supplies the oracle: expected pixels are defined independently of opinion, and checking reduces to a comparison. The standard leaves rate control and latency unspecified, along with encoder behavior under line-buffer back-pressure; someone must decide to write those checkers. Chapter 3 derives the rule from this case and the instruction-set simulator: a definitional oracle defines exactly one projection of correctness.

Knowing what to check. Choosing what to check now binds; the evidence is necessarily indirect because unasked questions leave no trace in tool output.

First-silicon success is the outcome in which the first fabricated parts are already fit for production. First-silicon success stood at 14 percent in 2024, the lowest value recorded in twenty years and the low point of a decline since 2012 [2]. A decade earlier it was about 30 percent [3]. On the FPGA side, 87 percent of projects reported non-trivial bugs escaping into production in 2024 [4]; that side is a different population measured by a different metric, not a point on the IC/ASIC curve. The two figures need not and cannot combine into one. Improved engines and wider adoption of mature techniques coexist with poor outcomes on both metrics; these observations do not establish that better engines alone improve outcomes.

The 2024 report traces complexity growth to new requirement layers beyond basic functionality: clocking, security, safety and hardware-software interaction requirements that did not exist years ago [2]. Each layer introduces questions that must be formulated before an engine can answer them. Design errors top the root-cause data; changing, incorrect or incomplete specifications remain a common concern for verification engineers and project managers [2]. A specification defect directs checking at the wrong claim.

The 2024 study reads the spread in verification time as a reuse effect: projects spending the least time typically reuse pre-verified IP, while those spending the most carry high proportions of newly developed IP [2]. Reused IP can cost less because someone has already worked out what to check, without its logic being simpler.

Three further signals concern the work of defining and interpreting checks. Debug, the human work of understanding a failure, took more of a verification engineer's time than any other activity in 2024 and is named the discipline's major bottleneck [2]. Automatic formal applications were the fastest-growing formal category in the 2024 data, growing at an 8.7 percent compound annual rate against 5.8 percent for property checking [2]; such an application supplies its own properties from a register description or a connectivity table, so for those narrow classes the work of deciding what to check moves upstream, to whoever wrote that description or that table (Chapter 15). A property set cannot establish whether it is sufficient, and Bormann's completeness criterion supplies the test from outside it. The criterion asks whether the properties allow unique determination of one output trace for each input trace; on a published project list, almost all the circuits verified to that criterion functioned properly after fabrication, and the few overlooked faults were traced to misapplication of the methodology rather than to the criterion [5] (Chapter 14 states its refinement in full). This implemented method is expensive, and what it tests specifically is the sufficiency of the property set, which no other part of the toolchain can assess.

The reference SoC illustrates the same constraint at block scale. A beat is a single data transfer within a burst. The neural accelerator's weight prefetcher wraps its buffer pointer one entry early when the weight stream ends mid-beat under DMA contention. It escaped 100 percent code coverage entirely: the line was executed and the branch was taken, but nothing about the structural metric could represent the conjunction. A tensor width class is a band of tensor widths that the datapath treats alike, and Chapter 6 defines the classes. Only a functional cross of weight precision $\times$ tensor width class $\times$ contention demands the scenario, and that cross exists only if somebody decided those were the three axes; no engine makes that choice.

Career specialization should follow the constraint at the relevant level. At block level, the model of what must be true should be assessed alongside throughput; on a full-chip firmware workload, cycles may still be the constraint. Both require the skill of translating specifications into claims precise enough for a machine to check.

26.2 Shift-left and shift-right

Shift-left means verification starting earlier and running against something other than finished RTL. Static analysis moves structural questions to the earliest point at which they can be asked: clock-domain crossing tools identify domain issues directly on an RTL model at earlier stages of the design flow because the class of metastability bug they target cannot be demonstrated on an RTL model in simulation at all and is hard to reproduce in the lab [2] (Chapter 13). Virtual platforms let firmware run against functional models of the memory map before the hardware exists, offering far more controllability and observability than debugging the same code over a debug port, and sometimes being the only option available while the hardware is still under development [6] (Chapter 18). Controllability is the ability to steer a design into a given internal condition using only drivable interfaces. Portable stimulus describes a scenario space once and retargets it across platforms, so the modelling

work precedes any particular environment (Chapter 9). Continuous integration binds the whole loop to the commit rather than to the milestone; one 2026 research-programme account reports that running UVM-based verification and FPGA-based checks inside CI increased productivity by making the verification flow transparent from RTL development [7].

Shift-left's specific benefit is finding defects in the *specification* earlier, beyond advancing bug-finding. Writing an assertion against a document rather than a design can expose a difference between its text and the assumed meaning; Chapter 11 recommends this as a low-cost way to detect specification ambiguity. This addresses the persistent specification problems in the root-cause picture of §26.1.

The sensor hub illustrates both the architectural work and its limits. The sensor hub's always-on 32 kHz island, wake sources and retention list are architectural decisions made and reviewed in this example long before RTL implementation. The design's power intent, in the form standardized by IEEE Std 1801-2024 (UPF), is which domains exist, what retains and which supply-state combinations are legal [8]. Power intent is the description held outside the design source, because the same logic is integrated into different power architectures; UPF, the unified power format, is that description in machine-readable form. This early architectural work does not simulate a design. This architectural work cannot reach the reference SoC's canonical escape in its event unit, because that escape is a property of the implementation's sequencing. In that bug a retention register loses its configuration when a wake event races the release of isolation, and the bug surfaced in power-aware gate-level simulation after RTL sign-off (Chapter 19). Isolation is the defined behavior of a signal while its driving logic is unpowered, and its release is the moment that behavior ends. Shift-left moves *claims* earlier without moving their evidence; confusing them can leave a project at week 40 with an early plan and no discharged rows.

Shift-right means the design continues to be checked after tape-out. **Validation** is checking real hardware rather than a model, the distinction Chapter 20 states in full; it is a planned stage with its own instruments rather than a fallback for what verification missed; and the industry commits silicon to that stage: in some modern SoCs over 20 percent of the design real estate, alongside a significant component of the CAD-flow effort, is devoted to silicon validation [1]. The flagship carries per-core processor trace, eight bus monitors with programmable triggers, a 64 KB on-chip trace buffer and a debug access port per power domain.

Shift-right covers what no pre-silicon platform provides: the real device running real software at real speed for real durations. It cannot supply observability beyond the hardware capacity. Every mechanism writes into a fixed store, so **signals × cycles ≤ buffer**: 64 KB buys 128 traced signals for 4,096 cycles, or 256 signals for 2,048, and that rectangle of signals against cycles is fixed at tape-out (Chapter 20). Lab work cannot enlarge that store, so post-silicon debug must deliver a pre-silicon reproducer, short enough to simulate and small enough to load while still failing; an explanation alone does not meet that requirement. Shift-right should therefore return problems to the middle of the flow.

The book's judgment is that these directions put the middle of the flow under strain. The block-level RTL environment, covered in this book's middle parts, is the only stage with both full observability and a real design. Shift-left puts pressure on block-level RTL verification by moving claims onto models that cannot carry the evidence. Shift-right adds pressure by promising that whatever remains will be caught on a platform that no longer offers full observability, and the cost of finding anything there rises for that reason. Both movements are correct in their own terms. Pursuing both can leave a strong early plan and lab with little block-level verification between them, despite the full observability and real design available at that stage.

Scenario. Late in the reference SoC project, the SoC integration team proposes cutting the crossbar's block-level constrained-random regression by half, on the grounds that its protocol properties are formally proven and its integration behaviour will be covered in the lab. **Approach.** The verification lead accepts the first argument and refuses the second: proven properties retire the protocol rows, so those runs are removed. The remaining plan rows concern *arbitration under sustained back-pressure across two managers*, which is proven only up to a bound and reachable in the lab but not diagnosable within its trace buffer, so those seeds stay. A bounded proof is a result stating only that no violation exists within a stated number of cycles from reset. **Why.** The two ends of the flow retired one class of evidence and neither of them retired the other. Only a retirement argument for each row identifies which half was kept; a percentage cut cannot do so.

26.3 What does not change

A single test distinguishes properties of the problem from tooling limits: whether each would remain true if every tool became a thousand times better. Confusing these categories leaves readers ten years later unable to identify which half of an account has become obsolete.

Six properties meet this test.

State-space size grows exponentially with the number of state bits, making abstraction necessary. A state space is the set of all the configurations a design's state elements can take. One DMA channel's 192 bits of architecturally visible state yield 2^{192} , about 10^{57} , configurations; simulation visits a vanishing fraction (Chapter 3 gives the arithmetic). A thousandfold faster engine adds capacity for ten state bits. A coherent hub keeps the cached copies of a line consistent across the caches that share it. In the coherent hub, the MESI state of every line in every cache forms a product space no tool can enumerate, so the proof runs against a model cut down to two caches and one address, small enough to converge and large enough to contain the livelock and starvation cases that motivate proving coherence. Abstraction remains necessary even with a thousand times more capacity because the space grows faster than any engine.

Simulation shows the presence of bugs, never their absence. Testing yields probabilistic assurance, not proof (Chapter 3). This is a statement about sampling, and no improvement in sampling rate converts a sample into a census.

Every practical oracle is partial. Reference models, scoreboards and assertions each check a projection of correctness; they must be combined so their blind spots do not overlap (Chapter 3). A partial oracle is a checker that verifies one projection of correctness rather than correctness itself. The radar front-end illustrates this within one design. Its FFT and detection pipelines use bit-exact arithmetic and therefore an equality oracle. One stage upstream, the receive chain from mixer through variable-gain amplifier to ADC needs a band, a window and a condition set for the same signal, and every downstream checker, coverage measure and pass/fail decision is rebuilt on those three (Chapter 21). The band is the tolerance the measured signal must stay inside, the window is the time interval over which it is measured, and the condition set is the set of operating conditions under which the band holds. An engine cannot convert that band into an equality, and someone must choose the band as part of the specification, whether or not the decision is recorded there.

A proof is a statement about a model, under assumptions, sometimes to a depth. “Formally verified” must state those three qualifiers as applicable: the model and assumptions, and any depth bound (Chapter 14). These enduring qualifications differ from the often-stated claim that “formal doesn’t scale,” which describes tooling and will go out of date. The qualifications remain: a model always abstracts something, assumptions always make claims about an environment, and a bound always limits the proof.

A plan specifies claims, and coverage samples an authored model. An environment has two separate aspects here: it records that a situation occurred, and it judges whether the design behaved correctly in it. The two draw on the same monitors, and their fidelities are traded off against one another ^[9]. Coverage therefore measures sampling rather than correctness (Chapter 6): “100 percent” describes the model’s extent without establishing its fidelity. This explains SOC-1284’s and the accelerator’s blind spots and survives the automation of Chapter 25: judgment still shapes a drafted plan’s claims, and the plan cannot detect what it cannot represent.

Activation, propagation and checking are three independent conditions, and controllability and observability bound the first two. A bug found requires all three: stimulus must activate it, the wrong value must propagate to a point the environment can see, and a check there must recognize the value as wrong. Each of the three can fail alone (Chapter 3). Observability is therefore designed into the hardware a year before it is needed, with a fixed capacity shared between signals and cycles rather than an adjustable observation range.

Three further claims describe current tools and practice. *Debug dominates verification time* was measured in 2014 and again in 2024 ^[2, 3]. The finding is a robust empirical regularity that describes current practice, and it is the most plausible candidate here for automation to change. Such a change would move the fourth of the constraints of §26.1, knowing what to check, rather than remove that constraint, since explaining a failure in seconds does not decide which questions to ask. *Formal is ca-*

capacity-limited to small blocks is a statement about engines. *Simulation is a billion times slower than silicon* measures today's tools rather than a permanent property. One further unsupported claim is that *70 percent of a project's overall effort is spent in verification*. Harry Foster is the author of the study series that supplies most of the measured numbers here. He identifies that figure as an example of the field's subjective and unsubstantiated claims; he points instead to a measured lineage: the Collett International Research studies of 2002 and 2004, the later of which had 201 eligible participants, and then the Wilson Research Group studies, with 1,886 eligible participants in 2014 [3]. The series applies that standard to itself: 1,492 eligible participants in the 2020 study, 980 in the 2022 one [10, 11]. A reported figure should include its sample size.

Worked example: a durability audit of four vplan rows. The DMA rows of Chapter 5 can be assessed by separating properties of the problem from contingencies of today's tooling. This short exercise tests the basis for retaining a practice. Bins are the counters that a coverage model keeps over sets of values, and an equivalence class is a set of input values assumed to exercise identical design behavior. A detection distance is the time and state between a mistake and the check that catches it.

Row	Durable	Contingent
DMA-F06a — 2D cross at the 4 KB boundary, closed by cg_2d_4kb at 100%	That the cross is a <i>chosen</i> set of equivalence classes, and that "100%" measures its extent, not its fidelity	That the cross has four axes and 72 cells; that a constraint solver budget forecloses the maximum-length straddle; that a human wrote the bins
DMA-F06b — p_no_4kb_crossing proven unbounded on the backend	That the proof is about a model, under assumptions, at a binding point somebody chose	That it converges unboundedly on this block at this size; that the property text is hand-written
DMA-N01a — scoreboard flags any write outside the legal destination range	That an end-to-end oracle pays a detection distance, and that it checks one projection	That it is cheap enough to leave enabled in every regression run
DMA-N01b — injection test proving the check fires	That a checker you have never seen fail is not evidence	That the proof of firing is a directed test rather than a generated one

The right-hand column identifies methodology that can become obsolete; the left identifies what outlasts it. The opening bug concerns F06b's left column: *the proof is about a model*, here the previous generation's backend.

26.4 Verification as a profession

Chapter 1's first pitfall is treating verification as a phase that starts when the RTL is done. Four independent observations establish verification as a profession.

Headcount. Demand for verification engineers has outgrown demand for design engineers since at least 2007: verification demand grew at a 12.5 percent compound annual rate through 2014 and design demand at 3.7 percent, and by that year the mean peak verification headcount per project already exceeded the design count [3]. The growth rates have since converged, starting from that inversion of the two headcounts, and in some market segments, including processors, a five-to-one ratio of verification to design engineers is not unusual [2].

Boundary. Organizational divisions no longer distinguish the work. In 2024, design engineers spent on average 49 percent of their time on verification activities [2]. Nearly half of design headcount thus represents verification effort, defining the profession by activity rather than organization. Chapter 24 takes up how such teams are built and how verifiers are trained.

Apparatus. The discipline has its own standards: an assertion language (IEEE Std 1800-2023) [12], a testbench methodology (IEEE Std 1800.2-2020) [13], a stimulus description language (Accellera PSS 3.0) [14], a coverage interchange format (Accellera UCIS 1.0) [15], a power-intent format (IEEE Std 1801-2024) [8], and as of March 2026 a standard for describing an IP's clock- and reset-domain-crossing interface so that different teams' tools can exchange collateral about it [16]. These standards identify the work of making claims portable between people rather than accelerating simulation. The newest standard's Accellera working group counted 154 members from 24 companies as of September 2025 [17]; the standard lists Agnisys, Arm, Blue Pearl Solutions, Google, IBM, Infineon, Intel, Marvell, Microsoft, NVIDIA, Qualcomm, Renesas, Siemens EDA, STMicroelectronics and Synopsys as eligible voting companies at the time of standardization [16]. Three years of collaboration among competitors indicates the importance they assigned to this problem.

Theory. The discipline has its own reasoning about state-space arguments and the oracle problem, coverage as sampling, and proofs qualified by model, assumptions and depth; these principles should inform daily engineering even when the engineer cannot name them.

Judgment is the professional contribution that tooling has been least able to absorb through four generations. Reasoning about what could be true rather than what is makes the work demanding, suiting some minds and frustrating others. The work also carries an unusual professional obligation, Chapter 2's independence of judgment: a willingness to say "no, this is not the way to do it" to a designer with more years on the block, or to a schedule that wants the block closed. This is a duty even when inconvenient, independent of personality.

Four weaknesses remain in the field, in the assessment offered here. Each offers an individual an opportunity to improve on current practice.

Escape analysis and subsequent plans. Escape analysis should feed changes into the next plan and sign-off checklist (Chapter 7 gives the five-question form, Chapter 20 the post-silicon version), rather than end with a post-mortem slide. Using the next plan and sign-off checklist to carry those findings into the next project provides an existing mechanism for accumulating organizational knowledge.

Effort estimation remains unresolved. Debug takes more of a verification engineer's time than any other activity, and it is unpredictable and varies significantly between projects; those findings together make one thing a stated, unresolved management problem: estimates of effort and schedule drawn from the previous project's data [2]. Such estimates should account for those limits. Three quarters of IC/ASIC projects were behind their original schedule in 2024, against a historical average of about two thirds [2].

Measurement of verification environments. Measurement of the design's behavior and measurement of a verification environment address different questions: the latter should establish whether its checks can still detect a change. Mutation tests (Chapter 14) are how that capability is demonstrated: each injects a deliberate change into the design to check that the environment's checks detect it. A checker has no demonstrated detection ability until it has been seen to fail; this applies to a whole environment as well as a single assertion.

Cross-domain sign-off has no common measure. A functional sign-off aggregates coverage and plan rows. A safety argument aggregates fault classification against per-level metric thresholds (Chapter 22). A security result is a list of named claims and named gaps with no percentage available at all, and there will not be one soon (Chapter 23). These three incommensurable results reach the same tape-out review without an accepted way to state a combined position.

IN PRACTICE

Three practices address these risks. Inherited plan rows should be re-derived: a row marked *verified in the previous generation* should be reopened and its assumptions reread against the new integration, the step the flagship's DMA program skipped. An explicit written register should identify what the current program is *not* verifying, including gaps as well as waivers, because named gaps can be reviewed while unnamed ones remain surprises. A waiver is a written record that accepts a specific hole in the evidence, with its reason, its risk argument, its owner and its expiry. A fraction of every program should fund block-level environments on the newest logic despite competing demands from early modeling and a well-equipped lab. Where the three practices lack an associated metric, budget reviews should make the contribution of all three explicit.

PITFALLS

1. **Reading the bottleneck as industry-wide.** *Symptom:* a methodology decision justified by "the industry has moved past simulation throughput" on a block whose regression does not fit in a night. *Cure:* the constraint should be identified per level and per block before any reallocation.
2. **Treating shift-left as evidence rather than as claims arriving early.** *Symptom:* a plan whose rows were written in month two and whose closure column is

still empty in month nine. *Cure*: every early row should specify a platform and a date for producing evidence; otherwise closure remains unscheduled.

3. **Reduced block-level verification between the two ends.** *Symptom*: strong architectural modeling and a well-equipped lab coexist with block environments staffed by whoever is free. *Cure*: middle-of-the-flow work should be cut only row by row with a retirement argument, never by percentage.
4. **Confusing a durable property with a tooling limit.** *Symptom*: a design rule of thumb inherited from a capacity constraint that lifted two tool generations ago, still shaping how blocks are partitioned. *Cure*: under the thousandfold test, a rule changed by a better engine is a budget decision that belongs in the plan for re-consideration.
5. **Skilling only at the ends.** *Symptom*: a team fluent in architectural models and lab bring-up that cannot debug a scoreboard mismatch at RTL. Bring-up is the phase in which the environment and the DUT first exchange checked transactions. *Cure*: both skills ultimately depend on block-level verification; a pre-silicon reproducer is useless if the team cannot run it.

SUMMARY

- Verification's binding constraint has moved from cycles to stimulus to checking to knowing what to check; each tooling generation resolved a constraint and exposed the next. The migration is per-level, not uniform: cycles still bind full-chip firmware runs.
- The measured outcomes limit what can be claimed for better engines. IC/ASIC first-silicon success fell to 14 percent in 2024 from about 30 percent a decade earlier [2, 3]; separately, 87 percent of FPGA projects reported non-trivial bugs escaping into production in 2024 [4]. Adoption of the mature techniques climbed and then saturated across the same span. These outcomes do not establish that better engines alone improve results.
- Shift-left buys specification defects found early; shift-right buys the real device at real speed. Their contributions are distinct; pursuing both without sustaining block-level verification leaves early claims and later debug costs with insufficient evidence between.
- What survives any improvement in tooling: exponential state spaces and abstraction as the reply; presence but never absence; partial oracles; proofs qualified by model, assumptions and depth; a plan as a set of claims and coverage as a sample of an authored model; and activation, propagation and checking as three separate conditions.
- Debug's share of effort, formal's capacity limits and the simulation-to-silicon speed ratio should not be taught as permanent properties. All three describe a practice and its tools, not the problem.
- Verification has the headcount, standards and theory of a profession; its activities also took 49 percent of design engineers' time in 2024 [2]. In the book's assessment its open problems are escape analysis that does not reach the next plan, un-

resolved estimation, environments that go unmeasured, and the lack of a common language for combining a functional, a safety and a security position at one review.

EXERCISES

Exercise 26.1 (calculation). §26.2 states that the flagship’s 64 KB on-chip trace buffer buys 128 traced signals for 4,096 cycles, or 256 signals for 2,048. Compute the storage each traced signal consumes per cycle under those two operating points. Compute the cycles that remain when the signal count of the second operating point is doubled. State the point in the program at which the resulting rectangle stops being adjustable, and give the reason §26.2 attaches to the limit.

Exercise 26.2 (calculation). §26.3 derives the configuration count of one DMA channel from its 192 bits of architecturally visible state. Compute that count as a power of two and state its order of magnitude in powers of ten. Compute how many further state bits an engine that is a thousand times faster brings within reach, and show the arithmetic that fixes the answer. State which of the six durable properties of §26.3 the result supports, and identify the two qualifiers §26.3 attaches to the 192 without which the count is a count of something else.

Exercise 26.3 (judgement). Two environments in the flagship program contend for machine time. The first is a block-level environment for the reference SoC’s DMA, whose nightly suite fits in a night. The second runs firmware on the flagship, where the workload does not complete between one working day and the next. Decide which of the four constraints of §26.1 binds in each case, and give the sentence of §26.1 that makes the answer differ between the two. State the pitfall of the chapter’s list that one program-wide answer would commit.

Exercise 26.4 (judgement). Late in the flagship program a proposal halves the flagship DMA’s block-level constrained-random regression, on the grounds that the 4 KB rule is now a formal property and the prototype will exercise what remains. Decide which rows such a proposal may retire and which it may not, taking the scenario of §26.2 as the model for the argument. State what the chapter’s opening bug establishes about the inherited proof of the 4 KB rule, using the durability audit of §26.3 for the qualifier that fails. Give the pitfall of the chapter’s list that a halving commits.

Exercise 26.5 (artifact). The row below reaches a tape-out review for the flagship’s DMA. State what §26.4 permits the empty field to assert, and give the reason that section states for the limit. Identify the two entries that the practices of §26.4 mark as defects, and state which practice each of them fails.

Row:	flagship DMA descriptor engine, tape-out review
Functional position:	authored coverage model closed, every plan row discharged or waived
Safety position:	fault classification complete, each per-level metric threshold met
Security position:	four named claims discharged, two named gaps open
Combined position:	

Inherited rows:	4 KB boundary proof, carried forward as verified in the previous generation
Register of gaps:	not maintained for this row

Exercise 26.6 (judgement). Three statements circulate in the flagship program as reasons to keep a practice in place. The first holds that the cross closing the plan row for the 2D cross at the 4 KB boundary has 72 cells; the second holds that every practical oracle checks one projection of correctness; the third holds that formal capacity limits verification to small blocks. Decide for each statement whether the thousandfold test of §26.3 makes it durable or contingent, and give the reason §26.3 supplies. State what the chapter’s list of pitfalls says a rule found to be contingent becomes.

FURTHER READING

From the corpus. The two short 2024 Wilson Research Group trend reports, IC/ASIC [2] and FPGA [4], provide the field’s survey record and are the most recent of the three editions held; Foster’s DAC 2015 paper [3] supplies the decade-earlier baseline and the series methodology. The two editions before that pair, from 2020 and 2022, are IC/ASIC [10, 11] and FPGA [18, 19], and reading two editions together shows the movement a single edition can only assert. The requirement-layer framing and adoption curves merit attention alongside the headline percentages. Bormann’s dissertation [5] addresses whether a property set is sufficient and warrants study even without adopting the methodology. Mishra et al. [1] describes changes in bug cost and visibility as a project moves right. The 2026 agentic-EDA survey [20] is a published projection of where automation is going and should be read as its authors’ expectation rather than as a finding: it proposes an autonomy taxonomy modeled on the levels used for self-driving cars, offers a near-, mid- and long-term roadmap against them, notes that as of its writing no LLM-driven autonomous agent had demonstrated a complete human-free tape-out of an industrial chip, and argues that trust should move from the model that generates the artifact to a mechanically verifiable proof. In this book’s interpretation, that argument makes the property somebody writes more important. The Accellera clock- and reset-domain-crossing interface standard [16], published March 2026, illustrates standardization for portability of claims between teams.

Not in the corpus (general references only): Wile, Goss and Roesner, *Comprehensive Functional Verification* (Morgan Kaufmann, 2005), a full-lifecycle treatment; and the DVCon and DAC proceedings, where the discipline debates its work publicly and where the survey series is presented.

Across twenty-six chapters, verification has been treated as establishing qualified claims about designs.

Verification does not itself produce design correctness. Chapter 3 explains why the state space cannot be enumerated and every practical oracle checks a projection;

Chapter 14 shows how a purpose-built completeness methodology addresses them for selected modules, at a cost. Verification establishes true claims about a design, stating the method and assumptions as well as what was not established. Sign-off states those claims and assigns named responsibility for them.

The book's instruments define and support these claims. A plan states claims in advance, while coverage records that their scope was exercised. A scoreboard can disprove a claim, and an assertion continuously checks a clause of it. A waiver states a limitation, while a proof establishes a clause without exceptions inside a named model. Escape analysis corrects a claim found to have been too broad.

SOC-1284 illustrates a false verification claim. Two hundred and fourteen directed tests passed; the dashboard was green and the coverage report accurate about everything it measured. They supported the false claim that *the DMA's splitting of 2D transfers is verified*, which no project instrument could refute. Only eleven days of margin and one constrained-random seed separated discovery in the project from discovery by a customer.

Every review, plan, dashboard and green regression should identify the exact claim and the conditions that would falsify it, so the results can be assessed as evidence for that claim.

REFERENCES

1. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
2. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
3. **P17** H. D. Foster, "Trends in Functional Verification: A 2014 Industry Study," Proc. 52nd Design Automation Conference (DAC '15), San Francisco, CA, 2015.
4. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
5. **T2** J. Bormann, "Complete Functional Verification," Ph.D. dissertation, University of Kaiserslautern, Germany, 2009 (English translation of the German dissertation 'Vollständige funktionale Verifikation', April 2009).
6. **T1** M. Schwarz, "Formal Hardware/Firmware Co-Verification of Optimized Embedded Systems," Ph.D. dissertation (Dr.-Ing.), Technische Universität Kaiserslautern, Germany, 2021.
7. **P25** S. Ahmed, A. Kropotov, R. I. Genovese, B. Homs, E. Merino, F. Urbani, et al., "Verification and Validation (V&V)-in-the-Loop for RISC-V Design: The Holistic Vision of BZL," arXiv preprint arXiv:2604.27013v1, April 2026.
8. **S10** IEEE, "IEEE Std 1801-2024, IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems (Revision of IEEE Std 1801-2018)," IEEE, 2024.
9. **B5** A. Piziali, "Functional Verification Coverage Measurement and Analysis," Springer, 2004.
10. **R3** H. D. Foster, "2020 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2020.
11. **R5** H. D. Foster, "2022 Wilson Research Group IC/ASIC Functional Verification Trends," Siemens EDA white paper, 2022.
12. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.

13. **S9** IEEE, "IEEE Std 1800.2-2020, IEEE Standard for Universal Verification Methodology Language Reference Manual (Revision of IEEE Std 1800.2-2017)," IEEE, 2020.
14. **S1** Accellera Systems Initiative, "Portable Test and Stimulus Standard, Version 3.0," Accellera, August 2024.
15. **S3** Accellera Systems Initiative, "Unified Coverage Interoperability Standard (UCIS), Version 1.0," Accellera, June 2012.
16. **S12** Accellera Systems Initiative, "Standard for IP Abstraction for Clock and Reset Domain Crossing Integration, Version 1.0," Accellera, March 2026.
17. **D36** D. Mills, B. Gascoyne, C. Choppali Sudarshan, D. Olopade, and D. K. Gupta, "Breakthrough in CDC-RDC Verification Defining a Standard for Interoperable Abstract Model," Proc. DVCon U.S., 2026.
18. **R4** H. D. Foster, "2020 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2020.
19. **R6** H. D. Foster, "2022 Wilson Research Group FPGA Functional Verification Trends," Siemens EDA white paper, 2022.
20. **P23** Z. Zang, Y. Song, A. Wang, B. W.-K. Ling, Q. Sun, Z. Lei, F. Yang, C. Zhuo, and J. Luo, "The Dawn of Agentic EDA: A Survey of Autonomous Digital Chip Design," arXiv preprint arXiv: 2512.23189v2, February 2026.

Appendix A — Glossary

Terms appear broadly in the order the book introduces them, except that entries the book deliberately contrasts are kept side by side — the pairs where one word carries a different sense in design and in verification are the reason this glossary exists, and separating them would hide exactly what the reader came to check. The *Introduced* column names the chapter that owns the definition, which is not always the chapter of first mention: a term is often used in passing well before the chapter that pins it down.

Term	Definition (one line)	Introduced	Italian
(test, seed) matrix	What a random regression actually runs — tests as rows, seeds as columns, each cell a run with its own outcome.	ch12	matrice (test, seed)
2D row	One line of a two-dimensional DMA transfer: a contiguous run of bytes. Consecutive rows start one stride apart, and each row is split into bursts on its own.	ch01	riga di un trasferimento 2D
4 KB boundary	The address line every 4096 bytes that an AXI burst must not cross (Arm IHI 0022H.c §A3.4.1): every burst stays inside one 4 KB address region, so it never spans two subordinates.	ch01	confine di 4 KB
action (PSS)	Unit of behaviour: atomic when it maps to one operation of the system, compound when it encapsulates a flow of others.	ch09	action (unità di comportamento in PSS)
activation condition	The part of a flow rule saying when tracking starts, such as the cycle a key is loaded. Without it a rule can forbid the design's intended behavior.	ch23	condizione di attivazione
active vs passive agent	Active instantiation emulates a device and drives it; passive builds only the monitor and observes — the switch that carries a block environment into a system one.	ch10	agent attivo e agent passivo
activity (PSS)	The flow of sub-actions a compound action encapsulates, stating scheduling relations rather than a schedule.	ch09	activity (flusso di sotto-azioni in PSS)
ADC (analog-to-digital converter)	Block turning a continuous input into numbers; its behavioral model has to agree with the schematic inside a band fixed before any run.	ch21	ADC (convertitore analogico-digitale)
adversarial stimulus	Traffic whose purpose is to be refused or to drive a design somewhere unspecified. A cooperative environment never emits it, so a security campaign run on one checks nothing.	ch23	stimolo ostile
adversary capability	What an opponent is granted, stated as an enumerable list rather than as attacks, which form an open set. Every security property names one.	ch23	capacità dell'avversario

Term	Definition (one line)	Introduced	Italian
agent	The reusable unit of an environment: sequencer, driver and monitor for exactly one interface.	ch10	agent (unità riusabile per una sola interfaccia)
agentic formal flow	Pipeline in which one agent drafts a plan, others generate and criticize properties, an engine proves them and coverage is assessed; its published indicator counts finished runs.	ch25	flusso formale agentico
always-on cell	Cell powered from the permanent supply. A route that starts and ends in permanent logic must be built from these, and an ordinary buffer inserted later breaks it.	ch19	cella sempre alimentata
AMBA (advanced microcontroller bus architecture)	Published family of on-chip bus protocols; the interfaces of the example systems, among them AXI and APB, are members of it.	ch01	AMBA (famiglia di protocolli di bus on-chip)
AMS (analog and mixed-signal)	Said of a design or a flow in which analog blocks and digital logic have to be verified together instead of apart.	ch21	AMS (analogico e a segnali misti)
AMS co-simulation (analog and mixed-signal)	Coupling the event-driven engine to a time-domain analog solver so both advance in one simulation, restoring loading and transients at a runtime cost that forbids nightly use.	ch21	co-simulazione AMS (analogico-digitale)
analog solver	Engine that finds node voltages and branch currents by iterating on a matrix, choosing each timestep for accuracy. It can fail to converge; a real-valued model cannot.	ch21	risolutore analogico
analysis port	Broadcast, non-blocking publication of an observed transaction to any number of subscribers, none of which may block the producer or modify what it receives.	ch08	analysis port (porta di pubblicazione delle osservazioni)
APB (advanced peripheral bus)	The simple low-bandwidth member of the AMBA family, carrying configuration registers and slow peripherals.	ch01	APB (bus AMBA per le periferiche)
approximately timed model	Model that adds latency estimates, arbitration policies and bus delays: fast enough for architectural questions about ratios and trends, and unable to return a verdict.	ch18	modello a tempificazione approssimata
arbitration mode	Policy by which a sequencer chooses among pending requests; only the strict modes grant the highest-priority one first.	ch10	modalità di arbitraggio
artifact ownership	Answering for an environment, a model or a plan rather than for a claim about the design. Discharged by the artifact being good, it leaves everyone blameless after an escape.	ch24	titolarità di un artefatto
ASIC (application-specific integrated circuit)	Chip fabricated for one purpose, whose masks cost enough that a functional flaw found afterwards forces another fabrication cycle.	ch01	ASIC (circuito integrato dedicato)

Term	Definition (one line)	Introduced	Italian
ASIL	Automotive safety integrity level, A to D with D most demanding. Assigned from a hazard's severity, exposure and controllability — a property of the hazard, never of your block.	ch22	ASIL (livello di integrità della sicurezza automobilistica)
ASIL decomposition	Assigning different integrity targets to regions of one design; the boundary is itself an obligation, since a fault in the lower region must not defeat the higher one's mechanisms.	ch22	decomposizione ASIL
assertion	A declarative statement of a property the design must hold, evaluated by a tool, whose falsehood indicates an error.	ch11	assertion (asserzione)
asset (security sense)	Something worth protecting, named before any property is written: a key, an entropy source, a boot measurement, or the configuration enforcing every other protection.	ch23	asset (bene da proteggere)
assumption (assume)	A property claimed of the environment rather than of the design: checked like an assertion in simulation, never checked in formal, where it deletes traces instead.	ch11	assunzione (assume)
assumption register	Reviewed table listing every assumption with an identifier, its wording, the party accountable for the behavior it names, and the evidence that discharges it.	ch14	registro delle assunzioni
asynchronous FIFO	Streaming crossing whose gray-coded pointers travel each way while the payload sits in dual-port memory, never entering a synchronizer.	ch13	FIFO asincrona
attribute	A parameter or dimension of a behavior whose value set defines its verifiable space.	ch05	attributo
autonomous monitor	Monitor whose observing thread starts at construction and runs continuously, so a late testbench never back-pressures the design.	ch08	monitor autonomo
AXI (advanced extensible interface)	The high-throughput AMBA interface: separate address, data and response channels, responses allowed out of order, identifiers separating concurrent streams.	ch01	AXI (interfaccia AMBA a canali indipendenti)
base failure rate	Rate carried by each failure mode of each element, computed from design and technology data. Architectural metrics combine these rates, so a detected fraction weights them.	ch22	tasso di guasto di base
batching	Covering many design cycles per host interaction so the link is crossed once per transaction; the speedup ceiling that follows is fixed by the interaction's own cost.	ch17	batching (raggruppamento delle transazioni)
beat	One data transfer within an AXI burst; a burst of 256 beats moves 256 data words.	ch01	beat (trasferimento elementare di un burst)

Term	Definition (one line)	Introduced	Italian
bind	Construct instantiating a checker into a design scope without modifying the design's source.	ch11	bind
binding constraint	The activity a doubled budget would be spent on. For verification it has migrated over decades toward deciding what to check, and it differs per level rather than globally.	ch26	vincolo dominante
bins	Counters over value sets — executable equivalence classes.	ch06	bins (contenitori di conteggio)
blended percentage	Single coverage figure mixing simulation hits with proof results, numerically valid but with no defined denominator, so a review cannot say what fraction it names.	ch16	percentuale mista
block verifier	The person answering for one block's features from extraction through sign-off, and the side of the block-to-system seam where a design's internals are actually understood.	ch24	verificatore di blocco
bound checker	Module carrying its own model of an interface — counters, queues — bound to a port when the rules outgrow what a single property can state.	ch11	bound checker (modulo legato con bind)
boundary cover	Cover on the condition an assumption restricts; when it reads zero, the assumption has never been approached and its simulation corroboration is empty.	ch16	boundary cover (cover sul limite dell'assunzione)
bounded model checking	Proof method that copies the combinational logic once per cycle of a fixed interval and asks a satisfiability solver for a violation inside it.	ch14	bounded model checking (verifica limitata di modello)
bounded proof	Result stating only that no violation exists within a stated number of cycles; a guarantee about the near future, not a proof.	ch14	prova limitata
bring-up	The phase in which environment and DUT first exchange checked transactions.	ch04	bring-up (avvio dell'ambiente sul DUT)
bug curve	Bug discoveries (or open/closed counts) plotted over time; the discipline's main convergence instrument.	ch07	curva dei bug
bug escape	Design flaw surviving verification into a later stage.	ch01	bug sfuggito alla verifica
bug footprint	Region of the coverage space a bug occupies.	ch06	impronta del bug
burst splitter	The DMA stage between the register frontend and the AXI backend that splits each transfer into bursts obeying the AXI address rules, the 4 KB boundary above all. Formerly called the midend legalizer; code identifiers keep the old name.	ch01	divisore di burst

Term	Definition (one line)	Introduced	Italian
campaign	Purpose-built run on no clock: launched for a stated reason, owned by someone, producing a written result, then stopping.	ch12	campagna
canonical bug	The book's recurring defect for one block, returned to whenever that block is the example. The DMA's is the main instance: the burst splitter mis-splits a negative-stride 2D row that crosses the 4 KB boundary, and only while the other DMA channel is active.	ch01	bug canonico
capability tier	One level of a graded adversary list, from unprivileged software up to invasive physical access. Exposing a physical mechanism to software moves a claim between tiers.	ch23	livello di capacità
CDC (clock-domain crossing)	Signal whose producing and consuming flip-flops run on clocks with no fixed phase relationship, so the sampling instant is unpredictable.	ch13	CDC (attraversamento di dominio di clock)
CDG	Coverage-directed test generation via feedback from coverage results to stimulus.	ch06	CDG (generazione dei test guidata dalla copertura)
champion	A named local expert for a technique inside one team: first point of contact, with an escalation path to central expertise and continuity across projects. Borrowed from the adoption-programme literature.	ch24	champion (referente locale della tecnica)
checking contract	The written list of verdicts an environment will produce, each with a named owner — and the list of verdicts it will not.	ch08	contratto di verifica (elenco dei verdetti dovuti)
CI (continuous integration)	Practice of merging changes often and running an automated tier of checks on each merge, so a break surfaces in minutes.	ch12	CI (integrazione continua)
clock domain	Every flip-flop taking its edge from one clock source, or from clocks derived from it; three clock sources need not give three domains.	ch13	dominio di clock
clock snooping	Taking a clock from inside the design instead of generating one in the environment. It yields fewer failing tests, a faster bring-up, and hidden clocking bugs.	ch24	clock snooping (clock prelevato dal design)
clocking block	Declaration fixing the moment interface signals are sampled or driven, so the testbench cannot race the design it watches.	ch08	clocking block
closure metric	The evidence a plan row names as what closes it: a covergroup at 100 %, a proven property, a passed test, a signed review.	ch05	metrica di chiusura
coarse-grained checking	What an abstract model of a hardware and software interface supports: the basic function is kept, everything finer is discarded, so only use cases at that granularity are checkable.	ch18	controllo a grana grossa

Term	Definition (one line)	Introduced	Italian
common timestamp source	Free-running counter software can read and monitors can sample, finer than the events to be ordered; without one, separate records of a failure cannot be merged.	ch18	sorgente comune di marca temporale
common-mode error	The same specification misreading encoded in both design and oracle, making a bug structurally invisible.	ch03	errore di modo comune
concurrent assertion	Clocked, temporal assertion evaluated at clock ticks over sampled values, checked on every run that executes it.	ch11	assertion concorrente
condition set	The supply, temperature, corner, frequency and load under which a stated band holds. A tolerance quoted without them is not a specification.	ch21	insieme delle condizioni
conditional sign-off	Sign-off taken now but contingent on named evidence arriving by a named date.	ch07	sign-off condizionato
cone of influence (COI)	The netlist region that can reach an observation point. A fault outside it cannot reach one under any workload whatsoever.	ch22	cono di influenza (COI)
confidentiality (information-flow sense)	Violated when an unauthorized reader can obtain protected state. The claim quantifies over pairs of executions, which is why one sampled execution cannot settle it.	ch23	riservatezza
configuration database	Store of typed values set against hierarchical path patterns, letting an integrator configure an environment without knowing its implementation.	ch10	database di configurazione
connect module	Element a simulator inserts automatically where a continuous port meets a discrete one, chosen from the disciplines and the directions involved.	ch21	connect module (modulo di conversione)
connectivity checking	Proof that every row of an integration table is implemented: named source to named destination, at the declared latency, under the declared enabling condition.	ch15	verifica di connettività
constrained-random	Automatic generation of legal-but-unlikely stimulus under constraints.	ch01	constrained-random (stimolo casuale vincolato)
constraint solver	The engine of a constrained-random generator that finds values satisfying every active constraint; not the analog solver of Chapter 21.	ch09	risolutore di vincoli
controllability	The ability to steer a design into a given internal condition using only drivable interfaces.	ch03	controllabilità
convolution job	The neural accelerator's unit of work: one convolution, programmed by software through the accelerator's registers (operand addresses, weight precision, tensor size) and run to completion.	ch01	job di convoluzione

Term	Definition (one line)	Introduced	Italian
corner (engine-selection sense)	Several individually rare conditions holding at once; simulation cost follows the product of their probabilities, while a proof engine is indifferent to that rarity.	ch16	corner (congiunzione di condizioni rare)
correlated errors	One misreading of one recurring pattern, reproduced identically across a generated batch. Sampling estimates nothing about them, so a batch is reviewed whole or rejected whole.	ch25	errori correlati
correlation expiry trigger	Named event that voids a correlation result: a schematic or specification revision, a new operating condition, a retuned parameter, or a model edited during debug.	ch21	evento di scadenza della correlazione
correlation tolerance	The per-port band, part of the specification and declared before any run, within which a model and the schematic must agree.	ch21	tolleranza di correlazione
corruption (power-aware sense)	Simulator behavior when a domain loses its supply: every node inside goes unknown until power returns, which turns an unprotected crossing into a visible failure.	ch19	corruzione (dei nodi di un dominio spento)
counterexample (CEX)	Input sequence a proof engine returns to exhibit a failure, carrying no history the failure does not need, unlike a regression trace.	ch14	controesempio (CEX)
counterparty rule	An assumption naming another team is not closed until someone from that team has read it in review and named the row that discharges it.	ch24	regola della controparte
cover property	Directive asking whether a scenario occurred: a count of matches in simulation, a reachability question answered with a witness in formal.	ch11	cover property
coverage closure	Converging every plan item to covered-or-consciously-excluded status.	ch04	chiusura della copertura
coverage differential	Bin-level comparison of two regressions, used to fail a commit that quietly made the suite ask less.	ch12	differenziale di copertura
coverage hole	Required stimulus/behavior not yet observed.	ch06	buco di copertura
coverage model	Multi-dimensional region defined by attributes and their values.	ch06	modello di copertura
covergroup	SystemVerilog container encapsulating a coverage model specification.	ch06	covergroup
coverpoint	One observed variable/expression partitioned into bins.	ch06	coverpoint
cross coverage	Cartesian combinations of coverpoints.	ch06	cross coverage (copertura incrociata)

Term	Definition (one line)	Introduced	Italian
cutpoint	Signal cut from its fan-in so it may take any value; an over-approximation that keeps proofs valid and can invent failures.	ch14	cutpoint (segnale liberato)
dangerous-undetected fault	Fault that corrupted an output while the mechanism stayed silent: the class a campaign exists to find, and the one that must be fixed or argued away.	ch22	guasto pericoloso non rilevato
data coherency (crossing sense)	The requirement that several bits crossing together be observed as a value the source actually drove; separate from resolving metastability on each bit.	ch13	coerenza dei dati (nell'attraversamento)
data tagging	Encoding the expected destination and transformation inside the payload, so each output monitor can decide correctness on its own.	ch08	marcatura dei dati
datapath equivalence	Word-level proof that an arithmetic block returns the same result as a bit-exact algorithmic reference for every operand, under a declared operand and result mapping.	ch15	equivalenza del datapath
deadlooping	Two automated stages oscillating between states without converging. A cap on iterations is what turns that into a hand-off to a person.	ch25	deadlooping (oscillazione senza convergenza)
debug-time share	Fraction of engineering time spent in debug, tracked as a project health metric.	ch07	quota di tempo speso in debug
decay (of a directed test)	A test that keeps passing while no longer exercising what it was written for, because the design grew past it.	ch09	decadimento (di un test diretto)
defeaturing	Reducing a part to a minimal bootable configuration without power management, security or complex firmware, then restoring features one at a time once it powers on reliably.	ch20	defeaturing (riduzione a configurazione minima)
definition of done	Per-deliverable exit criterion stated as an action ("passes in regression"), never a percentage.	ch04	definition of done (criterio di completamento)
delayed-lockstep	Redundant pair in which the second copy runs some cycles behind, so one disturbance reaches the two at different points and produces a mismatch instead of matching wrong answers.	ch22	lockstep sfasato
dependent failure analysis	Analysis of what a single physical event could do to both halves of a redundant pair at once.	ch22	analisi dei guasti dipendenti
design-for-debug (DfD)	On-die instrumentation for post-silicon observability.	ch01	design-for-debug (progettazione orientata al debug)
detection distance	The time and state between a mistake and the check that catches it; what an end-to-end oracle pays and an assertion does not.	ch08	distanza di rilevamento

Term	Definition (one line)	Introduced	Italian
detection point	Where the safety mechanism raises its alarm. A fault seen here is <i>detected</i> — which is not the same as observed.	ch22	punto di rilevamento
diagnostic coverage	The fraction of a failure mode’s faults a mechanism is claimed to catch — estimated in the FMEDA, measured by fault injection.	ch22	copertura diagnostica
directed test	Stimulus whose content and order are written out by hand, doing the same thing on every run.	ch09	test diretto
disable iff	Reset guard discarding any attempt in flight while its expression is true, ending it as neither pass nor failure; at most one per assertion, guarding the whole property.	ch11	disable iff
disposition	The recorded decision on an open item (fix / waive-as-errata / defer with justification).	ch07	disposizione (decisione registrata sul punto aperto)
distribution constraint (dist)	Weights over values and ranges that move probability mass without changing the legal set — and silently forbid whatever the list omits.	ch09	vincolo di distribuzione (dist)
DMA (direct memory access)	Engine moving data between memories on its own, without the processor copying it; the recurring example block of the book.	ch01	DMA (accesso diretto alla memoria)
DMA channel	One of the two independent transfer engines of the DMA, each with its own descriptor and state, both sharing the AXI backend. Not an AXI channel (write address, write data, response).	ch01	canale DMA
double assignment	Deliberately giving one feature two plan rows with two metrics, one closed by proof and one by coverage, because the engines fail differently.	ch16	doppia assegnazione
DPI (direct programming interface)	The SystemVerilog mechanism for calling C from the language and being called back, over a restricted set of argument types.	ch17	DPI (interfaccia di programmazione diretta)
drain time	Grace period after the last objection drops, so transactions still in flight reach the checkers.	ch10	drain time (tempo di svuotamento)
DSP (digital signal processor)	Processor specialized for signal arithmetic; in the example systems, a block clock gated off between bursts of work.	ch13	DSP (processore per segnali digitali)
durability audit	Reading a plan row twice: once for what would still be true if every engine improved a thousandfold, and once for what today’s tooling contributed to it.	ch26	audit di durabilità
durable property	A statement about the verification problem rather than about a tool, such as exponential state growth or the partiality of every practical oracle. Better engines do not retire it.	ch26	proprietà durevole

Term	Definition (one line)	Introduced	Italian
DUT (design under test)	The design being verified by the testbench.	ch02	DUT (progetto in verifica)
DVFS (dynamic voltage and frequency scaling)	Trading supply voltage and clock frequency at run time to save energy, which multiplies the operating points a plan has to cover.	ch19	DVFS (regolazione dinamica di tensione e frequenza)
ECC (error-correcting code)	Redundant bits kept with the data so a corrupted word can be spotted and, within the strength of the code, repaired.	ch22	ECC (codice a correzione d'errore)
EDA (electronic design automation)	The tool industry supplying simulators, proof engines and coverage databases; its license count is often what caps a regression.	ch08	EDA (automazione della progettazione elettronica)
emulation	Compiling a design onto a hardware execution fabric, synthesized and partitioned, where it runs under stimulus a host supplies; a model runs, so the work stays pre-silicon.	ch17	emulazione
emulator	Purpose-built machine with its own compiler, transactors and debug infrastructure, offering roughly a billion gates of capacity and observability through triggers and state saving.	ch17	emulatore
engine selection	Deciding per plan row rather than per block which engine discharges it, weighing quantification, rarity against volume, the cost of a witness, and observability.	ch16	scelta del motore di verifica
environment vs directive constraint	Constraints stating what the interface makes legal, which must be obeyed, versus constraints layered above them to steer a run toward chosen scenarios.	ch09	vincolo d'ambiente e vincolo direttivo
equivalence class	A set of input values assumed to exercise identical design behavior, used to shrink the test space.	ch03	classe di equivalenza
errata sheet	Published catalog of post-silicon bugs with workarounds.	ch01	errata (elenco pubblicato dei bug del silicio)
error detection latency	Cycles separating a fault's activation from the point where it becomes observable. For hard bugs the figure reaches millions, against trace windows a few thousand cycles deep.	ch20	latenza di rilevamento dell'errore
error injection	Deliberately driving erroneous stimulus to verify detection/recovery behavior.	ch02	iniezione di errori
errors re-found	Count of failures that re-discover an already-known open bug — the measure of how much of a night was spent learning nothing.	ch12	errori ritrovati
escape	A bug that crosses a sign-off boundary undetected — found on the wrong side of the boundary that was supposed to catch it. Ordinary-English uses of “escape” (“the obvious escape”, “escape routes”) collide with this term of art and must be reworded.	ch01	escape (bug che oltrepassa un confine di sign-off)

Term	Definition (one line)	Introduced	Italian
escape analysis	Blameless structured post-mortem of an escape that feeds changes into the next plan and checklist.	ch07	analisi degli escape
evaluation attempt	One evaluation of a property, started afresh at every tick of its clock and running to its own verdict, overlapping the others in flight.	ch11	tentativo di valutazione
exception clause	The part of a flow rule permitting one flow, such as a cipher's output. A rule is only as strong as the clause is narrow, so each needs an expiry.	ch23	clausola di eccezione
executable verification plan	Machine-readable plan whose items link to the coverage/checks/results that discharge them.	ch04	piano di verifica eseguibile
factory	Indirection through which components and objects are constructed, so a type can be replaced without editing the code that instantiates it.	ch10	factory (indirezione per la costruzione dei componenti)
failure signature	Deduplication key built from the first error with everything run-to-run stripped out; over-generic it swallows real finds, over-specific it reduces nothing.	ch12	firma del fallimento
fault injection	Safety sense: forcing a fault into internal design state to see whether a safety mechanism detects it. NOT ch02's <i>error injection</i> , which perturbs stimulus at the interface. The Italian must keep the two apart.	ch22	iniezione di guasti
fault reaction time interval	The budgeted time a mechanism has to act once it has detected, paired with the interval it has to detect in. Late detection counts as undetected.	ch22	intervallo di tempo di reazione al guasto
fault-tolerant time interval	The budgeted interval, measured from the moment a fault occurs, within which a mechanism must detect and react. Paired with the <i>fault reaction time interval</i> .	ch22	intervallo di tolleranza al guasto
feature extraction	Enumerating from spec, interviews and architecture everything that must be shown to work.	ch05	estrazione delle funzionalità
fidelity	Degree to which a coverage model captures actual behavioral requirements.	ch06	fedeltà (del modello di copertura)
field access policy	Declared behaviour of a register field on read and write (read-only, write-one-to-clear, and the rest), from which the model predicts the mirror.	ch10	politica di accesso del campo
FIFO (first-in first-out)	Queue returning entries in the order they arrived; its depth and its fill level are ordinary sources of verification state.	ch03	FIFO (coda a ordine di ingresso)

Term	Definition (one line)	Introduced	Italian
firmware-driven verification	Using the shipping boot sequence and driver as stimulus: deep along the single trajectory software takes, and silent about legal traffic no driver has reason to emit.	ch18	verifica guidata dal firmware
first-silicon success	First fabricated silicon is production-worthy.	ch01	successo al primo silicio
first-time success	The plan-defined set of features that must work in first silicon.	ch04	successo al primo colpo
first/second/third-order reuse	One environment across many testcases; its components in a system-level environment; those components in a different environment for a different design.	ch10	riuso di primo, secondo e terzo ordine
flaky test (flake)	Test whose outcome varies between runs with identical manifests; it has stopped being evidence, and it teaches the team to ignore red.	ch12	test instabile (flaky)
flow qualification	Evidence that the verification <i>environment</i> can fail: systematic faults are injected and what the environment detects is measured.	ch22	qualifica del flusso
FMEDA	Failure mode, effects and diagnostic analysis: element-by-element analysis asking how each element can fail, with what consequence and likelihood, and which safety mechanism handles it.	ch22	FMEDA (analisi dei modi di guasto, degli effetti e della diagnosi)
formal app	Packaged check whose properties the tool supplies, so a team provides only the design and a configuration; the intent lives in a machine-readable source.	ch15	formal app (applicazione formale)
formal property verification	Deciding by mathematical reasoning whether a design can violate a stated property, rather than sampling behavior with stimulus.	ch14	verifica formale di proprietà
FPGA (field-programmable gate array)	Device configured after manufacture, used to run a design orders of magnitude faster than simulation and with far less visibility.	ch01	FPGA (dispositivo logico programmabile)
FPGA prototype	A design's logic mapped onto commodity reconfigurable devices, running at megahertz with real peripherals attached: pre-silicon in time, and a check on hardware in kind.	ch18	prototipo FPGA
fresh-seed yield	Failures per unit of newly-seeded stimulus; discriminates a clean flat bug curve from a saturated one.	ch07	resa dei seed nuovi
front-door vs back-door access	Access driving real bus cycles over the real path, versus one reaching the simulation constructs directly by hierarchical path.	ch10	accesso front-door e accesso back-door
FSM (finite state machine)	Logic written as a set of states plus the transitions among them, the shape that state and transition coverage aim at.	ch06	FSM (macchina a stati finiti)

Term	Definition (one line)	Introduced	Italian
functional verification	Pre-fabrication establishment that a design implements its specification.	ch01	verifica funzionale
generator/monitor duality	One constraint used three ways: to generate legal traffic, to check a neighbour's output, and to assume in formal.	ch09	dualità generatore/monitor
GLS (gate-level simulation)	Running a testbench on the synthesized netlist. It exhibits phenomena nothing else produces and cannot sign off their absence, since it samples what a test happens to sensitize.	ch19	GLS (simulazione a livello di porte)
golden model	Model trusted by construction because it is the specification, stated at the specification's own level of abstraction; golden is not the same as complete.	ch08	golden model (modello preso come specifica)
good machine / faulty machine	The unmodified design and a copy carrying one hypothetical injected fault, compared at designated strobe times. ch22 deliberately writes "good", never "golden" — do not normalise it to ch08's bound <i>golden model</i> sense.	ch22	macchina buona / macchina guasta
gray-coded parallel synchronizer	One two-flop chain per bit of a bus, safe only when a single bit changes per transfer so any unresolved bit lands on a legal code.	ch13	sincronizzatore parallelo con codifica Gray
grey-box verification	Black-box verification augmented with design hooks for controllability/observability.	ch03	verifica grey-box
handshake synchronizer	Crossing that synchronizes only request and acknowledge while the source holds the data steady until the acknowledge returns, at the price of round-trip latency.	ch13	sincronizzatore a handshake
hardware redundancy	Replicated hardware whose outputs are compared or voted. NOT ch02's <i>redundancy</i> , which is two independent readings of a specification. Same word, unrelated mechanisms.	ch22	ridondanza hardware
hardware-assisted verification	Running a design on hardware instead of in an event-driven simulator, buying throughput and capacity while surrendering visibility, build turnaround and debug ergonomics.	ch17	verifica assistita da hardware
heterogeneous merge	Combining coverage from different verification processes; the interchange standard warns that such items are rarely equivalent and requires the result to be tagged as derived.	ch16	merge eterogeneo
hybrid co-emulation	Split in which host models run the mature parts of a system while selected register-transfer logic runs on the machine, joined by transaction-level channels.	ch17	co-emulazione ibrida

Term	Definition (one line)	Introduced	Italian
hybrid co-simulation	Partition running components outside the verification target as fast host models and the target itself as register-transfer logic, joined where the architecture already has transaction semantics.	ch18	co-simulazione ibrida
hybrid flow	One campaign running proof and simulation against a single plan, with work assigned by problem structure and one shared definition of legal traffic.	ch16	flusso ibrido
hypothesis quality	Score a language model awards for how closely a proposed cause resembles a known one. It does not order configurations the way an applied fix and a rerun do.	ch25	qualità dell'ipotesi
ICE (in-circuit emulation)	Use model in which real peripherals, hosts or instruments drive the emulated design, at the cost of replay, unattended running and state saving.	ch17	ICE (emulazione in circuito)
IEC 61508 (functional safety of programmable systems)	General industrial safety standard from which several sector standards descend; it fixes integrity levels and the evidence each level demands.	ch22	IEC 61508 (norma generale di sicurezza funzionale)
IEEE 1666 (SystemC standard)	Standard behind SystemC and its transaction-level interfaces, which supply an interoperability layer for models written above the level of signals.	ch08	IEEE 1666 (norma del linguaggio SystemC)
IEEE 1800 (SystemVerilog standard)	Language standard holding design constructs, testbench classes, constrained randomization, functional coverage and assertions in one document.	ch06	IEEE 1800 (norma del linguaggio SystemVerilog)
IEEE 1801 (power intent standard)	Standard notation for stating power intent apart from the design: domains, supply states, isolation, retention, and the legal combinations.	ch19	IEEE 1801 (norma per l'intento di potenza)
ignore_bins vs illegal_bins	“Not my job” exclusion vs “must never happen” (runtime error).	ch06	ignore_bins e illegal_bins (esclusione contro divieto)
image (emulation build)	One compiled build of a design on the platform; it is the unit that gets scheduled, so plan rows are grouped by it rather than by priority.	ch17	immagine (build compilata sulla piattaforma)
immediate assertion	Procedural assertion testing a non-temporal expression when control flow reaches it, where x or z counts as failure.	ch11	assertion immediata
implication (overlapping, non-overlapping)	Property operator making an obligation conditional, starting the consequent in the same tick or the next; an attempt whose antecedent is false succeeds having checked nothing.	ch11	implicazione (sovrapposta e non sovrapposta)

Term	Definition (one line)	Introduced	Italian
implicit vs explicit prediction	Updating the mirror only from accesses the model itself issued, versus updating it from a predictor fed by the bus monitor, which sees every access.	ch10	predizione implicita ed esplicita
in-fabric monitor	Check reimplemented as logic that latches its verdict into a register software can read, because the platform offers no assertion engine and no waveform to inspect.	ch18	monitor nel fabric (realizzato in logica)
induction	Route to an unbounded result: prove a predicate holds after reset and is preserved by every interval, then it holds for all time.	ch14	induzione
inference (PSS)	The tool completing a partial statement of intent with the actions it requires — and nothing it does not.	ch09	inferenza (completamento automatico dell'intento in PSS)
information-flow tracking	Labeling data as secret or untrusted and following where the label goes, reporting when a protected value reaches an observable place or an untrusted input steers control.	ch23	tracciamento del flusso di informazione
instruction-accurate model	Core model that executes the target instruction set and updates architectural state correctly while promising nothing about how many cycles any operation takes.	ch18	modello accurato all'istruzione
instrumentation manifest	Per-build record of which probes and monitors a bitstream actually contains, so a later investigation knows what remains connected from the previous one.	ch18	manifesto della strumentazione
integration verifier	The person answering for what only appears after assembly, to whom each delivered block arrives as a black box that cannot be learned in the time available.	ch24	verificatore di integrazione
intent curation	Creating and maintaining the machine-readable source an app reads; the real cost of a flow whose properties are otherwise free.	ch15	intent curation (cura della fonte di intento)
intent gate	The first human decision on a generated artifact: does it mean what the specification means. Its owner owns the plan row, and a passing proof is not evidence here.	ch25	gate di intento
IP (intellectual property)	Design block licensed or reused rather than written for this project; how much of a chip is reused governs its verification cost.	ch01	IP (blocco di proprietà intellettuale)
irritator	Background traffic generator run concurrently with the main test, whose only job is to perturb the DUT. Two senses: the testbench component (ch07) and the on-die version running on real silicon (ch20). Both translate the same way, but the Italian text must not imply a testbench when the subject is a fabricated part.	ch07, ch20	irritator (generatore di traffico di disturbo)

Term	Definition (one line)	Introduced	Italian
ISO 26262 (road vehicle functional safety)	Automotive safety standard fixing integrity levels, architectural metrics, and the assessments a hardware element has to undergo.	ch22	ISO 26262 (norma di sicurezza funzionale per autoveicoli)
isolation	Defined behaviour for a signal whose driving logic is not active.	ch19	isolamento
isolation cell	The cell implementing isolation: it passes values normally and clamps its output to a specified value when its control asserts. Its own cell stays powered — that is what lets it clamp.	ch19	cella di isolamento
ISS (instruction-set simulator)	Fast software model of a processor architecture, run beside the design so architectural state can be compared instruction by instruction.	ch01	ISS (simulatore del set di istruzioni)
JTAG (joint test action group)	The standard serial test and debug port; in the lab, the narrow window through which a fabricated part is driven and watched.	ch18	JTAG (porta standard di test e debug)
known-bad variant	Deliberately broken copy of the design kept in the repository and run regularly, to prove the environment can still fail.	ch08	variante deliberatamente difettosa
latent fault	A fault that cannot violate the safety goal alone and is neither detected nor perceived; it waits for a second fault.	ch22	guasto latente
LEC (logic equivalence checking)	Combinational equivalence between two closely related models, typically RTL and the netlist synthesized from it, proven cone by cone after state points are paired.	ch15	LEC (verifica di equivalenza logica)
leftover	An expectation still queued at end of test, never answered — the failure that produces no mismatch, only a queue that never empties.	ch08	residuo (attesa mai soddisfatta a fine test)
lemma	Auxiliary fact about internal state, proven on its own and then handed to a harder proof as an assumption needing no further discharge.	ch14	lemma
level-shifter	A cell translating a signal from one voltage swing to another.	ch19	level-shifter (cella di traslazione di livello)
lint	Structural check on the design description that reports questionable constructs a compiler accepts, such as inferred latches, width mismatches and unconnected ports.	ch13	lint
liveness property	A claim that something must eventually happen: it can never fail in simulation, where the attempt merely stays open, but a proof engine decides it.	ch11	proprietà di liveness (di vivacità)
localization	Naming the activating input sequence and the block holding the bug. Detection is cheap; this step dominates lab cost and can take weeks for one difficult bug.	ch20	localizzazione

Term	Definition (one line)	Introduced	Italian
lockstep	Safety sense: a pair of hardware cores executing identically with a comparator between them. NOT ch03's sense of stepping an instruction-set simulator against RTL.	ch22	lockstep (coppia di core in passo con comparatore)
LSB (least significant bit)	Lowest-weight bit of a word, and in analog work the unit in which the error budget of a converter is written.	ch21	LSB (bit meno significativo)
MAC (media access control / multiply-accumulate)	Two unrelated senses: the block terminating an Ethernet link, and the multiply-and-add lane of an arithmetic datapath. Context decides which.	ch01	MAC (controllo di accesso al mezzo / moltiplicazione-accumulo)
maturity stage	One of a small set of named, checklist-audited levels of verification maturity shared across blocks.	ch04	stadio di maturità
mechanical gate	Check a machine settles alone on a generated artifact: it compiles, its signals resolve, its antecedent is reachable, it is not a duplicate. All precede any human reading.	ch25	gate meccanico
metastability	The unstable condition of a flip-flop sampled too near a change, which resolves to a legal value only after an unpredictable delay.	ch13	metastabilità
metastability injection	Simulation instrumentation that randomly advances, delays or passes each synchronizer sample, so receiving logic is tested against all three legitimate arrival times.	ch13	iniezione di metastabilità
methodology owner	The person owning the base environment, the register generator flow, the regression infrastructure and the coverage merge on behalf of the other verifiers.	ch24	titolare della metodologia
metrics-driven verification (MDV)	Managing verification on quantitative measurements collected continuously from the process rather than on status estimates from people.	ch07	verifica guidata dalle metriche (MDV)
middle of the flow	Block-level work on register-transfer logic, the one stage with full visibility and a real design. Earlier claims and later debug both press on it; neither replaces it.	ch26	centro del flusso
milestone	A phase-gating event defined as a checkable demonstration, not a state estimate.	ch04	milestone (traguardo dimostrabile)
mined property	Property extracted from the design's own behavior, true by construction, so it detects later changes but finds no bug in the code it came from.	ch15	proprietà estratta dall'RTL
mirror vs desired value	What the model believes the design currently holds, versus a value set in the model alone and pushed to the design later.	ch10	valore del mirror e valore desiderato

Term	Definition (one line)	Introduced	Italian
model checking	Building the design’s finite transition system, computing the states reachable from reset, and deciding a property against that set with a trace when it fails.	ch14	model checking (verifica di modello)
model correlation	Checking a behavioural model against the schematic it abstracts: one self-checking testbench run against both, compared per port against tolerances declared in advance. Two words deliberately — bare “correlation” appears across ten chapters in the ordinary English sense.	ch21	correlazione del modello
model register	The table of every behavioural model a project regresses against: abstraction, circuit revision correlated against, tolerances, conditions covered, date, owner, and what breaks if it is wrong. Sibling of ch14’s assumption register.	ch21	registro dei modelli
modport	Restricted view of an interface; an inputs-only clocking modport makes a monitor’s passivity a compile-time property rather than a review item.	ch08	modport
MTBF (mean time between failures)	Expected interval between synchronizer failures; its exponent is the available settling window over the flop’s time constant, so small window losses cost disproportionately.	ch13	MTBF (tempo medio tra i guasti)
multi-point fault	A combination of faults that together violate the safety goal, classified by whether the mechanism detects it, merely perceives it, or misses it.	ch22	guasto multiplo
multi-stream (virtual) generator	One randomized object holding a sub-descriptor per stream, because constraints cannot be expressed across separate generators.	ch09	generatore multi-flusso (virtuale)
must-not-happen feature	Negative requirement enumerating an error the environment must be able to detect.	ch05	funzionalità in negativo (ciò che non deve accadere)
mutation-based generation	Producing hostile input by perturbing well-formed input. It suits a structured stream that stays almost legal, and not an interface that rejects malformed traffic outright.	ch23	generazione per mutazione
no-connection check	Structural test that no path exists in the cone between two points, discharging a forbidden-connection row more cheaply than a proof engine would.	ch15	verifica di assenza di collegamento
NoC (network-on-chip)	On-chip packet network taking the place of a bus once the number of communicating blocks makes one shared path untenable.	ch01	NoC (rete su chip)
non-convergence fallback	The plan’s stated answer, agreed before the work starts, to what happens when a proof does not converge; usually the row moves to simulation.	ch16	piano di ripiego per la non convergenza

Term	Definition (one line)	Introduced	Italian
notifier	The mechanism that makes a timing violation <i>functional</i> rather than merely printed: a timing check drives a notifier signal, which the cell model uses to corrupt its output.	ch19	notifier (segnale che rende funzionale una violazione di timing)
NRE	Non-recurring engineering cost.	ch01	NRE (costo di ingegneria non ricorrente)
objection	A raised claim that an activity must finish before its phase may end; a task phase lasts only while at least one is raised.	ch10	objection (impegno che tiene aperta la fase)
observability	How much internal design state is visible during debug.	ch01	osservabilità
observation point	Where a fault becomes visible to the outside world. A fault seen here is <i>observed</i> . Distinct from ch01's <i>observability</i> , which is the general property.	ch22	punto di osservazione
one-row-one-metric rule	Each vplan row carries exactly one measurable closure criterion; two evidences = two rows.	ch05	regola una riga una metrica
operational vs observational communication	Point-to-point blocking hand-off, where back-pressure is meaningful, versus broadcast non-blocking publication, where it must never be.	ch08	comunicazione operativa e comunicazione osservativa
oracle	The mechanism that decides whether an observed response is correct.	ch03	oracolo
orphan response	A response from the design that matches no outstanding expectation.	ch08	risposta orfana
over-approximation	Reduction that adds behaviors: a proof on the reduced model still holds on the original, while a failure may be impossible in the real design.	ch14	sovra-approssimazione
over-constraining	Narrowing stimulus below what the design must accept; the quiet form stays satisfiable and silently deletes the feature under test.	ch09	over-constraining (eccesso di vincoli)
own an outcome	Accountability stated as a claim about the design rather than about an artifact. Contrast <i>artifact ownership</i> , and note it is not a <i>definition of done</i> , which is an exit criterion rather than a unit of accountability.	ch24	farsi carico di un esito
partial oracle	A checker that verifies one projection of correctness rather than correctness itself.	ch03	oracolo parziale
path coverage	Coverage of decision sequences, not single branches.	ch06	path coverage (copertura dei cammini)
peek / poke	Back-door sample or deposit that bypasses a field's behaviour entirely, unlike back-door read and write, which mimic the front door's side effects.	ch10	peek / poke (lettura e deposito diretti in back-door)

Term	Definition (one line)	Introduced	Italian
phasing	The standard ordered steps every component runs together — build top-down, connect bottom-up, run concurrently — so independently written environments can be combined.	ch10	phasing (successione ordinata delle fasi)
PHY (physical layer)	Block driving and receiving the electrical signaling of a link, usually analog, hence modeled rather than simulated at gate level.	ch18	PHY (blocco di livello fisico)
plan row	One line of the verification plan: a feature, its attributes, a pass/fail criterion, a method and one closure metric.	ch05	riga del piano di verifica
plausible by construction	The property of an artifact from a system whose objective is a convincing continuation of its input: correctness is a frequent by-product rather than the target.	ch25	plausibile per costruzione
PLL (phase-locked loop)	Analog block synthesizing an on-chip clock from a reference; in the lab nothing digital runs before it has locked.	ch20	PLL (anello ad aggancio di fase)
poka-yoke	Mistake-proofing a human process by reducing it to foolproof steps.	ch02	poka-yoke
pool (PSS)	Sized store of resources or objects that actions claim; its depth is a rule the generator must satisfy, not a queue to wait in.	ch09	pool (deposito di risorse in PSS)
portable stimulus	Stimulus described once as a scenario space and retargeted to simulation, emulation, prototype and silicon.	ch09	portable stimulus (stimolo portatile)
power domain	A collection of instances treated as a group for power-management purposes, typically sharing a primary supply set.	ch19	dominio di potenza
power intent	Description of domains, isolation, retention and legal supply states, held outside the design description because the same logic is integrated into different power architectures.	ch19	intento di alimentazione
power state table (PST)	A statement of the legal combinations of supply states.	ch19	tabella degli stati di potenza (PST)
pre-silicon reproducer	What a lab investigation owes the design team: a failing stimulus brief enough for a simulator and narrow enough for a block-level environment. A narrative explanation does not qualify.	ch20	riproduttore pre-silicio
predicted-unreachable	Database flag recording that a coverage item was proven unreachable, so a warning is raised if a later change, constraint or test ever covers it.	ch16	predicted-unreachable (marcato come irraggiungibile)
predictor	Component computing the expected response from a monitored request — never from what the driver intended to send.	ch08	predictor (predittore della risposta attesa)

Term	Definition (one line)	Introduced	Italian
prime fault / collapsed fault	A prime fault represents an equivalence class; a collapsed fault produces the same observable behaviour as its prime, so only primes are simulated.	ch22	guasto primo / guasto collassato
product machine	Construction running two designs side by side on one input stream and reporting whether their outputs ever differ; the object a sequential equivalence proof explores.	ch15	macchina prodotto
proof core	The logic an engine actually used to establish a result, often smaller than the cone, so a bug inside the cone but outside it escapes.	ch14	proof core (nucleo della prova)
proof-carrying	Arriving with a mechanically checkable argument for itself. A generated property shipped with a cover on its antecedent and the sentence it came from is a small instance.	ch25	che porta con sé la propria prova
property-guided monitor generation	Proving a rule where an engine converges, then compiling the same rule into a synthesizable checker for long firmware runs, since a proof says nothing about software reachability.	ch23	generazione di monitor guidata dalle proprietà
protocol check (CDC sense)	Assertion check that a recognized synchronizer's preconditions really hold: pulse width, source-side stability, request-acknowledge sequencing, one changing bit at a time.	ch13	verifica di protocollo (per gli attraversamenti)
protocol monitor vs functional monitor	Passive component judging whether traffic on an interface is legal, versus one that only reconstructs and publishes what the design did.	ch08	monitor di protocollo e monitor funzionale
prototypability	Design property: the logic can be synthesized for a reconfigurable fabric at all, which rules out vendor memory macros, hand-gated clocks, latch structures and analog content.	ch18	prototipabilità
prototype-scope exclusion	Plan entry naming what a board cannot carry, such as an analog interface replaced by the platform's own, together with the named alternative that discharges it.	ch18	esclusione di perimetro del prototipo
provenance record	Repository entry for a generated artifact: the system and model under it, the exact document revision, the sampling settings, the checks passed, who accepted it, any later edit.	ch25	registro di provenienza
pseudo-static signal	Input declared to change only while its receiver is held in reset or disabled, so the setup may exempt it from crossing analysis.	ch13	segnale pseudo-statico
PSS (Portable Test and Stimulus Standard)	The standard defining that single declarative representation of scenario spaces, taking SystemVerilog as its constraint and coverage reference.	ch09	PSS (standard per test e stimoli portabili)

Term	Definition (one line)	Introduced	Italian
QED (quick error detection)	Software-only transformation of a test that shortens the interval between activation and failure, for instance by reading a location back immediately after every write.	ch20	QED (rilevamento rapido dell'errore)
quarantine	Named list of confirmed flaky tests, still run and reported but not gating, each with an owner and a date.	ch12	quarantena
radius (formal result)	Number recorded beside a formal verdict in the interchange database: how deep a bounded proof went, or how long its failure trace was, plus the clock.	ch16	radius (raggio del risultato formale)
RAL (register abstraction layer)	Object model of the design's memory-mapped registers and memories — blocks, registers, fields — commonly called the register model.	ch10	RAL (livello di astrazione dei registri)
random hardware fault	A physical defect arising during manufacturing or in operation, permanent or transient. It cannot be verified away — only detected and handled.	ch22	guasto hardware casuale
random stability	Localization of random generation per object and thread, so existing work keeps its sequence — provided new objects, threads and draws are appended, never inserted.	ch09	stabilità della generazione casuale
ranking	Selecting the (test, seed) pairs that contribute coverage and dropping the rest — a compressed suite at equal coverage, exploring nothing new.	ch12	ranking (selezione delle coppie che portano copertura)
RDC (reset-domain crossing)	Path where a register cleared by one asynchronous reset is sampled by a register in another reset domain, possibly under the same clock.	ch13	RDC (attraversamento di dominio di reset)
real I/O	Connecting a prototype to a physical console, network device or storage part, so the shipping driver runs against hardware rather than against a testbench substitute.	ch18	I/O reale
real number modeling (RNM)	Describing a block with real-valued signals updated on events in the ordinary simulator: no matrix to solve, no convergence failure, and no electrical relationship unless written in.	ch21	RNM (modellazione a numeri reali)
reasoning trace	Record linking the intent an agent was given to the action it took, so a reviewer audits the decision rather than only the output.	ch25	traccia del ragionamento
recipe (post-silicon sense)	The tuned combination of temperature, supply, part, elapsed time and feature set that brings a failure back on demand. Lacking one is not a property of the bug.	ch20	ricetta (condizioni che riproducono il guasto)
reconvergence model	Verification as reconciliation of a transformation and an independent second path sharing a common origin.	ch02	modello di riconvergenza

Term	Definition (one line)	Introduced	Italian
redundancy (verification sense)	A second, independent route to the same fact, used as the error-catching mechanism: a second interpretation of the specification by a different person, or a second access path to the same value.	ch02	ridondanza (nel senso della verifica)
redundant-state invariant	Assertion that two independently maintained representations of one fact agree — where silent corruption is most often caught.	ch11	invariante fra stati ridondanti
reference model	An independent executable of the specification, taken as golden, run on the same stimulus and compared with the design.	ch03	modello di riferimento
register app	Check proving RTL against a machine-readable register description: reset values, decode, and each field's read and write behavior for every value written.	ch15	register app (applicazione sui registri)
register description language	Machine-readable single source for a register map — addresses, fields and their behaviour — from which model, decode logic and headers are generated.	ch10	linguaggio di descrizione dei registri
regression suite	The test set re-run at fixed cadence to guarantee backward compatibility.	ch04	suite di regressione
repeat-N stability check	Running a test N times unchanged before admitting it to a gating tier, to establish that it is stable.	ch12	verifica di stabilità con N ripetizioni
required proof depth	The bound a bounded result must reach, derived from latency, micro-architecture and corner cases plus margin, and recomputable by a reviewer.	ch14	profondità di prova richiesta
reset synchronizer	Structure asserting reset asynchronously and releasing it on a clock edge, so that no flop meets the release inside its recovery window.	ch13	sincronizzatore di reset
residual fault	A fault that violates the safety goal where a mechanism <i>is</i> present, but which falls outside what that mechanism covers.	ch22	guasto residuo
respin	New fabrication cycle with corrected masks forced by an escape.	ch01	respin (nuovo giro di fabbricazione)
responder	Agent that answers requests that the design initiates; because its reply is under testbench control it is a driver, not a monitor.	ch08	responder (agente che risponde alle richieste del progetto)
restore-period condition	Predicate a retention strategy requires throughout its restore window. When it lapses the restore fails and the retained value is lost: the canonical wake-race escape.	ch19	condizione del periodo di ripristino
restrict	Directive constraining formal computation only; simulators ignore it.	ch11	restrict

Term	Definition (one line)	Introduced	Italian
retention	Enhanced functionality on selected sequential elements so their values survive the power-down of the primary supply.	ch19	retention (mantenimento dello stato)
retiming	Moving a register across combinational logic without changing the function; the mapping breaks, so equivalence needs a sequential proof with a declared latency relation.	ch15	retiming (ricollocazione dei registri)
retirement argument	The per-row reason for removing verification work, naming the evidence that replaces it. Cutting a regression by a percentage supplies none.	ch26	argomento di dismissione
reuse gap	Organizational shortfall named by a published taxonomy: knowledge about a block developed at one level and never applied at another, plus missing familiarity with the block itself.	ch24	lacuna di riuso
risk-driven verification	Allocating verification effort by bug likelihood × escape cost instead of pursuing completeness.	ch03	verifica guidata dal rischio
round-trip cost	What a single host interaction costs, counted in the design cycles that could have run instead: the call, the link crossing, host scheduling, the return. Largely fixed by the platform.	ch17	costo di andata e ritorno
RTL (register-transfer level)	The abstraction synthesizable design is written at: registers, the combinational logic between them, and clocks stated explicitly.	ch01	RTL (livello di trasferimento tra registri)
run	The unit of work: one execution with a specific seed and a specific set of source and tool revisions, producing its own messages and coverage.	ch09	run (singola esecuzione)
run manifest	Per-run record of everything needed to repeat it: seed, design and testbench revisions, exact tool build, configuration, host and result.	ch12	manifest della run
S1/S2/S3	Bug severity classes: S1 blocks tape-out; S2 fix-or-waive with an argument; S3 documentation or cosmetic.	ch07	S1/S2/S3 (classi di gravità dei bug)
safe fault	A fault that cannot violate the safety goal, either because it cannot reach safety-related logic or because its effect is tolerated.	ch22	guasto sicuro
safe state	The state a mechanism takes the system to when it fires: in a vehicle, a warning lamp, a limp mode, or a trip to the dealership.	ch22	stato sicuro
safety case	The written argument shipped as a deliverable, whose claims the evidence supports and whose gaps are argued explicitly.	ch22	safety case (argomentazione documentata di sicurezza)

Term	Definition (one line)	Introduced	Italian
safety mechanism	The design element that handles a failure mode — and itself a design, with bugs of its own.	ch22	meccanismo di sicurezza
sampled value	The value a concurrent assertion sees: the one held before the clock edge, which is why an assertion cannot race the design it watches.	ch11	valore campionato
sampling temperature	Setting governing how varied a model's output is. It is not a seed: identical input can give different output, so the artifact is versioned rather than the recipe.	ch25	temperatura di campionamento
SAT (Boolean satisfiability)	The decision problem under both constraint solving and proof engines: does an assignment exist that makes a Boolean formula true.	ch09	SAT (soddisfacibilità booleana)
satisfiable narrowing	The quiet over-constraint: assumptions remain consistent and most proofs stand, yet one clause has removed precisely the behavior the difficult property targets.	ch14	restringimento soddisfacibile
SCE-MI (Standard Co-Emulation Modeling Interface)	Published interface for driving an emulated design from untimed host code across vendors; the platform generates clocks and tracks model time, which is what allows replay.	ch17	SCE-MI (interfaccia standard di modellazione per la co-emulazione)
scenario (stimulus)	A sequence of stimulus interesting to the design and unlikely to arise from individually constrained-random items.	ch09	scenario (di stimolo)
scoreboard	The data structure holding the data expected to be received, filled by a predictor and consulted when an output is observed; loosely, the whole self-checking structure around it.	ch02	scoreboard (registro dei risultati attesi)
SDF back-annotation	Loading implementation delays into a simulation from a Standard Delay Format file. SystemVerilog takes only the timing constructs from that file and ignores the rest.	ch19	retroannotazione SDF
SEC (sequential equivalence checking)	Proof that two designs emit the same output sequence for every input sequence, dropping the matched-state assumption and paying a model-checking cost.	ch15	SEC (verifica di equivalenza sequenziale)
seed	The value fixing a run's random draws; replaying it reproduces the stimulus only while generator, sources and tool build are unchanged.	ch09	seed (seme del generatore casuale)
self-checking testbench	An environment that computes pass/fail itself instead of relying on human waveform inspection.	ch03	testbench autoverificante

Term	Definition (one line)	Introduced	Italian
self-help material	Answers reachable without asking a person: recurring questions, one sheet per application, worked exercises. It is where a local expert's answers go to survive reorganization.	ch24	materiale di autoconsultazione
sequence	Transient object generating stimulus, deliberately outside the component hierarchy so scenarios are not welded into the environment.	ch10	sequence (sequenza di stimolo)
sequence item	The transaction object a sequence produces and a driver consumes.	ch10	sequence item (transazione prodotta dalla sequence)
sequencer	Component holding pending sequence requests and choosing which one the driver receives each time the driver asks.	ch10	sequencer
sequential depth	The number of cycles needed to reach the farthest reachable state from reset; beyond it, a bounded safety result becomes a genuine proof.	ch14	profondità sequenziale
shaped cross	Cross with the reachable space stated via justified exclusions.	ch06	cross sagomato (con le esclusioni dichiarate)
shift-left	Moving a check earlier than the stage that traditionally performed it, so a defect is found before the artifact it would have damaged exists.	ch26	shift-left (anticipazione dei controlli)
shift-right	Moving a check after tape-out , onto the fabricated part — the lab and the field, not a pre-silicon platform. An emulator runs a model, so emulation is never shift-right however late it happens.	ch26	shift-right (controlli spostati dopo il tape-out)
side channel	Route by which a protected value can be inferred from an observable physical quantity rather than read. Verifiability depends on whether that quantity exists in the model.	ch23	canale laterale
sighting	A lab failure made repeatable under a stated set of conditions. It precedes assigning a debug team, and both precede any statement about a root cause.	ch20	sighting (avvistamento ripetibile del guasto)
sign-off review	The formal review that judges evidence against the plan and accepts residual risk.	ch04	revisione di sign-off
signal-level acceleration	Putting only the design on the platform while protocol behavior stays in host classes, so every pin event crosses the link. The canonical way to waste a fast machine.	ch17	accelerazione a livello di segnale
silent drift	Divergence among an interface rule's three forms that leaves both reports green, because an assumption tighter than the stimulus permits produces no failure and no hole.	ch16	deriva silenziosa

Term	Definition (one line)	Introduced	Italian
simstate	The operational capability a supply state supports, from NORMAL (full switching with characterised timing) down to CORRUPT (the supply cannot even hold existing state). Makes the level of corruption a declared property rather than a fixed behaviour.	ch19	simstate (capacità operativa dichiarata di uno stato)
single event upset (SEU)	The transient fault model: one bit flipped once, then left to propagate or die out.	ch22	single event upset (SEU, inversione di un bit per evento singolo)
single-point fault	A fault that violates the safety goal with no safety mechanism present to catch it.	ch22	guasto a punto singolo
smoke tier	The per-commit tier: fixed seeds, bounded runtime and zero tolerance for a known-failing member, proving only that the build is alive.	ch12	smoke tier (fascia per ogni commit)
SoC (system on chip)	Whole system integrated on one die, processors and accelerators and interconnect and memory and peripherals, verified at three levels.	ch01	SoC (sistema su chip)
soft constraint	Preference rather than requirement: discarded when it cannot hold alongside the active hard constraints, so a default can be overridden instead of fought.	ch09	soft constraint (vincolo preferenziale)
source cover	Evidence that a run actually placed a protected value into the tracked signals. Without one, a flow rule reports green over runs where nothing was ever tracked.	ch23	cover sulla sorgente
spatial merge	Combining coverage taken from different parts of one design; a mechanical operation, unlike a merge across verification processes.	ch16	merge spaziale
spec-hole log	Tracked list of questions extraction raised that the specification cannot answer, filed as spec defects.	ch05	registro dei buchi di specifica
speed adapter	Element between a slowly running emulated design and a real link running at its specified rate: it buffers, throttles and retimes, so content is real and timing is not.	ch17	adattatore di velocità
SPFM / LFM / PMHF	Single-point fault metric, latent fault metric, probabilistic metric for random hardware failures: the architectural metrics, each carrying a threshold per integrity level.	ch22	SPFM / LFM / PMHF (metriche architetturali della sicurezza)
spurious counterexample	Failure trace that the real design cannot produce, arising from a missing assumption or an over-approximated starting state, and cured by stronger invariants.	ch14	controesempio spurio
STA (static timing analysis)	Structural analysis establishing that every path meets its timing constraints, with no stimulus, and silent about what the design computes.	ch19	STA (analisi statica dei tempi)

Term	Definition (one line)	Introduced	Italian
stage inflation	A maturity stage claimed without its exit checklist, turning a demonstrable state into an assignable status word.	ch24	inflazione degli stadi
stale model	A learner whose training data no longer describes its subject. A stimulus model going stale shows in the coverage report; a generated checker going stale announces nothing.	ch25	modello obsoleto
state point	A register of a model, which a combinational equivalence run must pair with its counterpart before any comparison; unpaired ones yield a mapping report.	ch15	state point (punto di stato)
state restoration ratio	Restored plus traced states over traced states, the research measure of a trace signal set. No industrial report of its use in selecting signals exists.	ch20	rapporto di ripristino dello stato
state space	The set of all possible configurations of a design's state elements; 2^n for n state bits.	ch03	spazio degli stati
state-space explosion	The exponential growth of state count with design size that makes exhaustive analysis intractable.	ch03	esplosione dello spazio degli stati
static timing analysis (STA)	Structural examination of every timing arc against a constraints file across process, voltage, temperature and variation. It is the authority on timing; simulation is one sample.	ch19	analisi statica dei tempi (STA)
static verification	Analysis of the design description alone, needing no stimulus and no stated expectation, exhaustive over structure and silent about what the design computes.	ch13	verifica statica
stepping	A fabrication cycle with corrected masks — what Chapter 1 calls a respin , in the vocabulary of the lab. The two words denote the same event; <i>stepping</i> is how a bring-up team names it.	ch20	stepping (nuova maschera, nel lessico del laboratorio)
stream key	The tuple naming one ordered stream in a scoreboard: one dimension per independent source of concurrency, and computable from what the observer sees.	ch08	chiave di flusso
stride	The signed address step from the start of one 2D row to the start of the next; a negative stride walks memory downwards.	ch01	passo fra le righe (stride)
strobe time	Instant at which two copies of a design are compared during injection. It fixes what a campaign measures, alongside the choice of compared signals.	ch22	istante di confronto (strobe)
structural pruning	Removing from a fault list every site outside the cone of any observation point: an argument that holds under any stimulus, and cheaper than asking an engine.	ch22	potatura strutturale

Term	Definition (one line)	Introduced	Italian
stuck-at fault	The conventional model of a permanent defect: a node held at zero or at one.	ch22	guasto stuck-at
SVA (SystemVerilog Assertions)	The four-layer notation — Boolean, sequence, property, statement — in which temporal design rules are written once and read by simulator and proof engine alike.	ch11	SVA (assertion di SystemVerilog)
symbolic checking	Replacing enumerated port indices with constrained arbitrary ones, so one property covers every combination and the argument stays the same size as the design grows.	ch15	verifica simbolica
system slowdown factor	The divisor applied to a system clock so timing closes in fabric. Ratios to a board crystal that does not scale change with it, and crossing behavior with them.	ch18	fattore di rallentamento del sistema
systematic fault	A design bug: a mistake made during development, present in every part ever manufactured, and always permanent. Contrast <i>random hardware fault</i> .	ch22	guasto sistematico
tape-out	Freezing and releasing the design database for fabrication.	ch01	tape-out (rilascio del database per la fabbricazione)
temporal merge	Combining repeated executions of one process against one unchanged design; with the spatial case, one of the two merges the standard treats as mechanical.	ch16	merge temporale
test realization	Mapping abstract actions onto target code for one platform: portable describes the model, not the effort.	ch09	realizzazione del test
test selection	Filtering tests before simulating them, on the hypothesis that dissimilar tests hit dissimilar coverage.	ch12	selezione dei test
the crunch	The end-of-project concentration of the hardest verification work against an immovable date.	ch04	la stretta finale
thousandfold test	Asking whether a claim would survive engines a thousand times better. It separates a property of the problem from a limit of the tooling of the day.	ch26	test del fattore mille
threat model	Versioned document of assets, adversary capabilities and trust boundaries whose rows become checkable claims. A functional specification alone gives no reason to ask its questions.	ch23	modello delle minacce
tier	An admission policy with a runtime budget attached — what earns a place in a given regression run, not when that run is scheduled.	ch12	tier (fascia di ammissione con budget di tempo)

Term	Definition (one line)	Introduced	Italian
time bomb	Timeout ending a run that waits for something that never comes; useful only as an abnormal ending, since a suite routinely ending on it cannot tell success from deadlock.	ch12	time bomb (timeout che chiude una run bloccata)
timing exception	A human claim in the constraints file that some path need not be analyzed, false or multicycle. An annotated netlist run is what checks that claim.	ch19	eccezione di timing
timing side channel	Dependence of an operation's completion time on protected state. Cycles exist in the model, so this one is verifiable: a fixed-latency claim, or a rule ending at the done signal.	ch23	canale laterale temporale
TLM (transaction-level modelling)	Communication between components in whole transactions over standard interface handles, rather than through signals.	ch10	TLM (modellazione a livello di transazione)
toggle coverage	Per-bit 0→1 activity metric.	ch06	toggle coverage (copertura delle commutazioni)
tolerance token	Language construct attaching a band to a single real value, so that a coverage bin on a measured quantity can be hit at all despite finite precision.	ch21	token di tolleranza
tool qualification	The argument that a tool's output can be trusted as evidence, scoped to a version, a set of use cases, and often a configuration.	ch22	qualifica dello strumento
traceability	Permanent-ID linkage spec ↔ feature ↔ test/property/coverage ↔ result.	ch05	tracciabilità
transaction level	Abstraction whose unit is a whole operation — injected into the running simulation, terminated later by an observed result — rather than the design's clock cycles.	ch08	livello di transazione
transactor (bus-functional model)	Component converting between pins and transactions: every physical-level operation of one interface encapsulated in one place, so everything above it speaks transactions.	ch08	transactor (modello funzionale del bus)
transactor boundary	The cut between untimed host code and timed synthesized logic. Where it falls, rather than the platform's clock rate, decides the throughput actually obtained.	ch17	confine del transactor
transfer descriptor	The record software writes to program one DMA transfer: source and destination addresses, lengths, strides and channel. VMM and UVM texts use "descriptor" for a transaction object; this book always means the DMA record.	ch01	descrittore di trasferimento
transfer function	Testbench model reproducing the design's data transformation to predict its output: written by you, not golden, and as capable of misreading the specification as the design is.	ch08	funzione di trasferimento

Term	Definition (one line)	Introduced	Italian
triage	Turning failures into facts before debugging any: deduplicate by signature, classify as design bug, testbench bug or environment failure, then assign.	ch12	triage (classificazione dei fallimenti)
trigger-based debug	Debug built on state machines compiled into the platform that recognize an event sequence and start or stop capture, replacing waveform dumping bounded by a finite buffer.	ch17	debug a trigger
trust boundary	Surface where the capability tier changes. It makes assets and capabilities checkable, since every property reads as one asset not crossing one boundary under one capability.	ch23	confine di fiducia
two-flop synchronizer	Two chained destination flip-flops that give a metastable first stage time to settle before the second samples it; the building block of larger crossing structures.	ch13	sincronizzatore a due flip-flop
type vs instance override	Replacing a requested type everywhere, or only at one instance path; instance wins over type, and both must be registered before the parent builds its children.	ch10	override di tipo e override di istanza
UCIS	Unified Coverage Interoperability Standard — cross-tool coverage interchange.	ch06	UCIS (standard di interscambio dei dati di copertura)
under-approximation	Reduction that removes behaviors: a failure it reports is real, while a clean result proves nothing beyond the reduced model.	ch14	sotto-approssimazione
undetermined	Result in which the engine gave up, through timeout, memory exhaustion or an incomplete algorithm; neither a pass nor a failure.	ch14	indeterminato
unification layer	The verification plan, where each row closes on its own declared evidence, which is what makes a cross-engine report have a denominator.	ch16	livello di unificazione
unit-test fallacy	Reading a high pass rate on block-scale benchmarks as evidence about a system. Every published figure in this area concerns a block, a protocol or a curated benchmark.	ch25	fallacia del test unitario
unreachability app	Check submitting coverage targets to a model checker so an exclusion rests on a proof instead of an argument; the verdict inherits every environment constraint.	ch15	unreachability app (verifica di irraggiungibilità)
UPF (unified power format)	Machine-readable statement of a design's power intent, held outside the design source and read by simulation, synthesis and equivalence tools.	ch01	UPF (formato standard dell'intento di potenza)
UVM (Universal Verification Methodology)	The standardized base class library and API set for building modular, reusable, configurable verification components.	ch10	UVM (metodologia di verifica universale)

Term	Definition (one line)	Introduced	Italian
vacuous pass	Success of an implication whose antecedent never became true, so nothing was checked — neither a pass nor a failure.	ch06	successo vuoto
valid/invalid hole	Genuine stimulus gap vs a coverage-model bug.	ch06	buco valido / buco invalido
validation	Checking a design on real hardware rather than on a model — post-silicon on fabricated parts, or in the lab on a prototype. The load-bearing contrast is hardware vs model, not fabricated vs FPGA. NOT the Boehm “did we build the right thing?” sense, and not the umbrella term covering pre-silicon work.	ch20	validazione (prova su hardware reale, non su un modello)
validity pruning	Discarding stimulus as “can’t happen” before checking whether the interface can express it (anti-pattern).	ch02	potatura di validità (scarto dello stimolo ritenuto impossibile)
variable ordering (solve ... before)	Ordering the solver’s choices so a corner case becomes frequent, changing probabilities without changing the legal set.	ch09	ordinamento delle variabili (solve ... before)
verification component (VC)	Reusable testbench-side model of an interface, with a driver to stimulate and a monitor to observe.	ch02	verification component (componente di verifica)
verification gap	Shortfall between what must be verified and what can be (inherent/transient/self-induced).	ch01	divario di verifica
verification IP (VIP)	A verification component supplied ready-made for a standard protocol, typically purchased rather than written.	ch10	verification IP (VIP, componente di verifica acquistato)
verification lifecycle	The phased process from plan to sign-off, with feedback loops, run once per verification level.	ch04	ciclo di vita della verifica
virtual emulation	Use model with no physical target: clocks come from the platform and model time is tracked there, giving deterministic reruns, injection at a chosen cycle and simulator-like debug.	ch17	emulazione virtuale
virtual platform	Software model of a whole system, fast enough to run a real firmware binary long before stable logic exists, and without arbitration, back-pressure, latency or coherence.	ch18	piattaforma virtuale
virtual sequence / virtual sequencer	Sequence coordinating several interfaces at once, through a sequencer that only holds references to sub-sequencers and drives nothing itself.	ch10	virtual sequence / virtual sequencer
volatile (register field)	A field the design can change unobserved, so its mirrored value may not be trusted without a fresh read.	ch10	volatile (campo di registro)

Term	Definition (one line)	Introduced	Italian
volume criterion	A plan row about aggregate behavior over many transactions, such as throughput, occupancy or fairness; no proof discharges a statistical claim.	ch16	criterio di volume
waiver	Written record accepting a specific evidence hole: hole, reason, risk argument, owner, expiry.	ch07	waiver (deroga scritta)
weakness analysis	Row-by-row review challenge on correctness, precision, completeness.	ch05	analisi delle debolezze
white-box assertion	Assertion checking internal design signals rather than interface behavior.	ch02	assertion white-box (sui segnali interni)
witness	Stimulus sequence a proof engine returns to show that a cover target is reachable while every stated assumption still holds.	ch14	witness (traccia testimone)
witness cost	What a result costs to read: a counterexample arrives short and history-free, a simulation failure arrives with everything that preceded it.	ch16	witness cost (costo di ottenere la traccia)
word-level reasoning	Treating operands and results as numbers rather than bit vectors, which with decomposition is what makes an arithmetic proof converge.	ch15	ragionamento a livello di parola
X-dependent check	Check that needs an unknown value to reach an output. It stays in simulation, because a fabric of real gates has only two values to offer.	ch17	controllo dipendente da X
X-optimism	Simulation propagating a definite 0 or 1 where silicon would hold an unknown, which can mask design bugs — though asynchronous reset works because of it.	ch12	ottimismo sulle X
X-pessimism	Simulation returning an unknown where silicon would produce a definite value, filling waveforms with artifacts that cost tracing time but rarely hide design bugs.	ch13	pessimismo sulle X
X-propagation	The question of whether an unknown can reach a decision or an output, answered at gate level, by a simulator mode, or by proof.	ch13	propagazione delle X
zero-delay netlist	Netlist simulated with no annotated delays and no power intent: the configuration a first gate-level run should use, so each later addition has exactly one suspect.	ch19	netlist a ritardo nullo
zoom-out thinking	Periodic step back from task-level work to reassess project-level verification priorities.	ch02	sguardo d'insieme (rivalutazione periodica delle priorità)

Appendix B: Reading paths

This appendix provides reading paths through a book of about 500 pages. It reverses the table of contents: each ordered sequence of chapters claims to equip the reader for a specific task.

Every path is therefore falsifiable. Each one names the situation it addresses rather than the seniority of its reader, then the expected capability, then the omissions and what they cost.

Where only part of a chapter is needed the section number is given; Chapters 1–3 have unnumbered sections and are cited whole.

Three chapters for a minimal reading path

This selection aims to maximize immediate practical change per page rather than logical completeness, for readers whose work touches verification: designers, leads, architects, and managers, rather than verification engineers.

- **Chapter 2, *The Verifier's Mindset***: its opening scenario concerns a designer verifying her own block with stimulus limited to intended use, letting bugs escape.
- **Chapter 3, *The Verification Problem*** explains the rationale for coverage models, risk ranking, and waivers. It explains risk management rather than completeness, and it explains that every practical oracle is partial. This path assumes that most practitioners have been taught neither point.
- **Chapter 7, *Metrics-Driven Sign-off*** addresses the completion decision these readers will attend and the evidence to require (§7.4).

Chapter 1 is absent because this path assumes agreement that verification matters, and that agreement is the argument that Chapter 1 develops. Chapter 5 is absent: 8,000 words on planning for a reader who may never own a plan.

A first verification job

This path assumes SystemVerilog proficiency and an inherited environment with failing seeds to resolve. The intended outcomes are locating faults in an unfamiliar test-bench, writing reproducible stimulus, and triaging a regression without escalating every failure.

- **Chapter 2** addresses the initial change from training rewarded for making things work to the opposite verification posture.
- **Chapter 3** explains why the impossibility of testing everything, raised in week one in this scenario, is a methodological premise rather than an excuse.

- **Chapter 8, *Testbench Architecture*** separates the inherited environment into five responsibilities (§8.1); identifying which one owns a symptom directs fault localization.
- **Chapter 10, *UVM as a Methodology*** explains the rationale for the class taxonomy used in the UVM code this job requires (§10.2), rather than the API.
- **Chapter 9, *Stimulus: Directed to Random to Portable*** covers constraints (§9.3), assumed to occupy most of the first year, and the reproducibility needed for senior review (§9.4).
- **Chapter 12, *Regression Engineering*** supports morning triage: §12.5 distinguishes a test that has stopped providing evidence from a broken design.

Omits Part II entirely (planning, coverage theory, and sign-off). The reader will execute plan rows without knowing their origins and interpret coverage reports as scores rather than arguments. That material becomes necessary when a plan row is assigned to the reader. **Chapter 24 §24.1** follows when the reader takes ownership of a block; it restates ownership as an outcome.

A designer verifying a block alone

Without a verification engineer, the designer must verify and defend the block. The outcomes are the cheapest defensible checks, which begin without a testbench, and a list of designer reflexes that make an environment seem healthier than it is.

- **Chapter 2** addresses this designer: *Independence of judgment* gives three disciplines for a team of one. The essential discipline is prediction from the specification, never from RTL behavior.
- **Chapter 13, *Static Verification*** addresses the absence of a testbench: §13.1 describes analysis needing no stimulus or statement of expected output, running the first day RTL compiles.
- **Chapter 11, *Assertion-Based Verification***: §11.4 explains why the designer is better placed than any verifier to write internal invariants expressing micro-architectural assumptions absent from every specification.
- **Chapter 5, *Verification Planning***: even a one-person plan must specify each row's closure; §5.2's five fields give the minimum adequate form. §5.4 assumes an unavailable review audience, but its three criteria (correctness, precision, and completeness) also apply to the designer's own rows.
- **Chapter 10 §10.10** explains why a small block with one interface does not repay a UVM environment; its alternative is a module testbench with bound assertions and one covergroup.
- **Chapter 15 §§15.2-15.3** provides connectivity and register proofs the designer can run on the block without owning a formal methodology.

Chapter 24 §24.4 provides a later checklist of designer reflexes: explaining failures instead of reproducing them, building a recovering testbench, and debugging in waveforms.

Omits coverage modeling (Chapter 6) and stimulus at scale: the evidence establishes checks that fired rather than space covered, a weaker claim; this path treats that limitation as the first issue for review.

Formal verification after a decade of simulation

The next block is an arbiter or protocol converter whose specification is a set of properties. The first outcome is interpreting a formal result with the three qualifiers that Chapter 14 defines. The second is recognizing invalid passes, and the third is working an undetermined property in the order that costs an afternoon rather than a fortnight in this scenario. An undetermined property is one on which the engine gave up, through timeout, memory exhaustion, or an incomplete algorithm, and its result is neither a pass nor a failure.

- **Chapter 11** comes first even for readers who already write assertions. §11.3 is the asymmetry everything rests on: in simulation `assert` and `assume` are effectively synonyms; in formal an assumption is never checked and deletes traces instead. §11.8 names the other two shifts: covers ask whether a scenario is reachable rather than count how many times it occurs, and liveness properties become decidable.
- **Chapter 14, *Formal Property Verification*** covers the transition. §14.2 explains the hazard absent from simulation instincts: a *missing* assumption causes failures, while an *extra* one only produces more passes.
- **Chapter 15, *Formal Apps and Equivalence*** offers checks for next month without a formal methodology: connectivity and register maps, plus §15.4's unreachability proofs that replace coverage waivers with results.
- **Chapter 16, *Hybrid Flows*** retains simulation and supplies a decision procedure for each plan row, applicable in review (§16.4).

Omits the vendor's engine. §14.5's convergence tactics are generic; tool documentation supplies option names. It also omits ownership of an equivalence-checking flow; §15.6 places it in implementation, where it usually sits.

Producing and defending a verification plan

This path assumes senior review of the plan and accountability against it at sign-off. The outcomes are rows with one measurable exit each, defensible method fields, and a plan that accommodates specification gaps.

- **Chapter 4 §§4.2, 4.6** explains how timing and review membership determine a plan's value; both calendar commitments need negotiation before writing begins.
- **Chapter 5** supplies the planning method: §5.2's five fields, §5.5's depth selection for each feature, and §5.6's treatment of the reader's situation.

- **Chapter 6, Coverage: Theory and Practice** addresses coverage, and this path assumes that coverage will supply most of the closure metrics. §§6.3 and 6.5 prevent models that give evidence for the stimulus rather than for the design.
- **Chapter 16 §16.4** addresses optimism in method selection. Its question 6 addresses nonconvergence, otherwise left until week nine in this scenario.
- **Chapter 7 §7.4** supplies the sign-off checklist for the plan's last column; reading it during planning exposes fields otherwise left blank.

Omits the engines themselves. The reader will specify “formal” without knowing what makes a property converge and “emulation” without understanding the machine queue (§17.5). For a first plan, this path accepts the gap; a second plan requires the engine knowledge addressed by the path *Formal verification after a decade of simulation* above.

Signing off evidence produced by others

This path addresses responsibility for signing off a block or SoC. The outcomes are knowing which evidence to require, its form, and which passing results cannot fail.

- **Chapter 7: §7.4** lists seven required items; §7.3 distinguishes a clean flat bug curve from one flattened because stimulus stopped probing for bugs.
- **Chapter 6 §6.9** addresses others' waivers, assumed to account for most difficult decisions; the three kinds differ in ownership and expiry. A risk-accepted waiver for a deadline needs the sign-off owner's signature rather than the model owner's.
- **Chapter 14 §14.6** goes beyond a proof tick with three required review questions: checker-list completeness, absence of unintentional over-constraints, and achievement of required depth.
- **Chapter 12 §§12.2, 12.5** explains that a passing regression describes a (test, seed) matrix; leaving quarantine lists uncleared similarly undermines the evidence.
- **Chapter 19 §19.5** addresses proposals to run the whole suite at gate level or skip the tier entirely. The arithmetic rules out the first; five criteria select the handful of tests that answers the second.
- **Chapter 20 §20.5** belongs before sign-off. It classifies every escape by the first broken link in the chain that Chapter 3 defines (activation, propagation, or checking), and each break requires a different upstream change.

Omits how any of that evidence was produced. The reader audits work they could not have produced: adequate for a block, limited at SoC scale, where others fixed §20.2's design-for-debug budget at RTL freeze without consulting them.

Transition from software test or QA

This path assumes experience writing test frameworks, running continuous integration, and investigating flaky tests, the ones whose outcome varies between identically configured runs and that have therefore stopped being evidence. Those instincts transfer, but several terms do not. The outcomes are correct vocabulary and a list of existing habits that already apply.

- **Chapter 3** comes first because of the familiar oracle problem. The new distinction is that every practical hardware oracle is *partial*, requiring combinations whose blind spots do not overlap.
- **Chapter 1, *Why Verification Exists*** supplies the cost curve for readers accustomed to patchable systems.
- **Chapter 8** builds on familiar software separations: its five responsibilities (§8.1) map almost one-to-one onto them.
- **Chapter 12** covers familiar work with two corrections: §12.3 explains why running everything on every commit does not transfer when a unit costs minutes rather than milliseconds; §12.5 covers hardware-specific sources of flakiness.
- **Chapter 6** narrows “coverage”: §6.2 treats free but structurally blind code coverage; §6.3 treats functional coverage as a model a person writes from the specification.
- **Chapter 18 §18.4** covers firmware-driven verification, the closest match to prior software work, and attributes the problem to an incomplete interface contract rather than fault on either side.

One vocabulary distinction concerns **validation**: the meaning used here excludes the one expressed by “did we build the right thing?”. Chapter 20 defines it at §20.1 as checking a design on real hardware rather than on a model; everything before tape-out is *verification*.

Omits Part IV. The reader will continue encountering formal verification without a model for it; the path *Formal verification after a decade of simulation* is a later step.

Verification required by a standard

This path addresses a market requirement and external review of the resulting artifacts.

Common spine, all three branches: Chapter 5, then Chapter 7. Each regime converts ordinary planning and metrics into retained, traceable artifacts; these two practices explain the selection.

Safety. Chapter 22, *Safety Verification*: §22.1 gives the three obligations (process, evidence, traceability) and their overriding consequence: the argument itself becomes a deliverable. §22.5 scopes qualification to a tool *version* and *use cases*, so a mid-project upgrade requires a new argument rather than merely an IT ticket. Then **Chapter 12 §12.2** defines the run manifest that §22.5 makes the unit of the evid-

ence trail. Then **Chapter 19**: §22.3’s fault-injection campaigns and §19.5’s netlist tiers compete for the same machine hours.

Security. **Chapter 23, Security Verification** starts with planning (§23.2): assets and trust boundaries, with adversary capabilities enumerated rather than an open set of attacks. §23.6’s table routes each question to the engine that can answer it. Then **Chapter 11** prepares the reader to understand §23.3. A confidentiality claim quantifies over *pairs* of executions, and one assertion cannot express it. Reaching that understanding requires the ability to read an ordinary property. Then **Chapter 15 §15.2** supplies a security artifact for this path: a connectivity check with negative rows, in the build from day one.

Mixed-signal. This short branch ends with **Chapter 21 §21.5**. It assigns responsibility for the interface contract, for calibration sequences, for failure handling, and for the provenance of each tolerance, while excluding ownership of the analog block. It omits analog verification itself, including noise, linearity, and corner behavior. That belongs to the analog team, and Chapter 21 says so.

Entry points for a specific failure

Each entry is limited to two chapters and a handoff.

- **A bug escaped.** Chapter 7 §7.6 supplies five written, blameless questions; then Chapter 20 §20.5 identifies the first broken link and the upstream change each break requires. *Handoff*: if the answer to question 4 is “the metrics were fine and the plan was blind”, the problem lies upstream. The path *Producing and defending a verification plan* follows.
- **The regression is not trustworthy.** Chapter 12 §12.5 covers the recurring causes of flakiness and covers quarantine with a named owner and date; then Chapter 9 §9.4 defines a run as a seed *and* specific source files, and the source files are where reproducibility is usually lost. *Handoff*: a manifest missing the exact tool build is an infrastructure problem, requiring the gate in §12.3.
- **Coverage will not close.** Chapter 6 §6.8 covers the closure loop, including the invalid-hole branch this path assumes most teams skip; then Chapter 15 §15.4 replaces a waiver with proof, whose first limitation is inheriting every setup assumption, including those in scripts written by departed staff. *Handoff*: if the same holes recur every project, the model is downstream of a planning problem.

Reading dependencies

Chapter order differs from dependency, which determines what can be skipped.

- **Chapter 13 presupposes nothing.** §13.1 states that static analysis needs no stimulus or statement of expected output. It is readable on a project’s first day without a testbench or plan.

- **Chapter 11 is readable before Part III.** Its argument starts from the specification and the RTL rather than from an environment: §11.4 covers assertion authorship and placement, and §11.8 covers what happens when two engines read the same property text. Neither needs Chapter 8 or Chapter 10 first.
- **Chapter 21 §21.5 stands alone** as a boundary statement, which is why the mixed-signal branch above is one section long.

The following dependencies come from reading the chapters rather than their numbering:

- **Chapter 14 → Chapter 11 §11.3.** §14.2 depends entirely on the assert/assume asymmetry established in §11.3, as Chapter 11 explicitly states.
- **Chapter 10 → Chapter 8 §8.1.** §10.2 opens by taking Chapter 8’s “the reusable unit is one interface” as given and encoding it as a base class plus a flag.
- **Chapter 7 → Chapters 5 and 6.** §7.4 computes over artifacts the other two define: plan rows, closure metrics, waiver kinds. The checklist becomes auditable after the definitions in those chapters have been established.
- **Chapter 16 → Chapter 14.** §16.2 presents assumption and constraint duality as a flow; §16.4’s question 4 evaluates the cost of §14.4’s abstraction.
- **Chapter 20 §20.5 → Chapter 7 §7.6 and Chapter 3.** It reuses the five questions and Chapter 3’s activation-propagation-checking chain as its classifier.
- **Chapter 24 §24.3 → Chapter 4 §4.4.** Chapter 4 defines the maturity stages; §24.3 takes them as given and asks a narrower question about the conversations they change.

The chapters no path here reaches

Chapter 17, *Acceleration and Emulation* centers on one triage distinction (§17.1): the number of runs versus the length of one run. A wider farm answers the first and does nothing for the second, because cycle n of a boot depends on cycle $n-1$. It is relevant when run length is the problem.

Chapter 25, *AI and ML in Verification* addresses proposed tools that generate properties, testbenches, or plan rows. §25.4 names the four decision points a human must occupy and the damage of leaving each unowned.

Chapter 26, *The Road Ahead* is an argument rather than an instruction. It can be read last or omitted; it has no immediate operational effect, which excludes it from the three-chapter set.

Appendix C: The literature and its scope

This guide groups sources by topic, assessing their weight, distinctive contributions and gaps; each chapter’s reference list instead identifies the support for its claims. That list follows sentence order, so it cannot answer a topic-based reading question six months later.

Titles also obscure relevant content: the account of adoption across a large organization that this guide recommends appears in a formal property verification paper, and until Chapter 24 was written it was cited only in formal chapters. That paper covers a named local expert per team, escalation to a central group, continuity across projects and support for novices. Its title and citation pattern obscured it from readers with process questions. Section C.10 names every case of that kind.

C.1 Industry survey data

Initial reading: the 2024 Wilson Research Group IC/ASIC trend report (Foster, Siemens EDA). This short, current report supplies most quantitative framing in Chapters 1, 4, 7 and 26. Its separate FPGA companion covers a different population, one that is not a subset of IC/ASIC; it supplies Chapter 26’s escape figure.

Foster’s DAC 2015 paper, *Trends in Functional Verification: A 2014 Industry Study*, adds the series methodology, sample design and decade-earlier baseline for the 2024 numbers. Chapter 1 draws its 2014 and 2004 participant counts from Foster’s DAC paper, together with the paper’s verdict on the “70 percent of effort” figure, which it calls unsubstantiated; the 2024 participant count and margin of error come from the 2024 report.

Two counterweights: Mishra, Ray, Morad and Ziv’s post-silicon tutorial on what a bug costs as it moves right, and Narayan and Symons’ *I Created the Verification Gap*, a taxonomy of self-inflicted shortfall behind the management vocabulary of Chapters 1, 2, 4, 7 and 24.

Not covered. Any measured cost-per-stage series (Section C.12).

C.2 Methodology, testbench architecture and stimulus

Initial reading: Bergeron’s *Writing Testbenches using SystemVerilog*. Seventeen of the twenty-six chapters use it, and it alone treats the reconvergence model that Chapter 2 defines, together with bus-functional models, self-checking taxonomy and regression management in one unified account.

The *Verification Methodology Manual for SystemVerilog* (Bergeron, Cerny, Hunter, Nightingale) complements the explanatory testbench book with a set of rules. The rules concern the encapsulation of self-checking structures, recommendations on the

scoreboard that Chapter 2 defines, the layering of stimulus, seed discipline and the hierarchy of coverage guidelines. Chapters 8 and 9 use it for prescriptions resembling a standard.

Hollander, Morley and Noy's 2001 *The e Language: A Fresh Separation of Concerns* explains modern testbench architecture historically through decomposition, transaction-level framing and extension-as-configuration (Section C.10).

For stimulus, Yuan, Pixley and Aziz's *Constraint-Based Verification* covers over-constraint diagnosis and the constraint solver that Chapter 9 defines. On portable stimulus, which the same chapter defines, the reading is the Portable Test and Stimulus Standard itself, the working group's *PSS in the Real World*, and Gupta and Vax's *Test driving Portable Stimulus at AMD*, which gives its industrial limits.

Now included in this list, the *UVM Cookbook* idiom catalog is held; published by Siemens EDA's Verification Academy and not maintained by a community, it supports Chapters 8, 10, 11 and 17.

Not covered. Environment architecture as software design at book length.

C.3 Coverage, metrics and sign-off

Initial reading: Piziali's *Functional Verification Coverage Measurement and Analysis*. It covers taxonomy, model design method, fidelity and hole analysis for Chapters 5, 6, 7 and 26.

Clause 19 of the SystemVerilog standard is definitive for covergroup, bin, cross and option semantics; Accellera's Unified Coverage Interoperability Standard carries everything Chapter 16 says about merging results across engines, including its own caution against automatic merging.

On metrics as a management instrument, Meyer and Foster's *Metrics in SoC Verification* is the foundation of Chapter 7; Hristozkov, Pallister and Porter's *No Country For Old Men* is its modern industrial counterpart, with an in-house stack and measurements; Zhang shows what a management platform automates. For planning, Ehlers, Vargas and Carzola cover executable plans and assumption reviews, Marriott, Vance and McNeal deliverables and definitions of done, and Graham et al.'s *Planning for RISC-V Success* contributes the only published plan-row format here.

Four papers address closure automation at different levels: Kodi, Patil and Nair (constraint regeneration), Teplitsky et al. (constraint solver level), Ohana (deep reinforcement learning) and Feng, Chen and Muchandikar (coverage models for formal, which Chapter 6 defines).

Not covered. Coverage model quality: fidelity is discussed but never quantified.

C.4 Formal verification and its industrialisation

Initial reading: Kern and Greenstreet’s 1999 survey, *Formal Verification in Hardware Design*. It covers foundations, limits and the required content of a meaningful verification claim. Chapter 15 also calls it the corpus’s closest thing to a treatment of equivalence-checking algorithms.

Cerny, Dudani, Havlicek and Korchemny’s *SVA: The Power of Assertions in System-Verilog* is the reference for assertion semantics and for Chapter 14’s working vocabulary. Bormann’s dissertation is the principal reading here on property completeness. Schwarz’s dissertation treats bounded versus unbounded proof, sequential depth and spurious counterexamples from arbitrary starting states, all of which Chapter 14 defines. It extends that treatment across the hardware/firmware boundary for Chapter 18.

For practice, Tripathi, Saxena, Verma and Aggarwal demonstrate crossbar formal sign-off end to end: they derive the required proof depth that Chapter 14 defines, and they run a mutation experiment. Bromley and Sprott’s *Formal Verification in the Real World* covers complexity management and the traps of the waivers that Chapter 7 defines. Achutha Kiran Kumar, Seligman et al.’s *Making Formal Property Verification Mainstream* reports deployment, with a bug taxonomy by difficulty from already-simulated designs.

Not covered. Model-checking algorithms and the industrial C-to-RTL equivalence flow (Section C.12).

C.5 Standards consulted and limits of access

Three categories distinguish access from mention.

Worked from directly. IEEE 1800-2023 is cited across eight chapters. IEEE 1800.2-2020 is the normative UVM statement, with Accellera’s UVM 1.2 *User’s Guide* as the readable companion and the *Class Reference* for access policies and the arbitration modes that Chapter 10 defines. Then IEEE 1801-2024 (UPF) for Chapter 19, Accellera’s UCIS, the Portable Test and Stimulus Standard, SystemRDL, IEEE 1685-2022 (IP-XACT), the Verilog-AMS reference manual and the UVM Mixed-Signal standard for Chapter 21, and Accellera’s CDC/RDC IP Abstraction standard for Chapter 13. Accellera’s Standard Co-Emulation Modeling Interface reference manual covers Chapter 17’s boundary. IEEE 1666-2023 supports Chapter 8: Clause 9 names the transaction-level interoperability layer. The copy worked from is an institutional licensed download, citable here but not redistributable.

Named but not held. IEEE 1497 (the Standard Delay Format), referenced but not reproduced by the SystemVerilog standard. It is recommended in a chapter note and was never read. IEEE’s product record lists 1497-2001 as inactive-reserved, and it names the dual-logo IEC 61523-3:2004 as the replacement. That replacement is the standard that now defines the Standard Delay Format. The book’s claims about IEC 61523-3 are limited to what the product record says; that standard’s text remains unopened.

Unopened; never paraphrased as direct reading. ISO 26262 and DO-254/ED-80, the automotive and airborne safety regimes of Chapter 22. Everything Chapter 22 states about the automotive standard comes through Richter’s platform overview, Ahuja, Agarwal and Jana’s fault campaign, and Section C.7’s fault-injection deck; its DO-254 characterization is attributed to engineers experienced under both regimes, not the document. IEC 61508 is named only through the 2024 survey. Chapter 22 recommends reading these directly rather than paraphrasing them.

One caveat: the CDC/RDC working-group tutorial recurs at successive conferences with different presenters; the archive holds at least three editions, and the one cited is the 2026 U.S. edition, not a revision of the others.

Not covered. The corpus holds the VHDL reference manual, but the book uses SystemVerilog throughout; VHDL is mentioned twice in twenty-six chapters.

C.6 Emulation, prototyping and post-silicon

Initial reading: Mishra, Ray, Morad and Ziv’s post-silicon tutorial. For the later flow stages, it covers observability from simulator to fabric to silicon, the selection of trace signals, and the vocabulary that applies before a sighting, which Chapter 20 defines.

Rahman et al.’s *Emulation-based System-on-Chip Security Verification* classifies platforms: prototyping, ASIC-class emulation and hybrid co-emulation, the split that Chapter 17 defines. To that classification it adds the compile-versus-execute distinction, trace-buffer constraints and environment divergence.

Grove converts a conventional environment for acceleration; Hari, Krishnamurthy, Jain and Badaya explain the advantages of a virtual environment over an in-circuit one. On localization, Lin, Singh, Barrett and Mitra’s paper on symbolic quick-error-detection carries the trace-length data and the argument about the error detection latency that Chapter 20 defines. Mitra, Singh and Devarajegowda’s deck adds effort comparisons. Jain et al.’s *Never too late with formal* gives the method for reproducing a silicon bug in a proof environment.

Not covered. How emulation capacity is allocated across a program, and any narrative of bringing up silicon in the lab, other than the tutorial’s.

C.7 The domain obligations: safety, security, mixed-signal, power and timing

Initial safety reading: Richter’s *Unified Functional Safety Verification Platform*. It covers the systematic-versus-random split, the fault classification tree, the metric formulas and their failure-rate weighting. It also covers the FMEDA process that Chapter 22 defines, the analysis of failure modes, of their effects and of their diagnosis. FMEDA asks of each element how it can fail, with what consequence and likelihood, and which safety mechanism handles it. Ahuja, Agarwal and Jana supply the worked campaign and its fault-list reduction chain. Sesha Sai Kumar, Mouallem and Mazzawi’s *Fault Injection Analysis for Automotive Safety and Security* treats fault models in more detail, together with the strobes and the cone-of-influence pruning that Chapter 22 defines, and it extends into adversarial territory in Chapter 23.

Initial security reading: Rahman et al., followed by Hasan et al.’s survey, which organizes the same territory by workflow stage. Oberg’s root-of-trust talk states confidentiality and integrity as information-flow relations, the form in which Chapter 23 defines them.

Initial mixed-signal reading: Khan, Kashai and Fang, who apply metric-driven verification end to end to an analog block. Chang and Kundert supply the trust problem and the effort economics (caveat in Section C.11); Brennan, Ziller, Fotouhi and Osman give real-versus-analog modelling and the runtime comparisons.

Initial power and timing reading: Liu et al.’s *Low Power Verification with UPF*, alongside the UPF standard: a power-intent qualification checklist, an eleven-item isolation checklist and sequencing rules. Litterick’s *Full Flow Clock Domain Crossing* covers what happens to a synchronizer between RTL sign-off and silicon, and Chapter 19 draws on it.

Not covered. Side-channel measurement, the airborne regime in its own words, and any account of combining functional, safety and security positions.

C.8 People, process and adoption

Initial reading: Montesano and Litterick’s *Verification Mind Games*. Seven pages of concrete mindset guidance underpin Chapter 2 and most of Chapter 24’s guidance for engineers converting from design.

Making Formal Property Verification Mainstream is the corpus’s only account of making a technique mainstream in a large organization, and its only one that names roles and support structures. The roles it names are a named local expert for each cluster, called a champion, and a central expertise team that the champion can escalate to; the champions own the activity across projects. The support structures are ready-made property templates instead of a blank file, and self-help material for novices (the answers that a newcomer can reach without asking a person). The paper’s last point is this: practices that are absent from mainstream deliverables go unexercised. Bromley and Sprott’s companion planning slides discuss people explicitly (Section C.10).

Section C.3’s planning workshops carry the review mechanics; the 2024 IC/ASIC report the talent-gap data.

Not covered. Almost everything: Chapter 24 identifies three of its four review mechanics as the author’s judgment, as Chapter 12 does for a regression gate’s social layer.

C.9 Machine learning and language models

Initial reading: Yu, Foster and Fitzpatrick’s survey of machine learning applications in functional verification, then Bennett and Eder’s 2025 review: the first covers attempts, the second explains limited deployment, including the decay modes and the retraining obligation that Chapter 25 §25.1 takes up as the stale model.

For coverage-directed generation, Fine and Ziv’s 2003 Bayesian-network paper supplies the founding result and explains costs; despite few citation markers, it supports a whole section of Chapter 25. Ioannides and Eder review the field at its 2012 maturity; Jayasena and Mishra survey directed test generation in 2024 and refer readers back to them.

For generated properties, Fang et al.’s *AssertLLM* and Shih, Lin, Gupta and Malik’s *FLAG* should be read together, in that order, comparing their success definitions. Bai et al.’s *FVDebug* shows how results should be reported: a model-judged metric alongside a mechanically checked one. Kumar, Gadde, Radhakrishna and Lettnin’s *Saarthi* is the corpus’s one industrial agentic-formal deployment report, explicitly detailing failure modes. For landscape, Zang et al.’s agentic-EDA survey supplies the autonomy scale that Chapter 25 §25.5 applies to adoption decisions, and the governance vocabulary that goes with it; Pan et al. survey the wider field.

Three provenance caveats, which belong with the recommendation rather than a footnote. *Saarthi* is the one entry in this book’s reference file whose conference edition remains unresolved against the proceedings archive; cited without an edition, it takes its year from the file’s creation timestamp. The agentic-EDA survey’s year rests on one signal, the version stamp of the preprint, so Chapter 26 treats that year as the authors’ expectation rather than as a finding. This book deliberately cites *AssertLLM* and the BZL paper as preprints, because their published records diverge

from the artifacts read. The BZL paper is Ahmed et al.’s *(V&V)-in-the-Loop for RISC-V Design*. The published *AssertLLM* drops two words from its title, and the published BZL paper has a different title and an extra author.

Not covered. Most of it (Section C.12), as Chapter 25 acknowledges.

C.10 Sources filed under the wrong heading

Seven cases of titles or citation patterns obscuring relevant content.

- **Achutha Kiran Kumar, Seligman et al., *Making Formal Property Verification Mainstream*.** Titled and cited as formal verification, it alone in this corpus describes a technique-adoption program with named roles and support structures, supplying most of Chapter 24’s mentoring and process-integration material.
- **Rahman et al., *Emulation-based System-on-Chip Security Verification*.** Titled as security, it supplies the platform-and-workflow taxonomy for Chapters 17 and 18, which use it more than Chapter 23, as Chapter 23’s own note acknowledges.
- **Kern and Greenstreet, *Formal Verification in Hardware Design: A Survey*.** Titled as a formal survey, it supplies the book’s equivalence-checking theory; Chapter 15 calls it the corpus’s nearest treatment.
- **Foster, *Trends in Functional Verification: A 2014 Industry Study*.** Titled as survey data; Chapter 15 calls it the origin point for the automatic-formal-application category.
- **Hollander, Morley and Noy, *The e Language*.** Titled as a 2001 language paper; it is the historical rationale behind Chapter 8’s decomposition, Chapter 9’s constraint-oriented stimulus and Chapter 10’s extension-as-configuration.
- **Ahmed et al., *(V&V)-in-the-Loop for RISC-V Design*.** Titled as a methodology vision for one instruction-set family; it is the corpus’s only concrete account of hardware continuous-integration tiering (Chapter 12), of a multi-level environment with instruction-set-simulator co-simulation (Chapter 8), and of an FPGA validation flow (Chapter 18).
- **Bromley and Sprott, *Formal Verification in the Real World*.** A formal title carrying some of Chapter 24’s people material: time for experimentation, teamwork to prevent stalling, and accessible internal expertise.

C.11 Reading this literature critically

Each reading criterion below responds to a specific problem in this corpus.

Denominators and exclusions. *FLAG* reports coverage of 53 of 58 manually derived target properties across six open-source protocols, excluding one protocol from that count. The paper states the exclusion; summaries omitting it misreport the result.

Pass criteria and success rates. *Saarthi*’s primary success rate divides runs completing end-to-end verification by total runs, measuring pipeline completion rather than property correctness. Among the failure modes, its authors observed vacuous passes, which Chapter 6 defines as the successes of an implication whose antecedent never became true, so that nothing was checked and the result is neither a pass nor a failure. *AssertLLM*’s 89% means “passed a formal proof against known-good RTL”, after human engineers removed generations referencing unmappable signals.

Model-judged and mechanically checked metrics. *FVDebug*’s baseline table reports both, and the two metrics order the configurations in opposite ways. The model-judged metric there is best-hypothesis quality, the score that a language model awards for how closely a proposed cause resembles a known one. One configuration scored 0.783 on that metric and resolved 65.8% of failures within five attempts. Another configuration scored 0.474, the worst score in the table, and resolved 81.6%. Reporting only the model-judged metric measures output plausibility.

Baseline selection. Bennett and Eder note that random is the most common baseline here, and beating it says nothing about generalization. Fine and Ziv’s founding result is significant because it used an expert’s tuned directive file as baseline.

Publication years and measurement dates. Chang and Kundert’s mixed-signal model-creation figure, up to 80% of the effort of getting analog blocks into chip-level verification, comes from their own undated consulting experience. The paper itself is dated to 2015; the figures carry no year, and Chapter 21 says so.

Container metadata and bibliographic evidence. The book’s reference file dated Narayan and Symons’ talk to 2022 from every slide’s footer, an auto-date field re-rendered during bulk re-export. The archive puts it at 2015: wrong by seven years. A second entry’s metadata prints `Author: DVCon Europe` while the archive records it as DVCon India; that field belongs to the Word template. Timestamps, filenames and metadata authors corroborate at best.

Differences in source scope. Chapter 19 opens on one apparent conflict between two sources: the survey reports a metastability class needing a gate-level model with timing, while Litterick says back-end interference is answered by structural and timing analysis, not simulation. The comparison distinguishes exhibiting a failure from signing off the absence of that failure.

C.12 Corpus gaps

The following gaps are collected from the chapters that identify missing evidence.

- **No measured cost-per-stage series.** Chapter 1 says so, and refuses to treat the “cost grows tenfold per stage” rule as data.
- **Nothing on the social layer of a regression gate.** Chapter 12 marks its revert-window and ownership-by-commit guidance as practitioner judgment; no measurements exist here.

- **Equivalence-checking gaps concern algorithms and industrial flow.** Chapter 15 records limited algorithm coverage relative to industrial importance, with a 1999 survey the nearest treatment; it describes the industrial C-to-RTL flow without claiming source support here.
- **No security denominator.** Chapter 23: no enumeration of what an attacker might try, hence no denominator from which to calculate a percentage, and no settled definition of security coverage.
- **No study scoring generated testbenches or verification plans.** Chapter 25 states that the one corpus flow drafting a plan is graded only by downstream proof completion. The nearest adjacent evidence is negative: a 2023 survey found no reported code summarization in verification.
- **Every language-model capability figure was measured on a block, a protocol or a curated benchmark,** none on a system-on-chip. Chapter 25 names the unit-test fallacy for reading module-level pass rates as system-level capability.
- **Review mechanics are largely uncited.** Of Chapter 24's four mechanics for a productive review, only the last comes from the corpus.
- **No survey quantifies the shrinking gate-level tier.** Chapter 19 marks it as a trajectory that nothing measures.
- **Four standing failures of the field, from Chapter 26:** it does not learn from its escapes; it continues to produce estimates despite its inability to estimate; it rarely measures whether its own environments could still fail; and it has no common language for combining functional, safety and security positions at one review.

Chapter 26 recommends making gaps in verification plans explicit so they can be examined during review; the same recommendation applies to the supporting evidence.

Appendix D: Solutions to the exercises

Each chapter closes with four to six exercises of three kinds: a calculation whose inputs come from the chapter or from the book’s canonical numbers, a judgement between techniques that has a defensible answer, and the reading of an artifact such as a plan row, a log excerpt, a property or a coverage report. The solutions below give the reasoning, not only the result: the steps of a calculation with its derivation, the argument that decides a judgement, and the section of the chapter that carries the fact each answer rests on. Every solution was solved independently a second time before it was accepted; where the two readings disagreed, the statement of the exercise was made unambiguous.

Chapter 1

Solution 1.1. §1.2 puts the 2024 rate at 14 percent, so fourteen projects in a hundred reach production-worthy first silicon and the rest do not: $14 + 86 = 100$, which makes 86 in a hundred the number that pay for at least one further fabrication cycle. The section states the same quantity as a small integer ratio, and the arithmetic that connects the two is $6 / 7 = 0.857$, that is 85.7 in a hundred, which agrees with 86 to within a point; the ratio is the rounded form of the measured percentage rather than a second measurement. The rate describes IC/ASIC projects reaching first silicon in 2024. It describes no FPGA project, because an FPGA project has no metric directly comparable to a respin count, which §1.2 states where it reports the FPGA escape figure.

Solution 1.2. The two figures together entitle the slide to conclude that both populations fail at correctness at a high rate, and nothing about which discipline is weaker. Two properties bound the conclusion. First, the two headline percentages measure different things, one the fraction of projects whose first silicon was production-worthy and the other the fraction of projects that shipped production systems with non-trivial bugs; the FPGA paragraph of §1.2 states that FPGA projects have no metric directly comparable to a respin count, so no ordering can be read across the two. Second, the same paragraph states that the gap between the two percentages is smaller than the margin of error the studies declare, so even a comparison of like with like would not resolve at that precision. What §1.2 does support is the cultural diagnosis it gives directly, that many FPGA teams deliberately minimize pre-lab verification, a claim about practice rather than about relative discipline quality.

Solution 1.3. §1.3 staffs the reference SoC with four RTL designers and five dedicated verification engineers, so the project carries $4 + 5 = 9$ full-time equivalents, and the same section reports that IC/ASIC design engineers spent on average 49 percent of their time on verification in 2024. Verification therefore absorbs $5 + 4 \times 0.49 = 5 + 1.96 = 6.96$ equivalents, that is $6.96 / 9 = 0.77$, about 77 percent of the project’s engineering hours. Using the section’s own “near half” for the designers

rather than the survey average moves the result by one point and no further: $5 + 4 \times 0.5 = 7$, and $7 / 9 = 0.78$. Either arithmetic supports the claim §1.3 closes on, that well over half of all engineering hours go to verification on a project developing new IP. The dedicated team dominates the sum, contributing 5 of the 6.96 against the designers' 1.96, so the designers' verification time raises the figure without being what makes it exceed half.

Solution 1.4. §1.3 calls the 70 percent folklore, repeated from publication to publication without primary data, and puts a measured figure in its place: 57 percent of total project time on average, with wide dispersion in both directions, projects assembling pre-verified IP legitimately spending far less and projects with a high proportion of newly developed IP far more. A project assembling mostly pre-verified IP therefore belongs in the low tail and should budget below the average, not at a figure above it; the defensible planning claim is the majority of project time, with the excess over half set by how much of the design is new. The scope error lies in the third of the three 70 percent figures §1.3 collects: that one prices validation in the umbrella sense, pre-silicon and post-silicon work together, rather than the pre-silicon verification a plan of this kind budgets. The same source puts post-silicon validation alone at half, which is the figure §1.4 cites, so the umbrella number cannot be spent twice.

Solution 1.5. The absent fields are Escape analysis and Cost if escaped. Without the first, the report records what broke and not why the existing suite could not see it, so nothing forces the process change that would prevent the bug's siblings; §1.4 makes that field the reason a bug report is an economic document rather than a defect record. Without the second, the report carries no price for the escape, so a risk register prices this bug at the two engineer-days actually spent rather than at the downstream cost avoided. The omissions match pitfall 6, skipping the escape post-mortem, and pitfall 5, costing a respin as masks-plus-fab. The rest of the write-up is sound, and the found-by line is what makes it so: the 11-cycle counterexample is the evidence the severity line rests on, and it is available because the bug was caught where observability was still total.

Chapter 2

Solution 2.1. Both paths meet at the designer's interpretation, so the split leaves the closed loop of the opening intact. The reconvergence model locates what is actually verified at the point where the two paths meet (§2.1), and the two paths are the RTL and the *checks*, not the RTL and the stimulus: stimulus decides which states get visited, while the expected results decide what counts as correct, so moving stimulus authorship to a second engineer changes the visited set and leaves the verdict on the designer's reading. Writing the assertions and the reference model off the RTL is also the collapse §2.3 names in its own right, a reference reverse-engineered from what the RTL does instead of derived from the specification. The chapter's practice list prescribes the opposite split: the verification plan and the coverage model reviewed by someone else against the specification, the assertions written from the

specification text rather than the RTL, and an independent engineer owning the scoreboard’s reference model, which breaks the interpretation loop at the checks even when one person writes the stimulus. The deciding artifact is therefore the scoreboard’s reference model, with the assertions beside it.

Solution 2.2. §2.2 requires the least tolerant observer that the reference clock permits, even though no real receiver would be built that way. The mechanism is that recovery robustness and checking power move in opposite directions: the more tolerant the recovery algorithm, the less checking it performs, because a tolerant monitor adapts to data rates that are out of spec and so locks onto the very deviation it exists to detect, reporting a clean stream. Reusing the receiver IP’s algorithm imports exactly that tolerance, and fidelity to a real receiver is not the objective, since verification sets out to check the design as thoroughly as possible even where that means departing from reality (§2.2). On a 10G Ethernet MAC the same mistake takes the form of a receive monitor whose elastic-buffer model absorbs clock drift beyond what the specification permits: it accepts every frame, including the frames a compliant link partner would have rejected.

Solution 2.3. The covergroup declares four coverpoints: `cp_stride` with two bins, `cp_bnd` with three, `cp_len` with three, and `cp_ch1` with two. The cross `x_attack` therefore demands $2 \times 3 \times 3 \times 2 = 36$ cells. The axis in question is `cp_stride`, whose second bin is `neg`, and §2.2 places it in the model on reachability rather than on intended use: negative stride “is on the list not because anyone plans to use it that way but because the register field is signed, so it can happen”. That is the same argument §2.3 has the verifier make to the designer, where the operative question is whether the programming interface can express the case.

Solution 2.4. The comment names a 64-bit bus, so the largest legal `awsize` is 3 and one beat carries 2 to the third power = 8 bytes. A burst of 256 beats has `awlen` = 255, since the length field encodes the number of beats minus one, and it moves $256 \times 8 = 2048$ bytes. The property requires the offset plus that span to stay within the page, so $\text{offset} + 2048 \leq 4096$ and the largest admissible offset is $4096 - 2048 = 2048$: a burst starting there ends exactly on the boundary and satisfies the property, and any larger offset crosses it. The upper edge of the `in_burst` bin is 2048 as well, and the two quantities are not the same quantity: the first is a value of `awaddr[11:0]`, the offset into the page, while the second is a value of `bytes_to_4kb_boundary`, the headroom, which for this offset is $4096 - 2048 = 2048$ as well. What they share is their unit, the span of one maximum burst at the largest legal `awsize`, which is why the bin’s comment reads “crossing reachable in one burst, or exactly filled by one”: the bin boundary is drawn at the reach of a single burst rather than at a round address.

Solution 2.5. The summary establishes that the random environment reached every cell of the cross and that no test reported a failure, and it establishes nothing whatever about the burst splitter, because the last line records that no comparison was executed. The burst splitter is the DMA stage that splits each transfer into bursts that the AXI address rules allow. The rule it puts in question is *no coverage without checking* (§2.4): a coverage bin hit while nothing verified the outcome con-

verts ignorance into false confidence and is worse than an empty bin, so once the scoreboard line is read the coverage line becomes a liability rather than evidence. The tests line on its own matches pitfall 4, counting tests instead of checked coverage, since a growing pass rate over an unchecked design measures the testbench's tolerance and nothing else. §2.4 supplies the reading of the whole summary in the sentence it applies to the opening: when everything passes, the first hypothesis is that no check has fired.

Chapter 3

Solution 3.1. The widths sum to $32 + 32 + 32 + 32 + 32 + 16 + 8 + 8 = 192$ bits, which is the figure §3.1 reads off the table for one channel. The register set aside is `STATUS`, the read-only byte carrying busy, done and the backend error code, so one channel's programmable count is $192 - 8 = 184$ bits and the pair's is $2 \times 184 = 368$ bits, which is the exponent the chapter's opening uses when it states that stripping the read-only status bits leaves two to the 368th programmable configurations. The sixteen bits that sentence strips are the two `STATUS` bytes, $2 \times 8 = 16$. The predicate "architecturally visible" attaches to 192 for one channel and $2 \times 192 = 384$ for the pair, never to 184 or 368, because the visible count includes state that software can read but not write. The two are not interchangeable in an expression of the form two to the Nth because they answer different questions: the visible count bounds the states a checker must be able to distinguish, the programmable count bounds the states stimulus can command, and substituting one for the other silently changes what the exponent claims.

Solution 3.2. The class counts multiply to $16 \times 16 \times 10 \times 7 \times 4 \times 4 \times 8 \times 4 = 9,175,040$, about nine million, which is the figure §3.1 reads off the table; each count in that table carries a tilde, so the product is an order-of-magnitude estimate rather than a census. §3.1 calls a suite of a thousand hand-crafted tests exceptional, and one case per test gives $1000 / 9,175,040 = 1.1 \times 10^{-4}$ of the product, that is about one case in nine thousand, since $9,175,040 / 1000 = 9,175$. The remainder is what §3.1 calls the untested residue, and the section points at Chapter 1's escape data for what that residue does rather than at any further arithmetic. The exercise measures the second of the two reductions §3.1 describes, from about 10^{57} raw register states to 10^6 to 10^7 meaningful cases: abstraction removes most of the space, and a directed suite still samples what abstraction leaves at one part in nine thousand, which is why §3.1 puts stimulus automation and measurement after abstraction rather than instead of it.

Solution 3.3. The proposal degrades controllability, the first link of the three-link chain of §3.2. The mechanism is the one that section states for this bug: the buggy condition is a one-cycle coincidence between two independently timed events, and at SoC level aligning a timer interrupt with one specific instruction inside a firmware loop is nearly impossible to arrange deliberately and worse to reproduce, whereas at core level the IRQ lines are driven directly and randomization exercises the coincidence across thousands of runs. Observability is not the binding constraint, since the

ISS co-simulation oracle compares architectural state instruction by instruction and sees the corrupted flag in the cycle it commits, so the proposal trades away the link the environment cannot recover by other means. The criterion §3.2 sets is that a feature must be verified at a level where the testbench can both provoke it and see it fail, and the proposal matches pitfall 4, verifying at the wrong level, whose symptom is a block corner deferred to SoC tests where it is uncontrollable and unobservable.

Solution 3.4. The two readings are that every INCR burst the backend issued stayed inside its 4 KB page, and that the backend issued no INCR burst at all. The projection table of §3.3 states the second in its own row: a backend that hangs forever satisfies the property vacuously, because the implication never triggers and so the property never fails. The question the assertion leaves unanswered is whether a legal transfer ever completes, and §3.3 assigns that question to the scoreboard: for a data-moving design the testbench applies the specified transform to each input transaction, stores the expected output transactions in queues, and compares actual outputs against them as they emerge. The check it performs at end of test is that nothing expected was lost, and that completeness check is what separates a silent backend from a correct one. The table's other rows bound the assertion further, since it owns neither the data the split bursts carry, nor their sum against the programmed transfer, nor the read channel.

Solution 3.5. The passing regression establishes that the design and the testbench agree, not that either of them is right, which is the sentence §3.3 closes its oracle discussion on. §3.3 names the failure mode a common-mode error: every oracle encodes someone's reading of the specification, so when the scoreboard's author misreads the same ambiguous sentence the designer misread, model and design agree, the test passes, and the bug is structurally invisible to the environment. The defenses that section prescribes are organizational rather than technical: independent interpretation, with the verifier reading the specification rather than the RTL, review of the oracle itself, and treating every disagreement between model and design as a specification question whose resolution belongs in the specification text. A further oracle of the same kind is not among them because the fault lies not in any oracle's coverage but in the reading they share: an assertion or a second scoreboard written from the same sentence by the same reading agrees with the design for the same reason, so the additional projection multiplies the checks without touching the common cause. §3.3 adds the reason the resolution belongs upstream, since specification issues are among the leading root causes of the functional flaws that force respins.

Solution 3.6. The status line is entitled to claim activation and nothing more: the stimulus reached the scenario, and the plan row names no one who decides whether the result was right. The empty field is the oracle field, and the row commits pitfall 3, activation without a checker, whose cure the chapter's pitfall list states as every plan item naming its oracle, the check and not the stimulus. The oracle §3.3 assigns to this bug is the scoreboard's byte-accurate memory model, comparing final memory contents against the transform of the programmed transfer descriptor, the record that programs one DMA transfer. The reason it is the only one is that in the actual bug the emitted bursts were individually legal and the data landed wrong, so

nothing fires the assertion that watches the 4 KB boundary, the address line that AXI forbids a burst to cross. The plan row's constrained-random stimulus can visit the cell for an entire regression while the mis-split goes unnoticed, which is the shape of the third escape of §3.2, visible but unrecognized.

Chapter 4

Solution 4.1. The bug row of §4.5 reads 63 filed and 63 dispositioned, and its note splits the dispositions into 58 fixed and regressed, 4 not-a-bug after the specification was clarified, and 1 waived as a documentation erratum: $58 + 4 + 1 = 63$, so the three dispositions partition the filed set and none is left undisposed, which is what the closure strand of §4.5 demands when it asks that every open bug have a disposition. The plan-features row closes XB-01 through XB-07, while the plan excerpted in §4.2 lists six rows, XB-01 through XB-06, plus the out-of-scope entry; the table's own note says the seventh was added at plan review. That review is the requirements and plan review of §4.6, whose roster seats design, verification, software and system stakeholders. The hole it caught is the one the scenario of §4.6 records, a use case invisible from inside the block: no scenario in which the debug manager is the only active manager, issuing accesses into a quiesced system, was in the feature list, and the row added for it produced one directed test and one coverage bin that in week 16 caught an arbiter assumption. §4.6 credits the catch to its own principle that completeness needs outsiders, since requirements reviews seat stakeholders from outside the project team.

Solution 4.2. §4.2 gives the crossbar's deliverable schedule as environment skeleton 2 weeks, single-path bring-up 1 week, all-paths random with scoreboard 3 weeks, protocol assertion bind-in 1 week and coverage model 2 weeks, so the sum is $2 + 1 + 3 + 1 + 2 = 9$ weeks. The saving §4.2 reports runs from 10 to 40 percent of the overall verification schedule, which on that sum gives 10 percent of 9 weeks = 0.9 weeks at the low end and 40 percent of 9 weeks = 3.6 weeks at the high end. The sum is smaller than the schedule the saving applies to because §4.1 lists six phases while this deliverable list reaches only through the build phase and the first bring-up demonstration: the long middle of regression and coverage, closure and sign-off all sit outside it, and §4.3 calls that middle the longest phase of the lifecycle. The qualification §4.2 attaches is explicit, that the figures describe a single organization's reported experience and do not establish a ranking of lifecycle phases by return on investment, so the band indicates where the leverage is rather than supplying a coefficient to multiply a schedule by.

Solution 4.3. Stage claims license statements about verification maturity and not about RTL quality, which §4.4 states directly: a block can be at stage 2 with open bugs, because the claim is that the machinery to find and track them is mature rather than that the design is clean, and conflating maturity with cleanliness produces a dashboard of green process indicators over a red design. The inference about bug counts is unlicensed in both directions, and §4.4's own stage-2 line shows why: it claims that the bug-open curve has peaked and that closure is outpacing dis-

covery, which says nothing about the level at which the curve peaked, so the crossbar may well carry more open bugs than the DMA engine. What the crossbar's claim does license is what §4.1's scenario acts on: stage 2 means all planned stimulus and checkers are implemented, the coverage model is implemented and reporting, regression runs at its target cadence with a managed pass rate, and the block is trustworthy enough for others to build on, so the SoC lifecycle gates its own bring-up milestone on the crossbar reaching that stage. The DMA engine's stage 1 licenses only that its plan exists and has passed review, that its environment instantiates the current RTL and drives and checks real transactions, and that a smoke suite passes in regression, so nothing downstream may yet build on its results and the integration order follows from that, not from a comparison of bug counts.

Solution 4.4. §4.7 forbids exactly this extrapolation and names the reason: debug is the single largest consumer of verification engineer time, larger than testbench development, test writing or test planning, and debug effort is unpredictable and varies significantly from project to project, which invalidates the most natural estimation method, extrapolating from the last project. What carries across is the machinery rather than the numbers, and §4.7 lists it: schedule in demonstrable deliverables with linear progress so that slippage shows in weeks rather than quarters, buy schedule with planning up front where the leverage is, keep the maturity stages truthful so that a percentage cannot conceal stage 1 until RTL freeze, and protect the regression cadence and the debug loop. The two structural reasons for the spread are both in §4.7: projects at the low end lean on pre-verified IP reuse while projects at the high end carry high proportions of newly developed IP, which is why the crossbar, configured from a mature in-house pattern, and the neural accelerator, new, sit at opposite ends of the reference SoC's effort table despite comparable gate counts. The third variable is team composition, which §4.7 places inside the team and calls a first-order schedule variable that no methodology hides: individual skill spread reaches up to 10 times in productivity across engineers, and best-known methods simply not being in use is estimated to cost around 20 percent of productivity on its own, so an unchanged flow is not an unchanged effort.

Solution 4.5. The ticked line the blank field contradicts is the first, coverage closed against the plan, and the ticked line the disabled-test row contradicts is the second, full regression passing over a sustained window. Coverage closed against the plan requires, in §4.4's stage-3 wording, every exclusion written down and reviewed, and §4.5 admits declared exclusions as a feature of closure while forbidding the silent one, so the blank exclusions field is the field §4.5 forbids leaving blank: the crossbar's own summary in §4.5 records its exclusion count with the list attached, and this claim records neither the count nor a reason. Full regression passing over a sustained window fails on the claim's own next line, since §4.5 asks for all tests passing, repeatedly, across seeds rather than a single lucky green run, and the stage-2 checklist excerpt of §4.4 states the same condition as a line reading no test disabled, which this claim contradicts while reporting a pass rate that the disablement is what holds up. The chapter's In practice list requires the reviewer to treat a disabled test as an open bug against the environment, and names temporary disablement to keep the pass rate green as a stage claim without evidence; applied to the last row, that

adds two environment bugs to the open-bug count, so 7 filed and 7 dispositioned is no longer the whole set and the third tick, every open bug has a disposition, is not established either. The unticked line is not a defect: §4.4 says a block is at stage N while it works through that stage's checklist and exits into stage N+1 when every line is ticked, so a stage-3 claim with the sign-off review still open is the model being used as intended, and it is the ticks above it, not the empty box, that the reviewer must challenge.

Chapter 5

Solution 5.1. §5.5 quotes a coverage model for a 120-instruction ISA at roughly 3,600 lines of SystemVerilog, a further 9,000 lines for a 300-instruction extension, and a rate of 10 to 30 lines per instruction. The implied rates are $3,600 / 120 = 30$ lines per instruction and $9,000 / 300 = 30$ lines per instruction, so both totals sit at the top of the band rather than in its middle. At the low end of the same band the two instruction counts would give $120 \times 10 = 1,200$ lines and $300 \times 10 = 3,000$ lines, a factor of 3 below the quoted totals; what follows for quoting the totals as estimates is that they are the pessimistic end of a band presented as a single figure, so a plan that treats either as a mean over-provisions by up to that factor, and any schedule built on them should carry the band rather than the point. The conclusion §5.5 reports the source as drawing is that generation, not typing, is the only sane source for that class of model, and the section is careful to attribute both the arithmetic and the conclusion to that program rather than adopting them.

Solution 5.2. The blank field is the closure metric, and the rule that governs it is §5.2's one row, one metric: the field states the single measurable exit condition that closes the row, and a row needing two kinds of evidence is two rows, which is why §5.2 splits DMA-F06 into F06a and F06b and DMA-N01 into N01a and N01b, giving two features, four rows and four metrics. The failing field is the pass/fail criterion, and §5.2 rules out exactly this shape of sentence: it says that transfer completes is not a criterion, and that the field must state how the response will be judged correct and above all which errors to look for, so that an engineer who is not the block's expert can build a reliable check. Restated in the chapter's own pattern, the criterion reads: every destination byte equals its source byte, in order, exactly once, for every descriptor in the chain, and the done interrupt follows the last accepted write response of the last descriptor. The row also carries a feature name where §5.2 requires attributes, the dimensions and the values that matter, so the chain lengths and the sensitive values that select the interesting paths are absent, and the criterion above cannot be measured until they are named. The cross-reference the row lacks is the one §5.3 builds from the permanent identifier, tying the specification clause the feature came from to the testcases and properties that discharge it and the coverage points that measure it; §5.3 states the consequence flatly, that a feature with no cross-reference is not being verified.

Solution 5.3. The attribute list of DMA-F06a gives stride sign in three values, the 2D row's relation to the next boundary in four, burst length in three and the other

DMA channel in two, so `x_worst` has $3 \times 4 \times 3 \times 2 = 72$ cells. A 2D row is the contiguous run of bytes that makes up one line of a two-dimensional transfer, and the stride is the signed address step from the start of one row to the start of the next. §5.2 states that full coverage means 100 percent of the cells the model can legitimately reach, fewer than 72, and excludes the six maximum-length straddle cells, so $72 - 6 = 66$ cells count toward the plan row's closure. The premise the exclusion rests on is the generator's constraint solver budget, which caps a 2D row at 256 beats: a straddling 2D row therefore splits into pieces none of which can itself be a maximum-length burst. That premise belongs to a decision the team took and not to the model, since §5.2 says lifting the budget brings the cell back, and it is recorded on the plan row itself as a waiver with its justification rather than in a footnote. The same passage fixes the bug's footprint at two of the 72 cells and not three, `neg` crossed with `straddle` and the other channel active, in the one-beat and the 255-beat cells, because the 256-beat cell is one of the excluded ones. The reasons the Chapter 2 model counts fewer cells for the same feature are two refinements rather than a disagreement: its stride axis has one bin fewer, because a zero stride, with every 2D row starting at the same address, is its own splitting path rather than a degenerate case of the other two, and its boundary axis bins headroom in absolute bytes against fixed thresholds while this model bins it relative to what one burst of this 2D row would move, a frontier that moves with the burst length and that a fixed threshold cannot identify.

Solution 5.4. The row stays where it is, on the must-not-happen list of §5.1. That list carries the relevant and probable errors the environment must be able to detect, and a done interrupt raised before the last write response is accepted is an error of exactly that kind, one that software reading a status register cannot see; the designer's ground is a statement about one piece of software rather than about the design. §5.1 requires of every out-of-scope entry that it be a reviewed decision rather than an accident or an unexamined hope, written down with its review recorded, which is the form the DMA-X01 entry takes when it names a documented software constraint reviewed with design in week 3. The second question the verification lead asks in §5.4 is whether the RTL checks the condition or assumes it, and in the chapter's scenario the answer is that it assumes it; the same answer here means the row's risk rank rises rather than falls, because an unchecked assumption reachable by software the block's team does not control is where escapes breed, and §5.4 adds the companion move of a new must-not-happen row on the frontend for descriptors the design cannot honor. The lead's first question applies as well: the register interface is documented and other teams write their own descriptors, so the driver is not the only software that will ever program the block, and the interrupt can be enabled by code the driver's authors do not own.

Solution 5.5. §5.5 puts a per-feature decision in place of a uniform template, taken on two axes, the consequence of an escape and the likelihood of a bug, on top of the priority tiers that pre-draw the schedule cut line: must-have features define first-time success and get thorough verification across all configurations, should-have features get their basic operation verified with depth added only as time allows, and nice-to-have features are verified as time allows, which §5.5 notes today's schedules

almost guarantee means never. Walked across DMA-F06 the five drivers read: novelty high, since the 2D negative-stride path is new rather than silicon-proven reuse; structural complexity high, since the splitter's resumption arithmetic sits in state shared between two channels; specification quality high, since the document is silent on this case; controllability and observability low, since descriptors are programmed directly and every issued burst is visible on the backend interface; and cost of escape at the worst available, silent data corruption that no status flag reports. That ranking buys the depth \$5.5 spends when consequence and likelihood are both high, in every currency at once: two rows, a full cross in constrained-random simulation for the transport and a formal safety property for the invariant, with tighter closure criteria and a review of the checker itself. DMA-F01 inverts every answer, being re-used register logic generated with its own tests from one description through a register abstraction layer, with a fully specified access policy, complete observability and a consequence software can read back, so its row deliberately buys almost nothing: generated register tests and a review of the generator configuration. The favorable driver does not reduce DMA-F06's depth because \$5.5 states its role precisely, that good controllability and observability make depth affordable rather than unnecessary; depth is set by the product of consequence and likelihood, and both of those stay high whatever the observability.

Chapter 6

Solution 6.1. §6.2 reports a five-state machine with 14 possible transitions on which a single sequence reaches 100 percent state coverage while covering only 36 percent of the transitions, so the sequence traversed $14 \times 36 / 100 = 5.04$ transitions, that is 5 of them, since a traversed-arc count is an integer. The count checks back: $5 / 14 = 0.357$, that is 35.7 percent, which rounds to the 36 the section reports, while the next integer up gives $6 / 14 = 0.429$, that is 42.9 percent, which does not. A single sequence that visits every one of the five states must traverse at least $5 - 1 = 4$ arcs, so this sequence traversed one arc more than the minimum and either closed a loop or revisited a state once. The two things §6.2 says the metric cannot decide on its own follow from where the graph comes from: because the transitions are extracted from the implementation, the tool cannot identify unintended transitions or intended transitions that are missing, so the graph needs review against the specification, and it cannot decide which of the transitions it reports are unreachable under the protocols feeding the machine, which the section assigns to a property checker.

Solution 6.2. §6.6 declares the tensor width as `logic [8:0]`, a nine-bit field, and §6.3 gives `auto_bin_max` a default of 64, so automatic binning would slice $2^9 = 512$ values into 64 bins holding $512 / 64 = 8$ values each. The bin name §6.3 quotes for the lane boundary, `auto[16:23]`, fixes the alignment at multiples of eight, so the bin containing the architectural maximum begins at 256, since $256 / 8 = 32$ whole bins precede it, and runs from 256 up to $256 + 8 - 1 = 263$. What else falls inside that bin is the range 257 to 263, and the line that decides how the model must treat those values is `illegal_bins out_of_range` on `cp_width`, which declares the legal range to be 1 through 256 and everything outside it, the zero below as well as the values

above, a value the register checker must reject, so one reaching the datapath is a bug rather than a hole; §6.6 adds that illegal values are excluded from coverage, which is why `cp_width` still offers six bins. Automatic binning would therefore merge the architectural maximum with values that must never occur, in the same way §6.3 shows it merging widths 16 and 17, exactly the two the datapath treats differently, and both merges are the chapter's second pitfall, automatic bins on meaningful fields.

Solution 6.3. The result establishes that the modeled space was sampled and nothing about whether the design is correct, and §6.4 states the property that forces the distinction: verification aims to remove bugs, while coverage closure is only a well-defined, measurable derivative of that goal, and a derivative can be optimized directly, so a campaign whose stimulus is tuned only for bin hits can reach 100 percent with inactive checkers and no evidence that misbehavior would have been detected. The same section supplies the division of labor that the tool leaves untouched: coverage measures sampling while checkers establish correctness, and coverage methodology assumes that an orthogonal mechanism detects device misbehavior, so without output comparisons a full report merely counts jobs. The stopping criteria of §6.8 the overnight result does not by itself satisfy are three, and none of them is a percentage: a full reviewed model, a current exclusion file, and a stable period without model refinements, since both stimulus and model evolve during closure and §6.8 treats the discovery that new attributes are needed as part of that refinement. The reason §6.8 gives for tightening the checking review is that automation makes machine-optimized 100 percent arrive faster and with even less human examination of the scenarios, which is what turns §6.4's requirement into a gate: the coverage-model review must include the checkers observing the same events and identify, for every cover-point, who detects misbehavior in that scenario.

Solution 6.4. The latched report is evidence that the DMA contended for the weight-stream SRAM banks during the job, which is the situation the chapter's opening failure requires; the live report is evidence only that the DMA happened to be busy at the instant the job retired, and §6.6 says it fills the bin either way, regardless of whether the DMA contended during streaming. The decision that separates them is the third of §6.5's three, the choice of sampling, and within it correlation time as a first-class design decision: sampling must occur when the values are jointly valid and stable, which is why the model latches the contention flag across the weight-streaming window and samples all four attributes once, at job completion, and why §6.5 says sampling at an irrelevant instant gives each filled bin a misleading meaning. §6.6 draws the conclusion from the two reports being indistinguishable: the report cannot distinguish this defect from the original model, so functional coverage is exactly as good as the model, a model can be judged only by reading it, and coverage-model review against the vplan and the checkers must therefore be a flow gate rather than an optional step. The cell the opening failure occupies is `x_prec_width_dma: (PREC_2B, single, contended)`, the 2-bit precision crossed with the degenerate single-pixel geometry and contention, and the live-sampling model can fill that cell without the failure's situation ever having occurred.

Solution 6.5. None of the three lines covers the combination. `ifetch_wr` requires `cp_mgr.core_i`, `dbg_burst` requires `cp_burst.fixed` or `cp_burst.wrap`, and `rom_wr` requires `cp_sub.rom`, while this bin is a debug-manager INCR write to the APB bridge, which none of the three selects; the combination is therefore one of the 82 that §6.7 counts as justified and reachable, and what follows is that the record's stated reason is false as written rather than merely thin, since the shaped cross makes a falsifiable claim about intended reachability and this record contradicts it. The two unusable fields are the kind and the owner, and §6.9's second rule is the one that requires both: it classifies exclusions by kind, with different owners and expiry, as structural, tied to parameters and expiring on parameterization changes, specification, covering forbidden combinations and expiring on specification revisions, or risk-accepted, covering reachable cases judged too costly, which never expire and therefore require the most senior signature. §6.9's fourth rule is what an unowned entry cannot survive, since the exclusion file lives with the RTL and the vplan, is re-audited when either changes, and appears in the sign-off review as a first-class document, none of which an unassigned entry can be carried through. §6.9's first rule condemns the reason field independently: every exclusion carries its reason inline, in specification vocabulary, so that the review is a code review, and not reachable names no clause. If the team's real ground is a restriction in the sequence library, then §6.2's distinction applies, the one it draws between the crossbar's two stuck identifier bits: the bin is unreached rather than unreachable, as with the bit that appears structurally unused only because the managers never index past their assigned values. The classification then follows by elimination from §6.9's three kinds: not structural, since no parameter of the crossbar makes the combination impossible, and not specification, since nothing in the protocol forbids a debug write to the APB bridge, which leaves risk-accepted, a reachable case left uncovered by choice, and §6.2 requires that record to carry the owner and the expiry that the sequence library's next revision would retire. §6.2 also states the standard the claim of unreachability would have to meet, that misclassification is costly so the claim must name the clause that establishes it, and its companion bit is the warning against assuming a stimulus restriction, since that one was a testbench wiring bug line coverage could not see.

Chapter 7

Solution 7.1. §7.2 reports the pruned regression falling from 46.6 to 38.2 hours of compute, so the saving is $46.6 - 38.2 = 8.4$ hours, that is $8.4 / 46.6 = 0.18$, about eighteen percent of the original compute. The price the same sentence attaches is slightly lower reported coverage on the pruned points: the pruning drops cover-points already hit in the previous run, so the run no longer measures what it no longer collects, and the reported percentage on those points falls even though nothing about the design changed. §7.2 requires that trade to go into the sign-off record rather than living only in the script that implements it. The reason the exchange is admissible at all is that the loss is bounded and deliberate, and the reason it must be recorded is that a later reviewer reading a lower coverage figure cannot otherwise tell measurement pruning from lost closure.

Solution 7.2. The dashboard of §7.5 gives the accelerator 37 of 38 plan rows discharged and the DMA 44 of 47, so the undischarged shares are $38 - 37 = 1$, that is $1 / 38 = 0.026$, that is 2.6 percent, and $47 - 44 = 3$, that is $3 / 47 = 0.064$, that is 6.4 percent. Those shares are not what separates the two verdicts. Checklist item 1 of §7.4 asks for every plan row *discharged or dispositioned*, so the count of undischarged rows is not the quantity the item measures at all: the accelerator’s single row carries a written disposition, a waiver whose five fields §7.5 records in full, while §7.5 states of the DMA’s three rows that all three were opened in week 35 in reaction to the find and none carries evidence yet. The field that separates them is the expiry: the accelerator’s waiver names a date, the cross closes in SoC regression by week 41 or the block returns to review, which is what makes the decision conditional rather than deferred, and the DMA has no such condition to offer. §7.5 says exactly this of the DMA, that checklist item 1 alone therefore holds it.

Solution 7.3. §7.3 does not let the review read that tail as clean. The row that decides it is the last of the six-question table, where the last bugs were found: scattered, low severity and in modeled corners on the clean reading, clustered, severe and in a corner the plan never modeled on the blind reading, and the block described here matches the second pattern on that row while matching the first pattern on the other five. §7.3 gives the reason the one row overrides the five: because bugs cluster, a severe bug found late in a corner the coverage model did not describe is one sample from a region of the state space the instruments were not watching, so the find is evidence about the model as well as about the design. The two changes are therefore to extend the coverage model and the stimulus around the find, and then to require the discovery curve to flatten again under the expanded model, rather than fixing the bug and declaring the tail restored. §7.5 applies the same rule to the DMA, holding the block and requiring the curve to flatten under the new instruments before the checklist is reviewed again.

Solution 7.4. Item 2 does not admit the waiver. The property the reviewed exclusions carry and the proposal lacks is that they are true don’t-cares: §7.4’s checklist excerpt records the crossbar’s 14 exclusions as all reviewed, true don’t-cares, expiry “none”, and item 2 states that only a genuine don’t-care writes “none” there. A waiver taken to meet a deadline is the risk-accepted kind, which item 2 says is not a don’t-care and therefore requires a real expiry date and a senior signature; item 2 also names the consequence of the shortcut directly, that a waiver with an empty expiry is permanent by accident. The proposal matches pitfall 4, waivers without owner or expiry, whose cure is that every waiver retains all five fields and that sign-off re-reviews every waiver rather than counting them. The argument from precedent fails on the exclusion kind rather than on the field’s value: the same three characters mean *nothing to revisit* on a don’t-care and *no date was chosen* on an accepted risk.

Solution 7.5. The absent answer is question 4, misleading completion metrics. §7.5 supplies it: the DMA reported 96 percent coverage, a 99.2 percent pass rate and flat discovery, and the review’s own reading is that the green dashboard described coverage around an unseen region, because abort and resume had no plan row, cover-point or cross. In the region of the last find coverage sat at 71 percent, and §7.5

states why even that figure could not have raised the alarm: 71 percent cannot measure the hole itself, because the hole has no bins. The pattern is the one §7.6 names in question 4, missing coverage bins together with a starved bug curve interpreted as clean, and the starvation is documented in the same dashboard row, whose fresh-stimulus column reads no while the two engineers were in debug. What the post-mortem loses while question 4 stays blank is the audit of the instruments: questions 2 and 3 fix the stimulus and the checking for this class of bug, and question 5 files the new plan rows, but nothing in the record then says that the dashboard reported completion over a region it had no bins for, so the next block with the same shape of coverage report gets read the same way. §7.6 requires that every change listed under question 5 carry an owner and be filed into the next project's planning inputs, which is what turns the post-mortem into rows of the next plan and the next sign-off checklist rather than a lessons-learned document.

Chapter 8

Solution 8.1. §8.6 gives the crossbar five managers, four subordinates and ID width 4, so a manager-side ID field takes $2^4 = 16$ values, and the Step 4 key (manager port, direction, manager-side ID) addresses $5 \times 2 \times 16 = 160$ queues. The `stream_key` function of the same step confirms the layout: the ID occupies the low four bits, the direction bit 4 and the manager the three bits above it, so the encoding is injective and the 160 live keys sit inside $8 \times 2 \times 16 = 256$ addressable slots. The rejected two-coordinate key (subordinate, ID) addresses $4 \times 16 = 64$. Step 4 holds that the shorter key fails all three concurrency sources, and each failure has its own reason: the key carries no manager coordinate, so the five managers issuing at once collapse into whichever subordinate they reached; it carries no direction coordinate, so `ARID` and `AWID`, which name unrelated streams, merge into one space; and its identifier coordinate is therefore not per port and per direction, so the ordering the protocol guarantees, per-ID ordering within one port and one direction, is not the ordering one of its queues enforces. The signature Step 4 attributes to two managers holding the same identifier at once is a mismatch that disappears on a seed change rather than a reproducible bug: manager 1 and manager 2 may both have ID 3 outstanding to subordinate 0, and one queue then matches one manager's expectation against the other's response, under some arbitrations only. The merged identifier spaces carry a second and distinct signature, a read expectation popped by a write response.

Solution 8.2. Each of the five manager ports issues a 4-bit ID, so $2^4 = 16$ identities per port and $5 \times 16 = 80$ distinct stream identities per direction. Step 4 derives the subordinate-side field from the same two figures: five managers need three manager-identifying bits, and $4 + 3 = 7$, so the widened field encodes $2^7 = 128$ values, of which the remap uses 80. The map is therefore injective, and the widening exists for that reason, to avoid falsely serializing different managers' same-ID transactions, but which of the 128 encodings a given manager and manager-side ID land on is the design's ID-remap policy. Step 4 states the consequence: keying at the subordinate side is sound only with the widened ID, which obliges the predictor to reproduce that

remap policy exactly, so the checker acquires a second implementation of a design decision and a disagreement about it becomes indistinguishable from a routing bug. Manager-side keying is preferred because the key stays computable from what the observer sees: the monitor reads the port it is attached to, the direction and the ID the manager issued, and no part of the key depends on modeling the design.

Solution 8.3. The comparison establishes that the environment's own responders replied as the environment told them to, and §8.6's Step 3 says so directly: comparing against a subordinate-side response monitor checks the environment's own work. What it cannot establish is the crossbar's return path, because read data flows from subordinate through crossbar to manager, so a comparison taken before the crossbar cannot show whether subordinate 0's read data reached manager 2 rather than manager 3 in ID order. This is why Step 3 places the return-path check on the manager side and Step 4 keys it there. Of Step 0's verdicts, the two the scoreboard owns lose their mechanism: data integrity per ordered stream, and completeness, meaning nothing lost and nothing invented. Routing correctness survives, since Step 3's forward-path check is the comparison that legitimately uses the subordinate-side request monitor, and protocol legality survives with Step 2's per-interface checkers. The proposal therefore does not weaken the environment evenly: it removes exactly the verdict that only a manager-side observation can produce, and leaves an environment that still reports green.

Solution 8.4. `report()` returns the conjunction of its three zero tests, so with the leftover field nonzero it returns 0 and also raises an error naming the count, even though the orphan and mismatch counts are 0 and every one of the 256 responses that arrived matched its expectation. The behavior the nonzero field records is data the design did not deliver: §8.6's fourth detail states that a response that never arrives produces no mismatch, only a queue that never empties, so `leftover()` is what makes an otherwise silent failure visible, and §8.3 gives the general reading, that data remaining at simulation end was lost in the design. What the count leaves undetermined is whether that loss is an error: §8.3 says so in the same sentence, and gives the check that resolves it for an ordered stream, confirming assumed drops against the design's own drop-count statistics. The count also says nothing about where in the run the loss happened, which is what the capture index and prediction time of Step 6 are for. An environment without the field matches pitfall 4, no end-of-test completeness check, whose symptom is exactly this shape: every produced transaction matches, so the test passes while missing transactions go unnoticed.

Solution 8.5. The substitution costs the detection of undriven data. §8.6's fourth detail gives the rule: logical inequality on operands containing `x` or `z` yields `1'bx`, and `if` reads that value as false, so with `!=` a beat queue containing a single undriven beat takes the else branch and scores as matched. The failure made invisible is therefore a beat the design never drove, and it is made invisible in the direction that matters: the check reports a match rather than an error, so nothing in the log distinguishes it from a clean run. The case comparison `!==` compares `x` and `z` as values and reports the difference, which is why the scoreboard uses it. The other guard of the same step is `$isunknown`, applied to the ID at both entry points, and it defends

against the same class of value in the key rather than in the payload: `int'(id)` is a two-state cast, so without the guard an unknown ID collapses to 0 and the transaction is filed under a real key, corrupting the queues of that stream. Both guards exist because a two-state view of a four-state value silently substitutes a legal answer for an unknown one.

Solution 8.6. The substitution costs the environment its only golden oracle. §8.7 states the scope precisely: the descriptor model is golden because data-mover transfer semantics are the specification, and because it implements them at byte-movement abstraction rather than burst abstraction, so it predicts only where bytes end up and excludes splitting altogether. The burst-splitter model cannot inherit that status because it predicts those split decisions, which makes it a transfer function, and §8.4 states what a transfer function is: it does not exist a priori, is not golden by definition, and will contain as many errors as the design itself, so a disagreement first requires deciding which of the two implementations misread the specification. §8.7 draws the conclusion for this block, that the two models for one IP differ in the abstraction level each commits to, and that omitting the splitting details is what keeps the descriptor model maintainable. The over-specification §8.7 warns against is checking exact split sequences: doing so turns every legitimate change to the burst splitter's arbitration into a regression failure, so the checks it prescribes instead are that bursts stay within one 4 KB region each, cover the descriptor exactly once in aggregate and match byte counts, with coverage recording which split shapes appeared. Pitfall 5 names the same defect in general form, the model that predicts arbitration order, latency or burst decomposition and breaks the regression on legal design changes.

Chapter 9

Solution 9.1. §9.3 writes the constraint as `s -> d == 0` over a 1-bit `s` and a 32-bit `d`. When `s` is true the implication forces one solution, `d == 0`; when `s` is false `d` is unconstrained and contributes 2^{32} solutions, so the legal set has $1 + 2^{32}$ members, and `s` is true in exactly one of them. The value `d == 0` occurs in two of those members, once with `s` true and once with `s` false, so under uniformity over legal combinations its probability is $2 / (1 + 2^{32})$, which §9.3 calls effectively never. Adding `solve s` before `d` leaves the legal set identical and only changes the order in which the solver picks: `s` is chosen first over its two values, so `s` is true with probability $1 / 2$ and then `d == 0` is forced, and `s` false still yields `d == 0` with probability $1 / 2^{32}$, giving $1 / 2 + (1 / 2) \times (1 / 2^{32}) = 1 / 2 + 1 / 2^{33}$. That second term is the whole content of the word `slightly`: the figure exceeds one half by the chance that a freely chosen `d` happens to land on zero, which is why §9.3 writes slightly over 50 percent rather than exactly half. Because the legal set is unchanged, §9.3 notes that ordering, like `dist`, cannot cause the solver to fail.

Solution 9.2. The DMA's data path is 64 bits, so $64 / 8 = 8$ bytes per beat. The row starts $4096 - 4088 = 8$ bytes below the frontier, so the first piece is $8 / 8 = 1$ beat; the remainder is $4104 - 8 = 4096$ bytes, that is $4096 / 8 = 512$ beats, which exceeds

the 256-beat maximum burst and therefore splits again into 256 + 256. One piece of a straddling row is thus a maximum-length burst, so the combination of the `straddle` bin with the `max` bin becomes reachable, which is exactly the column §9.3 had shown to be empty. The premise the example lifts is `c_solver_budget`, which caps a row at 2048 bytes, that is $2048 / 8 = 256$ beats, one maximum burst and half a page; §9.3 states that a capped row reaches past at most one frontier and splits into exactly two pieces whose beat counts sum to at most 256, each at least one, so neither piece can be 256. The cap is an engineering choice about solve time rather than a property of the design, so §9.3 requires two things on vplan row DMA-F06a: that the six exclusions be recorded as depending on policy and revisited when the budget moves, and that the argument be marked destination-side, since `c_no_page_cross`, `c_addressable` and `dst_row[]` all speak about writes. Chapter 5's rule that 100 percent means 100 percent of the cells the model can legitimately reach is what makes the bookkeeping mandatory rather than optional, and §9.3 names the failure it prevents, a sign-off that quotes 72.

Solution 9.3. The weights had no effect because they were written over a raw field rather than over the quantity the coverage model measures. §9.3 shapes the expression the coverpoint samples: `c_frontier` distributes where row 0 ends modulo 4 KB, which is the relation `cp_boundary_dist` classifies, rather than the destination address by itself. The reason the choice matters is that the coverpoint bins a relation between the address and the row length, so a distribution over the address alone leaves the length free to move the relation anywhere inside a bin, and the report is unmoved for the same reason pitfall 5 gives, weights added, report unmoved, with its cure to write the distribution over the same expression the coverpoint samples. Two limits remain even when the constraint is written correctly. First, it targets one bin of four and not all four: §9.3 states that the third range maps onto `straddle` while the first two land somewhere among the remaining bins without choosing which, because those bins are classified against a burst's worth of headroom rather than against the frontier. Second, the expression names row 0 only, while `cp_boundary_dist` samples every row, so the shaping acts on the first of up to sixty-four rows; §9.3 adds that coverage reports are what establish whether two distributions over overlapping expressions achieved their intended effect, since the standard leaves the interaction between `dist` and solve order undefined.

Solution 9.4. The insertion broke random stability. §9.4 gives the mechanism that makes a seed reusable: the generator is localized to threads and objects, each class instance receives its own generator seeded from the thread that creates it and each new dynamic thread from its parent, so the values one object or thread produces are independent of the others. Stability then holds only as long as object creation, thread creation and random-number generation happen in the same order as before, and a draw inserted at the head of a sequence shifts every later draw in that thread, which is why §9.4 says one such call can change an entire regression. The call had to be appended after the existing ones, at the end of the code block, which §9.4 states preserves the earlier sequence and keeps previously created work reproducible. The same rule governs the two other kinds of addition, new objects and new threads, for the same reason: both consume a position in the creation order that later construc-

tions inherit. Hierarchical seeding is what fixes the scope: because each new thread is seeded from its parent, manually seeding one root thread pins a whole subsystem, which is the unit a reproduction command can therefore fix.

Solution 9.5. The shape of the report already answers check 1, checking the generator. §9.3 states the discriminator: a whole axis value absent from every combination it appears in, rather than scattered missing cells, indicates a disabled stimulus dimension, and the report shows exactly that, one value of one axis empty while every value of the other three axes is hit. The report also carries the second half of check 1, that a functional-coverage gap can indicate a fault in the generator or the environment rather than in the design, and the header line supplies the corroboration by recording no test failures: nothing is complaining, which is the signature of stimulus that never arrives rather than of behavior that misbehaves. For the axis at fault, check 2 asks which generator variable changes each axis, and §9.3 records what it finds for `cp_boundary_dist`: only a pinning constraint exists, which is `c_no_page_cross`, so no variable of the generator is free to move that axis at all. An owner and a due date is the wrong response because it treats the empty block as a difficult corner, which is the misreading §9.3 records as the outcome of this report, the block receiving an owner and a due date and remaining unresolved. The right response is check 3, requesting the relation that defines the missing cell so that a randomization failure names the conflict, and then removing the constraint, since §9.3 shows the request satisfiable in the base descriptor class and unsatisfiable in the over-constrained one. Pitfall 6 names the variant to avoid while waiting, raising the seed count against a hole that constraints have closed.

Solution 9.6. The report describes permitted behavior, not a defect. §9.6 states that `parallel` guarantees a synchronized start and the absence of any scheduling dependency across branches, and that two executions with no dependency between them may or may not overlap in time, from which §9.6 draws the conclusion directly: a contention scenario may produce no contention. The proposal is worse than ineffective, because §9.6 states that capacity constrains generation rather than imposing a queueing delay: the reference SoC has two channels, the pool is sized accordingly, and reducing it to one makes the activity illegal, so a scenario that would over-subscribe the pool is not generated at all rather than being generated and serialized. The construct that permits an entirely sequential order is `schedule`, which §9.6 says allows any legal order, including branches run one after another, so it widens rather than narrows the set of orders the tool may pick. Pool depth therefore acts on generation in both directions: it bounds how many channel-locking actions may hold a channel at one time, each with a distinct instance, and a request beyond that bound removes the scenario from the space instead of delaying it. Getting the overlap the team wants belongs elsewhere, in the launching arrangement rather than in the pool, which is the same distinction §9.5 draws when it says constraints specify relations but do not schedule execution.

Chapter 10

Solution 10.1. §10.2 makes an active agent a group of three components, a sequencer, a driver and a monitor, and the crossbar environment holds $5 + 4 = 9$ of them, so at block level it builds $3 \times 9 = 27$ leaf components, nine of each kind. Passive mode suppresses the construction of the sequencer and the driver and leaves the monitor, so the SoC environment of §10.8 builds 9 and $27 - 9 = 18$ components cease to exist. The instance override of §10.3 names one full instance path, so it replaces 1 driver and the remaining $9 - 1 = 8$ agents keep the standard one; a type override of the same pair of types matches every construction of that type and would therefore replace all 9, which is the difference §10.3 draws between targeting an instance and applying a policy. After the switch to passive neither form replaces anything, 0 in both cases, and the mechanism that fixes this is the moment of resolution: §10.3 states that the factory resolves an override when the child is constructed, so an override of a component nobody constructs is never consulted. The four responding agents are inside the nine and are configured active at block level, because by Chapter 8’s criterion a component that answers the crossbar’s requests under test-bench control is a driver rather than an observer.

Solution 10.2. The sequencer is using the default mode, plain FIFO, and the property that produces the delay is that plain FIFO grants in arrival order and reads no priority at all, so the three mainline sequences queued first are served first however the interrupt sequence is ranked. §10.5 lists six modes and identifies the two strict ones, strict FIFO and strict random, as the only modes that grant the highest-priority request first; of the other four, the weighted mode grants randomly by weight, the plain random mode grants randomly, plain FIFO grants by arrival, and the user-defined hook grants by whatever the project writes. This is pitfall 3 of the chapter, and its cure is to select a strict mode rather than to raise the priority further. Of the two further controls, prioritization is the less disruptive and `grab` with `ungrab` the more so: prioritization leaves every other sequence eligible for the grants the interrupt sequence does not take, whereas a `grab` gives one sequence exclusive use of the sequencer and stalls the rest for as long as it is held, which is why §10.5 presents the `grab` as the canonical interrupt-service pattern and prioritization as the gentler alternative. A sequence started with a parent sequence as its second argument takes that parent’s priority rather than the root default of 100, so a child of the virtual sequence of §10.5 runs at the virtual sequence’s priority and the parent’s `pre_do`, `mid_do` and `post_do` hooks fire around it.

Solution 10.3. `update` compares each register’s desired value against its mirrored value and drives bus cycles only where the two differ (§10.6). If `set` has given every register of the block a value unlike its mirror, the comparison holds throughout and the cost is one write per register: 40 registers differing = 40 front-door write cycles, exactly the cost of writing the block register by register. If `set` has left every desired value equal to its mirror, the comparison holds nowhere: 0 registers differing = 0 write cycles, and `update` returns without touching the bus. The count is therefore fixed by the number of registers whose desired and mirrored values disagree and not by the size of the block, which is what makes the model cheaper than forty writes in

the ordinary case where a test changes a handful of fields. On a volatile field the write that can be wrongly suppressed is the one the test asked for: §10.6 states that a volatile field can change without the model seeing it, so the mirror may already be stale, and a desired value that happens to equal a stale mirror compares equal and produces no cycle while the design holds something else. The property at work is the one §10.6 attaches to the mirror itself, that it records only what the model believes from the accesses it has seen, and nothing entitles it to be right about a field the design maintains on its own.

Solution 10.4. The model needs explicit prediction, which adds a predictor per bus interface, fed by that interface's monitor and looking up the register behind every observed address; §10.8 names explicit prediction as the fix for the register model at system level and §10.6 gives the reason, that an implicitly predicted mirror is updated only after operations the model issued itself, so firmware writes arriving through the crossbar and the APB bridge never reach it and the mirror is wrong within microseconds. The block-level check carries two defects, and only one of them is repaired by changing the prediction mode. The first belongs to §10.6 and was already present at block level: the overrun field is volatile, so comparing it against the mirror without a fresh read is not a check at any level of hierarchy, since the model has no standing to believe the value it last saw. The second belongs to §10.8: the check inherited the assumption that every access to the register originated in the environment, firmware now originates them, and the source of truth therefore moves from what the environment issued to what the monitor observed. §10.6 refuses the mirror the scoreboard role on a separate ground, that a mirror can predict a register the design does not update but cannot judge whether an updated value is right, so model and design may agree on a value both got wrong. §10.5 adds that a scoreboard keyed on the subordinate side loses its key when two managers write under one identifier: the crossbar widens identifiers on the way out, so the two writes arrive at the subordinate under different ones and the key no longer groups what it was meant to group; the interleaving check is unaffected, because it reasons about the order of beats on the write-data channel and never inspects an identifier.

Solution 10.5. The policy is `"W1C"`, write-one-to-clear: a written one clears the matching bit, and a read leaves the field alone (§10.6). The two statements the descriptor makes are about different things. `volatile` set to one speaks about the mirror: the field may change where the model cannot see it, so the mirrored value carries no entitlement and a comparison against it without a fresh read establishes nothing. The access policy speaks about prediction: the model computes the post-write value from the policy, so if the real design also clears the flag when software reads it, the register is modeled wrongly, the mirror diverges at every read, and the resulting mismatches are charged to the model rather than to the design. The ignored argument is `is_rand`, passed as 0 in the descriptor: §10.6 records that for any predefined policy outside the six the class reference lists, and `"W1C"` is outside that list, the library switches the field's randomization off whatever the argument holds, so the `rand` qualifier on the declaration records intent and editing the argument alone would not restore randomization. The test §10.6 builds from the two access paths reads the field through the front door, `peek`s it through the back door and

compares the two: in the canonical UART bug the front-door read followed by the clearing write leaves the model believing the flag is down while the back-door `peek` shows it still raised, and that disagreement is the firmware lockup made visible in two lines.

Chapter 11

Solution 11.1. In a block-level environment for the DMA engine the clause is an `assume`: the responses arrive at the DMA from outside, so the rule constrains the environment and the directive records where the obligation sits rather than changing what gets checked. In a block-level environment for the crossbar the same text is an `assert`: the crossbar issues those responses, so the clause is an obligation on the design being verified. §11.3 attaches this reason to the practice of writing a module's input rules and output guarantees explicitly, that an integration mistake then shows up as an assumption violation on the first system-level run, at the boundary where the two parties disagree, instead of as a symptom somewhere downstream. To a proof engine the two directives are not symmetric: an assertion is either proved or answered with a counterexample, and it is proved over the traces that survive the assumptions, while an assumption is never itself checked and instead removes traces from consideration. An assumption stronger than the real environment therefore deletes legal traces, and the assertions are established only over what is left, so the proof is a statement about a design narrower than the one that will be built; §11.3 makes that asymmetry the reason for reviewing the directive together with the property, and Chapter 14 turns it into assumption hygiene.

Solution 11.2. The length field on the maximum address carries one less than the beat count, so its value is $256 - 1 = 255$, the constant `c_max_burst` matches. The checker counts only beats before `WLAST`, so at the tick that last beat is accepted `w_beats` holds $256 - 1 = 255$ as well, and `w_beats != aw_len_q[0]` is false with 255 on both sides, meaning no error is flagged. A burst of one beat has a length field of $1 - 1 = 0$, and its single beat is already the last, so `w_beats` was never incremented and still holds 0; the comparison is false again with 0 on both sides. The identity behind both cases is that beats before the last = beats - 1 = the length field, so the queue of length fields and the counter of beats before the last are two names for one quantity and the encoding needs no correction anywhere in the checker. §11.5 declares `w_beats` one bit wider than the length field, `logic [8:0]` against the field's `[7:0]`, so that a manager driving beats past its own `WLAST` runs the counter past the maximum instead of wrapping it back onto a value the queued head would accept; a counter of the field's own width would return to 0 after the maximum burst and report an illegal burst as a legal one.

Solution 11.3. The parameter must be 7. A subordinate port carries the widened identifier, four bits at a manager port against seven at a subordinate one, and §11.5 binds the checker at 4 on the manager ports and at the wider value on the subordinate ports for that reason. If the parameter keeps its default the checker elaborates with a 4-bit `awid` and `bid` against 7-bit signals, and the connection fails elaboration

rather than misconnecting in silence, because `.*` matches only names that are also of equivalent type; §11.5 adds that catching this needs elaboration, since lint on its own does not resolve a `bind`. Both empty covers leave `a_w_burst_len` untested, in two different ways. `c_w_overlap` would have witnessed a second write address accepted while an earlier burst was still unfinished on the write-data channel, which on a subordinate port is the precondition of the canonical crossbar bug; the assertion has run on every one of those weeks and would still catch a short or long burst, but it has never once seen the interleaving case it exists for. `c_max_burst` would have witnessed an address requesting the maximum burst, so the checker’s beat-counting arithmetic has not been exercised at the length where wrapping is possible. §11.5 admits two readings of a cover at zero: no test has driven the design into the situation, which makes it a stimulus problem, or the design cannot reach the situation at all, which makes the property misformulated or points at something in the design blocking an expected state. The chapter assigns the first reading here and names the stimulus at fault, a bring-up library that runs one manager against one subordinate and then moves to the next pair, which arranges neither two managers with writes in flight at one subordinate nor a request for the longest burst. The two nonzero covers are the contrast worth reading beside them: `c_aw_stall` nonzero is what entitles the clean record of `a_aw_stable` to count as evidence instead of vacuity, and `c_same_id_pair` nonzero at this binding point concerns one manager’s own writes under one widened identifier, since a cross-manager collision cannot be seen here at all.

Solution 11.4. `ok |-> !err` succeeds vacuously on every tick where `ok` is false, and `err |-> !ok` succeeds vacuously on every tick where `err` is false, so two formulations that are logically equivalent report a clean run under opposite conditions and the choice between them decides what a clean run is worth (§11.7). The rule §11.7 gives is that the antecedent should be the rare, deliberately constructed condition, which turns a non-vacuous pass into evidence and an empty cover into an action item. With `err` rare, the formulation to keep is `err |-> !ok`: its antecedent cover counts the occasions on which the interesting condition was actually built, and a run of nothing but vacuous successes then shows up as a visible stimulus gap. The other formulation would be triggered by every idle tick in which `ok` happens to be low for unrelated reasons, and would accumulate a spotless record without the reviewed case ever arising. Where the antecedent is ordinary traffic, §11.7 holds a bare antecedent cover insufficient and asks instead for a witness that implies the antecedent and isolates the interesting case, or for the coverage model Chapter 5’s plan pairs with the boundary property. The model of that stronger witness is `c_full_bypass` of §11.6, which does not record that a push occurred but that a push was accepted at full because a pop retired in the same tick; that is precisely the exception clause of `a_no_overflow`, and a zero count there says the exception has never been tested.

Solution 11.5. 89% of 56 properties is $0.89 \times 56 = 49.84$, so about 50 of them were counted correct in both syntax and function and $56 - 50 = 6$ were not. The scored set is not the generator’s raw output: §11.8 records that human engineers first discarded the generations referring to signals the tool had failed to map, so the count entering the score has already had its unmappable cases removed, and the yield of

the generator itself is lower than the percentage suggests. The definition of correct behind the score is that the property passed a formal proof against RTL believed good, which measures agreement with an implementation rather than with the intent the specification carried. That definition admits the class §11.7 is about, the property whose antecedent is unreachable: it passes a proof on bug-free RTL without ever being in a position to fail, and so scores as correct while checking nothing. §11.8 draws the consequence rather than a number from this, that the result establishes no trend and that its criterion does not establish that a property checks the intended behavior, which is why generated properties belong in the drafting stage under the same review as hand-written ones.

Chapter 12

Solution 12.1. The proposal breaks two of the three smoke admission rules of §12.1 and satisfies the third. Fresh seeds break the fixed-seed rule, and §12.1's reason is that randomization can make the gate unreliable: a verdict that moves with the draw can no longer separate a bad commit from an unlucky seed. The unexplained failures break the zero-known-failures rule, and §12.1's reason is that a permanent failure prevents discrimination: a tier that is red whatever happens carries no information about the change that triggered it. The rule the test does satisfy is the runtime ceiling, since it finishes inside the tier's budget. Before any test enters a gating tier, §12.5 requires a repeat-N stability check, the same test run repeatedly under identical manifests to establish that its outcome does not move; §12.5 cites a published flow that runs an operating-system boot eight times for exactly this purpose, because that test cannot be parallelized. A confirmed flake belongs in quarantine, a named list whose members are still run and still reported but do not gate, and §12.5 requires each entry to carry an owner and a date, because a quarantine list that is never resolved preserves the instability instead of removing it.

Solution 12.2. The empty field is `tool`, and §12.2 states the requirement it fails: the field needs the patch level and not only the major version. Its absence permits a flake that no rerun can account for, because two runs whose manifests are identical may then have executed on two different simulator builds; §12.5 files this under provenance drift, noting that random stability covers randomization-related source changes and says nothing about a different build of the simulator. Step 3 of §12.5's worked example is where the same omission surfaced: rerunning the nine failing manifests reproduced six of them and not the other three, which the recorded manifests made impossible, and the farm turned out to be running two patch levels; pinning `tool` to the exact build made all nine reproduce deterministically. What that step established was provenance rather than the race itself, and §12.5 is explicit about why it had to come first, since without it every later experiment would have been unrepeatable and the intermittent failures would have been attributed vaguely to the design. The field the deduplication key of §12.4 must exclude is `seed`. A key carrying the seed makes every failure unique, which is over-specific classification and defeats the reduction the key exists for, in the way §12.4 illustrates with 37 failures spread over 34 buckets costing about as much work as 37 separate failures.

§12.4 keeps the seed in the manifest instead, and separates the two uses: the rule that a seed travels with a failure is about the failure report, while the deduplication key has to leave it out.

Solution 12.3. At one failure in 444 runs, 9 failures need $9 \times 444 = 3,996$ runs, which is the size of the campaign itself. The nightly tier contributes 260 DMA runs a night, so $3,996 / 260 = 15.4$, about fifteen nights of nightly DMA runs to gather the evidence the campaign gathered once. The campaign cost $4,000 \times 11$ minutes = 44,000 minutes = 733 CPU-hours, and on the reference SoC's 24 concurrent slots that is $733 / 24 = 30.6$ hours of wall clock, comfortably inside a weekend window of 48 hours. The comparison is §12.5's own reason for running a campaign rather than waiting: a rate near one in 490 is difficult to measure at 260 runs a night, because the nightly needs a fortnight to accumulate what a weekend campaign accumulates once, and those failures arrive one at a time across a fortnight of morning queues, where each was closed as the flaky one before the next appeared. The campaign turns a rate that looked like noise into a measured rate with a denominator, which is what steps 3 through 5 of the worked example then build on.

Solution 12.4. The random tests contribute $120 \times 7 = 840$ runs and the directed tests 60, so the nightly is $840 + 60 = 900$ runs. At a mean 11 minutes each that is $900 \times 11 = 9,900$ minutes = 165 CPU-hours, and on 24 concurrent licenses $165 / 24 = 6.9$ hours of wall clock. The night is 10 hours long, so $10 - 6.9 = 3.1$ hours remain, which at the same slot count is $3.1 \times 24 = 74.4$ further CPU-hours the night can still absorb, slightly under half of what the suite already costs. The weekend, at 40 seeds per random test, costs 891 CPU-hours, so $891 / 165 = 5.4$ times the nightly, and it is the tier where the seed depth lives rather than the tier where the runtime pressure is. §12.6 states that growth consumes the headroom in four months, so the computed headroom is a date rather than a margin, and the re-tiering, ranking and pruning of the flagship example are what this tier needs once that date arrives.

Solution 12.5. A ranked regression selects from the test-and-seed pairs that already exist and simulates nothing new, so the evidence it cannot produce is evidence about any scenario the suite has not run before: §12.6 records that ranking is unsuited to exploring the random space and to finding new bugs, and that its output is a compression of what the suite already covered. The property of the weekend tier that restores that evidence is that it stays unranked and raises the seed count, so fresh draws sample the constraint space instead of replaying the sample that scored well last time. The dashboard would then show Chapter 7’s flat-curve pathology, a green regression beside a coverage curve that has stopped moving, since a compression reproduces the coverage it was built from and cannot exceed it. The maintenance obligation §12.6 attaches to a ranking is regeneration, so that the compression stays a compression of the current design rather than of a superseded one, and the event that triggers it is a significant change to the design or the testbench. Of the flagship’s two scheduled tiers §12.6 leaves the weekend unranked and runs the random block set there at 20 seeds rather than the nightly’s 8, which is pitfall 4 of the chapter stated as an arrangement: an unranked tier and regeneration after design changes are both required, and a ranked nightly is safe only because the unranked weekend exists beside it.

Chapter 13

Solution 13.1. The count is the number of fast-clock ticks that span two destination periods, rounded up (§13.4). With the peripheral domain as the destination, one destination period is $1 / 50 \text{ MHz} = 20 \text{ ns}$, so two of them are $2 \times 20 = 40 \text{ ns}$; one system period is 5 ns, as §13.3 states, and $40 / 5 = 8$, so the count is 8. For the always-on destination §13.4 reports 12,500, and the ratio of the two counts is the ratio of the two destination periods: $31.25 \mu\text{s} / 20 \text{ ns} = 1562.5$, and $12,500 / 8 = 1562.5$ confirms it. The two counts together say that the cost of a level crossing in the downward direction is set by the destination period alone, so the same structure that is unremarkable toward the peripheral domain is unaffordable toward the always-on domain: §13.3 asks for a handshake or a toggle there and calls a level pulse in that direction a defect regardless of simulation results, while on the peripheral boundary it observes that pulse width is rarely an issue in either direction.

Solution 13.2. The two counts of §13.6 pair three domains with three boundaries and five domains with ten, and both follow from counting unordered pairs: $3 \times 2 / 2 = 3$ and $5 \times 4 / 2 = 10$, so the rule is $n \times (n - 1) / 2$ for n domains. One further asynchronous domain beyond the flagship’s five gives six, and $6 \times 5 / 2 = 15$ boundaries to classify. The condition that removes a boundary is the definition §13.1 gives: a clock domain is the set of flip-flops driven by one clock or by clocks derived from a common source, so two clocks sharing a source are one domain rather than two, and the pair between them carries no crossing at all; §13.1 adds that declaring such a boundary asynchronous buries the report in hundreds of crossings that are not crossings. When that condition holds for one pair of the six clocks, the design has five domains rather than six, so the count falls from 15 to $5 \times 4 / 2 = 10$, a reduction of $15 - 10 = 5$ boundaries rather than of one. The pair that disappears takes with it

every boundary either of its clocks would have formed separately with the other four, which is why §13.1 insists the setup record the clock relationships from the clocking specification instead of the clock count.

Solution 13.3. The proposal is path qualification, the second of the three fixes of §13.5: a control blocks the crossing while the source reset is active, so the destination samples a held value. It fails the requirement §13.5 attaches to that fix, that the qualifying control must itself be synchronous with the destination domain; a control generated by the soft-reset controller belongs to the source side, and §13.5 states that an asynchronous qualifier transfers the problem to another signal rather than removing it. Ordered reset assertions, the first fix, is not available either, because it requires the destination reset to be already asserted when the source reset asserts, and here the power-on reset is released and the destination register is live and clocking while the soft reset asserts, which is precisely the condition §13.5 uses to report this path. The fix §13.5 records for this island is the third, source reset synchronized to the destination clock, so the source register's output changes only on a clock edge and the crossing becomes an ordinary synchronous path; what makes it available is that the island's 32 kHz clock runs continuously, as §13.5 notes when it selects that option. The example is also the reason §13.5 separates the two disciplines: both registers share one clock, so the CDC report is empty and says nothing about a crossing whose domains are reset domains.

Solution 13.4. §13.6 licenses very little from that run. The whole-IP result it reports is that the same application, given a two-hour budget on an entire IP, returned bounded proofs and no violations although the IP held a known X-optimism bug, which §13.6 offers as an illustration of the application's capacity limit. So an empty report from a whole-block run under a time budget establishes that nothing was found inside the bounds reached, not that no unknown reaches a control point. The order §13.6 prescribes puts source finding first: formal reset analysis takes the reset vector and reports every register still unknown when the reset period ends, catching flops that no structural check can detect, and structural property analysis detects out-of-bound indexing at lower cost than a propagation proof; only then, with every source found and judged intended or not, is the proof scoped to the module where a given source is produced and consumed, small enough to converge. Each earlier step costs less because it asks a structural question over the whole block, while the proof asks a propagation question and pays for it in state space, which is why scoping it to one module is what makes it terminate. The dangerous error is X-optimism: as the crypto-engine example of §13.6 shows, an unreset control bit reads as unknown in simulation, the `if` on it takes the false branch, and the negative test passes for the wrong reason, whereas in silicon the flop powers up to a definite value which on some corners and some dice is one. X-pessimism is the more visible error but fills waveforms with artifacts rather than hiding design bugs.

Solution 13.5. The first bad declaration is the source given for `clk_periph`. Declaring it as `clk_sys` divided by four makes the two edge-aligned and derived from a common source, so by the definition of §13.1 they are one domain and the boundary carries no crossing; the asynchronous group in the excerpt is consistent with that

reading, since it lists only `clk_sys` and `clk_rtc`. What leaves the report is every crossing on the boundary §13.1 calls the widest one, the register interfaces of the APB subsystem, because §13.1 states that the peripheral clock has its own source and that declaring the boundary synchronous silently deletes the entire check. The second is `gray_coded evt_cause[2:0]`. §13.3 admits one two-flop synchronizer per bit for a gray-coded bus only because at most one bit can then go metastable and it resolves to the old or the new value, both legal codes; the event unit's cause bus changes two bits together on the transition from a GPIO wake to a comparator wake, so the declaration makes the structural check accept the existing per-bit synchronizers and report nothing, and §13.3 says in as many words that this one wrong, unreviewed setup line would have let the wake-up defect pass a clean CDC run. What leaves the report here is the crossing that carries the wake-up defect, so the report loses the only finding that mattered while every other line stays as it was. The blank field is the assumed mode of the `test_mode` constant, which the scenario of §13.1 requires the setup file to name for every constant or pseudo-static declaration. While it stays blank, a review cannot establish for which mode the constant holds, so it cannot establish which crossings the constant deletes legitimately and which it deletes in a configuration the design also ships; and since §13.1 observes that nothing in the RTL distinguishes a derived clock from an independent one, this file is the only place either error is visible, which is why §13.1 asks that it live beside the RTL and take the same review.

Chapter 14

Solution 14.1. The run of §14.3 is bounded at 40 cycles. `a_w_burst_len` is checked only at `WLAST`, and §14.3 states that on the engine's maximum burst of 256 beats that `WLAST` cannot arrive until the 256th data beat has been handshaken; each of those handshakes occupies at least one cycle of the interval, which is the reading §14.3 itself uses when it says that inside a 40-cycle interval no burst longer than about forty beats can have completed at all. A lower bound on the interval that could reach the guarded event on a maximum-length burst is therefore 256 cycles, and what the comparison settles, whatever the exact figure, is that the run cannot reach the guarded event at all: the bound would have to grow more than six times, since $256 / 40 = 6.4$. What the run does reach is bursts of about forty beats, so the part of the burst-length range covered is $40 / 256 = 0.156$, about one sixth, and the property is proven for short bursts and completely silent about long ones. `a_no_overflow` guards a datapath FIFO of 16 entries, so its guarded state is reached by 16 pushes with no intervening pop, and $16 < 40$ leaves the interesting state well inside the bound, with the paired `c_full` cover confirming it by witness. The low-cost check of §14.3 is that the antecedent or guarded state should be covered under the same bound before a bounded pass is accepted, and every related cover point must land below the bound; `a_no_overflow` satisfies it and `a_w_burst_len` does not, which is how one run on one block at one bound yields evidence for one property and a silence for the other.

Solution 14.2. §14.6 derives the window as $4 \times 16 = 64$, because each of the other four managers is served at most once ahead of this one, the arbiter being round-robin, and each holds the target address port for at most sixteen cycles by A1. Halving the acceptance bound to eight cycles leaves the port count and the arbitration policy alone and changes only the second factor, so the window becomes $4 \times 8 = 32$. The depth adds to the window the cycle at which the antecedent cover is first witnessed and the two cycles of margin, one for back-pressure and one for a safety net: the recorded row is $12 + 64 = 76$ and $76 + 2 = 78$, so the halved version is $12 + 32 = 44$ and $44 + 2 = 46$. The unchanged term is the witnessed cover cycle, 12, because the antecedent is five requests pending at one subordinate, which is a pattern of offers on the manager side; A1 bounds how long a granted address port stays held, not how long it takes five managers to have requests outstanding at one subordinate. §14.6 puts no latency term in this row because the obligation ends at the grant rather than at the subordinate's port, and the address handshake that follows is a separate obligation, itself bounded by A1. Two remarks belong with the recomputation. The property's own name carries the window, so a changed acceptance bound renames the obligation as well as the depth; and the second cross-check §14.6 asks for still holds, since 46 exceeds the eleven-cycle counterexample this chapter opens with, the longest the project has seen on this block.

Solution 14.3. The argument is right about one half of the reduction and wrong about the other. §14.4 defines over-approximation as adding behaviors and records that it is sound, so a success on the abstract model holds on the original; the one-address reduction is legitimate for this proof because the invariant is per line, and §14.4 lists what it preserves: the directory's transition rules for a line, every race between two requesters for it, the ordering of invalidations against data responses, and starvation of one requester by the other. What it deletes is named at the same time, anything needing a second address, anything needing capacity and therefore a conflict eviction, and anything needing a third sharer, each of which §14.4 sends to another method. Reducing the hub's many caches to two is not sound in that sense at all. §14.4 states that deleting requesters has no defined direction of error: the reduced model is neither a subset nor a superset of the original behavior but a different parameterization of it, a narrower sharer vector and arbitration over a smaller set, so neither implication direction is available and a success on it implies nothing by itself. The premise it rests on is a symmetry argument, that the hub treats its requesters interchangeably, and §14.4 is explicit that this is a claim about the RTL rather than a theorem: it must be checked in the code before use and recorded in the assumption register of §14.2 with the other unproven premises, since an abstraction argument is an assumption in every sense except the syntactic one. Fixed-priority arbitration by requester index inside the directory makes the claim false, and §14.4 gives the consequence: the two-cache proof would never show the lowest-priority requester starving behind the others.

Solution 14.4. As a class the four kinds of §14.6 are structural: they report which logic was involved, and §14.6 states directly that none of them says whether a check would have noticed a change to that logic. Cone-of-influence coverage reports the design space outside every asserted property's cone and so finds logic with no check

on it at all; input stimuli coverage ignores the assertions and reports what the constraints let the engine reach, turning the over-constraint hazard of §14.2 into a number; bounded proof coverage reports how much of a property's cone is reachable within the current bound; proof core coverage reports the logic actually used to establish a proof, which can be much smaller than the cone, so that a bug inside the cone but outside the core escapes the assertion even though the assertion is fully proven. That last kind is the most precise of the four and still answers a structural question, so it cannot tell the team whether an arbitration change would fail something. The instrument §14.6 assigns to that question is mutation: a deliberate design defect should make some assertion fail, which is the empirical complement to all four structural measures, and §14.6 notes that it grades the property set against designer bugs rather than adversarial changes. On scoping, §14.6 says proof core analysis should target a handful of highest-value properties rather than the entire suite, for two reasons it states: the measure carries the highest cost of the four, and such vendor-specific metrics are harder to interpret than simulation code coverage. A whole-suite run therefore spends the most expensive and least legible analysis on the rows that least need it, while leaving the team's actual question unanswered.

Solution 14.5. The cover result establishes that the new assumption deleted the traffic the property was written for. `c_b_pair` asks for two responses of the same ID in flight, and with at most one outstanding per ID no pair remains to order, so what run 2 proved is a rule about traffic the assumption had removed; §14.6 says exactly this of the register row it withdraws. The form of over-constraint is §14.2's satisfiable narrowing, the one that can reach sign-off without an obvious symptom: the assumptions stay consistent, the model stays far from empty, and because formal has no stimulus to measure, the deletion leaves no trace anywhere but in the assumption text. The low-cost manual check of §14.2 is that every assumption should have a design input or a free variable in its support, otherwise it is likely to be conflicting with design behavior rather than constraining the environment; this assumption has `awvalid` and `awid` in its support and passes, which is the point, since that check is aimed at the empty model and says nothing about narrowing. The skipped step is the first of §14.5's order, property validity: a deeper bounded run should have come before any strengthening, because an undetermined property is sometimes a false one whose counterexample lies past the last attempted bound, and finding it costs one overnight job rather than a week of strengthening. The record's own shape is the symptom the chapter's pitfall list names, a property that would not converge for hours proving in seconds shortly after an assumption was added, and the cure it gives is to delete the assumption and re-run: if the property collapses, the assumption was doing the proving. §14.6 records the discharge status of such an assumption as none, added during debug for convergence and never reviewed, and its disposition as withdrawn at sign-off, with the plan row leaving formal for simulation as a directed multi-ID sequence.

Chapter 15

Solution 15.1. §15.7 gives the two factors. An 8×8 residual block holds $8 \times 8 = 64$ coefficients, and at 16 bits each the operand of one invocation carries $64 \times 16 = 1024$ bits, so the input space is 2 raised to 1024 values, which is the figure the section states. The random blocks it mentions number a billion, and a billion is about 2 raised to 30, since 2 raised to 30 is 1073741824, so the sampled fraction is 2 raised to 30 divided by 2 raised to 1024, that is 1 divided by 2 raised to 994. What the comparison settles is the claim §15.7 opens with, that meaningful sampling by a coverage model is infeasible for arithmetic: the two exponents are not close enough for any increase in seed count to matter, and §15.7 adds the reason a larger sample would not help even if it were affordable, since the failure modes it lists, a rounding term applied at the wrong stage, a saturation clipping one bit late, an intermediate width one bit short, are each wrong for a sparse structured subset of operands that random stimulus meets with probability near zero. What it leaves is the flow §15.7 prescribes: a specification written as a mathematical relation between operands and result, word-level reasoning, and decomposition into properties about subunits, because a monolithic claim does not converge. The comparison says nothing about the reference the flow compares against, which the scenario of §15.7 treats separately.

Solution 15.2. The crossbar has five manager ports and four subordinate ports, so the exhaustive job proves $5 \times 4 = 20$ paths, the twenty combinations §15.2 names. On runtime the symbolic proof is the slower of the two: 162 seconds against 65 for all exhaustive per-path proofs together, a ratio of $162 / 65 = 2.49$, about two and a half times the cost, which is why §15.2 states that symbolic checking is not always faster and that runtime needs measurement. What §15.2 offers in its place is a property about the shape of the argument rather than its cost: an arbitrary manager and an arbitrary subordinate are constrained only to remain stable, and because the proof must hold for every value the indices could take, one property covers all of them, which keeps the connectivity argument constant-sized as the interconnect grows. The qualification that pushes the exhaustive count past twenty is stated in the same sentence: each path has to be checked per burst type and per identifier, so the per-path job multiplies by those attributes while the symbolic job does not, and the measured loss on twenty paths is the price of an argument whose size does not follow the port count.

Solution 15.3. The app discharges RTL against the register description, and §15.3 lists what that covers per register: the reset value the description states, for every reset; a decode that maps exactly one address to it; a read that returns what the policy says; a write that does what the policy says for every value written rather than for the values a test happened to pick; reserved fields that ignore writes; unimplemented addresses that behave as declared. It proves those statements over all values and orderings in one job, which is what replaces the register test's hours of front-door sweeps. What it leaves open is the description. §15.3 identifies this as the common-mode limit of Chapter 3: the verification model and the RTL check are generated from the same file, so an error in the description is invisible to both, encoded once and read twice, and nothing in the flow reviews whether bit 3 was the right bit.

So a clean report is evidence of conformance and not of correctness, and the review that stays a person's work is a human reading the description against the architecture document, which is the part the app leaves rather than the part it removes. The failing check is the one on the volatile field, and the two arguments that decide it are the ones §15.3 picks out of the canonical status register: the policy "W1C", a statement about every possible write value and both possible current values, and the `volatile` argument, which records that hardware can change the field without software's involvement. A check that reads the policy as leaving the field at 0 after the write ignores the second argument and therefore fails on correct hardware, since the overrun condition can set the flag in the same cycle software clears it; §15.3 says that this is what creates an incentive to disable the check. The reading a correct check has to carry is the one §15.3 states: a software write of 1 clears the field unless hardware sets it in the same cycle. Disabling the check removes the only exhaustive statement about the field and leaves the incentive in place for the next field like it.

Solution 15.4. §15.4 licenses no verdict from the app on those cells at all, and for two separate reasons. The first is that the app works on the design while the cross does not live there: the model's worst-case cross has $3 \times 4 \times 3 \times 2 = 72$ combinations, three stride signs by four boundary relations by three burst lengths by two states of the other channel, and the excluded column accounts for $3 \times 2 = 6$ of them, one for each stride sign and each other-channel state, which leaves $72 - 6 = 66$ cells to close; but the boundary relation is computed in the monitor, so §15.4 states that the app cannot see the cross and a machine verdict needs a design-level restatement. The second is the premise the exclusion rests on. §15.4 is explicit that these six are excluded by policy, the constrained-random generator's budget on a row, which Chapter 9 labels an engineering choice about solve time rather than a statement about the DUT, so the exclusion is valid as a stimulus claim and not as a design claim: true while the generator is unchanged, false when it widens. The design-level restatement returns reachable, with a trace showing how, because nothing in the design bounds a descriptor row at one maximum burst; the bound that empties the column lives in the generator, and a design-level cover does not inherit it. Handing the app that budget would make the verdict inherit it instead, which recreates the first limitation §15.4 records, where an input constraint in a tool setup script made a counter's increment line uncoverable and the line was duly reported unreachable, correctly, under an assumption no engineer had reviewed and that did not reflect the design's real environment. The disposition therefore remains a waiver in Chapter 7's written form, with the hole, reason, risk argument, owner and expiry, and the event that ends it is an edit to the constraint file, which §15.4 says must itself count as verification evidence.

Solution 15.5. The blank field is the closure metric. §15.2's practice section defines the row's metric as every generated property proven, none bounded, none waived, and it spells out why the middle clause has to be there: none bounded means that a property which exhausted its budget leaves the row open, because the default report does not distinguish a discharged property from one that ran out of time. The result as reported is a bounded one with no counterexample, so it does not close the row. §15.1 gives the reason in general terms, that an app uses the same engine as a hand-

written proof and a bounded result from an app is exactly as bounded as one from a specialist, and the chapter's pitfall list restates it as bounded results counted as complete, whose cure is that every generated property's proof status requires checking because bounded with no counterexample remains unfinished work. The missing directive is a cover on the antecedent, here on the enabling condition `cfg_irq_en[9]`: §15.2 records that the generated text lacks such a cover and that an intent row does not discharge Chapter 11's rule that every implication ships with a cover on its antecedent. Without it, a passing result cannot distinguish a proved connection from an enable that no legal trace can set, since the property is silent by construction on every cycle in which the enable is low. Both defects point the same way for this record, because a row reported bounded and with no witness on its antecedent carries no evidence of either kind, neither that the connection holds nor that the claim was ever exercised.

Chapter 16

Solution 16.1. The proof entitles the report to close the plan row it discharges and to leave the cross bins exactly as they stand. §16.3 sorts the pairing into its third disposition, the column of things that must never be summed: a functional coverage bin against a proof, because a hit count and a proof answer different questions, the bin counting occurrences of a situation in the executed runs and the proof holding that no legal trace violates a rule, which is a statement about the testbench and a statement about the design respectively. What closes instead is fixed by the one-row-one-metric rule §16.3 restates: a row discharged by a proof closes on the proof, with its depth, its assumption list and its own formal coverage evidence attached, and a proven row needs no functional bin because the proof discharges what the bin sampled, while a proof alone never closes the row. The two further measurements lost by marking the bins filled are the ones §16.3 names where it explains why the practice destroys two measurements at once: Chapter 7's coverage position can no longer corroborate a flat bug curve, and Chapter 12's coverage differential can no longer fail a commit that reduces the suite's scope. So the disagreement does not need a merge rule to settle it: the two engineers hold evidence of two kinds, and the plan row, not the dashboard, is where each kind is spent.

Solution 16.2. Under the metric with one bin per value the score is $2/256 = 0.0078$, that is 0.78%, and under the eight-bin automatic metric it is $2/8 = 0.25$, that is 25%; the factor between them is $25 / 0.78 = 32$, which is exactly the ratio of the two bin counts, $256 / 8 = 32$. The property that produces the factor is the one §16.3 states in defining a metric as the rule assigning observed values to bins: the denominator is a property of the metric, not of the design or of the data, so the same two observed values score differently under two partitions of the same space, and §16.3 adds that collection loses information that cannot be recovered afterwards, so the coarser score cannot be refined back into the finer one. The pair therefore establishes that a percentage combined from two engines whose metrics partition the space differently has no defined meaning, which is why §16.3 lists two percentages whose metrics partition the space differently among the pairs that must never be summed, and why

the reconciliation puts unification in the plan, where rows closed out of rows planned has a denominator that can be stated.

Solution 16.3. The row establishes what detector one is for: the same text that is assumed in the DSP wake controller's proof is bound as an assertion in the DSP's block-level simulation, where the signals it watches come from the testbench, so a nightly run in which it passes is evidence that no sequence has withdrawn an unacknowledged request, and therefore that the generator has not drifted outside the assumption. What the row leaves unestablished is whether the assumption was ever tested at all: the last two fields are detector two, the cover on the condition the assumption restricts, and the cover has never been hit. §16.2 gives the consequence directly: if that cover reads zero, the assertion has never evaluated non-vacuously, and the claim that simulation corroborates the formal environment is empty, which is the rule that every assumption ships with a cover on the condition it restricts. The state matches the fourth pitfall of the chapter's list, an assumption unapproached by tests, whose symptom is exactly this row, an assertion form passing in every simulation while its boundary cover reads zero, and whose cure is to count the empty cover as a failure. §16.2 also carries a caveat that decides how such a row is closed in general: where the restricted condition is ordinary traffic, a bare cover proves nothing, and the rule is discharged instead by a witness that implies the condition and isolates the interesting case; a request standing unacknowledged on this handshake is not ordinary traffic, which is why the chapter's own cover on that condition is the instrument the row needs.

Solution 16.4. Five rows carry four discharges, so $5 - 4 = 1$ row remains undischarged. §16.2 gives each row two run records, the run in which its assertion form last passed and the run in which its boundary cover was last hit, so the register holds $5 \times 2 = 10$ run dates once both detectors are applied to every row. The direction that leaves nothing behind is the second of §16.2's two: an assumption stronger than the generator's legal set, where the proof holds over a subset of the traffic the tests actually send, so simulation exercises behavior the proof never covered while both reports stay green, and neither a coverage hole nor a failure nor an artifact records the gap. The opposite direction is visible for free, because a generator narrower than the assumption shows up as a simulation coverage hole and goes to the classification of Chapter 6. For the four rows discharged elsewhere, §16.5 requires the transfer to carry the discharged assumption-register row and the boundary cover with it, under the heading that an assumption becomes a checker, and §16.2's crypto-engine scenario shows the same bookkeeping in place: the row keeps the assumption's text and gains a discharge pointer, and the assertion form joins the simulation regression that re-checks it against real traffic.

Solution 16.5. The method contradicts §16.1's corner-or-volume criterion. A latency distribution under sustained traffic is a volume criterion, aggregate behavior over many transactions, and §16.1 states that such a statistical criterion cannot be discharged by proof: formal verification generalizes and abstracts design behavior, which is why it cannot determine how often something happens. A proof of a bound over all traces can return one thing on this row, an existing worst case that exceeds

the envelope, which is a witness that some trace violates the bound; §16.1 makes the point on the GPU's second row, where an all-traces proof returns a worst-case stall occurring once in a billion frames and does not answer the row's question. What it cannot return is the distribution the criterion is stated over, so no proof result closes this row whatever its depth, and §16.1's own assignment for the flagship's mesh sends congestion, latency distribution and virtual-channel occupancy to simulation and emulation. The empty field is item six of §16.4's ordered procedure, the non-convergence fallback, which the plan review should define before week nine; §16.1 explains why the field is not optional, since a formal row on the critical path is an unbounded commitment unless the plan says in advance what happens when the proof does not converge, and Chapter 14's convergence ladder ends with handing the row to simulation as a planned outcome.

Solution 16.6. §16.5 requires the constraint set to accompany the verdict as evidence, reviewed as seriously as the waiver it replaces, because Chapter 15's caveat travels with the result: reachability is computed relative to the constraints, so a verdict without its constraint set is not interpretable. Besides the waiver's reason field, §16.3 puts the verdict on the coverage item itself, as a flag marking the item formally unreachable with respect to a particular test, and the interchange standard's classification distinguishes a don't-care from a predicted-unreachable point precisely so that a warning is raised if a predicted-unreachable point is later covered. That flag is what keeps the record valid as the design changes: if a later RTL change, a relaxed constraint or a new test reaches the excluded point, the regression warns, rather than the project retaining a file exclusion valid only for the old design, and the sixth pitfall of the chapter's list adds the other half of the cure, that the proof itself reruns at the cadence of the regression that consumes it. The pitfall avoided is that sixth one, a stale unreachability exclusion, whose symptom is a coverage exclusion justified by a proof against an RTL revision three months old. One check precedes all of this in §16.3: the waiver vocabulary applies only to the third class of the Cookbook's sorting, code unused in a given configuration, so a hole in the first two classes, stimulus never generated or a bug blocking the stimulus, owes a test or a fix instead of a waiver, and the three failed targeted tests are evidence of candidacy rather than of unreachability.

Chapter 17

Solution 17.1. One legal burst is the case $N = 64$, so the fraction of peak is $64 / (64 + 200) = 64 / 264 = 0.242$, about 24%. The batching table's middle column gives 7.4% for a short burst of 16 cycles and 56% for the 256 beats the protocol encodes, so 24% falls between the second and third rows, which is what §17.4 states of a single call on the AXI write data channel; the reason the third row is out of reach is arithmetic rather than protocol, since a beat is 64 bytes and $64 \times 64 = 4,096$ bytes, exactly the one 4 KB region the burst splitter permits. The speedup is $S(N) = N(1 + L) / (N + L)$, which tends to $1 + L = 201$ as N grows, and at $N = L$ it is $200 \times 201 / (200 + 200) = 100.5 = 201 / 2$, half the ceiling, as §17.4 states. The legal burst therefore leaves the half-ceiling point out of reach as well as the ceiling: $S(64) = 64 \times 201$

$/ 264 = 48.7$, well under 100.5, because 64 is well under $L = 200$. That is why §17.4 says the gap to the third row is not something a longer burst can close: the burst is capped by legality, the remedy is a coarser transaction such as a descriptor, and the general rule is that granularity should be comparable to round-trip cost.

Solution 17.2. The evidence closes the protocol-state half of the criterion and leaves the timing at the pins open, which is the split §17.3 states for a row closed by an instrument-driven compliance suite. The platform element that forces it is the speed adapter: the design executes at megahertz while the real link runs at its specified rate, so the adapter buffers and throttles traffic and re-times it, and the design receives real content at adapter-generated timing. Timing observed at the pins is therefore the adapter's, not the design's, and those rows stay open until silicon, which is why §17.3 says that a row recording a compliance pass without the split is claiming evidence it does not have, and why the sixth pitfall of the chapter's list names exactly this record. The column left empty is the image the row runs on, which §17.5 adds to Chapter 5's method field for every row that runs on hardware. Without it the row cannot be batched, and batching by image is the whole scheduling mechanism: the image is the scheduling unit, a low-priority row that fits an image already being built is nearly free, a high-priority row needing its own instrumentation costs a build, and §17.5 asks for images to be planned before rows are assigned because reversing the order leaves top-priority rows spread across unscheduled builds.

Solution 17.3. The two sections divide the cases. §17.2's amortization holds for an image reused across many hours or days of execution, and across regressions and campaigns: the build is paid once and divided by everything that runs on it, so a build supporting one run bears the full cost while a build supporting two hundred runs is amortized over the group. §17.5 states the other case in the placement model: the build B is paid per design revision, as is every nightly suite. A smoke tier exists to test a new revision, so every invocation of the tier is a new revision and a new build, and B is paid per invocation and never amortized; against a tier that §17.2 puts at under nine minutes end to end, most of it build, a build measured in hours to days makes the proposal slower by orders of magnitude, and the third pitfall of the chapter's list names it as buying capacity for a farm problem. The property that would reverse the decision is run length rather than run count: a workload whose single run exceeds simulation's reach, since a license pool shortens suites and never shortens a run. §17.5's admission criteria place such a workload in the first claim, workloads with no simulation answer, and a workload valued instead for iteration count under one image in the second, where the build cost is amortized across the sweep. The smoke tier has neither property, and §17.5's diagnostic cases add the reason to look elsewhere first: if profiling shows the design is a minority of runtime, the problem and the fix are software.

Solution 17.4. The machine figure implies $2.4 \text{ hours} = 2.4 \times 3,600 = 8,640$ seconds of execution for 4,800 runs, that is $8,640 / 4,800 = 1.8$ seconds per run, which agrees with the under two seconds §17.5 quotes at the optimistic end of the speed band. The farm figure follows from the same two counts: $4,800 \text{ runs} \times 30 \text{ minutes} =$

144,000 minutes = 2,400 CPU-hours, and $2,400 / 240 = 10$ hours of wall clock on 240 licenses, which is the $N \cdot s / W_{\text{sim}}$ term of the placement model. The term that decides against the move is B , the build, which §17.5 measures in hours to a day and which is paid per design revision, so a nightly suite pays it nightly: 2.4 hours of machine time plus a build of that size, incurred every night, against 10 hours on licenses the project already holds, yields no gain. Parallel width cannot offset it because W_{hw} is capacity divided by design size rather than a quantity that can be purchased, and §17.5 puts it at 1 for this image, so the 4,800 runs serialize on the one machine while the farm's width is what produced the 10-hour figure in the first place. At the pessimistic end of the band, §17.5 adds, the emulator loses on run time alone, before any build is counted.

Solution 17.5. The oracle belongs to the third of §17.4's three classes: an oracle that needs to see every beat costs one host interaction per beat and does not scale at all, so it has to be redesigned rather than ported. That is why the throughput belongs to the oracle: one interaction per beat is the $N = 1$ case of §17.4's arithmetic, which gives $f/(1 + L)$ and throws away between 95% and 99.95% of peak, so the rate the run advances at is fixed by the granularity of the checking and the round-trip cost, and the platform's clock rate does not enter. The record's second symptom points the same way, since no assertions were compiled into the image, and §17.2 makes those the one form of checking that crosses the boundary intact, costing logic and no interactions and needing no conversation with the host. §17.4 prescribes moving the per-cycle work onto the platform: a synthesizable digest accumulator hashes the ciphertext stream and the authentication tags, the host reads and compares one digest per checkpoint rather than one value per beat, and the per-beat comparison stays a short-vector check in the simulation tier. The cost §17.4 attaches is detection distance: a digest mismatch says the stream diverged somewhere in the last checkpoint interval and not where, so the checkpoint interval is what bounds the debug bisection, and the fifth pitfall of the chapter's list states the alternative, a per-beat scoreboard pinning throughput at transaction rate while the soak never finishes.

Solution 17.6. §17.3 licenses the use model for the campaign, because it licenses in-circuit emulation exactly when the environment cannot be modeled, and a third-party host the team does not own is one of the environments it names as justifying the acceptance of irreproducibility, alongside a compliance instrument and a driver stack the team does not own. The two losses that the gating requirement makes decisive are repeatability, since a real link partner's failure has no seed and no replay, and unattended operation, since someone must plug in the device; Chapter 12's stability check repeats a test before it may gate, and §17.3 states that Chapter 12's apparatus assumes a run is a function of recorded inputs and that a failure which cannot be reproduced cannot be triaged. The plan should therefore split the row the way §17.3 splits a compliance row: what closes on the in-circuit run is the interoperability claim, which needs no model of the other party and which no virtual environment can produce, while the half that gates the release stays open on this platform and closes on a virtual run over recorded traffic, where the same inputs can be re-

played. §17.3 supplies the boundary for that division: an environment is a candidate for recording when what it contributes is data, since noise, traffic patterns and workloads are indefinitely replayable, and only the parts that are genuinely unmodelable justify the irreproducible run.

Chapter 18

Solution 18.1. Dividing the system clock by ten puts the system domain at $200 / 10 = 20$ MHz and the peripheral domain, which drops with it, at $50 / 10 = 5$ MHz, so their ratio is $20 / 5 = 4$ and the 4:1 relationship survives, which is §18.1's point that dividing both by the same factor preserves the relationship. The always-on reference is a board crystal and does not scale, so the second ratio moves with the numerator alone: 20 MHz over 32 kHz is $20,000 / 32 = 625$, and $6,250 / 10 = 625$ confirms that the ratio has changed by exactly the slowdown factor. Every timing relationship across that boundary therefore changes, and §18.1 states the consequence for the canonical wake-cause bug, in which a 3-bit code crosses through per-bit synchronizers and briefly exposes a reserved code: reproduction depends on how many fast-domain sampling edges fall inside the slow domain's transition window, so a tenfold slower system clock leaves a tenth as many of them. A prototype that never reproduces the crossing and one that reproduces it constantly can be the same design at two clock settings, which is why §18.1 says the prototype is not the instrument for a crossing bug in either direction, and leaves it with the structural CDC analysis of Chapter 13, which found this bug in seconds.

Solution 18.2. The disagreement establishes nothing about which artifact is wrong. §18.2 states the argument where it warns against the virtual platform becoming the specification by default: when model and RTL disagree, neither is golden, because both were written by people reading the same document, so a disagreement is as likely to be two misreadings as one bug. Triage must therefore go to the specification rather than to the two implementations, which is what yields §18.5's three outcomes instead of two: the specification says X and the hardware does X, so the software is wrong and the driver is corrected; the specification says X and the hardware does Y, so the hardware is wrong and the RTL is corrected along with a test that would have caught the defect; and the specification does not say, which is the outcome that leaves the defect owned by neither team, because it is a specification defect and filing it as a firmware bug misclassifies it. The structural fix §18.2 takes from Chapter 15 generates the model's register behavior, the decode logic and the firmware header from one register description, which reduces disagreements to behavior rather than layout. The class of error that survives both the fix and the comparison is the common-mode one: where model and RTL agree wrongly, both having read the document the same way, comparison cannot surface the error, which is the same limit Chapter 3 places on any two-implementation check.

Solution 18.3. The readback establishes that the property was violated on this run, that the first offending burst is identified by its address together with its length and size fields, and that violations did not stop there: the counter stands at its maximum,

saturating rather than wrapping, so it reports that the violations exceeded its range rather than a wrong smaller number. What no readback of this form can establish is the context of any violation. §18.4 states it directly: the waveform is lost, so the fields give one address with the length and size of that burst, and no picture of what the interconnect was doing around them, and details of every violation after the first are lost as well. The counter field answers the question §18.4 distinguishes from the sticky bit's: whether the driver is systematically emitting illegal descriptors or whether one corner case fired once, a distinction the sticky bit alone cannot make since it answers a much weaker question, and a saturated counter is evidence for the first reading. The monitor satisfies the second admission condition of §18.1, an oracle that works with limited visibility, because the verdict reaches a register the driver can read rather than a waveform. §18.4 adds the two facts that make the readback worth its fabric: the check is evaluated at the instant of the violation, at full speed, on every burst of a workload no simulator will reach, so detection distance is zero cycles even though knowledge of the verdict waits for the driver's next poll, and the monitor had to be built before the bitstream, since adding one afterward requires a compilation.

Solution 18.4. The boot executes 20×10^6 cycles per second $\times$ 115 seconds = 2.3×10^9 cycles. The failing workload of the chapter's opening spans hundreds of millions of cycles, so the published boot is several times longer, and the opening records that the shorter of the two was already beyond the verification team's simulator for the quarter. The comparison therefore settles the first of §18.1's three admission conditions, that the workload needs more cycles than simulation can deliver, and settles it for either length. It leaves the other two open, because they are not about cycles: the workload's oracle must decide pass or fail without a waveform, through an offline bit-exact diff, self-checking firmware or an in-fabric monitor latching violations into a readable register, and its failure must be plausibly reproducible in a higher-visibility environment, since that is where debug will happen. The opening is the cost of failing them: with one console, a few LEDs and a trace buffer armed for another question three bitstreams earlier, and no way to reproduce the failure in simulation, two teams spent nine days deciding whose bug it was, on a bug §18.1 describes as straightforward once located.

Solution 18.5. The record supports the third of §18.5's three outcomes, the specification does not say, which makes this a specification defect owned by neither team: both evidence lines hold, the driver follows the documented sequence and the counters behave as the source describes, and the field the document leaves blank is exactly the one the failure turns on. What the filing line gets wrong is the fix it implies. §18.5 states that software cannot avoid this class of defect because nothing in the interface lets it infer the counter's value, so no correct driver can avoid it, and the only fix available to software is a workaround that must be specified before it can be written; filing it as a firmware bug therefore buys a driver patch and leaves the document wrong for the next driver, for the next derivative and for the customer. Classified correctly, §18.5 requires three artifacts alongside the entry: the document change, a checker enforcing the newly stated rule, and a plan row that owns it. The entry itself belongs in Chapter 5's spec-hole log, which exists for questions the spe-

cification cannot answer, and §18.5 adds that prototype triage needs a named owner spanning both teams, on a rotation drawn from both, since without it the ownership dispute costs a week per incident. The chapter’s own example of this outcome is the register field interpreted as two different fixed-point formats by two teams reading one line of documentation, where the remedy is a register-description entry rather than a patch.

Solution 18.6. The proposal fails the first condition and the second, and puts the third at risk. It fails the first because the boundary has no architectural transaction semantics: §18.3 requires a memory-mapped register interface, bus port or mailbox, and states that a boundary across microarchitecture, such as a pipeline, credit counter or handshake, has no transaction to send, since the channel can only carry operations; the published analysis §18.3 cites adds that the transaction-level standard’s interoperability guarantee reaches only memory-mapped bus protocols, so anything else depends on a payload defined locally by the project. It fails the second because the virtual side must be mature, every run trusting it without verification throughout execution, and a channel model written to fill this partition gap is a second design with no test plan of its own. The third condition, that all timing under verification stay on the RTL side, fails as well if the flow control is itself under verification, because the credit relationship would then span the untimed channel, and §18.3 states that every bug in the timing relationship between the two sides is unreachable by construction. The boundary §18.3 would accept is the cluster’s own architectural interface, its memory-mapped register interface and its bus port, the placement the sensor-hub scenario uses when it joins at the island’s register interface and wake-event line. Once placed there, the plan row must record the exclusion in Chapter 5’s terms, stating that cross-boundary timing between the two sides is out of scope on this platform and naming where it is covered instead, and it must carry the migration plan, since blocks move from virtual to RTL as the project matures; the fourth pitfall of the chapter’s list is what the record prevents, a passing system campaign hiding an untested cross-boundary interaction.

Chapter 19

Solution 19.1. The gate-level tier runs each of the 180 distinct tests once, so the base is the single-run cost of those tests and not the nightly’s total: $180 \times 11 = 1,980$ minutes = 33 CPU-hours. Twenty-four licenses across a ten-hour night afford $24 \times 10 = 240$ CPU-hours, so the tier exactly fills the night at $240 / 33 = 7.27$, which is the “around 7×” break-even §19.5 reports. The 165 CPU-hours of the nightly is the wrong base because it prices 900 runs rather than 180 distinct tests; taken as the base it gives $240 / 165 = 1.45$, a ratio five times too small. The two costs §19.5 gives for those same 180 runs settle which base is right: 1,320 CPU-hours at 40× and 330 CPU-hours at 10×, since $1,320 / 40 = 33$ and $330 / 10 = 33$, whereas the nightly base would have put those entries at $165 \times 40 = 6,600$ and $165 \times 10 = 1,650$.

Solution 19.2. The run establishes nothing about the netlist. §19.2 gives the mechanism: `!=` is logical inequality, whose result is `1'bx` when unknown bits make the re-

lation ambiguous, and `if (1'bx)` takes the false branch, so a fully corrupted sampled value is scored as correct; the run passes, coverage is collected and the plan row is discharged, which is why §19.2 calls this failure silent rather than pessimistic. The audit §19.2 requires is of the testbench, before its first netlist run: every sampled value is checked explicitly for unknowns before comparison, so unknown data carries its own verdict, and the comparison that remains is written with the case operator, whose result is always `1'b0` or `1'b1` and therefore always reaches the report. A history of reported mismatches leaves the objection unanswered because the two capabilities are different, and §19.2 separates them: a value corrupted only in bits that already differ from the expectation is still reported, since the relation is not ambiguous, so past reports establish that the checker fires on unambiguous relations and not that it rejects ambiguous ones. What escapes is the fully corrupted value, and the partially corrupted one whose unknowns sit precisely on the bits that would have carried the difference. The first clean zero-delay boot is also the prerequisite §19.2 puts in the plan for the additions that follow it, so an unaudited pass licenses those additions to rest on a result that was never evidence.

Solution 19.3. The two entries are the empty min/typ/max member and the unrecorded annotated-instance count. §19.3 makes the choice of triple member a string argument whose default value leaves the choice to the simulator, and states that a run with an unselected corner provides no evidence about any corner; the verdict recorded on the plan row is therefore unsupported by this run whatever the netlist does, and §19.3 assigns hold analysis at the best-case corner to static timing analysis in any case, so the row was asking a sample for a result the authority already owns. The missing count removes the only review §19.3 prescribes against missing annotation, which leaves any timing value the file does not carry at its pre-annotation value, frequently zero: the annotation log records every individual annotation, review is meant to compare its annotated-instance count against the netlist's, and without the log that comparison cannot be made. The two entries under it are exactly the symptom §19.3's pitfall on unaudited annotation names, a suspiciously fast back-annotated run with no violations, so on this record the absence of violations is as consistent with an unannotated block as with a clean one. Beside the corner, §19.3 requires the seed and the tool build in the run manifest. The remaining entries are sound, and one of them carries the section's third failure mode the right way round: annotating once at compile time and running the test cases against the same compiled model is the organization §19.3 prescribes, because annotation cost is why a gate-level tier is a small number of long runs rather than a large number of short ones.

Solution 19.4. The campaign can exhibit and cannot cover. §19.1 reconciles its two apparently opposed sources exactly here: gate-level simulation with timing can exhibit one timing violation, meaning one path, one corner and one alignment of edges, while whether a given alignment occurs in a given run is an accident of the stimulus, so a run samples the class instead of covering it. Sign-off requires coverage of the class, so the proposal asks a sample to discharge an exhaustive obligation, and the chapter's summary adds the sharper limit: metastability itself is not observable in simulation, so what such a run can show is the timing violation on the synchronizer path rather than the metastable event. The analysis that covers the class is structur-

al and timing analysis of the final implementation, which §19.3 describes examining every timing arc in the netlist against the constraints file across process variation, minimum and maximum voltage and temperature, and on-chip variation, with hold critical at the best-case corner and setup at the worst. The inserted structure is scan on the second flip-flop of a two-flop synchronizer: §19.1 reports that it adds multiplexer logic on the metastable path and erodes the settling window, degrading mean time between failures, while scan insertion on the first flip-flop has no such effect, and it names that finding a structural property of the implemented netlist which simulation could not have produced. What survives of the proposal is the split §19.1 prescribes for a proposed gate-level run: the obligations inside static analysis's scope go to the analysis, and a gate-level run is bought only where the purpose is to exhibit a phenomenon nothing else can produce.

Solution 19.5. §19.4 requires two of these entries to become negative properties. The retention filter must exclude the crypto engine's key registers, so that powering the domain down destroys the key, which §19.4 calls the entire security value of powering it down; as written the filter retains them, and §19.4 records why the defect survives the obvious test, since a test that checks correct encryption after power-up passes with the key still available. The directed test §19.4 prescribes instead powers the domain down and up, attempts encryption without reloading the key, and requires failure. The isolation entry must clamp `key_valid` to 0: clamping it to 1 tells the always-on island that a key is loaded while its domain is unpowered, which is why §19.4 treats a clamp value as a functional decision rather than a don't-care, and the busy entry is the second defect from the same list, since §19.4 states that clamping a busy output to 1 hangs the interconnect. The entry the section's domain figure already licenses is the clamp value 0 on the event unit's `irq_pending`: that figure shows the reference SoC's isolation cells clamping to 0 on the crossings that leave the switched domain, and `irq_pending` is one of them. It still owes the plan row and the rationale §19.4 requires for every clamp value, explaining how the receiving logic uses the clamped value throughout power-off. The restore-control entry contradicts step 3 of the power-up column, whose order inside the step is release reset and then assert restore; §19.4 records this as one of the three steps carrying a published failure, reset must be inactive at the restore edge, which is why that step has an order inside it at all.

Solution 19.6. §19.4 declares six flagship power domains, and the always-on domain contributes no factor because it is never off. The host cluster and the compute island each carry the three DVFS operating points plus off, four states each; the root of trust, the NoC and memory, and the peripherals are on or off, two states each. That is §19.4's own arithmetic, $4 \times 4 \times 2 \times 2 \times 2 = 128$. Giving the NoC and memory domain the same operating points raises its factor from two to four, so the bound becomes $4 \times 4 \times 4 \times 2 \times 2 = 256$. The power state table then does to the new figure what §19.4 says it does to the old one: it specifies the legal subset, so 256 is an upper bound and not a state count, most of the combinations being illegal by construction. What the bound is for is the checks it obliges, on the legal states and on their transitions, and since §19.4 notes that transitions far outnumber states and are where sequencing bugs occur, doubling the bound understates the growth in what has to be checked.

Chapter 20

Solution 20.1. §20.2 gives both halves of the arithmetic: the governing inequality is $\text{signals} \times \text{cycles} \leq \text{buffer}$, and sixty-four kilobytes is 524,288 bits, which is why 128 signals buys 4,096 cycles of history. Widening the set to 512 signals leaves $524,288 / 512 = 1,024$ cycles, the depth §20.2 lists for that width. The bisection then needs $\log_2(10^{12} / 1,024) = 29.9$, that is 30 halvings, against the 28 §20.4 derives for a 4,096-cycle window, so two more. Two extra runs at forty minutes each add $2 \times 40 = 80$ minutes of pure execution, and if only one run in three reproduces the failure the cost triples, so the addition becomes $3 \times 80 = 240$ minutes, four hours. That reconciles with §20.4's observation that a buffer twice as deep removes one of twenty-eight halvings rather than halving the cost: depth enters the count logarithmically, so the trade is one halving per factor of two in either direction, and quadrupling the traced width costs exactly the two halvings that doubling the depth twice would have bought.

Solution 20.2. The blank field is the oracle available at that moment, and the plan of §20.3 states it for this row: a data comparison plus the completion interrupt. The two earlier rows the check consumes are the L2 walking-pattern row B06 and the first-timer-interrupt row B08. B06 is needed because the verdict is a comparison of data in the memory that row proved, so without it a mismatch is as consistent with an L2 address-decode fault as with a DMA fault; B08 is needed because the completion interrupt half of the oracle rests on interrupt wiring, exception entry and vectoring, and §20.3's note on the order states that this row depends on both at once, follows both, and would otherwise produce an unattributable failure. Placing it after the heartbeat row breaks the rule §20.3 gives for the whole sequence, that each check must be decidable using only what preceding checks established. The two failure modes the placement leaves indistinguishable are the ones that rule names: the object checked and the machinery judging it, here a DMA that did not copy against an oracle, in memory and in the interrupt path, that has not been shown to work.

Solution 20.3. The passing suite establishes that the original mechanism is still absent and nothing at all about the descendant. §20.5 states the rule directly: inherited assets do not establish inherited coverage, and rerunning the old suite successfully does not establish that a previously found bug remains covered. The link that broke first is activation, because the condition the descendant needs, a split whose second half writes a line the host cluster holds Modified, is one the stimulus never creates: no test in the suite runs with a cache holding that line dirty. §20.5's table assigns activation a new plan row and a coverage cross that would have demanded the condition, and that is why the answer is not propagation: the corruption does stay invisible until the writeback, but on this platform no run reaches a writeback with the line dirty at all, so the condition never happens rather than happening unobserved. The inherited cross omits coherence state, which is what obliges the next generation to re-derive coverage against the new mechanism instead of inheriting the closure of the old one.

Solution 20.4. §20.6 obliges the team to supply evidence and to decline the recommendation. The evidence is the three findings §20.6 names: what the bug does, the necessary and sufficient conditions, and the workaround's coverage and exclusions. The recommendation is declined because the disposition decision requires information verification does not have, about committed customers, launch dates, the qualification plan, the cost of a mask revision and the competitive window; §20.6 adds that a severity label on its own omits precisely the evidence verification is uniquely qualified to supply. In the program §20.6 cites, severity was set by three factors together: the bug's impact on functional behavior, the performance cost of working around it where a workaround existed, and the complexity of the fix, and it could be determined only once the root cause was found and a fix proposed. The symptom cannot order the two bugs because two of those three factors are invisible in it, which §20.6 illustrates with this pair: a hang looks catastrophic and may be one register bit away from a free firmware workaround, while a one-bit corruption looks minor and may have no workaround at all.

Solution 20.5. Both entries are recorded at pre-sighting, and §20.4's pipeline lets only the first advance. The first has what pre-sighting analysis exists to produce, a stable recipe, so it exits as a sighting and owes the next stage an assigned debug team and an agreed plan to track, work around and address it, which is all §20.4 means by sighting disposition and not the fix or waive-as-errata decision. The second owes the exit the first has already reached: rerun under varied software, hardware, system and environmental conditions until a stable recipe exists, and §20.4's pitfall against treating cannot-reproduce as a property of the bug names the parameters to vary, temperature and voltage, part and elapsed time, and active feature set. The repetition on a second part is §20.4's first discriminator, between a physical silicon issue and a logic error: a failure following the part is a manufacturing or marginality problem and one following the design is a logic error, so the first entry, which shows the same symptom on another part under the same mixture, is a logic error. The standard §20.4 applies to the first entry's rate is the lab's weakened one, an error that appears reliably once in a few executions counting as reproducible, and §20.4 gives exactly the one-in-five rate as a working recipe there because five runs cost

minutes; the same rate in a nightly regression is Chapter 12's flake, the same failure on an identical manifest, and intolerable; the two entries must nevertheless be grouped before either is analyzed, because §20.4 records that two failures with one root cause can present as a crash in one test and a hang in another.

Chapter 21

Solution 21.1. The block is the 12-bit SAR ADC of the chapter's opening, so full scale is 4096 codes and 1% of it is $4096 / 100 = 40.96$, about 41 LSB, which is the figure the model register notes. In volts, one LSB against the 1.8 V reference is $1.8 / 4096 \approx 439 \mu\text{V}$, the value the opening derives, so the band is $41 \times 439 = 17,999 \mu\text{V}$, about 18 mV. The package states the same band directly, $0.01 \times 1.8 = 0.018 \text{ V}$, that is 18 mV, so the register's code form and the package's volt form are one band written twice, and the agreement is what lets a checker quote the constant while the plan row quotes the criterion. The form in use is the absolute one, since the percentage is taken of the fixed full-scale reference and therefore yields a constant band across the whole input range, which §21.5 gives as the reason the correlation check uses the absolute token. The other form, a percentage of the measured value, is the relative tolerance §21.1 says fails near a legitimate zero: taken of a measurement near the bottom of the range it shrinks with the signal and collapses toward zero, so the band would be far narrower there than the 439 μV of a single code.

Solution 21.2. The blank fields are the date and owner and the consequence, and each blocks a distinct claim. Without the date and owner the entry cannot support the claim that the correlation is current, because §21.4 makes the date what turns "the model is correlated" into a claim that can visibly go stale, and it cannot name who must re-run correlation when a trigger fires, a division §21.5 states as the analog engineer writing and correlating the model while the verification team owns its register row and its expiry. Without the consequence field the entry cannot support any statement about what else would be wrong if the model were wrong: §21.4 makes that field the answer to the only question a reviewer needs to ask about a model, and the filled form for the ADC names every ADC-path result in the chip-level regression and every plan row that closes on an ADC measurement. The conditions line covers nominal supply and one temperature point, which disqualifies the model from any run outside that nominal box, and the class §21.4 names for exactly this case is a power-management campaign: a model correlated at nominal supply says nothing about behavior at the corners of that supply, so an entire campaign can run against a model that was never correlated outside the box. The trigger such a run fires is the first of §21.4's expiry triggers, a new operating condition, and §21.5's practice of randomizing a nominal supply deliberately beyond its legal band is a run of precisely that kind.

Solution 21.3. The AMS boundary can discharge the algorithm and cannot discharge the distribution. §21.3 enumerates the algorithmic obligations it does discharge: that the training state machine converges against a channel model, that it terminates rather than loops when a step fails, that its retry and fallback paths are

reachable and correct, and that it respects the protocol's timing obligations throughout, with real models plus a few co-simulated runs covering that substantial algorithmic risk. What it cannot discharge is the obligation Chapter 20 names, the distribution of training outcomes across parts and temperature, and the platform where that one closes is the lab, on parts. The reason §21.3 gives is a property shared by every pre-silicon channel model: each trains against an idealization of signal integrity, so the spread that separates a part converging at the center of its acceptance window from one converging at the edge does not exist until there are parts. The consequence for the plan is that one row becomes two, since §21.3 requires the plan to carry both with the second closing in the lab, and requires that division to be documented before bring-up.

Solution 21.4. The two runtimes give $92.5 / 0.4 = 231.25$, the ratio §21.2's table rounds to $230\times$ for that row. One hour of real-model regression rerun with the analog solver in the loop therefore costs $1 \times 231.25 = 231.25$ hours, which is over nine days, and that is §21.3's own reading of the figure: at anything like that ratio an hour of real-model regression takes multiple days with analog solvers in the loop, and a job required overnight misses its deadline. The conclusion §21.3 draws is about scheduling and not about tuning, since it states that the runtime gap cannot be resolved by tuning: AMS co-simulation requires a campaign, a purpose-built run with a stated reason and a written result that then stops, rather than a regression tier with a nightly budget. The reason §21.2 gives against reusing the ratio is that each of its three measurements is against its own baseline, in a different design and a different year, the first row against a SPICE netlist and the other two against Verilog-AMS models of the same blocks; the three ratios measure separate things, so multiplying them would produce an unsupported composite claim, and every ratio must retain its baseline.

Solution 21.5. The cause is the shape of the bin rather than the volume of stimulus. §21.1 states that a coverage bin on a real value naming a single point can fail to be hit at all because floating-point representation has limited precision, and it gives the illustration: $0.1 + 0.2$ yields 0.30000000000000004 , which does not fall in a bin covering $\{0.3\}$, and the language standard names this explicitly as a problem for analog specifications that carry tolerance ranges. The remedy §21.1 takes from the same standard is a tolerance token attached to a single real value, which defines an implicit range so that a real measurement can land in the bin. The package of §21.5 writes the bin as `bins midscale = {[V_MIDSCALE +/- V_BIN_TOL]}`, using the absolute token, and the property that keeps it one bin is the one §21.5 states: its total width is smaller than the real interval, so a measurement near half-scale lands in it instead of falling between two points floating-point arithmetic will never produce exactly. Further runs cannot close the row because no additional sample can be exactly equal to a value the representation does not produce, so what has to change is the declaration and not the seed count, and the report's own reading of the span bins confirms the stimulus reached the region already.

Chapter 22

Solution 22.1. §22.3 reports the chain in one pass: the list starts from 3,606 faults, collapsing removes 973 and testability analysis another 144, leaving 2,489; structural cone-of-influence analysis marks 763 of those unobservable, leaving 1,726; formal observability analysis then clears a further 248, leaving 1,478 faults that reach the read-data port. The reduction therefore removed $3,606 - 1,478 = 2,128$ faults in total. Of that total, collapsing accounts for 973, the group §22.3 characterizes as dropped because an equivalent prime fault stays in the list to represent them. The remaining $144 + 763 + 248 = 1,155$ belong to the second group, since §22.3 states the dichotomy exhaustively: a collapsed fault has a representative, and every other fault is dropped by an argument that holds under any stimulus whatsoever. The two groups reconcile with the total, $973 + 1,155 = 2,128$. §22.3 gives the reason for the order directly: it reduces cost by applying the cheap arguments first, so the expensive engine sees only what the cheaper methods could not settle.

Solution 22.2. The four classes §22.3 reports are 225 neither observed nor detected, 23 detected without being observed, 966 both observed and detected, and 512 observed but not detected. The two classes holding the observed faults sum to $966 + 512 = 1,478$, the same count the reduction left reaching the read-data port, which is the arithmetic check that the classification and the reduction describe one campaign. The classification's own total is $225 + 23 + 966 + 512 = 1,726$, the faults §22.3 says entered detection analysis, so the difference is $1,726 - 1,478 = 248$, and that matches the count formal observability analysis cleared. Equal totals are not equal sets, and §22.3 forbids reading the identity as a safety verdict: a row marked not observed has two explanations its results cannot distinguish, either that the fault is safe and cannot affect the safety goal under any stimulus, or that it is dangerous and the workload failed to excite it. Before a fault left unobserved may be classified as safe, §22.3 therefore requires a structural or formal argument that it cannot violate the safety goal under any stimulus; short of that the fault is unresolved and must be resolved through better stimulus, proof, or a recorded engineering judgment with a named owner.

Solution 22.3. The FMEDA can do nothing with that percentage: §22.2 states that a single aggregate percentage across a whole block cannot be composed with anything. The property that makes it unusable is that architectural metrics are ratios of failure rates rather than of fault counts. Each failure mode of each design element carries a base failure rate computed from design data and technology data, and the metric formulas combine those rates through the safe, residual and multi-point fractions into a single number for the element and then for the part, so the campaign's detected fraction is an input applied to a weighted quantity and is not itself that quantity. §22.2 requires instead that the fault list be agreed with the safety engineer per failure mode and that results be kept separate by mode through reporting, which is what lets the FMEDA apply the base failure rates. Two campaigns with identical fault counts arrive at different metrics when one list oversamples a large but low-rate structure while undersampling a small critical one: that bias appears in no coverage report, because the report counts faults while the metric weighs rates.

Solution 22.4. The fault outside the cone of influence of every observation point may be removed. §22.3 separates the two on what their evidence quantifies over: structural and formal arguments answer under any stimulus, and a fault outside the cone of influence of every observation point cannot reach an observation point under any workload whatsoever, which is a statically established property of the netlist and scales from block to system; simulation answers only under this stimulus, and a workload that fails to excite a fault provides no evidence about its safety. The second fault therefore stays in the list as unresolved, and §22.3 leaves it three dispositions: better stimulus, a proof that it can never cause a difference at the observation point, or a recorded engineering judgment with a named owner. Recording it as safe instead is the inflation §22.3 warns against, since classifying every unexcited fault as safe can raise the reported metrics arbitrarily, and an assessor requires an argument for that classification.

Solution 22.5. The empty field is the FMEDA failure mode, and §22.5 makes it the link that keeps the plan row and the failure analysis from drifting apart: the two are maintained by different people in different tools, so each plan row should carry its failure-mode identifier, and without it the row sits off the chain that runs from the vehicle-level hazard through the safety requirement, the failure modes and the mechanism to the specific runs, the archived results and the report that quotes them. The criterion tests only the first of §22.4's three obligations, detection when required, and leaves the other two untested. The first of those is detection within the required interval, which §22.4 makes part of the requirement because a mechanism that reports an error long after the corrupted data has been consumed has not provided useful detection; the second is silence during healthy operation, and it is the one §22.4 discharges with a cross. The stimulus that cross requires is every legal power and clock transition of the watched domain crossed against the monitor's alarm, with the alarm asserted as an error, because the domain legitimately changes frequency during a low-power transition, legitimately stops during power-down and legitimately restarts with a ramp, and every one of those healthy events looks exactly like the failure the monitor exists to catch.

Chapter 23

Solution 23.1. The table's rows fall by tier as follows: C1 carries TM-01 alone; C2 carries TM-02, TM-04 and TM-05; C3 carries TM-03 and TM-06; C4 carries none. That is $1 + 3 + 2 = 6$, the table's six rows, and C2 carries the most, which follows from what §23.2 gives that tier: privileged software on the host cluster configures the interconnect, the memory management unit, the power controller and the debug fabric, so it stands at the most boundaries. Formal appears in the evidence column of TM-03, TM-04 and TM-05, so 3 of the 6 rows name it, and §23.3 states the same figure independently of the table, that three of the six rows use formal for at least part of their evidence and that TM-03 relies on it alone. C4 carries no row, and §23.2 records the reason rather than leaving it implicit: invasive physical access, up to decapsulation and direct probing of the die, is declared out of scope because counter-acting it is a packaging, sensor and countermeasure-design problem whose evidence

does not come from RTL verification, so a row against it would be one no pre-silicon activity could ever discharge, and writing the reason down is what exposes the escape risk instead of hiding it.

Solution 23.2. The report establishes non-adjacency and nothing more. §23.4 states that Chapter 15 proved the negative connectivity rows for this exact key ladder and was explicit that the result is structural: no path exists. TM-02's obligation is that key material influences no value on the memory path except through the cipher, and the path it turns on is the key bus from the ladder to the cipher core, which exists by design; §23.4 makes that the reason no cone-of-influence check can answer the question, because the cone must exist, so a check whose whole output is the absence of a cone has nothing to say about it. The result the chapter opens with is the published information-flow study of an unmodified, fully functional AES block taken from a public repository, encrypting correctly across thousands of test vectors and failing one rule, that the key must not flow to the data output, because the key exclusive-ORed with the data reaches the output pin as plaintext; there key and datapath are necessarily adjacent and a value carrying the key survives to the output regardless. TM-02's evidence column names two techniques, connectivity and information flow, and the outstanding one is information flow: it is what asks whether anything reachable downstream of the key bus depends on the key other than through the cipher's intended transformation.

Solution 23.3. The selector's reachability unseats TM-06. §23.5 states that physical side channels and fault injection are nominally C3 capabilities, requiring access to the package, and that software-controllable mechanisms for scaling voltage and frequency and for monitoring power consumption let an attacker mount those attacks without physical access; TM-06's obligation is that no key material be recoverable from the engine's power signature against a C3 adversary, so its justification was exactly the physical-access premise a software-reachable selector removes, and §23.5 says every property justified by the adversary needing physical access is then unsupported. TM-03 survives because its evidence never rested on that premise. Its obligation is that a debug request addressing the key region is never granted, unlocked or not, and §23.3 reads the assertion discharging it as a constraint on the grant rather than on the request: the adversary controls the request, only the grant belongs to the design, and constraining an adversary-controlled signal cannot establish a design guarantee in the first place. An invariant on the grant therefore holds however the request arrives and whatever tier produced it, which is why the same assertion deliberately does not mention the unlocked state either. The check §23.5 leaves to a verification engineer is to examine which side of the trust boundary contains the selector's enable, and §23.5 says that examination needs no analysis of the cryptography and none of power analysis.

Solution 23.4. The empty part is the activation condition, which §23.4's table describes as when tracking starts and gives, for this engine, as from the cycle a key is loaded rather than from reset. The shared-memory graphics scenario shows what a rule missing it forbids: there the rule must track everything one context wrote, starting at the switch into that context, and without the activation condition it forbids

that context's data from ever reaching memory, contradicting the intended memory writes. Of the two exception entries, §23.4 licenses the cipher core's ciphertext output as an argument, because ciphertext depends on the key and without that exception the rule fails immediately and correctly, so the flow it permits can be argued as declassification of a trusted transformation; the engine's busy flag records an unresolved defect instead, and §23.4 names that pattern in as many words, a signal added to the ignore list because the rule kept failing. The cost of the second entry runs past one signal: §23.5 makes the engine's completion signal a sink for a flow rule of exactly this kind, so an ignore list that grows into the engine's control signals gives up the timing claim §23.5 sets beside TM-06, and §23.4 warns that widening an exception by one signal declares a whole port secure. Before a review can accept either entry, §23.4 requires every exception to carry a row of its own, a written argument for why the flow it permits is safe, and the same expiry and owner as any waiver, since a widened exception is indistinguishable from a passing property.

Solution 23.5. The two claims do not belong in one row. §23.5 states it directly: the timing claim has a named sink and should receive its own row when found in review, precisely so that the distinct boundary TM-06 governs is preserved. §23.6 gives the property that separates them, that a verifiable security property must be expressible entirely as signals in an available model, as a source set, a sink set, an activation condition and an exception for a flow rule, or as an antecedent and a consequent for an invariant. The timing claim passes on the first form, because the operation's start and its completion are both signals and no key bit reaching the second is an ordinary flow rule; TM-06 fails the test because its physical quantity has no model representation and no signal list at all, so the model cannot settle it. The platforms follow from that split. The timing claim is answered pre-silicon, as an information-flow rule with the completion signal as its sink, since §23.5 states that a cycle count is a signal-level fact and a latency dependence on key material is present in the RTL and can be reasoned about there. TM-06's power claim is answered only on the fabricated part with measurement equipment attached, which §23.6 names as the one place the question exists and §23.5 puts on Chapter 20's calendar, leaving TM-06 meanwhile as a conditional row carrying a named countermeasure, a proxy comparison with its limitations written down, and a leakage assessment with a date and an owner.

Chapter 24

Solution 24.1. The crossbar serves five manager ports, so the interconnect must append enough bits to tell five sources apart: two bits distinguish four, which is one short, and three bits distinguish eight, which suffices, so three is the least width that will do. Added to the four-bit manager-side identifier, that gives a subordinate-side width of $4 + 3 = 7$ bits, the figure the designer states in the review. The consequence for the proposed cross is that it can never be hit: two managers presenting the same manager-side identifier arrive at the subordinate with different identifiers, so the cell the cross was built to sample does not exist by construction, and §24.2 records what such a cell costs, staying at zero without reporting a failure while absorbing stimulus effort during closure, until someone excludes it who no longer remembers whether it

was impossible or merely unreachd. The minutes record a replacement rather than a deletion because the hazard the cross was aimed at is real and would otherwise stay unmodeled: §24.2 puts its actual precondition at a second write address accepted while an earlier burst is still unfinished on the write-data channel, and that is what the replacement cross must sample. Recording the replacement and its reason is also what stops the original cross from being proposed again in the next generation.

Solution 24.2. Assignment three is the plan row for splitting at the 4 KB boundary on the 64-bit path, and its exclusion turns on the stimulus generator's constraint solver budget, which §24.4 states caps a 2D row at 2048 bytes. A beat on that path carries 64 bits, that is 8 bytes, so a maximum-length 2D row is $2048 / 8 = 256$ beats, one maximum burst. A straddling 2D row therefore splits into pieces whose beat counts sum to at most 256 and none of which can itself be 256, which is why the six affected cells stay empty. They belong in a waiver rather than a plain exclusion, and that is the whole lesson of the assignment: the cap is a team decision about solve time rather than a property of the design, so the record has to name the decision and has to expire, and §24.4 states that telling those two kinds of dead cell apart is what the four weeks are for. The exit criterion requires three things of the record: the generator's constraint solver budget named as the reason, an owner, and an expiry. §24.4 also names the failure the criterion exists to prevent, writing the cells off as a property of the design rather than as a choice the team made.

Solution 24.3. The second reply holds outcome ownership. §24.1's test is to ask an owner what would have to be true for their answer to be wrong: an artifact owner describes a bug in the artifact, and a monitor sampling the wrong signal is a bug in the block environment; an outcome owner describes a condition of the design they have not yet reached, and the accelerator never having run while another requester competed for memory is exactly such a condition. §24.1 states that a project unable to obtain the second kind of answer for a feature lacks an outcome owner for it. The escape the chapter opens with fell through the first of the three seams, block-to-system: each block arrives from its own team and is a black box to the SoC verification team, which has neither the time nor the background to understand the inner workings of several complex IPs at once, and §24.1 calls that a structural limitation rather than a staffing failure. The gap closes on the block side, where the internals are understood, by naming an owner for each cross-block outcome, with an obligation to state which system conditions the block environment did not present. The opening is that omission exactly: the accelerator's environment drove its ports directly, as planned, every artifact had a named owner, and demonstrable accelerator correctness while another component competed for memory had no owner at all.

Solution 24.4. The waiver line belongs on the checklist. §24.5's rule is that every line of a gating checklist must name an artifact a skeptic could regenerate, and whether each waiver carries an owner and an expiry date is objectively true or false: anyone can open the waiver list and look. Adequacy of the coverage model for the block's risk records only someone's willingness to make that judgment, and the chapter's pitfall list names adequate, sufficient and acceptable as the words that

mark a checklist line as a judgment recorded without review. The rejected obligation belongs in a named review item instead, and §24.5 requires two things of it there: the person who can make the judgment present, as its radar front-end example puts the DSP architect in the room, and a tolerance written down before the run rather than after it. §24.5 gives the reason for the split, that combining the mechanical obligation with the judgment conceals an unmade judgment: the mechanical half was always true, while the acceptance half had no owner, no threshold and no record of consideration.

Solution 24.5. The record breaks all four of the mechanics §24.2 names. The pre-read line breaks the first: the artifact should circulate before the meeting, which should open by asking whether people read it, and a plan handed out at the start makes the review a first reading, with the reviewers an audience rather than reviewers. The chair line breaks the second: someone other than the author should chair, so the author can answer questions instead of managing the clock. The actions line breaks the third: every finding should become an action with a name and a date, closed before the next gate rather than at it, and both findings to be addressed before sign-off carries no name, no date, and closure at the gate it should have preceded. The filed line breaks the fourth: the record should attach to the object reviewed, because links directly to plan objects are what keep the history retrievable during audits and quality reviews, and a page of meeting notes leaves it isolated from the plan. The assumption line is the counterparty rule's case: an assumption naming a team is not closed until someone from that team has read it in this review and named the row that discharges it, so the line cannot be recorded as closed while the SoC integration team has no representative in the room, and where no such row exists the review writes one, as §24.2's coherent-hub case does on both plans rather than on one.

Chapter 25

Solution 25.1. The framework's rate is a fraction of 56 properties: $0.89 \times 56 = 49.84$, so the figure stands for 50 correct out of 56, and $50 / 56 = 0.893$, which rounds to the 89% §25.2 prints. The ablation's rate is a fraction of every candidate the bare model produced: $6 / 71 = 0.0845$, the 8% the same section gives, of which 23 were syntactically correct and 6 passed proof. The 89% is the figure scored after removal. §25.2 states the condition twice, once in the sentence that introduces the result and once in the fourth column of the evidence table, which records that generations referencing unmappable signals were removed by human engineers before scoring. That removal changes the population the denominator counts: 56 is a set a person has already filtered, while 71 is the raw production count, so the two rates are measured over populations that are not comparable and their quotient measures nothing. What the pair does support is the section's own claim that the engineering around the model does most of the work, together with its warning that such engineering does not transfer for free to a specification written in a different house style.

Solution 25.2. The classifier proposal reports evidence about the use it proposes, and the assertion pilot does not. The property the first measurement has is a baseline the team owns, namely yesterday's triage decisions; those were made by a person and written down, and they remain available to disagree with, which is why the 88% can be re-measured next quarter on next quarter's failures and seen to fall. The pilot's proof result measures agreement between a generated property and the design it was generated to check, so it has no independent reference at all. §25.3 supplies the mechanism behind that verdict: a property derived from the design cannot disagree with the design, and the worked defect there shows a property that passes on today's RTL while demanding a rule the specification never stated. The passing proofs therefore establish compilation and consistency with the current RTL, which is a real result about the flow's mechanical output. They do not establish intent, the only thing a specification-derived property exists to capture, and §25.4 draws the consequence by ruling a passing proof out as evidence at the intent gate.

Solution 25.3. Ranking and selection over the existing nightly go to immediate adoption, because the first two criteria of §25.5 hold and the third does not: the team owns the runtime baseline and the coverage database used for checking, and no intent judgment is needed for either. §25.1 attaches one condition to that adoption, which is that a compressed suite detects whether a commit breaks behavior the suite already checks while the slower full sweep with fresh seeds must survive alongside it, since ranked regression simulates no new scenarios. Property generation from the crypto engine's specification goes to a pilot in a non-gating tier, because the third criterion applies: acceptance turns on an intent judgment that no run can settle, so the flow's throughput is that judgment's throughput and the reviewing capacity has to be budgeted before the pilot rather than discovered during it. §25.5 also asks the pilot to record a team-owned statistic from the first batch, the fraction of generated properties surviving the intent gate. Supervised classification of the video codec's failures qualifies for immediate adoption only where a labeled failure history already exists, and §25.1 shows that condition unmet for this block: golden-model mismatches dominate its failure population, and their useful label is a frame region and a coding tool, neither of which appears in the log unless somebody puts it there. The work that comes first is therefore labeling, and §25.1 names the affordable form of it, which is recording the triage engineer's confirmation in a form a model can read.

Solution 25.4. The two empty fields are the identity and version of the model underneath and the revision of the specification the artifact was given. Without the first, the review cannot tell which result the recorded flow version stands for, because §25.5 makes model identity a parameter of the result and cites the same flow varying from 4.33% to 41.43% end-to-end success on one benchmark tier according to which model was underneath. Without the second, a later reviewer cannot tell whether the property predates the change that made it wrong, which is the question a field failure two years after tape-out asks first. The filled entry the field list rejects is the human-gate line, which carries one batch signature for the whole batch. §25.5 requires a name for each artifact and the specification sentence used in review, and states that a batch signature is insufficient. The reason it gives is the division of labor between the two kinds of record: machine output records audit the machine,

while acceptance records audit the human decision that is under review, and a signature over a batch records no decision about this artifact.

Solution 25.5. §25.3 denies the inference because a generator's errors are not distributed the way a human author's are. A person writing a batch of properties makes idiosyncratic mistakes that are uncorrelated across the batch, so a sample estimates a rate and reading five of forty is defensible. A generator reproduces one misreading of one recurring pattern identically in every property derived from it, so a sample of five either catches the entire class or misses the entire class and yields no estimate of an error rate at all. The property that decides the point is that correlated errors make sampling uninformative, and §25.4 turns it into a rule of the flow: a review covers every artifact in a batch or rejects the batch. The two unmet admission conditions belong to the second and third human gates. The admission gate asks a generated check to earn a gating tier by running clean across a window of known-good history, and that window is the evidence, so without it the first false failure on a Monday morning can prompt engineers to bypass the check with a plusarg. The ownership gate asks that the responder to a failure be named in the file at merge, before the first failure, so that without a name the on-call engineer can read the code without being able to say what it was for, and the record cannot confirm that the accepting engineer was able to explain the artifact without regenerating it.

Solution 25.6. The two models within six points of each other on the easiest tier are the ones reporting 43.19% and 48.77%, since $48.77 - 43.19 = 5.58$ while the third is more than twenty points below both. On the hardest tier those same two report 9.00% and 41.43%, so their separation there is $41.43 - 9.00 = 32.43$ points. Across all three models the hardest-tier spread is $41.43 / 4.33 = 9.57$, a factor of more than nine between the best and the worst run of one unchanged flow. A proposal may therefore quote a completion rate only with the model named alongside it, because §25.2 states that the result characterizes the technique paired with one model rather than the technique. Two further conditions travel with the figure: the flow's own authors summarize overall efficacy at around 40%, and §25.2 reads that as 40% on designs of the benchmark's block scale. §25.5 makes the same requirement a field of the provenance record, which is where the identity and version of the model underneath have to be written down.

Chapter 26

Solution 26.1. The buffer holds $64 \times 1,024 = 65,536$ bytes, that is $65,536 \times 8 = 524,288$ bits. The first operating point spends $128 \times 4,096 = 524,288$ signal-cycles and the second spends $256 \times 2,048 = 524,288$, so both fill the same store exactly and the storage per traced signal per cycle is $524,288 / 524,288 = 1$ bit. Doubling the second operating point's signal count leaves the same store to spend, so $524,288 / 512 = 1,024$ cycles remain and the rectangle has been turned on its side rather than enlarged. The rectangle stops being adjustable at tape-out, because every shift-right mechanism writes into a fixed store and §26.2 states that lab work cannot enlarge it. The consequence §26.2 draws from that limit is the obligation on post-silicon

debug to deliver a pre-silicon reproducer, short enough to simulate and small enough to load while still failing, since an explanation alone does not meet the requirement. §26.3 states the same fact as a durable property of the problem, in that observability is designed in a year before it is needed and its fixed capacity is shared between signals and cycles.

Solution 26.2. One DMA channel's 192 bits of architecturally visible state yield 2^{192} configurations, which §26.3 puts at about 10^{57} , a space of which simulation visits a vanishing fraction. An engine a thousand times faster buys ten further bits, because $2^{10} = 1,024$ is the power of two nearest a thousand, so the reach moves from 192 bits to $192 + 10 = 202$ bits and the arithmetic is a multiplication against an exponential. The durable property the result supports is the first of the six, that state-space size grows exponentially with the number of state bits and abstraction is therefore necessary; §26.3 gives the reason in the same sentence, since the space grows faster than any engine. The two qualifiers §26.3 attaches to the figure are its subject and its predicate: the bits are *one DMA channel's*, and they are the *architecturally visible* state. Dropping the subject makes the count a claim about some other object, and dropping the predicate makes it a claim about every bit the block holds, so either substitution changes the exponent while leaving the sentence looking the same. §26.3 attributes the arithmetic behind the figure to Chapter 3 rather than deriving it here, which is where the register map has to be read.

Solution 26.3. For the block-level DMA environment the binding constraint is knowing what to check, and for the flagship firmware run it is cycles. §26.1 states that cycles have long ceased to bind for a block environment the size of the reference SoC's DMA because its nightly suite fits in a night, and that they still bind firmware runs on the flagship. The sentence that makes the answers differ is the one declaring that this constraint has not disappeared and has been relocated by level, which §26.1 supports with the 2024 observation that plateauing processor frequency scaling leaves simulation-based techniques struggling and makes acceleration essential for many designs. §26.1 draws the career consequence from the same sentence, asking that the model of what must be true be assessed alongside throughput at block level while cycles may still be the constraint on a full-chip firmware workload. One program-wide answer commits the first pitfall of the chapter's list, reading the bottleneck as industry-wide. Its symptom is a methodology decision justified by the claim that the industry has moved past simulation throughput, asserted on a block whose regression does not fit in a night, and its cure is to identify the constraint per level and per block before any reallocation.

Solution 26.4. The scenario of §26.2 supplies the shape of the argument: a proven property retires the plan rows it discharges, so the runs that existed only to exercise those rows can be removed, while rows proven only up to a bound and reachable in the lab but not diagnosable within its trace buffer keep their seeds. On the flagship DMA the 4 KB rows are of the second kind, and the reason is the qualifier the durability audit of §26.3 puts in the left-hand column of the row for the proven boundary property, that a proof is a statement about a model, under assumptions, at a binding point somebody chose. The opening bug establishes that the model was the previous

generation's backend: the proof was sound and it entered the new plan as an already discharged row, so nothing in the program ever asked whether it covered the new burst splitter. The chapter adds the claim the plan was missing, that a data-integrity check crossing into a coherent domain is a different claim requiring different evidence, which is why the scoreboard that caught the ancestor saw nothing. A halving commits the third pitfall of the chapter's list, reduced block-level verification between the two ends, whose cure is that middle-of-the-flow work should be cut only row by row with a retirement argument and never by percentage. The proposal's second ground fails for the same reason as the scenario's: the prototype can reach the behavior without being able to diagnose it inside the fixed trace rectangle.

Solution 26.5. The empty field may carry the three positions side by side, each with its own measure and its own named gaps, and it may not carry one aggregate or one percentage. §26.4 gives the reason as an absence of a common measure. A functional sign-off aggregates coverage and plan rows; a safety argument aggregates fault classification against per-level metric thresholds; a security result is a list of named claims and named gaps with no percentage available at all. The section adds that there will not be one soon, so the three incommensurable results reach the same tape-out review without an accepted way to state a combined position. The first defective entry is the inherited-rows line, which carries the boundary proof forward as verified in the previous generation. It fails the practice §26.4 puts first, that an inherited row should be reopened and its assumptions reread against the new integration, and the section names this as the step the flagship's DMA program skipped. The second defective entry is the register of gaps, recorded as not maintained for this row, which fails the practice asking for an explicit register of what the current program is not verifying, gaps as well as waivers, on the ground that named gaps can be reviewed while unnamed ones remain surprises.

Solution 26.6. The first statement is contingent. The durability audit of §26.3 places the number of axes and the count of 72 cells in the contingent column of the row for the 2D cross at the 4 KB boundary, together with the constraint solver budget that forecloses the maximum-length straddle and the fact that a person wrote the bins; what the same row places in the durable column is that the cross is a chosen set of equivalence classes and that 100% measures its extent rather than its fidelity. The second statement is durable, and it is the third of the six properties §26.3 lists, since every practical oracle checks a projection and the projections must be combined so their blind spots do not overlap. §26.3 illustrates it where an equality oracle serves a bit-exact pipeline while the receive chain one stage upstream needs a band that no engine can convert into an equality. The third statement is contingent, and §26.3 says so directly by grouping the capacity limits of formal with the claims that describe current tools and practice rather than the problem. The chapter's fourth pitfall states what follows for a rule found to be contingent: under the thousandfold test, a rule changed by a better engine is a budget decision that belongs in the plan for reconsideration.

Index

Page numbers are those of the printed edition. They are kept in the web version as a reference to it; a reader of the web version has the browser's own search, which no index can improve on. The index covers the chapters: it does not list the pages of the glossary, of the solutions or of this index, so a term is listed where the book discusses it and not where it is defined in a table or repeated in an answer.

- **(test, seed) matrix** 233, 236, 247, 573
- **action** 31, 37, 68, 73, 119, 122, 128, 184-186, 188, 202, 220, 225, 235, 327, 472, 484, 494, 496, 516
- **activation condition** 467-469, 473
- **active vs passive agent** 192, 202
- **activity** 29, 31, 40, 46, 71, 81, 101, 107-108, 110, 113, 124, 130, 177, 179, 181, 184-186, 196, 201, 207, 215, 236, 242, 298, 305, 311, 323, 334-335, 367, 402, 407, 438-439, 452-453, 461, 470, 481, 489, 505, 523, 525, 530-532, 582
- **ADC** *see* analog-to-digital converter
- **advanced extensible interface** 24, 31, 33, 38-39, 41, 43, 49, 51, 57-58, 63, 66, 83, 86-87, 89, 92-93, 104, 108-109, 113, 125, 130, 137, 146, 152, 154, 159, 162, 168-169, 176, 189-190, 192, 208-209, 215-216, 220, 223-224, 228, 231, 236-237, 241-242, 245, 274-275, 277, 280, 290-291, 296, 303, 314, 326, 338, 343, 345, 354, 363, 373, 396, 413, 458, 463-464, 477, 480, 522
- **advanced microcontroller bus architecture** 33-34, 38, 49, 63, 83, 104, 125, 152, 168, 189, 192, 208, 215, 231, 277, 283, 291, 296, 303, 314, 360, 373, 477
- **advanced peripheral bus** 33-34, 38, 66, 69, 75, 83, 117, 124, 157-158, 193, 200-201, 204, 206, 238, 255, 260, 291, 296
- **adversarial stimulus** 472
- **adversary capability** 459-462, 475, 575
- **agent** 65, 70, 101, 120, 155-158, 162, 165, 167, 190-196, 202-207, 283, 464, 501, 504-505, 512, 516, 518, 534
- **agentic formal flow** 504, 519
- **always-on cell** 383, 390
- **AMBA** *see* advanced microcontroller bus architecture
- **AMS** *see* analog and mixed-signal
- **AMS co-simulation** 422, 426-427, 435
- **analog and mixed-signal** 157, 357, 422, 424-428, 435-437, 457, 579
- **analog solver** 424, 426-427, 435-436
- **analog-to-digital converter** 422-423, 425-426, 431, 433, 435, 528
- **analysis port** 149, 157, 193, 205
- **APB** *see* advanced peripheral bus

- **application-specific integrated circuit** 25-29, 34, 36-38, 49, 59, 63, 66, 75, 78, 82-83, 104, 107, 125, 137, 142-143, 189, 191, 207, 270, 292, 296, 298, 313, 352, 363, 372-373, 395, 417, 420, 438-439, 457, 473, 477, 497, 524, 531-532, 534-535, 577, 580, 582
- **approximately timed model** 360
- **arbitration mode** 197, 205-207, 579
- **artifact ownership** 480-482, 495
- **ASIC** *see* application-specific integrated circuit
- **ASIL** 244, 409, 439-440, 442, 452, 457
- **ASIL decomposition** 440
- **assertion** 28-29, 31, 35-37, 40, 42, 47-48, 56-58, 60-62, 69, 71-74, 91, 100, 102, 107, 110-111, 118, 122-125, 145, 148, 157, 161-163, 171, 191, 195, 204, 209-215, 217-221, 223-231, 254-255, 260-261, 263-264, 266-270, 275, 277-279, 282, 284-285, 287-289, 291, 293-296, 298, 314, 318-323, 326, 328-329, 338, 342, 346, 349, 351, 362, 365, 367-368, 377, 384-386, 400, 424, 429, 448-450, 454, 464-466, 481, 483, 488, 490, 499, 503-505, 507-508, 510, 518, 520, 524, 526, 528, 530-531, 535, 571-572, 575-576, 579
- **asset** 34, 39, 204, 410, 413, 459-465, 467-468, 471, 475-476, 488, 575
- **assumption** 39, 45, 67-69, 74-77, 79-80, 82, 97, 101, 103, 118, 133-134, 141, 151, 153, 164, 171, 183, 209, 213-214, 226, 229-231, 267, 272-278, 282, 284-295, 297-298, 303, 307, 311-312, 315-316, 318-323, 327-330, 339, 345, 359-360, 377, 381, 387, 389, 407, 411, 414, 417-418, 422-423, 431-432, 441, 454, 460, 464, 466-467, 471, 474, 482, 484-486, 494-497, 528-532, 535, 571-572, 575-576, 578
- **assumption register** 274, 278, 284, 290-291, 293, 315, 320-321, 328, 330, 431
- **asynchronous FIFO** 259-260, 269-270
- **attribute** 86, 88, 90, 92-93, 97-100, 102-103, 106-107, 110, 112-116, 119, 121, 123-124, 128, 248, 264, 270, 310, 316, 452, 456, 500, 503, 522, 574
- **autonomous monitor** 156
- **AXI** *see* advanced extensible interface
- **base failure rate** 442, 454
- **batching** 334, 343, 350
- **bind** 69, 93, 185, 197, 209, 214, 218-219, 229-231, 320, 374, 523-524, 526, 532-533
- **binding constraint** 523, 532
- **bins** 43-44, 48, 79, 88, 94-95, 106, 110-117, 120-124, 128, 130, 132, 137-140, 177-178, 186, 188, 238, 244, 246, 315, 322, 329, 423-424, 433-434, 436, 501, 529
- **blended percentage** 324, 328-329
- **block verifier** 481
- **Boolean satisfiability** 179, 504-505, 507

- **bound checker** 228, 277, 291, 378
- **boundary cover** 320-321, 327-329
- **bounded model checking** 278-279, 295, 409
- **bounded proof** 53, 61, 135, 266-267, 273, 281, 288, 294, 299, 321, 327, 527
- **bring-up** 24, 30-32, 35, 64-67, 69-70, 72, 80-81, 88, 131, 146, 148-149, 163-164, 169-170, 186-187, 204, 219-220, 228, 233, 253, 300, 305, 338, 352, 357, 359, 368-370, 376, 378, 391, 396-397, 402, 404-405, 418-420, 423, 428, 436, 441, 463, 486, 488, 532
- **bug curve** 65, 74, 76, 127-132, 138, 140, 229, 323, 438, 458, 573
- **bug escape** 24-25, 60, 358
- **campaign** 45, 52, 106, 111, 124, 176-177, 188, 195, 234-236, 241-243, 248, 251, 253-255, 268, 299, 314-315, 338, 346, 348-349, 352, 370, 390, 394, 427, 430, 434-435, 440-450, 452-457, 466, 470, 472-473, 476, 575, 580-581
- **capability tier** 461, 466, 470, 474-475
- **CDC** *see* clock-domain crossing
- **CDG** 120, 124, 500, 517, 520, 582
- **champion** 489-490, 494-495, 497, 582
- **checking contract** 158, 161
- **CI** *see* continuous integration
- **clock domain** 25, 28, 34, 54, 254, 260, 263, 265, 269-271, 352, 371, 395, 405, 581
- **clock snooping** 488
- **clock-domain crossing** 34, 133, 135, 222, 237-238, 252, 254-255, 258-265, 267-271, 298, 356, 362, 388, 407, 525, 536, 579-580
- **clocking block** 149, 156, 164, 240, 242, 391
- **coarse-grained checking** 358, 371
- **common timestamp source** 367
- **common-mode error** 58, 147, 302, 310, 360
- **concurrent assertion** 211-212, 229, 365, 464
- **condition set** 423, 435, 528
- **conditional sign-off** 127, 134, 136, 141, 471
- **cone of influence** 282, 294, 299, 323, 377, 444-445, 456-457, 468
- **confidentiality** 416, 458-459, 461, 464, 467, 469, 476, 515, 575, 581
- **configuration database** 190-195, 205
- **connect module** 426-427
- **connectivity checking** 299, 311
- **constrained-random** 24, 27-28, 31, 36, 39, 43, 60, 65, 70, 72, 86, 91-92, 99, 103, 107, 120, 131, 137, 142, 169, 171-172, 181, 183, 186-187, 204, 236, 243-245, 248, 256, 287, 303, 316, 325, 336, 339, 347, 351, 355, 363, 371, 390, 426-427, 430, 440, 459, 475, 522, 524, 527, 533, 535

- **continuous integration** 204, 235, 237-238, 247, 249, 293, 526, 574
- **controllability** 50, 54-56, 61, 66, 95, 98, 253, 359, 397-398, 400, 415, 445, 525, 528
- **corner** 31, 34, 39-40, 42, 44, 46, 53, 56, 59-61, 63, 73, 86, 88, 102, 104, 110, 113, 127, 132, 136-137, 140-141, 150, 155, 171, 173, 175, 177, 181, 183, 187, 266, 290, 307, 315-317, 322, 324, 328, 336, 366, 375-376, 379, 381, 390, 392-394, 400, 423-425, 430-431, 480, 482, 493, 496, 575
- **correlated errors** 517
- **correlation expiry trigger** 430
- **correlation tolerance** 429
- **corruption** 24, 31-32, 37, 55, 62, 71, 98-99, 108, 136-137, 222, 229, 265, 279, 354, 382, 385, 388, 394-395, 399, 410-411, 413, 416-417, 419, 444, 450, 481
- **counterexample** 31, 37, 54, 72, 116, 161, 214, 224, 272-273, 275, 278-282, 284, 286, 290, 292-295, 309, 311, 313, 316-317, 321, 325-328, 504-507, 516, 579
- **counterparty rule** 484, 486, 496
- **cover property** 110, 213, 217-218, 221-222, 224, 261, 264, 303, 319, 327, 386, 449, 465, 469, 509
- **coverage closure** 45, 64, 66, 106, 111, 119-120, 124-125, 131, 139, 250, 302, 440, 472
- **coverage differential** 234, 247, 323
- **coverage hole** 65, 78, 119, 122, 276, 316, 319, 327, 330, 417
- **coverage model** 31-32, 35-36, 43, 47, 60, 65, 69-70, 72-73, 76, 90, 94-95, 98-99, 103-104, 107, 109-111, 114-115, 117-125, 128, 132, 134, 139, 141-142, 155, 158, 162-163, 171-172, 174-175, 177-179, 187, 193, 195, 203-205, 224-225, 233, 295-296, 303-304, 308, 312-314, 316, 322-323, 331, 358, 388, 392, 411-412, 414, 417, 422-423, 434, 439, 459, 480-481, 483, 485, 487, 492, 495-497, 500-501, 517, 522, 529, 533, 570, 578
- **covergroup** 43-44, 92-94, 96, 103, 106, 109-113, 115-118, 124, 204, 257, 316, 323, 325, 412, 433-434, 436, 571, 578
- **coverpoint** 43-44, 48, 62, 94, 106, 110-111, 113-117, 122, 130, 137, 141, 177, 187, 412, 424, 433, 436
- **cross coverage** 31, 127, 138
- **cutpoint** 282, 295, 321-322
- **dangerous-undetected fault** 446
- **data coherency** 259, 262
- **data tagging** 153, 167
- **datapath equivalence** 308
- **deadlooping** 512
- **debug-time share** 130, 135, 139-140, 505
- **decay** 63, 95, 131, 171, 258, 268, 494, 500, 516, 582
- **defeathering** 402

- **definition of done** 68
- **delayed lockstep** 440
- **dependent failure analysis** 450, 453
- **design under test** 42, 44, 47-48, 65-66, 68, 95-97, 101, 120, 132, 172, 174, 177, 180, 186, 193, 204, 219, 300, 304, 341-343, 423, 501, 532
- **design-for-debug** 29-30, 398-399, 476, 573
- **detection distance** 155, 162, 317, 338, 346, 367, 400, 529
- **detection point** 443, 445
- **diagnostic coverage** 438, 440-442, 445, 447, 454-455
- **digital signal processor** 33, 260, 281, 307, 319-320, 329, 362, 369, 387, 401, 447-448, 493
- **direct memory access** 24-25, 29, 31, 33-34, 39-41, 43-48, 50-52, 55-58, 60-62, 64, 66, 73, 77, 81, 86-90, 92-94, 96-99, 103, 106-108, 110, 112-115, 117, 127-130, 132, 135-137, 141-142, 146, 151, 154, 157, 159, 162-163, 169-170, 173-174, 176-178, 181-182, 184, 188, 190, 193, 198, 202, 209, 214, 222-223, 230, 233, 236-237, 240-243, 245-246, 248, 266, 274-275, 277, 280, 287, 291, 294, 303, 310, 312, 325-326, 334, 336, 343-345, 347, 351, 354, 357, 359, 362-363, 365-367, 370-371, 374, 383, 396, 404, 409, 412-413, 418-419, 452, 463, 472, 480-481, 486, 490, 522-523, 525, 527, 529, 531, 533, 535
- **direct programming interface** 344-345
- **directed test** 31, 44, 60, 63, 69, 74, 77, 86, 91, 119, 125, 139, 169-172, 181, 186-187, 189, 235, 244, 248, 256, 284, 297, 299, 326, 328, 364, 388, 460, 465, 501, 520-522, 529, 535, 582
- **disable iff** 43, 58, 93, 209-210, 212-213, 217-218, 221-222, 224, 227, 229, 261, 264, 274, 276-277, 283, 299, 303, 319, 365, 385-386, 449, 465, 469, 508-509
- **disposition** 65, 71-72, 74, 81, 127, 133-134, 136, 140-141, 295, 304, 310-312, 322, 329, 406-407, 414-416, 418, 456, 468, 474
- **distribution constraint** 179
- **DMA** *see* direct memory access
- **double assignment** 325, 329
- **DPI** *see* direct programming interface
- **drain time** 196, 202, 205
- **DSP** *see* digital signal processor
- **durability audit** 529, 533
- **durable property** 532-533
- **DUT** *see* design under test
- **DVFS** *see* dynamic voltage and frequency scaling
- **dynamic voltage and frequency scaling** 267, 387-388, 394, 396, 470
- **ECC** *see* error-correcting code
- **EDA** *see* electronic design automation

- **electronic design automation** 38, 49, 63, 83, 104, 125, 143, 149, 167, 189, 207-208, 215, 231, 243, 265, 270, 296, 313, 342, 352-353, 373, 395, 420, 457, 477, 497, 506, 520-521, 530, 534-536, 577-578, 582
- **emulation** 30, 34, 167, 183-185, 187, 203, 228, 238, 243, 271, 318, 321, 333, 335, 337-340, 346-349, 351-353, 357, 363, 372-373, 397, 410, 471-472, 477, 482, 573, 576, 579-580, 583
- **emulator** 119, 150, 165, 167, 210, 262, 268, 334-340, 346-349, 351, 354-355, 357, 361, 363, 365, 370, 471-472, 476
- **engine selection** 316
- **environment vs directive constraint** 173
- **equivalence class** 445, 529
- **errata sheet** 33
- **error detection latency** 406, 418-419, 580
- **error injection** 42, 87, 93, 96, 99, 101, 155, 443
- **error-correcting code** 45, 56, 157, 165, 195, 299, 432, 438, 440-441, 443, 446-447, 449-450, 452
- **errors re-found** 234, 239
- **escape** 24-25, 27, 30-33, 35-36, 47, 53-54, 59-61, 65-67, 71, 74-75, 80, 82, 94, 97-99, 127, 134, 137-142, 289-290, 358, 374, 378, 384, 386, 390, 392, 397, 399, 410-414, 417-419, 461, 480-481, 492-496, 513, 522, 526, 530, 532, 535, 570, 573, 577, 585
- **escape analysis** 31, 35, 127, 134, 137-141, 410-412, 414, 417, 493, 513, 522, 530, 532, 535
- **evaluation attempt** 209, 212, 231
- **exception clause** 459, 474-476
- **executable verification plan** 95
- **factory** 130, 190-191, 193-194, 204-207
- **failure signature** 340
- **fault injection** 438, 441-443, 447, 451-452, 456-457, 470, 477, 581
- **fault reaction time interval** 447
- **fault-tolerant time interval** 438, 447
- **feature extraction** 67, 87, 103
- **fidelity** 48, 111, 121, 123-124, 305, 307-308, 361-362, 470, 483-484, 495, 528-529, 578
- **field access policy** 87, 89, 98, 198-201, 207, 312
- **field-programmable gate array** 26-27, 29, 35-38, 56, 61, 75, 80, 82-83, 138, 141-143, 167, 184, 191, 204, 208, 243, 293, 296, 298, 313, 337, 339, 352, 354-356, 358, 363, 365, 372-373, 399, 420, 439, 457, 524, 526, 532, 534-536, 577, 583
- **FIFO** *see* first-in first-out

- **finite state machine** 108, 139, 303, 306-307, 311, 383, 386, 388, 460, 465
- **firmware-driven verification** 363, 574
- **first-in first-out** 33, 42, 50-51, 88, 107, 130, 137, 142, 156, 162, 171, 197, 204, 213, 222-223, 225, 242, 259-260, 269-270, 279-280, 294, 304, 317, 398, 401, 441
- **first-silicon success** 26-27, 36, 59, 75, 78, 80, 82, 137, 141, 417, 420, 524, 532
- **first-time success** 78, 82, 97
- **first/second/third-order reuse** 191-192, 195-196
- **flaky test** 233, 240, 574
- **flow qualification** 451
- **FMEDA** 441, 443, 447, 452-457, 581
- **formal app** 249, 271, 297-298, 311, 313, 395
- **formal property verification** 53, 60, 143, 272, 296, 331, 410, 464-465, 489, 498, 572, 577, 579, 582-583
- **FPGA** *see* field-programmable gate array
- **FPGA prototype** 29, 339, 354-355, 399
- **fresh-seed yield** 132, 137, 139-140, 458
- **front-door vs back-door access** 199-201, 206, 301, 377-378
- **FSM** *see* finite state machine
- **functional verification** 25, 37-38, 49, 63, 82-83, 103-104, 124-125, 143, 147-148, 167, 189, 207-208, 231, 250, 270, 296, 313, 352-353, 373, 395, 420, 453, 455, 457, 477, 483, 497, 501, 506, 519-520, 534-536, 577-578, 582-583
- **gate-level simulation** 55, 60, 137-139, 235, 258, 307, 374-377, 381, 392-393, 395, 441, 526
- **generator/monitor duality** 171
- **GLS** *see* gate-level simulation
- **golden model** 60, 162-163, 166, 358, 412, 429, 482
- **good machine / faulty machine** 443
- **gray-coded parallel synchronizer** 259
- **grey-box verification** 56, 63
- **handshake synchronizer** 259
- **hardware redundancy** 447-448
- **hardware-assisted verification** 334, 346, 352
- **heterogeneous merge** 321, 330
- **hybrid co-emulation** 337, 339, 352, 372, 580
- **hybrid co-simulation** 361
- **hybrid flow** 251, 315, 322, 326, 572
- **hypothesis quality** 505, 507, 584
- **ICE** *see* in-circuit emulation
- **IEC 61508** 457, 580

- **IEEE 1497** 395, 580
- **IEEE 1666** 150, 361, 579
- **IEEE 1685** 199, 208, 579
- **IEEE 1800** 109-110, 124, 179, 191, 225, 231, 275, 433, 579
- **IEEE 1801** 383-384, 393, 579
- **ignore_bins vs illegal_bins** 110, 114-118, 121, 123-124, 158, 459
- **image** 39, 44, 162-163, 334, 336, 338, 347-351, 367-368, 370, 449, 462, 485
- **immediate assertion** 209, 212-213
- **implication** 32, 58, 188, 209-210, 212, 224-225, 229, 261, 264, 282-283, 300, 320, 386, 474, 487, 513, 584
- **implicit vs explicit prediction** 201, 203, 205
- **in-circuit emulation** 339-341, 351-352
- **in-fabric monitor** 357, 367, 371
- **induction** 272, 279-280, 295
- **inference** 81, 132, 140, 184, 440, 480, 515
- **information-flow tracking** 466, 476
- **instruction-accurate model** 359-361
- **instruction-set simulator** 29, 34, 55, 57, 60, 154, 167, 448, 524
- **instrumentation manifest** 370
- **integration verifier** 480-481
- **intellectual property** 27-29, 32, 36-37, 48, 61, 66, 73, 77, 83, 127, 150, 162-163, 183, 185, 191, 199, 201, 205, 207-208, 243-244, 247, 257, 263-264, 267, 269, 271, 278, 299, 338-339, 401, 417, 461, 464, 472, 476, 481-482, 488, 498, 524, 530, 536, 579
- **intent curation** 299, 305
- **intent gate** 511, 515-516
- **IP** *see* intellectual property
- **irritator** 67, 132, 135, 141, 400, 417
- **ISO 26262** 439-442, 446, 450-452, 457, 580
- **isolation** 55, 88, 103, 138, 214, 382-386, 390-394, 401, 526, 581
- **isolation cell** 382-384
- **ISS** *see* instruction-set simulator
- **joint test action group** 34, 266, 300, 357, 359, 396, 399, 404
- **JTAG** *see* joint test action group
- **known-bad variant** 165, 289, 451, 483
- **latent fault** 442, 449
- **least significant bit** 199, 207, 301, 422, 431, 434-435
- **LEC** *see* logic equivalence checking
- **leftover** 152, 160-161, 165-166

- **lemma** 278, 284-286, 291-292, 294, 318, 327
- **level-shifter** 382-383, 388, 394
- **lint** 133, 135, 164, 219, 234, 237-238, 252, 254-257, 267-268, 270
- **liveness property** 229, 572
- **localization** 397, 406-407, 410, 418-420, 571, 580
- **lockstep** 57, 440-443, 447-448, 450, 453
- **logic equivalence checking** 297, 305-308, 310-312, 375
- **LSB** *see* least significant bit
- **MAC** *see* media access control / multiply-accumulate
- **maturity stage** 64, 72, 78, 80, 480, 486, 495, 576
- **mean time between failures** 252, 258, 268, 270, 375, 395
- **mechanical gate** 511, 519
- **media access control / multiply-accumulate** 25, 34, 42, 48, 100, 110, 114-115, 134, 150, 152, 157, 181, 183, 201, 221, 241, 307, 381
- **metastability** 252, 258-259, 262, 268-271, 281, 335, 338, 345, 348, 352, 375, 392, 394-395, 407, 417, 420, 525, 584
- **metastability injection** 262, 268, 270, 335
- **methodology owner** 481, 490
- **metrics-driven verification** 127, 139-140, 142
- **middle of the flow** 523, 526-527
- **milestone** 46, 64-66, 70, 73, 78, 80, 82, 121, 125-126, 140, 236, 267, 287, 325, 331, 334, 498, 526
- **mined property** 304, 311
- **mirror vs desired value** 198
- **model checking** 53, 63, 154, 251, 272, 278-279, 295, 307, 409
- **model correlation** 428-429, 435-436
- **model register** 360, 431-432, 435
- **modport** 156, 164, 167
- **MTBF** *see* mean time between failures
- **multi-point fault** 442
- **multi-stream (virtual) generator** 181
- **must-not-happen feature** 89-90, 432, 459
- **mutation-based generation** 473
- **network-on-chip** 34-35, 118, 228, 245, 330, 334, 336, 347, 356, 363, 387, 394, 396, 404, 418, 462
- **no-connection check** 300
- **NoC** *see* network-on-chip
- **non-convergence fallback** 325, 330
- **notifier** 380-381, 392, 394

- **objection** 190, 196, 202, 205, 231, 393
- **observability** 27, 29-30, 35, 50, 54-56, 59, 61, 63, 66, 88, 98, 110, 223, 253, 315-316, 328, 338, 348-349, 352, 355, 357, 359, 368, 370, 372, 396-402, 414, 418, 445-446, 452, 465, 476, 525-528, 580
- **observation point** 299-300, 443-447, 454, 456
- **one-row-one-metric rule** 90, 323, 453
- **operational vs observational communication** 149
- **oracle** 25, 37, 43, 46, 50, 54-62, 129, 146-147, 154-155, 162, 165-166, 186, 215, 223-224, 231, 236, 305, 317, 323, 346, 350-351, 357-358, 365, 367, 371, 399, 402, 404, 412, 418, 422-423, 432, 435, 459, 472-473, 488, 503, 524, 528-530, 532, 534, 570, 574
- **orphan response** 165
- **over-approximation** 282-283, 294, 468
- **over-constraining** 176, 276, 467
- **own an outcome** 490
- **partial oracle** 57, 162, 223-224, 528, 532
- **path coverage** 108
- **peek / poke** 199-200
- **phase-locked loop** 270, 363, 402, 404, 422-425, 434, 436
- **phasing** 190, 196, 204-205
- **PHY** *see* physical layer
- **physical layer** 30, 132, 235, 334, 355, 357, 370, 376, 404-405, 423, 428, 436
- **plausible by construction** 500, 508
- **PLL** *see* phase-locked loop
- **poka-yoke** 41
- **pool** 184-185, 188, 346-347
- **portable stimulus** 169, 183, 187, 189, 353, 373, 525, 578
- **Portable Test and Stimulus Standard** 184-186, 189, 353, 365, 530, 536, 578-579
- **power domain** 34, 55, 138, 382-383, 386-388, 394, 401, 405, 462, 526
- **power intent** 374, 376, 378-379, 382, 384, 388-392, 526
- **power state table** 382, 387-388, 392, 394
- **pre-silicon reproducer** 406, 410, 414, 418, 526, 532
- **predicted-unreachable** 323, 328, 330
- **predictor** 148, 150-151, 157-161, 164-167, 201, 203, 205, 250
- **prime fault / collapsed fault** 299, 445-446, 455
- **product machine** 307, 313
- **proof core** 288, 294-295
- **proof-carrying** 516

- **property-guided monitor generation** 472
- **protocol check** 252, 254, 260, 269
- **protocol monitor vs functional monitor** 157, 161, 365
- **prototypability** 356
- **prototype-scope exclusion** 357
- **provenance record** 513, 519
- **pseudo-static signal** 254
- **PSS** *see* Portable Test and Stimulus Standard
- **QED** *see* quick error detection
- **quarantine** 241, 573, 575
- **quick error detection** 409-410, 420, 457
- **radius** 42, 321-322, 330
- **RAL** *see* register abstraction layer
- **random hardware fault** 438, 440, 455
- **random stability** 180, 187, 237, 241, 249
- **ranking** 59-61, 68, 98-99, 103, 130, 206, 243-249, 307, 335, 390, 502, 515, 570
- **RDC** *see* reset-domain crossing
- **real I/O** 355
- **real number modeling** 422, 424-426, 428, 435
- **reasoning trace** 516
- **recipe** 327, 402, 406-408, 414, 417, 419
- **reconvergence model** 40-41, 48-49, 305, 577
- **redundancy** 41, 44, 48, 80, 199, 222, 229, 447-448, 483, 495
- **redundant-state invariant** 222, 279, 318, 327
- **reference model** 47-48, 57, 61, 91, 99, 111-112, 146-147, 154-155, 158, 165, 167, 203, 252, 287-288, 309, 317, 336, 346, 369, 388, 399, 411, 448, 458, 482, 493, 528
- **register abstraction layer** 98, 198
- **register app** 301-302
- **register description language** 201, 207-208, 301-302, 314
- **register-transfer level** 27, 29-31, 35-37, 40-41, 44-45, 47-48, 51, 54-57, 59-61, 65-67, 69-76, 79-81, 87-88, 96-97, 101, 106-108, 121-122, 125-126, 131, 134, 137-140, 148, 154-155, 159, 163-164, 171, 180, 186, 192, 209, 211, 214-215, 226, 228-229, 232, 235-236, 238, 240, 244, 246, 249, 252-259, 265-267, 269-271, 273-274, 284, 287, 290, 292-293, 298, 301-311, 313-314, 323, 327-328, 331, 333-335, 337, 339, 345, 350, 354-363, 367-372, 374-377, 379, 381-382, 385-395, 397-400, 405, 407, 429, 431, 448, 451, 459, 461-463, 468-470, 474, 481-483, 488, 490-492, 495, 498-499, 503, 505-507, 509-510, 513, 515-519, 523, 525-527, 529, 532, 571, 573, 576, 579, 581, 584-585
- **regression suite** 70, 234-235, 243

- **repeat-N stability check** 241, 340, 367, 512
- **required proof depth** 288, 290, 322, 579
- **reset synchronizer** 263, 450
- **reset-domain crossing** 133, 234, 237, 252, 254-255, 263-264, 267-271, 387, 536, 579-580
- **residual fault** 442
- **respin** 24-28, 30-32, 35-36, 41, 58, 60, 78, 82, 99, 397-398
- **responder** 155-156, 158-160, 163-165, 193, 196, 364, 519
- **restore-period condition** 384-385
- **restrict** 53, 156, 187, 212-214, 320
- **retention** 34, 55, 60, 138-139, 300, 382-386, 390-392, 394, 401, 487, 526
- **retiming** 306-307, 311-312
- **retirement argument** 527, 532
- **reuse gap** 488
- **risk-driven verification** 59, 459
- **round-trip cost** 343, 349-350
- **RTL** *see* register-transfer level
- **run** 24, 26, 34, 39, 44, 49, 52, 58, 60, 62, 64-65, 71, 74, 76-78, 80, 89, 91, 95-96, 100-101, 118-120, 122-123, 128, 130-131, 134, 137, 139, 141-142, 149, 154-155, 162, 164, 170-173, 175-177, 180-181, 183-187, 196-197, 202-203, 210, 214-215, 218-219, 223, 225, 227-228, 230-231, 234-241, 243-248, 253, 256-257, 259, 267-269, 272, 275-276, 279-281, 286-287, 290, 292-295, 297, 299-300, 303, 305, 309-311, 315-318, 320-322, 325-327, 330, 334-340, 347-351, 354-358, 361-365, 367-372, 374-381, 384, 386-394, 396-397, 399, 406-410, 412, 414, 417-419, 422, 426-430, 432, 434-436, 444, 447-454, 457-458, 460, 466, 469-472, 482-483, 490-491, 493, 496, 500-501, 503-505, 507, 514-516, 519, 522, 525, 529, 532, 571, 573-576, 579
- **run manifest** 234, 237, 247-248, 327, 340, 379, 392-393, 451, 457, 490, 574
- **S1/S2/S3** 133
- **safe fault** 441-442, 446
- **safe state** 448
- **safety case** 439
- **safety mechanism** 438, 441-443, 447-449, 452, 455, 581
- **sampled value** 209, 211, 221, 229, 231, 261-262, 378, 464, 508
- **sampling temperature** 514
- **SAT** *see* Boolean satisfiability
- **satisfiable narrowing** 276
- **SCE-MI** *see* Standard Co-Emulation Modeling Interface
- **scenario** 25, 27, 29, 31, 41-42, 45-46, 48, 55, 60-61, 66, 70, 77, 79, 87-88, 95, 97, 100-101, 103, 109-113, 116, 120, 122-123, 130, 136, 138, 147-148, 150, 152,

- 155, 157, 169, 171, 173, 175, 181-187, 189, 193-198, 201, 204-206, 221, 226, 229, 235, 244, 246, 255, 257-258, 260, 263, 266, 273, 276, 281, 287, 290, 300, 302, 305, 309, 317, 320, 324-326, 336, 338-340, 346, 348-349, 352, 358, 360, 362, 364-365, 369, 376-377, 381, 386-387, 390-391, 398, 401, 405, 407, 409, 416-417, 419, 422, 424, 426, 428, 432, 440, 443, 447-449, 459, 465-466, 468, 472, 476, 482, 485, 487, 489, 493, 496, 502, 522, 525, 527, 533, 570, 572-573
- **scoreboard** 24, 31, 39, 43-44, 47-49, 57-58, 60-62, 65, 69-70, 86, 89, 91-94, 96, 98-99, 102, 111, 122, 137, 146, 149-150, 152-154, 158-159, 161, 163-167, 169-170, 179, 192-193, 196, 198-199, 202-206, 209, 215-216, 233, 242-243, 252, 315-316, 325, 342, 346, 350-351, 374, 378, 380-381, 392-393, 458, 490, 522, 528-529, 532, 535, 578
 - **SDF back-annotation** 374, 379-380
 - **SEC** *see* sequential equivalence checking
 - **seed** 24, 39, 45, 62, 64, 67, 71, 73-75, 119-120, 129, 131-132, 134-135, 137, 139-140, 142, 159, 165, 169, 179-181, 186-189, 209, 233-237, 239-249, 252-253, 266, 268, 317, 327-328, 336, 340, 344-345, 367, 376, 379, 390, 392, 422, 451, 458, 488, 490, 493, 502, 514-515, 517, 527, 535, 570, 573, 575, 578
 - **self-checking testbench** 29, 429
 - **self-help material** 490, 495, 582
 - **sequence** 30-31, 42, 51, 53-54, 60, 64, 72-73, 87, 98, 101, 108-110, 123-124, 132, 135, 138-139, 142, 146, 148-150, 152, 157-159, 161, 163, 165, 169-171, 176, 180-181, 183-185, 187-188, 190, 193, 195-198, 201-202, 204-207, 209, 211-212, 219-220, 224, 236, 241, 262, 264, 266, 270, 273-274, 276, 290-291, 301, 305-308, 320-321, 324-325, 335-336, 338, 340, 342, 355, 358-359, 363-365, 372, 376, 383-384, 387-389, 392, 394, 397, 401, 404-406, 409-410, 413, 416, 422, 426, 431-432, 435, 444-445, 449, 462-464, 472, 474, 490, 495, 501, 570, 575
 - **sequence item** 195, 204, 207
 - **sequencer** 149, 191-193, 197, 201-202, 206, 252, 342, 385
 - **sequential depth** 279, 295, 579
 - **sequential equivalence checking** 296-297, 305-308, 311-312, 314, 321
 - **shaped cross** 117-118, 121, 124
 - **shift-left** 523, 525-527, 531-532
 - **shift-right** 523, 525-527, 532
 - **side channel** 33, 459, 469-470
 - **sighting** 406-407, 414, 419, 580
 - **sign-off review** 64-66, 72, 74-76, 81, 120, 134-135, 272, 294, 483-484, 513, 519
 - **signal-level acceleration** 341
 - **silent drift** 329
 - **simstate** 382, 388, 393-394
 - **single event upset** 444

- **single-point fault** 442
- **smoke tier** 234, 248, 338, 351
- **SoC** *see* system on chip
- **soft constraint** 176
- **source cover** 469
- **spatial merge** 321
- **spec-hole log** 100, 369
- **speed adapter** 339-340, 350-351
- **SPFM / LFM / PMHF** 442, 455
- **spurious counterexample** 275, 279, 293-295, 579
- **STA** *see* static timing analysis
- **stage inflation** 480, 487
- **stale model** 502, 582
- **Standard Co-Emulation Modeling Interface** 340, 352-353, 579
- **state point** 305-308, 311-312
- **state restoration ratio** 401
- **state space** 25, 29, 50-51, 53, 59, 61, 111, 131-132, 180, 204, 273, 278-279, 283, 308, 429, 527, 532, 534
- **state-space explosion** 50
- **static timing analysis** 374-376, 381, 392-395
- **static verification** 133, 252-253, 267-268, 571
- **stepping** 32, 397-398, 412, 414-416, 418-419, 448
- **stream key** 151-153, 159, 166
- **strobe time** 443
- **structural pruning** 445
- **stuck-at fault** 444, 447
- **SVA** *see* SystemVerilog Assertions
- **symbolic checking** 300
- **system on chip** 24-25, 27-29, 31, 33-36, 38, 50-51, 54-57, 60-66, 68, 72-73, 75, 77, 79-83, 86, 88, 94, 96, 127-128, 130, 135-137, 140, 142, 146, 154, 170, 183-184, 190-192, 195, 201-202, 206, 209, 215, 233, 236, 238, 241, 243, 245, 247-249, 252, 254-257, 259, 267, 269-270, 272, 278, 287, 291, 297, 299-300, 305, 310, 312-313, 318, 320, 334-336, 338, 347, 350-352, 354-356, 358, 363, 365, 371, 373-374, 383-384, 387-389, 393-396, 398, 400, 405, 409, 412, 419-420, 422, 424, 440, 458, 464, 466, 477, 480-481, 487, 496-497, 516, 518, 522-528, 533, 535, 573, 578
- **system slowdown factor** 356
- **systematic fault** 438, 440, 455

- **SystemVerilog Assertions** 110, 209, 211, 215, 226, 228, 231, 270, 287, 296, 499, 503, 520, 524, 579
- **tape-out** 24, 31, 35, 42, 45, 60, 63, 66, 75-76, 78-79, 95, 98, 127, 133, 135, 256, 313, 334, 346, 349, 354, 357, 374, 397, 399-400, 402, 405, 407, 414-415, 418, 430, 438, 471-472, 489, 501, 503, 513, 516-518, 522, 526, 531, 533-534, 574
- **temporal merge** 321
- **test realization** 185
- **test selection** 125, 244, 249-250, 391, 502
- **the crunch** 46, 64, 78
- **thousandfold test** 532, 534
- **threat model** 289, 402, 459-463, 466, 470, 473-475, 477
- **tier** 97-98, 102, 233-238, 241, 244-245, 247-249, 268, 293, 321, 326, 328, 335, 338, 346-347, 351, 369, 379, 389-392, 427, 435, 454, 461, 466, 470, 474-475, 489, 505, 507, 511-513, 515, 518-519, 573, 575, 585
- **time bomb** 241, 247, 249
- **timing exception** 376, 393
- **timing side channel** 469
- **TLM** *see* transaction-level modelling
- **toggle coverage** 108-109
- **tolerance token** 423, 433, 437
- **tool qualification** 438, 450-451, 457
- **traceability** 67, 95, 101-102, 129, 133, 139, 142, 284, 424, 439, 451-453, 456, 463, 512, 574
- **transaction level** 148
- **transaction-level modelling** 150, 167, 194, 361-362, 373
- **transactor** 148-150, 201, 203, 334, 337, 339-345, 349-350, 352
- **transactor boundary** 334, 341, 350, 352
- **transfer function** 154, 158, 162, 165, 167, 482
- **triage** 29, 31, 76, 120, 122, 128, 130-133, 136, 140-141, 181, 190, 202, 233-234, 238, 240, 245, 247, 340, 350-351, 355, 367-372, 376, 392, 406-407, 416, 419, 468, 472, 499, 501-503, 515-518, 571, 576
- **trigger-based debug** 338
- **trust boundary** 459-462, 470, 475, 575
- **two-flop synchronizer** 252, 258-260, 262, 375, 394
- **type vs instance override** 194-195, 206
- **UCIS** 119, 124-125, 321-322, 329, 331, 530, 536, 579
- **under-approximation** 282-283, 294-295
- **undetermined** 272-273, 284, 286, 291, 293, 295, 572
- **unification layer** 323

- **unified power format** 34, 138, 382, 395, 526, 579, 581
- **unit-test fallacy** 516, 585
- **Universal Verification Methodology** 28, 38, 40, 145, 149, 167-168, 190-195, 198-199, 203-205, 207-208, 231, 256-257, 266, 341, 352-353, 373, 422, 436-437, 524, 526, 536, 571, 578-579
- **unreachability app** 302, 312
- **UPF** *see* unified power format
- **UVM** *see* Universal Verification Methodology
- **vacuous pass** 225-226, 449, 469, 505, 507, 513, 584
- **valid/invalid hole** 119-120
- **validation** 25, 27-28, 30, 38, 56, 63, 83, 168, 189, 249, 271, 339, 352, 354-355, 357-358, 372-373, 397-399, 402, 406, 412, 417-418, 420, 457, 476-477, 521, 526, 535, 574, 583
- **validity pruning** 42, 47
- **variable ordering** 175
- **verification component** 41-42, 48, 157, 191, 203-205, 488-489
- **verification gap** 32, 35, 37-38, 46, 49, 52, 59, 78, 82-83, 143, 292, 488, 498, 577
- **verification IP** 185, 205, 247, 482
- **verification lifecycle** 65-66
- **virtual emulation** 340, 352
- **virtual platform** 337, 354, 358-361, 367, 371, 373, 525
- **virtual sequence / virtual sequencer** 159, 193, 197-198, 202, 206
- **volatile** 59, 199, 206-207, 301-302
- **volume criterion** 317
- **waiver** 94, 113-114, 121-123, 126-127, 133-136, 138-141, 177, 253, 256-257, 267, 271, 278, 288, 290, 293, 295, 297, 302-304, 310-311, 323-324, 327, 330-331, 391, 415, 417, 431, 453, 460, 463, 467, 474, 487, 491, 496, 511-512, 518, 531, 535, 570, 572-573, 575-576, 579
- **weakness analysis** 96, 101, 103, 483
- **white-box assertion** 42, 221, 481, 488
- **witness** 214, 219, 225-226, 231, 273, 280-281, 295, 315-317, 320, 324, 327-328, 509
- **witness cost** 315-317, 328
- **X-dependent check** 345
- **X-optimism** 235, 240, 265-267
- **X-pessimism** 265-266
- **X-propagation** 253-255, 265-270, 298-299, 325
- **zero-delay netlist** 379, 389-390
- **zoom-out thinking** 46